

Bachelorarbeit - Gesamter Quellcode

Wellen-Erkennung und Griff mit myCobot 280 JN + RealSense D435i
Export 2026-08-31 aus ~/ros2_ws · 103 Dateien

Inhaltsverzeichnis

- .box_teach.json
- .place_offset.json
- .vida_target_latch.json
- DEVLOG.md
- box_map.yaml
- calib_check.py
- calib_check_points.json
- calib_touch_results.md
- cam_viewer.py
- camera_center.py
- check_grasp.py
- confirm_key.py
- cyclonedds.xml
- fixture_corner.json
- go_to_zero.sh
- measure_zero.py
- moveit_working.rviz
- preview_reach.sh
- reach_rings.py
- record_cam_publisher.py
- run_approach.sh
- run_approach_test.sh
- run_pick_preview.sh
- run_pick_test.sh
- run_pick_tilt.sh
- run_place_calib.sh
- rviz_preview.rviz
- src/mycobot_calibration/config/charuco_params.yaml
- src/mycobot_calibration/config/charuco_params_gripper.yaml
- src/mycobot_calibration/launch/calibrate_eye_on_base.launch.py
- src/mycobot_calibration/launch/calibrate_full.launch.py
- src/mycobot_calibration/launch/calibrate_session.launch.py
- src/mycobot_calibration/launch/calibrate_standalone.launch.py
- src/mycobot_calibration/mycobot_calibration/__init__.py
- src/mycobot_calibration/mycobot_calibration/charuco_detector.py
- src/mycobot_calibration/mycobot_calibration/estop_button.py
- src/mycobot_calibration/mycobot_calibration/joint_encoder_publisher.py
- src/mycobot_calibration/mycobot_calibration/joint_pose_gui.py
- src/mycobot_calibration/mycobot_calibration/numerical_ik.py
- src/mycobot_calibration/package.xml
- src/mycobot_calibration/scripts/calibrate_handeye.py
- src/mycobot_calibration/scripts/camera_point_marker.py
- src/mycobot_calibration/scripts/cartesian_jog.py
- src/mycobot_calibration/scripts/click_arc.py
- src/mycobot_calibration/scripts/click_place.py
- src/mycobot_calibration/scripts/click_place_moveit.py
- src/mycobot_calibration/scripts/click_to_go.py
- src/mycobot_calibration/scripts/generate_charuco_board.py
- src/mycobot_calibration/scripts/generate_charuco_gripper.py
- src/mycobot_calibration/setup.cfg
- src/mycobot_calibration/setup.py
- src/mycobot_demo/CMakeLists.txt
- src/mycobot_demo/launch/approach_target.launch.py
- src/mycobot_demo/launch/goto_clicked_point.launch.py
- src/mycobot_demo/launch/pick_place.launch.py
- src/mycobot_demo/launch/pick_tilt.launch.py

- src/mycobot_demo/package.xml
- src/mycobot_demo/rviz/approach.rviz
- src/mycobot_demo/src/approach_target.cpp
- src/mycobot_demo/src/goto_clicked_point.cpp
- src/mycobot_demo/src/pick_place_cartesian.cpp
- src/mycobot_demo/src/pick_tilt.cpp
- src/mycobot_hardware/CMakeLists.txt
- src/mycobot_hardware/include/mycobot_hardware/mycobot_hardware.hpp
- src/mycobot_hardware/mycobot_hardware_plugin.xml
- src/mycobot_hardware/package.xml
- src/mycobot_hardware/scripts/estop_relay.py
- src/mycobot_hardware/scripts/mycobot_bridge.py
- src/mycobot_hardware/src/mycobot_hardware.cpp
- src/mycobot_moveit_config/CMakeLists.txt
- src/mycobot_moveit_config/config/cartesian_limits.yaml
- src/mycobot_moveit_config/config/joint_limits.yaml
- src/mycobot_moveit_config/config/kinematics.yaml
- src/mycobot_moveit_config/config/moveit_controllers.yaml
- src/mycobot_moveit_config/config/mycobot.srdf
- src/mycobot_moveit_config/config/ompl_planning.yaml
- src/mycobot_moveit_config/config/ros2_controllers.yaml
- src/mycobot_moveit_config/launch/demo.launch.py
- src/mycobot_moveit_config/launch/moveit_rviz.launch.py
- src/mycobot_moveit_config/package.xml
- src/mycobot_moveit_config/rviz/moveit.rviz
- src/mycobot_world/CMakeLists.txt
- src/mycobot_world/launch/world.launch.py
- src/mycobot_world/package.xml
- src/mycobot_world/rviz/world.rviz
- src/mycobot_world/urdf/mycobot_world.urdf.xacro
- src/vida_vision/launch/vida_detector.launch.py
- src/vida_vision/package.xml
- src/vida_vision/setup.cfg
- src/vida_vision/setup.py
- src/vida_vision/vida_vision/__init__.py
- src/vida_vision/vida_vision/detection.py
- src/vida_vision/vida_vision/vida_detector_node.py
- start_mycobot.sh
- target_latch.py
- teach_boxes.py
- teach_click.rviz
- teach_setup.sh
- tools/umlaut_convert.py
- view_camera.sh
- view_overlay.py
- visit_boxes.py
- vo_rsp.launch.py

.box_teach.json

```
{  
  "kirmizi": [  
    0.2792440758590879,  
    0.011926408390758186,  
    0.058029951004597416  
  ],  
  "sari": [  
    0.24606200933459718,  
    -0.0969909153642651,  
    0.03720116562681315  
  ],  
  "beyaz": [  
    0.1747361137975782,  
    -0.18581033247647383,  
    0.041158810271655424  
  ],  
  "siyah": [  
    0.1152564438599393,  
    -0.24477349284346506,  
    0.0547735042779398  
  ],  
  "yesil": [  
    -0.008352656125774544,  
    -0.269284920872774,  
    0.053600176006962194  
  ]  
}
```

.place_offset.json

```
{  
  "x": -0.007,  
  "y": -0.011,  
  "z": 0.0  
}
```

.vida_target_latch.json

```
{  
  "frame_id": "robot_base",  
  "px": 0.11950728722106596,  
  "py": -0.13409965081658165,  
  "pz": 0.02012665369210065,  
  "ox": -0.6899152713496087,  
  "oy": -0.15497392799627882,  
  "oz": 0.15497392799627882,  
  "ow": 0.6899152713496087,  
  "info": "SIYAH Schraube conf=0.88 | x=120 y=-134 z=20mm"  
}
```

DEVLOG.md

ros2_ws Geliştirme Günlüğü

Bu dosya oturumlar arası kaybolmasın diye `**~/ros2_ws/` içinde**` tutulur.

****Her yeni Claude Code oturumunda ÖNCE bu dosyayı oku**** – son durum, doğrulanmış gerçekler ve sıradaki adım burada.

ANLIK DURUM (yarın açıldığında okunacak özet)

2026-08-30 – PLAN: Kamera 66→40-45cm İNDİRİLECEK + YENİDEN KALİBRASYON (henüz yapılmadı)

- ****Gerekçe (kullanıcı):**** 66cm veri için çok yüksek; gripper Welle-kutusunu okülde ediyor. Hedef: ~40-45cm'e sabitle, yeniden el-göz kalibrasyonu. Kullanıcı gripper'a sabitlenecek ChArUco görselini yükleyecek (matris boyutu ondan teyit edilecek; mevcut config: `**6×3, kare 16mm, marker 12mm, DICT_4X4_50**` – ``charuco_params_gripper.yaml``; board farklıysa yaml + kumpas ölçümü güncellenmeli).

- ****ETKİLENEN HER ŞEY (envanter):**** (1) URDF ``mycobot_world.urdf.xacro`` satır ~187 ``sensor_d435i`` origin (yeni `T_origin = T_calib ◦ (bottom_screw→optical)-1` – dönüşüm scripti scratchpad'de kaybolmuş, YENİDEN yazılacak); (2) URDF ``camera_arm`` joint z 0.670→yeni bottom_screw z (~0.45-0.50) + uç x'leri/uzunluk (2026-07-14'teki gibi boşluk kalmasın), ``column`` 0.76m yeterli; (3) ``~/ros2/easy_handeye2/calibrations/mycobot_d435i_eob.calib`` yeni sonuçla; (4) BAYATLAYACAKLAR: ``place_offset.json`` (-7,-11mm), ``GRASP_Z_OFFSET=-0.025``, ``calib_check_points.json`` → kalibrasyon sonrası `calib_check` ile YENİDEN ölç; (5) memory (handeye+kaiser) + DEVLOG. URDF symlink-install → rebuild GEREKMEZ.

- ****FOV/menzil kontrolü:**** renk ~69°×42° → 45cm'de ~62×35cm, 40cm'de ~55×31cm kapsama (66cm'de ~92×52 idi) – pick bölgesi ($r \approx 100-330\text{mm}$) kenar payları DARALIR, montaj sonrası canlı görüntüyle doğrula. D435 min-Z: 720p'de ~28cm – kalibrasyon pozlarında board'u lensden $\geq 30\text{cm}$ tut (veya 848×480 kullan).

- ****PROSEDÜR (2026-07-14 şablonu):**** yedek (``calibration_backups/<tarih>/`` URDF+calib) → kamerayı indir/sabitle → ``calibrate_session.launch.py`` + ``calibrate_handeye.py`` (charuco_params_GRIPPER ile!) → 16+ poz freehand (çeşitli rotasyon, board köşeleri tam görünür) → ****Park**** hesapla, Horaud/Daniilidis ile $\leq 1\text{cm}$ uyum kontrolü (****Tsai-Lenz YASAK**** – z çökertir) → $z \approx$ cetvelle ölçülen yükseklik sanity → .calib kaydet → URDF origin + camera_arm güncelle → RViz/tf2_echo teyit → `calib_check` 2-nokta cetvel ölçümü → yeni offset → (opsiyonel) robot-dokunma testi → DEVLOG+memory+git commit.

2026-07-16 (akşam) – KALİBRASYON TAM DENETİMİ + CANLI TEST: extrinsic SAĞLAM, asıl açık = ofset uygulaması + CONF=0.85 fazla sıkı

- ****Statik denetim (bayat veri avı):**** Runtime zinciri temiz: URDF origin (0.15598/-0.11415/0.66987) ↔ ``~/ros2/easy_handeye2/calibrations/mycobot_d435i_eob.calib`` (0.14406/-0.14646/0.65809, Park) birebir; canlı ``tf2_echo`` de aynı → ****runtime'da eski veri YOK****. Δ ``~/ros/easy_handeye/*.yaml`` (ROS1-yolu) yuvarlak placeholder (0.22/-0.15/0.55) BAYAT ama hiçbir şey okumuyor – karışıklık diye arşivlenebilir. ****DOKÜMANLAR bayattı:**** tez/sunumlar/A-Z/memory hâlâ Haziran değerini (0.197/-0.143/0.475 + "47cm") taşıyordu → HEPSİ 0.144/-0.146/0.658 + ~66cm'e düzeltildi, 3 PDF yenilendi, memory (handeye+kaiser+index) güncellendi.

- ****ANA AÇIK (pick sapmasının nedeni):**** ölçülmüş algı ofseti (-7,-11)mm SADECE ``place_offset.json``'da ve onu SADECE ``click_place_moveit.py`` okuyor – ****pick zinciri (vida_detector→pick_tilt) ofseti HİÇ uygulamıyor!**** Pick XY'de ~+7.11mm sapıyor; z sadece ``GRASP_Z_OFFSET=-0.025`` yara bandıyla. ÖNERİ: ofseti `vida_detector`'a parametre olarak ekle (yayınlanan /vida/target'a uygula) YA DA `pick_tilt`'e; sonra robot-dokunma testiyle kesinleştir (altyapı hazır: `click_to_go+cartesian_jog+GRIPPER_DOWN_QUAT` fix).

- ****Z sapması (2026-07-15 cetvel ölçümünden):**** yakın ($r \approx 176\text{mm}$) z✓ doğru; uzak ($r \approx 327\text{mm}$) +5.7mm ($\approx 2^\circ$ eğim izi). XY sabit ofset: yakın (+11,+9), uzak (+2,+13). Yeni ölçüm gerekmedi – o veri güncel kalibrasyona ait.

- ****Canlı algı testi (robot hareketsiz, kamera+rsp+detector):**** conf=0.85 → 32 döngü ****SIFIR tespit****; conf=0.5 → sahnedeki tek nesne conf=0.64'te titrek tespit (lin=0.49 güvenilmez, z=-10mm şüpheli depth). ****CONF=0.85 kararı**

sahada ÇALIŞMAZ** (dünkü gerçek pick tespitleri 0.72-0.75, latch 0.72 idi!) → öneri: CONF=0.65-0.70'e çek.
display_min_conf 0.85 kalabilir.

- **Hız iyileştirmeleri CANLI DOĞRULANDI:** model 16s + warmup 70s = 87s'de "Modell bereit" (eskiden görünmez 102s). △ Yeni gözlem: 848x480'de döngü ~3.4s/kare (424x240'ta ~1s idi) – statik sahne için kabul edilebilir, istenirse profil geri alınabilir.

- Kamera "HW not ready" 4 satırı bilinen zararsız init (DEVLG bilinen sorun #3). pkill self-match tuzağına bir kez daha basıldı (exit 144) – pattern'siz `for p in \$(pgrep -f ...)` yöntemi kullanıldı.

- **OFFSET PICK ZİNCİRİNE EKLENDİ (kullanıcı onayı):** `vida\_detector` yeni `offset\_x/y/z` parametreleri – düzeltme SADECE yayınlanan `/vida/target`/marker/info'ya eklenir (iç geometri+overlay ROH kalır → geri-projeeksiyon vidanın üstünde kalır; sticky roh `\_last\_pub\_p` ile). `run\_pick\_preview.sh` değerleri `place\_offset.json`'dan okuyup `\_p` ile geçirir (click_place ile TEK kaynak). Canlı doğrulandı: "Ziel-Offset AKTIV: (-7.0, -11.0, 0.0) mm". Log artık "BASE(korr.)" gösterir.

- **CONF kalıcı 0.70** (0.85 canlı testte hiçbir şey bulamadı; gerçek tespitler 0.72-0.75): script + node + detection.py imzası; tez/2 sunum/A-Z 0.70'e güncellendi, 3 PDF yenilendi.

- **SIRADAKİ ADIM:** robot-dokunma testi (kullanıcı başında: vida yerleştir → pick → cetvel/dokunuş teyidi → gerekirse place_offset.json'u nudge ile güncelle; dosya artık pick'i de besliyor).

2026-07-16 (gece olayı incelemesi) – NOT-AUS OLAYI KÖK NEDEN + ZİNCİR SERTLEŞTİRME + KAMERA/HIZ İYİLEŞTİRME + TEZ GENİŞLETME

- **OLAY (02:01-02:12 pick_preview koşusu):** kullanıcı NOT-AUS'a bastı, robot durmadı, güç düğmesiyle kapatıldı. Log analizi (`logs/pick\_preview\_20260716\_020108/`): grasp 02:11:39'da bitti, onay beklerken Ctrl+C → cleanup `goto\_zero` (02:12:25, MoveIt'siz çarpışma-kontrolsüz düz süpürme) → stack (relay+bridge dahil) öldürüldü → `park\_zero\_pymycobot` ham süpürme. **Bu pencerede NOT-AUS zinciri ÖLÜYDÜ** (buton penceresi ekranda ama relay+bridge yok) → basış boşluğa gitti. İkincil bulgu: rcutils stdout blok-tamponlu → `estop\_button.log` 0 byte (adli iz YOK).

- **Canlı testler (xdotool ile gerçek tıklama):** buton→`/estop`→relay→sahte-bridge-soket zinciri TEK TEK ÇALIŞIYOR; tıklama `estop 1` + 2s heartbeat üretti. Yani mekanizma sağlam, olay cleanup-boşluğu + geri-bildirimsizlikti.

- **FIX'ler:** (1) `estop\_button.py`: flush'lu print loglama + `/estop\_ack` aboneliği → pencerede "BRIDGE BESTAETIGT" / 2sn'de ack yoksa "KEINE BRIDGE-ANTWORT – NETZSCHALTER!" + `mlockall(MCL\_CURRENT)` (swap-donması önlemi; **MCL_FUTURE YASAK** – testte CycloneDDS `pthread\_create` EAGAIN ile öldü, publisher sessizce çalışmaz oldu!). (2) `estop\_relay.py`: başarılı soket yazımı sonrası `/estop\_ack` yayını. (3) `run\_pick\_preview.sh`: `RCUTILS\_LOGGING\_BUFFERED\_STREAM=0`; `estop\_active()` (latched `/estop` echo) → STOP aktifken cleanup `goto\_zero` VE pymycobot-park ATLANIR.

- **Kamera/hız fix'leri (aynı script):** kamera **848x480x15 renk+depth, pointcloud KAPALI** (Faz A'da RViz yok, kimse kullanmıyor; pencere kalitesi şikâyeti çözüldü – renk=depth profil eşitliği korundu). Detector Faz A'da KAMERADAN ÖNCE başlar (CUDA warmup ~100s kamera/stack açılışıyla paralel; toplam ~3-4dk → ~2dk). `vida\_detector\_node`: model yükledikten sonra **dummy warmup inference** → "Modell bereit" artık gerçekten hazır demek; DETECTOR_WAIT 120→240. `cam\_viewer.py`: overlay gelene dek **ham kamera görüntüsü** fallback (siyah pencere bitti), PANEL_H 480 (native), INTER_AREA/CUBIC, tüm yazılar ince (thickness 1 + kontur). Detector overlay: **"ZIEL: ROT/GELB/..."** hedef rengi hem işaretçide hem sağ-üstte sabit (istek).

- **Ölçülen gerçek:** model yükleme 15s + İLK CUDA inference 102s (Nano Maxwell cuDNN init) + sonrası ~1s/kare @1Hz. 3-4dk'lık algı bundandı.

- `display\_min\_conf` 0.9→0.85 + **CONF kalıcı 0.5→0.85** (kullanıcı isteği): run_pick_preview.sh varsayılanı, node `conf` parametresi ve `detection.py` imza varsayılanı 0.85 – artık hedef adayı da yalnız ≥0.85 tespitler. DİKKAT: başarılı pick reçetesi CONF=0.5 idi; detector hedef bulamazsa `CONF=0.5 ./run\_pick\_preview.sh` ile eskiye dön. Sunum (Folie 5+19) ve tez (Listing 4.2, 4.4.1, 5.5) 0.85'e güncellendi; ilk-başarı-0.5 tarihsel notu korundu. Tüm PDF'ler yenilendi.

- Not: bridge'de `WARN poll: 'int' object is not iterable` tek-seferlik zararsız polling istisnası (döngü atlanıyor) – pymycobot get_radians iç parse hatası; izlemede.

- **TEZ (BA) GENİŞLETİLDİ:** `~/Downloads/ba/BA...\_ERWEITERT\_2026-07-16.docx` (orijinale DOKUNULMADI). 12.821→17.324 kelime, LibreOffice render 142 sayfa. 213 paragraf dil temizliği (mil parçası→Welle, Şekil→Abbildung, Tablo→Tabelle, Kaynak→Quelle, 6 tam-Türkçe gövde paragrafı Almancaya çevrildi; Türkçe Özet/kapak bilinçli korundu). YENİ: 4.2.1-2 (gerçek kamera parametreleri+QoS), 4.4.1 (detect_screws kodu), 4.6+ (PCA kodu+head_aware_grip), 4.7+ (_depth_m kodu), 4.8+ (qrot/deproject/TF2 kodu), 4.9.1-3 (3D-PCA, grasp-frame kodu, eleme+konsens), 4.10.1-4 (5 paket, bridge wire-protokol, 2-faz RAM, rmem/locale/log), 4.11.1-4 (TRAC-İK port, hız profili, fraction, çarpışma

dersleri), 4.12+ (gerçek durum makinesi, gripper open-loop), **4.13 NOT-AUS zinciri (olay dersi dahil)**, **4.14 Jetson laufzeit ölçümleri**, F.6 (easy_handeye2: Tsai-Lenz z-hatası-Park, 0.197/-0.143/0.475), Anhang A detaylı kütüphane listesi, 5.5+ (ilk uçtan-uca pick + açık noktalar), **Anhang H: 4 kod listing'i**. Stil: CodeBlock (Courier 8.5pt) + CodeCaption. Script: scratchpad/extend_ba.py.

- **TEZ ŞEKİL NUMARALARI DÜZELTİLDİ (devam):** 30 caption belge sırasına göre 1..30; 3 başıboş liste satırı gövdeden silindi; metin içi referanslar en-yakın-caption eşlemesiyle güncellendi; ön liste gerçek caption'lardan yeniden kuruldu; "21a" Türkçe yer tutucu Almancalaştırıldı. Script: scratchpad/fix_fig_numbers.py. **Tablo numaraları da düzeltildi** (fix_tab_numbers.py): 13 caption 1..13 ardışık, mükerrer 5/6/7 + "2a" giderildi, ilk tabloya eksik caption eklendi, Tablolar Listesi yeniden kuruldu; render 142 sf sağlam.

- **KOMBİNE SUNUM (devam):** kullanıcının Kurzvortrag'ı (22 folie, didaktik) + bizim matematik/kod destesi birleştirildi → `ba/Praesentation\_Wellen\_Roboter\_KOMBINIERT\_2026-07-17.pptx/.pdf` + kombine Sprechernotizen (29 folie; 9 yeni didaktik folie eklendi). Kaynak dosyalara dokunulmadı. Script: scratchpad/make_pptx_kombi.py.

- **IK/ERİŞİM HALKALARI DOKÜMANTE EDİLDİ (devam):** Sunuma yeni Folie 15 (halkaların Monte-Carlo-FK herleitung'u, ring içi IK-reddi nedenleri, aktif TRAC-İK konfigürasyonu), 20 folie korundu (eski Folie 2 başlığa birleşti, numaralar otomatik); teze 4.11.5 Erreichbarkeitsanalyse; A-Z referansa 2 giriş (134). PDF'ler yenilendi.

- **PROF SUNUMU HAZIRLANDI (2026-07-17 için):** `~/Downloads/ba/Praesentation\_Wellen\_Roboter\_2026-07-17.pptx` – 20 folie Almanca, matematik (her formülde sembol açıklama kutusu) + gerçek kod ağırlıklı; folie 18 = aks 1-2 backlash (~1cm sarkma, kalibrasyon emeği vurgusu). Yanında `Sprechernotizen\_Praesentation\_2026-07-17.docx` (folie başına serbest konuşma cümleleri; PPTX notes alanında da aynı). Render kontrolü LibreOffice-PDF ile yapıldı, vektör-ok glif hatası düzeltildi.

- **AÇIK:** dokunuş testi hâlâ yarım (önceki oturum); cleanup'taki `goto\_zero`'nun MoveIt'li güvenli versiyonu düşünülebilir.

2026-07-16 (devam) – DOKUNUŞ TESTİ ALTYAPISI + 3 KÖK-NEDEN FIX (test YARIM, kullanıcı tez isteğiyle durdu)

- **Dokunuş testi akışı kuruldu:** gerçek stack (bridge+3 controller+estop_relay+estop_button) + kamera 720p/720p + `click\_to\_go.py` (tıkla→robot gitsin) + `cartesian\_jog.py` (ince ayar). Plan: yakın vidaya tıkla→E→hover→jog ile mm hizala→TF `robot\_base→tcp` oku→offset = tcp_xy – algılanan_xy (nudge 1cm kuantizasyonu YOK).

- **FIX 1 – `click\_place\_moveit.py` tuş kodları:** NumLock açık numpad 8/2/4/6 KP_-keysym gönderir (1114034=0x10FFB2), tanınmıyordu; AYRICA bu OpenCV/GTK build tuşlara **0x100000 öneki** ekliyor → ok tuşları da baştan beri ölüydü. Fix: `kx&0xFFFF` normalizasyonu + KP_rakam/KP_+– bağları (ARROW_KEYS genişletildi).

- **FIX 2 – `click\_to\_go.py` CLICK_HOVER_M:** env değişkeniyle opsiyonel hover payı (default 0=eski davranış; kalibrasyonda 0.04). Sebep: araç tıklanan noktanın kendi z'sine iner, XY sapması varken vidaya yandan çarpar.

- **FIX 3 (BÜYÜK) – `numerical\_ik.py` GRIPPER_DOWN_QUAT bayattı:** eski değer (-0.7071,0.7071,0,0) ESKİ URDF'in [0,-1.57,0,-1.57,0,0] FK'sından türetilmişti; gripper/tcp değişince o oryantasyon İMKÂNSIZ oldu → /compute_ik HER pozisyonda -31 → click_to_go+cartesian_jog "keine IK/nicht erreichbar" (kullanıcının gördüğü hata buydu; 0-pozdan FAHRE hatası da bu). Yeni değer (-0.70711,0,0,0.70711) = tcp-Y-ekseni aşağı (tcp gripper_base'den +Y 110mm → parmak yönü Y). TRAC-İK canlı testi: yakın bölge TÜM yaw'larda çözüyor. Modül build/ hardlink → rebuild gerekmedi.

- ▲ **ERİŞİM KISITI:** uzak vida (297,-137, r≈327mm) dik-gripper'la ERİŞİLEMEZ (tüm yaw -31) → dokunuş testi YAKIN vidayla (106,-141, r≈176mm).

- **KALDIĞI YER:** araçlar açık, akış hazır, kullanıcı tıklayıp E+jog yapacaktı → tez isteğiyle DURDU. Dokunuş testi + offset kesinleştirme AÇIK. `place\_offset.json` şu an ölçülen algı-düzeltilmesiyle önceden dolduruldu: x=-0.007 y=-0.011 z=0.

- Ayrıca: cartesian_jog'un IK'sı da artık çalışır durumda (aynı fix) – FAHRE alanına mm cinsinden mutlak hedef yazılabilir.

2026-07-15/16 oturum – ALGI KALİBRASYON ÖLÇÜMÜ YAPILDI: extrinsic SAĞLAM, hata SABİT OFFSET (~+7,+11mm)

- **calib_check ölçümü tamamlandı** (oturma 36 prosedürü): 2 vida (yatık, en geniş 15mm/şaft 10mm), yakın r=176mm + uzak r=327mm, kullanıcı cetvelle ölçtü.

- **Kurulum:** robot GEREKMEDİ – sadece `robot\_state\_publisher` (params-file ile; URDF CLI'da `p robot\_description:=` OLMUYOR, arguments.c parse hatası → yaml params dosyası şart) + realsense. **Kamera 1280x720x15 renk + 1280x720x15 depth + align SORUNSUZ çalıştı (pointcloud KAPALI iken)** – "424x240 zorunlu" kısıtı pointcloud+tam-stack durumu içinmiş. DİKKAT Galactic sürücü tuhaflığı: renk 720p + depth 424x240 karışımında aligned_depth 424x240 kalıyor ama camera_info'su 1280x720 diyor → tıklama matematiği BOZULUR; renk=depth profili

eşit tut.

- ****Sonuçlar (robot_base, mm):**** yakın: kamera[106,-141,13] vs cetvel[95,-150,~10-15] → Δ(+11,+9,z✓); uzak: kamera[297,-137,20] vs cetvel[295,-150] → Δ(+2,+13,z+5..7).

- ****Doğrulan 3 şey:**** (1) kamera yüksekliği 658mm FİZİKSEL TEYİT (kullanıcı ölçtü – 2026-07-14'ten beri bekleyen teyit KAPANDI); (2) vida-arası mesafe kamera 191mm = cetvel ~191mm → ölçek/rotasyon hatası YOK (ilk 200mm izlenimi taban-ortası referans belirsizliğiydi); (3) derinlik zinciri birebir (z_cam 643/644 = beklenen 658-15..10).

- ****KARAR: SABİT OFFSET yeterli**** – kamera X'te ~+7mm, Y'de ~+11mm fazla okuyor (±5mm cetvel belirsizliği). Düzeltme ≈ (-7,-11)mm. Küçük kalıntı: Z hatası uzakta +7mm'e büyüyor (~2° eğim izi) – XY'yi etkilemiyor, kabul edildi.

- ****SIRADAKİ ADIM (robot gerekir):**** ofseti robot dokunuşuyla kesinleştir – gerçek stack + `click\_place\_moveit.py` → nudge (başlangıç -7,-11mm) → `S` ile `.place\_offset.json`. Eski `.place\_offset.json` + `GRASP\_Z\_OFFSET=-0.025` BAYAT, bu ölçümle değiştirilecek.

- Ölçüm verisi: `~/ros2\_ws/calib\_check\_points.json` (2 nokta). Bu oturumda robot hareket ETMEDİ.

2026-07-15 oturum – Almanca denetim + UMLAUT KARARI + script/proje analizi + İLK GIT COMMIT

- ****Batch 2 durumu doğrulandı:**** config yamll/srdf + `charuco\_detector` docstring ZATEN Almanca (önceki oturumda bitmiş); Türkçe yorum kalıntısı ****SIFIR**** (tüm bizim-paketler tarandı). ****KALAN Almanca işi:**** 5 seyrek-yorumlu launch (`world.launch.py` 12, `moveit\_rviz.launch.py` 9, `goto\_clicked\_point` 5, `pick\_place` 4, `approach\_target` 3 yorum) + `generate\_charuco\_board.py` ~4 İngilizce satır + `mycobot\_demo` ****C++ rebuild**** (binary hâlâ eski).

- **△ **TESPİT: "Batch 1 umlaut ile yapıldı" kaydı YANLIŞ**** – repoda tek bir gerçek umlaut yok, HER ŞEY ASCII-Almanca (fuer/ueber/laedt). ****KULLANICI KARARI (2026-07-15):** TÜM repo yorumları gerçek umlauta (ü/ö/ä/ß) çevrilecek; cv2 pencere metinleri ASCII kalır. ****HAZIRLIK TAMAM, UYGULANMADI** (kullanıcı analiz isteğiyle beklemeye alındı): yedek tar scratchpad'de + ~600 kelimelik dağarcık analizi + güvenli plan = yorum/docstring-scoped dönüşüm (string literal'lere DOKUNMA), ß-kuralları (gross→groß, ausser→außer, schliess→schließ, stoss→stoß, weiss→weiß, mass→maß, oess→öß, Gauss→Gauß; dass/muss/Messung/Kompromiss ss KALIR), sahte-pozitif korumaları: `que/aue/eue` dizileri + tam-kelime skip (true, value, queue, Daemon, Koeffizient, Shoemake, Rodrigues, aktuell/manuell/visuell, zuerst) + BÜYÜK-HARF token'lar (BESTAETIGEN gibi UI-literal referansları) atlanır + `zurueck` typo düzeltilmesi.

- ****E-STOP GERÇEĞİ (analizde çıktı):**** `/estop` enforcement SADECE `joint\_encoder\_publisher.py`'de (kalibrasyon bridge'i). ****ANA bridge (`mycobot\_hardware/scripts/mycobot\_bridge.py`) /estop DİNLEMİYOR**** → `run\_pick\_preview.sh`/`run\_place\_calib.sh` çalışırken kırmızı buton ETKİSİZ. İki run scripti `estop\_button`'u başlatmıyor da. Memory düzeltildi → EN YÜKSEK ÖNCELİKLİ AÇIK: enforcement'ı ana bridge'e taşı + run scriptlerine estop_button ekle.

- ****İLK GIT COMMIT ATILDI:**** repo bugüne dek commit'sizdi (her şey untracked = tez verisi risk altındaydı). `.gitignore` eklendi (build/install/log/logs/pycache/src_backup*/upstream'ler). Upstream 4 paketin yerel değişiklikleri `patches/upstream\_local\_mods/\*.patch` (450 satır) + untracked dosyaları `.tgz` + `patches/UPSTREAM\_MANIFEST.md` (URL+branch+SHA) olarak donduruldu → upstream'ler ignore edilse de tam yeniden-kurulabilir.

- ****run_pick_preview.sh + run_place_calib.sh incelemesi + proje geliştirme listesi kullanıcıya sunuldu**** (özet: ortak-lib çıkarma ~150 satır duplikasyon, estop entegrasyonu, TCP kalibrasyonu band-aid yerine kalıcı, place adımı pick akışına, vision-doğrulamalı grasp, Daemon, Koeffizient, Shoemake, Rodrigues, Rodrigues ölçümü hâlâ bekliyor, log virikimi 12MB, welle rename).

- ****E-STOP ANA YOLA ENTEGRE EDİLDİ (kullanıcı: "önce l'i yap")****

- ****`mycobot\_bridge.py` (ana bridge):**** yeni wire-komut `estop <0|1>`. Merkezi `\_enqueue()` (iki socket girişi de buradan): estop-aktivasyonunda flag ANINDA set + bekleyen kuyruk boşaltılır (stop sonrası komut "nachlauf" edemez); worker (tek seri thread) `mc.stop()`×3 uygular (tork KALIR, kol düşmez); aktifken `send\_radians/set\_gripper/release\_all` hem enqueue'da hem `\_exec`'te (çifte savunma) düşürülür; `power\_on/shutdown/estop 0` geçer. ****6/6 kukla-MC birim testi geçti.****

- ****`estop\_relay.py` (YENİ, mycobot_hardware/scripts):**** bridge bilinçli ROS'suz (pybind11 tuzağı) → küçük tek-thread rclpy node `/estop` (transient_local) → cmd socket'e `estop 1/0` yazar; STOP aktifken ****2s heartbeat**** (bridge yeniden başlarsa stop durumunu devralır). CMakeLists PROGRAMS'a eklendi.

- ****`demo.launch.py`:**** gerçek donanımda (`USE\_FAKE\_HARDWARE` != true) estop_relay otomatik başlar.

- ****`click\_place\_moveit.py`:**** pymycobot'la portu DOĞRUDAN açtığı için kendi `/estop` aboneliği eklendi: STOP-anında `mc.stop()`×3 (pymycobot `thread\_lock=True` iç kilidi doğrulandı → spin-thread'den güvenli) + plan iptal + `\_exec\_thread` waypoint döngüsü flag'le kesilir (bekleme 50ms dilimli) + E/O/C/R estop aktifken reddedilir.

- ****İki run scripti:**** controller'lar aktif olunca `estop\_button` otomatik başlar (DISPLAY yoksa terminal-alternatifi yazdırılır); sweep+cleanup'a estop_button/estop_relay eklendi.

- **Δ Kapsam notu:** cleanup'taki `park\_zero\_pymycobot` bilinçli olarak estop'a bakmaz (kullanıcı Ctrl+C = park isteği). Kalan FAZ 2: RViz estop paneli.

- ****DOĞRULAMA (hepsi geçti):**** py_compile+bash -n ✓; 6/6 birim test (kukla MC) ✓; ****CANLI duman testi**** (gerçek port, robot HAREKETSİZ, bridge `--no-power-on`, sonunda `kill -9` → release_all çalışmadı, kol düşmedi): socket `estop 1`→STOP ✓, `send\_radians` düştü ✓, topic→relay→bridge tam zincir ✓, 2s heartbeat ✓ (7× estop 1), FREIGABE ✓; `demo.launch.py -p` gerçek-hw'de relay VAR / fake'te YOK ✓. Build: mycobot_hardware+moveit_config+**mycobot_demo** (Almanca C++ rebuild BEKLEYEN İŞ DE KAPANDI – `strings pick\_tilt`: Türkçe 0, `SCHRITT` var).

- ****BUILD DERSİ:**** Nano'da `colcon build` default paralel derleme mycobot_demo'da OOM (`cclplus Killed`) → `MAKEFLAGS=-j1 --executor sequential` ile geçti (~1.5 dk/dosya; ilk paralel deneme 1s 5dk sürüp patladı).

- ****YAN BULGU:**** bridge testinde `get\_gripper\_value(1)` sürekli ****geçerli 95**** döndürdü – "readback hep 255" bilgisi bu firmware durumunda geçersiz olabilir; gripper hareketiyle doğrulanmalı (memory güncellendi).

- **Δ `estop\_relay.py`'ye `chmod +x` gerekti** (Write ile oluşturulan dosya – `ros2 run` "No executable found" verdi; symlink-install src'nin bitini kullanıyor).

2026-07-15 (devam) – SIRADAKİ AKTİF İŞ: TCP ~2.5cm ofset kök-neden kalibrasyonu (analiz yapıldı, FİZİKSEL ÖLÇÜM bekliyor)

- ****Kök-neden haritası çıkarıldı (kod okundu):**** `tcp` frame URDF'te `gripper\_base\_to\_tcp` fixed joint, `origin xyz="0 0.110 -0.01"` (gripper_base'den +y 110mm ileri, z -10mm). Yorum: "tcp-Pose ungefähr ~9cm... später mit präziser Messung justierbar" → yani zaten kaba, hassas ölçümle düzeltilmesi planlanmış.

- ****Yara-bantları (kalıcı çözümle değiştirilecek):**** (1) pick: `run\_pick\_preview.sh` `GRASP\_Z\_OFFSET=-0.025`; (2) place: `click\_place\_moveit.py` `place\_offset.json` + `DEFAULT\_OFF` (satır 47-51). İkisi de TCP model-sapmasını nokta-nokta maskeler, kök-çözüm DEĞİL.

- ****Araç HAZIR:**** `~/ros2\_ws/calib\_check.py` – SALT-OKUNUR (robotu HAREKET ETTİRMEZ, porta dokunmaz), kamera+TF dinler, tıklanan pikselin robot_base 3B koordinatını yazar/kaydeder (`pixel\_to\_base` matematiği click_place ile AYNI, 7×7 medyan derinlik). click_place ÇALIŞIRKEN paralel açılabilir. Tuşlar: sol=nokta, sağ=sil, k=kaydet (`calib\_check\_points.json`), q=çık.

- ****ÖLÇÜM PROSEDÜRÜ** (oturum 36 planı, kullanıcı+robot gerekir): yakın (~15cm) + uzak (~30cm) 2 fiziksel hedef koy → calib_check ile her birine tıkla → yazılan base koordinatını CETVELLE ölçülen gerçek konumla karşılaştır. KARAR: XY hatası iki noktada ~aynı (yön+miktar) → SABİT offset (nudge+5 yeterli, hızlı); hata uzakta büyüyor/yön değişiyor → EXTRINSIC/TCP hatası (URDF tcp origin'i düzelt VEYA yeniden kalibre).

- **Δ ****KRİTİK NOT:**** kalibrasyon 2026-07-14'te YENİLENDİ** (kamera ~47-66cm) → eski `place\_offset.json`/GRASP_Z_OFFSET değerleri BAYAT olabilir; ölçümü yeni extrinsic'le SIFIRDAN yap.

- ****Durum:**** kod analizi + prosedür hazır. Sıradaki adım fiziksel: robotu+kamerayı kur, iki hedef koy, calib_check ile ölç. (Bu oturumda robot bağlı DEĞİL, ölçüm yapılmadı.)

2026-07-15 (devam) – UMLAUT DÖNÜŞÜMÜ TAMAMLANDI (tüm repo yorumları+docstring'ler gerçek ü/ö/ä/ß)

- ****Karar uygulandı:**** kullanıcının "TÜM repoyu umlauta çevir" kararı hayata geçti. ****606 benzersiz kelime, 69 dosya, ~1540 yerde**** ASCII-Almanca (fuer/ueber/laedt) → gerçek umlaut (für/über/lädt). cv2 pencere metinleri + tüm string literal'ler ASCII KALDI.

- ****Yöntem** (`scratchpad/umlaut.py`): ****`.py` için `tokenize` ile SADECE COMMENT + docstring STRING token'ları** (string literal'ler yapısal olarak korundu); ****`.sh` tam-satır `#`; ****`.yaml` satır-içi `#`; ****`.cpp` ****`//`+`/\*`/` (string-atlama state machine); **\*\*`.xml/.srdf/.xacro` **\*\*`<!-- -->`. **\*\*`ss→ß` SADECE elle-onaylı FORCE sözlüğünden (gross→groß, ausser→außer, schliess→schließ, stoss→stoß, weiss→weiß, mass→maß AMA Masse→Maße[ölçü], Ausreisser→Ausreißer, Gauss→Gauß, -mäßig; dass/muss/Messung/Kompromiss/flüssig/lässt ss KALDI). Sahte-pozitif korumaları: prev-sesli + prev-q (Quelle/Sequenz/Frequenz), -uell Latince sıfatlar (aktuell/manuell/visuell ATLA ama füllen ÇEVİR), eu/au+e sahte-ue (neue/steuert/bauen/Dauer ATLA), İngilizce (true/value/queue/request) + özel-ad (Rodrigues/Shoemake/Daemon-Thread) + Koeffizient ATLA, BÜYÜK-HARF token'lar (UI-literal ref) ATLA, ****`zuerst` ATLA.****************

- ****BULUNAN+DÜZELTİLEN BUG:**** ilk conv_ue'de `''` in `qQ` Python'da ****True**** döndürüyordu (boş string alt-dizi) → kelime-başı `ue` (ueber/uebergeben...) yanlış atlandı; `prev in ('q','Q')` ile düzeltilip yeniden uygulandı (idempotent, tam-kelime). Ayrıca `zurueck` yazım hatası (ue) → `zurück` düzeltildi.

- ****DOĞRULAMA (hepsi geçti):**** (1) string literal sayımı ÖNCE=SONRA bit-aynı (Modell bereit/ZIEL/ROBOTER/GESPERRT... değişmedi); (2) grep kontratı ('grep -q "...") değişmedi; (3) tokenize taraması: docstring-OLMAYAN umlaut'lu string = ****0**** (string literal'lere hiç dokunulmadı); (4) 47 bizim .py 'py_compile' OK; (5) tüm .sh 'bash -n' OK; (6) C++ string literal'lerinde umlaut = 0; (7) upstream 4 pakete umlaut sızmadı (scope korundu [[feedback_ros2ws_scope]]); (8) şüpheli-kalıp taraması (umlaut-sesli-komşu/ß-adayı-kaldı/İng-bozuldu) = 0; kalan 133 ASCII kelimenin hepsi meşru (kısa-sesli ss ya da İngilizce/özel-ad).

- **△ **NOT:**** C++ değişikliği SADECE yorumlarda → binary rebuild GEREKMEZ (runtime string'leri değişmedi). Python symlink-install → rebuild gerekmez.

- Dil artık %100 gerçek-umlaut Almanca (yorumlar+docstring'ler). cv2 overlay'ler bilinçli ASCII-Almanca [[feedback_german_ui_overlay]]. Yorum kararı: yorum/docstring gerçek umlaut, pencere metni ASCII.

2026-07-14 oturum – KALİBRASYON YENİDEN YAPILDI + Not-Aus eklendi

- ****Yeni el-göz kalibrasyonu TAMAM:**** 6x3 gripper board, 16 poz, OpenCV/Park. Park/Horaud/Daniilidis ~1cm içinde uyuştu (z güvenilir, çökme yok – geçen seferki Tsai-Lenz z-çökmesi sorunu bu sefer YOK). Sonuç 'robot_base→camera_color_optical': trans(0.14406,-0.14646,0.65809) quat(0.70446,0.70947,0.01973,0.00212). Kamera ~47→~66cm.

- ****URDF güncellendi:**** 'mycobot_world.urdf.xacro' sensor_d435i origin = 'xyz="0.15598 -0.11415 0.66987" rpy="-2.46695 1.53111 0.68173"' (T_calib→(bottom_screw→optical)⁻¹, eski değere karşı doğrulandı). Scriptler+RViz TF'ten okuyor → sonraki launch'ta otomatik güncel. Hardcoded kamera pozu SADECE URDF'te (grep doğrulandı).

- ****Not-Aus (E-stop) eklendi + test edildi:**** bkz [[project_estop_system]]. 'estop_button' penceresi + bridge '/estop' bloğu. MoveIt yolu + RViz paneli FAZ 2.

- **△ **AÇIK:**** kalibrasyon stack + estop_button + bridge hâlâ çalışıyor; robot 0'a PARKLANDI (2026-07-14). Kamera fiziksel ~66cm teyidi kullanıcıdan alınabilir.

- ****RViz sim teyit:**** TF robot_base→camera_color_optical = [0.144,-0.146,0.658] (kalibrasyonla birebir). Kamera yeni ~66cm konumunda.

- ****STAND GEOMETRİSİ DÜZELTİLDİ:**** kamera yükselince 'camera_arm' (kamerayı tutan çubuk) eski z=0.486'da kalmıştı → ~17cm boşluk. Düzeltildi: 'camera_arm' box 0.175→0.229, origin z 0.486→0.670 (=yeni bottom_screw z), yakın uç x 0.21→0.156 (yeni kamera x). 'column' (0→0.76m) yeterince yüksek, değişmedi. Çubuk artık kameraya değiyor.

- ****AKTİF ÖNCELİK – BACHELORARBEIT ALMANCA DOKÜMANTASYON:**** kullanıcı TÜM bizim-paket kodun (a) Almanca olduğunu doğrula, (b) HER kod parçasının yanına DETAYLI Almanca açıklama ekle istiyor (tez için). Kapsam: bizim paketler (mycobot_calibration/world/hardware/demo/moveit_config + vida_vision + kök .sh). Upstream HARIÇ [[feedback_ros2ws_scope]]. cv2 pencereleri ASCII-güvenli Almanca [[feedback_german_ui_overlay]].

- ****DENETİM SONUCU (2026-07-14, 52 dosya):**** %73 (38) tam Almanca+iyi belgeli; %19 (10) karışık (İngilizce docstring+Almanca kod); %13 (7) az/eksik açıklama.

- ****İŞ GEREKTİREN dosyalar (İngilizce/Türkçe kalıntı ya da detay-eksik):**** 'joint_pose_gui.py' (İng docstring), 'numerical_ik.py' (detay-eksik + s31 İng), 'scripts/click_place.py'+ 'click_place_moveit.py'+ 'click_to_go.py' (İng docstring→DE), 'mycobot_calibration/setup.py'+ 'vida_vision/setup.py' (Türkçe→DE), 'vida_vision/detection.py' (PCA inline yorum-eksik), 'vida_detector_node.py' (orta). C++ demo + hardware + click_arc + estop/bridge + kök .sh zaten tam Almanca+detaylı.

- ****KARAR (kullanıcı):**** kod yorumlarında GERÇEK umlaut (ü/ö/ä/ß); cv2 pencereleri ASCII-güvenli; TÜM dosyalara detay.

- ****BATCH 1 BİTTİ+DOĞRULANDI (2026-07-14):**** 9 eksik/karışık dosya detaylı Almanca (umlaut) dokümente edildi: 'joint_pose_gui.py', 'numerical_ik.py', 'click_place.py', 'click_place_moveit.py', 'click_to_go.py', 'detection.py', 'vida_detector_node.py', 'vida_vision/setup.py' + 'mycobot_calibration/setup.py' (description). Tümü py_compile OK; CLASS_NAMES + "Modell bereit"/"ZIEL" + cv2 overlay + kod korundu. Dil artık ~%100 Almanca.

- **△ **BULGU (batch sırasında):**** 'click_place.py::release()' içinde tanımsız 'ik6(...)' çağırısı → NameError riski (önceden var olan bug, kod değiştirilmedi, yoruma işlendi). Kullanıcıya bildirildi.

- ****KALAN:**** zaten-Almanca 38 dosyadan seyrek-yorumlu olanlar (launch dosyaları, config yamllar, world.launch.py, charuco_detector İng docstring). C++ demo/hardware/click_arc/bridge/estop/.sh zaten detaylı – yeniden işlemeye gerek yok.

- ****KARAR (kullanıcı 2026-07-14):**** kalan iş = SADECE seyrek-yorumlu dosyalar (launch + config yamllar/srdf + charuco_detector docstring). BATCH 2 başlatıldı. NOT: eski "yarım kalan çeviri"nin config-yorumları (Türkçe) bu

batch'te de ele alınacak.

- ****ÇALIŞAN SÜREÇLER** (oturum sonu temizlik için):** sim RViz (demo.launch.py fake_hw, bg `bvctb6x1t`), estop_button (bg `bx7m05gvj`), gerçek bridge KAPALI. Robot 0'da parkta. Gerçek robot işine dönlürse: sim'i kapat → estop'lu bridge'i başlat → go_to_zero.

- ****CHECKPOINT** (kullanıcı "buraya kadar kaydet" 2026-07-14):** Yapılanlar: yeni kalibrasyon+URDF, Not-Aus sistemi, stand fix, Almanca doküman batch 1 (9 dosya). Memory güncel: [[project_handeye_calibration]] [[project_estop_system]] [[feedback_zero_pose_gripper_open]].

2026-07-14 (eski) – KALİBRASYON YENİDEN YAPILACAK (setup değişti)

Kullanıcı ChArUco'yu gripper'a (yeniden) sabitledi + kamera yüksekliği/robota yakınlığı değişti → 2026-06-02 eye-to-hand kalibrasyonu BAYAT. ****Yeni kalibrasyon geliştirilecek (planlanıyor, henüz yapılmadı).****

- Eski kalibrasyon YEDEKLENDİ: `~/ros2\_ws/calibration\_backups/20260714\_002718/` (URDF tam kopya + yaml + README.txt) + `~/ros/easy\_handeye/mycobot\_handeye\_eye\_on\_base\_backup\_20260714\_002718.yaml`.

- ▲ Tutarsızlık: aktif `mycobot\_handeye\_eye\_on\_base.yaml` (0.22,-0.15,0.55) ≠ URDF Park sonucu (origin 0.20961,-0.11021,0.48567). ****Runtime URDF'i kullanır.**** Yeni kalib URDF `sensor\_d435i` origin'ini güncellemeli.

- Ayrıca: sıfır-park artık gripper açık içeriyor → `~/ros2\_ws/go\_to\_zero.sh` (yeni).

YARIM KALAN İŞ – "HER ŞEYİ ALMANCAYA ÇEVİR" (2026-06-26, buradan DEVAM ET)

Kullanıcı: tüm repo metnini (pencere yazıları + terminal mesajları + KOD YORUMLARI) Almanca'ya çevir; sohbet Türkçe kalsın. Sadece ****bizim paketler**** (upstream `mycobot\_ros2`/`realsense-ros`/`easy\_handeye2`/`trac\_ik` + `src\_backup` HARIÇ). Dil: ****ASCII-güvenli Almanca**** (ü→ue, ö→oe, ä→ae, ß→ss) çünkü OpenCV fontu umlaut basamaz. Bkz [[feedback_german_ui_overlay]].

****BİTEN** (9 paralel ajan, hepsi py_compile/bash -n OK):**

- `vida\_detector\_node.py`, `cam\_viewer.py`, `calib\_check/camera\_center/check\_grasp/confirm\_key/measure\_zero.py`

- `reach\_rings/record\_cam\_publisher/target\_latch/teach\_boxes/view\_overlay/visit\_boxes/vo\_rsp.launch.py`

- `click\_to\_go/click\_place/click\_arc/click\_place\_moveit/camera\_point\_marker.py`

- `calibrate\_handeye/generate\_charuco\_board/generate\_charuco\_gripper.py`,
`charuco\_detector/joint\_encoder\_publisher/joint\_pose\_gui.py`, `mycobot\_hardware/scripts/mycobot\_bridge.py`

- Tüm shell scriptler: `run\_\*.sh` + `preview\_reach/view\_camera/start\_mycobot/teach\_setup.sh`

- C++ (kaynak çevrildi): `mycobot\_demo/src/{pick\_tilt,goto\_clicked\_point,pick\_place\_cartesian,approach\_target}.cpp`

- Tüm `launch.py`'ler, `detection.py`, `numerical\_ik.py`, `cartesian\_jog.py`, `mycobot\_world.urdf.xacro` (yorumlar)

- ****GREP KONTRATI DOĞRULANDI:**** detector üretici `Modell bereit` (sat.303) + `ZIEL` (sat.742) ↔ scriptler `grep -q "Modell bereit"`/`grep -q "ZIEL"`. Senkron. (ros2-çıktısı grep'leri `arm\_controller.\*active` vb. DOKUNULMADI.)

□ ****KALAN 2 İŞ (resume):****

1. ****Config yorumları** (7 dosya) hâlâ Türkçe** – SADECE `

```
`cd ~/ros2_ws && export LC_ALL=C LC_NUMERIC=C && source /opt/ros/galactic/setup.bash && source
install/setup.bash && colcon build --packages-select mycobot_demo --symlink-install`
```

(Python paketleri symlink/hardlink → rebuild GEREKMEZ; sadece C++.)

△ **BİLİNÇLİ ÇEVİRİLMİYENLER** (veri-ID, DOKUNMA):** `CLASS\_NAMES=["sari","beyaz","siyah","yesil","kirmizi"]`
(detection.py + pick_tilt.cpp – model.names ile birebir eşleşir), `robot-vida-projesi` (proje özel adı).

△ **STACK AÇIK:** gerçek robot + RViz hâlâ bağlı (`/dev/ttyTHS1`, PID'ler 28264/28272 + rviz 29918). Kapatınca robotu 0'a parkla.

Tarih: 2026-06-25 oturum 37 – **UI** Almancaya çevrildi + 2. kamera kaldırıldı + kamera standı plaka kenarına düzeltildi + gerçek robot RViz ile bağlandı.

□ **PENCERE YAZILARI ALMANCA** (`run\_pick\_preview.sh` FAZ A penceresi):** İki yerde çizilen overlay metinleri çevrildi (OpenCV fontu ü/ö/ä basamaz → ASCII-güvenli Almanca: ue/oe/ae/ss).

- **detector overlay** (`src/vida\_vision/vida\_vision/vida\_detector\_node.py` `\_draw`): merkez→`Mitte`, `ROBOT yesil...` (ic kirmizi=cok yakin)→`ROBOTER gruen...` (innen rot=zu nah), KILITLI→`GESPERRT`, KAFA→`KOPF`, HEDEF→`ZIEL`, `N vida`→`N Schrauben`. (build/ kopyası src ile **aynı inode/hardlink** → rebuild GEREKMEDİ.)

- **cam_viewer** (`cam\_viewer.py`): pencere başlığı + `bekleniyor...`→`warten...`, `ONAYLA`→`BESTAETIGEN`, `ONAY GONDERILDI`→`BESTAETIGUNG GESENDET`, panel etiketi.

§ **2. KAMERA** (record_cam) KALDIRILDI (kullanılmıyor):** `cam\_viewer.py`'den CAM2 aboneliği+panel+`np.hstack` silindi → tek panel (640px). `run\_pick\_preview.sh`'tan `record\_cam\_publisher.py` başlatması çıkarıldı (REC_PID boş kaldı → kill/sweep no-op, zararsız). NOT: `run\_pick\_tilt.sh`/`run\_approach.sh` hâlâ record_cam başlatıyor ama cam_viewer artık göstermiyor (zararsız); istenirse onlardan da çıkarılır.

KAMERA STANDI plaka uzak-kenar ortasına düzeltildi (`src/mycobot\_world/urdf/mycobot\_world.urdf.xacro`):** Kullanıcı RViz'de çubuğun plaka (platform_top 50×40) kenarından ~5cm öne kaydığını fark etti. Plaka uzak kenarı X=0.40, Y-ortası=-0.125. Düzeltme (kamera pozu/satır 183 **DEĞİŞMEDİ** – kalibrasyon sabit): column 0.335→0.385** (dış yüz≈0.40=kenar), tutucu kelepce 0.3225→0.3725**, camera_arm 12.5cm@0.2725 → 17.5cm@0.2975** (kameraya değen yakın uç 0.21 **SABİT**, uzak uç çubuğa uzatıldı). Y zaten ortadaydı. Kurulu xacro src'ye **symlink** → rebuild gerekmedi, sadece stack restart. Çubuk geriye gidince collision açısından da daha güvenli.

烙 **GERÇEK ROBOT BAĞLI + RViz AÇIK** (oturum sonu durumu):** `demo.launch.py use\_camera:=false use\_fake\_hardware:=false use\_rviz:=true` çalışıyor – 3 controller active, bridge `/dev/ttyTHS1`'e bağlı, gerçek eklem açıları geliyor (kol ~0'da, gripper~92), RViz move_group'a bağlı (MotionPlanning hazır, Image display yok → segfault yok). RViz tek başına `moveit\_rviz.launch.py` ile de tazelenabiliyor (stack'e dokunmadan). △ **STACK HÂLÂ AÇIK** – kapatınca robotu 0'a parkla + background temizle. Pkill self-match tuzağına dikkat (bu oturumda exit 144 yedik → PID ile veya bracket-pattern öldür). [[feedback_pkill_self_match]] [[feedback_background_node_cleanup]] [[feedback_servo_drop_on_shutdown]]

□ **PLACE kalibrasyonu** (oturum 36) **HÂLÂ AÇIK:** `calib\_check.py` ile yakın/uzak ölçüm + sabit-offset vs extrinsic kararı bekliyor (aşağıda oturum 36).

• önceki oturum:

****Tarih:** 2026-06-23 oturum 36 – **PLACE KALİBRASYON TEŞHİSİ:** "tıklanan nokta ile gripper ucunun gittiği yer baya uzak" sorununun kök neden analizi + salt-okunur algı testi aracı (`calib\_check.py`) yazıldı. ÖLÇÜM BEKLİYOR.**

□ ****SORUN (kullanıcı):**** `click\_place\_moveit.py`'de ekranda tıklanan nokta ile gripper ucunun ulaştığı nokta birbirinden epey uzak.

☞ ****TEŞHİS – sapma İKİ ayrı zincirin karışımı, ayırmak şart:****

1. ****Tasarım payı:**** `HOVER\_OFF=0.13` → gripper bilerek hedefin ****13 cm ÜSTÜNE**** gider (hover). Dikey boşluğun çoğu BU olabilir, hata değil. Gerçek hatayı ölçerken sadece ****XY****'ye bak.

2. ****Algı zinciri**** (piksel+derinlik → robot_base): kamera intrinsics + aligned depth + ****hand-eye extrinsic****. Extrinsic URDF'te ELLE yazılı sabit poz: `src/mycobot\_world/urdf/mycobot\_world.urdf.xacro:183` → `\*\*Park\*\*, 17 poz; Tsai-Lenz z'yi çöktürmüştü). `calib_touch_results.md` doğrulaması ~1.5cm demiş AMA confound'lu (gripper oryantasyonu + tcp=parmak ucu).

3. ****Hareket zinciri**** (hedef → eklem açısı → gerçek uç): `click\_place\_moveit.py`'nin KENDİ analitik FK/IK modeli (`\_0`,`\_FGB`,`\_FTCP`) + pymycobot `send\_radians` ****AÇIK-DÖNGÜ servo**** (mycobot280 tek başına ~30mm tekrar hatası).

% ****KİLİT İPUCU:**** `calib\_touch\_results.md`'de Δx SABİT DEĞİL, BÜYÜYÖR (+25mm → +41mm). Sabit offset olsaydı her yerde aynı olurdu → bu ****extrinsic rotasyon/ölçek hatasına**** işaret eder → `nudge`+`S` ile kaydedilen sabit offset (`.place\_offset.json`) bunu TAM düzeltemez, tek noktada maskeler. (Teyit için temiz ölçüm lazım, aşağıda.)

****YENİ ARAÇ –** `~/ros2\_ws/calib\_check.py` (SALT-OKUNUR, sözdizimi OK): ****click_place ÇALIŞIRKEN** paralel açılabilir, ****seri porta/pymycobot'a DOKUNMAZ, robotu HAREKET ETTİRMEZ****. Sadece kamera+TF dinler; tıklanan pikselin robot_base 3B koordinatını ekrana yazar/loglar (`pixel\_to\_base` matematiği click_place ile AYNI: 7×7 medyan derinlik). Amaç: ALGI zincirini hareketten izole etmek. Tuşlar: Sol=nokta, Sağ=son-sil, k=kaydet (`calib\_check\_points.json`), q=çık.

□ ****YARIN İLK İŞ – ölçüm topla, sonra fork:**** İki fiziksel hedef koy (robota ****yakın ~15cm**** + ****uzak ~30cm****), `calib\_check.py` ile her birine tıkla → yazılan base koordinatını ****cetvelle ölçülen gerçek konumla**** karşılaştır. Karar:

- İki noktada XY hatası ****~aynı (yön+miktar)**** → SABİT offset → `click\_place\_moveit.py`'de nudge ile hizala + `S` (kaydet `.place\_offset.json`). Hızlı.

- Hata ****uzakta büyüyor / yön değişiyor**** → ****extrinsic**** hatası → easy_handeye2 yeniden (`calibrate\_full.launch.py`) VEYA URDF kamera pozunu temiz ****top-down**** dokunmayla düzelt (gripper oryantasyonunu SABİT tut ki tcp x-y = parmak ucu = vida x-y olsun).

- Tıklanan koordinat doğru ama uç yine kayıyorsa → ****hareket**** hatası (analitik IK modeli / açık-döngü servo), ayrı iş.

****Tarih:** 2026-06-21/22 oturum 35 – 0-PARK + çeyrek-daire demo + `click\_arc.py` + kamera FOV (RGB=1280×720) + ****PLACE:** fixture görüldü (delikli blok), teach denendi, `click\_place\_moveit.py` yazıldı – vida DİK + çarpışmasız plan ÇALIŞTI, kalan: XY ofset (nudge ile düzeltilecek).****

烙 ****ELLE 0-PARK:**** Kullanıcı isteğiyle robot sıfır noktasına gönderildi: doğrudan `pymycobot` reçetesi (`send\_radians([0]\*6, 25)` → `is\_moving==0` bekle → `set\_gripper\_value(100)`). Sonuç: tüm eklemler 0, en büyük |açı| = ****0.021 rad (~1.2°)****, servolar torklu (kol düşmedi), gripper açık. Port `/dev/ttyTHS1` (Jetson UART; USB değil) boştaydı, ROS stack çalışmıyordu. LC_NUMERIC=C set edildi. [[feedback_servo_drop_on_shutdown]]

[[feedback_locale_lc_numeric_c]]

□ ****ÇEYREK-DAİRE HAREKETİ** (platformdan $\geq 3\text{cm}$, sürtmesiz) – yöntem önemli, ileride tekrar kullanılır:**** Kullanıcı "kinematik sınırlar içinde platformdan 3cm yukarıda çeyrek daire çizdir, gripper en altı sürtmesin" dedi.**
****Yaklaşım:**** (1) URDF `mycobot\_280\_jn` zincirinden FK + sayısal IK kuruldu (numpy, salt offline hesap). (2) ****Bridge 1:1 doğrulandı**** → `send\_radians` açıları = URDF eklem açıları (işaret/offset YOK), yani URDF-FK gerçek donanımla birebir uyuşur. (3) ****Platform üst yüzeyi = robot_base frame'inde $z=0$ **** (mycobot_world.urdf.xacro: platform_top joint $z=-0.0125$ + yarı kalınlık 0.0125). (4) ****KİLİT FİKİR – sabit-yükseklik garantisi:**** gripper dik-aşağı + tcp $z=0.045$ 'te bir poz IK ile çözülüp ****SADECE joint1 süpürülür**** ($\pm 45^\circ = 90^\circ$ yay, $R=0.13\text{m}$). joint1 dönüşü Z'yi ve yarıçapı DEĞİŞTİRMEZ → tüm yay boyunca yükseklik matematiksel olarak sabit, frame-konvansiyonu hatalarına başışık. (5) Süpürme+geçişler önceden simüle edildi: min kol-frame Z = ****4.5cm**** (tcp en alt; fingertip=tcp → platformdan $\geq 3\text{cm}$, $\sim 1.5\text{cm}$ pay), 6 eklem de limit içinde, engellerden (kolon/kelepçe $x \approx 0.32$) uzak. Sıra: sıfır-hover(12cm)→iniş(speed18)→13-adım yay(speed22)→çıkış→sıfır(speed28). Canlı çalıştı, robot sıfırda bitti (max sapma 0.023 rad), gripper açık. Hesap+yörünge: `/tmp/quarter\_circle.json` (geçici).
[[feedback_command_speed_real_speed_cap]]

****`click\_arc.py` (YENİ araç, `src/mycobot\_calibration/scripts/click\_arc.py`) – kamera tıkla → gidişat tahmini → yay çiz:**** click_to_go.py temel alındı. Kullanıcı kamera görüntüsüne ****5 nokta**** tıklar ($\geq 3 + P$), nokta piksel-aligned depth→3B kamera→TF robot_base'e dönüşür. Noktalara ****en-iyi-uyum (least-squares) çember**** (`fit\_circle`) oturtulur ($\approx$ doğruysa doğru-mod), ****tıklanan span (yeşil) + ~ 30 ekstrapolasyon "tahmin" (turuncu)**** yay üretilir, her waypoint ****kendi gömülü numpy FK/IK'sıyla**** (gripper dik-aşağı) çözülür + limit/ $\geq 3\text{cm}$ /engel doğrulanır (geçersizse E hareket etmez; tahmin kısmı erişilemezse otomatik kısaltılır). ****E**** → tek `JointTrajectory` (hover→iniş→yay→çıkış→home) `/arm\_controller/follow\_joint\_trajectory` action'a gönderilir. Yükseklik: tcp $z=0.045$ (platform=0), `MIN\_CLEAR=0.030`. Hız: `\_build\_traj`'te segment-taban $1.0\text{s} \rightarrow 0.12\text{s}$ + `VEL\_FREE 0.6→1.4` (önceki $1.0\text{s}/\text{segment}$ yapay yavaşlığı düzeltildi; gerçek tavan bridge `command\_speed=30`). IK Jetson'da yavaştı (39s) → ****sıcak-başlangıç**** (ilk wp multi-seed, sonrası önceki çözümden) + arka-thread → $\sim 2-6\text{s}$, GUI donmaz.

****MİMARİ NETLEŞTİ** (kalıcı bilgi):**** gerçek stack = `demo.launch.py use\_fake\_hardware:=false use\_camera:=... use\_rviz:=...`. `MyCobotHardware` (ros2_control plugin) ****kendi içinde `mycobot\_bridge.py`'yi spawn edip portu (`/dev/ttyTHS1`) açar**** ve socket'ten konuşur → ****hem `/arm\_controller/follow\_joint\_trajectory` (ros2_control) hem `/tmp/mycobot\_bridge.sock.cmd` socket**** kullanılabilir; bridge `send\_radians` açıları = URDF açıları (1:1). Bu oturumda click_arc bu stack'le çalıştı (camera ayrı `rs\_launch`, RViz kapalı).**

📷 ****KAMERA FOV ANALİZİ** (kalıcı bilgi, [[feedback_camera_fov_resolution]]):**** Kullanıcı "robotun sağ tarafı/dizme alanı kamerada görünmüyor, yükseltsem/çözünürlük artsa bozulur mu" sordu. `rs-enumerate-devices -c`: ****D435i renk FOV $\sim 70^\circ \times 43^\circ$ TÜM 16:9 profillerde**** ($424 \times 240 = 69.85^\circ$, $1280 \times 720 = 70.19^\circ$ – fark ihmal); ****4:3 profiller 55° (daha DAR yatay)****. Yani çözünürlük FOV'u GENİŞLETMEZ. Derinlik/stereo $\sim 87^\circ$ (renkten geniş) ama okülüzyon kalır. Viewer'da "daha geniş" görünmesi = sadece yüksek-çöz+büyük-pencere netliği → ****ROS renk profili $424 \times 240 \rightarrow 1280 \times 720$ 'ye çıkarıldı**** (kalibrasyon BOZULMAZ: intrinsics camera_info'dan canlı, extrinsic TF kamera oynamadıkça geçerli) + click_arc penceresi 1280×720 . Görünmeyen ARKA kısımlar FOV değil okülüzyon/bakış-açısı → çözüm 2.kamera / kamerayı taşı+yeniden-kalibre / robotu-parçayı döndür.**

□ ****PLACE** (vidaları dik dizme) – bu oturumun ASIL İŞİ, çok ilerleme + kalan tek sorun:******

□ ****FIXTURE görüldü**** (kullanıcı foto, Downloads): ****3B baskı siyah delikli blok, 4 sütun $\times$ 5 satır = 20 delik****, vidalar ($\sim 44\text{mm}$ renkli kafa) ****şaft delikte → dik durur****, renye göre satır satır (siyah/sarı/beyaz/yeşil/kırmızı). Düz yüzeyde vida durmaz → bu fixture şart.

□ ****TEACH denendi** (sonra terk edildi):**** pymycobot `release\_all\_servos()` ile free-drive → gripper'ı köşe deliğe elle götür → `get\_radians`+FK ile poz kaydet (`fixture\_corner.json`: sol_ust, sag_alt). ****AMA fixture sonradan TAŞINDI → bu kayıtlar GEÇERSİZ.**** Ders: pymycobot serial-close torku BIRAKMAZ (kol düşmez) – sadece ros2_control bridge shutdown'ı bırakır. Teach grid için 3 köşe gerekir (TL+TR+BL, dönük olabilir).**

% ****KİLİT PROBLEM – vida YATAY tutuluyor, DİK dizilmeli:**** pick top-down → vida gripper'da ****yatay**** (eksen

yaklaşmaya dik). Deliğe dik sokmak için vida ****dik**** olmalı. Gripper rijit → ya regrasp ya ****bilek döndürme****.
Çözüm A (kullanıcı seçti): gripper'ı ****YAN**** çevir (yaklaşma YATAY) → vida eksenini aşağı → dik. IK:
pos(3)+****"gripper_base eksenini AŞAĞI"***** hizala (azimut serbest, geodezik so3 değil cross yeter); `A` tuşu eksenini
Z→X'Y değiştirir (vidanın gerçek eksenini hangisiyse). Robotta ****vida DİK çıktı****.

△ ****FIXTURE base'e ÇOK YAKIN olamaz**** yakın hedefte (≤ 16 cm) yan-yatık poz kolu geri katlayıp ****bileği robot_base kutusuna sokuyor**** (RViz'de son pozda kırmızı). IK'ya ****base-kutu reddi**** eklendi (`in\_base\_box`, $0.15 \times 0.11 \times 0.11$ z0-0.11). Fixture ****~22-24cm uzağa + ~5cm yükseğe**** alınınca temiz poz var (test , kullanıcı taşıdı).
`HOVER\_OFF=0.13` (bilek kutu üstünde kalsın + "yüksekten bırak").

****`click\_place\_moveit.py` (YENİ, asıl place aracı, `src/mycobot\_calibration/scripts/`):**** Önce ****`click\_place.py`** yazıldı (pymycobot DOĞRUDAN send_radians) ama ****çarpıştı/sürttü**** → DERS: ****IK sadece HEDEF pozunu çözer, YOLU değil;** çarpışmasız yol için PLANLAYICI (MoveIt) gerekir. ****Doğru mimari: `demo.launch use\_fake\_hardware:=true use\_rviz:=true` (move_group+RViz+collision model, **port'a dokunmaz**) + **pymycobot gerçek hareket** + ayrı `rs\_launch` (1280x720). Araç: kamera tıklama→3B hedef-eksen-hizalama IK joint goal-****move_group plan_only**** (`req.start\_state`=pymycobot `get\_radians` GERÇEK açılar, fake /joint_states değil!)-****RViz önizleme**** (DisplayTrajectory)→****E onayı****→planlanan yörüngeyi ****pymycobot ile izleyerek**** uygula (subsample, 0.5 rad/s). Tuşlar: O/C grip, P plan, E çalıştır, A eksen, ****8/2/4/6 hedefi 1cm kaydır (xy), +/- z**** (her nudge yeniden planlar), H home, R bırak. ****RViz: `moveit\_rviz.launch.py rviz\_config:=~/ros2\_ws/rviz\_preview.rviz`**** (Loop Animation açık; pencereyi KAPATMA = rviz ölür). `vo\_rsp.launch.py` repo KÖKÜNDE → dosya yoluyla başlat.**

****CANLI SONUÇ (oturma sonu):**** robot çarpmadan gitti, ****vida DİK****, RViz çarpışmasız önizleme çalıştı. ****KALAN TEK SORUN: XY offset**** – vida deliğinin "gerisinde" düşüyor (bilinen ~2.5cm TCP hatası + yan-tutuş geometrisi). ****ÇÖZÜM hazır:**** nudge tuşları (8/2/4/6) ile RViz'de hizalama→E→kameradan bak, döngü. ****YARIN:**** nudge ile vidayı delik üstüne hizala, doğru offset'i bul → araca ****kalıcı varsayılan düzeltme**** olarak göm → sonra renk-delik grid'i (3 köşe teach, fixture TAŞINDIĞI için yeniden) + sayaç + seri dizme.

△ ****STACK DURUM (oturma sonu):**** Oturma sonunda robot ****0'a parklandı (pymycobot, gripper açık)****; arka plan ROS (move_group-fake + rviz + kamera) kapatıldı (RAM). Yarın: önce DEVLÖG oku → fixture'ı yerinde bırak (taşıma!) → stack'i kur (demo.launch fake+rviz + rs_launch 1280x720) → `python3 src/mycobot\_calibration/scripts/click\_place\_moveit.py`. `[[feedback_servo_drop_on_shutdown]]`
`[[feedback_background_node_cleanup]]`

****Önceki (2026-06-18 oturma 34)**** – ****TOP-DOWN (DİK) İNİŞ ZORLANDI** → pick canlı çalıştı + sıfıra hızlı döndü** + $\propto$ hız (command_speed 15→30, rapid %100) + overlay sadeleştirme + view_camera.sh + kamera-FOV kenar elemesi.

□ ****EĞİK İNİŞ → KOMŞU VIDALARA ÇARPMAYA ÇÖZÜLDÜ (asıl kazanım):**** Eski davranış: `grasp\_quat\_from\_axis` ölçülen PCA tilt'ini (vida ekseninin düşey bileşeni) yaklaşma eksenine katıyordu → eğik vida ya da ****gürültülü tilt ölçümü**** (DEVLÖG'da tilt $2^{\circ} \rightarrow 73^{\circ}$ zıplama tuzağı) eğik yaklaşma eksenini → hover'dan grasp'e iniş ****yana süzülerek**** komşu vidalara çarpıp sahneyi bozuyordu. Canlı logda küçük 5° tilt bile ($\times 10$ cm hover = ~9mm yanıl) sık kümede komşuları oynatıyordu. ****FIX:**** detector'a `top\_down=True` (varsayılan) param eklendi → `grasp\_quat\_from\_axis(..., top\_down=True)` : eY (yaklaşma) HER ZAMAN (0,0,-1) dümdüz-aşağı, sadece ****yaw**** (yatay uzun-eksen) korunur, eğik PCA YOK SAYILIR. pick_tilt değişmedi – `approachAxis` otomatik (0,0,-1) alıyor → hover tam üstte, iniş dümdüz aşağı (yanıl=0). Mod etiketi artık ****DİK****. Bu oturma 20'de çalışan top-down modunun kalıcı hali. ****Vidaları aralıklı+ortaya koyunca PICK CANLI ÇALIŞTI**** (gerçek robot).

烙 ****ADIM 7 – vida aldıktan sonra SIFIRA hızlı dön (yeni):**** Eskiden lift sonrası havada tutup bekliyordu. Şimdi kavrama onayı (5b)+lift sonrası robot vidayı ****tutarak home(0)'a `rapid\_vel` (%100) ile DİREKT (önizleme/onay yok) döner****. Log: `>> ADIM 7 ... SIFIR (home) noktasına hızlı dön`. (pick_tilt.cpp, build+strings doğrulandı.)

$\propto$ ****HIZ – asıl kapı `command\_speed` (MoveIt scaling DEĞİL):**** Açık-döngü bridge her `send\_radians`'a SABİT

`command\_speed` veriyor → serbest-uzay gerçekte BUNA takılıyor (scaling %100 olsa bile). URDF'te `command\_speed default=15`'ti → **30 yapıldı** (mycobot_world.urdf.xacro:198, build+kurulu doğrulandı). Serbest-uzay ~2x hızlı; **descend korunur** (yavaş DESCEND_VEL_SCALE=0.10 trajectory-timing'le sınırlı, command_speed'e takılmıyor). Ayrıca `run\_pick\_preview.sh` `RAPID\_VEL\_SCALE` 0.40→**1.00**. Descend sarsılırsa command_speed 20-25'e çek. [[feedback_command_speed_real_speed_cap]]

粒 **OVERLAY SADELEŞTİRME (kullanıcı isteği):** `display\_min\_conf` (0.9): 0.9 altı vidalar ÇİZİLMEZ; çizilende sadece **nokta** (erişim halkası + `renk conf`/'r=' yazıları KALDIRILDI); alt xyz bilgisi küçük-ince-**beyaz**; **HEDEF işareti MAVİ** (vida renkleriyle karışmasın); üst sayaç `shown/total (conf>=)` . Robot-base reach polygon (yeşil/amber çember) DURUYOR.

□ **HEDEF RENGİ capture'da:** detector `/vida/target\_info` (String, "KIRMIZI vida conf=.. x.. y.. z..") yayınlıyor; `target\_latch.py capture` dinleyip ENTER'da yazıyor + JSON'a kaydediyor → FAZ B (kamera kapalı) replay'de de yazıyor. Artık kamera kapanınca hangi RENK vidaya gideceği belli.

☞ **`view\_camera.sh` (yeni):** SADECE kamera overlay + reach halkaları izleme; `use\_fake\_hardware:=true` → gerçek robota/porta dokunmaz (TF URDF'ten statik, eye-to-hand). Stack(TF)+kamera+detector+cam_viewer. ▲ DERS: `set -u` altında ROS `setup.bash` source PATLIYOR → `set +u ... set -u` ile sar (run_pick_preview zaten sarıyordu, ilk versiyonum sarmamıştı → hiç açılmadı, düzeltildi).

□ **KAMERA-FOV KENAR ELEMESİ (yeni):** `frame\_margin\_frac` (0.04): vida hedef olmadan ÖNCE kamera karesinin güvenli iç bölgesinde olmalı VE maskesi kare kenarına değmemeli (yarı-kesik/kenar derinliği güvenilmez). Bu, mevcut "kutu duvarı" köşe elemesinden (corner_edge_frac) AYRI katman – biri FOV kenarı, diğeri vida-yığının kenarı.

RViz tekrar-izleme: `rviz\_preview.rviz` `Loop Animation: true` → planlanan yörünge sürekli döngüde oynar, ENTER'a (gerçek hareket) basmadan istediğin kadar izlenir.

烙 **gripper park'ta AÇIK (yeni tercih):** 3 park scriptinin `park\_zero\_pymycobot()`'una `send\_radians([0]\*6)` sonrası `set\_gripper\_value(100)` eklendi → sıfıra dönünce/sıfırdarken parmaklar açık (tutulan vida bırakılır). Elle 0-park'ta da uygula. [[feedback_servo_drop_on_shutdown]]

□ **`home` = SRDF isimli poz, 6 eklem de 0** (= sıfır noktası). pick akışı bununla başlar (`ready` pozu var ama kullanılmıyor).

□ **SIRADAKİ:** yeni hız (command_speed 30) ile descend pürüzsüzlüğünü canlı doğrula; top-down pick'i tekrarla. Açıklar: TCP ~2.5cm offset, gripper readback geçersiz, **place adımı yok** (ADIM 7 sıfıra getiriyor ama bırakmıyor). Oturum sonu: robot SIFIRDA, gripper açık, port serbest, ROS temiz.

‡ önceki oturum:

Önceki (2026-06-16 oturum 33) – **RViz Tegra SEGFAULT kök neden ÇÖZÜLDÜ (gdb)** + **hedef seçimi "en ortada + en iyi görünen" + sert köşe elemesi KODLANDI+build** + robot sıfır/gripper-aç (pymycobot).

RViz `exit code -11` (segfault) ÇÖZÜLDÜ – kök neden gdb ile bulundu: RViz pencere açılır açılmaz çöküyordu (iki ardışık denemede de aynı yer). RAM (2.6Gi boş) / GPU (Tegra X1 nvgpu donanım GL) / OOM **DEĞİL**. `gdb -batch -ex run -ex bt` backtrace: `\_\_libc\_free(0x2a3)` ← `XFree` ← `RenderSystem\_GL.so` ←

`rviz\_rendering::RenderWindowImpl::resize` ← `exposeEvent`. Sistematik display-bisect (her config'i `timeout` ile çalıştır, rc=139 segfault / rc=124 hayatta): bare-rviz , sadece-Grid , tam moveit.rviz . İkiye bölünce: ****`rviz\_default\_plugins/Image` (CameraImage) + `moveit\_rviz\_plugin/MotionPlanning` AYNI config'te → segfault.**** Tek tek doğrulandı: MP+Image , MP+PointCloud2 . ****FIX:**** `moveit.rviz`'den (hem `src/mycobot\_moveit\_config/rviz/` hem install kopyası) ****sadece Image/CameraImage display'i silindi**** (MotionPlanning+ReachRings+DepthCloud korundu); yedek `.bak\_cameraimage\_segfault`. 2D kamera görüntüsü gerekirse `cam\_viewer.py`/rqt ile AYRI pencerede bak – RViz içinde MotionPlanning ile birlikte koyma. [[feedback_rviz_tegra_cameraimage_segfault]] Artık `demo.launch.py` use_camera:=false ile RViz açık kalıyor.

□ ****HEDEF SEÇİMİ YENİDEN YAZILDI** – "en ortada + en iyi görünen" tek vida (kullanıcı isteği: kutuya tüm vidalar konuldu, geniş gripper KÖŞE vidasını alamaz):** Eski mantık sadece en yüksek confidence'ı (artı sticky) seçiyordu, konum hiç hesaba katılmıyordu. Yeni (`vida\_detector\_node.py` `\_on\_timer` seçim bloğu, build):

- ****Merkez referansı = aday vidaların piksel-centroid'i**** (= vida yığını/kutu merkezi; kalibrasyon gerekmez, kutu nereye konursa uyum sağlar).

- ****SERT köşe elemesi**** (kullanıcı seçti): adayların eksen-hizalı bbox'ı = kutu iç sınırı; kenara `corner\_edge\_frac` (0.15)×bbox kadar yakın vidalar (kutuyu duvarı → gripper çarpar) aday-DİŞİ silinir. ****Hepsi kenardaysa hedef YAYINLANMAZ, robot bekler.**** `corner\_min\_screws` (4) altında eleme atlanır (son birkaç vidayı sonsuza dek köşe sayıp mahsur bırakmamak için).

- ****Skor = `center\_weight` (1.0)×merkezlik + `conf\_weight` (1.0)×confidence**** → max seçilir. Sticky takip eleme-sonrası korundu.

- ****Overlay görseli:**** magenta "merkez" işareti (`\_draw` yeni `centroid` param) → kullanıcı kararı gözle görür.

- Yeni paramlar launch'a da eklendi (`vida\_detector.launch.py`). ****Sentetik test 4 senaryo GEÇTİ**** (3×3 ızgara→tam orta; orta düşük-conf→yine orta; <4 vida→eleme yok; halka→None=bekle). Δ ****DONANIMDA test EDİLMEDİ**** – kutuyla canlı doğrulanacak; köşe elemesi fazla/az gelirse `corner\_edge\_frac` oynat (0.18-0.25 daha agresif).

烙 ****Robot sıfır + gripper aç (pymycobot, port boştu):**** oturum başında bridge ölüydü (socket bayat), port serbest → `MyCobot('/dev/ttyTHS1',1000000)` `send\_angles([0]\*6,30)` (zaten ~1.5° içindeydi, torklu sıfırda) + `set\_gripper\_value(100,50,1)` → ****grripper açıldı (kullanıcı gözle teyit etti).**** [[feedback_servo_drop_on_shutdown]] [[feedback_gripper_no_position_readback]]

□ ****SIRADAKİ:**** kutuyla canlı pick – yeni merkez-seçimi + RViz-fix birlikte doğrula; `corner\_edge\_frac` saha-ayarı; (kalan açıklar: TCP ~2.5cm offset, gripper open-loop, place adımı yok).

‡ önceki oturum:

****Önceki (2026-06-16 oturum 32)**** – ****İKİNCİ BAŞARILI PICK (gerçek robot, reçete tekrarlandı)**** + gerçek-robot+RViz desync mimarisi belgelendi + iniş-reach tuzağı CANLI doğrulandı.

賭 ****İKİNCİ BAŞARILI PICK:**** `run\_pick\_preview.sh` (`CONF=0.5 GRASP\_Z\_OFFSET=-0.025 DESCEND\_VEL\_SCALE=0.10 PREVIEW\_CONFIRM=1`) ile robot vidayı ****aldı + kaldırdı**** . Oturum 31 reçetesi ikinci kez tuttu → reçete tekrarlanabilir.

□→ ****İNİŞ-REACH TUZAĞI CANLI** (ilk deneme başarısız, restart ile çözüldü):** İlk capture ****r=195mm (yeşil bandın ÜST sınırı)**** + eğik yönelim (latch quat ow=0.667 ox=-0.744, z=-0.022) → `[descend] cartesian fraction = 0.544 → 0.803 < 0.95` → ****iniş SESSİZCE iptal****, `home-hover` döngüsü (retry 1/3, 2/3). Kullanıcı "ENTER'a basıyorum inmiyor" gördü. ****DERS (canlı doğrulandı):**** ENTER ****sadece HOVER**** kapısını onaylar; descend önizleme kapısı ****fraction<0.95'te HİÇ GÖSTERİLMEZ**** → arka planda sessiz fail; ENTER asla indiremez. Hedef FAZ A'da latch'li → vidayı oynatmak yetmez, ****RESTART (yeni capture) ŞART****. Vida ****daha yakın (~170-180mm) + daha düz**** konunca iniş

tamamlandı. → ****r=195 yeşil bandın kenarı = GÜVENLİ DEĞİL; ideal 170-180mm + minimum tilt.**** (oturma 31 notu " $r \leq 200$ " doğrulandı ama 195 bile eğik yönelimle 0.54'e düşebiliyor – reach + tilt birlikte belirleyici.)

□ ****GERÇEK-ROBOT ↔ RViz DESYNC = TASARIM, bozukluk değil (kullanıcı sordu, koddan teşhis):**** kol ile RViz aynı anda hareket etmiyor çünkü köprü ****gevşek/açık-döngü****, sıkı servo-loop değil. ****3 ayrı zaman tabanı:**** (1) ``mycobot_hardware.cpp` write()` → ``send_radians` <hedef> <command_speed≈30>`` → servolar **\*\*sabit hızda, asenkron,** kendi profiliyle gider; yörünge hız profilini takip ETMEZ → RViz'in planlanan-yörünge animasyonu gerçek koldan farklı sürede biter. (2) ``/joint_states`` = bridge'in seri ``get_radians()`` okuması **\*\*~20Hz + ~50ms round-trip\*\*** (``--rate 20``, "serial round-trip ~50ms") → RViz robot modeli fiziksel kolu **\*\*~50-100ms geriden\*\*** takip eder. (3) ``write()`` ``write_period_s`` + ``change_threshold_rad`` ile ****seyrek/eşikli**** komut yazar (nokta-nokta akıtmaz). Lockstep beklenmez; pick için sorun değil (önizleme+çarpışma RViz'de ****plan**** üzerinden; gerçek hareket onaylı planın açık-döngü yürütmesi). Sıkılaştırma sınırlı: `command_speed` yörünge süresi kabaca eşlenebilir, poll ~20Hz seri tavanı, ~50ms gecikme pycobot ile yok edilemez.

⚙ ****OTURUM YÖNETİMİ DERSLERİ:**** Stack teardown'da yine ****kill self-match (exit 144)**** vurdu – ``pgrep -f`` ile for-loop'taki desen metinleri (``move_group``, ``mycobot_hardware``...) ****kendi komut satırımı**** eşleştirip shell'i öldürdü → ****sadece sayısal PID ile `kill`**** çözdü (desen YOK). Her teardown'da bridge ölünce ****servo-drop**** (J4 -128°'ye düştü) → ****pycobot `send\_radians([0]\*6,25)` park**** kurtardı (torklu, kol 0'da). `[[feedback_kill_self_match]]` `[[feedback_servo_drop_on_shutdown]]`. ****Oturma sonu durumu:**** sadece RViz çalışıyor (kamerasız, ``demo.launch.py` use_camera:=false``, en hafif mod), robot sıfır noktasında, port ttyTHS1 bridge'de.

• önceki oturma:

****Önceki (2026-06-15 oturma 31)**** – ****İLK BAŞARILI UÇTAN UCA PICK (gerçek robot)**** + base-kasası çarpma kök nedeni ÇÖZÜLDÜ + İKİ FAZLI RViz-ÖNİZLEME MİMARİSİ kuruldu+çalıştı.

□ ****İLK BAŞARILI PICK (gerçek robot, `run\_pick\_preview.sh` ile):**** detect→kaydet→detector kapat→RViz önizle→her hareket ENTER onay→****iniş→kavra→kaldır = vida parmaklar arasında havada.**** Tüm zincir gerçek robotta çalıştı: base-box fix (katlanma/çarpma YOK), her hareket RViz'de önizlendi+onaylandı, yavaş iniş, ****~2.5cm offset yüksekliği düzeltti****, gripper -0.74'te ~10-12mm plastik vidayı TUTTU (grip-sıkılığı yeterli).

% ****BAŞARILI KAVRAMA REÇETESİ (kaydet):**** ``CONF=0.5`` + ``GRASP_Z_OFFSET=-0.025`` + ``DESCEND_VEL_SCALE=0.10`` + ``PREVIEW_CONFIRM=1``; algılanan hedef $z \approx 2\text{mm}$ → grasp $z \approx -0.023$; $r \approx 200\text{mm}$ (150-200mm band); düz vida top-down; gripper closed=-0.74 / open=0.15.

⊗ ****gripper readback CANLI doğrulandı GEÇERSİZ:**** komut -0.74 iken ``/joint_states` gripper_controller=0.063``; firmware bu oturumda 9-20 raporladı ama komutla/gerçek kapalı state'le uyumuyor (readback güvenilir) → ****model gripper'ı gerçek kapalı göstermez**** (RViz parmak açıklığı güvenilir; kol/çarpışma/yükseklik güvenilir). Bkz `[[feedback_gripper_no_position_readback]]`.

□ ****AÇIK (kalibrasyon kümesi):**** (1) TCP frame ~2.5cm fazla (``gripper_base-tcp=0.110``; offset band-aid, kalıcı=ChArUco) (2) gripper readback open-loop (bridge son-komutu raporlamalı; firmware value 9-20 raporladı ama güvenilir) (2b) ****gripper AÇMA ros2_control yolu firmware'i hareket ettirİYOR**** – pick sonrası gripper_controller→0.15 "success" dedi ama açmadı (firmware value 9'da takılı); ****pycobot `set\_gripper\_value(100,50,1)`** doğrudan açtı (20→99)**. Bridge gripper write açık-yönde bozuk. Acil gripper açma yolu: stack kapat→port serbest→pycobot set_gripper_value(100). (3) place adımı yok (pick_tilt sadece tutar). • aynı oturma, önceki iş:

□ ****OTURUM 31 EK DENEMELER (başarılı pick sonrası 2 ders):****

- ****REACH: iniş için $r \leq 200\text{mm}$ **** Vida $r=235\text{mm}$ 'de: hover ULAŞILABİLİR ama ****cartesian iniş TAMAMLANAMIYOR**** (``fraction`` 0.65-0.76 < 0.95, kol neredeyse tam uzanmış). İlk başarılı pick $r=201\text{mm}$ 'deydi (sınır). Vidayı ****yeşil banda (~170-195mm, ideal 180mm)**** koy. Δ Önizleme akışında hedef FAZ A'da latch'lenir → iniş başarısızsa vidayı oynatmak yetmez, RESTART (yeni capture) gerekir. NOT: iniş ``fraction<0.95`` ise descend gate GÖSTERİLMEYEN iptal eder (kullanıcı "ENTER'a basıyorum inmiyor" görür – aslında arkada sessiz fail).

- ****D435i STREAMING STALL (autosuspend DEĞİL):**** bugün ~6+ kez başlat/durdur sonrası kamera node "RealSense Node Is Up!" + profil açık + `power/control=on` AMA ****frame akıyor**** (detector "henüz color yok" 30sn+). Çek/dur USB akışını takıyor. ****ÇÖZÜM:** stack kapat → D435i USB çek-tak (ya da sysfs unbind/rebind) → restart.** Bkz [[feedback_jetson_usb_autosuspend]].

□ ****KOL KENDİ BASE KASASINA ÇARPMA + İKİ FAZLI RViz-ÖNİZLEME (oturum 31 ilk yarısı):****

烙 ****DONANIM DENEME 7-8 (offset deneyleri):****

- ****Deneme 7 (offset=0, CONF=0.5):**** tam zincir çalıştı, 2. onay + KILITLI overlay canlı çalıştı, AMA parmaklar vidanın ****~2.5cm ÜSTÜNDE boş kapandı**** (kullanıcı ölçtü: fingertip+zemin 2.5cm; kamera2+overlay'den boş gripper doğrulandı). Vida DÜZ yatıyordu ama detector tilt'i ****2°~73°** zıplatıp yanlış 72°'de kilitledi**. ****Z zincir hatası ≈2.5cm net ölçüldü.****

- ****Deneme 8 (offset=-0.020):**** grasp'e ulaştı AMA ****kol KENDİ base kasasına çarptı**** → kök neden avı.

□ ****KÖK NEDEN – BASE KASASI COLLISION MODELDE YOKTU (ÇÖZÜLDÜ+DOĞRULANDI):**** `robot\_base` BOŞ link'ti (`<link name="robot\_base"/>`, collision yok). Robotun fiziksel base kasası (USB/HMI girişli ~15×11×11cm kutu, kol+yer arası) collision modelinde ****HIÇ yoktu****. Mekanizma (kullanıcı tarif etti): ****vida çok yakın → gripper'ı dik tutmak için joint2 sertçe döner → forearm(joint3) base kasasına katlanır → MoveIt orada model'de hiçbir şey olmadığı için "serbest" görür → plan kabul → fiziksel çarpışma.**** ****FIX:**** `mycobot\_world.urdf.xacro` robot_base linkine gerçek-ölçü collision box eklendi (`box 0.15×0.11×0.11`, origin z=0.055, kol ekseninde ortalı).

△ ****YANLIŞ İLK DENEME (geri alındı):**** önce SRDF `joint1×joint3 "Never"` disable'ı silindi; ama `check\_state\_validity` gösterdi ki joint1 collision ****SİLİNDİRİ (r=4.5cm, 15.6cm boy)**** forearm'la ****neredeyse HER öne-uzanma pozunda**** kesişiyor → robot hiçbir vidaya uzanamıyordu → disable ****GERİ EKLENDİ****. Doğru fix tek başına robot_base box.

****DOĞRULAMA (`/check\_state\_validity`, fake-hw sim):**** HOME + tüm normal/uzanma pozları GEÇERLİ; ****sadece base'e katlanan pozlar **** (`robot\_base×joint3`, ağır katlanmada +joint4). Yedek: `config/mycobot.srdf.bak\_20260615\_joint1joint3`. ****Bu, oturum 28/30 SRDF-"Never"-gizleme sınıfının BASE versiyonu.**** Bkz [[feedback_base_box_collision_missing]].

****YENİ MİMARİ – İKİ FAZLI RViz-ÖNİZLEME (kullanıcı fikri, KURULDU+smoke-test GEÇTİ):**** Nano'da detector(2.1GB) +RViz aynı anda RAM'e sığmaz. Vida statik → ****FAZ A:**** tespit→`.vida\_target\_latch.json`'a KAYDET→detector+kamera KAPAT (RAM boşalır); ****FAZ B:**** RViz aç→kaydedilen hedefi `/vida/target`'a replay+marker-pick_tilt planla→RViz'de yörlünge+ÇARPIŞMA(kırmızı) gör→terminal ENTER onayı(×2)→çalıştır. Dosyalar: `run\_pick\_preview.sh`, `target\_latch.py` (capture/replay), `confirm\_key.py`, `rviz\_preview.rviz` (MotionPlanning=çarpışma görseli), `moveit\_rviz.launch.py` (RViz'i MoveIt param'larıyla başlatır – standalone `ros2 run rviz2` "No Planning Scene Loaded" veriyordu). ****pick_tilt'e SONRADAN eklendi:**** `preview\_confirm` param (home/hover/iniş/kaldır her biri plan→RViz önizle→`pick/confirm` ENTER→execute; cartesian DisplayTrajectory ile yayınlanır) + `descend\_vel\_scale=0.10` (yavaş iniş). ****run_pick_preview.sh`PREVIEW\_CONFIRM=1` set eder. **DERS:**** RViz önizlemesi ancak collision modeli kadar iyidir – base çarpışması RViz'de de görünmezdi (box'ı ekleyene kadar). "Sim'de herşey biliniyor" sadece model gerçeği tam yansıtırsa doğru.

⊗ ****servo-drop yine canlı doğrulandı:**** kapanışta joint5 boşalınca joint6+gripper sarktı → pymycobot park kurtardı (kol 0'da). [[feedback_servo_drop_on_shutdown]]. Robot oturum boyunca her durakta `arm\_controller` 0-trajectory (bridge canlı) + pymycobot park ile güvende tutuldu.

**** (Bu ilk-yarı SONRAKİ'leri YAPILDI – üstteki "İLK BAŞARILI PICK'e bak):**** kuru çalıştırma + gerçek robotta pick tamamlandı. Geriye kalan açık işler üstteki "AÇIK (kalibrasyon kümesi)" maddesinde toplandı (TCP ~2.5cm, gripper readback/write, place adımı; ayrıca tilt 2°~73° zıplaması [bu turda olmadı], 3-çaplı vida yamuk geometrisi). † önceki oturum:

****Önceki (2026-06-14 oturum 30)** – **KULLANICI İSTEĞİYLE 4 GELİŞTİRME KODLANDI+DERLENDİ (donanım/sim testi BEKLİYOR).****

****OTURUM 30 – 4 özellik (hepsi derlendi, HENÜZ test edilmedi):****

1. ****Kavrama sonrası 2. onay**** (`pick\_tilt.cpp` ADIM 5b): kavradıktan SONRA, kaldırmadan ÖNCE robot grasp pozunda DURUR, ikinci `/pick/confirm` ('g') bekler → "vida gerçekten tutuldu mu?" kontrolü. Kavrayamadıysa Ctrl+C. (Eskiden tek onay vardı, o da hover'da inişten önceydi.) Binary `strings` ile doğrulandı (ADIM 5b VAR).

2. ****Kafa-farkında grip**** (`detection.py` `head\_aware\_grip` + detector): maske şekinden vida KAFASINI bulur (uzun-eksen iki ucundan geniş=kafa), grip noktasını kafanın hemen ALTINDAKİ düz şafta kaydırır (`grip\_inset\_frac=0.30`). Geçerli derinlik varsa hedef o noktadan üretilir, yoksa centroid'e düşer. Param: `grip\_under\_head=True`, `grip\_inset\_frac`. Sentetik maske testi: kafa doğru uçta, grip şafta kaydı (ratio 3.8). Δ DONANIMDA doğrulanmadı – inset_frac ayar gerektirebilir.

3. ****Ekran kilitli hedef**** (`vida\_detector\_node.py` `\_draw`): son yayınlanan hedef + 3B eksen CYAN "KILITLI" işaretiyle her karede sabit çizilir (`\_last\_pub\_p`/`\_last\_axis\_base` base'den geri-projekte) → robot vidanın üstünü kapatıp canlı tespit kaybolrsa bile hedef görünür kalır.

4. ****Uzun eksen her zaman + kafa işareti****: seçilen vidada turuncu uzun-eksen çizgisi HER karede (sadece güvenilir halde değil), KAFA ucu kırmızı daire+etiket, HEDEF yeşil çarpı grip noktasında.

- `cam\_viewer.py` ipucu güncellendi (g = 2 aşamalı onay). Tüm değişiklikler `colcon build` TEMİZ (mycobot_demo + vida_vision), py_compile temiz.

5. ****GÜVENİLİR 0-PARK (cleanup fix)**** Kullanıcı canlı gözledi – Ctrl+C'de `mycobot\_bridge` kapanırken servolar boşalıp robot DÜŞÜYOR; action-tabanlı `goto\_zero` kurtarmıyor. `run\_pick\_tilt.sh` + `run\_approach.sh` `cleanup()`'ına ****`park\_zero\_pymycobot()`**** eklendi: stack öldükten + port boşaldıktan SONRA pymycobot `power\_on()`'+`send\_radians([0]\*6,25)` ile 0'a park, servolar TORKLU kalır (kol düşmez). bash -n temiz. Detay+yöntem: [[feedback_servo_drop_on_shutdown]]. NOT: pkill -f self-match tuzağı bu oturumda yine vurdu (exit 144) – PID/bracket ile öldür.

6. ****cam_viewer overlay etiket çakışması fix**** sol panelde cam_viewer'ın "Kameral D435i + tespit (HEDEF)" etiketi ile detector overlay'inin "N vida" yazısı üst üste biniyordu. `\_panel(..., show\_label=False)` eklendi; sol panel (D435i overlay) artık etiket çizmiyor, sadece detector'ın "N vida" yazısı kalıyor. Sağ panel etiketi duruyor.

烙 **DONANIM DENEME 4-5 (oturum 30, canlı):**

- ****Deneme 4 (offset YOK)**** Tam zincir çalıştı – hover→1.'g'→descend fraction 1.000→kavra→****ADIM 5b 2.onay bekledi (YENİ özellik ÇALIŞTI)****. AMA kullanıcı ölçtü: XY mükemmel (vidanın tam üstü+ortası, kafa-altı grip OK) ama ****dikeyde ~1.5-2.5cm YÜKSEK**** kapandı → boşa kavrama. 2. kameradan + overlay'den doğrulandı; ****yeni "KILITLI" cyan overlay de çalıştı**** (occlusion'da hedef sabit kaldı). İptal sırasında bridge ölünce servo düştü→pymycobot park kurtardı (servo-drop dersi canlı doğrulandı).

- ****Deneme 5 (`GRASP\_Z\_OFFSET=-0.02`)**** kullanıcı seçti. Stack açıldı, robot home, AMA ****detector vidayı tespit edemedi ("vida yok")**** – vida önce amber kenarda+beyaz kâğıt üstündeydi; yeşil çembere taşındı, görünür ama ****conf=0.75'te hâlâ tespit yok.**** Muhtemel sebep: vidanın 3-çap geometrisi (aşağıda) + foreshortening + conf eşiği.

- ****KULLANICI GEOMETRİ İÇGÖRÜSÜ**** vidanın ****3 farklı çapı var (kafa / orta gövde / alt-uç)**** → düz yatsa bile doğal YAMUK durur (2 bölüm yere değer, 3.'sü havada). Bu, tek bir `GRASP\_Z\_OFFSET`'in her pozda çalışmamasını ve tilt/derinlik okumasının kırılma açılarını açıklıyor. İdeal: en geniş kararlı silindirik bölümden tut.

□ ****AÇIK SORUN**** yeşil çemberdeki görünür vida conf 0.75'te tespit edilmiyor → conf düşür (0.5?) ya da vidayı düz/gövdesi net göster + beyaz zemini kaldır. Robot deneme 5 sonunda 0'a parklandı (pymycobot, max|açı|=1.0°), süreçler kapatıldı.

烙 **DONANIM DENEME 6 (CONF=0.5, kullanıcı kendi terminalinde) – BASE/SELF COLLISION + 2 ÖNEMLİ BULGU:**

- Vida yeşil çembere kondu, tespit oldu, hover r=174mm z=-0.019, 'g' onay, descend fraction=1.000 "success" AMA ****kol fiziksel olarak base kutusuna/kendine çarptı**** (MoveIt collision görmedi → MODEL-GERÇEK uyumsuzluğu: base collision box veya enkoder ofseti). Acil durduruldu (pick_tilt SIGINT + action 0'a + pymycobot park). Robot 0'da güvende.

- ****REACH ÇEMBERİ DİSK DEĞİL DONUT OLMALI**** yeşil 0-200mm dolu disk çiziliyordu; ama ****r < ~150mm top-down'da kol aşırı katlanıp kollar birbirine/base'e çarpıyor**** (kullanıcı gözledi). DÜZELTME: `reach\_radius\_min=0.15` param

+ overlay'e iç KIRMIZI çember + per-vida halka donut mantığı ($r < \min$ veya $r > \max$ =kırmızı, $[\min, 200]$ =yeşil, $(200, 260]$ =amber). vida_vision derlendi. İdeal band **150-190mm**. Yeşil=200mm(20cm), amber=260mm(26cm) robot_base'den; 280mm resmi max SERBEST-uzay (top-down+platform'da daha az, çalışmıyor).

- **"0 vida ama ekranda hedef işaretli + robot oraya gidiyor" AÇIKLAMASI:** pick_tilt son YAYINLANAN `/vida/target`'ı latch'leyip ona gider (anlık tespiti tekrar kontrol etmez); detector sadece güvenilir tespitte yayınlar. Ekrandaki işaret = yeni "KILITLI" overlay'i (son hedefi occlusion'da bile gösterir). Yani fantom değil, latch'li son hedef – AMA risk: tek kötü hedef yayınlanırsa pick ona commit eder. ▲ İleride: hedef yayınlanmayalı X sn olduysa pick_tilt timeout/iptal eklenebilir.

□ **KÖK NEDEN 4 – SRDF FIXTURE COLLISION KONTROLÜ KAPALIYDI (collision'ın ASIL sebebi, ÇÖZÜLDÜ):** Aktif `mycobot.srdf`'te **column/camera_arm/tutucu_kelepce x tüm kol+gripper** çiftleri `disable\_collisions` reason="Never" ile işaretliydi (~35 çift) → MoveIt kolun **direğe/kelepçeye/kamera koluna** çarpmasını planlamada HİÇ görmüyordu → "execute success" der ama kol fiziksel olarak column'a (33.5cm dikey direk = kullanıcının "base kutusu"su) çarpıyordu. Kutu ölçüleri DOĞRU; sorun kontrolün KAPALI olmasıydı. **DÜZELTME:** 35 fixture x kol/gripper "Never" çifti silindi (132→97; yedek `mycobot.srdf.bak\_20260614`). **KORUNDU:** `platform\_top x gripper/parmaklar` (vida masa üstünde; gripper kavramak için masaya yaklaşmalı, açılırsa grasp reddedilir), fixture-fixture adjacency, robot_base çiftleri. XML geçerli, `mycobot\_moveit\_config` derlendi. ▲ TEST EDİLMEDİ: artık MoveIt direğe çarpan planı reddeder (güvenli) ama bazı grasp'ler "erişilemez" olabilir (column-free yol yoksa) – donanımda doğrula. Detay: [[feedback_srdf_fixture_collision_disabled]].

□ **SONRAKİ:** `./run\_pick\_tilt.sh` ile donanım denemesi – (a) overlay'de KAFA doğru uçta mı, grip kafa-altı şafta mı düşüyor, (b) iki onaylı akış ('g' hover, 'g' kavrama sonrası) çalışıyor mu, (c) kafa-altı grip kavramayı iyileştirdi mi (oturum 29'un ~1cm sorunu + GRASP_Z_OFFSET ile birlikte), (d) robot üstü kapatınca KILITLI işaret kalıyor mu. Robotu önce 0'a park et.

Önceki (oturum 29, 2026-06-12): **GENEL PROJE DENETİMİ + KÖK NEDEN 3 BULUNDU+DÜZELTİLDİ + SİM UÇTAN UCA GEÇTİ (oturum 28'in bekleyen kapısı KAPANDI).**

□ **KÖK NEDEN 3 – SRDF'te parmak-içi çift eksikti:** `gripper\_left2 x gripper\_left3` ve `gripper\_right2 x gripper\_right3` disable_collisions YOKTU. 4-bar kardeş linklerin yeni collision box'ları gripper **AÇIK (0.15)** pozda tasarım gereği üst üste biniyor → gripper açıldıktan sonra HER plan "invalid start state" ile reddediliyordu (FixStartStateCollision jiggle'ı da kurtaramıyor). Oturum 28'in 21:56/22:16 koşularının home'dan sonra asla ilerleyememesinin eksik parçası buydu. **DÜZELTME:** `mycobot.srdf`'e 2 satır eklendi (yorumlu, satır ~121).

☞ **demo.launch.py `position\_only\_ik: True→False`:** move_group bu paramı pose-goal IK örnekleyicisinde kullanıyor; True iken yönelim zorlanmıyor/çelişiyordu (kinematics.yaml=false ve pick_tilt.launch.py=False ile de çelişkiliydi). Artık 3 yer de tutarlı False – tilt-hızalı grasp için ÖN KOŞUL.

SİM UÇTAN UCA GEÇTİ (fake hardware, 2 senaryo): home→gripper aç→hover→'g' onay→**descend fraction=1.000**→kavra→**lift fraction=1.000**→TAMAM. Hem **top-down** hem **20° eğik** sentetik `/vida/target` ile; final TCP pozisyonu (0.050, -0.170, 0.060) ve quaternion'u komutla **birebir** (tf2_echo doğrulandı) → yönelim zorlaması + paralel iniş ÇALIŞIYOR. Eğik hedef doğrudan grasp yöneliminde, top-down 180°-flip'te çözüldü.

22:16 koşusu analizi: o koşu GERÇEK robottu (bridge loglu; "fake-hardware sim" hiç koşulmamış). `gripper\_base x column` teması model hatası DEĞİL – FK doğrulaması sıfır pozda ~20cm boşluk veriyor (gripper x≈0.05-0.12, column x≈0.32-0.35); robot fiziken kolona yakın pozda kalmıştı. **DONANIM ÖNCESİ KURAL:** robotu önce 0'a park et** (stack kapalıyken pymycobot `send\_radians([0]\*6, 25)`).

□ **DENETİM AÇIK MADDELERİ (kod incelemesi, henüz uygulanmadı):** (1) detector `conf` hâlâ 0.50 – oturum 26 kararı 0.75'ti; `run\_pick\_tilt.sh` detector'ı parametresiz `ros2 run` ile başlatıyor. (2) Konsensüs sadece MOD'a bakıyor, vida KİMLİĞİNE değil – iki vida arasında pozisyon zıplaması hâlâ mümkün (deterministik en-yakın/merkez seçim yok). (3) `PYTHONUNBUFFERED=1` run_pick_tilt.sh'de yok (oturum 28 dersi uygulanmamış). (4) `\_mask\_points\_3d`'te MAD k=2.5 hardcoded (`self.depth\_mad\_k` kullanılmalı). (5) pick_tilt'te retry sayacı yok – hover/iniş hep başarısız olursa sonsuz döngü (gözetimli kullanımda kabul). (6) rmem sysctl kalıcı değil (script her koşuda yapıyor; `/etc/sysctl.d/99-ros-rmem.conf` önerisi). (7) udev autosuspend kuralı VAR ama bu boot'ta D435i

`power/control=auto` (script telafi ediyor; kuralın reboot'ta çalıştığını doğrula). (8) `vida\_detector.launch.py` 21 paramdan 3'ünü geçiriyor (script launch kullanmadığından düşük öncelik).

□ ****OTURUM 29 DERSİ:**** pkill bracket-pattern ([v]ida) yetmiyor – komut satırındaki HERHANGİ bir düz-metin eşleşmesi (echo mesajındaki "pick_tilt durdu" dahil!) self-kill yapar (exit 144). pkill çağırın komutun TAMAMINDA hedef adı düz yazma.

****DENETİM İYİLEŞTİRMELERİNİN HEPSİ UYGULANDI** (aynı oturum, devamı):

1. ****conf 0.50→0.75**** (node default + launch default + `run\_pick\_tilt.sh` artık `-p conf=\$CONF` geçiyor; env: `CONF`/`MODE\_CONSENSUS`/`STICKY\_RADIUS`).
2. ****Yapışkan deterministik aday seçimi**** (`vida\_detector\_node.py`): tüm geçerli-derinlik+TF adayları toplanır; son YAYINLANAN hedefe `sticky\_radius` (0.05m) içinde aday varsa 0 seçilir, yoksa en yüksek conf → conf jitter'ı hedefi iki vida arasında zıplatamaz. `self.\_last\_pub\_p` steady yayında güncellenir.
3. ****PYTHONUNBUFFERED=1**** run_pick_tilt.sh'ye eklendi (detector log gecikmesi biter).
4. ****MAD k hardcoded fix****: `\_mask\_points\_3d` artık `self.\_depth\_mad\_k` kullanıyor; mode karşılaştırması dereceye sadeleşti (`\_grasp\_tilt\_min\_deg`).
5. ****pick_tilt.cpp retry sayaçları****: 5 ardışık hover başarısızlığı → home'a dön (poz tazele); 3 iniş başarısızlığı → VAZGEÇ (sonsuz döngü biter). Derlendi, binary ASCII-fragment ile doğrulandı.
6. ****run_pick_tilt.sh 0-DOĞRULAMA****: goto_zero sonrası gerçek `/joint\_states` okunur (rclpy inline); en büyük |açı| >0.09 rad (~5°) ise ABORT – 22:16 vakası (JTC "success" dedi ama robot fiziken 0'da değildi → MoveIt start-collision) bir daha sessiz ilerleyemez. Snippet canlı stack'e karşı test edildi.
7. ****Kalıcılık****: `/etc/sysctl.d/99-ros-rmem.conf` (8MB, sysctl --system doğrulandı) + udev autosuspend kuralı `udevadm trigger` ile tetiklendi → D435i `power/control=on` (kural reboot'ta da çalışır).
8. `vida\_detector.launch.py`'ye 11 operasyonel parametre eklendi (conf default 0.75).

****Sim E2E YENİ binary+detector default'larıyla TEKRAR GEÇTİ**** (home→hover[eğik, grasp yöneliminde]→onay→descend 1.000→lift 1.000). Detector yeni paramlarla temiz açılıyor ("Model hazır", param hatası yok).

烙 ****DONANIM DENEME 3** (aynı oturum, akşam) – İLK TUR ISKALADI → TEŞHİS → -1.5cm OFFSET → İKİNCİ TUR LOG'DA TAM GEÇTİ:

- ****Tur 3.1 (offsetsiz)**** Tüm zincir çalıştı – 0-doğrulama OK (0.026 rad), kırmızı vida conf 0.93-0.95, konsensus 3/3, yapışkan seçim sabit (BASE 124,-138,1mm 10+ karede milimetrik), hover grasp yöneliminde 0.9s'de çözüldü, descend+lift fraction 1.000. Yeni z-kapısı hover sonrası gelen çöp kareyi (z=-65910mm!) reddetti . ****AMA** parmaklar vidanın ~1cm ÜSTÜNDE kapandı – kavrayamadı.
- ****TEŞHİS (kullanıcı ölçtü)**** hover'da parmak ucu-vida merkezi ****6cm****, iniş 5cm → 1cm eksik. Model TCP'yi gerçekten ~1-1.5cm AŞAĞIDA sanıyor (model-gerçek zincir hatası). Ayrıca KULLANICI İÇGÖRÜSÜ doğru çıktı: vida eğik/kafası havada duruyordu ama detector z=1-3mm (platform!) okudu – ****_depth_m` 11x11 pencere** medyanı vidadan GENİŞ** (vida bu mesafede ~6px) → medyan = platform derinliği, vida pikselleri aykırı diye eleniyor. Yatan vidada maskeleniyordu (oturum 26 bu yüzden çalıştı), dik/havada vidada patlar. ****TODO**: hedef konumu maske 3B bulut medyanından al** (pts zaten PCA için hesaplanıyor) – ama önce offset tek değişkenle doğrulansın.
- ****Tur 3.3 (`GRASP\_Z\_OFFSET=-0.015`)**** grasp z=-0.009 komutlandı. İlk iniş cartesian fraction 0.70 → ****yeni retry** sayacı tasarlandığı gibi çalıştı** (1/3 → home → yeni hedefle yeniden hover) → ikinci tur: descend 1.000 → kavra → lift 1.000 → " TAMAM" (log). Kullanıcının fiziksel kavrama onayı bu yazım anında bekleniyor.
- ****YENİ GÖZLEM (kullanıcı)**** iniş sırasında eklemeler titreyerek/sarsılarak gidiyor ("komik"). Muhtemel sebep: cartesian 10 waypoint × 5mm adım (EEF_STEP) + %20 hız + seri-bus servolara akan küçük pozisyon adımları → kesik kesik ilerleme. İyileştirme adayları: EEF_STEP 5→10mm, descend için VEL_SCALE 0.20→0.30, bridge speed parametresi; fonksiyonel değil kozmetik ama bilek hassasiyetini etkileyebilir.

□ ****SONRAKİ**** (1) kavrama fiziksel onayı + (varsa) vida gripper'dayken yeni tur, (2) hedef z'yi maske-bulut medyanına taşı (kalıcı fix; offsetle birlikte yeniden kalibre et), (3) iniş titremesi için EEF_STEP/hız denemesi, (4) model-gerçek ~1.5cm zincir hatasının kökü (TCP/URDF) – ChArUco gripper board ile yeniden kalibrasyon hâlâ

masada. • önceki oturum:

****Önceki (oturum 28):**** ****TILT-PICK İLK DONANIM DENEMELERİ + İKİ BÜYÜK KÖK NEDEN BULUNDU+DÜZELTİLDİ:** (1) gripper collision mesh'i mm-birim DAE → MoveIt'te 58-METRELİK blob (çarpmışma kontrolü fiilen yoktu), (2) detector ilk-örnek latch tuzağı. Sim doğrulaması o oturumda KOŞUYOR sanılıyordu – oturum 29 bulgusu: o koşular gerçek robottu ve KÖK NEDEN 3 (parmak-içi SRDF çifti) yüzünden zaten geçemezdi.**

□ ****Kumpas doğrulaması TAMAMLANDI (oturum 27'nin yarım işi):**** taze vision-only stack'le (yeni `vo\_rsp.launch.py` + kamera + detector) uzunluk 45-49mm ↔ kumpas 45mm ; kalınlık percentile fix sonrası 15.3→**12.6-13.4mm** (kumpas 10mm; kalan +3mm = YOLO maske kenarı ~0.7px/kenar, gripper için güvenli yönde – KABUL). Pozisyon kaya gibi stabil (oturum 26 z-zıplaması taze stack'te YOK).

吝 ****TILT ALGISI CANLI DOĞRULANDI (robotsuz):**** düz vida tilt=3-6° `top-down`; kullanıcı eğik koyunca ****tilt=28° → TILT-HIZALI****; elle karıştırırken lin=0.41 → hedef yayınlanmadı ("vidayı karıştır") . Dik koyunca YOLO göremiyor ("vida yok" – sadece kafa görünür, foreshortening); ****~20-30° + gövde kameraya görünür**** ideal. ~10°'lik eğim eşiğin (15°) altında top-down sayılır.

烙 ****DONANIM DENEME 1 (`run\_pick\_tilt.sh`):**** home gripper hover plan+execute AMA (a) latch'lenen hedef İLK güvenilir örnekti = düz-cluster karesi (z=-2mm tilt=0) → hover top-down geldi (kod: "Hedef hover anında DONDURULUR" `pick\_tilt.cpp:10`); (b) ****kullanıcı kolun katlanırken KENDİNE ÇARPTIĞINI gördü**** → Ctrl+C. Ayrıca hover'dayken gripper kamera-vida arasına girince detector çöp okudu (z=256mm).

□ ****KÖK NEDEN 1 – SRDF:**** 28 çift `disable\_collisions reason="Never"` (7 gripper linki × joint2/3/4 + column) → MoveIt gripper↔ön-kol çarpmasını HİÇ kontrol etmiyordu → katlanan IK çözümü "geçerli" sayıldı. ****DÜZELTME:**** 28 satır silindi (158-130; yedek `config/mycobot.srdf.bak\_20260611`).

□ ****KÖK NEDEN 2 (daha derin) – DAE BİRİM TUZAĞI:**** Deneme 2'de home bile planlanamadı ("invalid start state", temas: gripper_base×column). Sebep: gripper collision'ları görsel `.dae`'leri kullanıyordu; dae'ler `\*\*MoveIt/assimp collision yükleyici birimi YOK SAYAR\*\* → gripper\_base 58×72×54\*\*metre\*\* blob → her şeye "değişiyor". 0 28 `Never` satırı da vaktiyle bu yüzden eklenmiş (sorunu gizlemiş). **\*\*DÜZELTME:\*\*** 7 gripper linkinin collision'ı mesh→\*\*gerçek ölçülü box\*\* (dae vertex'lerinden ölçüldü; ör. base 58×71.5×53.7mm, merkez ofsetli; yedek `mycobot_280_jn_adaptive_gripper.urdf.bak_20260611`; görsel mesh DEĞİŞMEDİ). FK+AABB doğrulaması: sıfır pozda açık çiftlerin hiçbirisi çakışmıyor .

****DETECTOR YENİ KAPILAR (kodlandı+derlendi+install doğrulandı):**** (1) `mode\_consensus=3` – son 3 cycle aynı modda hemfikir olmadan `/vida/target` YAYINLANMAZ (logda "konsensus bekleniyor (TILT-HIZALI, 2/3)") → tek-karelik yanlış top-down latch'i biter; (2) `target\_z\_min/max=[-0.02,0.12]m` – BASE z sınırı dışıysa "depth cop" diye red (oturum 26'nın z-sanity maddesi). Yeni paramlar: `mode\_consensus`, `target\_z\_min`, `target\_z\_max`.

****ŞU AN:**** fake-hardware sim smoke test koşuyor (home planı yeni SRDF+box'larla geçiyor mu). ****SONRAKİ:**** sim geçerse donanım deneme 3 – vida r=150-190mm (129mm çok yakındı, IK'yı katlanmaya zorladı), eğim 20-30°, gövde kameraya görünür; Claude logu izler, 'g' işaretini verir.

□ ****OTURUM 28 DERSLERİ:**** (1) `pkill -f vida\_detector` kendi komut satırını vurur → `pkill -f "[v]ida\_detector"` ya da PID. (2) Detector log ~2dk buffer'lı → `PYTHONUNBUFFERED=1`. (3) Robotun yakın bölgesi (r≤130mm) dik/tilt grasp için IK'yı aşırı katlanmaya zorlar – vida 150-190mm bandında olsun. (4) Collision mesh olarak mm-birimli dae KULLANMA – primitive box kullan (assimp birim bug'ı). • önceki oturum:

****Önceki (oturum 27):**** ****VİZYON ÖLÇÜM ↔ KUMPAS KARŞILAŞTIRMASI: UZUNLUK/ÖLÇEK DOĞRULANDI,** kalınlık şişkin çıktı → percentile fix kodlandı ama YENİDEN-ÖLÇÜM YARIM KALDI (oturum DEVLOG'a yazılmadan kapandı; bu kayıt sonradan

transkriptten çıkarıldı).**

****YENİ ARAÇLAR (oturum 27):**** (1) `~/ros2\_ws/vo\_rsp.launch.py` – ****vision-only TF launch****: SADECE robot_state_publisher (kamera→robot_base statik TF zinciri), controller/RViz/robot YOK – detector'ı base-frame'de robota dokunmadan koşturmak için. (2) `vida\_detector\_node.py`'ye ****_mask_dims_mm****: vida uzunluk+kalınlık MUTLAK mm (ölçek doğrudan depth'ten `z/fx`, 44mm varsayımı YOK); overlay/log'a `uz=.. kal=..mm` eklendi. Derlendi.

□ ****KUMPAS KARŞILAŞTIRMASI (kullanıcı ölçtü):**** Kumpas: toplam uzunluk ****45mm****, kafa çapı 7mm, shaft çapı 5mm, ****en geniş (orta çıkıntı) çapı 10mm**** (geniş gövde uzunluğu 11mm). Vision: uzunluk ****46-48mm**** (****ölçek/kalibrasyon DOĞRULANDI****, ~+2mm maske payı) – kalınlık ****15.3mm**** (~%50 fazla; sebep: kısa-eksen max-min, maske kenarı aykırı pikselleri şişiriyor + ölçülen şey başlık/en-geniş silüet). ****FIX KODLANDI+DERLENDİ****: genişlik artık `np.percentile(minor, 97)-np.percentile(minor, 3)` (robust). ****AMA fix'ten SONRA yeniden ölçüm ALINAMADI**** – log buffer + pkill self-match (exit 144) kaosu içinde oturum kapandı. ****SONRAKİ: vision-only'yi taze aç (`rs\_launch.py` 424x240 align_depth + `vo\_rsp.launch.py` + detector), `kal=10mm` çıkıyor mu doğrula.**** Not: `pkill -f vida\_detector` KENDİ komut satırıyla eşleşip shell'i öldürüyor – `pkill -f "[v]ida\_detector"` ya da PID ile öldür. Detector log'u ağır buffer'lı (~2dk flush) – `PYTHONUNBUFFERED=1` ile başlat. Süreçler temiz kapandı (oturum 28 başında doğrulandı, hayalet yok). † önceki oturum:

****Önceki (oturum 26, 2026-06-09):**** △ ****Bu oturum YANLIŞ (eski) yolu test etti (Claude hatası) + erişim ÇÖZÜLDÜ + asıl bloker DEPTH ÇÖP/HEDEF TİTREŞİMİ + overlay'e 2B eksen çizimi eklendi + robot 0'a park.****

****CLAUDE HATASI**** DEVLOG dibindeki oturum 19'dan devam etti, en üstteki oturum 25'i ATLADI → kullanıcıyı ESKİ `run\_pick\_test.sh` (`pick\_place\_cartesian`, oryantasyonu ****YOK SAYAR****: tilt=0 sabit + yaw=atan2(y,x) radyal) koşturdu. ****DOĞRU GÜNCEL YOL = oturum 25'in `run\_pick\_tilt.sh` / `pick\_tilt.cpp`'si**** (detector'ın tam grasp quaternion'ını KULLANIR). ****YARIN: `./run\_pick\_tilt.sh` koş, `run\_pick\_test.sh` DEĞİL.**** (Kullanıcının "pick node çöpe atmasını" isteği = pick_tilt zaten yapıyor.)

****ERİŞİM DARBOĞAZI ÇÖZÜLDÜ (oturum 19 açık maddesi):**** Vidayı robota ~15-20cm (r≈195mm) yakına koyunca pick_place_cartesian'da pick_pre çözüldü (yaw=-20° tilt=0°), ****[pick_descend] cartesian fraction=1.000**** → robot vidaya İNDİ, gripper kapandı (-0.74), `[pick\_lift]=1.000` → kavradı+kaldırdı. Aynı kol/IK olduğundan ****pick_tilt için de geçerli: hedefi yeşil çember (r<200mm) içinde tut.**** (Oturum 19'un "Unable to sample valid goal" hatası = erişim limiti, çözüldü.)

****PLACE erişilemez**** (sadece pick_place_cartesian): place=(0.15,0.15,0.05) ****+Y**** tarafı, 7 yaw'ın hepsi abort → "hedef erişilemez". Pick ****+Y****'de çalıştı; kanıtlanmış erişilebilir taraf -Y. (pick_tilt'te place YOK – tut+kaldır+bekle.)

□ ****ASIL BLOKER (PAYLAŞILAN detector – HER İKİ yolu da etkiler): DEPTH ÇÖP + HEDEF TİTREŞİMİ.**** Taze detector logu: yeşil vida conf 0.76-0.78, lin 0.85, tilt 4-5° (yatıyor – doğru) AMA `/vida/target` BASE konumu 2 değer arası ZIPLIYOR: `(311,261,-606)mm` ve `(215,-78,+304)mm` – z fiziksel İMKANSIZ (gerçek ~5mm). 21:17 pick testinde z=4mm DOĞRUYDU → ****depth kalitesi DEĞİŞKEN/kararsız.**** 2 alternatif konum = muhtemelen 2 yeşil tespit (gripper'da kalan vida? yansıma) + conf jitter → her cycle farklı aday latch'leniyor. ****DÜZELT:**** conf 0.50→0.75; ****base-z sanity check (z∉[-20,+120]mm reddet)****; tek vida ile test ya da deterministik "en-yakın/merkez" seçim; gripper'daki vidayı çıkar.

****YENİ KOD (`vida\_detector\_node.py`, derlendi):**** `\_draw`'a ****2B overlay EKSEN çizimi**** eklendi – MAGENTA çizgi (`p\_base ± axis\_base·4cm` projeksiyon) = ölçülen vida eksen (vida gövdesi boyunca uzanmalı); SARI daire (`p\_base` geri-projekte) = deprojection/TF tutarlılık kontrolü (vidanın üstüne düşmüyorsa kalibrasyon/depth sorunu). `axis3d=(p\_base,axis\_base)` reliable dalında geçildi. (oturum 25 sadece 3B `/vida/axis\_marker` çiziyordu; bu RViz'siz 2B doğrulama için.) `colcon build vida\_vision` TEMİZ.

△ ****Overlay TESLİM sorunu:**** 40+ dk ayakta kalan hayalet stack'te DDS yorgun – `/vida/overlay` hiçbir YENİ subscriber'a teslim olmadı (2 grabber + view_overlay boş), `ros2 topic hz/echo/info` tamamen takıldı (daemon

wedged). Doğrulama için TAZE restart şart.

粒 **TEMİZLİK + ROBOT 0'DA:** Hayalet stack `run\_pick\_test.sh` cleanup'ından SAĞ KALMIŞTI (PGID 13324: demo.launch+move_group+camera+robot_state_pub+ros2_control+bridge – oturum 18 iç-içe-stack tuzağı TEKRAR). Hepsi + detector + viewer + ros2 daemon öldürüldü; `/dev/ttyTHS1` boş, socket silindi. Stack öncüne robot vıdayı tutar pozda kaldı → kullanıcı isteğiyle **doğrudan** pymycobot `send\_radians([0]\*6, 25)` ile 0'a park** (stack kapalı, goto_zero action'ı yoktu). ONCE `[-4.7,-88.9,-24.3,-57.2,9.5,-51.3]°` → SONRA `~0°` ($\pm 1.2^\circ$ = bilinen enkoder ofseti). Robot ŞU AN 0'da, servo torklu (tutuyor); gripper hâlâ vıdayı tutuyor olabilir (yarın çıkar).

□ **DERSLER:** (1) symlink-install binary'de stale kontrolü `stat -c %Y` symlink'in KENDİ tarihini verir (YANILTIR) → bu oturumda yanlış "stale" alarmı + gereksiz (zararsız) rebuild. `[ src -nt bin ]` ya da `readlink -f`/`stat -L` ile HEDEFİ stat et. (2) `run\_pick\_test.sh` cleanup ana stack'i ÖLDÜRMÜYOR (script bug) – test sonu `pgrep`/`ps` ile doğrula. (3) **Her oturum başında DEVLOG'un EN ÜSTÜNDEKİ ANLIK DURUM'u oku, dibini değil.**

□ **SONRAKİ (yarın):** (1) **./run_pick_tilt.sh** (DOĞRU tilt-farkında yol; 5cm hover için `APPROACH\_HEIGHT=0.05`) – `run\_pick\_test.sh` DEĞİL. (2) **Önce overlay'i taze vision-only ile aç** (kamera `rs\_launch.py` 424x240x15 align_depth + detector + `view\_overlay.py`) → MAGENTA eksen vida boyunca mı, SARI daire vida üstüne mi düşüyor (kalibrasyon/depth). (3) DEPTH+TİTREŞİM düzelt (conf 0.75 + base-z sanity + deterministik seçim + gripper'daki vıdayı çıkar). (4) 1-2cm konum: hand-eye doğrula / XY ofset / kavrama noktası silüet-merkez yerine gövde. † önceki oturum:

Önceki (oturum 25): **TILT-FARKINDA PICK** planlandı + Part A (detector 3B tilt) KODLANDI + yeni ChArUco board üretildi. Kullanıcı isteği: gripper vidanın 5-10cm üstüne dik gelsin → **hover'da onay bekle** → vidanın gerçek 3B **tilt'**ine göre **paralel** inip kavra → kaldır → tut (place YOK). Plan onaylandı (`~/claudes/plans/shimmering-wondering-bachman.md`). **Kararlar:** (1) Tilt güven-kapısı `linearity≥0.6 && N≥25`; GEÇERSE ölçülen tilt'le kavra (tilt<15°=doğal top-down, ≥15°=tilt-hizalı), GEÇMEZSE `/vida/target` YAYINLANMAZ → robot hover'da bekler, **kullanıcı vidaları elle karıştırır** (kör top-down fallback YOK). (2) Grasp sonrası tut+kaldır+bekle. (3) Hover-onay (`/pick/confirm` std_msgs/Empty, cam_viewer'da 'g' tuşu). **TEKNİK BULGU:** mevcut `gripperOrientation(yaw,tilt)` 2-DOF parametrizasyonu gerçek tilt-grasp'i İFADE EDEMEZ (tilt'i yaw yönünde uygular; "eksen boyunca eğim + eksene dik parmak" aynı anda kurulamaz) → detector grasp yönelimini vida 3B ekseninden **grasp-frame matrisi** ile kurar (tcp+Y=yaklaşmaeksen, tcp+Z=eksen boyunca, tcp+X=parmak1eksen); düz vida → doğal top-down. **YAPILDI** (Part A, `vida\_detector\_node.py`): **mat_to_quat`+`grasp_quat_from_axis`** (modül), **\_mask\_points\_3d`** (mask piksel-ham depth→optik 3B, MAD eleme), **_axis_3d`** (PCA→eksen+linearity), **\_on\_timer`** güven-kapısıyla tam-quaternion `/vida/target` (geçmezse yayınlamaz+overlay "vidayı karıştır"), `/vida/axis\_marker` (magenta eksen oku), **_publish_marker`** artık yaklaşma okunu çizer. Yeni paramlar: **\_axis\_subsample=3**, **\_axis\_min\_points=25**, **\_axis\_min\_linearity=0.6**, **\_grasp\_tilt\_min\_deg=15**. **py\_compile temiz.** **PART B+C de TAMAM** (aynı oturum): **pick\_tilt.cpp`** (home→hedef SÜRESİZ bekle→hover[grasp q + 180°-flip IK fallback, yaklaşma eksenı boyunca geri]→`/pick/confirm` bekle→cartesian paralel iniş→close→lift→tut) + **pick_tilt.launch.py`** (position\_only\_ik:False) + CMake/package.xml(std\_msgs) + **cam\_viewer.py`** 'g' tuşu→`/pick/confirm` + **run_pick_tilt.sh`**. **colcon build TEMİZ**; binary tazeliği `strings`'le DOĞRULANDI\*\* (yeni "target yok" VAR, eski "gelmedi" YOK – mtime/symlink/UTF-8 yanıltıcıydı). **Sim/donanım testi BEKLİYOR** (gerçek kolu hareket ettirir, otonom koşturulmadı). **YENİ BOARD:** **generate\_charuco\_gripper.py`** → **boards/charuco_6x3_gripper.{png,pdf}`** (96x48mm, 16mm kare, 12mm marker, DICT\_4X4\_50) + **config/charuco\_params\_gripper.yaml`** (aktif 5x5 EZİLMEDİ). Gripper'a yapıştırılıp yeniden kalibrasyon için; kalibrasyonu YÜKSEK çözünürlükte (≥848x480) çalıştır (424x240'ta marker decode olmaz). **SONRAKİ (DONANIM testi – kullanıcı koşar):** **cd ~/ros2_ws && ./run_pick_tilt.sh`** (5cm için **APPROACH_HEIGHT=0.05**). (1) Algı: belirgin eğik (~30°) vida → overlay'de tilt°/lin makul mu, "[TILT-HIZALI]" mı; düz vida → tilt≈0 "[top-down]"; gürültülüde "vidayı karıştır" + robot bekliyor mu. (2) Hover'da gripper vida eksenine PARALEL mi (sim'de görsel doğrula – grasp-frame konvansiyonu). (3) 'g' onayı → paralel iniş + kavrama. Eğik IK çok zorsa: yeşil çembere yaklaştı / **_axis_min_linearity`** düşür / **\_grasp\_tilt\_min\_deg`** ayarla. Kalibrasyon (Part C) doğru kavrama için ÖN KOŞUL. † önceki oturum:

Önceki (oturum 24): **position_only_ik`** BULGUSU + kalibrasyon DOĞRULANDI. **approach\_target.launch.py`** kinematiği **position_only_ik:True`** idi → TRAC-İK sadece TCP **konumunu** çözüp **oryantasyonu** (90° dik) HİÇ zorlamıyordu → gripper konuma gidiyor ama EĞİK kalıyordu. **Bu, oturum 23'ün "gripper yana-yatık" gözleminin muhtemel BİRİNCİL sebebi** (model→gerçek açığını aramadan önce bunu ele al). **position\_only\_ik:False`** yapıldı (launch param, rebuild gerekmez). Donanım testi: False ile planlama zorlaştı (çoğu yaw IK-fail, OMPL "Unable to sample valid goal", ~6s/yaw) ama sonunda **yaw=+171° tilt=0°** çözüldü → Execute success → gripper hedefin 10cm üstüne DİK gitti, gripper açıldı. **KALİBRASYON DOĞRULANDI:** gripper vidanın TAM ÜSTÜNDE (kullanıcı 2x onayladı "tam üzerinde") → eye-to-hand XY hâlâ geçerli. **Servo zero-offset:** arm 0'a gidince **get_angles`** sabit $\sim \pm 1^\circ$ ofset **[0.61,-0.52,-0.52,-1.05,-0.7,-0.61]** (3 denemede AYNI = enkoder ofseti, rastgele/sarkma değil); tam 0.0

fiziksel imkansız + servo-zero recalib hand-eye'ı bozar → ****DOKUNMA.**** Yan işler: `camera\_center.py` (kamera orta piksel 212,120@424x240, pyrealsense2 standalone), cam_viewer raw-mode (`CAM1\_TOPIC=/camera/color/image\_raw` → tespitsiz canlı), `run\_approach\_test.sh` (tek-kamera/view_overlay 5cm wrapper – goto_zero-on-exit YOK, Ctrl+C'de robot SARKTI; doğru script `run\_approach.sh`). Kapatma: SIGINT trap YİNE çalışmadı (oturum 23 notu doğrulandı) → elle sıralı park→nodes→bridge→socket→port; robot 0'da, port boşta, temiz. ****SONRAKİ:** position_only_ik:False ile gripper'in GERÇEKTEN dik gelip gelmediğini net doğrula – hâlâ yana-yatıkrsa oturum 23'ün URDF telafi roll/pitch yolu; düzeldiyse yana-yatıklık ÇÖZÜLDÜ.****** † önceki oturum:

****Önceki (oturum 23):**** Yaklaşma modu YENİ gripper modeliyle donanımda tekrar koştı + gripper YANA-YATIKLIK teşhisi. `run\_approach.sh` çalıştı: controller 2s, YOLO 25s, hedef bulundu (beyaz vida conf=0.91, BASE x=148 y=-123 z=222mm, r≈192mm yeşil kenar), fallback yaw=-135° tilt=0° çözüldü → ****Execute success** → robot hedefin 10cm üstüne dik gitti → gripper açıldı (oturum 21 davranışı yeni modelle TEKRARLANDI).****** † ****AMA** kullanıcı gerçek gripper'in DİK DURMADIĞINI, YANA YATIK olduğunu gördü.****** Teşhis: bu ****yaw** işareti GELEMEZ****** – approach_target.cpp `setRPY(-90°,0,yaw)` ile temiz dikey komut ediyor, +0.8406 yaw bilek eksen (flange Z=J6) etrafında SAF DÖNÜŞ (parmakları çevirir, asla yana yatıramaz). Yana yatıklık = ****ayrı model+gerçek kalibrasyon açığı**** (olası: robot eklem sıfır-offset J5/J6, gripper braket roll uyumsuzluğu, ya da 110mm gripper ağırlığıyla bilek yerçekimi sarkması). ****SONRAKİ:** yatıklığın YÖNÜ+DERECESİ ölçülecek; sabit bilek/mount yatıklığıysa URDF'e telafi roll/pitch ekle (gripper_base_to_tcp veya mount), model gerçek yatıklığı bilince "dik komut" gerçek bileği dikleştirir.****** Yaw işareti (+0.8406) bu belirtiden bağımsız; oturum 22 görsel onayı geçerli. Robot 0'da park, tüm node'lar temiz kapatıldı. † önceki oturum:

****Önceki (oturum 22):**** Stand/gripper GÖRSEL KALİBRASYON. camera_arm boşluğu kapatıldı (çubuk 12.5cm + column/clamp -5cm), gripper RViz'de sıfırda gerçek gibi eğik gösteriliyor (mount yaw +0.84rad=48.16°), gripper TCP 110mm. Robot 0'da park.

****Önceki (oturum 21):**** ****BASİT YAKLAŞMA MODU DONANIMDA ÇALIŞTI.**** Robot 0'dan başladı → siyah vida r=156mm (yeşil bölge) → yaw=-122° tilt=0° dümdüz dik çözüldü → hedefin 10cm üstüne gitti → gripper açtı → orada bekledi. ****TAM ÜSTÜNE GİTTİ**** (kullanıcı onayladı). Yol: 1)RViz segfault(Tegra Image display)→2)RAM thrash(RViz dondu)→****RViz TAMAMEN ÇIKARILDI****, sadece 2-kameralı cv2 penceresi(sol=/vida/overlay erişim çemberleri+seçili nokta)+terminal→3)erişim retry-loop+kalıcı yeşil(200)/amber(260) çember→4)tail yarışı fix(`> log` + `tail -F`). Robot: 0'dan→10cm üst 90° dik→gripper aç→bekle (0'a DÖNMEZ). KALICI dosyalar aşağıda. SONRAKİ: madde 17 interaktif pick (iniş+kavrama+place+onay) ya da TCP kalibrasyonu.

****KALICI ARTEFAKTLAR (oturum 21 – çalışan yaklaşma modu):****

- `~/ros2\_ws/run\_approach.sh` – tüm akış (stack use_rviz:=false + record_cam + cam_viewer + detector + approach_target + tam temizlik)

- `~/ros2\_ws/cam\_viewer.py` – 2 kamera tek cv2 penceresi (sol=/vida/overlay, sağ=/record_cam)

- `~/ros2\_ws/src/mycobot\_demo/src/approach\_target.cpp` – 0→10cm üst 90° dik→gripper aç→bekle, erişene kadar retry-loop, en güncel hedef

- `~/ros2\_ws/src/mycobot\_demo/launch/approach\_target.launch.py`

- `~/ros2\_ws/src/vida\_vision/.../vida\_detector\_node.py` – erişim çemberi KALICI varsayılan reach_radius=0.20 / reach_radius_max=0.26

- `~/Desktop/Vida\_Yaklasma.desktop` – masaüstü kısayolu

- `demo.launch.py` – `rviz\_config` arg (artık kullanılmıyor ama duruyor)

- `approach.rviz` (kamarasız) – yaklaşma modunda KULLANILMIYOR (RViz çıkarıldı), ileride lazım olursa duruyor
Kullanıcı interaktif pick planını şimdilik bıraktı; istedi: masaüstü kısayolu → RViz(gerçek robot+sim) + YOLO hedefi marker + ADIM1 robot 0'dan başlar + ADIM2 hedefin 10cm üstü 90° dik gripper açık + RViz'de 2 kamera. Yeni: `approach\_target.cpp/.launch.py`, `approach.rviz`, `run\_approach.sh`, `~/Desktop/Vida\_Yaklasma.desktop`, demo.launch.py'ye `rviz\_config` arg. Derlendi (16:21). ****SONRAKİ:** `./run\_approach.sh` ile donanım testi.****** † önceki oturum:

****Önceki (oturum 20):**** ****ERİŞİM/İK DARBOĞAZI AŞILDI.**** (a) Erişim çemberi kamera overlay'ine + click_to_go'ya eklendi (yeşil<220mm güvenli, amber<300mm limit). (b) Oturum 19 fail'inin ASIL sebebi: yaw+tilt fallback kodu

derlenmemiřti (stale binary, 4. kez) – derlendi. (c) gripper close fix uygulandı (doğrudan controller). (d) **21:11 testi: PICK UÇTAN UCA ÇALIŞTI** – open→pick_pre(yaw=-13°)→descend 1.0→`gripper closed OK` (abort yok)→lift 1.0 → **robot vidayı tutup kaldırdı.** (e) jump_threshold 0→5 fix (descend artık temiz). (f) **TAM pick→place→bırak zinciri ÇALIŞTI** (place -y'de). (g) tilt KALDIRILDI (sadece dümdüz-ařağı) → temiz tilt=0 full döngü çalıştı. (h) Kalan precision: TCP kalibrasyonu (~1cm offset, yöntem madde 15, ÖLÇÜM bekliyor). (i) **2. kamera /dev/video3 (video kayıt) bulundu+çalışıyor**, record_cam_publisher.py yazıldı. (j) **İNŞAATTA: interaktif pick** (RViz preview+onay+dar-taraftan tutuş+kayıt kamera) – kararlar+plan madde 17, SONRAKİ: pick_interactive.cpp. Hareket/eriřim/IK/place/gripper HEPSİ SAĞLAM.

Önceki (oturum 19): (a) Stale binary çözüldü (constraint'li 18:22 binary → constraint'siz 20:31 derlendi). (b) Yeni test (20:52) constraint'siz kořtu, plan artık 25s→6s; AMA pick_pre HÂLÂ başarısız. ASIL KÖK NEDEN ortaya çıktı: OMPL `Unable to sample any valid states for goal tree` → pre-grasp pozunun GEÇERLİ IK ÇÖZÜMÜ YOK. Vida r≈254mm'de (myCobot 280 ~280mm limitine çok yakın), dik (top-down) tutuşla ulaşlamıyor VEYA hedef IK stand'a (column/platform_top/tutucu_kelepce) çarpıyor. Robot bu yüzden 'ready' pozunda (yukarıda, vidadan uzak) takılıp çıkıyor.**

Faz durumu: FAZ 1-6 , FAZ 5 kalibrasyon , FAZ 9 (TRAC-İK) . **Vida pick-and-place – Faz 2/4 ; Faz 5 grasp: constraint çözüldü, řimdi ERİřİM/ULAřILABİLİRLİK darboğazı (vida çok uzakta).**

3) Test sonucu (20:52 run, constraint'siz binary): Akıř artık doğru ilerliyor – Ready , Gripper open , sonra `>> [PICK] joint plan → pick\_pre` (constraint satırı YOK, rebuild doğrulandı). Plan ~6s sürdü (eski 25s değıl). Hedef: x=0.226 y=-0.117 z=0.008 (stabil, conf 0.89-0.90, kırmızı vida). pick_pre = z+0.10 = **0.108**. **stack.log kesin hata:** `[ompl] RRTConnect.cpp:253 - arm/arm: Unable to sample any valid states for goal tree` x4 → `No motion plan found`. Bu = hedef pozun valid IK'sı YOK (yol değıl, GOAL sorunu).

- **Geometri:** yatay yarıçap $r = \sqrt{(0.226^2 + 0.117^2)} = **254.5 \text{ mm}**$. myCobot 280 max eriřim ~280mm. Dik gripper (gripperDown, +Y→-Z) ile bu yarıçapta TCP'yi tam ařağı çevirmek bilek katlanmasını gerektirir; r≈254mm'de kola neredeyse hiç pay kalmaz → IK infeasible. pre-grasp z=0.108 en kötü nokta (3B mesafe $\sqrt{(0.254^2 + 0.108^2)}=0.276\text{m} \approx$ tam limit).

- **Alternatif hipotez:** hedef IK çözümleri stand fixture'larına (column / platform_top / tutucu_kelepce / baseboard – hepsi collision scene'de) çarpıyor olabilir. "Unable to sample valid goal" ikisini de üretir.

- **AYIRT ETME TESTİ (sıradaki):** vidayı robota 5-8cm DAHA YAKIN koy, tekrar çalıştır. Plan tutarsa → eriřim limiti idi. Hâlâ patlarsa → collision; o zaman pick pozunu RViz/PlanningScene'de validity kontrol et. Ayrıca `APPROACH\_HEIGHT` 0.10→0.05 denenebilir (pre-grasp'i alçaltır ama r aynı kalır, kısmi fayda).

- NOT: Detector + kalibrasyon SAĞLAM – hedef (x=0.226 y=-0.117 z=0.008) run'lar arası tutarlı tekrarlandı, titreřim yok. Sorun tespit/kalibrasyon DEĞİL, vida konumu robotun dik-tutuş zarfının dıřında.

2026-06-07 oturum 21 – BASİT YAKLAřMA modu (RViz + 2 kamera + masaüstü kısayolu)

Kullanıcı interaktif pick planını (madde 17, onay butonu + dar-taraf kavrama + place) **řimdilik bıraktı**; yerine daha basit, net bir hedef istedi: **sadece pre-grasp + RViz + 2 kamera izleme + masaüstü kısayolu.**

İstenen davranıř: Masaüstü kısayolu → RViz açılır (gerçek robot bağılı, simülasyon görünür) → YOLO'nun seçtiğı hedef RViz'de marker olarak gösterilir → **ADIM 1:** robot 0 noktasından (SRDF home, tüm eklemler 0) başlar → **ADIM 2:** gripper'ı hedefin tam üstüne, 90° dik (top-down), 10cm yukarıda götürür, gripper'ı açar, orada durur. İniř/kavrama/place YOK. RViz'de hem 1. kamera (D435i) hem 2. kamera (kayıt /dev/video3) izlenir.

Yazılan/değıřen dosyalar:

- `src/mycobot\_demo/src/approach\_target.cpp` (YENİ) – pick_place_cartesian'ın sade hali. home→hedef bekle→hedefin 10cm üstü (yaw ızgarası fallback, tilt=0 dik)→gripper aç (doğrudan controller)→marker yayınla (transient_local küre /approach/target_marker)→Ctrl+C'ye kadar ayakta kal (RViz açık kalsın).

- `src/mycobot\_demo/launch/approach\_target.launch.py` (YENİ) – pick_place.launch.py kopyası, executable=approach_target, param approach_height (env APPROACH_HEIGHT).

```
- `src/mycobot_demo/rviz/approach.rviz` (YENİ) – moveit.rviz + Marker(/approach/target_marker transient_local) + Marker(/vida/target_marker) + Image(/camera/color/image_raw best_effort) + Image(/record_cam/image_raw reliable).

- `src/mycobot_moveit_config/launch/demo.launch.py` – **yeni `rviz_config` launch arg** (default moveit.rviz). Özel RViz config geçilebilirsin diye (parse testi OK, 14 entity).

- `src/mycobot_demo/CMakeLists.txt` – approach_target executable + deps (visualization_msgs dahil), `install(DIRECTORY ... rviz)`.
```

- `run\_approach.sh` (YENİ, çalıştırılabilir) – sistem hazırlık→stack(use_rviz:=true rviz_config:=approach.rviz use_camera:=true)→record_cam_publisher.py→vida_detector→approach_target.launch.py; Ctrl+C'de tam temizlik (bridge+socket+rviz dahil).

- `~/Desktop/Vida\_Yaklasma.desktop` (YENİ, trusted) – gnome-terminal'de run_approach.sh çalıştırır.

****Derleme:**** `colcon build --packages-select mycobot\_demo --symlink-install` (binary 16:21 taze). bash -n run_approach.sh . demo.launch.py parse . ****DONANIM TESTİ BEKLİYOR**** (henüz çalıştırılmadı).

****RAM:**** RViz + YOLO + kamera aynı anda Nano'da swap zorlar ([[feedback_jetson_nano_ram_pressure]]). Kullanıcı RViz'i açıkça istedi → script çalıştırıyor ama uyarı basıyor.

****Kullanıcı netleştirmesi (aynı oturum, son akış):****

- Robot eylemi = ****SADECE yaklaşma****, 0'a geri DÖNMESİN – hedefin 10cm üstünde 90° dik gelip ****orada bekler**** (node zaten böyle: home→yaklaş→gripper aç→Ctrl+C'ye kadar bekle; node değişmedi).

- ****RViz HİÇ kapanmaz.**** Sadece ****detector****, node hedefi latch'ledikten sonra kapanır (Nano RAM). `run\_approach.sh`'e eklendi: detector tespit→node `/vida/target` latch (`Hedef: x=` log'u)→watcher detector'ı öldürür. RViz+kameralar (D435i realsense node'undan) + `/approach/target\_marker` (transient_local) açık kalır.

- Sıra netleştirdi: önce RViz (pgrep rviz2 + 2s), SONRA detector. Node arka planda, çıktı `tail -f --pid` ile canlı; Ctrl+C → tam temizlik.

**** RViz SEGFAULT (1. donanım denemesi) + DÜZELTME:**** İlk testte RViz açılıp hemen kapandı (exit -11 = SIGSEGV), kamera pencereleri "error". Kök neden = oturum 7'deki bilinen Tegra OGRE/X11 render-pencere bug'ı: ****RViz Image display'leri**** ekstra render penceresi açıp resize segfault'unu tetikliyor. Ayrıca `approach.rviz`'de Image display'leri yanlışlıkla `Panels:` bölümüne de koymuştum (geçersiz). ****ÇÖZÜM:**** (a) `approach.rviz` kamerasız, kanıtlanmış moveit.rviz layout'una indirildi (sadece robot + 2 hedef Marker; Hide Right Dock:true, ekstra panel yok). (b) Kameralar RViz DIŞINDA: yeni ****cam_viewer.py**** (cv2, 2 kamera tek pencere yan yana, best-effort D435i + reliable record_cam, cv_bridge'siz – view_overlay.py deseni, bu makinede kanıtlı). (c) `run\_approach.sh` cam_viewer'ı başlatır (DISPLAY varsa), sweep+cleanup'a eklendi. Derlendi. ****DERS:** bu Jetson'da RViz'e Image display KOYMA – kamera her zaman ayrı cv2 penceresinde.**

**** 2. deneme: RViz açıldı + kameralar açıldı AMA en sonda DONDU, model vida tespiti yapamadı.**** Kök neden = klasik Nano RAM thrash: full stack + RViz + YOLO + 2 kamera aynı anda → swap, detector inference donuyor ([[feedback_jetson_nano_ram_pressure]] + oturum 13). ****KULLANICI KARARI:** RViz'i tamamen çıkar. ****** Sadece kamera penceresi + terminal yeter; önemli olan kameradan seçilen noktayı görmek + robotun fiziksel olarak o noktanın 10cm üstünde 90° dik durması (gerçek robotta izlenir, sim gerekmez).

****RViz ÇIKARILDI (3. sürüm):****

- `run\_approach.sh`: stack artık `use\_rviz:=false`. RViz bekleme bloğu + rviz_config + RVIZ_CFG kaldırıldı. RAM rahatladı.

- `cam\_viewer.py`: sol panel artık ****/vida/overlay**** (YOLO tespiti + SEÇİLEN vida "HEDEF" işaretli) – "kameradan seçili noktayı gör" isteği. Sağ panel kayıt kamerası. Detector ölünce overlay son karede donar (seçili nokta görünür kalır). best-effort sub (reliable pub'dan da alır).

- Detector hâlâ hedef latch'lenince kapanıyor (robot hareketi + YOLO eşzamanlı RAM thrash; oturum 13'ün ardışık-yap

kuralı).

- approach.rviz/demo.launch.py rviz_config arg duruyor ama artık kullanılmıyor (zararsız).

- Desktop kısayolu güncellendi (RViz yok).

- Node DEĞİŞMEDİ (rebuild gerekmez). bash -n + py syntax ✓.

****3. deneme sonucu (kısmi başarı):**** RViz yok → ****detector DONMADAN tespit yaptı**** (kırmızı vida conf 0.94, BASE x=216 y=-116 z=12mm). RAM 3.2GB/403MB free, sorun yok. AMA (a) robot vidaya ****ERİŞEMEDİ**** – $r=\sqrt{(216^2+116^2)}=245\text{mm}$, 10cm yükseklikte (z=112) dik-limit ~265mm → sınıra 20mm → tüm yaw IK fail → node "erişilemez" deyip öldü. (b) cam1 (overlay) gelmedi çünkü node latch edince watcher detector'ı ****hemen öldürdü**** → overlay durdu.

****4. sürüm düzeltmeleri (oturum 21):****

- ****Detector ARTIK öldürülüyor**** (watcher kaldırıldı) → `/vida/overlay` canlı kalır, sol kamera panelinde seçilen nokta + erişim çemberleri sürekli görünür.

- ****Node retry-loop'a çevrildi**** (`approach\_target.cpp`): erişilemezse ölmüyor, EN GÜNCEL hedefi okuyup 5sn'de bir tekrar deniyor. Robot home'da bekler; vida yeşil bölgeye taşınınca otomatik gider. Başarınca gripper açıp orada bekler. Abone callback latch yerine "her zaman en güncel" yapıldı. Binary 17:10 taze.

- ****Erişim çemberleri gerçeğe ayarlandı + KALICI yapıldı****: `vida\_detector\_node.py` varsayılanı `reach\_radius=0.20` / `reach\_radius\_max=0.26` (eski 0.22/0.30; 0.30 kısıtsız limitti, top-down'da yanıltıcı). Artık overlay'de HER ZAMAN çizilir, script'te -p gerekmez. Yeşil=200mm güvenli (r<200 ilk yaw'da çözülür), amber=260mm dik-limit. vida_vision derlendi.

- ****4. test (kısmi başarı):**** kırmızı vida r=246mm (amber) → ilk turda 6 yaw fail (~6sn/yaw timeout), 5sn bekle, 2. turda yaw=+61° çözüldü → robot 10cm üstüne dik gitti, gripper açtı → TAMAM. retry-loop çalıştı. Ders: r=246mm erişiliyor ama YAVAŞ (~80sn deneme); r<200 (yeşil) ilk denemede gider. Kullanıcı tüm vidaları yeşil çembere taşıdı.

**** 5. test – tail YARIŞI bug'ı + fix:**** Watcher'ı kaldırınca node launch ile `tail -f \$NODE\_LOG` arasındaki gecikme kalktı → tail, approach.log oluşmadan açmaya çalıştı → "No such file" → tail çıktı → script her şeyi kapattı (node hiç çalışmadı). Detector siyah vidayı r=156mm (YEŞİL!) seçmişti, erişilebilirdi. ****FIX:**** node launch'tan önce `> \$NODE\_LOG` (dosyayı önceden oluştur) + `tail -F` (retry). bash -n OK.

**** SON TEST BAŞARILI (2026-06-07 17:36):**** tail yarışı düzelince node düzgün çalıştı. Hedef siyah vida x=0.133 y=-0.081 z=0.001 (**r=156mm, yeşil**) → pre-grasp `yaw=-122° tilt=0°` çözüldü → robot ****hedefin TAM 10cm üstüne 90° dik gitti**** → gripper açtı → bekledi. Kullanıcı "tam üstüne gitti süper" onayladı. ****YAKLAŞMA MODU TAMAM.****

****+ 0-NOKTASI PARK (kullanıcı isteği):**** Terminal kapanınca servo serbest kalıp kol DÜŞMESİN diye → `run\_approach.sh`'e `goto\_zero()` eklendi (arm_controller `/follow\_joint\_trajectory` action'ına 6 eklem=0, 5sn). ****Açılışta**** (controller'lar aktif olunca) ve ****kapanışta**** (cleanup'ta, app node'ları öldükten SONRA ama stack ölmeden ÖNCE) çağrılıyor → kol her zaman 0'a park eder, sonra stack kapanır. Eklem sırası ros2_controllers.yaml ile birebir.

**** SONRAKİ:**** İki yön: (a) madde 17 interaktif pick – bu çalışan yaklaştırmaya iniş+kavrama+place+onay ekle; (b) TCP kalibrasyonu (madde 15) – gripper ~1-2cm yan sapma.

2026-06-09 oturum 25 – Tilt-farkında pick planı + Part A (detector 3B tilt) + yeni ChArUco board

Kullanıcı tilt-farkında kavrama istedi: gripper vidanın üstüne dik gelir, ****bekler/onay****, sonra

vidanın gerçek 3B duruşuna göre ****paralel**** inip kavrar. Önce mevcut kod incelendi, plan modunda

tasarlandı (`~/claude/plans/shimmering-wondering-bachman.md`, ONAYLANDI), Part A kodlandı.

****1) Mevcut durum tespiti:****

- `vida\_detector\_node.py` sadece 2D mask açısından düzlem-içi yaw hesaplıyordu, `/vida/target`'a ****yaw-only**** quaternion basıyordu. Tilt (3B eğim) YOKtu (madde 7 yarım).
- `approach\_target.cpp` gelen orientation'ı YOK SAYIP (`screw=m->pose.position`) hep top-down iniyor.
- `pick\_place\_cartesian.cpp` tilt iskeleti VAR ama ****kapalı**** (`tilts{0.0}`, satır 130). Yeniden kullanılacak parçalar: `gripperOrientation(yaw,tilt)`, `cartesianMove` (computeCartesianPath + JUMP_THRESHOLD=5), `lift`, `setGripper` (direct gripper_controller), `approachWithFallback`.

****2) TEKNİK BULGU – 2-DOF parametrizasyon yetersiz:**** `gripperOrientation(yaw, tilt) =

setRPY(- $\pi/2$ +tilt, 0, yaw)` tilt'i ****yaw yönünde**** uygular. Bir vidada eğim ****eksen boyunca****,

parmaklar ise ****eksene dik**** olmalı (birbirine dik) → tek (yaw,tilt) çifti ikisini aynı anda

kuramaz. ÇÖZÜM: grasp yönelimini vida 3B ekseninden ****grasp-frame matrisiyle**** kur:

`eY(tcp+Y)=yaklaşma=masa-aşağıının eksene dik bileşeni`, `eZ(tcp+Z)=-eksen`, `eX(tcp+X)=eY×eZ=

parmak-kapanış (eksene dik)`. Düz yatan vida (eksen yatay) → eY=(0,0,-1) yani ****doğal top-down****.

Dik vida → yaklaşma tanımsız → None (güvenilmez). Konvansiyon top-down halinde eski

`gripperOrientation(azimut+90,0)` ile birebir örtüşüyor (matematikte doğrulandı).

****3) KARARLAR (kullanıcı seçti):****

- ****Tilt güven-kapısı:**** `linearity≥0.6 && N≥25`. GEÇERSE ölçülen tilt'le kavra (tilt<15°=top-down, ≥15°=tilt-hızlı). ****GEÇMEZSE** `/vida/target` YAYINLANMAZ → robot hover'da bekler, kullanıcı vidaları elle karıştırıp net eksen oluşturunca otomatik devam. ****Kör top-down fallback YOK.**** (Reliable-flat vida → tilt≈0 = doğal top-down ile kavranır; "bekle" yalnız ÖLÇÜM güvenilmezse.)
- Grasp sonrası: tut → kaldır → ****bekle**** (place/bırak YOK).
- Hover → ****onay bekle****: `/pick/confirm` (std_msgs/Empty); cam_viewer penceresinde ****'g'** tuşu ile publish (Jetson'da approach pipeline RViz'siz; stdin'e bağlı değil, setsid altında güvenli).

****4) YAPILDI – Part A (`src/vida\_vision/vida\_vision/vida\_detector\_node.py`):****

- Modül: `mat\_to\_quat(R)` (rotmat=quat), `grasp\_quat\_from\_axis(s\_base)` (yukarıdaki grasp-frame).
- `\_mask\_points\_3d(mask)`: mask>127 pikselleri alt-örnekle (stride=axis_subsample) → ham `self.\_depth` (mm) ile eşle, >0 filtre, ****MAD aykırı eleme**** → optik-frame Nx3 (vektörize deproject).
- `\_axis\_3d(pts)`: kovaryans → `np.linalg.eigh` → en büyük özvektör=eksen, `linearity=( $\lambda_1-\lambda_2$ )/ $\lambda_1$ `, N.
- `\_on\_timer`: ekseni base'e taşı (merkez + eksen ucu 5cm `\_to\_base` farkı) → tilt=asin|z|, güven kapısı. Reliable → `\_publish\_target(p, grasp\_q)` + `\_publish\_marker` (yaklaşma oku) + `\_publish\_axis` (magenta eksen oku `/vida/axis\_marker`) + overlay "tilt=.. lin=.. [mod]". Değilse target YOK + overlay "tilt GÜVENİLMEZ - vidayı karıştır" + warn log.
- `\_publish\_target`/`\_publish\_marker` artık quaternion alıyor (yaw_to_quat değil). Yeni paramlar:

```
`axis_subsample=3, axis_min_points=25, axis_min_linearity=0.6, grasp_tilt_min_deg=15`.

- **`python3 -m py_compile` TEMİZ.** Python symlink-install → rebuild gerekmez, node reload yeter.

**colcon build + sim/donanim testi HENÜZ YOK.**

**5) YENİ ChArUco board (yeniden kalibrasyon için):** Kullanıcı 10×5cm gripper marker istedi.
En mantıklı = ChArUco (mevcut `charuco_detector.py` hattı + alt-piksel doğruluk + kısmi-görünüm
dayanıklı + tek-marker poz-flip belirsizliği yok). `scripts/generate_charuco_gripper.py` yazıldı →
`boards/charuco_6x3_gripper.{png,pdf}` (96×48mm, 16mm kare, 12mm marker, **DICT_4X4_50**) +
`config/charuco_params_gripper.yaml`. **Aktif `charuco_params.yaml` (5×5) EZİLMEDİ** – basıp
gripper'a takınca geçilecek. Δ Kalibrasyonu **≥848×480 (tercihen 1280×720)** color ile çalıştır;
424×240'ta 12mm marker ~8px → decode olmaz (offline işlem, depth eşzamanlı gerekmez).

**6) YAPILDI – Part B+C (aynı oturum):**

- `src/mycobot_demo/src/pick_tilt.cpp` (YENİ, pick_place_cartesian temelli; place YOK, hover-onay VAR):
  `approachAxis` (q'dan tcp+Y), `flipAboutApproach` (q•Ry(π) – yaklaşma eksenini sabit, grip simetrik),
  `cartesianMove`/`lift`/`setGripper` (kopya), `moveToHover` (grasp q olmazsa flip olmazsa erişilemez).
  main: home→grripper aç→`/vida/target` SÜRESİZ bekle→reach-loop{hover→`/pick/confirm` bekle→descend→
  close→lift→tut}. approach_height param (0.10), grasp_z_offset.

- `pick_tilt.launch.py` (position_only_ik:False), `CMakeLists.txt`+`package.xml` (std_msgs eklendi,
  executable+install).

- `cam_viewer.py`: `Empty` import + `/pick/confirm` publisher + cv2 'g' tuşu (flash "ONAY GONDERİLDİ").

- `run_pick_tilt.sh` (run_approach.sh kalıbı; pick_tilt.launch.py + detector AÇIK kalır + 'g' onay notu + tam
  cleanup).

  **TUZAK + FIX (ilk koşuda):** `sweep_ros_ghosts`'taki `pkill -f "[p]ick_tilt"` deseni **wrapper'ın KENDİ
  adı `run_pick_tilt.sh` ile eşleşip script'i SIGTERM'ledi** → trap cleanup → sweep → tekrar self-kill →
  SONSUZ "TEMİZLİK" döngüsü (Ctrl+C de tetikliyor; çıkış `Ctrl+\` SIGQUIT veya başka terminalden
  `pkill -9 -f run_pick_tilt.sh`). FIX: desen → `mycobot_demo/pick_tilt` (kurulu node yolu) +
  `pick_tilt.launch.py` (ikisi de wrapper adıyla eşleşmez) + cleanup'a re-entry guard (`_CLEANED` + `trap -`).

  **DERS: wrapper adı node/launch adını içeriyorsa pkill deseni wrapper'ı vurabilir – spesifik yol/uzantı kullan.**

- **colcon build --packages-select mycobot_demo --symlink-install` TEMİZ** (1min20s). Binary tazeliği
  **`strings`'le doğrulandı** (yeni "target yok" VAR, eski "gelmedi" YOK). Δ install/ path SYMLINK →
  build/ artifact'e; mtime + UTF-8 grep yanıltıcı, ASCII fragment'le doğrula. py_compile (detector+cam_viewer)
  temiz.

** SONRAKİ (DONANIM testi – kullanıcı koşar, otonom koşturulmadı):**

- `cd ~/ros2_ws && ./run_pick_tilt.sh` (5cm: `APPROACH_HEIGHT=0.05 ./run_pick_tilt.sh`).

- Doğrula: (a) eğik vida → "[TILT-HIZALI]", düz → "[top-down]", gürültülü → "vidayı karıştır" + robot bekler;
  (b) hover'da gripper vida eksenine PARALEL mi (sim görsel – grasp-frame konvansiyon kontrolü);
```


(c) 'g' → paralel iniş + kavrama. Eğik IK zorsa axis_min_linearity/grasp_tilt_min_deg ayarla.

- Part C (kalibrasyon, ÖN KOŞUL): 6x3 board bas-gripper'a yapıştır→`charuco\_params\_gripper.yaml` aktif→
≥848x480 ile easy_handeye2/Park-URDF güncelle. Kapatma: [[feedback_background_node_cleanup]].

2026-06-08 oturum 24 – position_only_ik bulgusu, kalibrasyon doğrulama, servo zero-offset

Kullanıcı önce kamera orta noktasını görmek, sonra **kalibrasyonun hâlâ doğru olup olmadığını donanımda doğrulamak** istedi. Oturum 23'ün "gripper yana-yatık" teşhisine kritik bir alternatif kök bulundu.

1) `camera\_center.py` (YENİ, kalıcı): D435i color'ı `pyrealsense2` ile DOĞRUDAN açar (ROS/stack gerekmez, Nano yormaz), tam orta piksele kırmızı nokta + sarı artı çizer. **Orta piksel = (212, 120)** (424x240 akış). ESC/q kapat, s → `~/camera\_center.png`. Δ ROS realsense node'u ile aynı anda çalışmaz (cihaz tek process'e açılır).

2) Kalibrasyon doğrulama yöntemleri (kullanıcıya anlatıldı): eye-to-hand kalibrasyon sadece **kamera robota göre fiziksel kayarsa** bozulur (yazılım kendiliğinden değişmez). Hızlıdan kesine: (a) `tf2\_echo robot\_base camera\_color\_optical\_frame` = (0.197,-0.143,0.475) – yazılım sağlamlığı, ama fiziksel kaymayı YAKALAMAZ; (b) RViz PointCloud + robot model overlay (gripper bulutu mesh'e oturuyorsa OK) – en hızlı kayma testi; (c) dokunma testi (`/vida/target` vs `tf2\_echo robot\_base tcp`), baseline ~1.5cm (oturum kalibrasyonu 2026-06-02).

3) ASIL BULGU – `position\_only\_ik:True` gripper'ı eğik bırakıyordu: İlk donanım testinde gripper konuma gitti ama DİK değildi. Kök: `approach\_target.launch.py`'da kinematik override `position\_only\_ik:True` → TRAC-İK sadece TCP konumunu çözer, **istenen top-down oryantasyonu tamamen yok sayar** → gripper'ın açısı "ne çıkarsa o". Bu, **oturum 23'ün yana-yatıklık gözleminin muhtemel birincil sebebi** (oturum 23 model+gerçek açığı sanmıştı; ama o test de position_only_ik:True altındaydı). **DÜZELTME:** `position\_only\_ik: True → False` (`approach\_target.launch.py:41`). Launch param, **rebuild gerekmez** (symlink-install).

4) `position\_only\_ik:False` donanım testi – gripper TAM ÜSTTE: `run\_approach.sh` (masaüstü "Vida Yaklasma" kısayolu) ile koştı. Hedef beyaz vida x=0.187 y=-0.030 z=0.003 (r=189mm). False ile planlama zorlaştı – birçok yaw IK-fail (OMPL `Unable to sample any valid states for goal tree`, ~6s/yaw timeout), AMA `yaw=+171° tilt=0°` çözüldü (2.9s) → **Execute success → robot hedefin 10cm üstüne dik gitti → gripper açıldı.** **Kullanıcı gripper'ın vidanın TAM ÜSTÜNDE olduğunu 2x onayladı** → eye-to-hand XY kalibrasyonu HÂLÂ GEÇERLİ. (NOT: 10cm çünkü run_approach.sh varsayılanı APPROACH_HEIGHT=0.10; 5cm için env ver.)

5) Servo zero-offset (tam 0.0 mümkün değil): Robotu arm_controller ile 0'a gönderince `get\_angles` = `[0.61,-0.52,-0.52,-1.05,-0.7,-0.61]`° (joint sırası map'lendi). 3 kez tekrar gönderildi → HER SEFER AYNI değer. Sabit olması = **servo enkoder sıfır-ofseti** (sarkma/rastgele değil); robot fiziksel olarak 0'da, sadece okuma ±1° ofsetli. Okumayı 0.0 yapmanın tek yolu servo-zero recalib = **şu anki pozu yeni 0 tanımlar → hand-eye + URDF bozulur → YAPMA.** myCobot 280 için ±1° normal tekrarlanabilirlik; MoveIt/JTC tolere ediyor; IK ile gidildiği için TCP konumu önemli, eklem okuması değil.

6) Yan araçlar: `run\_approach\_test.sh` (YENİ – tek kamera/view_overlay, approach_target 5cm wrapper; run_pick_test.sh'in orkestrasyonunu kopyalar). Δ **Bunda goto_zero-on-exit YOK** → Ctrl+C'de robot sarktı ([6.67,-69.87,-91.31,...]). Doğru/tam script `run\_approach.sh` (çift kamera/cam_viewer + açılış/kapanış 0-parkı). `cam\_viewer.py` raw-mode: `CAM1\_TOPIC=/camera/color/image\_raw python3 cam\_viewer.py` → sol panel tespitsiz canlı D435i (detector durdurulabilir). Panel başlığı kodda sabit "tespit" yazsa da içerik ham.

Δ Kapatma (oturum 23 notu doğrulandı): `run\_approach.sh`'e SIGINT gönderildi → trap cleanup ÇALIŞMADI, setsid çocuk node'ları sağ kaldı. Elle sıralı kapatıldı: (1) arm_controller action ile 0'a park (stack ayaktayken), (2) PID + bracket-pkill ile move_group/ros2_control/bridge/realsense/cam_viewer/record_cam/robot_state_publisher öldür, (3) `/tmp/mycobot\_bridge.sock` sil, (4) `fuser /dev/ttyTHS1` boşta doğrula. Bkz [[feedback_background_node_cleanup]]. (Bracket-trick `pkill -f "[c]am_viewer.py" self-match'i önler.)

****7)** 3B tilt fit (BAŞLANDI, YARIM – oturum sonunda durduruldu):** Vidanın gerçek 3B oryantasyonunu (tilt) depth'ten çıkarma planı netleşti. ****Yöntem:**** seçilen vidanın `d["mask"]` piksellerini ham aligned-depth ile optik-frame 3B noktalara çevir (`\_mask\_points\_3d`) → 3B PCA ile uzun-eksen yönü + linearity güveni (`\_axis\_3d`) → eksenin iki ucunu `\_to\_base` ile base'e taşı → eksenin masa düzleminden (base XY) kalkıklığı = ****tilt****. ****Robot davranışını DEĞİŞTİRMEZ**** (approach_target zaten `/vida/target` orientation'ını yok sayıp top-down gidiyor) – sadece log + magenta ARROW marker `/vida/axis\_marker`; kapılar: `linearity≥0.6` (param `axis\_min\_linearity`) + `N≥25` geçerli derinlik. Δ D435i + ince parlak vida depth'i gürültülü → sadece ****belirgin eğik (≥~25-30°)**** vida güvenilir ölçülür, düz yatan vida noise'a gömülür (linearity kapısı eler). ****UYGULAMA DURUMU:**** sadece `from geometry\_msgs.msg import PoseStamped, Point` (import, `vida\_detector\_node.py:17`) eklendi; `\_mask\_points\_3d`/_`\_axis\_3d`/_`\_publish\_axis`/_`\_on\_timer` entegrasyonu + `pub\_axis` publisher + `axis\_min\_linearity` param + ****colcon build HENÜZ YOK.**** Tam kod oturum 24 konuşmasında mevcut. ****Devam:**** helper'ları ekle → build → sim akış → eğik vidayla tilt'i gerçekle karşılaştır.

**** SONRAKİ:**** position_only_ik:False ile gripper'ın gerçekten dik gelip gelmediğini net ölç. Hâlâ yana-yatıksa → oturum 23'ün yolu (URDF telafi roll/pitch veya eklem zero-offset). Düzeldiyse → yana-yatıklık ÇÖZÜLDÜ, madde 17 interaktif pick'e geç. (Takas: position_only_ik:False planlamayı yavaşlatır/zorlaştırır – vida yeşil çemberde r<200mm tutulmazsa "erişilemez".)

2026-06-08 oturum 23 – Yaklaşma yeni modelle tekrar + gripper yana-yatıklık teşhisi

`run\_approach.sh` yeni gripper modeliyle (yaw +0.8406, TCP 110mm) ilk kez donanımda koştı ve ****oturum 21 davranışını tekrarladı**** (0=hedefin 10cm üstü=dik=gripper aç=bekle). Akış: controller 2s aktif, açılış 0-parkı, YOLO modeli 25s, hedef (beyaz vida conf=0.91, BASE x=148 y=-123 z=222mm, r≈192mm), ilk yaw denemeleri (-45°, +45°) IK'sız, fallback ****yaw=-135° tilt=0° çözüldü**** → Execute success → gripper açıldı.

**** Yeni bulgu – gripper YANA YATIK (dik değil):**** Kullanıcı gerçek robotta gripper'ın tabana dik 90° durmadığını, yana meyilli durduğunu gördü. ****Bu yaw işareti GELMEZ**** (kanıt: `approach\_target.cpp:52 gripperOrientation() = `setRPY(-M_PI_2+tilt, 0, yaw)` → TCP'yi temiz dikey komut eder; mount yaw +0.8406 = flange Z=J6 bilek eksen etrafında saf dönüş → parmakları yatayda çevirir, gripper'ı yana yatıramaz). Yana yatıklık = ayrı ****model+gerçek kalibrasyon açığı****. Olası kökler: (1) robot eklem sıfır-offset (myCobot gerçek 0 ≠ model 0, özellikle bilek J5/J6), (2) fiziksel gripper braketi roll'ü URDF ile uyumuyor, (3) 110mm gripper ağırlığıyla zayıf bilek yerçekimi sarkması. ****SONRAKİ:**** yatıklık yönü (robot doğru/uzaya/yana) + derecesi ölç; sabit bilek/mount yatıklığıysa URDF'e telafi roll/pitch ekle (gripper_base_to_tcp ya da joint6output_to_gripper_base), rebuild → model gerçek yatıklığı bilince "dik komut" gerçek bileği gerçekten dikleştirir. Eğer poza göre değişiyorsa eklem sıfır-offset kalibrasyonu gerekir.

****Yaw işareti durumu:**** +0.8406 işareti bu yana-yatıklıktan BAĞIMSIZ. Yaw sadece parmak yönünü (azimut) belirler; doğrulaması "parmak çizgisi yönü" ile yapılır, oturum 22 görsel onayı geçerli sayılıyor. Kullanıcı parmak-çapraz şikayeti bildirmedi.

****Δ Operasyonel not – run_approach.sh cleanup trap GÜVENİLMEZ:**** Script SIGINT alınca trap çalıştı ama `setsid` ile AYRI process-group'larda başlatılan çocuk node'lar (demo.launch ağacı, mycobot_bridge, detector, cam_viewer, approach_target) SAĞ KALDI; `/dev/ttyTHS1` tutulu kaldı. Bir sonraki sefer kapatırken: ÖNCE arm_controller action ile robotu 0'a park et (stack ayaktayken), SONRA `pkill -9` / PID ile demo.launch+move_group+ros2_control+realsense+mycobot_bridge+robot_state_publisher öldür, socketleri (`/tmp/mycobot\_bridge.sock`) sil, `fuser /dev/ttyTHS1` ile portu doğrula. Bkz [[feedback_background_node_cleanup]]. (TODO: trap'i process-group'ları doğru süpürecek şekilde düzelt – `setsid` PGID'leri zaten kaydediliyor ama SIGINT tek prosese gidince çocuklar kalıyor.)

2026-06-07 oturum 22 – Stand/gripper görsel kalibrasyon (camera_arm boşluğu, gripper roll, TCP uzunluğu)

Yaklaşma modu çalıştıktan sonra kullanıcı sim'i gerçeğe uydurma turuna girdi. Tüm değişiklikler URDF/görsel (kamera hand-eye pozu KORUNDU). Görüntüleme için kamerasız `/tmp/view\_stand.launch.py` + `/tmp/view\_stand.rviz`

(RobotModel+TF, Image yok → Tegra segfault'undan kaçınır) kullanıldı.

****1) camera_arm boşluğu kapatıldı**** (`mycobot\_world.urdf.xacro`): çubuk 13→****12.5cm****, kameraya değecek konuma alındı (origin x 0.3225→0.2725, yakın uç camera_bottom_screw ~0.21). column+clamp ~5.25cm robota yaklaştırıldı (column x 0.3875→0.335, clamp 0.375→0.3225) ki çubuk her iki uca da bağlı dursun. Δ column MoveIt collision scene'de – 5cm yakınlaşma planlamayı çok az etkiler.

****2) gripper roll – RViz sıfırda gerçek gibi gösteriliyor**** (`mycobot\_280\_jn\_adaptive\_gripper.urdf`, ÜST AKIŞ düzenlendi): gripper flanşa ~****48.16°**** dönük monteli. Ölçüm: robot 0'dayken jog GUI ile gripper düz olana kadar döndürüldü → ****J6 (6. kol eklemi) = -48.16° (-0.84 rad)**** = düz konum; yani joint6=0'da gripper +48.16° eğik. `joint6output\_to\_gripper\_base` origin rpy `1.579 0 0` → `1.579 0 0.8406` (tool eksen=flange Z'de yaw). Artık model joint6=0'da gerçek gibi eğik. **** SIGN DOĞRU (kullanıcı RViz'de onayladı 2026-06-07).**** NOT: bu URDF yaklaşımı seçildi (hardware servo-offset DEĞİL) – model gerçeğe uyuyor, planlama tutarlı kalıyor.

****3) gripper uzunluğu 110mm**** (`mycobot\_world.urdf.xacro`): `gripper\_base\_to\_tcp` y 0.105→****0.110**** (kullanıcı gövdeyi ölçtü = 110mm).

****Ölçümler (sim TF):** gripper_base→tcp(parmak ucu/kavrama) = **10.5→11.0 cm**;** joint6_flange(kol flanşı)→tcp = ****13.9 cm****. Robotun taban-uç efektör max erişimi = ****280mm**** (resmi spec; önceki 570 yanlıştı = çap).

****Robot:**** oturum sonunda pymycobot ile 0 noktasına gönderildi (servolar kilitli, kol düşmez).
`/tmp/zero\_robot.py`.

****Geçici araçlar (/tmp):** view_stand.launch.py + view_stand.rviz (stand görüntüleyici), cam_viewer.py kalıcı kopya** ~/**ros2_ws**'te, gripper_zero_gui.py (jog kalibrasyon GUI), zero_robot.py.

**** TÜMÜ ONAYLANDI (kullanıcı "herşey normal" + "kaydet" 2026-06-07).**** Gripper eğikliği RViz=gerçek aynı yön, camera_arm/column görünümü OK, TCP 110mm.

**** SONRAKİ:**** Yaklaşma modunu yeni TCP (110mm) + yeni gripper modeliyle donanımda tekrar test; sonra madde 17 interaktif pick (iniş+kavrama+place) ya da TCP ince kalibrasyonu.

2026-06-05 oturum 20 – erişim zarfı hesabı + kamera overlay'ine "erişim çemberi"

****1) Dik-tutuş erişim zarfı (URDF FK Monte-Carlo ile hesaplandı):**** Gripper tam dik (top-down, 90°) tutarken parmak ucunun (TCP, ~14cm gripper dahil) ulaşabildiği ****max yatay yarıçap YÜKSEKLİĞE bağlı.**** robot_base = joint1 eksen, r = yatay yarıçap. Referans tablo:

z (mm)	max r (mm)	z (mm)	max r (mm)
---:	---:	---:	---:
0 (masa)	~300–310	110 (pre-grasp)	265
50	286	150	223
90	275	190	135

Kısıtsız (herhangi açı): 3B 570mm, yatay 417mm. **** Δ DÜZELTME (oturum 21):** bu 570 YANLIŞ/yanıltıcı – taban J1'den GERÇEK erişim yarıçapı = RESMİ 280mm (flanşa, gripper'sız). 570 muhtemelen çalışma alanı ÇAPI (2×280) ya da

robot_base'i stand kolonunun dibinden alıp 3B mesafeye kolon yüksekliğini katmış. Gripper+TCP (~+140mm) ile yatay TCP ~417mm OLABİLİR ama sadece kol tam yatayken, vida tutmak için kullanılamaz.** Resmi "280mm" flanşa, gripper'sız. **Oturum 19 başarısızlığının kökü doğrulandı:** vida r=254mm + pre-grasp z=108mm → o yükseklikte sınır ~265mm, yani vida sınırın sadece ~11mm içinde → geçerli IK kümesi ~sıfır ölçü → OMPL "Unable to sample valid goal". Çözüm: vidayı r<200mm'ye getir VEYA `APPROACH\_HEIGHT` 0.10-0.05 (z=50'de sınır 286mm). Hesap saf kinematik – stand/öz-çarpışma hariç, ±5° tolerans. (script: /tmp/reach.py, reach2.py)

2) Kamera overlay'ine erişim çemberi eklendi (`vida\_detector\_node.py`): robot_base XY düzleminde, platform yüksekliğinde (`reach\_z=0.008`) iki çember `/vida/overlay`'e projekte ediliyor – **yeşil=güvenli (`reach\_radius=0.22`), amber=kinematik limit (`reach\_radius\_max=0.30`)**. Her tespit edilen vida base yarıçapına göre halka ile işaretleniyor: içeride=yeşil, sınırda=amber, dışarıda=kırmızı + `r=NNN` etiketi. Robot merkezi ROBOT işaretiyle gösteriliyor. Yeni yardımcılar: `\_opt\_tf` (base-optik tek TF lookup), `\_proj` (pinhole projeksiyon), `\_reach\_polygon`, `\_screw\_base`. Tümü try/except + otf-guard'lı, çizim başarısızsa overlay bozulmaz. Projeksiyon yönü mevcut `\_to\_base`/_`deproject` ile birebir tutarlı (ters). `colcon build --packages-select vida\_vision --symlink-install` ile derlendi (binary 20:05, taze). Parametreler launch'tan/CLI'dan ayarlanabilir.

3) Çember click_to_go penceresine de eklendi (`mycobot\_calibration/scripts/click\_to\_go.py`): aynı yeşil/amber çember + tıklanan hedefe `r=NNN` mm ERISIR/SINIRDA/ERISMEZ` etiketi. `_base_to_cam_RT`/\_`proj`/\_`draw_reach` (quaternion→R, tek TF lookup). Script doğrudan `python3` ile çalışıyor (install kopyası/CMake yok) → rebuild gerekmez. Canlı test edildi: overlay penceresi (PNG ile, vida r=253→amber doğru) + click_to_go penceresi (ekran görüntüsü, yeşil+amber yay görünür) İKİSİ DE çalışıyor.

4) KÖK NEDEN BULUNDU – pick yine STALE BINARY (4. kez!): Kullanıcı "yeşil çembere koydum yine IK hatası" dedi. İnceleme: `pick\_place\_cartesian.cpp` kaynağı **21:23**'te `approachWithFallback` ile güncellenmiş (pre-grasp'ta 6 yaw × 3 tilt=0/20/35° = 18 yönelim dener – dümdüz aşağı tutamazsa gripper'ı eğer). AMA kurulu binary **20:31**'den = fallback YOK, sadece tek dümdüz-aşağı poz → r≈255mm'de IK fail. Kullanıcının paylaştığı log (dün 19:52) daha da eski (`>>` Orientation constraint AÇ` versiyonu). Üstelik hedef x=0.227 y=-0.116 → \*\*r=255mm = AMBER bölge, yeşil değildi.\*\* `colcon build --packages-select mycobot_demo --symlink-install` → binary bugün \*\*20:32 taze\*\*, `strings` "pick_pre (yaw+tilt fallback)" doğrular.

5) TEST SONUCU (2026-06-05 20:36, taze fallback binary) – ERİŞİM/IK ÇÖZÜLDÜ, İLK KEZ TAM İNİŞ: Vida hedefi x=0.176 y=-0.040 z=0.003 → **r=180mm (yeşil çember İÇİNDE ✓)**. Akış:

- Ready , Gripper open

- `>> [PICK] joint plan → pick\_pre (yaw+tilt fallback)` – fallback ÇALIŞTI: yaw=-13° tilt=0° planlanmadı → yaw=+77° tilt=0° planlanmadı → **yaw=-103° tilt=0° ÇÖZÜLDÜ** (2.27s, execute success). İlk iki yaw ~6s timeout/abort, üçüncü tuttu. (Not: tilt=0 yani dümdüz aşağı çözüm bulundu – tilt'e gerek kalmadı, yaw seçimi yetti.)

- `>> [PICK] cartesian descend` → **`[pick\_descend] cartesian fraction = 1.000`** – robot vidaya TAM indi. **Hiçbir önceki oturumda buraya gelinmemişti** – erişim darboğazı RESMEN aştı.

- `>> [PICK] gripper close` → **`Gripper plan başarısız: closed`** → exit 1. YENİ TAKILMA NOKTASI.

6) GRIPPER CLOSE FIX UYGULANDI (çözüm (a) – doğrudan controller). `setGripper()` artık MoveIt `gripper.plan()` (collision-aware) KULLANMIYOR; doğrudan `/gripper\_controller/follow\_joint\_trajectory` action'ına tek-eklem (`gripper\_controller`) pozisyonu gönderiyor (open=0.15, closed=-0.74; SRDF değerleri). Böylece inmiş pozdaki "start state in collision" abort'u ortadan kalkıyor – kapama hareketi planlamaya ihtiyaç duymaz. Değişiklikler: `pick\_place\_cartesian.cpp` (yeni include'lar rclcpp_action/control_msgs/trajectory_msgs, setGripper yeniden yazıldı, `MoveGroupInterface gripper` → `gripper\_client` action client, 3 çağrı yeri güncellendi), `CMakeLists.txt` + `package.xml` (rclcpp_action+control_msgs+trajectory_msgs deps). **Derlendi, binary 20:53 TAZE** `strings` "gripper -> %s OK" doğrular). Robot ayrıca pymycobot ile SIFIRA (home, tüm eklem 0) gönderildi (`/tmp/home\_robot.py`, speed 30) – testten önce güvenli başlangıç.

7) PICK TAM ÇALIŞTI (2026-06-05 21:11 testi) – gripper close fix DOĞRULANDI: Vida r=180mm. Akış: Ready ✓ → `gripper -> open (pos=0.15) OK` ✓ → pick_pre fallback yaw=-13° çözüldü ✓ → descend fraction=1.0 ✓ → **`>> [PICK] gripper close` → `gripper -> closed (pos=-0.74) OK`** (ABORT YOK – fix çalıştı!) ✓ → retreat `[pick\_lift] fraction=1.0` ✓ → **robot vidayı tutup KALDIRDI.** Pick zinciri uçtan uca tamam.

****8)** YENİ darboğaz: PLACE konumu erişilemez. **** place=(0.15,0.15,0.05) → **r=212mm, place_pre z=150mm****. z=150'de dik-tutuş limiti ~223mm → place sınırın 11mm içinde → 18 yaw/tilt'in HEPSİ planlanamadı (pick'teki aynı sınır sorunu, bu kez place'te). Place pozisyonu zarf dışında, fix gerekmez – sadece ****erişilebilir place gir.**** Place env'den ayarlanır (`run\_pick\_test.sh`: PLACE_X/Y/Z, default 0.15/0.15/0.05). İyi seçenek: `PLACE\_X=0.15 PLACE\_Y=0.10 PLACE\_Z=0.02` (r=180mm, place_pre z=120mm → reach ~258mm, bol pay).

****9)** Δ 21:42 testi – pick YİNE tam çalıştı (pick_pre yaw=-139°, descend 1.0, gripper closed OK, lift 1.0) AMA place(0.15,+0.10,0.02 → r=180mm) HÂLÂ tüm yaw/tilt'te patladı. **** BEKLENMEDİK: pick r=180'de (y=-0.041) sorunsuzdu ama place r=180'de (y=+0.10) hiç çözülmedi. Hipotez: (a) +y bölgesi -y'den farklı erişilebilir (kol/stand asimetrisi) – erişim çemberi y-simetrik varsayıyordu, YANLIŞ olabilir; (b) vidayı tutarken pick_lift config'inden +y'ye gövde-üstü geçiş collision/İK'da zor; (c) place_pre z=120 + across-body. Kullanıcı testi durdurdu, robot pymycobot ile SIFIRA döndürüldü (gripper hâlâ KAPALI). **SONRAKİ:** place'i pick'in çalıştığı tarafa yakın seç (y NEGATİF, ör. PLACE_Y=-0.10) VEYA gerçek MoveIt IK ile +y/-y erişim farkını ölç. Çember muhtemelen y-asimetrik düzeltilmeli.**

****10)** DESCEND "saçma hareket" KÖK NEDENİ + FIX – JUMP_THRESHOLD. **** Kullanıcı gözlemi: pick_pre'ye (2) gidiş IK'sı temiz AMA descend'de (2→3) gripper saçma/ışınlanır gibi hareket ediyor (14-35s sürüyor). KÖK NEDEN: `cartesianMove` → `computeCartesianPath(..., JUMP\_THRESHOLD=0.0, ...)` – **jump_threshold=0 = eklem-uzayı süreksizlik kontrolü KAPALI** (MoveIt'in klasik tuzağı). 5mm'lik ardışık waypoint'ler arasında IK farklı konfigürasyona (bilek/dirsek flip) sıçrayabiliyor; fraction=1.0 dese de yol içinde dev eklem atlamaları var → execute edince çığırın reconfig. **FIX: `JUMP\_THRESHOLD 0.0 → 5.0`** (`pick\_place\_cartesian.cpp:56`). Derlendi, binary 22:06 taze. Etki: süreksiz yol artık reddedilir → descend ya düz iner ya temiz fail (fraction<0.95). Eğer fail ederse SONRAKİ adım: pick_pre fallback'i "düz inişe uygun konfigürasyon" seçecek şekilde geliştir (şu an ilk planlanan yaw'ı alıyor, descend-uyumluluğuna bakmıyor). Ayrıca "tam üstte değil" gözlemi kısmen TCP kalibrasyonu (gripper_base_to_tcp=(0,0.105,-0.01) KABA, ~1cm sapma – URDF yorumu doğruluyor).**

****11)** TAM PICK→PLACE ZİNCİRİ TAMAMLANDI (2026-06-05 ~22:10, jump_threshold=5.0 + place -y). **** Akış: pick_pre çözüldü(yaw=-138 **tilt=20**) → descend fraction=1.000 (jump_threshold ile artık SÜREKSİZLİK YOK, temiz) → gripper closed → pick_lift 1.000 → place_pre çözüldü(yaw=+146 tilt=0, -y'de) → place descend 1.000 → place gripper open (vida bırakıldı) → retreat. Script exit 0. **Hareket zinciri uçtan uca ÇALIŞIYOR.** (Place +y başarısız, -y başarılı → erişim y-ASİMETRİK doğrulandı.)**

****12)** KALAN: GRASP KALİTESİ (kavrama) – vidayı tutamıyor. İki kök neden (kullanıcı gözlemi): **** (1) Gripper vidanın **1-2cm yanında**, tam üstünde değil. (2) Vida platformda YATAY/düz dururken gripper ****eğik (tilt=20°)**** geliyor. ANALİZ: `approachWithFallback` önce tilt=0 (6 yaw) dener; bu run'da TUM tilt=0 başarısız → tilt=20°'ye düştü. **Tilt bir tercih değil, erişim tavizi** – vida tilt=0'ın IK'sının olmadığı bir noktadaydı. Düz vida için eğik yaklaşım KÖTÜ: (a) fingerlar açıyla gelince yüzeye/yana kayıyor, (b) ~10cm tool boyu × sin20° yana ötele + (c) TCP kalibrasyonu KABA (`gripper\_base\_to\_tcp=(0,0.105,-0.01)`, URDF "~1.5cm sapma" diyor) → 1-2cm offset. ****ÇÖZÜM YÖNÜ:**** (A) düz vida için tilt'i KALDIR/kısıtla (sadece tilt=0) → ya düz iner tutar ya temiz "erişilemez" der, bozuk eğik kavrama olmaz; vidayı tilt=0'ın çalıştığı güvenli bölgeye koy. (B) TCP kalibrasyonunu ölç/düzeltil (gerçek parmak-kavrama noktası) → kalan ~1cm offset gider. Gripper açık-döngü (firmware get_gripper_value=255, [[feedback_gripper_no_position_readback]]) – tutuş GÖZLE doğrulanmalı.**

****13)** TILT KALDIRILDI (çözüm A uygulandı, 22:33). **** `approachWithFallback` tilts `{0, 20°, 35°}` → `{0.0}`.** Artık SADECE dümdüz-aşağı × 6 yaw. Düz vidada bozuk eğik kavrama imkânsız; ya düz iner-tutar ya temiz "erişilemez". Bedeli: erişim biraz daralır → vidayı yeşil çember içine, sınırdan uzağa koy. Derlendi, binary 22:33 taze.

**** (eski) SONRAKİ:**** Vidayı tilt=0'ın çalıştığı güvenli bölgeye koy → `PLACE\_X=0.15 PLACE\_Y=-0.10 PLACE\_Z=0.02 ./run\_pick\_test.sh`.

****14)** TILT=0 RUN BAŞARILI (full döngü temiz). **** Vida r=163mm. pick_pre yaw=+23° **tilt=0** (4 yaw fail sonra tuttu) → descend 1.0 (temiz, jump_threshold) → gripper closed → pick_lift 1.0 → place_pre yaw=-34° tilt=0 → place descend 1.0 → gripper open (bıraktı) → place_lift 1.0 → Home'a dön → exit 0. **Tüm tilt=0, hareket temiz.**** Yazılım tarafı SAĞLAM. Kalan: kavrama precision'ı (gripper ~1-2cm yanına iniyor → TCP kalibrasyonu).

****15) TCP KALİBRASYONU – yöntem belgelendi, ÖLÇÜM BEKLİYOR (uygulanmadı).**** `gripper\_base\_to\_tcp=(0,0.105,-0.01)` el-tahmini. Gerçek parmak-kavrama noktası mesh içinde → saf-geometriden çıkmıyor, fiziksel ölçüm şart. Yöntem: gripper tilt=0 + bilinen yaw ile vidaya inip kapanınca, parmak merkezi+vida farkı (ex ileri/geri, ey sol/sağ) = TCP hatası. Düzeltme (yaw ile döndür): ` $\Delta x = ex \cdot \cos(yaw) + ey \cdot \sin(yaw)$ `, ` $\Delta z = -ex \cdot \sin(yaw) + ey \cdot \cos(yaw)$ `, yeni offset=`(0+Δx, 0.105-Δyükseklik, -0.01+Δz)`. SONRAKİ: kontrollü kavrama + cetvel ölçümü → hesapla → URDF güncelle → doğrula.

****16) İKİNCİ KAMERA (video kayıt) BULUNDU + ÇALIŞIYOR.**** Platform yanına USB 2.0 Camera bağlandı = ****~/dev/video3**** (D435i video0-2'de; "USB 2.0 Camera", MJPG, 640x480→1024x768, 30fps). İlk kare auto-exposure ısınmadan KARANLIK; ısınınca normal (parlaklık ~125). 90° YAN monte → kayıta döndürülecek. Amaç: robot kavrarken video kaydı. ****~/ros2_ws/record_cam_publisher.py` YAZILDI\*\*** (OpenCV→ROS Image, cv\_bridge'siz, /dev/video3 → `/record_cam/image_raw`, ROTATE env=cw default, standalone python3, rebuild gerekmez). Test edildi: Publisher count 1 ✓.

****17) BÜYÜK ÖZELLİK PLANI – interaktif pick (RViz preview + onay + dar-taraftan tutuş). İNŞAATTA.**** Kullanıcı isteği: detection-kamera penceresinde işaretle + RViz'de hedef nokta marker → robot gitmeden ÖNCE simülasyonda göster → ****ONAY**** → pick_pre (vidanın 10cm üstü, gripper dik) → parmak aç → düz in → ****vidanın dar tarafından tut**** (detector grip_yaw kullan, uzun eksene dik) → kaldır → bırak. + 2. kamera RViz'de. KARARLAR (kullanıcı seçti): onay=****RViz buton****, 2.kamera=****ROS topic+RViz****, sonrası=****tut-kaldır-bırak****. PAKET DURUMU: moveit_visual_tools YOK + apt erişimi YOK → onay butonu ****interactive_markers**** (VAR) ile tıklanabilir "ONAYLA" kutusu; usb_cam YOK → record_cam_publisher.py (kendi Python yayıncım). YAPILACAK: (a) `pick\_interactive.cpp` node – grip_yaw grasp + RViz marker + DisplayTrajectory preview + interactive-marker onay + pick→lift-place→release (jump_threshold=5, setGripper direct-controller mevcut koddan); (b) `pick\_interactive.launch.py` (moveit params); (c) `run\_pick\_interactive.sh` (demo.launch use_rviz:=true + D435i + detector + record_cam + node); (d) RViz config (target marker + /record_cam/image_raw Image + interactive marker + Trajectory). record_cam_publisher.py = TAMAM.

**** SONRAKİ:**** pick_interactive.cpp'yi yaz (madde 17a). Temel = pick_place_cartesian.cpp; ekle: grip_yaw'1 /vida/target quaternion'undan al → pick_pre yaw=grip_yaw (+ grip_yaw+180° fallback, dar-taraf korunur), DisplayTrajectory ile /display_planned_path preview, interactive_markers "ONAYLA" butonu (atomic flag), onay sonrası execute. DERS: stale-binary tuzağı 16/18/19/20'de DÖRT kez – her derlemede `stat` doğrula.

2026-06-04 oturum 19 – "yukarıda durup değmiyor" teşhisi: stale binary (orientation constraint) + detector hedef titreşimi

****Şikâyet:**** `./run\_pick\_test.sh` her seferinde robot hedefin FARKLI bir noktasının ÜSTÜNDE, tam dik duruyor ama vidaya DEĞMİYOR.

****1) Asıl sebep – YİNE stale binary (değmeme):**** Kaynak `pick\_place\_cartesian.cpp` 19:11'de düzenlenip orientation path constraint KALDIRILMIŞTI (dosya başı yorumu: RRTConnect + rejection-sampling kısıtlı planlama saniyelerce timeout/abort ediyor). Ama kurulu binary 18:22'den → hâlâ constraint'li versiyon koşuyordu. Log kanıtı: `>> Orientation constraint AÇ` satırı (yeni kaynakta YOK) + `Planning request accepted ... 25 sn sonra ... aborted` + `[pick\_pre] joint plan başarısız`. ****Tüm pick_*/pick_place.log'larda `cartesian fraction` satırı HİÇ YOK**** = robot descend'e hiç ulaşmıyor, o yüzden vidaya değmiyor. Robot pre-grasp/ready'de takılıp kalıyor. ****colcon build --packages-select mycobot_demo --symlink-install` → binary artık 20:31, kaynaktan yeni.****

****2) İkincil sebep – "farklı nokta" (değme sorunundan bağımsız):**** `vida\_detector\_node.py:236` hedefi "derinliği geçerli olan EN YÜKSEK CONFIDENCE'lı vida" diye seçiyor. Loglar hedefin oynadığını gösterdi: x=0.152 / 0.190 / 0.227 ... Bir runda beyaz vida conf=0.57 → BASE z=****282mm**** (fiziksel imkânsız çöp deprojection). Birden fazla vida / conf titreşimi → her cycle farklı vida latch'leniyor. ****İyileştirme önerileri (henüz UYGULANMADI):**** `conf` 0.50→0.75; tek-vida ile test ya da "en yakın/en merkezi" deterministik seçim; detector'a base-z sanity check (z ∈ [-20,+120]mm dışını reddet).

**** SONRAKİ:**** ERİŞİM darboğazı doğrulandı (test 20:52 → `Unable to sample valid goal`). ****Vidayı robota ~5-8cm**

daha yakın koyup ($r < 200\text{mm}$ hedefle) `cd ~/ros2\_ws && ./run\_pick\_test.sh` tekrarla.** Plan tutar + log'da `[pick\_descend] cartesian fraction = ...` çıkarsa erişim limiti idi, çözüldü. Hâlâ patlarsa stand collision → PlanningScene'de pick pozu validity kontrolü + gerekirse stand collision geometrisini gözden geçir. Yardımcı: `APPROACH\_HEIGHT` 0.10→0.05. Sonra "farklı nokta" için detector conf 0.50→0.75.

> Δ DERS (tekrar eden tuzak): kaynağı her düzenledikten sonra MUTLAKA `colcon build`. Oturum 16, 18 ve 19'da ÜÇ kez stale-binary aynı tuzağa düşüldü – test öncesi `stat` ile binary > kaynak mtime doğrula.

2026-06-04 oturum 18 – stale binary + camera_arm URDF düzeltme + test-müdahale temizliği

1) Stale binary: Oturum 16'da `TOL\_TIGHT` kaynakta 0.40→0.50 yazılmıştı ama `colcon build` YAPILMAMIŞTI. Kurulu binary 16:19'dan, kaynak 17:16'dan → kullanıcının çalıştırdığı test aslında ESKİ 0.40 binary'le koştı, 0.50 hiç denemedi. **colcon build --packages-select mycobot_demo --symlink-install` ile derlendi** (binary artık 18:22, kaynaktan yeni).

2) camera_arm URDF: `mycobot\_world.urdf.xacro` – kamerayı dik kolona tutan yatay ara parça (`robot\_base\_to\_camera\_arm`) hâlâ eski 65cm seviyesindeydi ($z=0.648$), kamera ise kalibrasyon yüksekliğinde (~48cm) → RViz'de kopuk görünüyordu. **z: 0.648 → 0.486** (camera_bottom_screw $z=0.48567$ hizası). Kamera macro origin'i (kalibrasyon, $z=0.48567$) DEĞİŞTİRİLMEDİ – vision'ı bozmamak için. Symlink-install → rebuild gerekmez, sadece robot_state_publisher restart.

3) Test-müdahale kazası + temizlik: URDF değişikliğini RViz'e yansıtmak için stack restart edilirken, kullanıcının 0 SIRADA çalışan `run\_pick\_test.sh`'i (18:44 başlamış, 0.50'yi test ediyordu) fark edilmedi; relaunch onun alt-stack'ini kesti → pick node "Ready'e git" adımıyla action-server kayıbdan kilitlendi (zombi). İki stack iç içe geçti. **Kullanıcı kararı: tam temizlik.** Her iki stack + vida_detector + view_overlay + bridge + port + soketler süpürüldü. Sistem ŞU AN tertemiz (node yok, /dev/ttyTHS1 boş, soket yok).

** SONRAKİ:** Kullanıcı `cd ~/ros2\_ws && ./run\_pick\_test.sh` ile 0.50'yi SIFIRDAN test edecek (artık doğru binary + temiz sistem). Çıkışta `logs/pick\_\*/pick\_place.log` oku. Plan başarısızsa 0.50→0.60. NOT: Claude bundan sonra restart/stack işlemi yapmadan önce `run\_pick\_test.sh` çalışıyor mu diye `pgrep -f run\_pick\_test` ile kontrol etsin.

2026-06-04 oturum 17 – SRDF yükleme hatası düzeltildi

Bağlam: Oturum 16 testi çalıştırıldı. Tespit ZİNCİRİ TAM ÇALIŞTI (yeşil vida BASE (168,-185,34)mm conf 0.82, grip_yaw 119° bulundu). Ama `pick\_place\_cartesian` MoveGroupInterface kurarken düştü: `Could not find parameter robot\_description\_semantic ... Unable to construct robot model` (exit 250).

Kök neden: `run\_pick\_test.sh` node'u düz `ros2 run mycobot\_demo pick\_place\_cartesian` ile başlatıyordu → SRDF parametresi node'a HİÇ verilmiyordu. SRDF topic olarak yayınlanmaz, parametre olarak geçilmeli. Zaten DOĞRU yapan `pick\_place.launch.py` vardı (robot_description + semantic + kinematics yüklüyor) ama script onu kullanmıyordu.

Düzeltilme:

- `pick\_place.launch.py`: place_x/y/z + grasp_z_offset env'den okunup Node parametrelerine eklendi (PLACE_X/PLACE_Y/PLACE_Z/GRASP_Z_OFFSET).

- `run\_pick\_test.sh:200`: `ros2 run ... -p ...` yerine `export PLACE\_\*; ros2 launch mycobot\_demo pick\_place.launch.py`. Launch dosyası install'da symlink → rebuild GEREKMEDİ.

SONRAKİ: Aşağıdaki oturum 17b'ye bakınca SRDF düzeldi ama execution segfault'a takıldı – devamı orada.

2026-06-04 oturum 17b – ros2_control SEGFAULT teşhisi: öksüz mycobot_bridge port çakışması

****Bağlam:**** SRDF düzeltmesinden sonra test tekrar çalıştı. İLERLEME: SRDF yüklendi, robot modeli + IK kuruldu, MoveIt çözümü BULDU. Ama ilk hareket adımı ("Ready'e git, KISITSIZ") execution'da `ABORTED`:

`move\_group: Action client not connected to action server: arm\_controller/follow\_joint\_trajectory`. → orientation constraint sorunu DEĞİL.

****Kök neden (stack.log):**** `ros2\_control\_node` BAŞLANGIÇTA ****SIGSEGV (exit -11)**** ile öldü – `arm\_controller` "Loading" aşamasında, bridge `power\_on OK` + soket kurduktan hemen sonra. Respawn yok → arm_controller action server hiç açılmadı → move_group bağlanamadı.

****SEGFAULT'un asıl sebebi – HAYALET BRIDGE:**** `mycobot\_hardware` (C++) ayrı bir alt-süreç `mycobot\_bridge.py` başlatıp `/dev/ttyTHS1` + `/tmp/mycobot\_bridge.sock` üzerinden konuşuyor. `run\_pick\_test.sh` cleanup'ı bridge'i HİÇ öldürmüyordu (ne preflight ne exit). ros2_control ölünce/öldürülünce bridge ****öksüz kalıp**** portu tutmaya devam ediyor. Sonraki çalışmada taze ros2_control'ün bridge'i çakışıp segfault ediyor. ****DOĞRULANDI:**** test sonrası `fuser /dev/ttyTHS1` → öksüz `mycobot\_bridge.py` (pid 17007) portu tutuyordu; ayrıca 16:20 çalışmasından kalan realsense (10dk, %55 CPU) + 16:30'dan move_group/vida_detector(%41 RAM) hepsi canlıydı. Cleanup tamamen başarısızdı.

****Yapılan düzeltmeler (`run\_pick\_test.sh`):****

- Ortak `sweep\_ros\_ghosts()` fonksiyonu – ****mycobot_bridge.py`'ı + stale soketleri\*\*** (`/tmp/mycobot_bridge.sock`) dahil her şeyi TERM-KILL ile süpürür. Hem preflight hem exit'te çağrılıyor.
- Preflight + exit'te `fuser -k -9 /dev/ttyTHS1` ile port hâlâ tutuluyorsa zorla bırak.
- RC artık logdan gerçek node durumunu okuyor (`ros2 launch` node ölse bile 0 dönüyordu → sahte " BAŞARIYLA"). Artık "process has died exit code N" varsa HATA der.
- Eski hayaletler ELLE temizlendi (port + soket şu an boş/temiz).

**** SONRAKİ (donanım testi):**** Sistem artık TERTEMİZ. `cd ~/ros2\_ws && ./run\_pick\_test.sh` tekrar.

- Eğer segfault GİTTİYSE → execution çalışır, orientation-constrained joint plan (TOL_TIGHT=0.40) gerçek robotta denenir. Plan başarısızsa 0.40-0.50.

- Eğer TEMİZ sistemde HÂLÂ segfault ederse → port çakışması değil, `mycobot\_hardware` C++ read/write veya bridge protokolünde bağımsız bir bug var; o zaman ros2_control'ü tek başına çalıştırıp (`ros2 run mycobot\_hardware ...` / demo.launch fake=false) izole et, gdb/coredump bak.

- Diğer açık konular: en-üst vida depth stratejisi, gripper mapping [0,0.7]→URDF[-0.74,0.15].

2026-06-04 oturum 16 – Constraint gevşetme (sıkı plan başarısız oldu)

****Bağlam:**** Oturum 15 sonrası kullanıcı `run\_pick\_test.sh` çalıştırdı; terminalde joint plan ****"başarısız"**** verdi, sonra sistem dondu, kullanıcı kapattı. Hata = orientation constraint çok sıkı.

****Yapılan:**** `pick\_place\_cartesian.cpp:56` – `TOL\_TIGHT` ****0.30 → 0.40**** rad (~17° → ~23°). Bu X/Z eksen toleransı (grripper "dik kalma" kısıtı); Y (yaklaşım/yaw) zaten `TOL\_FREE = π`, dokunulmadı.

****Derleme :** ****`colcon build --packages-select mycobot\_demo --symlink-install` geçti (51s).**

**** SONRAKİ (donanım testi):**** `cd ~/ros2\_ws && ./run\_pick\_test.sh` → çıkışta `logs/.../pick\_place.log` oku. Yine

"joint plan başarısız" derse 0.40-0.50 aç; hata farklıysa loga göre bak. Diğer açık konular oturum 15'teki gibi (en-üst vida depth stratejisi, gripper mapping [0,0.7]→URDF[-0.74,0.15]).

2026-06-04 oturum 15 – Üretici önerisiyle pick&place: 3-aşama + yönelim kısıtı

****Bağlam:**** Üreticiden IK/MoveIt önerisi geldi: (1) end-effector'a orientation constraint (gripper dik), (2) hareketi 3 aşamaya böl: pre-grasp → linear descent → close gripper.

****Tespit:**** `pick\_place\_cartesian.cpp` 3 aşamayı ZATEN yapıyordu (sabit-poz demo). `goto\_clicked\_point.cpp` ise constraint yerine ayrıık yaw/tilt brute-force taraması yapıyor (eski "hiçbir yaw'da çözüm yok" hatasının kaynağı).

****Yapılan – `pick\_place\_cartesian.cpp` baştan yazıldı:****

- ****Detector entegrasyonu:**** `/vida/target` (PoseStamped, robot_base) abone; SADECE pozisyon alınıyor (detector orientation'ı sadece Z-yaw, yaw-serbest çalıştığımız için yönelimi `gripperDown()` ile biz kuruyoruz). 10sn timeout.

- ****Orientation constraint**** (`uprightConstraint`): tcp link, X/Z tolerans ****SIKI (0.30 rad ~17°)****, ****Y tolerans SERBEST (π)****. Geometri: TCP ****+Y = yaklaşım eksenii**** (gripper aşağı bakınca +Y-dünya -Z), yani Y etrafında dönme = yaw → serbest. Kısıt ready→pick→place joint planlarında aktif, home/ready'den ÖNCE `clearPathConstraints` (yoksa katlanma planlanamaz).

- ****Yaw-serbest**** (kullanıcı kararı): vida simetrik kavrama, IK rahatlasın.

- `setPlanningTime` 3→8s, `NumPlanningAttempts` 5→10 (kısıtlı planlama yavaş).

- Parametreler: `grasp\_z\_offset`, `place\_x/y/z`.

- `gripperDown(yaw)` = `setRPY(- $\pi/2$ ,0,yaw)` (goto ile aynı konvansiyon, literal (1,0,0,0) DEĞİL).

****Derleme** : ****`colcon build --packages-select mycobot\_demo` geçti (tf2 header `.hpp`→`.h` Galactic fix).**

**** SONRAKİ (donanım testi):**** (1) stack+kamera+detector başlat, `ros2 run mycobot\_demo pick\_place\_cartesian` ÇALIŞTIR – gerçek robotta dene. (2) Constraint çok sıkıysa (joint plan başarısız) TOL_TIGHT 0.30→0.40 gevşet. (3) En-üst vida depth stratejisi hâlâ açık (oturum 14'ten). (4) Gripper mapping [0,0.7]→URDF[-0.74,0.15] hizalama pick öncesi.

****Resume:**** `demo.launch.py use\_camera:=true use\_rviz:=false` + `ros2 run vida\_vision vida\_detector` + `ros2 run mycobot\_demo pick\_place\_cartesian`.

****2026-06-04 ek – terminal-only test scripti + canlı overlay (Claude kapalı çalışsın diye):****

- `~/ros2\_ws/run\_pick\_test.sh` – TEK komutla tüm zinciri kurar: sistem fix (rmem/USB/GPU fan) → stack (RViz KAPALI) → controller bekle → vida_detector (Model hazır + ilk HEDEF bekle) → `pick\_place\_cartesian` → çıkışta TÜM node'ları temizler (hayalet bırakmaz). Loglar `~/ros2\_ws/logs/pick\_<zaman>`. Env override: `PLACE\_X/Y/Z`, `GRASP\_Z\_OFFSET`, `SIM=1`, `VIEW=0`.

- `~/ros2\_ws/view\_overlay.py` – RViz/rqt yerine HAFİF cv2 (GTK3) penceresi, `/vida/overlay` canlı. Script `VIEW=1` (default) ile otomatik açar. `q`/ESC kapatır. Headless: `SAVE\_ONLY=1`.

- `vida\_detector\_node.py::\_draw` güncellendi: SEÇİLEN vida artık ****gövdesi yeşil boyalı + yeşil kontur + kalın yeşil çerçeve + siyah-zeminli büyük "HEDEF" etiketi**** (eski sarıydı). Derlendi .

- HÂLÂ donanım testi bekliyor – kullanıcı `./run\_pick\_test.sh` çalıştırıp `logs/.../pick\_place.log`'u okuyacak.

oturum 14 (eski) – FAZ 5 (KAMERA KALİBRASYONU) TAMAMLANDI (easy_handeye2/Park, URDF'e işlendi, doğrulandı).

Faz durumu: FAZ 1-6 , **FAZ 5 (artık atlanmadı)**, FAZ 9 (TRAC-IK) . **Vida pick-and-place – Faz 2/4 ; transform artık DOĞRU kalibrasyona dayanıyor.**

2026-06-02 oturum 14 – HAND-EYE KALİBRASYON ÇÖZÜLDÜ + URDF'e uygulandı

Kamera indirildi: yükseklik 65cm → **46-47cm** (kullanıcı standı indirdi). Eski URDF kamera pozu (z=0.648) geçersizdi.

Kalibrasyon: `easy\_handeye2` eye_on_base, `calibrate\_full.launch.py use\_rviz:=false` (RViz/depth kapalı = Nano donmasın). Örneklemeye **CLI servisleriyle** (rqt donuyor): take_sample/compute/save. 17 poz, bol rotasyon (J4/J5 eğim + J6 in-plane) + mesafe çeşitliliği.

ASIL KEŞİF – algoritma: easy_handeye2 varsayılanı **OpenCV/Tsai-Lenz** bu geometride z'yi çökertiyor (z=0.157 yanlış, geçmiş tüm başarısızlıkların sebebi büyük ihtimalle buydu). `set\_algorithm` ile denenen 5 algoritmadan **OpenCV/Park** ve **Horaud** doğru z=0.475 verdi. Park kullanıldı.

SONUÇ `robot\_base→camera\_color\_optical\_frame`: trans **(0.197,-0.143,0.475)** quat **(0.716,0.697,0.013,0.033)**. Kayıt: `~/ros2/easy\_handeye2/calibrations/mycobot\_d435i\_eob.calib`.

URDF'e işlendi: `mycobot\_world.urdf.xacro:154` d435i macro origin → xyz="0.20961 -0.11021 0.48567" rpy="2.75146 1.49976 -0.41745" (T_origin = T_calib ◦ (bottom_screw→optical)⁻¹). İzole ROS_DOMAIN_ID'de rsp+tf2_echo ile Park'a **birebir** doğrulandı.

Doğrulama (4 bağımsız): Park 0.475 = Horaud 0.475 = düz-board ölçümü 0.470 = cetvel 46-47cm. Robot-dokunma: board ortası hesap (0.169,-0.130,-0.002) vs gerçek tcp (0.177,-0.136,-0.014) → ~1.5cm hata (x,y 6-8mm).

Kod değişikliği: `charuco\_detector.py` artık **StaticTransformBroadcaster** (sparse ~0.7fps tespitte TF bayatlayıp "Take Sample" yanıp sönmessin diye); `calibrate\_full.launch.py`'ye `use\_rviz` arg + depth kapatma eklendi. Snapshot aracı `/tmp/grab\_overlay.py` (overlay'i PNG'ye basıp inline gösterme – Nano'da rqt_image_view açmadan).

Vida detector yeni kalibrasyonla TEST EDİLDİ : **`demo.launch.py use\_camera:=true use\_rviz:=false` (TF doğrulandı = yeni Park değeri yüklü) + `vida\_detector`. Beyaz vida BASE **(217,-89,40)mm** conf 0.79 stabil. **z=40mm DOĞRU** – vidalar KUTUDA (~30 adet, ~4cm yukarıda); z gerçek yükseklik. (Eski (232,-106,28) eski poza dayalıydı, geçersiz.)

Derinlik filtresi iyileştirildi (`vida\_detector\_node.py` `\_depth\_m`): 5x5 düz medyan → 11x11 + MAD aykırı eleme + min geçerli örnek; param `depth\_win`/`depth\_min\_valid`/`depth\_max\_k`. MAD okumayı değiştirmede = depth tutarlı.

SONRAKİ (Faz 5 grasp): (1) dağınık kutuda EN ÜSTTEKİ vida için depth stratejisi (geniş medyan komşuları katıyor → pencereyi küçült veya min/alt-percentil; tepe vida kameraya en yakın). (2) goto_target + gerçek pick. (3) gripper mapping [0,0.7]→URDF[-0.74,0.15] hizala. Detector resume: demo use_camera:=true use_rviz:=false → `ros2 run vida\_vision vida\_detector`.

2026-06-01 oturum 13 – (ESKİ – artık oturum 14'te kalibrasyonla çözüldü) Transform doğrulama, robot-dokunma

****Amaç:**** detector'ın verdiği BASE koordinatı DOĞRU mu? Yöntem (kullanıcı seçti): robotu vidaya ****fiziksel dokundur**** → `tf2\_echo robot\_base tcp` = ground-truth → detector değeriyle karşılaştır. (Cetvel değil robot, çünkü aynı frame tanımı, cetvel hatası yok.)

****Bu oturumda yapılan hazırlık + öğrenilenler:****

- ****`demo.launch.py`'ye `use\_rviz` arg eklendi**** (default true). `use\_rviz:=false` → RViz başlamaz = Jetson Nano RAM tasarrufu. Doğrulama RViz gerektirmiyor (joint_pose_gui + tf2_echo CLI). Dosya symlink-install, rebuild gerekmez.

- ****Resume launch:**** `ros2 launch mycobot\_moveit\_config demo.launch.py use\_camera:=true use\_rviz:=false` (LC_ALL=C LC_NUMERIC=C LANG=C şart).

- ****rmem KALICI DEĞİL:**** her boot `rmem\_max` 212KB'a dönüyor. Kamera frame'i için `sudo sysctl -w net.core.rmem\_max=8388608` GEREK, üstelik ****realsense node başlamadan ÖNCE**** (node soketini o anki rmem ile kurar). Akış gelmiyorsa: rmem'i ayarla → stack'i tamamen kapat → yeniden başlat. (TODO: kalıcı sysctl.d + udev kuralı.)

- ****USB power fix:**** `/sys/bus/usb/devices/\*/power/control = on` (D435i autosuspend, [[feedback_jetson_usb_autosuspend]]).

- ****`ros2 topic hz /camera/...` YANILTIYOR:**** "Terminated"/veri yok gösterir AMA kamera çalışıyor olabilir – image topic'leri best-effort QoS, `topic hz` default reliable abone bağlanamaz. ****Gerçek kontrol:**** best-effort QoS'lu küçük subscriber (bkz `/tmp/cam\_check.py` mantığı) ya da realsense log'unda "RealSense Node Is Up!" + "Open profile". Bu oturumda 7sn'de 16 renk + 17 camera_info frame doğrulandı → kamera SAĞLAM.

- ****TF doğrulandı (stack ayakta):**** `camera\_color\_optical\_frame` → robot_base = trans ****[0.245, -0.158, 0.637]**** quat ****[0.707, 0.707, 0, 0]****. `tcp` → robot_base (katlanmış duruş) = ****[0.020, -0.060, 0.168]****.

**** DOĞRULAMA NOKTASI #1 – detector okuması (KAYITLI, yarın bununla karşılaştır):****

- Kırmızı vida, piksel ****(238,118)****, z=609mm. 20 frame ortalaması: ****BASE x=232, y=-106, z=29 mm**** (±3mm, çok stabil).

- (Not: `vida\_yaw` +40° ↔ -128° arası zıplıyor = PCA uzun-eksen 180° belirsizliği; gripper simetrik, kavramada sorun değil ama ileride tek yöne sabitlenmeli.)

**** YARIN İLK İŞ – kaldığımız yer (robot-dokunma, henüz YAPILMADI):****

1. Stack'i başlat (yukarıdaki resume komutu) + rmem fix. RViz'e gerek yok.

2. ****Detector'ı çalıştırma ŞART DEĞİL**** doğrulama için – nokta #1 değeri (232,-106,29) zaten kayıtlı. (Kamera/vida yeri değişmediyse aynı; değiştiyse detector'ı kısa aç, yeni değeri al.)

3. `ros2 run mycobot\_calibration joint\_pose\_gui` (DISPLAY=:0). Arayüz: Move duration spinbox + 6 joint slider (J1-J6) + butonlar ****Send to robot / All zero / Sync sliders ← current****.

4. ****Güvenli prosedür:**** acil-dur elinin altında → "Sync sliders ← current" (ilk sıçramayı önler) → duration 5-6s → gripper'ı vidanın ÜSTÜNE yüksekte getir → J2/J3 küçük adım → parmak ucu (TCP) vida ucuna DEĞ (bastırma).

5. Temas anında `tf2\_echo robot\_base tcp` oku = ground-truth.

6. ****Delta = detector(232,-106,29) – tcp(ölçülen).**** 2-3 farklı vidada tekrarla. Sabit delta → URDF kamera origin'ine ofset (`mycobot\_world.urdf.xacro:152` `xyz="0.2575 -0.125 0.648"`). Konuma göre değişen delta → rpy/derinlik-scale hatası.

****`Jetson Nano RAM dersi (bu oturum doğrulandı):**** YOLO detector node'u tek başına ****2.1GB RSS****. Full stack +

detector → 3.4GB dolu, 256MB kalır, ****ŞİDDETLİ swap thrash****: ilk inference ~6-7 dk grind (swap I/O, process D-state), sonra warmup ile ~1Hz'e oturuyor. Detector kapatınca anında 2.6GB boşalıyor. ****Robot dokunma adımı**nda detector'a gerek yok → kapat, RAM rahat, jogging akıcı. ****Vida okuması + robot hareketi AYNI ANDA gerekmiyor**; ardışık yap. Bkz `[[feedback_jetson_nano_ram_pressure]]`. RViz'i de açma (`[[feedback_no_visible_primitives]]` değil; RAM için).

****Oturum sonu temizlik**** tüm node'lar ``kill -9`` ile kapatıldı (pkill -f bu ortamda tutmuyor, doğrudan PID gerek). RAM temiz. Bkz `[[feedback_background_node_cleanup]]`.

2026-05-31 oturum 12 – Gripper parmakları + collision kırmızı + vida_vision paketi

****1. Gripper parmakları RViz'de görünmüyordu.**** Kök sebep: ``mycobot_hardware.cpp``'de ``get_gripper_value()`` daima ****255**** döndürüyor (firmware adaptive gripper pozisyonunu raporlamıyor – donanım probe'uyla doğrulandı: ``is_gripper_moving``/``protect_current`` çalışıyor, sadece pozisyon yok). Eski kod 255'i geçerli sanıp $255/100 \times 0.7 = 1.785$ rad'a map ediyordu → URDF limiti ``[-0.74, 0.15]`` dışı → mimic parmaklar gövdeye katlanıp görünmez oluyordu.

- ****Fix 1 (hardware.cpp):**** ``if (grip >= 0 && grip <= 100)`` – geçersiz okuma reddedilir.

- ****Fix 2 (mycobot_bridge.py):**** ``_read_gripper()`` eklendi – ****açık-döngü****: ``get_gripper_value(1)`` dener, geçersizse son ****komutlanan**** değere düşer (``last_gripper_cmd``, ``set_gripper``'da yakalanıyor). Parmaklar artık 0.0'da görünür; komut verilince izler. Gerçek encoder okuması YOK (firmware kısıtı). Bkz memory `[[feedback_gripper_no_position_readback]]`.

- ****C++ mapping uyumsuzluğu (gelecek iş):**** komut yolu ``[0, 0.7] rad`` kullanıyor ama URDF limiti ``[-0.74, 0.15]``; MoveIt'ten gripper'ı tam açmak için hizalanmalı.

****2. Platform/gripper hareket edince kırmızıya dönüyordu = MoveIt collision.**** SRDF'te ``column`` tüm link'lere disable edilmişti (FCL mesh false-positive) ama ``platform_top``/``tutucu_kelepce``/``camera_arm`` hareketli kol+gripper'a karşı AÇIK bırakılmıştı. Robot baseboard üstünde durduğundan gripper aşağı uzanınca FCL çarpışma sanıyordu.

- ****Fix (mycobot.srdf):**** `platform_top/tutucu_kelepce/camera_arm ↔ joint2-6/joint6_flange` tüm gripper link'leri disable eklendi (117-158 çift). NOT: gripper'ın baseboard'a fiziksel çarpma kontrolü artık yok = kullanıcı sorumluluğunda (column'la aynı ödünleşme).

****3. Vida pick-and-place projesi başladı (YENİ).**** Kullanıcı Windows'ta YOLO11-seg modeli eğitti (280 foto, 5 renk vida: sari/beyaz/siyah/yesil/kirmizi), USB ile ``~/Schreibtisch/tasima``'ya getirdi (large ``best.pt`` 55MB + `best.py/inference_template.py`).

- ****Hedef:**** D435i kutudaki vidaları tespit → en iyi/en üstte olanı seç → robot tepeden 90° dik yaklaşıp gripper'la al → başka yere bırak.

- ****Mimari kararı:**** ROS 2 entegre (standalone DEĞİL) – cihaz tek-erişimli (D435i USB + ttyTHS1), realsense2_camera + mycobot_bridge zaten tutuyor. Mevcut goto/MoveIt/TRAC-IK/collision altyapısı kullanılacak.

- ****Faz 0 :** Jetson ortamı hazır – torch 1.13, ultralytics 8.4.33, cv2, pyrealsense2, CUDA True (Tegra X1), Python 3.8 (Galactic ile aynı).

- ****Faz 1 :** Jetson benchmark – yolol11-seg: yükleme 9.4s, inference ****671ms/kare****, ****GPU 214MB**** → large OLDUĞU GİBİ kullanılıyor (small/TensorRT gerekmez). ``model.names`` koddaki sırayla eşleşiyor.

- ****Faz 2 (CANLI ÇALIŞIYOR):**** ``~/ros2_ws/src/vida_vision`` paketi kuruldu (ament_python). ``detection.py`` (inference_template.py'den port, template-mask'sız: maske PCA → merkez+açı+renk) + ``vida_detector_node.py`` (color topic → tespit → ``/vida/overlay``). ****Canlı test:**** 3 vida tespit, kırmızı conf=0.92, merkez+açı stabil. ``cv_bridge`` KULLANILMADI (elle Image↔numpy). 1 Hz timer (0.67s inference birikmesin diye color callback'ten ayrı).

- ****Kamera:**** ``demo.launch.py use_camera:=true`` ile aç (default false). align_depth=true, 424x240x15. rmem 8MB + USB power/control=on zaten ayarlı.

- **Faz 4** (3B + transform CANLI ÇALIŞTI): **vida_detector_node.py** genişletildi –
`/camera/aligned\_depth\_to\_color/image\_raw` (16UC1 mm, pencere medyanı) + `/camera/color/camera\_info` (K) abone.
Akış: seçilen vida merkez piksel → `\_deproject` (pinhole, optik frame) → `\_to\_base` (tf2 lookup
`camera\_color\_optical\_frame` → `robot\_base`, quaternion ile elle döndürme) → `PoseStamped /vida/target` + `Marker
/vida/target_marker`. Vida uzun-ekseni yaw'ı: merkez + eksen boyunca 2. nokta deproject edip base'de açı → grip_yaw
= +90° (dik kavra).

- **Canlı sonuç (stabil):** kırmızı vida conf=0.83, z≈608mm, **BASE** x=232 y=-106 z=28 mm, vida_yaw=42°
grip_yaw=132°. Kareler arası çok tutarlı → transform deterministik.

- **Overlay'de seçilen vida vurgulanıyor:** kalın sarı kutu + nişangah + "HEDEF" + sol altta base koordinatı
(bir sürü kırmızı vida arasından hangisi seçildi belli olsun diye). Seçim kuralı şu an: derinliği geçerli + en
yüksek confidence (TODO: en-üst/izolasyon skoru).

- **DOĞRULAMA YAPILMADI:** x=232,y=-106,z=28 STABİL ama DOĞRU mu bilinmiyor (hand-eye geçmişi). Sonraki
oturum ilk iş: HEDEF vidasının gerçek X/Y/Z'sini ölç → overlay'deki değerle karşılaştır → fark sabitse URDF kamera
pozuna ofset düzeltmesi gir.

- **JETSON NANO BELLEK DARBOĞAZI:** full stack (RViz+move_group+realsense) + YOLO detector (1354MB) + agent →
RAM 3.5GB dolu + **1.8GB swap** → **ŞİDDETLİ thrash**. İlk inference 240s (swap), warmup sonrası ~1Hz'e dönüyor ama UI
donuyor. Operasyonda: **RViz'i Nano'da çalıştırma** (uzaktan/headless), gereksiz uygulama (caja vs) kapat. Olası
ileri çözüm: detector'ı ayrı/daha hafif tut, RViz başka makinede.

Vida pipeline'ı YENİDEN ÇALIŞTIRMA (resume komutları):

...

1. Stack + kamera (LC_NUMERIC=C ZORUNLU):

```
cd ~/ros2_ws && source /opt/ros/galactic/setup.bash && source install/setup.bash
```

```
export LC_NUMERIC=C
```

```
ros2 launch mycobot_moveit_config demo.launch.py use_camera:=true
```

2. Detector (ayrı terminal, aynı source + LC_NUMERIC=C):

```
ros2 run vida_vision vida_detector
```

Görüntü: `ros2 run rqt_image_view rqt_image_view /vida/overlay`

Marker: RViz Add > By topic > /vida/target_marker (Fixed Frame=robot_base)

...

Model `src/vida\_vision/weights/vida\_large.pt` (git'te yok, USB `~/Schreibtisch/tasima`'dan kopyalandı). Kamera ön-
koşulları (rmem 8MB, USB power/control=on) kalıcı görünüyor.

Sıradaki: (1) transform doğruluk ölçümü/ofset, (2) seçim skoru (en-üst/izole), (3) Faz 5: goto_target
entegrasyonu + gerçek pick + place. Gripper komut mapping uyumsuzluğu ([0,0.7] vs URDF [-0.74,0.15]) pick öncesi
düzeltmeli.

2026-05-30 oturum 11 – goto_clicked_point "Approach plan başarısız" teşhisi + düzeltme

Şikâyet: Turuncu dairenin içinde nereye tıklanırsa tıklansın `Planning request aborted` → `Approach plan
başarısız – hiçbir yaw'da çözüm bulunamadı`. Loglarda ayrıca `unknown goal response, ignoring...` / `unknown result
response` hataları.

İki ayrı kök sebep bulundu:

1. ****ÇİFT NODE ÇAKIŞMASI (asıl blocker).** Önceki oturumda Claude'un background'da başlattığı ESKİ binary'li `goto\_clicked\_point` node'u (PID 67683, 23:38) hâlâ ayakta ve `/tmp/goto\_clicked\_point.log`'a yazıyordu. Kullanıcı kendi node'unu yeniden başlatınca İKİ node aynı anda `/clicked\_point` dinleyip aynı `move\_group`'a goal gönderdi → rclcpp_action goal UUID karışması = `unknown goal response, ignoring` race → planlar abort. Kullanıcı eski node'un logunu izlediği için yeni binary'nin loglarını hiç görmedi. ****Çözüm:** eski launch ağacı `kill` edildi, tek node ile temiz relaunch yapıldı.**

2. ****GEOMETRİK / DİK-KISIT (ikincil).** Tıklanan noktalar robota çok yakındı (10–15 cm) ve `z≈0` (platform/tabana düzlemi = omuz `joint2` z=0.158 m'nin 15 cm altı). Gripper'ı DİK aşağı tutarak bu kadar yakına ulaşmak iç ölü bölgede → çözümsüz. Gerçek çalışma alanı bir HALKA (≈13–25 cm yarıçap), turuncu daire sadece dış sınırı gösteriyordu.

****`src/mycobot\_demo/src/goto\_clicked\_point.cpp` değişiklikleri:****

- `MIN\_REACH\_XY = 0.13` eklendi; `onClick`'te `dist\_xy < MIN\_REACH\_XY` ise uyarıp reddediyor (boşa 4 sn plan denemesi yok).

- `publishReachBoundary` → `publishCircle()` helper; artık DIŞ turuncu (25 cm) + İÇ kırmızı (13 cm) daire çiziliyor. Tıklanabilir bölge = halka.

- `downwardOrientation(yaw)` → `gripperOrientation(yaw, tilt)`. Approach denemesi artık yaw × tilt: önce dik (tilt=0), sonra 20° ve 35° eğim. Yakın/alçak noktalarda kol dışarıdan eğik yaklaşabiliyor. Descend sabit oryantasyonla dik iniyor.

- "Hazır" mesajı halka anlatacak şekilde güncellendi.

****Build:**** `colcon build --symlink-install --packages-select mycobot\_demo` (1min 6s). symlink-install olduğu için install→build symlink, ama ÇALIŞAN process eski binary'yi RAM'de tutuyordu → ****derleme sonrası node'u mutlaka relaunch et.****

3. ****ÇİFT TAM LAUNCH STACK'İ (gerçek asıl blocker – sonradan bulundu).** goto node tekilleştirilince bile 17.5 cm'lik (halka içi, erişilebilir) noktada hâlâ abort + `unknown goal response` geldi. `ps` ile bakınca ****İKİ tam `demo.launch.py` çalışıyordu:****

- 23:36 nesli (launch 66306): RSP 66396 + ros2_control 66398 + realsense 66443 + move_group 66408

- 00:28 nesli (launch 73493): RSP 73534 + ros2_control 73536 + move_group 73546 + rviz2 73564 + realsense 73567

Kullanıcı 00:28'de yeni demo.launch başlatmış ama eski 23:36'yı kapatmamış. İKİ `move\_group` aynı `move\_action` server'ını yayınlıyor → goal gönderince iki server cevap veriyor → `unknown goal response, ignoring` race + abort. Ayrıca iki controller_manager + iki realsense. ****Çözüm:** tüm 23:36 nesli öldürüldü (kill 66306 66396 66398 66408 66443). Geriye tek temiz 00:28 nesli kaldı.**

****`src/mycobot\_demo/src/goto\_clicked\_point.cpp` değişiklikleri:****

- `MIN\_REACH\_XY = 0.13` eklendi; `onClick`'te `dist\_xy < MIN\_REACH\_XY` ise uyarıp reddediyor (boşa 4 sn plan denemesi yok).

- `publishReachBoundary` → `publishCircle()` helper; artık DIŞ turuncu (25 cm) + İÇ kırmızı (13 cm) daire çiziliyor. Tıklanabilir bölge = halka.

- `downwardOrientation(yaw)` → `gripperOrientation(yaw, tilt)`. Approach denemesi artık yaw × tilt: önce dik (tilt=0), sonra 20° ve 35° eğim. Descend sabit oryantasyonla dik iniyor.

- "Hazır" mesajı halka anlatacak şekilde güncellendi.

****Build:**** `colcon build --symlink-install --packages-select mycobot\_demo` . symlink-install olsa da ÇALIŞAN process eski binary'yi RAM'de tutuyordu → derleme sonrası relaunch şart.

****Runtime durumu (oturum sonu – bu haliyle kaydedildi):****

- Çalışan TEK temiz stack: 00:28 nesli (PID'ler: launch 73493, RSP 73534, ros2_control 73536, move_group 73546, rviz2 73564, realsense 73567) + goto_clicked_point yeni binary (launch 76923, node 76927, log `/tmp/goto\_clicked\_point.log`).

- ****KAMERA RİSKİ:**** Tıklarken görülen pointcloud muhtemelen ESKİ realsense'ten (66443) geliyordu; o öldürüldü. Yeni realsense (73567) ayakta ve topic'te `Publisher count: 1`, AMA `ros2 topic hz /camera/depth/color/points` 7 sn veri görmedi (D435i tek-erişimli USB; 73567 açılışta cihazı meşgul bulup stream açamamış olabilir). ****Kullanıcı RViz'de pointcloud hâlâ görünüyor mu kontrol etmeli.**** Görünmüyorsa: realsense'i (veya tüm demo.launch'ı) tek sefer temiz relaunch et; USB autosuspend hub'larda `auto` görünüyor (memory: D435i için `power/control=on` gerekebilir, bkz [[feedback_jetson_usb_autosuspend]]).

****Sıradaki adım / dikkat:****

- Önce RViz'de pointcloud + kırmızı iç daire görünüyor mu doğrula. Pointcloud yoksa kamerayı çöz.

- Sonra halka (kırmızı iç ↔ turuncu dış) ARASINA tıkla. Artık tek move_group olduğu için `unknown goal response` GELMEMELİ.

- Hâlâ abort gelirse: bu sefer gerçekten collision/octomap veya reach; move_group terminalindeki abort sebebini (start/goal in collision vs no plan) oku. Loglar `yaw=... tilt=...` formatında.

- ****Önemli ders:**** Hem Claude'un hem kullanıcının başlattığı node/launch'lar oturum sonunda kapatılmazsa çift `move\_group`/çift stack hayaleti oluşuyor; teşhise HER ZAMAN `ps -eo pid,lstart,cmd | grep` ile kaç process var diye bakarak başla. Bkz [[feedback_background_node_cleanup]].

2026-05-27 oturum 10 – IK Orientation + TCP Offset + Click-to-Go Testi

****Yapılanlar:****

1. ****`position\_only\_ik: false` yapıldı**** – `kinematics.yaml`'da `position\_only\_ik: true` idi, IK solver orientation'ı görmezden geliyordu. Gripper açılı geliyordu. `false` yapıldı → TRAC-İK artık orientation constraint'i de çözüyor.

2. ****Gripper-down quaternion düzeltildi**** – `numerical\_ik.py`'daki `GRIPPER\_DOWN\_QUAT` yanlıştı `(1,0,0,0)`. FK ile doğru değer bulundu: `(-0.7071, 0.7071, 0.0, 0.0)` (joints=[0,-1.57,0,-1.57,0,0] pozisyonundan).

3. ****TCP frame kalibre edildi**** – Orijinal `xyz="0 0.07 -0.01"` gripper parmak ucundan ~3.5cm içeride kalıyordu. click_to_go ile fiziksel test yapılarak `xyz="0 0.105 -0.01"` olarak ayarlandı. Artık tcp gripper parmak ucunda.

4. ****click_to_go.py APPROACH_OFFSET kaldırıldı**** – 5cm güvenlik ofseti kaldırıldı, gripper hedefe tam gidiyor.

5. ****RViz Image display Jetson'da segfault**** – moveit.rviz'e eklenen Camera RGB/Depth display'ler OGRE crash'e neden oldu (exit -11). Kaldırıldı. Kamera görüntüsü için `rqt\_image\_view` veya click_to_go penceresi kullanılacak.

6. ****Hand-eye kalibrasyon analizi**** – Eski kalibrasyon dosyaları (`~/ros/easy\_handeye/`) incelendi. ****Ana hata: yanlış TF frame isimleri**** (`g\_base` vs `robot\_base`, `joint6\_flange` vs `tcp`). Ayrıca URDF'teki kamera transform'u TF ağacında çakışma yaratmış olabilir.

****Değiştirilen dosyalar (oturum 10):****

- `src/mycobot\_moveit\_config/config/kinematics.yaml` - `position\_only\_ik: true` → `false`
- `src/mycobot\_world/urdf/mycobot\_world.urdf.xacro` - tcp offset `y=0.07` → `y=0.105`
- `src/mycobot\_calibration/mycobot\_calibration/numerical\_ik.py` - GRIPPER_DOWN_QUAT düzeltildi
- `src/mycobot\_calibration/scripts/click\_to\_go.py` - APPROACH_OFFSET kaldırıldı
- `src/mycobot\_calibration/scripts/camera\_point\_marker.py` - YENİ, kameradan tıklanan noktayı RViz'de marker olarak gösterir
- `src/mycobot\_moveit\_config/rviz/moveit.rviz` - Image display kaldırıldı (segfault fix)

❖ SONRAKİ ADIMLAR

1. **click_to_go ile daha fazla test** - farklı pozisyonlarda sapma ölçümü
2. **Kamera Z kalibrasyonu ince ayar** - URDF'teki kamera transform'u (`xyz="0.22 -0.15 0.55"`) doğrulama
3. **FAZ 5 (hand-eye kalibrasyon)** - doğru frame'lerle tekrar denenebilir, veya mevcut elle ölçüm yeterli
4. **FAZ 7 (perception)** - nesne algılama pipeline'ı
5. **FAZ 8 (manipulation)** - pick & place

2026-05-26 oturum 9 - OMPL Time Parameterization Fix + Gerçek Robot Testi

****Yapılanlar:****

1. **Seri port sorunu çözüldü** - robot fiziksel restart (USB/güç çıkar-tak) ile düzeldi. Bridge `power\_on OK`, `primed positions from bridge`, `Activated` - hiç WARN yok.
2. **KRİTİK BUG: OMPL planning pipeline `request\_adapters` eksikti!**
 - `ompl\_planning.yaml`'da `planning\_plugin` ve `request\_adapters` tanımlı değildi
 - MoveItConfigsBuilder (Galactic) bu parametreleri otomatik eklemiyor
 - **Sonuç:** OMPL planner sadece geometrik path ürettiyordu - 42 nokta, **hepsi `time\_from\_start=0`**, velocity yok
 - JTC (JointTrajectoryController) "time not strictly increasing" ile goal reject ediyordu
 - MoveIt logda: `Goal request rejected` → `Failed to send trajectory` → `ABORTED`
 - **Çözüm:** `ompl\_planning.yaml`'a eklendi:

```
```yaml  

planning_plugin: ompl_interface/OMPLPlanner

request_adapters: >-

 default_planner_request_adapters/AddTimeOptimalParameterization

 default_planner_request_adapters/FixWorkspaceBounds
```



```
default_planner_request_adapters/FixStartStateBounds
default_planner_request_adapters/FixStartStateCollision
default_planner_request_adapters/FixStartStatePathConstraints
...
```

- Yeniden launch'ta doğrulandı: `Using planning request adapter 'Add Time Optimal Parameterization'` ✓

3. **\*\*Doğrudan `ros2 action send\_goal` ile gerçek robot hareket etti\*\*** – JTC goal kabul etti, joint5\_to\_joint4 - 1.804 → -0.023 (sıfır noktası). Bu, bridge+hardware+controller zincirinin çalıştığını kanıtladı.

4. **\*\*RViz Plan & Execute ÇALIŞIYOR\*\*** – Kullanıcı RViz'den interactive marker ile hedef seçip Plan & Execute yaptı, **\*\*gerçek robot hareket etti\*\***. Tam MoveIt 2 pipeline doğrulandı: TRAC-IK → OMPL + TimeOptimalParameterization → JTC → mycobot\_hardware → bridge → pymycobot → fiziksel robot.

**\*\*Değiştirilen dosyalar (oturum 9):\*\***

- `src/mycobot\_moveit\_config/config/ompl\_planning.yaml` – `planning\_plugin` + `request\_adapters` eklendi

**\*\*Çalışan tam pipeline (oturum 9 sonrası doğrulanmış):\*\***

...

RViz MotionPlanning → MoveGroup (TRAC-IK + OMPL + TimeOptimalParameterization)

→ FollowJointTrajectory action → arm\_controller (JTC)

→ ros2\_control → mycobot\_hardware (C++ HW interface)

→ Unix socket → mycobot\_bridge.py (Python subprocess)

→ pymycobot.send\_radians() → /dev/ttyTHS1 @ 1M baud → fiziksel robot

...

**\*\*Başlatma:\*\***

- `~/ros2\_ws/start\_mycobot.sh` veya masaüstü `MyCobot Real Robot` kısayolu

- Robot fiziksel restart gerekebilir (USB/güç çıkar-tak) eğer bridge hata veriyorsa

**### ☯ SONRAKİ ADIMLAR**

1. **\*\*FAZ 5 (hand-eye kalibrasyon)\*\*** – kamera-robot transform'u kalibre et

2. **\*\*FAZ 7 (perception)\*\*** – nesne algılama pipeline'ı

3. **\*\*FAZ 8 (manipulation)\*\*** – pick & place

---

**### 2026-05-26 oturum 8 – TRAC-IK Kurulumu + MoveIt IK Solver Geçiş**

**\*\*Yapılanlar:\*\***

1. **\*\*TRAC-IK kaynak koddan derlendi\*\*** (Galactic'te apt paketi yok):

- `~/ros2\_ws/src/trac\_ik/` – TRAC Labs resmi repo, `rolling` branch clone'landı
- `trac\_ik\_lib` + `trac\_ik\_kinematics\_plugin` Galactic'e uyarlandı:
  - `generate\_parameter\_library` kaldırıldı → doğrudan `node->declare\_parameter()` / `get\_parameter()` ile parametre okuma
  - Header uzantıları: `urdf/model.hpp` → `.h`, `tf2\_kdl.hpp` → `.h`, MoveIt `kinematics\_base.hpp` → `.h`, `robot\_model.hpp` → `.h`, `robot\_state.hpp` → `.h`
  - CMake: `tf2\_kdl::tf2\_kdl` target Galactic'te yok → `ament\_target\_dependencies()` kullanıldı
- `trac\_ik`, `trac\_ik\_examples`, `trac\_ik\_python` → `COLCON\_IGNORE`
- Build OK: `trac\_ik\_lib` (1m13s) + `trac\_ik\_kinematics\_plugin` (1m8s)

2. **\*\*kinematics.yaml güncellendi:\*\***

```
```yaml
```

```
arm:
```

```
  kinematics_solver: trac_ik_kinematics_plugin/TRAC_IKKinematicsPlugin
```

```
  kinematics_solver_search_resolution: 0.005
```

```
  kinematics_solver_timeout: 0.5
```

```
  position_only_ik: true
```

```
  solve_type: Distance
```

```
  epsilon: 1e-5
```

```
```
```

3. **\*\*demo.launch.py hardcoded KDL override kaldırıldı\*\*** – Satır 72-81'de

`moveit\_config.robot\_description\_kinematics` Python dict ile KDL hardcoded'lanmıştı. TRAC-IK + Distance ile değiştirildi.

4. **\*\*Trajectory timestamp fix:\*\*** `moveit\_controllers.yaml` → `allowed\_start\_tolerance: 0.0` (eskisi 0.01). MoveIt mevcut durumu `time\_from\_start=0` ile trajectory'ye ekleyince JTC "Time between points not strictly increasing" hatası veriyordu.

**\*\*TRAC-IK doğrulandı:\*\***

```
```
```

```
[move_group-6] IK Using joint joint2 -2.9321 2.9321
```

```
...
```

```
[move_group-6] Using solve type Distance ← ✓ TRAC-IK aktif
```

```
```
```

Plan başarılı, RViz'de trajectory görüntülendi.

**\*\*△ DEVAM EDEN SORUNLAR:\*\***

1. **\*\*Seri port iletişimi kopuk\*\*** – `mycobot\_bridge` robotla konuşmıyor:

```

[mycobot_bridge] WARN poll: read failed: device reports readiness to read

but returned no data (device disconnected or multiple access on port?)

[mycobot_bridge] WARN poll: 'int' object is not iterable

```

Execute tıklanınca controller goal'ü reddediyor (`Goal request rejected`). Bu hata bridge'in seri porta yazamamasından kaynaklanıyor.

- Bridge process öldükten sonra bile portu tutuyor (orphan process) – `kill -f mycobot\_bridge` ile temizle

- Robot fiziksel restart gerekebilir (USB/güç kablosu çıkar-tak)

2. **\*\*Goal rejected (ikincil)\*\*** – `allowed\_start\_tolerance: 0.0` ile timestamp sorunu çözülmüş olmalı ama seri port kopukluğu nedeniyle test edilemedi.

**### ⦿ SONRAKİ OTURUM ADIMLARI**

1. **\*\*Seri port temizliği:\*\***

```bash

kill -f mycobot_bridge; kill -f ros2_control_node; kill -f move_group; kill -f rviz2

fuser /dev/ttyTHS1 # boş dönmeli

```

2. **\*\*Robot fiziksel restart\*\*** – güç kablosu çıkar, 5sn bekle, tak

3. **\*\*Yeniden launch:\*\***

```bash

export LC_NUMERIC=C && source /opt/ros/galactic/setup.bash && source ~/ros2_ws/install/setup.bash && ros2 launch mycobot_moveit_config demo.launch.py

```

4. **\*\*Bridge hata vermiyorsa\*\*** → Plan & Execute test et → gerçek robot hareket etmeli

5. **\*\*Eğer hâlâ Goal rejected geliyorsa\*\*** → bridge.py'daki poll/read hatasını debug et

**### Değiştirilen dosyalar (oturum 8)**

- `src/trac\_ik/` – YENİ, rolling branch clone + Galactic uyarlamaları

- `trac\_ik\_lib/src/trac\_ik.cpp` – `urdf/model.hpp` → `.h`

- `trac\_ik\_kinematics\_plugin/CMakeLists.txt` – `generate\_parameter\_library` kaldırıldı, `ament\_target\_dependencies` eklendi
- `trac\_ik\_kinematics\_plugin/package.xml` – `generate\_parameter\_library` depend kaldırıldı
- `trac\_ik\_kinematics\_plugin/include/trac\_ik/trac\_ik\_kinematics\_plugin.hpp` – ParamListener → basit member değişkenler
- `trac\_ik\_kinematics\_plugin/src/trac\_ik\_kinematics\_plugin.cpp` – parametre okuma manual, header `.h` düzeltmeleri
- `src/mycobot\_moveit\_config/config/kinematics.yaml` – KDL → TRAC-IK Distance
- `src/mycobot\_moveit\_config/config/moveit\_controllers.yaml` – `allowed\_start\_tolerance: 0.01` → `0.0`
- `src/mycobot\_moveit\_config/launch/demo.launch.py` – hardcoded KDL override → TRAC-IK

---

### 2026-05-25 oturum 7 – RViz Segfault Çözümü + Kamera Düzeltmesi

**\*\*Çözülen kritik sorunlar:\*\***

1. **\*\*RViz2 MotionPlanning segfault (exit -11)\*\*** – `moveit.rviz`'deki `QMainWindow State` hex-encoded dock layout Jetson Tegra'nın OGRE GL driver'ında `RenderWindow::resize()` sırasında geçersiz X11 pointer ile `XFree()` çağrısına neden oluyordu. **\*\*GDB backtrace:\*\*** `\_\_GI\_\_\_libc\_free(0x2ad) → XFree → RenderSystem\_GL.so.1.12.1 → RenderWindowImpl::resize → RenderWindow::exposeEvent`.

- **\*\*Sorun mesh boyutu veya thermal değildi\*\*** – saf X11/OGRE pencere layout bug'ı

- **\*\*Çözüm:\*\*** `moveit.rviz` yeniden yazıldı – aynı MotionPlanning plugin, basitleştirilmiş Window Geometry (MotionPlanning panel dock'ları kaldırıldı, pencere boyutu küçültüldü). Eski config `moveit.rviz.bak` olarak yedeklendi.

- Nisan'da çalışıyordu çünkü: (a) eski URDF mesh'leri bulunamıyordu (basit render), (b) 21 Mayıs'taki workspace yeniden yapılandırmasında yeni panel layout oluşturuldu

2. **\*\*D435i mesh boyutu optimize edildi\*\*** – `\_d435i.urdf.xacro`'ya `use\_mesh` pass-through eklendi, `mycobot\_world.urdf.xacro`'da `use\_mesh="false"` ayarlandı → 16MB `.dae` yerine basit kutu collision. GPU yükü düştü.

3. **\*\*D435i USB kablosu\*\*** – Kamera sürekli disconnect oluyordu (`No such device`). Kullanıcı kabloyu çıkarıp tekrar taktı → kamera stabil 11fps çalışıyor. USB portu `2-1.4` → `2-1.1` olarak değişti (her takışta değişebilir).

4. **\*\*Orphan ROS process'leri\*\*** – Önceki launch'lardan kalan `realsense2\_camera\_node`'lar kamerayı kilitliyordu ("device busy"). Temizlendi.

**\*\*start\_mycobot.sh güncellendi:\*\***

- GPU railgate kapatma + fan max eklendi (RViz OGRE stabilite)
- Launch sonrası otomatik arm + gripper sıfır noktasına gönderme eklendi (her açılışta robot home'a gider)
- `use\_camera:=true` (kamera stabil çalışıyor)

**\*\*Masaüstü kısayolları güncellendi:\*\***

- `MyCobot\_Simulation.desktop` – `sim` argümanı ile fake hardware
- `MyCobot\_Real.desktop` (YENİ) – gerçek robot modu

**\*\*Gerçek robot test edildi:\*\***

- `USE\_FAKE\_HARDWARE=false` ile launch → bridge bağlandı, power\_on OK, controller'lar aktif
- `ros2 action send\_goal` ile arm + gripper sıfır noktasına başarıyla gönderildi
- Fiziksel robot hareket etti, RViz senkron

**### ⚠ DEVAM EDEN SORUNLAR**

1. **\*\*IK error -31 (NO\_IK\_SOLUTION)\*\*** – MoveIt KDL solver her noktada başarısız. **\*\*Sonraki adım:\*\*** TRAC-IK solver'a geç.

2. **\*\*Kamera-robot transform yaklaşık\*\*** – Hand-eye kalibrasyon yapılmadı.

3. **\*\*USB autosuspend kalıcı udev rule\*\*** – Her boot'ta `start\_mycobot.sh` script ile kapatılıyor ama kalıcı `/etc/udev/rules.d/` rule'u henüz yazılmadı.

**### ⚙ YARIN/SONRAKI OTURUM ADIMLARI**

1. **\*\*IK solver düzelt (öncelik):\*\*** `kinematics.yaml` → TRAC-IK

2. **\*\*Kalıcı udev rule\*\*** – D435i USB autosuspend

3. **\*\*Kalıcı sysctl\*\*** – `/etc/sysctl.d/99-ros-cyclonedds.conf`

**### 2026-05-25 önceki oturum – Click-to-Go + Kamera Düzeltmeleri**

**\*\*Çözülen kritik sorunlar:\*\***

1. **\*\*CycloneDDS socket buffer limiti\*\*** – Jetson'da `rmem\_max` default 212KB, D435i image mesajları ~300KB → topic var ama frame akıyor. **\*\*Çözüm:\*\*** `sysctl net.core.rmem\_max=8388608` (start\_mycobot.sh'a eklendi).

2. **\*\*"Out of frame resources"\*\*** – D435i tam çözünürlükte Jetson bellek sınırını aşıyor. **\*\*Çözüm:\*\*** 424x240@15fps, pointcloud/infra/gyro/accel kapalı.

3. **\*\*USB autosuspend\*\*** – D435i her boot'ta uyuyor. `start\_mycobot.sh`'a Intel USB cihazları için otomatik `echo on > power/control` eklendi.

4. **\*\*Kamera TF düzeltildi\*\*** – URDF'teki placeholder → gerçek fiziksel konum **\*\*xyz="0.22 -0.13 0.65"\*\*** rpy="0 π/2

π".

**\*\*Click-to-Go script güncellendi\*\*** (`src/mycobot\_calibration/scripts/click\_to\_go.py`):

- TF ağacı tabanlı, Marker yayınlayan, approach offset'li versiyon

**### Değiştirilen dosyalar (oturum 7)**

- `src/mycobot\_moveit\_config/rviz/moveit.rviz` – QMainWindow layout basitleştirildi (segfault fix). Eski: `moveit.rviz.bak`
- `src/mycobot\_world/urdf/mycobot\_world.urdf.xacro` – D435i `use\_mesh="false"` (16MB .dae → basit kutu)
- `src/realsense-ros/realsense2\_description/urdf/\_d435i.urdf.xacro` – `use\_mesh` parametresi pass-through eklendi
- `start\_mycobot.sh` – GPU railgate + fan max + otomatik sıfır noktası gönderme eklendi, `use\_camera:=true`
- `~/Schreibtisch/MyCobot\_Simulation.desktop` – `sim` argümanı eklendi
- `~/Schreibtisch/MyCobot\_Real.desktop` – YENİ, gerçek robot kısayolu

**### Doğrulanen yeni gerçekler**

- **\*\*RViz segfault sebebi:\*\*** OGRE 1.12.1 `RenderSystem\_GL` + Jetson Tegra X11 driver'ı → `QMainWindow State` dock layout değiştiğinde `RenderWindowImpl::resize()` crash. Çözüm: basit window layout.
- **\*\*D435i USB path değişken\*\*** – her takıp çıkarmada `2-1.4` / `2-1.1` arasında değişebilir. `start\_mycobot.sh`'daki loop tüm Intel USB cihazlarını buluyor.
- **\*\*Kamera 11fps\*\*** (424x240@15fps ayarında) – stabil, disconnect yok (kablo yeniden takıldıktan sonra)
- **\*\*Gerçek robot sıfır noktası çalışıyor\*\*** – `ros2 action send\_goal` ile arm + gripper başarıyla kontrol edildi
- **\*\*GPU railgate + fan\*\*** – RViz stabilitesi için `railgate\_enable=0` + fan PWM 255 şart
- KDL IK solver myCobot 280'de çok kötü performans gösteriyor (TRAC-İK tavsiye)

---

**### Önceki oturum (2026-05-24) – Eski ANLIK DURUM (referans)**

**\*\*Tamamlandı:\*\***

- \*\*Bashrc düzeltildi\*\*** – eski `catkin\_ws/devel/setup.bash` referansı + `QT\_QTA\_PLATFORMTHEME` bozuk satır temizlendi. Yerine ROS 2 Galactic + ros2\_ws auto-source + `LC\_NUMERIC=C` export eklendi. Yedek: `~/bashrc.bak.20260524\_165148`. **\*\*ARTIK her yeni terminal hazır\*\*** – source komutu gereksiz. roslshell/ros2shell launcher'ları kullanılsın.
- \*\*joint\_pose\_gui\*\*** yazıldı (`mycobot\_calibration/joint\_pose\_gui.py`) – tkinter slider GUI, action send\_goal ile robotu joint joint kontrol eder. `ros2 run mycobot\_calibration joint\_pose\_gui` veya kalibrasyon launch'ından otomatik açılır. MoveIt'ı atlar.
- \*\*Robot home pozisyonu\*\*** kısmen – `ros2 action send\_goal /arm\_controller/follow\_joint\_trajectory ...` SUCCEEDED ama joint 4 (joint5\_to\_joint4) gerçek robotta 2.629 rad'da takıldı, URDF limit 2.6179 → 0.65° AŞIM.

**\*\*TIKANDIĞIMIZ NOKTA – kamera yayın yapmıyor:\*\***

- `rs-enumerate-devices` cihazı **\*\*OK\*\*** görüyor (S/N 244222071012, FW 05.17.00.10) → hardware sağlam
- RealSense node "RealSense Node Is Up!" log atıyor → init OK
- `ros2 topic info /camera/color/image\_raw` → Publisher count: 1 (discovery OK)
- AMA `ros2 topic hz` ve `topic echo` → BOŞ, hiç frame gelmiyor
- dmesg'de `usb\_suspend\_both: status 0` + `uvcvideo: Failed to query UVC control 1` → USB autosuspend nedeniyle iç state bozulmuş
- `sudo echo on > /sys/.../power/control` ile autosuspend kapatıldı ama mevcut node bozuk state'te kaldı, kill+restart denemeleri çalışmadı

**\*\*Kullanıcı bilgisayarı kapatıp açacak\*\*** (USB controller tegra-xhci tam reset için tek çözüm).

**\*\*Plan & Execute sorunu (ikincil):\*\*** MoveIt joint 4 limit aşımı nedeniyle start state invalid → Plan "aborted". Robot limit içine girince çözülür. Şimdilik joint\_pose\_gui ile bypass.

**\*\*RViz Image display crash:\*\*** Kalıcı eklenen `/charuco\_detector/image\_annotated` display'i kamera kesintisinde çöktü, sim launch'ı çekti. Alternatif: `rqt\_image\_view` (ayrı pencere, RViz bağımsız).

### ### ☪ YARIN/REBOOT SONRASI ADIMLAR

1. **\*\*USB autosuspend kapat (her boot'ta gerek – kalıcı udev rule yapılacaktır):\*\***

```
```bash
```

```
echo on | sudo tee /sys/bus/usb/devices/2-1.1/power/control /sys/bus/usb/devices/2-1/power/control
```

```
```
```

2. `~/ros2\_ws/start\_mycobot.sh` → "You can start planning now!"

3. **\*\*2. terminal:\*\*** `ros2 topic hz /camera/color/image\_raw` → 30 Hz olmalı (boşsa USB sorunu sürüyor)

4. **\*\*3. terminal:\*\*** `ros2 launch mycobot\_calibration calibrate\_eye\_on\_base.launch.py`

5. **\*\*4. terminal (RViz yerine):\*\*** `ros2 run rqt\_image\_view rqt\_image\_view /charuco\_detector/image\_annotated`

6. joint\_pose\_gui'den robot poz seç → board kamerada görünür → easy\_handeye2 "Take Sample" → 15-20 örnek → Compute → kaydet

**\*\*TODO bir dahaki oturum:\*\***

- USB autosuspend için kalıcı udev rule (`/etc/udev/rules.d/99-realsense-no-autosuspend.rules`)
- joint 4 limit aşımı sorununa kalıcı çözüm (joint\_limits.yaml veya URDF patch #4)
- RViz Image display kalıcı `Enabled: true` ayarı veya tamamen kaldır (rqt\_image\_view daha güvenli)

### 2026-05-24 oturum – FAZ 6'nın hikayesi

Dün pybind11 embedded Python segfault'a takılmıştı. Bugün **\*\*subprocess+IPC\*\*** ile baştan yazıldı, sorun kökten

çözüldü.

**\*\*Mimari:\*\***

- `mycobot\_bridge.py` (scripts/) – pymycobot.MyCobot280 sahibi tek Python process. Unix socket server `/tmp/mycobot\_bridge.sock`. Tek worker thread: komut queue + 20Hz polling.
- `mycobot\_hardware.cpp` – pybind11 TAMAMEN kaldırıldı. POSIX socket client. on\_configure'da `posix\_spawn` ile bridge başlatır, 12s connect retry. Read thread sadece socket okur (Python YOK). write() socket'e komut yazar.
- **\*\*Protokol:\*\*** newline-delimited, space-separated text
  - `state <ts> r1..r6 grip\n` (bridge → client, 20Hz)
  - `send\_radians r1..r6 speed\n`, `set\_gripper v s\n`, `power\_on\n`, `release\_all\n`, `shutdown\n` (client → bridge)

**\*\*Doğrulan davranış:\*\***

- Sim başlatınca bridge subprocess otomatik fork edilir, log'da görünür (`bridge pid=...`)
- 3 controller "Configured and started" (segfault YOK)
- `/joint\_states` ~100Hz, gerçek robot pozlarını yansıtır
- Robot manuel hareket → RViz model senkron takip
- on\_deactivate → `release\_all` → servolar serbest

**### ❖ SIRADAKİ İŞ – FAZ 5 (kalibrasyon)**

**\*\*Kullanıcı tercihi (2026-05-24):\*\*** Robot kalibrasyon sırasında ELLE değil PROGRAMLA hareket ettirilecek. Manuel hareketten sonra servolar tam kilitlenmediği için sample pozisyonu drift ediyor, kalibrasyon bozuluyor. **\*\*Çözüm:\*\*** her joint için slider UI (rqt\_joint\_trajectory\_controller) → robot komutla istenen poza gider, servolar kilitli kalır, sample alırken hareket etmez.

**\*\*Yapılacaklar:\*\***

1. Robotu home pozisyonuna gönder (tüm açılar 0) – başlangıç poza temizle
2. `calibrate\_eye\_on\_base.launch.py`'ya `rqt\_joint\_trajectory\_controller` ekle (joint slider UI)
3. Kalibrasyonu başlat: 15-20 farklı poz → her birinde `Take Sample` (easy\_handeye2 rqt panel)
4. Compute → 5 algoritma karşılaştır → en stabili YAML'a kaydet
5. demo.launch.py'a `static\_transform\_publisher` ile entegre

**### Bugün öğrenilen kritik bilgiler**

**\*\*Galactic pybind11 2.2.7 + bg thread = SEGFAULT\*\*** (kalıcı not [[pybind-galactic-threading]]). Subprocess+IPC ile çöz: Python ayrı process'te, C++ pure POSIX. Bu çözüm `mycobot\_hardware/` paketinde tam çalışıyor – gelecek embedded Python ihtiyaçlarında aynı pattern kullan.

**### Bugün ne yapıldı**



**\*\*Kamera düzeltildi:\*\*** `mycobot\_world.urdf.xacro` – D435i rpy `0,  $\pi/2$ ,  $\pi$ ` (lens dik aşağı bakar, kullanıcının fiziksel kurulumuna uydu).

**\*\*Charuco detector annotated overlay\*\*** eklendi (`charuco\_detector.py`):

- `/charuco\_detector/image\_annotated` topic – yeşil markörler + ChArUco köşeleri + RGB pose eksenini + status banner (READY/PARTIAL/NO BOARD)

- `moveit.rviz`'a "ChArUco Detection" Image display KALICI eklendi

**\*\*FAZ 6 – mycobot\_hardware paketi yazıldı\*\*** (`src/mycobot\_hardware/`):

- C++ HardwareInterface + pybind11 embedded Python (pymycobot bridge) seçildi

- Robot port test edildi: ``/dev/ttyTHS1 @ 1000000 baud`` kesin doğru

- 5 dosya: package.xml, CMakeLists.txt, mycobot\_hardware\_plugin.xml, include/.../mycobot\_hardware.hpp, src/mycobot\_hardware.cpp

- URDF güncellendi: `fake\_components` → `mycobot\_hardware/MyCobotHardware` (env `USE\_FAKE\_HARDWARE=true` ile sim'e dönülür)

- `start\_mycobot.sh` argüman alır: `./start\_mycobot.sh sim` → fake\_components

**\*\*Galactic API uyumsuzlukları aşıldı:\*\***

- `read()/write()` parametresiz (Humble parametrelili)

- `CallbackReturn` namespace dışı global (using statement)

- `HW\_IF\_POSITION/VELOCITY` ayrı header: `hardware\_interface/types/hardware\_interface\_type\_values.hpp`

- pybind11 2.2.7 vendor: `py::module` (`py::module\_` yok), `py::list` → `std::vector<double>` otomatik cast yok (manuel iter)

- `libpython3.8.so.1.0` RTLD\_GLOBAL ile dlopen ŞART (yoksa termios C ext "PyExc\_TypeError undefined symbol")

- scoped\_interpreter sonrası `g\_gil\_release` global tutulmalı (yoksa main thread GIL'i tutar, alt thread'ler bekler)

**\*\*Build:\*\*** OK (`colcon build --packages-select mycobot\_hardware --symlink-install`)

**\*\*Runtime:\*\*** pymycobot ready ✓ → Primed positions ✓ → Activated ✓ → controller\_manager 100Hz ✓ → **\*\*SEGFault (exit - 11)\*\***

**### ÇÖZÜLEMİYEN SORUN:** pybind11 2.2.7 thread-safety

- bg\_thread (background pymycobot polling) Activated'tan hemen sonra segfault tetikliyor

- serial\_mutex\_ ekledim, GIL release fix yaptım, hâlâ crash

- En son durum: **\*\*bg\_thread + write() pymycobot çağrıları KAPATILDI\*\*** (kod yorum satırında)

- Bu haliyle: `joint\_states` prime'dan gelen SABİT değerlerle yayımlanır, gerçek robot pozisyonu takip edilmez

- Build temiz, AMA test edilmedi (yarın yapılacak)

### ☉ YARIN BAŞLARKEN

**\*\*1) Mevcut "Python disabled" durumu test et:\*\***

```
```bash
```

```
~/ros2_ws/start_mycobot.sh
```

Başka terminal:

```
export LC_ALL=C && source /opt/ros/galactic/setup.bash && source ~/ros2_ws/install/setup.bash
```

```
ros2 daemon stop && ros2 daemon start
```

```
ros2 control list_controllers      # 3 active gözükmeli
```

```
ros2 topic hz /joint_states        # 50 Hz akmalı (SABİT değerler ama akıyor olmalı)
```

```
```
```

Eğer akıyorsa → hardware iskeleti OK, sadece thread mimarisi sorun.

**\*\*2) Threading sorununu kalıcı çöz – 4 seçenek:\*\***

- **\*\*a) Subprocess + IPC\*\*** (önerilen, en güvenli): pymycobot ayrı Python process'inde, ZeroMQ/Unix socket ile state paylaş, C++ plugin sadece socket okur. Thread safety derdi yok.

- **\*\*b) rclcpp::TimerBase\*\* + SingleThreadedExecutor:** Update loop'u 10Hz'e indir, read+write timer callback'te ana thread'de yap. GIL contention yok.

- **\*\*c) pybind11 source build\*\*:** 2.10+ sürümü WS içine derle, vendor'u override et. Risk: ROS bağımlılıkları kırılabilir.

- **\*\*d) pymycobot serial protocol'ünü C++'ta direkt yaz\*\*:** Python tamamen elimine. ~400 satır, 4 saat. Protokol Atom/Basic SDK kaynaklarında dökümante.

Karar: muhtemelen **\*\*a) subprocess + IPC\*\*** veya **\*\*b) rclcpp timer\*\*** – kullanıcıyla karar verilecek.

**\*\*3) FAZ 6 tamamlanınca FAZ 5 (kalibrasyon) devam:\*\***

- Board fiziksel hazır + mount edilmiş (2026-05-23)

- `ros2 launch mycobot_calibration calibrate_eye_on_base.launch.py``

- Robotu manuel teach mode → 15-20 poz topla → rqt'da "Compute" → en stabil algoritma → YAML kaydet

### Hızlı sağlık kontrolü

```
```bash
```

```
export LC_ALL=C && source /opt/ros/galactic/setup.bash && source ~/ros2_ws/install/setup.bash
```

```
ros2 control list_controllers      # 3 controller "active" görmeli
```

```
ros2 topic info /camera/color/image_raw # "Publisher count: 1" görmeli
```

```
ros2 run tf2_ros tf2_echo robot_base tcp # default: [0.079, -0.065, 0.526]
```

```
ros2 service list | grep compute      # /compute_ik, /compute_cartesian_path
```

```
```
```

**\*\*Bilinen tek (kabul edilmiş) sorun:\*\*** `joint1\_jet.dae` (alt taban mesh'i) RViz'de render olmuyor – kullanıcı 2026-05-21'de kabul etti.

**\*\*Bashrc notu:\*\*** `~/bashrc` her terminal açılışında `/home/er/catkin\_ws/devel/setup.bash` aramaya çalışıyor (dosya yok) → her seferinde "Datei nicht gefunden" hatası. Zararsız ama kafa karıştırıcı, kalibrasyon sonrası temizlenebilir.

---

## 2026-05-22 – Oturum 2 – FAZ 5 başlatıldı

### Ne değişti

- **\*\*Master plan revize:\*\*** 5 → **\*\*6 paket\*\***. Yeni paket: `mycobot\_calibration` (kullanıcı onayladı).
- `src/easy\_handeye2/` upstream klonu eklendi (Marco Esposito, branch=master, commit b42cae6) – **\*\*Galactic'te zero-port derlendi\*\*** (msgs 2 dk, paket 8 sn). aruco\_ros'a gerek yok; detection katmanını biz yazdık.
- `src/mycobot\_calibration/` paketi açıldı (ament\_python):
  - `mycobot\_calibration/charuco\_detector.py` – RGB stream'i okur, ChArUco pose'unu `camera\_color\_optical\_frame` → charuco\_board` TF olarak yayınlr (cv\_bridge BYPASS – system cv\_bridge OpenCV 4.8'le bozuk, manuel numpy çevirimi)
  - `scripts/generate\_charuco\_board.py` – A4 ChArUco 5x7 PNG + PDF üretir (square=30mm, marker=22mm, DICT\_5X5\_100)
  - `config/charuco\_params.yaml` – board geometrisi, detector parameters
  - `boards/charuco\_5x7\_a4.{png,pdf}` – baskıya hazır (300 DPI, A4 portrait, alt kısımda 50 mm referans cetveli)
  - `launch/calibrate\_eye\_on\_base.launch.py` – detector + easy\_handeye2 `calibrate.launch.py`'yi birlikte ayağa kaldırır

### Hangi dosyalar

- Yeni: `src/easy\_handeye2/` (upstream, read-only)
- Yeni: `src/mycobot\_calibration/` (tam paket)
- Memory: `~/claude/projects/-home-er/memory/project\_ros2ws\_master\_plan.md` (5 → 6 paket)

### Doğrulan

- `colcon build --packages-select easy\_handeye2\_msgs easy\_handeye2 mycobot\_calibration` ✓ sıfır hata
- `ros2 pkg executables mycobot\_calibration` → `charuco\_detector` ✓
- Detector 6 sn smoke test: config log, clean shutdown, exit 0 ✓
- Board generator: PNG 2480x3508 @ 300 DPI = tam A4 ✓

### Bilinen sorunlar / dikkat

- **\*\*cv\_bridge boost init exception\*\*** (OpenCV 4.8 ABI mismatch) – detector bypass ediyor, perception (FAZ 7) için aynı yöntem kullanılacak veya cv\_bridge yeniden derlenecek
- easy\_handeye2 detection sağlamaz – bizim charuco\_detector zorunlu

- Robot serial port hâlâ belirsiz (`/dev/ttyTHS1` vs `THS2`) – FAZ 5 capture aşamasında netleşecek

### Sonraki adım (kullanıcı + Claude)

1. **Kullanıcı**: PDF baskı (100% scale, fit-to-page KAPALI), cetvel doğrulama (50 mm), karton mount
2. **Claude (capture aşaması)**:
  - a. `start\_mycobot.sh` ile demo.launch.py ayağa kaldır (robot + MoveIt + D435i)
  - b. `ros2 launch mycobot\_calibration calibrate\_eye\_on\_base.launch.py` – detector + rqt calibrator
  - c. Robotu manuel teach mode'a al (`release\_all\_servos`), 15-20 farklı poz topla
  - d. rqt'da "Compute" → 5 algoritma karşılaştır → en stabil olanı seç
  - e. Sonucu YAML'a kaydet, `static\_transform\_publisher` ile demo.launch.py'a entegre et
3. **Doğrulama**: `tf2\_echo robot\_base camera\_color\_optical\_frame` stabil + reprojection error < 1 px

### FAZ 5 launch komutu (test edileceği zaman)

```
```bash
```

```
# Terminal 1: tam sistem
```

```
~/ros2_ws/start_mycobot.sh
```

```
# Terminal 2: kalibrasyon
```

```
export LC_ALL=C && source /opt/ros/galactic/setup.bash && source ~/ros2_ws/install/setup.bash
```

```
ros2 launch mycobot_calibration calibrate_eye_on_base.launch.py
```

```
```
```

```

```

## 2026-05-21 – Oturum 1 ÖZET – FAZ 1+2+3+4 KAPALI

### Faz durumu tablosu

| Faz | Durum | Detay |
|-----|-------|-------|
|-----|-------|-------|

|     |     |     |
|-----|-----|-----|
| --- | --- | --- |
|-----|-----|-----|

|                            |        |                                                                                                                                          |
|----------------------------|--------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>**1 – Description**</b> | KAPALI | `robot_base` root + `tcp` (gripper_base z+0.090) + gripper mimic ✓ + D435i REP-103 ✓. TF: `robot_base+tcp=[0.079,-0.065,0.526]` default. |
|----------------------------|--------|------------------------------------------------------------------------------------------------------------------------------------------|

|                        |        |                                                                                                                                                                                  |
|------------------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>**2 – MoveIt2**</b> | KAPALI | ros2_control fake_components + 3 controller active + /compute_ik + /compute_cartesian_path. <b>**Interactive marker + Plan &amp; Execute kullanıcı tarafından test edildi.**</b> |
|------------------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                         |        |                                                                                                                                                    |
|-------------------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>**3 – Temizlik**</b> | KAPALI | 7 arm linkine primitive collision blok eklendi (upstream patch #3). Self-collision check çalışır. Collision warning'leri sıfır. Locale fix kalıcı. |
|-------------------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------|

|                          |        |                                                                                                                                                                                                                           |
|--------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>**4 – RealSense**</b> | KAPALI | D435i canlı topic'leri ✓ (`/camera/color/image_raw`, `/camera/depth/image_rect_raw`, `/camera/color/camera_info`, bonus pointcloud + imu). demo.launch.py'da `use_camera:=true` opsiyonu. Masaüstü kısayolu default true. |
|--------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```
5 – Hand-eye	BEKLEMEDE	Kalibrasyon paketi hazır, robot+kamera çalışıyor. **SIRADAKİ.**
6 – Hardware bridge	KAPALI	mycobot_hardware + mycobot_bridge.py – subprocess+IPC, Plan & Execute gerçek robotta ✓
7 – Perception	HİÇ	mycobot_perception paketi, FAZ 6 sonrası.
8 – Manipulation	HİÇ	mycobot_manipulation paketi.
9 – TRAC-IK	KAPALI	Kaynak koddan derlendi (rolling-Galactic port), kinematics.yaml + demo.launch.py güncellendi. `Using solve type Distance` doğrulandı.
10 – Son temizlik	HİÇ	Lint, dokümantasyon.
```

### ### Workspace yapısı

```
...
~/ros2_ws/
├─ DEVLOG.md (bu dosya)
├─ start_mycobot.sh (executable; LC_ALL=C + source + launch use_camera:=true)
├─ src/
| ├─ mycobot_ros2/ (upstream humble – read-only, COLCON_IGNORE'lar var)
| | ├─ mycobot_description/ (URDF + meshes)
| | └─ mycobot_280/mycobot_280jn/ (slider GUI scripts)
| ├─ realsense-ros/ (upstream tag 4.51.1)
| | └─ realsense2_description/ (sadece bu aktif)
| ├─ mycobot_world/ (BİZİM – robot+gripper+D435i birleştirme)
| | ├─ urdf/mycobot_world.urdf.xacro (robot_base, tcp, ros2_control fake_components)
| | ├─ launch/world.launch.py (sadece display)
| | └─ rviz/world.rviz
| └─ mycobot_moveit_config/ (BİZİM – MoveIt 2 config)
| ├─ config/{mycobot.srdf, kinematics.yaml, joint_limits.yaml, ompl_planning.yaml, moveit_controllers.yaml, ros2_controllers.yaml, cartesian_limits.yaml}
| ├─ launch/demo.launch.py (use_camera:=true/false opsiyonu)
| └─ rviz/moveit.rviz
└─ src_backup_20260521_205720/ (eski iterasyon yedeği, COLCON_IGNORE)
├─ build/ install/ log/ (colcon çıktıları)
└─ ~/Desktop/MyCobot_Simulation.desktop (masaüstü kısayolu, scripti çağırır)
...
```

### ### Upstream'e uygulanan ÜÇ patch (yeniden clone'da kaybolur)

```
1. `mycobot_description/urdf/mycobot_280_jn/mycobot_280_jn_adaptive_gripper.urdf:89` → `lower="-2.932"1` → `"-
```

2.9321"`` (typo)

2. `mycobot\_280/mycobot\_280jn/launch/slider\_control\_adaptive\_gripper.launch.py:81` → `package="mycobot\_280pi"` → `mycobot\_280jn`` (typo)

3. `mycobot\_description/urdf/mycobot\_280\_jn/mycobot\_280\_jn\_adaptive\_gripper.urdf` – 7 arm link'ine primitive `` (cylinder/box) eklendi: joint1, joint2, joint3, joint4, joint5, joint6, joint6\_flange

### FAZ 5 için pre-flight check (hazır)

- pymycobot 4.0.4b5 (pip) ✓
- OpenCV 4.8.0 + ArUco modülü ✓
- ros-galactic-realsense2-camera 4.51.1 ✓
- D435i USB: 8086:0b3a, S/N 244222071012, FW 05.17.00.10 ✓
- Robot serial port: `/dev/ttyTHS1` (777 perm) ya da `/dev/ttyTHS2` (660 perm) – upstream `/dev/ttyTHS0` arıyor, **\*\*bizimkinde farklı\*\***, FAZ 5'te netleşecek

### FAZ 5'e başlarken alınacak kararlar (yarın)

1. **\*\*Marker tipi:\*\***

- ChArUco 5x7 board (önerim – daha doğru, en yaygın)
- Tek büyük ArUco marker (daha kolay)
- Veya kullanıcının önerdiği başka bir tip

2. **\*\*Robot hareket modu:\*\***

- Manuel teach mode (önerim – güvenli, send\_coords yasağı ihlal edilmez)
- pymycobot.send\_radians() (joint-space, master plan'a uygun ama hareket öncesi kollu)
- FAZ 6'yı önce yap (zaman alır, FAZ 5'i erteler)

3. **\*\*Marker mount:\*\***

- Robot ucuna karton+bant ile geçici mount
- Veya 3D-printed adapter

4. **\*\*Marker baskı:\*\***

- A4 kağıda yazdır
- Veya ekranda göster (daha basit ama daha az doğru)

### Galactic'e özgü çözülen API farkları (Humble vs Galactic) – referans

- `MoveItFakeControllerManager` Galactic'te YOK → `moveit\_simple\_controller\_manager` + ros2\_control kullan

- `mock\_components/GenericSystem` Humble adı → Galactic'te **`fake\_components/GenericSystem`**
- `MoveItConfigsBuilder.\_\_init\_\_(package\_name=...)` Humble+; Galactic'te `robot\_name` → paket adı  
`<robot\_name>\_moveit\_config` türetir
- `planning\_scene\_monitor(publish\_robot\_description=...)` Galactic'te yok (sadece publish\_planning\_scene + 3  
publish\_\*\_updates)
- `to\_moveit\_configs()` `cartesian\_limits.yaml`'i implicit yüklüyor – boş bir dosya gerek
- `robot\_description\_kinematics` **\*\*RViz parametrelerine MUTLAKA verilmeli\*\*** – yoksa "No active joints or end  
effectors found" → marker görünmez
- **\*\*LC\_ALL=C zorunlu\*\*** – sistem de\_DE.UTF-8 olduğunda float parse bozulur, marker rendering bozulur, kinematics  
yüklenmez

### Doğrulanmış gerçekler (cross-session güvenle kullanılabilir)

**\*\*Donanım:\*\***

- Jetson Nano aarch64, ROS 2 Galactic, Ubuntu 20.04
- D435i: 848x480 depth @30Hz + 1280x720 color @30Hz + gyro 200Hz + accel 100Hz
- Robot serial: /dev/ttyTHS1 veya /dev/ttyTHS2

**\*\*Kurulu yazılım:\*\***

- pymycobot 4.0.4b5 (pip), OpenCV 4.8.0 + ArUco
- ros-galactic-realsense2-camera, realsense2-description (apt)
- moveit, moveit-configs-utils 2.3.4
- ros2-control 1.6.0, ros2-controllers 1.5.1, joint-trajectory-controller 1.5.1

**\*\*Mesh'ler:\*\***

- joint2..7.dae upstream ile byte-identical
- joint1\_jet.dae upstream'le farklı (1964 byte) ama yapısal aynı; RViz/Ogre render etmiyor ama bu kabul edildi
- STL export hazır: `urdf/mycobot\_280\_jn/joint1\_jet.stl` (3.88 MB) – ileride istenirse

### Test komutları (referans)

```bash

=== HER ŞEYİ BAŞLAT (önerilen) ===

~/ros2_ws/start_mycobot.sh

Ya da masaüstündeki ikona çift tıkla

=== Manuel başlatma (eğer scripti kullanmıyorsan) ===

export LC_ALL=C LC_NUMERIC=C LANG=C

```
source /opt/ros/galactic/setup.bash && source ~/ros2_ws/install/setup.bash
```

```
# Tam sistem (robot + MoveIt + kamera)
```

```
ros2 launch mycobot_moveit_config demo.launch.py use_camera:=true
```

```
# Sadece simülasyon (kamera kapalı)
```

```
ros2 launch mycobot_moveit_config demo.launch.py
```

```
# Sadece display (MoveIt'siz)
```

```
ros2 launch mycobot_world world.launch.py
```

```
# Sadece kamera
```

```
ros2 launch realsense2_camera rs_launch.py pointcloud.enable:=true
```

```
# === DOĞRULAMA ===
```

```
ros2 control list_controllers # 3 active controller
```

```
ros2 service list | grep compute # compute_ik, compute_cartesian_path
```

```
ros2 topic info /camera/color/image_raw # Publisher count: 1
```

```
ros2 run tf2_ros tf2_echo robot_base tcp # ~[0.079, -0.065, 0.526]
```

```
ros2 run tf2_ros tf2_echo robot_base camera_color_optical_frame
```

```
# === BUILD ===
```

```
colcon build --packages-select mycobot_world mycobot_moveit_config --symlink-install
```

```
...
```

```
### Kalıcı dosyalar (yarın açıldığında bulunacaklar)
```

- `~/ros2\_ws/start\_mycobot.sh` – wrapper script
- `~/ros2\_ws/DEVLOG.md` – bu dosya
- `~/Desktop/MyCobot\_Simulation.desktop` – masaüstü kısayolu
- `~/claude/projects/-home-er/memory/` altında 6 memory dosyası:
 - MEMORY.md (index)
 - user_robotics_setup.md
 - project_handeye_calibration.md (eski catkin_ws – sadece tarihsel)
 - project_ros2ws_master_plan.md (10 faz detaylı)
 - feedback_ros2ws_scope.md
 - feedback_devlog_in_repo.md

- feedback_no_visible_primitives.md
- feedback_locale_lc_numeric_c.md

Kalıcı kurallar (her oturum hatırla)

- ROS 2 **Galactic**. Yeni terminal: **`export LC\_ALL=C`** + **`source /opt/ros/galactic/setup.bash && source ~/ros2\_ws/install/setup.bash`**. ROS 1 source etme, catkin_ws source etme.
- Workspace **sadece ~/ros2_ws**.
- Build: **`colcon build --symlink-install`** workspace root'ta.
- Motion planning **sadece MoveIt 2**, **`send\_coords`** YASAK.
- Hedef frame: **`tcp`**, ana referans frame: **`robot\_base`**.
- Resmi upstream paketler **read-only** (mycobot_ros2, realsense-ros). Eklemeler kendi paketlerimizde.
- Her faz test-gated – bir faz stabilize olmadan sonraki faza geçme.
- Ana RViz/MoveIt sahnesinde test için **primitive visual** koyma – debug için ayrı URDF/launch.
- Her fazdan sonra 6 maddelik rapor: ne değişti, hangi dosyalar, hangi test, hangi warning, runtime durumu, sonraki adım.

Bilinen sorunlar / kabul edilenler

1. **~~`joint1\_jet.dae` render~~** – **kullanıcı 2026-05-21'de kabul etti**. STL hazır kalıyor.
2. Octomap warning **`"No 3D sensor plugin(s) defined"`** – FAZ 5 sonrası **sensors_3d.yaml** ile çözüldü olur. Şimdilik zararsız.
3. RViz init'te **"HW not ready"** 4 satır (depth_module advanced params) – non-fatal, kamera tam stream.
4. RViz **"Action server: /recognize_objects not available"** – moveit object recognition kullanmıyoruz, ignore.
5. RViz **"class_loader namespace collision"** – RViz plugin loader uyarısı, zararsız.

box_map.yaml

```
# Mit teach_boxes.py eingelernt – Mittelpunkte der Sortierboxen je Farbe (Frame robot_base, in m).

# pick_tilt.launch.py lädt diese Datei über die Umgebungsvariable BOX_MAP

# (Standardpfad: ~/ros2_ws/box_map.yaml).

# Hinweis: Die Parameternamen (box_sari usw.) sind Daten-IDs und müssen mit

# CLASS_NAMES des Detektors übereinstimmen – NICHT übersetzen.

/**:

  ros__parameters:

    box_sari: [0.2461, -0.0970, 0.0372]

    box_beyaz: [0.1747, -0.1858, 0.0412]

    box_siyah: [0.1153, -0.2448, 0.0548]

    box_yesil: [-0.0084, -0.2693, 0.0536]

    box_kirmizi: [0.2792, 0.0119, 0.0580]
```

calib_check.py

```
#!/usr/bin/env python3

"""
NUR-LESE WAHRNEHMUNGSTEST – Genauigkeit der Kamera->robot_base Zuordnung.

Kann parallel geöffnet werden WAEHREND click_place_moveit.py LAEUFT: rührt
pymycobot/seriellen Port NICHT an, BEWEGT den Roboter NICHT. Hört nur Kamera + TF.
Schreibt die 3D-Position des angeklickten Pixels in robot_base auf den Bildschirm
und loggt sie ins Terminal.

Zweck: die Hälfte "angeklickter Punkt -> base-Koordinate, die der Roboter SIEHT"
isolieren. Wenn du die echte physische Position mit dem Lineal misst und vergleichst,
siehst du den KAMERA-Fehler (extrinsisch/Tiefe) getrennt von der Roboterbewegung.

Verwendung:

python3 calib_check.py

Linksklick = Punkt wählen (base-Koordinate wird geschrieben + bleibt fest)
Rechtsklick = letzten Punkt löschen
k           = alle Punkte speichern (calib_check_points.json)
q           = beenden
"""

import math, json, threading
import cv2, numpy as np
import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy
from sensor_msgs.msg import Image, CameraInfo
from geometry_msgs.msg import PointStamped
import tf2_ros, tf2_geometry_msgs # noqa

CAMERA_FRAME = 'camera_color_optical_frame'
ROBOT_BASE   = 'robot_base'
OUT_FILE     = 'calib_check_points.json'

def img_bgr(m):
```

```

a = np.frombuffer(m.data, np.uint8)

if m.encoding.lower() == 'rgb8':

    return cv2.cvtColor(a.reshape((m.height, m.width, 3)), cv2.COLOR_RGB2BGR)

return a.reshape((m.height, m.width, 3)).copy()


def depth_arr(m):

    if m.encoding == '16UC1':

        return np.frombuffer(m.data, np.uint16).reshape((m.height, m.width))

    if m.encoding == '32FC1':

        return np.frombuffer(m.data, np.float32).reshape((m.height, m.width))

    raise ValueError(m.encoding)


class CalibCheck(Node):

    def __init__(self):

        super().__init__('calib_check')

        self._K = None

        self._bgr = None

        self._depth = None

        self._lock = threading.Lock()

        self._status = 'Warten auf Kamera...'

        self._pts = []          # [{'u', 'v', 'base':[x,y,z], 'z_cam', 'r'}]

        self._tf = tf2_ros.Buffer()

        tf2_ros.TransformListener(self._tf, self)

        qos = QoSProfile(reliability=ReliabilityPolicy.RELIABLE,

                        history=HistoryPolicy.KEEP_LAST, depth=1)

        self.create_subscription(CameraInfo, '/camera/color/camera_info', self._oninfo, qos)

        self.create_subscription(Image, '/camera/color/image_raw', self._oncolor, 10)

        self.create_subscription(Image, '/camera/aligned_depth_to_color/image_raw', self._ondepth, 10)


    def _oninfo(self, m):

        self._K = np.array(m.k, np.float64).reshape(3, 3)


    def _oncolor(self, m):

        try:

            with self._lock:

```

```

        self._bgr = img_bgr(m)

        if self._status.startswith('Warten'):

            self._status = 'Bereit – klicke auf einen Punkt'

except Exception:

    pass

def _ondepth(self, m):

    try:

        with self._lock:

            self._depth = depth_arr(m)

    except Exception:

        pass

def pixel_to_base(self, u, v):

    """GLEICHE Mathematik wie click_place_moveit.pixel_to_base (7x7 Median-Tiefe)."""

    if self._K is None:

        return None, None

    with self._lock:

        depth = self._depth

        if depth is None:

            return None, None

        h, w = depth.shape

        if not (0 <= v < h and 0 <= u < w):

            return None, None

        reg = depth[max(0, v - 3):v + 4, max(0, u - 3):u + 4]

        val = reg[reg > 0]

        if len(val) == 0:

            return None, None

        z = float(np.median(val))

        z = z / 1000.0 if z > 100 else z          # mm -> m (16UC1)

        fx, fy = self._K[0, 0], self._K[1, 1]

        cx, cy = self._K[0, 2], self._K[1, 2]

        pc = PointStamped()

        pc.header.frame_id = CAMERA_FRAME

        pc.header.stamp = self.get_clock().now().to_msg()

        pc.point.x = (u - cx) * z / fx

        pc.point.y = (v - cy) * z / fy

```

```

pc.point.z = z

try:

    pb = self._tf.transform(pc, ROBOT_BASE, timeout=rclpy.time.Duration(seconds=2.0))

except Exception as e:

    self.get_logger().warn(f'TF: {e}')

    return None, z

return np.array([pb.point.x, pb.point.y, pb.point.z]), z


def on_click(self, u, v):

    pb, zcam = self.pixel_to_base(u, v)

    if pb is None:

        self._status = f'Depth/TF nicht verfuegbar ({u},{v}) z_cam={zcam}'

        self.get_logger().warn(self._status)

        return

    r = math.hypot(pb[0], pb[1])

    self._pts.append({'u': u, 'v': v,

                     'base': [float(pb[0]), float(pb[1]), float(pb[2])],

                     'z_cam': float(zcam), 'r': float(r)})

    self._status = (f'#{len(self._pts)} base=[{pb[0]*1000:.0f},{pb[1]*1000:.0f},'

                   f'{pb[2]*1000:.0f}]mm r={r*1000:.0f}mm Tiefe={zcam*1000:.0f}mm')

    self.get_logger().info('>> ' + self._status)


def clear_last(self):

    if self._pts:

        self._pts.pop()

        self._status = f'letzter Punkt geloescht ({len(self._pts)} verbleiben)'


def save(self):

    try:

        with open(OUT_FILE, 'w') as f:

            json.dump({'points': self._pts}, f, indent=2)

            self._status = f'{len(self._pts)} Punkte gespeichert -> {OUT_FILE}'

            self.get_logger().info(self._status)

    except Exception as e:

        self._status = f'Speicher-FEHLER: {e}'


def frame(self):

```

```

with self._lock:

    if self._bgr is None:

        return None

    f = self._bgr.copy()

h, w = f.shape[:2]

for i, p in enumerate(self._pts):

    u, v = p['u'], p['v']

    col = (0, 255, 0)

    cv2.drawMarker(f, (u, v), col, cv2.MARKER_CROSS, 22, 2)

    cv2.circle(f, (u, v), 10, col, 2)

    b = p['base']

    cv2.putText(f, f"#{i+1} [{b[0]*1000:.0f},{b[1]*1000:.0f},{b[2]*1000:.0f}]mm",

                (u + 12, v - 6), cv2.FONT_HERSHEY_SIMPLEX, 0.45, col, 1, cv2.LINE_AA)

cv2.rectangle(f, (0, 0), (w, 26), (40, 40, 40), -1)

cv2.putText(f, self._status, (8, 18), cv2.FONT_HERSHEY_SIMPLEX, 0.5,

            (255, 255, 255), 1, cv2.LINE_AA)

cv2.rectangle(f, (0, h - 22), (w, h), (40, 40, 40), -1)

cv2.putText(f, 'NUR-LESE (Roboter BEWEGT SICH NICHT) Links:Punkt Rechts:letzten-loeschen k:speichern
q:beenden',

            (4, h - 6), cv2.FONT_HERSHEY_SIMPLEX, 0.42, (180, 180, 180), 1, cv2.LINE_AA)

return f

def mouse(ev, x, y, fl, node):

    if ev == cv2.EVENT_LBUTTONDOWN:

        node.on_click(x, y)

    elif ev == cv2.EVENT_RBUTTONDOWN:

        node.clear_last()

def main():

    rclpy.init()

    node = CalibCheck()

    threading.Thread(target=lambda: rclpy.spin(node), daemon=True).start()

    cv2.namedWindow('Calib Check (nur-lese)', cv2.WINDOW_NORMAL)

    cv2.resizeWindow('Calib Check (nur-lese)', 1280, 720)

    cv2.setMouseCallback('Calib Check (nur-lese)', mouse, node)

```

```

print('\n CALIB CHECK – BEWEGT den Roboter NICHT. Klicke auf einen Punkt, base-Koordinate wird geschrieben.\n')

try:
    while rclpy.ok():
        fr = node.frame()

        if fr is not None:
            cv2.imshow('Calib Check (nur-lese)', fr)

            k = cv2.waitKey(50) & 0xFF

            if k in (ord('q'), ord('Q')):
                break

            elif k in (ord('k'), ord('K')):
                node.save()

except KeyboardInterrupt:
    pass

finally:
    cv2.destroyAllWindows()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```


calib_check_points.json

```
{
  "points": [
    {
      "u": 629,
      "v": 559,
      "base": [
        0.29694717129854753,
        -0.13667176548136936,
        0.020216751244700237
      ],
      "z_cam": 0.643,
      "r": 0.32688957466092944
    },
    {
      "u": 620,
      "v": 288,
      "base": [
        0.10562440248411603,
        -0.14063002534695143,
        0.012612990611160502
      ],
      "z_cam": 0.644,
      "r": 0.1758787037398523
    }
  ]
}
```

calib_touch_results.md

Transform doğrulama – robot-dokunma ölçümleri (2026-06-01 oturum 14)

Yöntem: detector BASE vs robot TCP temas (ground-truth). Δ = detector – tcp.

| Trial | Detector BASE (mm) | TCP temas (mm) | Δx | Δy | Δz | Not |
|-------|--------------------|-----------------|------------|------------|------------|--|
| 1 | (150, -70, -5) | (125, -62, -14) | +25 | -8 | +9 | conf düşük (0.25-0.48); reverse yöntem (önce dokun, sonra ölç) |
| 2 | (176, -108, -9) | (135, -109, -9) | +41 | +1 | 0 | conf yüksek (0.79-0.85); forward yöntem; gripper temas oryantasyonu trial-1'den farklı |

Analiz (2 trial)

- Δx : +25, +41 → her ikisinde de detector x'i GERÇEK temastan ileride (pozitif bias), ama sabit değil (büyüyor)
- Δy : -8, +1 → küçük, manuel dokunma gürültüsü içinde
- Δz : +9, 0 → küçük
- KONFOUND: iki dokunmada gripper oryantasyonu çok farklıydı (quaternion) + `tcp` frame'i tam parmak ucu değil → x farkının bir kısmı bu ofsetin dönmesinden, saf transform hatası değil.
- Sonraki: temiz sonuç için DİK (top-down) sabit oryantasyonla 1-2 dokunma daha; o zaman tcp x-y = parmak ucu x-y = vida x-y olur.

KALİBRASYON iter-1 uygulandı (2026-06-01)

- Dosya: `src/mycobot\_world/urdf/mycobot\_world.urdf.xacro:152`
- Kamera origin x: $0.2575 \rightarrow 0.2245$ (-33mm = ortalama Δx)
- TF doğrulama: camera_color_optical_frame→robot_base x: 0.245 → 0.212 ✓ (-33mm)
- Build: mycobot_world --symlink-install OK
- y, z dokunulmadı (gürültü içinde)
- BEKLEYEN: temiz top-down dokunmayla artık (residual) hata ölçümü → fazla/eksik düzeltme ayarı
- UYARI: veri konfound'lu (gripper oryantasyonu + tcp-parmak ucu ofseti + robot servo ~30mm). iter-2 için kartezyen GUI + sabit vida ile DİK dokunma önerilir.

cam_viewer.py

```
#!/usr/bin/env python3

# cam_viewer.py – D435i Kamera (Detector Overlay) in einem einzigen cv2-Fenster.
#
# Warum nicht RViz: Das RViz Image-Display löst auf diesem Jetson Tegra einen
# OGRE/X11 Render-Fenster-Resize-Bug aus und bringt RViz zum Segfault (DEVLOG
# Sitzung 7 + Sitzung 21). Deshalb werden die Kameras AUSSERHALB von RViz in einem
# leichten OpenCV-Fenster gezeigt (gleich wie view_overlay.py, auf dieser Maschine
# bewährte Methode). Ohne cv_bridge.
#
# Verwendung: python3 ~/ros2_ws/cam_viewer.py    (q / ESC zum Schließen)
#   env: CAM1_TOPIC, CAM2_TOPIC, PANEL_H

import os

import numpy as np

import cv2

import rclpy

from rclpy.node import Node

from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy

from sensor_msgs.msg import Image

from std_msgs.msg import Empty

# CAM1 = Detector Overlay (YOLO-Erkennung + AUSGEWAHLTE Schraube als "ZIEL" markiert).
#   Wenn der Detector stirbt, friert das letzte Bild ein (der ausgewählte Punkt
#   bleibt sichtbar). Ein best-effort Sub empfängt auch von einem reliable Pub
#   → am kompatibelsten.

CAM1 = os.environ.get("CAM1_TOPIC", "/vida/overlay")          # D435i + Erkennungs-Overlay

# Rohbild-Fallback: solange der Detector noch laedt/warmuppt (auf dem Nano 1-2 min),
# wird das ROHE Kamerabild gezeigt statt eines schwarzen "warten..."-Fensters.

RAW = os.environ.get("RAW_TOPIC", "/camera/color/image_raw")

# 480 = native Hoehe des 848x480-Streams → 1:1, kein unscharfes Hochskalieren.

PANEL_H = int(os.environ.get("PANEL_H", "480"))

def img_to_bgr(msg):

    h, w = msg.height, msg.width

    enc = msg.encoding
```

```

buf = np.frombuffer(msg.data, dtype=np.uint8)

try:

    if enc == "rgb8":

        return cv2.cvtColor(buf.reshape(h, w, 3), cv2.COLOR_RGB2BGR)

    if enc == "bgr8":

        return buf.reshape(h, w, 3)

    if enc in ("mono8", "8UC1"):

        return cv2.cvtColor(buf.reshape(h, w), cv2.COLOR_GRAY2BGR)

    return buf.reshape(h, w, -1)[: , : , :3]

except Exception:

    return None

```

```

class CamViewer(Node):

    def __init__(self):

        super().__init__("cam_viewer")

        self.fl = None

        self.raw = None          # Rohbild-Fallback bis das erste Overlay eintrifft

        self._overlay_seen = False

        be = QoSProfile(depth=1, reliability=ReliabilityPolicy.BEST_EFFORT,

                        history=HistoryPolicy.KEEP_LAST)

        self.create_subscription(Image, CAM1, self.cb1, be)

        self.create_subscription(Image, RAW, self.cb_raw, be)

        # pick_tilt HOVER-Bestätigung: Taste 'g' -> /pick/confirm (std_msgs/Empty).

        self.pub_confirm = self.create_publisher(Empty, "/pick/confirm", 1)

        self._confirm_flash = 0   # kurzer "BESTAETIGUNG GESENDET" Zähler am Bildschirm

        self.win = "Kamera D435i + Erkennung    -    g=BESTAETIGEN  q/ESC=schliessen"

        cv2.namedWindow(self.win, cv2.WINDOW_NORMAL)

        cv2.resizeWindow(self.win, int(PANEL_H * 848 / 480), PANEL_H + 20)

        self.timer = self.create_timer(1.0 / 15.0, self.tick)

        self.get_logger().info(f"cam_viewer: {CAM1} (Fallback: {RAW})")

    def cb1(self, m):

        self.fl = img_to_bgr(m)

        self._overlay_seen = True

    def cb_raw(self, m):

```

```

# Nur solange kein Overlay kam dekodieren – danach unnoetige CPU-Last sparen.

if not self._overlay_seen:

    self.raw = img_to_bgr(m)

def _panel(self, img, label, show_label=True):

    h = PANEL_H

    if img is None:

        p = np.zeros((h, int(h * 1.4), 3), np.uint8)

        cv2.putText(p, label + ": warten...", (10, h // 2),

                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)

        return p

    sc = h / float(img.shape[0])

    # INTER_AREA beim Verkleinern (scharf), INTER_CUBIC beim Vergroessern (weniger

    # Treppen als das Standard-INTER_LINEAR) – Bildqualitaets-Wunsch 2026-07-16.

    interp = cv2.INTER_AREA if sc < 1.0 else cv2.INTER_CUBIC

    if abs(sc - 1.0) < 1e-3:

        r = img.copy()

    else:

        r = cv2.resize(img, (max(1, int(img.shape[1] * sc)), h), interpolation=interp)

    # show_label=False: im D435i-Overlay-Panel kein Label zeichnen – das Detector-

    # Overlay schreibt bereits seinen eigenen "N Schrauben" + ZIEL Text, damit es

    # sich nicht überlagert.

    if show_label:

        cv2.putText(r, label, (8, 26), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

    return r

def tick(self):

    frame = self.f1

    if frame is None and self.raw is not None:

        # Detector laedt noch – Rohbild zeigen, mit Hinweis.

        frame = self.raw.copy()

        cv2.putText(frame, "Detector laedt (CUDA-Warmup ~1-2 min)...", (8, 24),

                    cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 0, 0), 3, cv2.LINE_AA)

        cv2.putText(frame, "Detector laedt (CUDA-Warmup ~1-2 min)...", (8, 24),

                    cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 255), 1, cv2.LINE_AA)

    combo = self._panel(frame, "Kamera D435i + Erkennung (ZIEL)", show_label=False)

    # Bestätigungs-Hinweis + Flash (duenn, mit Kontur – Wunsch 2026-07-16: Texte nicht fett)

```

```

hint = "g = BESTAETIGEN  (1: Hover->ab  2: gegriffen->heben)"

cv2.putText(combo, hint, (8, combo.shape[0] - 12),

             cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 3, cv2.LINE_AA)

cv2.putText(combo, hint, (8, combo.shape[0] - 12),

             cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 255), 1, cv2.LINE_AA)

if self._confirm_flash > 0:

    cv2.putText(combo, "BESTAETIGUNG GESENDET", (combo.shape[1] // 2 - 120, 40),

                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 0), 3, cv2.LINE_AA)

    cv2.putText(combo, "BESTAETIGUNG GESENDET", (combo.shape[1] // 2 - 120, 40),

                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 1, cv2.LINE_AA)

    self._confirm_flash -= 1

cv2.imshow(self.win, combo)

k = cv2.waitKey(1) & 0xFF

if k in (27, ord('q')):

    rclpy.shutdown()

elif k == ord('g'):

    self.pub_confirm.publish(Empty())

    self._confirm_flash = 15    # ~1 s (15 Hz)

    self.get_logger().info("/pick/confirm gesendet (Bestaetigung: Hover->ab ODER gegriffen->heben)")

```

```

def main():

    rclpy.init()

    node = CamViewer()

    try:

        rclpy.spin(node)

    except KeyboardInterrupt:

        pass

    finally:

        try:

            cv2.destroyAllWindows()

        except Exception:

            pass

    node.destroy_node()

    if rclpy.ok():

        rclpy.shutdown()

```

```
if __name__ == "__main__":  
    main()
```

camera_center.py

```
#!/usr/bin/env python3

"""Öffnet das D435i-Farbbild und markiert das exakte MITTE-Pixel mit Punkt + Kreuz.

Jetson Nano: 424x240@15fps zwingend (volle Auflösung gibt 'Out of frame resources').

Beenden: im Fenster ESC oder q. Mit 's' wird das Bild als ~/camera_center.png gespeichert.

"""

import os

import cv2

import numpy as np

import pyrealsense2 as rs

W, H, FPS = 424, 240, 15

pipe = rs.pipeline()

cfg = rs.config()

cfg.enable_stream(rs.stream.color, W, H, rs.format.bgr8, FPS)

profile = pipe.start(cfg)

# Echte Stream-Auflösung aus dem Profil holen (falls das Gerät anders liefert, bleibt kompatibel)

vsp = profile.get_stream(rs.stream.color).as_video_stream_profile()

w, h = vsp.width(), vsp.height()

cx, cy = w // 2, h // 2

print(f"[camera_center] Stream {w}x{h} | MITTE-Pixel = ({cx}, {cy})")

win = "D435i - MITTE-PUNKT (q/ESC beenden, s speichern)"

cv2.namedWindow(win, cv2.WINDOW_NORMAL)

cv2.resizeWindow(win, w * 2, h * 2) # zum bequemen Sehen 2x vergrößern

try:

    while True:

        frames = pipe.wait_for_frames()

        color = frames.get_color_frame()

        if not color:

            continue

        img = np.asanyarray(color.get_data())
```



```

# Kreuz (dünn, gelb) – über das ganze Bild

cv2.line(img, (cx, 0), (cx, h), (0, 255, 255), 1)

cv2.line(img, (0, cy), (w, cy), (0, 255, 255), 1)

# Exakter Mittelpunkt – roter gefüllter Kreis + weißer Ring

cv2.circle(img, (cx, cy), 5, (0, 0, 255), -1)

cv2.circle(img, (cx, cy), 7, (255, 255, 255), 1)

cv2.putText(img, f"({cx},{cy})", (cx + 10, cy - 10),

            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1, cv2.LINE_AA)


cv2.imshow(win, img)

k = cv2.waitKey(1) & 0xFF

if k in (27, ord('q')):

    break

if k == ord('s'):

    out = os.path.expanduser("~/camera_center.png")

    cv2.imwrite(out, img)

    print(f"[camera_center] Gespeichert: {out}")

finally:

    pipe.stop()

    cv2.destroyAllWindows()

```

check_grasp.py

```
#!/usr/bin/env python3

"""

check_grasp.py – diagnostiziere ein fehlgeschlagenes Abstiegsziel.

Führt auf Hover- / Oberflächen- / Grasp-Höhen eine IK + Kollisionsprüfung durch;

listet auf, in welcher Höhe der Abstieg an welchem Link-Paar hängen bleibt.

Verwendung: source /opt/ros/galactic/setup.bash && source install/setup.bash

python3 check_grasp.py

"""

import sys

import rclpy

from rclpy.node import Node

from geometry_msgs.msg import PoseStamped

from moveit_msgs.srv import GetPositionIK, GetStateValidity

FRAME = "robot_base"

GROUP = "arm"

EEF = "tcp"

# Letztes fehlgeschlagenes Latch: x=179 y=-32 z=3mm, offset -0.025 → grasp z=-0.022

X, Y = 0.179, -0.032

QUAT = (-0.735, -0.031, 0.029, 0.677)

HEIGHTS = [

    ("hover    (+78mm)", 0.078),

    ("Oberflaeche (0mm)", 0.000),

    ("grasp     (-22mm)", -0.022),

]

def pose_at(z):

    ps = PoseStamped()

    ps.header.frame_id = FRAME

    ps.pose.position.x = X

    ps.pose.position.y = Y

    ps.pose.position.z = z
```

```

ps.pose.orientation.x, ps.pose.orientation.y, ps.pose.orientation.z, ps.pose.orientation.w = QUAT
return ps

```

```

def main():

```

```

    rclpy.init()

    node = Node("check_grasp")

    ik = node.create_client(GetPositionIK, "/compute_ik")

    sv = node.create_client(GetStateValidity, "/check_state_validity")

    for c, n in ((ik, "/compute_ik"), (sv, "/check_state_validity")):

        if not c.wait_for_service(timeout_sec=10.0):

            print(f"!! {n} nicht vorhanden – laeuft move_group?"); rclpy.shutdown(); return

```

```

def solve_ik(z, avoid):

```

```

    req = GetPositionIK.Request()

    req.ik_request.group_name = GROUP

    req.ik_request.ik_link_name = EEf

    req.ik_request.pose_stamped = pose_at(z)

    req.ik_request.avoid_collisions = avoid

    req.ik_request.timeout.sec = 2

    f = ik.call_async(req)

    rclpy.spin_until_future_complete(node, f, timeout_sec=8.0)

    return f.result()

```

```

def validity(robot_state):

```

```

    req = GetStateValidity.Request()

    req.robot_state = robot_state

    req.group_name = GROUP

    f = sv.call_async(req)

    rclpy.spin_until_future_complete(node, f, timeout_sec=8.0)

    return f.result()

```

```

print(f"\n=== Ziel x={X} y={Y} (r={ (X*X+Y*Y)**0.5*1000:.0f}mm), quat={QUAT} ===")

```

```

for name, z in HEIGHTS:

```

```

    # 1) Gibt es eine kollisionsVERMEIDENDE IK? (die Trajektorie verlangt das)

    r_avoid = solve_ik(z, True)

    ok_avoid = r_avoid and r_avoid.error_code.val == 1

```

```

# 2) Löse eine IK, die die Kollision IGNORIERT, dann schaue WELCHES Paar kollidiert

contacts = []

valid = None

r_free = solve_ik(z, False)

joints = None

if r_free and r_free.error_code.val == 1:
    v = validity(r_free.solution)

    if v is not None:
        valid = v.valid

        contacts = [f"{c.contact_body_1} x {c.contact_body_2}" for c in v.contacts]

        js = r_free.solution.joint_state

        joints = {n: p for n, p in zip(js.name, js.position)}

    tag = "    ERREICHBAR (kollisionsfreie IK vorhanden)" if ok_avoid else "    KEINE kollisionsfreie IK"

    print(f"\n[{name}]  {tag}")

    if joints:
        order = ["joint2_to_joint1", "joint3_to_joint2", "joint4_to_joint3",
                  "joint5_to_joint4", "joint6_to_joint5", "joint6output_to_joint6"]

        vals = [joints.get(k, joints.get(k.replace("_to_", "_"), 0.0)) for k in order]

        # auch mit rohen Namen versuchen

        if all(v == 0.0 for v in vals):
            vals = list(joints.values())[:6]

            print("    Gelenke(rad): " + " ".join(f"{x:+.2f}" for x in vals))

    if valid is not None:
        print(f"    geometrischer IK-Status: {'GUELTIG' if valid else 'KOLLISION'}")

        if contacts:
            for cpair in sorted(set(contacts)):
                print(f"        ↳ {cpair}")

    elif not r_free or r_free.error_code.val != 1:
        print("    (in dieser Hoehe konnte gar keine IK geloest werden – ausser Reichweite/Limit)")

node.destroy_node()

rclpy.shutdown()

if __name__ == "__main__":
    main()

```

confirm_key.py

```
#!/usr/bin/env python3

# =====

# confirm_key.py - 'g'-Bestätigung in der RViz-Vorschau-Phase (da cam_viewer geschlossen ist).

#

# Jedes ENTER im Terminal (oder 'g'+ENTER) → veröffentlicht /pick/confirm (std_msgs/Empty).

# pick_tilt wartet zweimal auf Bestätigung: (1) im HOVER die Trajektorie in RViz prüfen→bestätigen,

# (2) nach dem Greifen "gehalten?"→bestätigen. Mit Ctrl+D / Ctrl+C beenden.

# =====

import sys

import rclpy

from rclpy.node import Node

from std_msgs.msg import Empty

def main():

    rclpy.init()

    n = Node("confirm_key")

    pub = n.create_publisher(Empty, "/pick/confirm", 1)

    print("[confirm_key] BEREIT. Pruefe die Trajektorie in RViz; zum BESTAETIGEN ENTER in diesem Terminal druecken.")

    print("[confirm_key] (pick_tilt wartet auf 2 Bestaetigungen: nach HOVER + nach dem Greifen.) Ctrl+C=beenden")

    try:

        for _ in sys.stdin:

            pub.publish(Empty())

            print("[confirm_key] ✓ /pick/confirm gesendet.")

    except KeyboardInterrupt:

        pass

    n.destroy_node()

    rclpy.shutdown()

if __name__ == "__main__":

    main()
```

cyclonedds.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<CycloneDDS xmlns="https://cdds.io/config" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="https://cdds.io/config https://raw.githubusercontent.com/eclipse-cyclonedds/cyclonedds/master/
etc/cyclonedds.xsd">

  <Domain>

    <General>

      <NetworkInterfaceAddress>lo</NetworkInterfaceAddress>

    </General>

    <Internal>

      <MinimumSocketReceiveBufferSize>10MB</MinimumSocketReceiveBufferSize>

    </Internal>

  </Domain>

</CycloneDDS>
```

fixture_corner.json

```
{
  "corners": {
    "sol_ust": {
      "q": [
        -1.051,
        -0.187,
        -2.503,
        1.141,
        0.063,
        2.939
      ],
      "tcp": [
        0.022612238107800983,
        -0.1564526730286088,
        -0.009185400933429816
      ]
    },
    "sag_alt": {
      "q": [
        -1.534,
        -0.798,
        -1.637,
        0.965,
        0.061,
        -0.713
      ],
      "tcp": [
        -0.04361613626237343,
        -0.22600154540886133,
        -0.021219875708968022
      ]
    }
  },
  "date": "2026-06-22"
}
```

go_to_zero.sh

```
#!/usr/bin/env bash

# Führt den echten Roboter in die Nullstellung: alle 6 Gelenke = 0 rad,
# Greifer OFFEN. send_radians lässt die Servos unter Drehmoment (kein Absturz
# beim Loslassen). Nutzt direkt pymycobot über /dev/ttyTHS1 – der ros2_control
# Stack darf dabei NICHT am selben Port hängen (Sim-Modus ist ok).

set -euo pipefail

export LC_ALL=C LC_NUMERIC=C

PORT="${MYCOBOT_PORT:-/dev/ttyTHS1}"
BAUD="${MYCOBOT_BAUD:-1000000}"
SPEED="${ZERO_SPEED:-30}"

python3 - "$PORT" "$BAUD" "$SPEED" <<'PY'

import sys, time

from pymycobot import MyCobot280

port, baud, speed = sys.argv[1], int(sys.argv[2]), int(sys.argv[3])

mc = MyCobot280(port, baud)

time.sleep(1.5)

print("aktuelle Winkel:", mc.get_angles())

mc.send_radians([0, 0, 0, 0, 0, 0], speed) # Nullstellung, Servos unter Drehmoment

time.sleep(3)

mc.set_gripper_value(100, 50, 1) # 100 = offen, adaptiver Greifer (Open-Loop)

time.sleep(2)

print("Nullstellung + Greifer offen fertig; Winkel:", mc.get_angles())

PY
```


measure_zero.py

```
#!/usr/bin/env python3

"""
measure_zero.py – Gelenk-Null-Wiederholbarkeit: Kalibrierungsdrift oder Servo-
Toleranz? Bei jedem Versuch fährt der Arm in eine kleine Pose und wird zurück auf 0
gesendet, der echte Encoder wird aus /joint_states gelesen. N Versuche → meldet die
Offset-Konsistenz jedes Gelenks.

Läuft über laufendes move_group/arm_controller (Port muss nicht frei sein).
"""

import math
import statistics as st
import rclpy

from rclpy.node import Node
from rclpy.action import ActionClient
from control_msgs.action import FollowJointTrajectory
from trajectory_msgs.msg import JointTrajectoryPoint
from sensor_msgs.msg import JointState
from builtin_interfaces.msg import Duration

ARM = ["joint2_to_joint1", "joint3_to_joint2", "joint4_to_joint3",
      "joint5_to_joint4", "joint6_to_joint5", "joint6output_to_joint6"]
SHORT = ["j1", "j2", "j3", "j4", "j5", "j6"]
WIGGLE = [0.20, -0.20, 0.20, -0.20, 0.20, -0.20]
ZERO = [0.0] * 6
TRIALS = 5


class M(Node):
    def __init__(self):
        super().__init__("measure_zero")

        self.cli = ActionClient(self, FollowJointTrajectory,
                                "/arm_controller/follow_joint_trajectory")

        self.latest = {}

        self.create_subscription(JointState, "/joint_states", self._cb, 10)

    def _cb(self, msg):
```

```

    for n, p in zip(msg.name, msg.position):
        self.latest[n] = p

def goto(self, pos, secs):
    g = FollowJointTrajectory.Goal()
    g.trajectory.joint_names = ARM
    pt = JointTrajectoryPoint()
    pt.positions = list(map(float, pos))
    pt.time_from_start = Duration(sec=secs)
    g.trajectory.points = [pt]
    self.cli.wait_for_server()
    f = self.cli.send_goal_async(g)
    rclpy.spin_until_future_complete(self, f)
    gh = f.result()
    if gh is None or not gh.accepted:
        self.get_logger().warn("Ziel abgelehnt"); return
    rf = gh.get_result_async()
    rclpy.spin_until_future_complete(self, rf)

def settle_read(self, ticks=50, stable_eps=0.15):
    # Langer Spin (~5s) + verifiziere, dass die letzte Messung STABIL ist (das Servo
    # ist offene Schleife und kann sich nach Ende der Action noch bewegen). Prüfe,
    # ob die letzten 10 Messungen innerhalb von eps liegen.
    hist = []
    for _ in range(ticks):
        rclpy.spin_once(self, timeout_sec=0.1)
        hist.append([math.degrees(self.latest.get(n, float("nan"))) for n in ARM])
    last = hist[-1]
    window = hist[-10:]
    moved = max(max(abs(w[i] - last[i]) for w in window) for i in range(6))
    stable = moved < stable_eps
    return last, stable

def main():
    rclpy.init()
    node = M()

```

```

if not node.cli.wait_for_server(timeout_sec=10.0):

    print("!! arm_controller Action nicht vorhanden – laeuft move_group/controller?")

    rclpy.shutdown(); return

rows = []

for t in range(1, TRIALS + 1):

    node.get_logger().info(f"Versuch {t}: wiggle → 0 ...")

    node.goto(WIGGLE, 3)

    wig, _ = node.settle_read(ticks=30)    # liveness: sollte ~+11.5° sehen

    node.goto(ZERO, 3)

    deg, stable = node.settle_read(ticks=50)

    rows.append(deg)

    flag = "" if stable else "  ▲ NICHT STABIL (uebersprungen)"

    if not stable:

        rows.pop()    # nicht gesetzte Messung aus der Zusammenfassung entfernen

    print(f" Versuch {t}: wiggle≈{wig[0]:+.1f}° | Null: " +

          " ".join(f"{SHORT[i]}={deg[i]:+.2f}" for i in range(6)) + flag)

print("\n=== ZUSAMMENFASSUNG (Grad) ===")

print(f"{'Gelenk':4} {'Mittel':>9} {'std(Streuung)':>13} {'min..max':>14}  Kommentar")

for i in range(6):

    col = [r[i] for r in rows]

    mean = st.mean(col)

    sd = st.pstdev(col)

    rng = max(col) - min(col)

    # konsistente Drift: |Mittel| groß ABER Streuung klein → Kalibrierung

    # zufällige Toleranz: Streuung groß verglichen mit |Mittel| → Servo-Toleranz

    if abs(mean) > 0.5 and sd < 0.3:

        note = "  KONSISTENTER Offset (Kalibrierungsdrift wahrscheinlich)"

    elif rng > 0.6:

        note = "  gestreut (Servo-Toleranz/Backlash)"

    else:

        note = "✓ nahe Null + stabil"

    print(f"{SHORT[i]:4} {mean:+9.2f} {sd:13.2f} {min(col):+6.2f}..{max(col):+.2f}  {note}")

node.destroy_node()

rclpy.shutdown()

```

```
if __name__ == "__main__":  
    main()
```

moveit_working.rviz

Panels:

- Class: rviz_common/Displays

Help Height: 78

Name: Displays

Property Tree Widget:

Expanded: null

Splitter Ratio: 0.5

Tree Height: 78

Visualization Manager:

Class: ''

Displays:

- Alpha: 0.5

Cell Size: 0.10000000149011612

Class: rviz_default_plugins/Grid

Color: 160; 160; 164

Enabled: true

Line Style:

Line Width: 0.029999999329447746

Value: Lines

Name: Grid

Normal Cell Count: 0

Offset:

X: 0

Y: 0

Z: 0

Plane: XY

Plane Cell Count: 10

Reference Frame: <Fixed Frame>

Value: true

- Acceleration_Scaling_Factor: 0.1

Class: moveit_rviz_plugin/MotionPlanning

Enabled: true

Move Group Namespace: ''

MoveIt_Allow_Approximate_IK: false

MoveIt_Allow_External_Program: false

MoveIt-Allow_Replanning: false

MoveIt-Allow_Sensor_Positioning: false

MoveIt_Planning_Attempts: 10

MoveIt_Planning_Time: 5

MoveIt_Use_Cartesian_Path: false

MoveIt_Use_Constraint_Aware_IK: true

MoveIt_Workspace:

Center:

X: 0

Y: 0

Z: 0

Size:

X: 2

Y: 2

Z: 2

Name: MotionPlanning

Planned Path:

Color Enabled: false

Interrupt Display: false

Links:

All Links Enabled: true

Expand Joint Details: false

Expand Link Details: false

Expand Tree: false

Link Tree Style: Links in Alphabetic Order

Loop Animation: false

Robot Alpha: 0.5

Robot Color: 150; 50; 150

Show Robot Collision: false

Show Robot Visual: true

Show Trail: false

State Display Time: 0.05 s

Trail Step Size: 1

Trajectory Topic: display_planned_path

Planning Metrics:

Payload: 1

Show Joint Torques: false

Show Manipulability: false

Show Manipulability Index: false

Show Weight Limit: false

TextHeight: 0.07999999821186066

Planning Request:

Colliding Link Color: 255; 0; 0

Goal State Alpha: 1

Goal State Color: 250; 128; 0

Interactive Marker Size: 0.029999999329447746

Joint Violation Color: 255; 0; 255

Planning Group: arm

Query Goal State: true

Query Start State: false

Show Workspace: false

Start State Alpha: 1

Start State Color: 0; 255; 0

Planning Scene Topic: monitored_planning_scene

Robot Description: robot_description

Scene Geometry:

Scene Alpha: 0.8999999761581421

Scene Color: 50; 230; 50

Scene Display Time: 9.999999747378752e-05

Show Scene Geometry: true

Voxel Coloring: Z-Axis

Voxel Rendering: Occupied Voxels

Scene Robot:

Attached Body Color: 150; 50; 150

Links:

All Links Enabled: true

Expand Joint Details: false

Expand Link Details: false

Expand Tree: false

Link Tree Style: Links in Alphabetic Order

Robot Alpha: 1

Show Robot Collision: false

Show Robot Visual: true

Value: true

Velocity_Scaling_Factor: 0.1

Enabled: true

Global Options:

Background Color: 48; 48; 48

Fixed Frame: robot_base

Frame Rate: 30

Name: root

Tools:

- Class: rviz_default_plugins/Interact
- Hide Inactive Objects: true
- Class: rviz_default_plugins/MoveCamera
- Class: rviz_default_plugins/Select
- Class: rviz_default_plugins/FocusCamera

Transformation:

Current:

Class: rviz_default_plugins/TF

Value: true

Views:

Current:

Class: rviz_default_plugins/Orbit

Distance: 0.5159747004508972

Enable Stereo Rendering:

Value: false

Focal Point:

X: 0

Y: 0

Z: 0.30000001192092896

Focal Shape Fixed Size: true

Focal Shape Size: 0.05000000074505806

Invert Z Axis: false

Name: Current View

Near Clip Distance: 0.009999999776482582

Pitch: 0.8799999952316284

Target Frame: <Fixed Frame>

Value: Orbit (rviz_default_plugins)

Yaw: 5.858185291290283

Saved: null

Window Geometry:

Displays:

collapsed: false

Height: 700

Hide Left Dock: false

Hide Right Dock: true

Width: 1024

X: 0

Y: 24

preview_reach.sh

```
#!/bin/bash

# =====

# preview_reach.sh – NUR BILD-Vorschau (RUEHRT DEN ROBOTER NICHT AN, KEIN IK).

#

# Zweck: ohne das pick-Skript zu starten sehen, WO HIN man die Schraube legen soll.

# Kamera + statisches TF (vo_rsp) + detector + cam_viewer werden geöffnet; Roboter/move_group/

# serieller Port (/dev/ttyTHS1) werden GAR NICHT genutzt → der Arm bewegt sich nicht, nichts wird geplant.

#

# Im Kamerafenster (linkes Panel /vida/overlay):

#   GRUENER Kreis = sichere Senkrecht-Griff-Zone (r < 200mm) → IK löst im ersten Versuch.

#   AMBER Kreis  = kinematisches Senkrecht-Limit (200-260mm) → erreichbar, aber langsam/angestrengt.

#   Neben jeder Schraube r=XXX (mm) + Ringfarbe: grün/amber/rot.

# Lege die Schraube so, dass ihr Ring GRUEN ist → belastet das IK nicht.

#

# Verwendung: cd ~/ros2_ws && ./preview_reach.sh      (Ctrl+C = beenden)

# =====

set -u

CONF="{CONF:-0.75}"

export LC_ALL=C LC_NUMERIC=C LANG=C PYTHONUNBUFFERED=1

WS="$HOME/ros2_ws"

STAMP="$(date +%Y%m%d_%H%M%S)"

LOGDIR="$WS/logs/preview_$STAMP"

mkdir -p "$LOGDIR"

CAM_LOG="$LOGDIR/camera.log"; TF_LOG="$LOGDIR/tf.log"

DET_LOG="$LOGDIR/detector.log"; CAMV_LOG="$LOGDIR/cam_viewer.log"

CAM_PID=""; TF_PID=""; DET_PID=""; CAMV_PID=""

log() { echo -e "\033[1;36m[preview]\033[0m $*"; }

warn() { echo -e "\033[1;33m[preview]\033[0m $*"; }

err() { echo -e "\033[1;31m[preview]\033[0m $*" >&2; }

kill_group() {

    local pid="$1" name="$2"


```

```

[ -z "$pid" ] && return 0

if kill -0 "$pid" 2>/dev/null; then

    log "$name wird beendet..."

    kill -TERM -- "$pid" 2>/dev/null

    for _ in $(seq 1 8); do kill -0 "$pid" 2>/dev/null || break; sleep 0.5; done

    kill -0 "$pid" 2>/dev/null && kill -KILL -- "$pid" 2>/dev/null

fi

}

cleanup() {

    [ "${_CLEANED:-0}" = "1" ] && return 0

    _CLEANED=1; trap - EXIT INT TERM; echo

    log "AUFRAEUMEN..."

    kill_group "$CAMV_PID" "cam_viewer"

    kill_group "$DET_PID" "detector"

    kill_group "$TF_PID" "vo_rsp(TF)"

    kill_group "$CAM_PID" "kamera"

    pkill -f "cam_viewer.py" 2>/dev/null

    pkill -f "vida_detector" 2>/dev/null

    pkill -f "robot_state_publisher" 2>/dev/null

    pkill -f "realsense2_camera" 2>/dev/null

    sleep 1

    pkill -9 -f "realsense2_camera" 2>/dev/null

    log "Logs: $LOGDIR"

}

trap cleanup EXIT INT TERM

# ---- D435i autosuspend-Ausgleich (damit die Kamera senden kann) ----

for dev in /sys/bus/usb/devices/*/idVendor; do

    [ "$(cat "$dev" 2>/dev/null)" = "8086" ] && \

        echo on | sudo tee "$(dirname "$dev")/power/control" >/dev/null 2>&1

done

set +u

source /opt/ros/galactic/setup.bash

source "$WS/install/setup.bash"

set -u

```

```

cd "$WS" || { err "ros2_ws nicht vorhanden"; exit 1; }

log "DER ROBOTER WIRD NICHT ANGERUEHRT – nur Kamera + TF + detector (KEIN IK/Bewegung)."
```

---- 1) Kamera (D435i, 424x240x15, aligned depth) ----

```

log "Kamera wird gestartet → $CAM_LOG"

setuid bash -c "exec ros2 launch realsense2_camera rs_launch.py \
    align_depth.enable:=true depth_module.profile:=424x240x15 \
    rgb_camera.profile:=424x240x15" >"$CAM_LOG" 2>&1 &

CAM_PID=$!
```

---- 2) Statisches TF (vo_rsp: Kamera↔robot_base, ohne Roboter) ----

```

log "Statisches TF (vo_rsp) wird gestartet → $TF_LOG"

setuid bash -c "exec ros2 launch '$WS/vo_rsp.launch.py'" >"$TF_LOG" 2>&1 &

TF_PID=$!
```

---- 3) Detector (YOLO + Reach-Kreis-Overlay) ----

```

log "vida_detector wird gestartet → $DET_LOG"

setuid bash -c "exec ros2 run vida_vision vida_detector --ros-args -p conf:=$CONF" >"$DET_LOG" 2>&1 &

DET_PID=$!
```

log "Auf Modell warten ('Modell bereit', max 120s)..."

```

ok=0

for i in $(seq 1 120); do

    kill -0 "$DET_PID" 2>/dev/null || { err "Detector zu frueh gestorben:"; tail -n 25 "$DET_LOG"; exit 1; }

    grep -q "Modell bereit" "$DET_LOG" 2>/dev/null && { ok=1; log "Modell geladen (${i}s)"; break; }

    sleep 1

done

[ "$ok" = "1" ] || { err "Modell nicht geladen. Siehe $DET_LOG"; exit 1; }
```

---- 4) Kamerafenster ----

```

if [ -n "${DISPLAY:-}" ]; then

    log "cam_viewer wird geoeffnet (linkes Panel = Reach-Kreise + Erkennung) → $CAMV_LOG"

    setuid bash -c "exec python3 '$WS/cam_viewer.py'" >"$CAMV_LOG" 2>&1 &

    CAMV_PID=$!

else

    warn "Kein DISPLAY – Fenster konnte nicht geoeffnet werden. Overlay-Topic: /vida/overlay"
```

```
fi
```

```
log "=====
```

```
log " VORSCHAU BEREIT. Die Schraube naeher zum Roboter bringen, bis ihr Ring GRUEN ist."
```

```
log " GRUEN = belastet IK nicht | AMBER = angestrengt | ROT = nicht erreichbar"
```

```
log " Beenden: Ctrl+C"
```

```
log "=====
```

```
# Detector-Log live anzeigen (Ctrl+C = alles beenden)
```

```
tail -n +1 -F "$DET_LOG" --pid="$DET_PID"
```

reach_rings.py

```
#!/usr/bin/env python3

"""

reach_rings.py – Reichweite-Grenzen des Roboters in RViz als Ring-Marker zeigen.

RViz-Äquivalent des Donuts (grün/amber/rot) im Detector-Overlay: horizontale
Kreise in der robot_base-Achse. Horizontal r = zeigt die hypot(x,y)-Bänder.


    red  r=0.15 : innere Grenze – darunter klappt der Arm ein/stößt an die Basis
    green r=0.18 : ideales Pick-Zentrum
    green r=0.20 : außerhalb des grünen Bandes (Reichweite-Kante)
    amber r=0.26 : absolutes Max (im freien Raum; bei Plattform top-down weniger)


Verwendung:  source /opt/ros/galactic/setup.bash && source install/setup.bash

              python3 reach_rings.py

In RViz:      Add → MarkerArray → Topic: /reach_rings

"""

import math

import rclpy

from rclpy.node import Node

from rclpy.qos import QoSProfile, DurabilityPolicy

from visualization_msgs.msg import Marker, MarkerArray

from geometry_msgs.msg import Point

FRAME = "robot_base"

Z = 0.002 # direkt über der Tischebene


# (id, Radius m, (r,g,b), Beschriftung)

RINGS = [

    (0, 0.15, (0.90, 0.10, 0.10), "innen 15cm"),

    (1, 0.18, (0.10, 0.90, 0.10), "ideal 18cm"),

    (2, 0.20, (0.10, 0.80, 0.10), "gruen 20cm"),

    (3, 0.26, (1.00, 0.65, 0.00), "max 26cm"),

]


def circle(mid, r, rgb, width=0.004, npts=96):
```

```

m = Marker()

m.header.frame_id = FRAME

m.ns = "reach_ring"

m.id = mid

m.type = Marker.LINE_STRIP

m.action = Marker.ADD

m.scale.x = width

m.color.r, m.color.g, m.color.b = rgb

m.color.a = 0.9

m.pose.orientation.w = 1.0

for i in range(npts + 1):

    a = 2.0 * math.pi * i / npts

    p = Point()

    p.x = r * math.cos(a)

    p.y = r * math.sin(a)

    p.z = Z

    m.points.append(p)

return m

```

```

def label(mid, r, rgb, text):

    m = Marker()

    m.header.frame_id = FRAME

    m.ns = "reach_label"

    m.id = mid

    m.type = Marker.TEXT_VIEW_FACING

    m.action = Marker.ADD

    m.scale.z = 0.018

    m.color.r, m.color.g, m.color.b = rgb

    m.color.a = 0.95

    m.pose.position.x = r * 0.707

    m.pose.position.y = -r * 0.707 # in den vorderen-rechten Quadranten schreiben

    m.pose.position.z = Z + 0.005

    m.pose.orientation.w = 1.0

    m.text = text

    return m

```

```

def main():

    rclpy.init()

    node = Node("reach_rings")

    qos = QoSProfile(depth=1)

    qos.durability = DurabilityPolicy.TRANSIENT_LOCAL # latched → spät beitretendes RViz bekommt es

    pub = node.create_publisher(MarkerArray, "/reach_rings", qos)

    arr = MarkerArray()

    for mid, r, rgb, txt in RINGS:

        arr.markers.append(circle(mid, r, rgb))

        arr.markers.append(label(100 + mid, r, rgb, txt))

    def tick():

        for mk in arr.markers:

            mk.header.stamp = node.get_clock().now().to_msg()

        pub.publish(arr)

    tick()

    node.create_timer(2.0, tick) # alle 2s auffrischen (für neue Abonnenten)

    node.get_logger().info(f"reach_rings aktiv → /reach_rings ({len(RINGS)} Ringe, frame={FRAME})")

    try:

        rclpy.spin(node)

    except KeyboardInterrupt:

        pass

    finally:

        node.destroy_node()

        rclpy.shutdown()

if __name__ == "__main__":

    main()

```


record_cam_publisher.py

```
#!/usr/bin/env python3

# record_cam_publisher.py – die USB-2.0-Kamera neben der Plattform (/dev/video3)
# auf ein ROS-Image-Topic veröffentlichen. Zweck: Aufnahme + Beobachtung in RViz,
# während der Roboter die Schraube hält.
#
# Kamera 90° seitlich montiert → Korrektur per ROTATE env (cw/ccw/180/none).
# Ohne cv_bridge (wie das vida_detector-Muster), in Galactic wenig Abhängigkeiten.
#
# Verwendung (separates Terminal):
#   source /opt/ros/galactic/setup.bash && source ~/ros2_ws/install/setup.bash
#   python3 ~/ros2_ws/record_cam_publisher.py
#   # Parameter per env: DEV, WIDTH, HEIGHT, FPS, ROTATE, TOPIC

import os
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import Header
import numpy as np
import cv2

DEV      = int(os.environ.get("REC_DEV", "3"))          # /dev/video3
WIDTH    = int(os.environ.get("REC_WIDTH", "640"))
HEIGHT   = int(os.environ.get("REC_HEIGHT", "480"))
FPS       = float(os.environ.get("REC_FPS", "15"))
ROTATE   = os.environ.get("REC_ROTATE", "cw").lower()  # cw / ccw / 180 / none
TOPIC    = os.environ.get("REC_TOPIC", "/record_cam/image_raw")
FRAME    = os.environ.get("REC_FRAME", "record_cam")

_ROT = {
    "cw": cv2.ROTATE_90_CLOCKWISE,
    "ccw": cv2.ROTATE_90_COUNTERCLOCKWISE,
    "180": cv2.ROTATE_180,
}
```

```

def bgr_to_image(img, frame_id):

    msg = Image()

    msg.header = Header()

    msg.header.frame_id = frame_id

    msg.height, msg.width = img.shape[:2]

    msg.encoding = "bgr8"

    msg.is_bigendian = 0

    msg.step = msg.width * 3

    msg.data = np.ascontiguousarray(img).tobytes()

    return msg


class RecordCam(Node):

    def __init__(self):

        super().__init__("record_cam_publisher")

        self.pub = self.create_publisher(Image, TOPIC, 1)

        self.cap = cv2.VideoCapture(DEV, cv2.CAP_V4L2)

        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, WIDTH)

        self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, HEIGHT)

        try:

            self.cap.set(cv2.CAP_PROP_AUTO_EXPOSURE, 3) # UVC auto

        except Exception:

            pass

        if not self.cap.isOpened():

            self.get_logger().error(f"/dev/video{DEV} konnte nicht geoeffnet werden!")

        else:

            self.get_logger().info(

                f"/dev/video{DEV} geoeffnet -> {TOPIC} ({WIDTH}x{HEIGHT}@{FPS} rot={ROTATE})")

            # ein paar Frames verwerfen, damit sich die Auto-Belichtung einpegelt

            for _ in range(10):

                self.cap.read()

            self.timer = self.create_timer(1.0 / max(FPS, 1.0), self.tick)

            self._warned = False

    def tick(self):

        ok, frame = self.cap.read()

        if not ok or frame is None:

```

```

        if not self._warned:

            self.get_logger().warn("kein Frame empfangen", throttle_duration_sec=5.0)

            self._warned = True

        return

    if ROTATE in _ROT:

        frame = cv2.rotate(frame, _ROT[ROTATE])

    self.pub.publish(bgr_to_image(frame, FRAME))

def destroy_node(self):

    try:

        self.cap.release()

    except Exception:

        pass

    super().destroy_node()

def main():

    rclpy.init()

    node = RecordCam()

    try:

        rclpy.spin(node)

    except KeyboardInterrupt:

        pass

    finally:

        node.destroy_node()

        if rclpy.ok():

            rclpy.shutdown()

if __name__ == "__main__":

    main()

```

run_approach.sh

```
#!/bin/bash

# =====

# run_approach.sh – ANNAEHERUNG an Ziel (KEIN RViz – nur Kamerafenster + Terminal).

#

# Benutzerwunsch (2026-06-07, aktuell):

#   - KEIN RViz (RViz+YOL0+Kamera gleichzeitig füllten den Nano-RAM und froren ihn ein,
#     Erkennung war nicht möglich). Nur ein cv2-Fenster mit 2 Kameras + dieses Terminal reicht.
#   - Im Kamerafenster ist der GEWAHLTE Punkt sichtbar (linkes Panel = YOLO-Overlay /vida/overlay,
#     mit "ZIEL" markiert). Rechtes Panel = Aufnahmekamera.
#   - SCHRITT 1: Roboter startet bei Punkt 0 (home, alle Gelenke 0).
#   - SCHRITT 2: bringt den Greifer GENAU UEBER das Ziel, 90° SENKRECHT, 10cm höher, OEFFNET, wartet.
#   - Der Roboter steht physisch senkrecht über diesem Punkt (am echten Roboter beobachtet).
#   - BEIM START und BEIM BEENDEN parkt der Roboter zu Punkt 0 (beim Beenden bevor der Stack
#     stirbt → auch wenn die Servos frei werden, FAELLT der Arm NICHT).

#

# Ablauf:

#   1) Systemvorbereitung (locale, rmem, USB power, GPU fan) [fragt sudo]
#   2) Stack: demo.launch.py use_camera:=true use_rviz:=false (KEIN RViz)
#   3) record_cam_publisher.py (2. Kamera → /record_cam/image_raw) + cam_viewer.py
#   4) vida_detector (YOLO) – auf "Modell bereit" + erstes "ZIEL" warten
#   5) approach_target AUSFUEHREN (0 → 10cm über Ziel, Greifer auf, warten)
#     wenn das Ziel gelatcht ist, wird der DETECTOR beendet (Roboterbewegung + YOLO gleichzeitig = RAM-thrash)
#   6) Kamerafenster + Terminal bleiben offen; zum Beenden Ctrl+C → komplettes Aufräumen

#

# Verwendung: cd ~/ros2_ws && ./run_approach.sh

#   APPROACH_HEIGHT=0.10 ./run_approach.sh   # Annäherungshöhe (m)

#   SIM=1 ./run_approach.sh                  # Simulation ohne Roboter

# Logs: ~/ros2_ws/logs/approach_<zeit>/

# =====

set -u

APPROACH_HEIGHT="${APPROACH_HEIGHT:-0.10}"

SIM="${SIM:-0}"

DETECTOR_WAIT="${DETECTOR_WAIT:-120}"

CTRL_WAIT="${CTRL_WAIT:-60}"
```

```

export LC_ALL=C

export LC_NUMERIC=C

export LANG=C


WS="$HOME/ros2_ws"

STAMP="$(date +%Y%m%d_%H%M%S)"

LOGDIR="$WS/logs/approach_$STAMP"

mkdir -p "$LOGDIR"


STACK_LOG="$LOGDIR/stack.log"

REC_LOG="$LOGDIR/record_cam.log"

CAMV_LOG="$LOGDIR/cam_viewer.log"

DET_LOG="$LOGDIR/detector.log"

NODE_LOG="$LOGDIR/approach.log"


STACK_PID=""

REC_PID=""

CAMV_PID=""

DET_PID=""

NODE_PID=""


log() { echo -e "\033[1;36m[run_approach]\033[0m $*"; }

warn() { echo -e "\033[1;33m[run_approach]\033[0m $*"; }

err() { echo -e "\033[1;31m[run_approach]\033[0m $*" >&2; }


kill_group() {
    local pid="$1" name="$2"

    [ -z "$pid" ] && return 0

    if kill -0 "$pid" 2>/dev/null; then
        log "$name wird beendet (pgid=$pid)..."

        kill -TERM -- "$pid" 2>/dev/null

        for _ in $(seq 1 10); do kill -0 "$pid" 2>/dev/null || break; sleep 0.5; done

        kill -0 "$pid" 2>/dev/null && { warn "$name erzwungen (KILL)"; kill -KILL -- "$pid" 2>/dev/null; }
    fi
}

```

```
# Verwaiste mycobot_bridge.py + veralteter Socket = segfault beim nächsten Lauf. Aufkehren.
```

```
sweep_ros_ghosts() {  
    pkill -f "record_cam_publisher" 2>/dev/null  
    pkill -f "cam_viewer.py" 2>/dev/null  
    pkill -f "vida_detector" 2>/dev/null  
    pkill -f "approach_target" 2>/dev/null  
    pkill -f "approach_target.launch.py" 2>/dev/null  
    pkill -f "demo.launch.py" 2>/dev/null  
    pkill -f "rviz2" 2>/dev/null  
    pkill -f "move_group" 2>/dev/null  
    pkill -f "ros2_control_node" 2>/dev/null  
    pkill -f "mycobot_bridge.py" 2>/dev/null  
    pkill -f "realsense2_camera" 2>/dev/null  
    pkill -f "robot_state_publisher" 2>/dev/null  
    sleep 2  
    pkill -9 -f "mycobot_bridge.py" 2>/dev/null  
    pkill -9 -f "ros2_control_node" 2>/dev/null  
    pkill -9 -f "move_group" 2>/dev/null  
    pkill -9 -f "realsense2_camera" 2>/dev/null  
    pkill -9 -f "rviz2" 2>/dev/null  
    rm -f /tmp/mycobot_bridge.sock /tmp/mycobot_bridge.sock.cmd 2>/dev/null  
}
```

```
# Roboter über den arm_controller zu Punkt 0 (alle Gelenke 0) senden. Der Stack
```

```
# (ros2_control + arm_controller) muss AKTIV sein. Damit beim Beenden die Servos nicht frei werden
```

```
# und der Arm fällt, wird ZUERST zu 0 geparkt, DANN der Stack getötet.
```

```
ARM_JOINTS="[joint2_to_joint1, joint3_to_joint2, joint4_to_joint3, joint5_to_joint4, joint6_to_joint5,  
joint6output_to_joint6]"
```

```
goto_zero() {  
    local secs="${1:-5}"  
    command -v ros2 >/dev/null 2>&1 || return 0  
    if ros2 action list 2>/dev/null | grep -q "/arm_controller/follow_joint_trajectory"; then  
        log "Roboter wird zu Punkt 0 gesendet (${secs}s, Servo soll nicht frei werden)..."  
        timeout 20 ros2 action send_goal /arm_controller/follow_joint_trajectory \  
            control_msgs/action/FollowJointTrajectory \  
            "{trajectory: {joint_names: $ARM_JOINTS, points: [{positions: [0.0,0.0,0.0,0.0,0.0,0.0], time_from_start:  
{sec: $secs}}]}"}" \  
            >/dev/null 2>&1 \  
    fi  
}
```

```

    && log "Roboter bei Punkt 0." || warn "Senden zu 0 nicht abgeschlossen (trotzdem weiter)."
```

else

```

    warn "arm_controller action nicht vorhanden – Roboter kann nicht zu 0 gesendet werden."
```

fi

```

}
```

Nachdem Stack/bridge beendet sind, mit pymycobot GARANTIERT zu 0 parken + Servos UNTER DREHMOMENT lassen.

Warum: action-basiertes goto_zero ist unzuverlässig – beim Beenden gibt mycobot_bridge die Servos

frei und LAESST den Roboter FALLEN (Lektion Sitzung 30). Diese Funktion öffnet den Port direkt,

sendet zu 0 und lässt die Servos energetisiert (haltend) → der Arm bleibt bei 0 hängen, fällt nicht.

```

park_zero_pymycobot() {
    [ "${SIM:-0}" = "1" ] && return 0
    local port="${MYCOBOT_PORT:-/dev/ttyTHS1}"
    for _ in $(seq 1 10); do fuser "$port" >/dev/null 2>&1 || break; sleep 0.5; done
    if fuser "$port" >/dev/null 2>&1; then
        warn "pymycobot Park: $port noch belegt – wird uebersprungen (Arm kann fallen, von Hand kontrollieren!);"
        return 0
    fi
    log "GARANTIERTER Park zu 0 mit pymycobot (Servos bleiben unter Drehmoment, Arm faellt nicht)..."
    if MYCOBOT_PORT="$port" timeout 35 python3 - <<'PYEOF'
import os, time
try:
    from pymycobot import MyCobot280
    mc = MyCobot280(os.environ.get("MYCOBOT_PORT", "/dev/ttyTHS1"), 1000000)
    time.sleep(0.6)
    try:
        mc.power_on(); time.sleep(0.4)      # heruntergefallene/freie Servos neu energetisieren
    except Exception:
        pass
    mc.send_radians([0.0] * 6, 25)
    for _ in range(14):
        time.sleep(1.0)
        if mc.is_moving() == 0:
            break
    try:
        mc.set_gripper_value(100, 50, 1)  # beim Park Finger OFFEN (gehaltene Schraube loslassen)
        time.sleep(1.0)

```

```

except Exception as ge:

    print("Greifer-Oeffnen Warnung:", ge)

a = mc.get_radians()

if a and len(a) == 6:

    print("groesster |Winkel| nach Park = %.3f rad" % max(abs(x) for x in a))
except Exception as e:

    print("pymycobot Park FEHLER:", e)
PYEOF

then

    log "pymycobot Park beendet – Roboter bei 0, Servos halten."

else

    warn "pymycobot Park Timeout/fehlgeschlagen – Arm von Hand kontrollieren."

fi
}

cleanup() {

    echo

    log "AUFRAEUMEN – Hintergrund-nodes werden beendet..."

    # Erst die App-nodes stoppen (sie geben das Armkommando frei), DANN während der Stack noch aktiv ist
    # den Roboter zu 0 parken, GANZ ZULETZT den Stack töten → auch wenn die Servos frei werden, fällt der Arm nicht.

    kill_group "$NODE_PID" "approach node"

    kill_group "$DET_PID" "detector"

    kill_group "$CAMV_PID" "cam_viewer"

    kill_group "$REC_PID" "record_cam"

    goto_zero 5                                # Park beim Beenden (Stack noch aktiv)

    kill_group "$STACK_PID" "stack"

    sweep_ros_ghosts

    if fuser /dev/ttyTHS1 >/dev/null 2>&1; then

        warn "/dev/ttyTHS1 NOCH belegt – erzwungen toeten"; fuser -k -9 /dev/ttyTHS1 2>/dev/null

    fi

    park_zero_pymycobot                        # GARANTierter Park zu 0 (Servos unter Drehmoment, Arm faellt nicht)

    log "Logs: $LOGDIR"

}

trap cleanup EXIT INT TERM

# ---- 1) Systemvorbereitung ----

log "Systemvorbereitung (sudo evtl. noetig)..."

```



```

sudo sysctl -w net.core.rmem_max=8388608 net.core.wmem_max=8388608 \
    net.core.rmem_default=8388608 net.core.wmem_default=8388608 >/dev/null 2>&1
echo 0 | sudo tee /sys/devices/57000000.gpu/railgate_enable >/dev/null 2>&1
sudo sh -c 'echo 255 > /sys/devices/pwm-fan/target_pwm' 2>/dev/null
for dev in /sys/bus/usb/devices/*/idVendor; do
    [ "$(cat "$dev" 2>/dev/null)" = "8086" ] && \
        echo on | sudo tee "$(dirname "$dev")/power/control" >/dev/null 2>&1
done

sweep_ros_ghosts
if fuser /dev/ttyTHS1 >/dev/null 2>&1; then
    warn "/dev/ttyTHS1 vom vorherigen Lauf belegt – wird erzwungen freigegeben"
    fuser -k -9 /dev/ttyTHS1 2>/dev/null; sleep 1
fi

log "KEIN RViz – um den RAM zu entlasten nur Kamerafenster + Terminal."

# ---- ROS-Umgebung ----
set +u
source /opt/ros/galactic/setup.bash
source "$WS/install/setup.bash"
set -u
cd "$WS" || { err "ros2_ws nicht vorhanden"; exit 1; }

if [ "$SIM" = "1" ]; then
    export USE_FAKE_HARDWARE="true"; warn "SIM MODE – fake_components (kein Roboter)"
else
    export USE_FAKE_HARDWARE="false"; log "ECHTER Roboter @ ${MYCOBOT_PORT:-/dev/ttyTHS1}"
fi

# ---- 2) Stack (KEIN RViz + Kamera) ----
log "Stack wird gestartet (use_camera:=true use_rviz:=false) → $STACK_LOG"
setsid bash -c "exec ros2 launch mycobot_moveit_config demo.launch.py \
    use_camera:=true use_rviz:=false" >"$STACK_LOG" 2>&1 &
STACK_PID=$!

# ---- 3) Warten bis Controller active sind ----

```

```

log "Auf Controller warten (max ${CTRL_WAIT}s)..."

ok=0

for i in $(seq 1 "${CTRL_WAIT}"); do

    kill -0 "$STACK_PID" 2>/dev/null || { err "Stack zu frueh gestorben! Letzte Zeilen:"; tail -n 25 "$STACK_LOG";
exit 1; }

    if ros2 control list_controllers 2>/dev/null | grep -q "arm_controller.*active"; then

        ok=1; log "Controller AKTIV (${i}s)"; break

    fi

    sleep 1
done

[ "$ok" = "1" ] || { err "Controller wurde nicht aktiv innerhalb ${CTRL_WAIT}s. Siehe $STACK_LOG"; exit 1; }

# ---- 3a) START-PARK – Roboter soll sofort zu Punkt 0 gehen ----
goto_zero 5

# ---- 3b) 2. Kamera-Publisher (Aufnahmekamera /dev/video3) ----
log "record_cam_publisher (2. Kamera) wird gestartet → $REC_LOG"
setsid bash -c "exec python3 '$WS/record_cam_publisher.py' " >"$REC_LOG" 2>&1 &
REC_PID=$!

# ---- 3b2) Kamera-Viewer (2 Kameras in einem cv2-Fenster) ----
# Linkes Panel = /vida/overlay (YOLO-Erkennung + GEWAELTER Punkt mit "ZIEL" markiert),
# rechtes Panel = Aufnahmekamera. (Sobald der Detector startet, füllt sich das Overlay.)
if [ -n "${DISPLAY:-}" ]; then

    log "cam_viewer (2-Kamera-Fenster) wird gestartet → $CAMV_LOG"

    setsid bash -c "exec python3 '$WS/cam_viewer.py' " >"$CAMV_LOG" 2>&1 &

    CAMV_PID=$!

else

    warn "Kein DISPLAY – Kamerafenster uebersprungen."

fi

# ---- 4) Detector (YOLO) starten, auf Modell + erstes ZIEL warten ----
log "vida_detector wird gestartet → $DET_LOG"

# Reach-Kreise (grün=200mm sicher, amber=260mm Senkrecht-Limit) sind jetzt in vida_detector_node.py

# als PERMANENTER Standard – werden immer im Overlay gezeichnet, kein extra -p nötig.

setsid bash -c "exec ros2 run vida_vision vida_detector" >"$DET_LOG" 2>&1 &
DET_PID=$!

```

```

log "Auf Detector-Bereitschaft warten: 'Modell bereit' (max ${DETECTOR_WAIT}s)..."

ok=0

for i in $(seq 1 "$DETECTOR_WAIT"); do

    kill -0 "$DET_PID" 2>/dev/null || { err "Detector zu frueh gestorben! Letzte Zeilen:"; tail -n 25 "$DET_LOG";
exit 1; }

    grep -q "Modell bereit" "$DET_LOG" 2>/dev/null && { ok=1; log "Modell geladen (${i}s)"; break; }

    sleep 1

done

[ "$ok" = "1" ] || { err "Modell nicht geladen innerhalb ${DETECTOR_WAIT}s. Siehe $DET_LOG"; exit 1; }


log "Auf erstes Schrauben-Ziel ('ZIEL') warten (max 60s)..."

ok=0

for i in $(seq 1 60); do

    kill -0 "$DET_PID" 2>/dev/null || { err "Detector gestorben! Siehe $DET_LOG"; exit 1; }

    if grep -q "ZIEL" "$DET_LOG" 2>/dev/null; then

        ok=1; log "Schraube gefunden → $(grep 'ZIEL' "$DET_LOG" | tail -n1)"; break

    fi

    [ $((i % 10)) -eq 0 ] && warn " ...noch kein Ziel (${i}s). Letzte: $(tail -n1 "$DET_LOG")"

    sleep 1

done

[ "$ok" = "1" ] || warn "Innerhalb 60s kein Ziel – wird trotzdem versucht (node hat 30s-Timeout)."
```

---- 5) approach_target AUSFUEHREN (im Hintergrund) ----

```

log "=====
```

log " ANNAEHERUNG wird ausgefuehrt (0 → \${APPROACH_HEIGHT}m ueber Ziel, Greifer senkrecht+offen)"

log " Detector bleibt AN → im linken Kamera-Panel ist der gewaehlte Punkt (ZIEL) live sichtbar."

log " Der Roboter steht physisch 10cm ueber dem Ziel 90° senkrecht und wartet."

```

log "=====
```

```

export APPROACH_HEIGHT

: > "$NODE_LOG"    # tail-Race verhindern – Datei vorab anlegen (sonst beendet sich tail sofort und schliesst alles)

setsid bash -c "exec ros2 launch mycobot_demo approach_target.launch.py" >>"$NODE_LOG" 2>&1 &

NODE_PID=$!
```

HINWEIS: detector wird JETZT NICHT MEHR GETOETET – /vida/overlay soll live bleiben, damit im linken

Kamera-Panel der gewählte Punkt ständig sichtbar ist (Benutzerpriorität). Da RViz weg ist,

ist der RAM ohnehin entspannt. Wenn der Roboter nicht erreicht, die Schraube näher zum Roboter legen (grüne Zone).

```
# Node-Ausgabe live anzeigen; der node wartet bis Ctrl+C (Roboter steht über dem Ziel).
```

```
log "Node-Ausgabe (Ctrl+C = alles beenden):"
```

```
tail -n +1 -F "$NODE_LOG" --pid="$NODE_PID"
```

```
# Hierher kommt man, wenn der node endet (Ctrl+C) → cleanup trap läuft.
```

```
log "Wird beendet..."
```

run_approach_test.sh

```
#!/bin/bash

# =====

# run_approach_test.sh – Schrauben-ANNAEHERUNGSTEST (nur pre-grasp, KEIN Abstieg).

# Identische Orchestrierung wie run_pick_test.sh; einziger Unterschied: statt pick_place

# läuft approach_target → der Roboter WARTET APPROACH_HEIGHT UEBER der Schraube,

# Greifer 90° senkrecht (top-down), Greifer offen. Steht bis Ctrl+C.

#

# Zweck: per AUGES prüfen, ob der Greifer genau über der Schraube + genau 90° senkrecht ist.

#

# Verwendung:

#   cd ~/ros2_ws && ./run_approach_test.sh           # 5 cm darüber (Standard)

#   APPROACH_HEIGHT=0.10 ./run_approach_test.sh      # 10 cm darüber

#   SIM=1 ./run_approach_test.sh                    # Simulation ohne Roboter

#

# Bei Position PRUEFEN; danach IM TERMINAL Ctrl+C → alles wird aufgeräumt.

# Logs: ~/ros2_ws/logs/approach_<zeit>/{stack,detector,approach}.log

# =====

set -u

# ---- Einstellbare Parameter -----

APPROACH_HEIGHT="${APPROACH_HEIGHT:-0.05}" # wie viele m ueber der Schraube er stehen wird

SIM="${SIM:-0}"

VIEW="${VIEW:-1}"

DETECTOR_WAIT="${DETECTOR_WAIT:-120}"

CTRL_WAIT="${CTRL_WAIT:-60}"

export LC_ALL=C

export LC_NUMERIC=C

export LANG=C

WS="$HOME/ros2_ws"

STAMP="$(date +%Y%m%d_%H%M%S)"

LOGDIR="$WS/logs/approach_$STAMP"

mkdir -p "$LOGDIR"
```

```

STACK_LOG="$LOGDIR/stack.log"

DET_LOG="$LOGDIR/detector.log"

APP_LOG="$LOGDIR/approach.log"


STACK_PID=""

DET_PID=""

VIEW_PID=""


log() { echo -e "\033[1;36m[run_approach]\033[0m $*"; }

warn() { echo -e "\033[1;33m[run_approach]\033[0m $*"; }

err() { echo -e "\033[1;31m[run_approach]\033[0m $*" >&2; }


kill_group() {
    local pid="$1" name="$2"

    [ -z "$pid" ] && return 0

    if kill -0 "$pid" 2>/dev/null; then
        log "$name wird beendet (pgid=$pid)..."

        kill -TERM -- "$pid" 2>/dev/null

        for _ in $(seq 1 10); do kill -0 "$pid" 2>/dev/null || break; sleep 0.5; done

        kill -0 "$pid" 2>/dev/null && { warn "$name erzwungen (KILL)"; kill -KILL -- "$pid" 2>/dev/null; }
    fi
}


sweep_ros_ghosts() {
    pkill -f "view_overlay.py"          2>/dev/null
    pkill -f "vida_detector"           2>/dev/null
    pkill -f "approach_target"         2>/dev/null
    pkill -f "approach_target.launch"  2>/dev/null
    pkill -f "pick_place_cartesian"    2>/dev/null
    pkill -f "demo.launch.py"          2>/dev/null
    pkill -f "move_group"              2>/dev/null
    pkill -f "ros2_control_node"       2>/dev/null
    pkill -f "mycobot_bridge.py"       2>/dev/null
    pkill -f "realsense2_camera"       2>/dev/null
    pkill -f "robot_state_publisher"   2>/dev/null

    sleep 2

    pkill -9 -f "mycobot_bridge.py"    2>/dev/null

```

```

    pkill -9 -f "ros2_control_node"      2>/dev/null

    pkill -9 -f "move_group"             2>/dev/null

    pkill -9 -f "realsense2_camera"      2>/dev/null

    rm -f /tmp/mycobot_bridge.sock /tmp/mycobot_bridge.sock.cmd 2>/dev/null
}

cleanup() {
    echo

    log "AUFRAEUMEN – Hintergrund-nodes werden beendet..."

    kill_group "$VIEW_PID" "overlay-viewer"

    kill_group "$DET_PID" "detector"

    kill_group "$STACK_PID" "stack"

    sweep_ros_ghosts

    if fuser /dev/ttyTHS1 >/dev/null 2>&1; then

        warn "/dev/ttyTHS1 NOCH belegt – verbleibenden Halter erzwungen toeten"

        fuser -k -9 /dev/ttyTHS1 2>/dev/null

    fi

    log "Logs: $LOGDIR"

    log " lesen: tail -n 80 $APP_LOG"
}

trap cleanup EXIT INT TERM

# ---- 1) Systemvorbereitung -----

log "Systemvorbereitung (sudo evtl. noetig)..."

sudo sysctl -w net.core.rmem_max=8388608 net.core.wmem_max=8388608 \

    net.core.rmem_default=8388608 net.core.wmem_default=8388608 >/dev/null 2>&1

echo 0 | sudo tee /sys/devices/57000000.gpu/railgate_enable >/dev/null 2>&1

sudo sh -c 'echo 255 > /sys/devices/pwm-fan/target_pwm' 2>/dev/null

for dev in /sys/bus/usb/devices/*/idVendor; do

    [ "$(cat "$dev" 2>/dev/null)" = "8086" ] && \

        echo on | sudo tee "$(dirname "$dev")/power/control" >/dev/null 2>&1

done

sweep_ros_ghosts

if fuser /dev/ttyTHS1 >/dev/null 2>&1; then

    warn "/dev/ttyTHS1 vom vorherigen Lauf belegt – wird erzwungen freigegeben"

    fuser -k -9 /dev/ttyTHS1 2>/dev/null; sleep 1

```

```

fi

set +u

source /opt/ros/galactic/setup.bash

source "$WS/install/setup.bash"

set -u

cd "$WS" || { err "ros2_ws nicht vorhanden"; exit 1; }

if [ "$SIM" = "1" ]; then

    export USE_FAKE_HARDWARE="true"

    warn "SIM MODE – fake_components (kein Roboter)"

else

    export USE_FAKE_HARDWARE="false"

    log "ECHTER Roboter @ ${MYCOBOT_PORT:-/dev/ttyTHS1}"

fi

# ---- 2) Stack starten (RViz AUS) -----

log "Stack wird gestartet (use_camera:=true use_rviz:=false) → $STACK_LOG"

setsid bash -c "exec ros2 launch mycobot_moveit_config demo.launch.py \

    use_camera:=true use_rviz:=false" >"$STACK_LOG" 2>&1 &

STACK_PID=$!

# ---- 3) Controller active -----

log "Auf Controller warten (max ${CTRL_WAIT}s)..."

ok=0

for i in $(seq 1 "$CTRL_WAIT"); do

    if ! kill -0 "$STACK_PID" 2>/dev/null; then

        err "Stack zu frueh gestorben! Letzte Zeilen:"; tail -n 25 "$STACK_LOG"; exit 1

    fi

    if ros2 control list_controllers 2>/dev/null | grep -q "arm_controller.*active"; then

        ok=1; log "Controller AKTIV (${i}s)"; break

    fi

    sleep 1

done

[ "$ok" = "1" ] || { err "Controller wurde nicht aktiv innerhalb ${CTRL_WAIT}s. Siehe $STACK_LOG"; exit 1; }

# ---- 4) Detector starten -----

```



```

log "vida_detector wird gestartet → $DET_LOG"

setsid bash -c "exec ros2 run vida_vision vida_detector" >"$DET_LOG" 2>&1 &

DET_PID=$!

log "Auf Detector-Bereitschaft warten: 'Modell bereit' (max ${DETECTOR_WAIT}s)..."

ok=0

for i in $(seq 1 "$DETECTOR_WAIT"); do

    kill -0 "$DET_PID" 2>/dev/null || { err "Detector zu frueh gestorben!"; tail -n 25 "$DET_LOG"; exit 1; }

    grep -q "Modell bereit" "$DET_LOG" 2>/dev/null && { ok=1; log "Modell geladen (${i}s)"; break; }

    sleep 1

done

[ "$ok" = "1" ] || { err "Modell nicht geladen innerhalb ${DETECTOR_WAIT}s. Siehe $DET_LOG"; exit 1; }

# ---- 4b) Live-Overlay-Fenster -----

if [ "$VIEW" = "1" ] && [ -n "${DISPLAY:-}" ]; then

    log "Overlay-Viewer wird geoeffnet (DISPLAY=$DISPLAY) – im Fenster 'q'/ESC = schliessen"

    setsid bash -c "exec python3 '$WS/view_overlay.py'" >"$LOGDIR/viewer.log" 2>&1 &

    VIEW_PID=$!

elif [ "$VIEW" = "1" ]; then

    warn "Kein DISPLAY – Overlay-Fenster uebersprungen."

fi

log "Auf erstes Schrauben-Ziel ('ZIEL') warten (max 60s)..."

ok=0

for i in $(seq 1 60); do

    kill -0 "$DET_PID" 2>/dev/null || { err "Detector gestorben!"; tail -n 25 "$DET_LOG"; exit 1; }

    if grep -q "ZIEL" "$DET_LOG" 2>/dev/null; then

        ok=1; log "Schraube gefunden → $(grep 'ZIEL' "$DET_LOG" | tail -n1)"; break

    fi

    [ $((i % 10)) -eq 0 ] && warn " ...noch kein Ziel (${i}s). Letzte: $(tail -n1 "$DET_LOG")"

    sleep 1

done

[ "$ok" = "1" ] || warn "Innerhalb 60s keine Schraube – approach wird trotzdem versucht (hat eigenen 30s-Timeout)."
```

---- 5) approach_target AUSFUEHREN (Roboter fährt zur Position, WARTET dort) --

```

log "=====
```

log " ANNAEHERUNG wird ausgefuehrt – \${APPROACH_HEIGHT}m (=\$(awk "BEGIN{printf \"%.0f\\", \$APPROACH_HEIGHT*100}")cm)

ueber der Schraube, 90° senkrecht"

log " Wenn der Roboter die Position erreicht, WARTET er DORT. Pruefen; danach Ctrl+C."

log " log: \$APP_LOG"

log "=====

export APPROACH_HEIGHT

ros2 launch mycobot_demo approach_target.launch.py 2>&1 | tee "\$APP_LOG"

Hierher kommt man nur bei Ctrl+C / wenn der node stirbt → cleanup trap läuft.

run_pick_preview.sh

```
#!/bin/bash

# =====

# run_pick_preview.sh – ZWEIPHASIGER pick: VISION → (detector aus) → RViz VORSCHAU.

#

# Idee (Benutzer, 2026-06-15): Auf dem Nano passen detector(2.1GB)+RViz nicht gleichzeitig
# in den RAM. Schraube STATISCH → Koordinate einmal erfassen und SPEICHERN, detector+Kamera
# BEENDEN (RAM wird frei), RViz öffnen, gespeicherten Punkt markieren, in RViz die Trajektorie+
# KOLLISION (rot) sehen, bestätigen, echten Roboter ausführen.

#

# PHASE A (detector an, kein RViz): stack + Kamera + cam_viewer + detector.
#   Schraube hinlegen, Erkennung im Overlay stabilisieren → in cam_viewer/Terminal ENTER (capture).
# UEBERGANG: detector + cam_viewer + record_cam + Kamera BEENDEN (RAM wird frei).
# PHASE B (RViz an, kein detector): RViz + Ziel-replay (marker) + pick_tilt.
#   In RViz Trajektorie+Kollision prüfen → im Terminal ENTER = BESTAETIGUNG (x2).

#

# Verwendung: cd ~/ros2_ws && ./run_pick_preview.sh

#   APPROACH_HEIGHT=0.05 GRASP_Z_OFFSET=-0.02 ./run_pick_preview.sh

# Logs: ~/ros2_ws/logs/pick_preview_<zeit>/

# =====

set -u

APPROACH_HEIGHT="${APPROACH_HEIGHT:-0.10}"

# Standardwert des erfolgreichen pick-Rezepts (2026-06-15): TCP ~2.5cm überdefiniert → Pflaster.

# Override: GRASP_Z_OFFSET=-0.03 ./run_pick_preview.sh

GRASP_Z_OFFSET="${GRASP_Z_OFFSET:-0.025}"

# Modell-Laden (~15s) + ERSTE CUDA/cuDNN-Inferenz (~100s Warmup auf dem Nano) –
# der Detector meldet "Modell bereit" erst NACH dem Warmup (Sitzung 2026-07-16).

DETECTOR_WAIT="${DETECTOR_WAIT:-240}"

CTRL_WAIT="${CTRL_WAIT:-60}"

CAM_WAIT="${CAM_WAIT:-40}"

# Kamera-Profil: 848x480 statt 424x240 – schärferes Fenster + bessere Erkennung

# kleiner Wellen. OHNE pointcloud (Phase A hat kein RViz, niemand braucht sie –
# spart CPU/RAM, 720p+720p@15 ohne pointcloud ist auf dem Nano getestet).

# Farb- und Tiefenprofil IMMER gleich halten (aligned_depth bricht sonst).

CAM_PROFILE="${CAM_PROFILE:-848x480x15}"
```

```

# 0.85 → 0.70 (2026-07-16 Live-Test): 0.85 war zu streng – reale Pick-Erkennungen

# lagen bei conf 0.72-0.75, mit 0.85 fand der Detector NICHTS. 0.70 filtert

# Unsicheres, laesst echte Wellen durch. Altes Rezept: CONF=0.5 ./run_pick_preview.sh

CONF="${CONF:-0.70}"

MODE_CONSENSUS="${MODE_CONSENSUS:-3}"

STICKY_RADIUS="${STICKY_RADIUS:-0.05}"

# IK-Innengrenze (m): der ROTE innere Kreis im Overlay. Schrauben mit Base-XY-Radius

# kleiner als dieser sind dem Roboter ZU NAHE (beim Top-Down-Grasp überbiegt der Arm) → kein Ziel.

# Override: REACH_MIN=0.16 ./run_pick_preview.sh

REACH_MIN="${REACH_MIN:-0.15}"


# Vorschau-Ablauf: jede Bewegung in RViz anzeigen und ENTER-Bestätigung abwarten.

PREVIEW_CONFIRM="${PREVIEW_CONFIRM:-1}"

DESCEND_VEL_SCALE="${DESCEND_VEL_SCALE:-0.10}"

# Rapid-traverse-Geschwindigkeit für freien Raum (home/hover/Rückkehr-zu-Null) – Abstieg/Greifen NICHT BETROFFEN.

# 1.00 = volles Rabbit: bei langsam zittert der Servo stick-slip, schnelle Fahrt ist glatt (kein Kontakt, kein Risiko).

# Bei Ruckeln: mit RAPID_VEL_SCALE=0.85 ./run_pick_preview.sh zurücknehmen.

RAPID_VEL_SCALE="${RAPID_VEL_SCALE:-1.00}"

export LC_ALL=C LC_NUMERIC=C LANG=C PYTHONUNBUFFERED=1

# rcutils-Logs UNGEPUFFERT in die Log-Dateien schreiben – sonst bleiben z.B.

# estop_button-Meldungen im 4KB-stdout-Puffer stecken und gehen bei SIGKILL

# verloren (estop_button.log war beim Vorfall 2026-07-16 deshalb LEER).

export RCUTILS_LOGGING_BUFFERED_STREAM=0

export APPROACH_HEIGHT GRASP_Z_OFFSET PREVIEW_CONFIRM DESCEND_VEL_SCALE RAPID_VEL_SCALE


WS="$HOME/ros2_ws"

STAMP="$(date +%Y%m%d_%H%M%S)"

LOGDIR="$WS/logs/pick_preview_$STAMP"

mkdir -p "$LOGDIR"

TARGET_FILE="$WS/.vida_target_latch.json"

RVIZ_CFG="$WS/rviz_preview.rviz"


STACK_LOG="$LOGDIR/stack.log"; CAM_LOG="$LOGDIR/camera.log"

REC_LOG="$LOGDIR/record_cam.log"; CAMV_LOG="$LOGDIR/cam_viewer.log"

DET_LOG="$LOGDIR/detector.log"; CAP_LOG="$LOGDIR/capture.log"

RVIZ_LOG="$LOGDIR/rviz.log"; REPLAY_LOG="$LOGDIR/replay.log"; NODE_LOG="$LOGDIR/pick_tilt.log"

```

```
ESTOP_LOG="$LOGDIR/estop_button.log"
```

```
STACK_PID=""; CAM_PID=""; REC_PID=""; CAMV_PID=""; DET_PID=""
```

```
RVIZ_PID=""; REPLAY_PID=""; NODE_PID=""; TAIL_PID=""; ESTOP_PID=""
```

```
log() { echo -e "\033[1;36m[preview]\033[0m $*"; }
```

```
warn() { echo -e "\033[1;33m[preview]\033[0m $*"; }
```

```
err() { echo -e "\033[1;31m[preview]\033[0m $*" >&2; }
```

```
kill_group() {
```

```
    local pid="$1" name="$2"; [ -z "$pid" ] && return 0
```

```
    if kill -0 "$pid" 2>/dev/null; then
```

```
        log "$name wird beendet (pgid=$pid)..."
```

```
        kill -TERM -- "$pid" 2>/dev/null
```

```
        for _ in $(seq 1 10); do kill -0 "$pid" 2>/dev/null || break; sleep 0.5; done
```

```
        kill -0 "$pid" 2>/dev/null && { warn "$name erzwungen (KILL)"; kill -KILL -- "$pid" 2>/dev/null; }
```

```
    fi
```

```
}
```

```
sweep_ros_ghosts() {
```

```
    pkill -f "estop_button"          2>/dev/null
```

```
    pkill -f "estop_relay"           2>/dev/null
```

```
    pkill -f "record_cam_publisher"  2>/dev/null
```

```
    pkill -f "cam_viewer.py"         2>/dev/null
```

```
    pkill -f "target_latch.py"       2>/dev/null
```

```
    pkill -f "confirm_key.py"        2>/dev/null
```

```
    pkill -f "vida_detector"         2>/dev/null
```

```
    pkill -f "mycobot_demo/pick_tilt" 2>/dev/null
```

```
    pkill -f "pick_tilt.launch.py"   2>/dev/null
```

```
    pkill -f "demo.launch.py"        2>/dev/null
```

```
    pkill -f "rviz2"                 2>/dev/null
```

```
    pkill -f "move_group"            2>/dev/null
```

```
    pkill -f "ros2_control_node"     2>/dev/null
```

```
    pkill -f "mycobot_bridge.py"     2>/dev/null
```

```
    pkill -f "realsense2_camera"     2>/dev/null
```

```
    pkill -f "robot_state_publisher" 2>/dev/null
```

```
    sleep 2
```

```

    pkill -9 -f "mycobot_bridge.py" 2>/dev/null

    pkill -9 -f "ros2_control_node" 2>/dev/null

    pkill -9 -f "move_group" 2>/dev/null

    pkill -9 -f "realsense2_camera" 2>/dev/null

    pkill -9 -f "rviz2" 2>/dev/null

    pkill -9 -f "mycobot_demo/pick_tilt" 2>/dev/null

    rm -f /tmp/mycobot_bridge.sock /tmp/mycobot_bridge.sock.cmd 2>/dev/null
}

ARM_JOINTS="[joint2_to_joint1, joint3_to_joint2, joint4_to_joint3, joint5_to_joint4, joint6_to_joint5,
joint6output_to_joint6]"

goto_zero() {

    local secs="${1:-5}"; command -v ros2 >/dev/null 2>&1 || return 0

    if ros2 action list 2>/dev/null | grep -q "/arm_controller/follow_joint_trajectory"; then

        log "Roboter wird zu Punkt 0 gesendet (${secs}s)..."

        timeout 20 ros2 action send_goal /arm_controller/follow_joint_trajectory \

            control_msgs/action/FollowJointTrajectory \

            "{trajectory: {joint_names: $ARM_JOINTS, points: [{positions: [0.0,0.0,0.0,0.0,0.0,0.0], time_from_start:
{sec: $secs}}]}}" \

            >/dev/null 2>&1 && log "Roboter bei Punkt 0." || warn "Senden zu 0 nicht abgeschlossen."

    else

        warn "arm_controller action nicht vorhanden – Senden zu 0 nicht moeglich."

    fi
}

# Ist NOT-AUS gerade aktiv? /estop ist transient_local (gelatcht) – solange das
# estop_button-Fenster lebt (cleanup beendet es zuletzt), ist der Zustand lesbar.
# Rueckgabe 0 = STOP aktiv. Beim Vorfall 2026-07-16 fuhr das Aufraeumen (goto_zero +
# pymycobot-Park) trotz gedrucktem NOT-AUS weiter, weil relay+bridge schon tot
# waren – diese Pruefung schliesst die Luecke.

estop_active() {

    command -v ros2 >/dev/null 2>&1 || return 1

    local out

    out=$(timeout 5 ros2 topic echo --once \

        --qos-durability transient_local --qos-reliability reliable \

        /estop std_msgs/msg/Bool 2>/dev/null)

    [ "${out#*data: }" != "$out" ] && echo "$out" | grep -q "data: true"
}

```

```

park_zero_pymycobot() {
    local port="${MYCOBOT_PORT:-/dev/ttyTHS1}"

    if estop_active; then
        err "NOT-AUS AKTIV – pymycobot-Park wird NICHT ausgefuehrt (Arm bleibt stehen)."
        err "Zum Parken: im NOT-AUS-Fenster FREIGABE druecken und ~/ros2_ws/go_to_zero.sh nutzen."
        return 0
    fi

    for _ in $(seq 1 10); do fuser "$port" >/dev/null 2>&1 || break; sleep 0.5; done

    if fuser "$port" >/dev/null 2>&1; then
        warn "pymycobot Park: $port noch belegt – wird uebersprungen (Arm von Hand kontrollieren!); return 0
    fi

    log "GARANTierter Park zu 0 mit pymycobot (Servos unter Drehmoment, Arm faellt nicht)..."

    MYCOBOT_PORT="$port" timeout 35 python3 - <<'PYEOF'

import os, time

try:
    from pymycobot import MyCobot280

    mc = MyCobot280(os.environ.get("MYCOBOT_PORT", "/dev/ttyTHS1"), 1000000)

    time.sleep(0.6)

    try: mc.power_on(); time.sleep(0.4)

    except Exception: pass

    mc.send_radians([0.0]*6, 25)

    for _ in range(14):
        time.sleep(1.0)

        if mc.is_moving() == 0: break

    try:
        mc.set_gripper_value(100, 50, 1) # beim Park Finger OFFEN (gehaltene Schraube loslassen)

        time.sleep(1.0)

    except Exception as ge: print("Greifer-0effnen Warnung:", ge)

    a = mc.get_radians()

    if a and len(a) == 6:
        print("groesster |Winkel| nach Park = %.3f rad" % max(abs(x) for x in a))

except Exception as e:
    print("pymycobot Park FEHLER:", e)

PYEOF

log "pymycobot Park beendet – Roboter bei 0."
}

```

```

cleanup() {
    [ "${_CLEANED:-0}" = "1" ] && return 0
    _CLEANED=1; trap - EXIT INT TERM; echo
    log "AUFRAEUMEN – nodes werden beendet..."
    kill_group "$TAIL_PID"    "tail"
    kill_group "$NODE_PID"    "pick_tilt"
    kill_group "$REPLAY_PID"  "target_replay"
    kill_group "$RVIZ_PID"    "rviz"
    kill_group "$DET_PID"     "detector"
    kill_group "$CAMV_PID"    "cam_viewer"
    kill_group "$REC_PID"     "record_cam"
    kill_group "$CAM_PID"     "camera"

    if estop_active; then
        err "NOT-AUS AKTIV – goto_zero wird uebersprungen (kein automatisches Fahren!)"
    else
        goto_zero 5
    fi
    kill_group "$STACK_PID"   "stack"
    sweep_ros_ghosts
    fuser /dev/ttyTHS1 >/dev/null 2>&1 && fuser -k -9 /dev/ttyTHS1 2>/dev/null
    park_zero_pymycobot

    # NOT-AUS-Fenster: sweep hat es meist schon beendet; hier Rest-Aufräumen.
    kill_group "$ESTOP_PID"   "estop_button"

    log "Logs: $LOGDIR"
}

trap cleanup EXIT INT TERM

# ---- Systemvorbereitung ----

log "Systemvorbereitung (sudo evtl. noetig)..."

sudo sysctl -w net.core.rmem_max=8388608 net.core.wmem_max=8388608 \
    net.core.rmem_default=8388608 net.core.wmem_default=8388608 >/dev/null 2>&1

echo 0 | sudo tee /sys/devices/57000000.gpu/railgate_enable >/dev/null 2>&1

sudo sh -c 'echo 255 > /sys/devices/pwm-fan/target_pwm' 2>/dev/null

for dev in /sys/bus/usb/devices/*/idVendor; do
    [ "$(cat "$dev" 2>/dev/null)" = "8086" ] && \
        echo on | sudo tee "$(dirname "$dev")/power/control" >/dev/null 2>&1

```



```

done

sweep_ros_ghosts

fuser /dev/ttyTHS1 >/dev/null 2>&1 && { warn "Port belegt – wird freigegeben"; fuser -k -9 /dev/ttyTHS1
2>/dev/null; sleep 1; }

set +u

source /opt/ros/galactic/setup.bash

source "$WS/install/setup.bash"

set -u

cd "$WS" || { err "ros2_ws nicht vorhanden"; exit 1; }

export USE_FAKE_HARDWARE="${USE_FAKE_HARDWARE:-false}"

[ "$USE_FAKE_HARDWARE" = "true" ] && warn "FAKE HARDWARE (sim) – kein Roboter"

# =====

# PHASE A – VISION

# =====

log "##### PHASE A – VISION (detector an) #####"

log "Stack (move_group+control+bridge, OHNE KAMERA, KEIN RViz) → $STACK_LOG"

setsid bash -c "exec ros2 launch mycobot_moveit_config demo.launch.py \

    use_camera:=false use_rviz:=false use_fake_hardware:=$USE_FAKE_HARDWARE" >"$STACK_LOG" 2>&1 &

STACK_PID=$!

log "Auf Controller warten (max ${CTRL_WAIT}s)..."

ok=0

for i in $(seq 1 "$CTRL_WAIT"); do

    kill -0 "$STACK_PID" 2>/dev/null || { err "Stack zu frueh gestorben!"; tail -n 25 "$STACK_LOG"; exit 1; }

    ros2 control list_controllers 2>/dev/null | grep -q "arm_controller.*active" && { ok=1; log "Controller AKTIV ($

{i}s)"; break; }

    sleep 1

done

[ "$ok" = "1" ] || { err "Controller wurde nicht aktiv. Siehe $STACK_LOG"; exit 1; }

# NOT-AUS-Taster: eigenes, immer-im-Vordergrund-Fenster; publiziert /estop.

# Enforcement: estop_relay (startet demo.launch.py bei echter Hardware automatisch)

# leitet den Zustand an den seriellen Bridge weiter -> Bewegung stoppt SOFORT.

if [ -n "${DISPLAY:-}" ]; then

    log "NOT-AUS-Taster (estop_button) → $ESTOP_LOG"

    setsid bash -c "exec ros2 run mycobot_calibration estop_button" >"$ESTOP_LOG" 2>&1 &

```

```

    ESTOP_PID=$!

else

    warn "Kein DISPLAY – NOT-AUS-Fenster nicht moeglich. Terminal-Alternative:"

    warn "  ros2 topic pub --once --qos-durability transient_local /estop std_msgs/msg/Bool \"{data: true}\""

fi

goto_zero 5


# Detector FRUEH starten: Modell-Laden (~15s) + CUDA-Warmup (~100s) laufen dann
# PARALLEL zum Kamera-Start statt danach (Sitzung 2026-07-16: 3-4min → ~2min).
# Er wartet einfach mit "noch kein color"-Warnungen, bis die Kamera sendet.
# Wahrnehmungs-Offset (Lineal-Messung: Kamera liest ~+7/+11mm versetzt) aus
# .place_offset.json – DIESELBE Quelle wie click_place_moveit.py. Der Detector
# addiert die Korrektur auf das veroeffentlichte /vida/target (2026-07-16:
# vorher wurde die gemessene Korrektur im Pick-Pfad NIE angewendet).
read -r OFF_X OFF_Y OFF_Z <<< "$(python3 -c "
import json

try:

    d = json.load(open('$WS/.place_offset.json'))

    print(d.get('x', 0.0), d.get('y', 0.0), d.get('z', 0.0))

except Exception:

    print(0.0, 0.0, 0.0)")"

log "Wahrnehmungs-Offset: x=$OFF_X y=$OFF_Y z=$OFF_Z m (aus .place_offset.json)"


log "vida_detector VORAB gestartet (conf=$CONF, reach_min=${REACH_MIN}m) → $DET_LOG"

setuid bash -c "exec ros2 run vida_vision vida_detector --ros-args \

    -p conf:=$CONF -p mode_consensus:=$MODE_CONSENSUS -p sticky_radius:=$STICKY_RADIUS \

    -p reach_radius_min:=$REACH_MIN \

    -p offset_x:=$OFF_X -p offset_y:=$OFF_Y -p offset_z:=$OFF_Z" >"$DET_LOG" 2>&1 &

DET_PID=$!


log "Kamera (rs_launch $CAM_PROFILE, OHNE pointcloud, eigene Gruppe) → $CAM_LOG"

setuid bash -c "exec ros2 launch realsense2_camera rs_launch.py \

    pointcloud.enable:=false align_depth.enable:=true \

    depth_module.profile:=$CAM_PROFILE rgb_camera.profile:=$CAM_PROFILE" >"$CAM_LOG" 2>&1 &

CAM_PID=$!

```

```

log "Auf Kamera-Topic warten: /camera/color/image_raw (max ${CAM_WAIT}s)..."

ok=0

for i in $(seq 1 "$CAM_WAIT"); do

    kill -0 "$CAM_PID" 2>/dev/null || { err "Kamera zu frueh gestorben!"; tail -n 25 "$CAM_LOG"; exit 1; }

    [ "$(ros2 topic info /camera/color/image_raw 2>/dev/null | grep -c 'Publisher count: [1-9]')" -ge 1 ] && { ok=1;
log "Kamera sendet (${i}s)"; break; }

    sleep 1

done

[ "$ok" = "1" ] || { err "Kamera sendet nicht innerhalb ${CAM_WAIT}s. Siehe $CAM_LOG"; exit 1; }


# 2. Kamera (record_cam) entfernt – nicht verwendet (Benutzer, 2026-06-25).

if [ -n "${DISPLAY:-}" ]; then

    log "cam_viewer (Overlay-Anzeige) → $CAMV_LOG"

    setsid bash -c "exec python3 '$WS/cam_viewer.py'" >"$CAMV_LOG" 2>&1 &

    CAMV_PID=$!

fi


log "Auf Detector-Warmup warten (Modell + erste CUDA-Inferenz, max ${DETECTOR_WAIT}s)..."

ok=0

for i in $(seq 1 "$DETECTOR_WAIT"); do

    kill -0 "$DET_PID" 2>/dev/null || { err "Detector zu frueh gestorben!"; tail -n 25 "$DET_LOG"; exit 1; }

    grep -q "Modell bereit" "$DET_LOG" 2>/dev/null && { ok=1; log "Modell geladen (${i}s)"; break; }

    sleep 1

done

[ "$ok" = "1" ] || { err "Modell nicht geladen. Siehe $DET_LOG"; exit 1; }


log "====="

log " SCHRAUBE HINLEGEN (150-190mm, 20-30° geneigt). Wenn die Erkennung im Overlay stabil ist:"

log " IN DIESEM TERMINAL ENTER = Ziel SPEICHERN (danach werden detector+Kamera beendet). "

log "====="

# capture FOREGROUND – wenn der Benutzer ENTER drückt, wird das Ziel gespeichert und kehrt zurück

python3 "$WS/target_latch.py" capture "$TARGET_FILE" 2>&1 | tee "$CAP_LOG"

[ -f "$TARGET_FILE" ] || { err "Ziel nicht gespeichert – wird beendet."; exit 1; }


# =====

# UEBERGANG – VISION BEENDEN (RAM freigeben)

# =====

```

```

log "##### UEBERGANG – detector+Kamera werden beendet (RAM wird frei) #####"

kill_group "$DET_PID" "detector"; DET_PID=""

kill_group "$CAMV_PID" "cam_viewer"; CAMV_PID=""

kill_group "$REC_PID" "record_cam"; REC_PID=""

kill_group "$CAM_PID" "camera"; CAM_PID=""

sleep 2

log "Freier RAM: $(free -h | awk 'NR==2{print $7}')"

# =====

# PHASE B – RViz VORSCHAU + PICK

# =====

log "##### PHASE B – RViz VORSCHAU #####"

if [ -n "${DISPLAY:-}" ]; then

    # RViz mit MoveIt-Parametern starten (moveit_rviz.launch.py) – standalone

    # 'ros2 run rviz2' bekam kein robot_description_semantic und lieferte "No Planning Scene

    # Loaded" + unsichtbarer Roboter (Sitzung 31 Fix).

    RVIZ_ARG=""; [ -f "$RVIZ_CFG" ] && RVIZ_ARG="rviz_config:=$RVIZ_CFG"

    log "RViz (moveit_rviz.launch.py, $RVIZ_CFG) → $RVIZ_LOG"

    setsid bash -c "exec ros2 launch mycobot_moveit_config moveit_rviz.launch.py $RVIZ_ARG" >"$RVIZ_LOG" 2>&1 &

    RVIZ_PID=$!

else

    warn "Kein DISPLAY – RViz uebersprungen (keine Vorschau moeglich, pick laeuft trotzdem)."

```

```
log " In RViz die Trajektorie + KOLLISION (rot) pruefen."

log " Fuer die BESTAETIGUNG in DIESEM TERMINAL ENTER druecken (pick_tilt wartet 2 mal):"

log "    1) nach HOVER – wenn die Trajektorie sicher ist ENTER → abstieg+greifen"

log "    2) nach dem Greifen – wenn gehalten ENTER → anheben"

log " Ctrl+C = alles beenden + Roboter zu 0 parken."

log "===== "

# confirm_key FOREGROUND – hält das Skript am Leben; ENTER → /pick/confirm

python3 "$WS/confirm_key.py"

log "confirm_key beendet – wird beendet..."
```

run_pick_test.sh

```
#!/bin/bash

# =====

# run_pick_test.sh – Schrauben-pick&place-Test, KOMPLETT über Terminal.

# KEIN Claude nötig. KEIN RViz (Nano RAM-Ersparnis). Alles wird geloggt.

#

# Ablauf:

# 1) Systemvorbereitung (locale, rmem, USB power, GPU fan) [fragt sudo]

# 2) Stack starten: demo.launch.py use_camera:=true use_rviz:=false

# 3) Warten bis Controller active sind

# 4) vida_detector starten, warten bis "Modell bereit" + erstes "ZIEL" kommt

# 5) pick_place_cartesian AUSFUEHREN (auf Bildschirm + Log)

# 6) Beim Beenden ALLE Hintergrund-nodes aufräumen (keine Geister hinterlassen)

#

# Verwendung:

# cd ~/ros2_ws && ./run_pick_test.sh

# # Place-Punkt ändern:

# PLACE_X=0.12 PLACE_Y=-0.10 PLACE_Z=0.05 ./run_pick_test.sh

# # Abstiegs-Offset zur Schraube (in Schraubenoberseite versenken/anheben):

# GRASP_Z_OFFSET=0.005 ./run_pick_test.sh

# # NUR Simulation (ohne Roboter):

# SIM=1 ./run_pick_test.sh

#

# Logs: ~/ros2_ws/logs/pick_<zeit>/{stack.log,detector.log,pick_place.log}

# =====

set -u

# ---- Einstellbare Parameter (per env override) -----

PLACE_X="${PLACE_X:-0.15}"

PLACE_Y="${PLACE_Y:-0.15}"

PLACE_Z="${PLACE_Z:-0.05}"

GRASP_Z_OFFSET="${GRASP_Z_OFFSET:-0.0}"

SIM="${SIM:-0}"

VIEW="${VIEW:-1}" # 1 = Live-Overlay-Fenster (ohne RViz). Wenn RAM sehr knapp VIEW=0

DETECTOR_WAIT="${DETECTOR_WAIT:-120}" # max Wartezeit fuer detector-Modell+erstes Ziel (s)

CTRL_WAIT="${CTRL_WAIT:-60}" # max Wartezeit fuer controller active (s)
```

```

# ---- Locale (damit Qt/MoveIt floats nicht kaputtgehen) -----
export LC_ALL=C
export LC_NUMERIC=C
export LANG=C

WS="$HOME/ros2_ws"

STAMP="$(date +%Y%m%d_%H%M%S)"
LOGDIR="$WS/logs/pick_$STAMP"
mkdir -p "$LOGDIR"

STACK_LOG="$LOGDIR/stack.log"
DET_LOG="$LOGDIR/detector.log"
PICK_LOG="$LOGDIR/pick_place.log"

# Gruppenleiter-PIDs der Hintergrundprozesse (per setsid jeder in eigener process group)
STACK_PID=""
DET_PID=""
VIEW_PID=""

log() { echo -e "\033[1;36m[run_pick_test]\033[0m $*"; }
warn() { echo -e "\033[1;33m[run_pick_test]\033[0m $*"; }
err() { echo -e "\033[1;31m[run_pick_test]\033[0m $*" >&2; }

# Eine process group erst sanft, dann erzwungen töten
kill_group() {
    local pid="$1" name="$2"

    [ -z "$pid" ] && return 0

    if kill -0 "$pid" 2>/dev/null; then
        log "$name wird beendet (pgid=$pid)..."
        kill -TERM -- "$pid" 2>/dev/null

        for _ in $(seq 1 10); do kill -0 "$pid" 2>/dev/null || break; sleep 0.5; done

        kill -0 "$pid" 2>/dev/null && { warn "$name erzwungen (KILL)"; kill -KILL -- "$pid" 2>/dev/null; }
    fi
}

# Alle ROS-node-Geister aufkehren (gemeinsam für preflight + exit genutzt).

```

```

# KRITISCH: wenn mycobot_bridge.py, ros2_control_node sterben, bleiben sie VERWAIST und
# halten /dev/ttyTHS1 weiter; beim nächsten Lauf SEGFAULTet eine frische bridge wegen
# Port/Socket-Konflikt. Daher bridge + veraltete Sockets UNBEDINGT aufräumen.

sweep_ros_ghosts() {

    pkill -f "view_overlay.py"          2>/dev/null
    pkill -f "vida_detector"            2>/dev/null
    pkill -f "pick_place_cartesian"     2>/dev/null
    pkill -f "pick_place.launch.py"     2>/dev/null
    pkill -f "demo.launch.py"           2>/dev/null
    pkill -f "move_group"               2>/dev/null
    pkill -f "ros2_control_node"         2>/dev/null
    pkill -f "mycobot_bridge.py"         2>/dev/null # <-- verwaiste bridge (haelt Port)
    pkill -f "realsense2_camera"        2>/dev/null
    pkill -f "robot_state_publisher"    2>/dev/null

    sleep 2

    # Wenn etwas TERM widerstanden hat, erzwingen
    pkill -9 -f "mycobot_bridge.py"      2>/dev/null
    pkill -9 -f "ros2_control_node"      2>/dev/null
    pkill -9 -f "move_group"             2>/dev/null
    pkill -9 -f "realsense2_camera"      2>/dev/null

    rm -f /tmp/mycobot_bridge.sock /tmp/mycobot_bridge.sock.cmd 2>/dev/null
}

cleanup() {

    echo

    log "AUFRAEUMEN – Hintergrund-nodes werden beendet..."

    kill_group "$VIEW_PID" "overlay-viewer"

    kill_group "$DET_PID" "detector"

    kill_group "$STACK_PID" "stack"

    sweep_ros_ghosts # inkl. bridge + verwaiste nodes + veralteter Socket

    # Prüfen: ist der Port noch belegt?

    if fuser /dev/ttyTHS1 >/dev/null 2>&1; then

        warn "/dev/ttyTHS1 NOCH belegt – verbleibenden Halter erzwungen toeten"

        fuser -k -9 /dev/ttyTHS1 2>/dev/null

    fi

    log "Logs: $LOGDIR"

    log " lesen: tail -n 80 $PICK_LOG"

```



```

}

trap cleanup EXIT INT TERM

# ---- 1) Systemvorbereitung (sudo – fragt evtl. Passwort, am Anfang erledigen) ----
log "Systemvorbereitung (sudo evtl. noetig)..."

sudo sysctl -w net.core.rmem_max=8388608 net.core.wmem_max=8388608 \
    net.core.rmem_default=8388608 net.core.wmem_default=8388608 >/dev/null 2>&1

echo 0 | sudo tee /sys/devices/57000000.gpu/railgate_enable >/dev/null 2>&1

sudo sh -c 'echo 255 > /sys/devices/pwm-fan/target_pwm' 2>/dev/null

for dev in /sys/bus/usb/devices/*/idVendor; do
    [ "$(cat "$dev" 2>/dev/null)" = "8086" ] && \
        echo on | sudo tee "$(dirname "$dev")/power/control" >/dev/null 2>&1
done

# ---- Geister aus dem vorherigen eingefrorenen Lauf aufkehren -----
# (BESONDERS verwaiste mycobot_bridge.py + veralteter Socket – sonst SEGFAULTet
# ein frisches ros2_control; siehe Diagnose 2026-06-04 Sitzung 17)
sweep_ros_ghosts

if fuser /dev/ttyTHS1 >/dev/null 2>&1; then
    warn "/dev/ttyTHS1 vom vorherigen Lauf belegt – wird erzwungen freigegeben"
    fuser -k -9 /dev/ttyTHS1 2>/dev/null; sleep 1
fi

# ---- ROS-Umgebung -----
# ROS setup.bash nutzt undefinierte Variablen → set -u beim sourcen ausschalten.
set +u

source /opt/ros/galactic/setup.bash
source "$WS/install/setup.bash"

set -u

cd "$WS" || { err "ros2_ws nicht vorhanden"; exit 1; }

if [ "$SIM" = "1" ]; then
    export USE_FAKE_HARDWARE="true"
    warn "SIM MODE – fake_components (kein Roboter)"
else
    export USE_FAKE_HARDWARE="false"

    log "ECHTER Roboter @ ${MYCOBOT_PORT:-/dev/ttyTHS1}"

```

```

fi

# ---- 2) Stack starten (RViz AUS) -----
log "Stack wird gestartet (use_camera:=true use_rviz:=false) → $STACK_LOG"
setsid bash -c "exec ros2 launch mycobot_moveit_config demo.launch.py \
    use_camera:=true use_rviz:=false" >"$STACK_LOG" 2>&1 &
STACK_PID=$!

# ---- 3) Warten bis Controller active sind -----
log "Auf Controller warten (max ${CTRL_WAIT}s)..."
ok=0
for i in $(seq 1 "$CTRL_WAIT"); do
    if ! kill -0 "$STACK_PID" 2>/dev/null; then
        err "Stack zu frueh gestorben! Letzte Zeilen:"; tail -n 25 "$STACK_LOG"; exit 1
    fi
    if ros2 control list_controllers 2>/dev/null | grep -q "arm_controller.*active"; then
        ok=1; log "Controller AKTIV (${i}s)"; break
    fi
    sleep 1
done
[ "$ok" = "1" ] || { err "Controller wurde nicht aktiv innerhalb ${CTRL_WAIT}s. Siehe $STACK_LOG"; exit 1; }

# ---- 4) Detector starten, auf Modell + erstes ZIEL warten -----
log "vida_detector wird gestartet → $DET_LOG"
setsid bash -c "exec ros2 run vida_vision vida_detector" >"$DET_LOG" 2>&1 &
DET_PID=$!

log "Auf Detector-Bereitschaft warten: 'Modell bereit' (max ${DETECTOR_WAIT}s)..."
ok=0
for i in $(seq 1 "$DETECTOR_WAIT"); do
    kill -0 "$DET_PID" 2>/dev/null || { err "Detector zu frueh gestorben! Letzte Zeilen:"; tail -n 25 "$DET_LOG";
    exit 1; }
    grep -q "Modell bereit" "$DET_LOG" 2>/dev/null && { ok=1; log "Modell geladen (${i}s)"; break; }
    sleep 1
done
[ "$ok" = "1" ] || { err "Modell nicht geladen innerhalb ${DETECTOR_WAIT}s. Siehe $DET_LOG"; exit 1; }

```

```

# ---- 4b) Live-Overlay-Fenster (gewählte Schraube GRUEN + ZIEL) -----

if [ "$VIEW" = "1" ]; then

    if [ -n "${DISPLAY:-}" ]; then

        log "Overlay-Viewer wird geoeffnet (DISPLAY=$DISPLAY) – im Fenster 'q'/ESC = schliessen"

        setsid bash -c "exec python3 '$WS/view_overlay.py' \

            >"$LOGDIR/viewer.log" 2>&1 &

        VIEW_PID=$!

    else

        warn "Kein DISPLAY – Overlay-Fenster uebersprungen. Fuer Headless-PNG:"

        warn " SAVE_ONLY=1 python3 $WS/view_overlay.py (→ /tmp/vida/overlay_latest.png)"

    fi

fi

log "Auf erstes Schrauben-Ziel ('ZIEL') warten (max 60s)..."

ok=0

for i in $(seq 1 60); do

    kill -0 "$DET_PID" 2>/dev/null || { err "Detector gestorben! Siehe $DET_LOG"; exit 1; }

    if grep -q "ZIEL" "$DET_LOG" 2>/dev/null; then

        ok=1

        log "Schraube gefunden → $(grep 'ZIEL' "$DET_LOG" | tail -n1)"

        break

    fi

    # auch 'keine Schraube' melden, damit wir nicht blind warten

    if [ $((i % 10)) -eq 0 ]; then

        warn " ...noch kein Ziel (${i}s). Detector letzte Zeile: $(tail -n1 "$DET_LOG")"

    fi

    sleep 1

done

if [ "$ok" != "1" ]; then

    err "Innerhalb 60s keine Schraube erkannt. Ist eine Schraube im Kamerabild? Siehe $DET_LOG"

    err "pick_place wird trotzdem versucht (hat eigenen 10s-Timeout)..."

fi

# ---- 5) pick_place_cartesian AUSFUEHREN -----

log "====="

log " PICK & PLACE wird ausgefuehrt"

log " place=($PLACE_X,$PLACE_Y,$PLACE_Z) grasp_z_offset=$GRASP_Z_OFFSET"

```

```

log " log: $PICK_LOG"

log "===== "

set +e

# HINWEIS: wir nutzen ros2 launch – pick_place.launch.py lädt das SRDF (robot_description_semantic)
# + kinematics-Parameter der MoveGroupInterface. Reines "ros2 run" blieb ohne SRDF
# und stürzte mit "Unable to construct robot model" ab.

export PLACE_X PLACE_Y PLACE_Z GRASP_Z_OFFSET

ros2 launch mycobot_demo pick_place.launch.py 2>&1 | tee "$PICK_LOG"

RC=${PIPESTATUS[0]}

set -e 2>/dev/null


# ros2 launch liefert 0, selbst wenn ein node darin stirbt → echten Status aus dem LOG ziehen.
# Ist ein node mit Nicht-Null beendet? ('process has died ... exit code N' N!=0)

if grep -qE "process has died.*exit code [1-9]" "$PICK_LOG" 2>/dev/null; then

    RC=1

fi


echo

if [ "$RC" = "0" ]; then

    log " pick_place ERFOLGREICH beendet"

else

    err " pick_place mit FEHLER beendet – obiges Log / $PICK_LOG ansehen"

fi


# cleanup trap läuft bei EXIT (detector + stack werden beendet)

exit "$RC"

```

run_pick_tilt.sh

```
#!/bin/bash

# =====

# run_pick_tilt.sh – TILT-BEWUSSTER PICK (KEIN RViz – Kamerafenster + Terminal).

#

# Ablauf (Sitzung 25, Plan: ~/.claude/plans/shimmering-wondering-bachman.md):

#   home (0) → auf /vida/target warten (detector sendet, wenn er einen ZUVERLAESSIGEN tilt findet; sonst nicht
#
#           → Roboter wartet, Benutzer mischt die Schrauben)
#
#   → HOVER (von der Grasp-Pose entlang der Annäherungsachse APPROACH_HEIGHT zurück, Finger offen)
#
#   → im cam_viewer-Fenster 'g' = BESTAETIGUNG → PARALLELER Abstieg → greifen → anheben → halten.
#
# KEIN RViz (Nano RAM + Tegra segfault). Linkes Panel /vida/overlay (zeigt tilt=.. lin=.. [mod];
# wenn unzuverlässig "Schrauben mischen"), rechtes Panel Aufnahmekamera.
# Detector bleibt AN (pick_tilt liest ständig das aktuellste Ziel).
#
# Verwendung: cd ~/ros2_ws && ./run_pick_tilt.sh
#
#   APPROACH_HEIGHT=0.05 ./run_pick_tilt.sh   # Hover-Höhe (m), 5cm
#
#   SIM=1 ./run_pick_tilt.sh                   # Simulation ohne Roboter
#
# BESTAETIGUNG: im Kamerafenster 'g' (oder: ros2 topic pub --once /pick/confirm std_msgs/msg/Empty '{}')
#
# Logs: ~/ros2_ws/logs/pick_tilt_<zeit>/
# =====

set -u

APPROACH_HEIGHT="${APPROACH_HEIGHT:-0.10}"

SIM="${SIM:-0}"

DETECTOR_WAIT="${DETECTOR_WAIT:-120}"

CTRL_WAIT="${CTRL_WAIT:-60}"

# Detector-Schwellen (Sitzung 29): per env override-bar.

CONF="${CONF:-0.75}"

MODE_CONSENSUS="${MODE_CONSENSUS:-3}"

STICKY_RADIUS="${STICKY_RADIUS:-0.05}"

export LC_ALL=C

export LC_NUMERIC=C

export LANG=C

# Python-Logs sofort ausgeben (detector-Log war 2min gepuffert – Lektion Sitzung 28).
```

```

export PYTHONUNBUFFERED=1

WS="$HOME/ros2_ws"

STAMP="$(date +%Y%m%d_%H%M%S)"

LOGDIR="$WS/logs/pick_tilt_$STAMP"

mkdir -p "$LOGDIR"

STACK_LOG="$LOGDIR/stack.log"
REC_LOG="$LOGDIR/record_cam.log"
CAMV_LOG="$LOGDIR/cam_viewer.log"
DET_LOG="$LOGDIR/detector.log"
NODE_LOG="$LOGDIR/pick_tilt.log"

STACK_PID=""
REC_PID=""
CAMV_PID=""
DET_PID=""
NODE_PID=""

log() { echo -e "\033[1;36m[run_pick_tilt]\033[0m $*"; }
warn() { echo -e "\033[1;33m[run_pick_tilt]\033[0m $*"; }
err() { echo -e "\033[1;31m[run_pick_tilt]\033[0m $*" >&2; }

kill_group() {
    local pid="$1" name="$2"

    [ -z "$pid" ] && return 0

    if kill -0 "$pid" 2>/dev/null; then
        log "$name wird beendet (pgid=$pid)..."

        kill -TERM -- "$pid" 2>/dev/null

        for _ in $(seq 1 10); do kill -0 "$pid" 2>/dev/null || break; sleep 0.5; done

        kill -0 "$pid" 2>/dev/null && { warn "$name erzwungen (KILL)"; kill -KILL -- "$pid" 2>/dev/null; }
    fi
}

# Verwaiste mycobot_bridge.py + veralteter Socket = segfault beim nächsten Lauf. Aufkehren.
sweep_ros_ghosts() {

    # ACHTUNG: das Muster "pick_tilt" passt auf den eigenen Namen des Wrappers (run_pick_tilt.sh)

```

```

# und tötet das Skript → cleanup-sweep→tot Endlosschleife. Nur mit node ("mycobot_demo/pick_tilt"
# = installierter Executable-Pfad) und launch ("pick_tilt.launch.py") Mustern matchen.

pkill -f "record_cam_publisher" 2>/dev/null

pkill -f "cam_viewer.py" 2>/dev/null

pkill -f "vida_detector" 2>/dev/null

pkill -f "mycobot_demo/pick_tilt" 2>/dev/null

pkill -f "pick_tilt.launch.py" 2>/dev/null

pkill -f "demo.launch.py" 2>/dev/null

pkill -f "rviz2" 2>/dev/null

pkill -f "move_group" 2>/dev/null

pkill -f "ros2_control_node" 2>/dev/null

pkill -f "mycobot_bridge.py" 2>/dev/null

pkill -f "realsense2_camera" 2>/dev/null

pkill -f "robot_state_publisher" 2>/dev/null

sleep 2

pkill -9 -f "mycobot_bridge.py" 2>/dev/null

pkill -9 -f "ros2_control_node" 2>/dev/null

pkill -9 -f "move_group" 2>/dev/null

pkill -9 -f "realsense2_camera" 2>/dev/null

pkill -9 -f "rviz2" 2>/dev/null

pkill -9 -f "mycobot_demo/pick_tilt" 2>/dev/null

rm -f /tmp/mycobot_bridge.sock /tmp/mycobot_bridge.sock.cmd 2>/dev/null
}

# Roboter zu Punkt 0 parken (während der Stack läuft; auch wenn beim Beenden die Servos frei werden, soll er nicht
fallen).

ARM_JOINTS="[joint2_to_joint1, joint3_to_joint2, joint4_to_joint3, joint5_to_joint4, joint6_to_joint5,
joint6output_to_joint6]"

goto_zero() {

    local secs="${1:-5}"

    command -v ros2 >/dev/null 2>&1 || return 0

    if ros2 action list 2>/dev/null | grep -q "/arm_controller/follow_joint_trajectory"; then

        log "Roboter wird zu Punkt 0 gesendet (${secs}s, Servo soll nicht frei werden)..."

        timeout 20 ros2 action send_goal /arm_controller/follow_joint_trajectory \

            control_msgs/action/FollowJointTrajectory \

                "{trajectory: {joint_names: $ARM_JOINTS, points: [{positions: [0.0,0.0,0.0,0.0,0.0,0.0], time_from_start:
{sec: $secs}}]}}" \

            >/dev/null 2>&1 \

```

```

    && log "Roboter bei Punkt 0." || warn "Senden zu 0 nicht abgeschlossen (trotzdem weiter)."
```

else

```

    warn "arm_controller action nicht vorhanden – Roboter kann nicht zu 0 gesendet werden."
```

fi

```

}
```

Nachdem Stack/bridge beendet sind, mit pymycobot GARANTIERT zu 0 parken + Servos UNTER DREHMOMENT lassen.

Warum: action-basiertes goto_zero ist unzuverlässig – beim Beenden gibt mycobot_bridge die Servos

frei und LAESST den Roboter FALLEN (Lektion Sitzung 30). Diese Funktion öffnet den Port direkt,

sendet zu 0 und lässt die Servos energetisiert (haltend) → der Arm bleibt bei 0 hängen, fällt nicht.

```

park_zero_pymycobot() {
    [ "${SIM:-0}" = "1" ] && return 0

    local port="${MYCOBOT_PORT:-/dev/ttyTHS1}"

    for _ in $(seq 1 10); do fuser "$port" >/dev/null 2>&1 || break; sleep 0.5; done

    if fuser "$port" >/dev/null 2>&1; then

        warn "pymycobot Park: $port noch belegt – wird uebersprungen (Arm kann fallen, von Hand kontrollieren!);"
        return 0

    fi

    log "GARANTIERTER Park zu 0 mit pymycobot (Servos bleiben unter Drehmoment, Arm faellt nicht)..."

    if MYCOBOT_PORT="$port" timeout 35 python3 - <<'PYEOF'

import os, time

try:

    from pymycobot import MyCobot280

    mc = MyCobot280(os.environ.get("MYCOBOT_PORT", "/dev/ttyTHS1"), 1000000)

    time.sleep(0.6)

    try:

        mc.power_on(); time.sleep(0.4)      # heruntergefallene/freie Servos neu energetisieren

    except Exception:

        pass

    mc.send_radians([0.0] * 6, 25)

    for _ in range(14):

        time.sleep(1.0)

        if mc.is_moving() == 0:

            break

    try:

        mc.set_gripper_value(100, 50, 1)  # beim Park Finger OFFEN (gehaltene Schraube loslassen)

        time.sleep(1.0)

```



```

except Exception as ge:

    print("Greifer-Oeffnen Warnung:", ge)

a = mc.get_radians()

if a and len(a) == 6:

    print("groesster |Winkel| nach Park = %.3f rad" % max(abs(x) for x in a))
except Exception as e:

    print("pymycobot Park FEHLER:", e)
PYEOF

then

    log "pymycobot Park beendet – Roboter bei 0, Servos halten."

else

    warn "pymycobot Park Timeout/fehlgeschlagen – Arm von Hand kontrollieren."

fi
}

cleanup() {

    # Re-entry guard: falls trap EXIT/INT/TERM + ein mögliches self-signal von sweep cleanup

    # erneut auslöst, die Endlosschleife abbrechen (nur einmal laufen).

    [ "${_CLEANED:-0}" = "1" ] && return 0

    _CLEANED=1

    trap - EXIT INT TERM

    echo

    log "AUFRAEUMEN – Hintergrund-nodes werden beendet..."

    kill_group "$NODE_PID" "pick_tilt node"

    kill_group "$DET_PID" "detector"

    kill_group "$CAMV_PID" "cam_viewer"

    kill_group "$REC_PID" "record_cam"

    goto_zero 5                                # Park beim Beenden (Stack noch aktiv)

    kill_group "$STACK_PID" "stack"

    sweep_ros_ghosts

    if fuser /dev/ttyTHS1 >/dev/null 2>&1; then

        warn "/dev/ttyTHS1 NOCH belegt – erzwungen toeten"; fuser -k -9 /dev/ttyTHS1 2>/dev/null

    fi

    park_zero_pymycobot                        # GARANTierter Park zu 0 (Servos unter Drehmoment, Arm faellt nicht)

    log "Logs: $LOGDIR"

}

trap cleanup EXIT INT TERM

```

```

# ---- 1) Systemvorbereitung ----

log "Systemvorbereitung (sudo evtl. noetig)..."

sudo sysctl -w net.core.rmem_max=8388608 net.core.wmem_max=8388608 \
    net.core.rmem_default=8388608 net.core.wmem_default=8388608 >/dev/null 2>&1

echo 0 | sudo tee /sys/devices/57000000.gpu/railgate_enable >/dev/null 2>&1

sudo sh -c 'echo 255 > /sys/devices/pwm-fan/target_pwm' 2>/dev/null

for dev in /sys/bus/usb/devices/*/idVendor; do
    [ "$(cat "$dev" 2>/dev/null)" = "8086" ] && \
        echo on | sudo tee "$(dirname "$dev")/power/control" >/dev/null 2>&1
done

sweep_ros_ghosts

if fuser /dev/ttyTHS1 >/dev/null 2>&1; then
    warn "/dev/ttyTHS1 vom vorherigen Lauf belegt – wird erzwungen freigegeben"
    fuser -k -9 /dev/ttyTHS1 2>/dev/null; sleep 1
fi

log "KEIN RViz – um den RAM zu entlasten nur Kamerafenster + Terminal."

# ---- ROS-Umgebung ----

set +u

source /opt/ros/galactic/setup.bash

source "$WS/install/setup.bash"

set -u

cd "$WS" || { err "ros2_ws nicht vorhanden"; exit 1; }

if [ "$SIM" = "1" ]; then
    export USE_FAKE_HARDWARE="true"; warn "SIM MODE – fake_components (kein Roboter)"
else
    export USE_FAKE_HARDWARE="false"; log "ECHTER Roboter @ ${MYCOBOT_PORT:-/dev/ttyTHS1}"
fi

# ---- 2) Stack (KEIN RViz + Kamera) ----

log "Stack wird gestartet (use_camera:=true use_rviz:=false) → $STACK_LOG"

setuid bash -c "exec ros2 launch mycobot_moveit_config demo.launch.py \
    use_camera:=true use_rviz:=false" >"$STACK_LOG" 2>&1 &

```

```

STACK_PID=$!

# ---- 3) Warten bis Controller active sind ----

log "Auf Controller warten (max ${CTRL_WAIT}s)..."

ok=0

for i in $(seq 1 "$CTRL_WAIT"); do

    kill -0 "$STACK_PID" 2>/dev/null || { err "Stack zu frueh gestorben! Letzte Zeilen:"; tail -n 25 "$STACK_LOG";
exit 1; }

    if ros2 control list_controllers 2>/dev/null | grep -q "arm_controller.*active"; then

        ok=1; log "Controller AKTIV (${i}s)"; break

    fi

    sleep 1

done

[ "$ok" = "1" ] || { err "Controller wurde nicht aktiv innerhalb ${CTRL_WAIT}s. Siehe $STACK_LOG"; exit 1; }

# ---- 3a) START-PARK ----

goto_zero 5

# ---- 3a2) 0-PRUEFUNG (Sitzung 29): JTC sagt ohne Toleranz "success"; wenn der Roboter physisch
# nicht 0 erreicht hat (Klemmen/Anstoßen), blockiert MoveIt mit start-state-collision.
# Echte /joint_states werden gelesen und |Winkel| geprüft; >5° → ABORT.

if [ "$SIM" != "1" ]; then

    ZMAX=$(timeout 15 python3 - <<'PYEOF' 2>/dev/null

import rclpy

from rclpy.node import Node

from sensor_msgs.msg import JointState

ARM = {"joint2_to_joint1", "joint3_to_joint2", "joint4_to_joint3",
        "joint5_to_joint4", "joint6_to_joint5", "joint6output_to_joint6"}

rclpy.init()

node = Node("zero_check")

got = {}

def cb(m):

    for n, p in zip(m.name, m.position):

        if n in ARM:

            got[n] = p

node.create_subscription(JointState, "/joint_states", cb, 10)

```

```

import time

t0 = time.time()

while time.time() - t0 < 10 and len(got) < len(ARM):

    rclpy.spin_once(node, timeout_sec=0.2)

print(f"{max(abs(v) for v in got.values()):.4f}" if got else "NaN")

node.destroy_node(); rclpy.shutdown()

PYEOF

)

if [ -z "$ZMAX" ] || [ "$ZMAX" = "NaN" ]; then

    warn "θ-Pruefung: /joint_states nicht lesbar (trotzdem weiter).\"

elif python3 -c "import sys; sys.exit(0 if float('$ZMAX') > 0.09 else 1)"; then

    err "ROBOTER NICHT BEI 0 (groesster |Winkel|=${ZMAX} rad > 0.09≈5°).\"

    err "Der Arm koennte physisch klemmen/nicht erreicht haben – von Hand loesen, ggf.\"

    err "bei beendetem Stack mit pymycobot send_radians([0]*6,25) parken, dann neu starten.\"

    exit 1

else

    log "θ-Pruefung OK (groesster |Winkel|=${ZMAX} rad).\"

fi

fi

# ---- 3b) 2. Kamera-Publisher ----

log "record_cam_publisher (2. Kamera) wird gestartet → $REC_LOG"

setsid bash -c "exec python3 '$WS/record_cam_publisher.py'" >"$REC_LOG" 2>&1 &

REC_PID=$!

# ---- 3b2) Kamera-Viewer (2 Kameras + 'g' Bestätigungstaste) ----

if [ -n "${DISPLAY:-}" ]; then

    log "cam_viewer (2 Kameras, 'g'=BESTAETIGEN) wird gestartet → $CAMV_LOG"

    setsid bash -c "exec python3 '$WS/cam_viewer.py'" >"$CAMV_LOG" 2>&1 &

    CAMV_PID=$!

else

    warn "Kein DISPLAY – Kamerafenster uebersprungen. Bestaetigung: ros2 topic pub --once /pick/confirm std_msgs/msg/Empty '{}'"

fi

# ---- 4) Detector (YOLO) starten, auf Modell warten ----

log "vida_detector wird gestartet → $DET_LOG"

```

```

setuid bash -c "exec ros2 run vida_vision vida_detector --ros-args \

  -p conf:=$CONF -p mode_consensus:=$MODE_CONSENSUS -p sticky_radius:=$STICKY_RADIUS" >"$DET_LOG" 2>&1 &

DET_PID=$!

log "Auf Detector-Bereitschaft warten: 'Modell bereit' (max ${DETECTOR_WAIT}s)..."

ok=0

for i in $(seq 1 "$DETECTOR_WAIT"); do

  kill -0 "$DET_PID" 2>/dev/null || { err "Detector zu frueh gestorben! Letzte Zeilen:"; tail -n 25 "$DET_LOG";
exit 1; }

  grep -q "Modell bereit" "$DET_LOG" 2>/dev/null && { ok=1; log "Modell geladen (${i}s)"; break; }

  sleep 1

done

[ "$ok" = "1" ] || { err "Modell nicht geladen innerhalb ${DETECTOR_WAIT}s. Siehe $DET_LOG"; exit 1; }

warn "HINWEIS: pick_tilt erhaelt /vida/target nur bei ZUVERLAESSIGEM tilt. Wenn das Overlay 'Schrauben mischen'"

warn "      sagt, die Schraube deutlich GENEIGT hinlegen/mischen; der Roboter wartet im Hover."

# ---- 5) pick_tilt AUSFUEHREN ----

log "====="

log " TILT-BEWUSSTER PICK (hover=${APPROACH_HEIGHT}m → 'g' Bestaetigung → paralleler Abstieg → greifen → anheben)"

log " BESTAETIGUNG: Taste 'g' im Kamerafenster."

log "====="

export APPROACH_HEIGHT

: > "$NODE_LOG" # tail-Race verhindern

setuid bash -c "exec ros2 launch mycobot_demo pick_tilt.launch.py" >>"$NODE_LOG" 2>&1 &

NODE_PID=$!

log "Node-Ausgabe (Ctrl+C = alles beenden):"

tail -n +1 -F "$NODE_LOG" --pid="$NODE_PID"

log "Wird beendet..."

```

run_place_calib.sh

```
#!/bin/bash

# =====

# run_place_calib.sh – PLACE (Schraube senkrecht einsetzen) XY-Offset-KALIBRIERUNG.

#

# Zweck (Sitzung 35→36): click_place_moveit.py läuft live, aber die Schraube

# fällt "hinter" das Loch (TCP ~2.5cm + Seitengriff-Geometrie → XY-Offset).

# Dieses Skript baut den Kalibrier-Stack auf; der Benutzer richtet im Werkzeug per nudge (8/2/4/6,+/-)

# die Schraube über das Loch aus, der funktionierende Offset wird gefunden → als PERMANENTER

# Standard (DEFAULT_OFF) ins Werkzeug eingebettet.

#

# ARCHITEKTUR (DEVLOG Sitzung 35): Stack FAKE hardware → ros2_control RUEHRT DEN PORT NICHT AN;

# den Port (/dev/ttyTHS1) öffnet click_place_moveit.py per pymycobot.

# move_group plant einen kollisionsfreien WEG, RViz (rviz_preview.rviz, Loop Anim)

# zeigt die Vorschau, mit ENTER/E folgt pymycobot dem Plan und steuert den echten Roboter.

#

# Komponenten: stack(fake,ohne Kamera,ohne rviz) + RViz(preview) + Kamera

#               + click_place_moveit.py (FOREGROUND, öffnet DEN Port)

#

# Verwendung: cd ~/ros2_ws && ./run_place_calib.sh

#             COLOR_PROFILE=848x480x15 ./run_place_calib.sh # wenn die Kamera zu stark auslastet

# Logs: ~/ros2_ws/logs/place_calib_<zeit>/

# =====

set -u

# Kamera-Profile – in Sitzung 35 lief Farbe 1280x720 (detector aus, RAM reichlich).

# Bei "Out of frame resources" mit COLOR_PROFILE=848x480x15 erneut versuchen.

COLOR_PROFILE="${COLOR_PROFILE:-1280x720x15}"

DEPTH_PROFILE="${DEPTH_PROFILE:-848x480x15}"

CTRL_WAIT="${CTRL_WAIT:-60}"

CAM_WAIT="${CAM_WAIT:-40}"

MG_WAIT="${MG_WAIT:-40}"

export LC_ALL=C LC_NUMERIC=C LANG=C PYTHONUNBUFFERED=1

# KRITISCH: demo.launch.py liest die hardware-Auswahl aus der UMGEBUNGSVARIABLE USE_FAKE_HARDWARE

# (das launch-Argument 'use_fakeHardware:= ' wird IGNORIERT). Ohne fake öffnet ros2_control

# MyCobotHardware → mycobot_bridge.py den Port → mit dem tool DOPPELTER ZUGRIFF → serial platzt.
```

```

export USE_FAKE_HARDWARE=true

WS="$HOME/ros2_ws"

STAMP="$(date +%Y%m%d_%H%M%S)"

LOGDIR="$WS/logs/place_calib_$STAMP"

mkdir -p "$LOGDIR"

RVIZ_CFG="$WS/rviz_preview.rviz"

TOOL="$WS/src/mycobot_calibration/scripts/click_place_moveit.py"


STACK_LOG="$LOGDIR/stack.log"; RVIZ_LOG="$LOGDIR/rviz.log"

CAM_LOG="$LOGDIR/camera.log"; TOOL_LOG="$LOGDIR/tool.log"

ESTOP_LOG="$LOGDIR/estop_button.log"

STACK_PID=""; RVIZ_PID=""; CAM_PID=""; ESTOP_PID=""


log() { echo -e "\033[1;36m[place]\033[0m $*"; }

warn() { echo -e "\033[1;33m[place]\033[0m $*"; }

err() { echo -e "\033[1;31m[place]\033[0m $*" >&2; }


kill_group() {

    local pid="$1" name="$2"; [ -z "$pid" ] && return 0

    if kill -0 "$pid" 2>/dev/null; then

        log "$name wird beendet (pgid=$pid)..."

        kill -TERM -- "$pid" 2>/dev/null

        for _ in $(seq 1 10); do kill -0 "$pid" 2>/dev/null || break; sleep 0.5; done

        kill -0 "$pid" 2>/dev/null && { warn "$name erzwungen (KILL)"; kill -KILL -- "$pid" 2>/dev/null; }

    fi
}


sweep_ros_ghosts() {

    pkill -f "estop_button"          2>/dev/null

    pkill -f "estop_relay"           2>/dev/null

    pkill -f "click_place_moveit"    2>/dev/null

    pkill -f "demo.launch.py"        2>/dev/null

    pkill -f "moveit_rviz.launch"     2>/dev/null

    pkill -f "rviz2"                 2>/dev/null

    pkill -f "move_group"             2>/dev/null

    pkill -f "ros2_control_node"      2>/dev/null

```

```

    pkill -f "mycobot_bridge.py"      2>/dev/null

    pkill -f "realsense2_camera"      2>/dev/null

    pkill -f "robot_state_publisher" 2>/dev/null

    sleep 2

    pkill -9 -f "ros2_control_node" 2>/dev/null

    pkill -9 -f "move_group"          2>/dev/null

    pkill -9 -f "realsense2_camera" 2>/dev/null

    pkill -9 -f "rviz2"                2>/dev/null

    rm -f /tmp/mycobot_bridge.sock /tmp/mycobot_bridge.sock.cmd 2>/dev/null
}

park_zero_pymycobot() {
    local port="${MYCOBOT_PORT:-/dev/ttyTHS1}"

    for _ in $(seq 1 10); do fuser "$port" >/dev/null 2>&1 || break; sleep 0.5; done

    if fuser "$port" >/dev/null 2>&1; then

        warn "pymycobot Park: $port noch belegt – wird uebersprungen (Arm von Hand kontrollieren!)"; return 0
    fi

    log "GARANTierter Park zu 0 mit pymycobot (Servos unter Drehmoment, Arm faellt nicht)..."

    MYCOBOT_PORT="$port" timeout 35 python3 - <<'PYEOF'

import os, time

try:

    from pymycobot import MyCobot280

    mc = MyCobot280(os.environ.get("MYCOBOT_PORT", "/dev/ttyTHS1"), 1000000)

    time.sleep(0.6)

    try: mc.power_on(); time.sleep(0.4)

    except Exception: pass

    mc.send_radians([0.0]*6, 25)

    for _ in range(14):

        time.sleep(1.0)

        if mc.is_moving() == 0: break

    try:

        mc.set_gripper_value(100, 50, 1)  # beim Park Finger OFFEN (gehaltene Schraube loslassen)

        time.sleep(1.0)

    except Exception as ge: print("Greifer-0effnen Warnung:", ge)

    a = mc.get_radians()

    if a and len(a) == 6:

        print("groesster |Winkel| nach Park = %.3f rad" % max(abs(x) for x in a))

```



```

except Exception as e:

    print("pymycobot Park FEHLER:", e)
PYEOF

    log "pymycobot Park beendet – Roboter bei 0."
}

cleanup() {

    [ "${_CLEANED:-0}" = "1" ] && return 0
    _CLEANED=1; trap - EXIT INT TERM; echo

    log "AUFRAEUMEN – nodes werden beendet..."

    kill_group "$CAM_PID"    "camera"

    kill_group "$RVIZ_PID"   "rviz"

    kill_group "$STACK_PID"  "stack"

    sweep_ros_ghosts

    fuser /dev/ttyTHS1 >/dev/null 2>&1 && fuser -k -9 /dev/ttyTHS1 2>/dev/null

    park_zero_pymycobot

    # NOT-AUS-Fenster: sweep hat es meist schon beendet; hier Rest-Aufräumen.

    kill_group "$ESTOP_PID"  "estop_button"

    log "Logs: $LOGDIR"
}

trap cleanup EXIT INT TERM

[ -f "$TOOL" ] || { err "Werkzeug nicht vorhanden: $TOOL"; exit 1; }

# ---- Systemvorbereitung (rmem/gpu/fan/usb-power) ----

log "Systemvorbereitung (sudo evtl. noetig)..."

sudo sysctl -w net.core.rmem_max=8388608 net.core.wmem_max=8388608 \

    net.core.rmem_default=8388608 net.core.wmem_default=8388608 >/dev/null 2>&1

echo 0 | sudo tee /sys/devices/57000000.gpu/railgate_enable >/dev/null 2>&1

sudo sh -c 'echo 255 > /sys/devices/pwm-fan/target_pwm' 2>/dev/null

for dev in /sys/bus/usb/devices/*/idVendor; do

    [ "$(cat "$dev" 2>/dev/null)" = "8086" ] && \

        echo on | sudo tee "$(dirname "$dev")/power/control" >/dev/null 2>&1

done

sweep_ros_ghosts

fuser /dev/ttyTHS1 >/dev/null 2>&1 && { warn "Port belegt – wird freigegeben"; fuser -k -9 /dev/ttyTHS1
2>/dev/null; sleep 1; }

```

```

set +u

source /opt/ros/galactic/setup.bash

source "$WS/install/setup.bash"

set -u

cd "$WS" || { err "ros2_ws nicht vorhanden"; exit 1; }

# ---- Stack: move_group + FAKE control (RUEHRT DEN PORT NICHT AN), ohne Kamera, ohne rviz ----
log "Stack (move_group+FAKE control, ohne Kamera, KEIN RViz) → $STACK_LOG"

setsid bash -c "exec ros2 launch mycobot_moveit_config demo.launch.py \
    use_camera:=false use_rviz:=false use_fake_hardware:=true" >"$STACK_LOG" 2>&1 &

STACK_PID=$!

log "Auf Controller warten (max ${CTRL_WAIT}s)..."

ok=0

for i in $(seq 1 "$CTRL_WAIT"); do

    kill -0 "$STACK_PID" 2>/dev/null || { err "Stack zu frueh gestorben!"; tail -n 25 "$STACK_LOG"; exit 1; }

    ros2 control list_controllers 2>/dev/null | grep -q "arm_controller.*active" && { ok=1; log "Controller AKTIV (${i}s)"; break; }

    sleep 1

done

[ "$ok" = "1" ] || { err "Controller wurde nicht aktiv. Siehe $STACK_LOG"; exit 1; }

# NOT-AUS-Taster: publiziert /estop (gelatcht). Enforcement übernimmt hier das
# Werkzeug selbst (click_place_moveit.py abonniert /estop und stoppt pymycobot
# sofort) – der Stack ist FAKE, es läuft kein serieller Bridge.

if [ -n "${DISPLAY:-}" ]; then

    log "NOT-AUS-Taster (estop_button) → $ESTOP_LOG"

    setsid bash -c "exec ros2 run mycobot_calibration estop_button" >"$ESTOP_LOG" 2>&1 &

    ESTOP_PID=$!

else

    warn "Kein DISPLAY – NOT-AUS-Fenster nicht moeglich. Terminal-Alternative:"

    warn "  ros2 topic pub --once --qos-durability transient_local /estop std_msgs/msg/Bool \"${data: true}\""

fi

log "Auf move_group (/move_action) warten (max ${MG_WAIT}s)..."

ok=0

```

```

for i in $(seq 1 "$MG_WAIT"); do

    ros2 action list 2>/dev/null | grep -q "/move_action" && { ok=1; log "move_group BEREIT (${i}s)"; break; }

    sleep 1

done

[ "$ok" = "1" ] || warn "move_action nicht erschienen – du kannst im Werkzeug trotzdem mit P planen."

# ---- RViz-Vorschau (rviz_preview.rviz, Loop Animation) ----

if [ -n "${DISPLAY:-}" ]; then

    RVIZ_ARG=""; [ -f "$RVIZ_CFG" ] && RVIZ_ARG="rviz_config:=$RVIZ_CFG"

    log "RViz-Vorschau (moveit_rviz.launch.py, $RVIZ_CFG) → $RVIZ_LOG"

    setid bash -c "exec ros2 launch mycobot_moveit_config moveit_rviz.launch.py $RVIZ_ARG" >"$RVIZ_LOG" 2>&1 &

    RVIZ_PID=$!

else

    warn "Kein DISPLAY – RViz uebersprungen (keine Vorschau moeglich)."

```

```

sleep 1

done

log "====="
log " KALIBRIER-SCHLEIFE – Werkzeug wird FOREGROUND geoeffnet."
log " 1) Auf das Fixture-Loch (wo die Schraube eingesetzt wird) klicken."
log " 2) Mit 0 Greifer auf → Schraube horizontal laden → mit C greifen."
log " 3) P = planen (in RViz kollisionsfreier Weg + Schraube SENKRECHT Vorschau)."
log " 4) E = ausfuehren → in der Kamera die Lage der Schraube zum Loch ansehen."
log " 5) Mit 8/2/4/6 (xy 1cm), +/- (z 1cm) NUDGE → jeder nudge plant neu."
log "    Wiederholen, bis die Schraube genau ueber dem Loch ist, den FUNKTIONIERENDEN Offset NOTIEREN."
log "    (im Terminal-Log erscheint die Zeile 'offset x=.. y=.. z=..cm')"
log " 6) Q = beenden. Den funktionierenden Offset mir sagen → ich bitte ihn PERMANENT ins Werkzeug ein."
log " Ctrl+C / Q = alles beenden + Roboter zu 0 parken."
log "====="
# Werkzeug FOREGROUND – öffnet DEN Port; wenn der Benutzer Q drückt, kehrt es zurück, dann parkt cleanup.
python3 "$TOOL" 2>&1 | tee "$TOOL_LOG"

log "Werkzeug beendet – wird aufgeraemt..."

```

rviz_preview.rviz

Panels:

- Class: rviz_common/Displays

Help Height: 78

Name: Displays

Property Tree Widget:

Expanded: null

Splitter Ratio: 0.5

Tree Height: 78

Visualization Manager:

Class: ''

Displays:

- Alpha: 0.5

Cell Size: 0.10000000149011612

Class: rviz_default_plugins/Grid

Color: 160; 160; 164

Enabled: true

Line Style:

Line Width: 0.029999999329447746

Value: Lines

Name: Grid

Normal Cell Count: 0

Offset:

X: 0

Y: 0

Z: 0

Plane: XY

Plane Cell Count: 10

Reference Frame: <Fixed Frame>

Value: true

- Acceleration_Scaling_Factor: 0.1

Class: moveit_rviz_plugin/MotionPlanning

Enabled: true

Move Group Namespace: ''

MoveIt_Allow_Approximate_IK: false

MoveIt_Allow_External_Program: false

MoveIt_Allow_Replanning: false

MoveIt_Allow_Sensor_Positioning: false

MoveIt_Planning_Attempts: 10

MoveIt_Planning_Time: 5

MoveIt_Use_Cartesian_Path: false

MoveIt_Use_Constraint_Aware_IK: true

MoveIt_Workspace:

Center:

X: 0

Y: 0

Z: 0

Size:

X: 2

Y: 2

Z: 2

Name: MotionPlanning

Planned Path:

Color Enabled: false

Interrupt Display: false

Links:

All Links Enabled: true

Expand Joint Details: false

Expand Link Details: false

Expand Tree: false

Link Tree Style: Links in Alphabetic Order

Loop Animation: true

Robot Alpha: 0.5

Robot Color: 150; 50; 150

Show Robot Collision: false

Show Robot Visual: true

Show Trail: false

State Display Time: 0.05 s

Trail Step Size: 1

Trajectory Topic: display_planned_path

Planning Metrics:

Payload: 1

Show Joint Torques: false

Show Manipulability: false

Show Manipulability Index: false

Show Weight Limit: false

TextHeight: 0.07999999821186066

Planning Request:

Colliding Link Color: 255; 0; 0

Goal State Alpha: 1

Goal State Color: 250; 128; 0

Interactive Marker Size: 0.029999999329447746

Joint Violation Color: 255; 0; 255

Planning Group: arm

Query Goal State: true

Query Start State: false

Show Workspace: false

Start State Alpha: 1

Start State Color: 0; 255; 0

Planning Scene Topic: monitored_planning_scene

Robot Description: robot_description

Scene Geometry:

Scene Alpha: 0.8999999761581421

Scene Color: 50; 230; 50

Scene Display Time: 9.999999747378752e-05

Show Scene Geometry: true

Voxel Coloring: Z-Axis

Voxel Rendering: Occupied Voxels

Scene Robot:

Attached Body Color: 150; 50; 150

Links:

All Links Enabled: true

Expand Joint Details: false

Expand Link Details: false

Expand Tree: false

Link Tree Style: Links in Alphabetic Order

Robot Alpha: 1

Show Robot Collision: false

Show Robot Visual: true

Value: true

Velocity_Scaling_Factor: 0.1

- Class: rviz_default_plugins/Marker

Enabled: true

Name: Vida Hedef (preview)

Topic:

Depth: 5

Durability Policy: Volatile

History Policy: Keep Last

Reliability Policy: Reliable

Value: /vida/preview_marker

Namespaces: {}

Value: true

Enabled: true

Global Options:

Background Color: 48; 48; 48

Fixed Frame: robot_base

Frame Rate: 30

Name: root

Tools:

- Class: rviz_default_plugins/Interact

Hide Inactive Objects: true

- Class: rviz_default_plugins/MoveCamera

- Class: rviz_default_plugins/Select

- Class: rviz_default_plugins/FocusCamera

Transformation:

Current:

Class: rviz_default_plugins/TF

Value: true

Views:

Current:

Class: rviz_default_plugins/Orbit

Distance: 0.5159747004508972

Enable Stereo Rendering:

Value: false

Focal Point:

X: 0

Y: 0

Z: 0.30000001192092896

Focal Shape Fixed Size: true

Focal Shape Size: 0.05000000074505806

Invert Z Axis: false

Name: Current View

Near Clip Distance: 0.00999999776482582

Pitch: 0.8799999952316284

Target Frame: <Fixed Frame>

Value: Orbit (rviz_default_plugins)

Yaw: 5.858185291290283

Saved: null

Window Geometry:

Displays:

collapsed: false

Height: 700

Hide Left Dock: false

Hide Right Dock: true

Width: 1024

X: 0

Y: 24

src/mycobot_calibration/config/charuco_params.yaml

```
# ChArUco-Board-Parameter – muss mit dem physischen Board des Benutzers übereinstimmen.

#

# Diese Datei beschreibt die Geometrie des gedruckten ChArUco-Kalibrierusters.

# Der charuco_detector-Node rekonstruiert daraus per solvePnP die 6-DoF-Pose

# des Boards relativ zur Kamera. Sind die Maße hier falsch, ist die geschätzte

# Pose maßstäblich verzerrt und die gesamte Hand-Auge-Kalibrierung wird ungenau.

#

# Board (vom Benutzer am 2026-05-22 mit dem Messschieber vermessen):

#   - 5 x 5 ChArUco (Anzahl der Schachbrettfelder in X- und Y-Richtung)

#   - Feldkantenlänge (Schachbrettfeld): 18 mm

#   - Markerkantenlänge (schwarzes Bitfeld des ArUco): 12 mm -> Verhältnis 0.667

#   - Außenmaß: ~90 x 89 mm

#   - Wörterbuch: DICT_4X4_50 (per bildbasierter Detektion 12 Marker, IDs 0-11)

#

# WENN DU ÄNDERST: diese Datei und der (ggf.) neue Board-Ausdruck müssen synchron sein.

# Falsche Feld-/Markerlänge = maßstäblich fehlerhafte Kalibrierung.

/**:

  ros__parameters:

    # squares_x / squares_y: Anzahl der Schachbrettfelder je Achse. Bestimmt zusammen

    # mit square_length die physische Boardgröße und die Zahl der inneren Ecken.

    squares_x: 5

    squares_y: 5

    # square_length_m: Kantenlänge eines Schachbrettfeldes in Metern. Setzt den

    # absoluten Maßstab der PnP-Pose – der wichtigste Wert für die Genauigkeit.

    square_length_m: 0.018

    # marker_length_m: Kantenlänge des schwarzen ArUco-Markers innerhalb der

    # weißen Felder. Muss < square_length_m sein (hier Verhältnis 12/18 = 0.667).

    marker_length_m: 0.012

    # aruco_dict: verwendetes ArUco-Wörterbuch. Muss exakt dem Wörterbuch

    # entsprechen, mit dem das Board generiert/gedruckt wurde, sonst keine Detektion.

    aruco_dict: DICT_4X4_50
```

src/mycobot_calibration/config/charuco_params_gripper.yaml

```
# ChArUco 6x3 Greifer-Board-Parameter – muss mit dem gedruckten PDF übereinstimmen.

#

# Dieses kleinere, längliche 6x3-Board wird am Greifer angebracht und dient der
# eye-in-hand-artigen Nachkalibrierung bzw. der Verifikation der Greifer-Spitze.
# Erzeugt von generate_charuco_gripper.py. Ändert sich ein Wert, muss das Board
# neu generiert und ausgedruckt werden.
# Bei einer Nachkalibrierung den Detector mit DIESER Datei starten (statt des alten 5x5).

/**:

ros__parameters:

  # squares_x / squares_y: 6 Felder in X-, 3 in Y-Richtung – längliches Format,
  # das flach auf die schmale Greifer-Oberfläche passt.

  squares_x: 6

  squares_y: 3

  # square_length_m: Kantenlänge eines Schachbrettfeldes in Metern (setzt den Maßstab).
  # 2026-08-30: Neuer Ausdruck auf 3D-gedruckter Halteplatte vermessen (Messschieber):
  # lange Kante 96.5mm/6 = 16.08mm, kurze Kante 47mm/3 = 15.67mm – Druck leicht
  # anisotrop, Drucker liess sich nicht auf 100% zwingen (Nutzer-Entscheid: so verwenden).
  # OpenCV erlaubt nur EINEN Wert -> Mittelwert 15.875mm; Rest-Fehler max ~1.3%/Achse,
  # wird in der Verifikationsphase (calib_check + Offset-Messung) aufgefangen.

  square_length_m: 0.015875

  # marker_length_m: Kantenlänge des ArUco-Markers innerhalb der Felder (< square_length_m).
  # Mit dem gleichen Druck-Massstab skaliert: 12mm * (15.875/16) = 11.9mm.

  marker_length_m: 0.0119

  # aruco_dict: ArUco-Wörterbuch; muss dem beim Generieren verwendeten entsprechen.

  aruco_dict: DICT_4X4_50
```

src/mycobot_calibration/launch/calibrate_eye_on_base.launch.py

```
# Hand-Auge-Kalibrierung – eye-on-base (Kamera ortsfest, Board am Roboter/Greifer).

# Setzt voraus, dass demo.launch.py (Roboter + MoveIt + D435i-Kamera) BEREITS läuft; dieser

# Launch fügt nur die kalibrierspezifischen Nodes HINZU, ohne Treiber/Roboter erneut zu

# starten (vermeidet Port-Konflikt am seriellen /dev/ttyTHS1 und Kamera-Doppelbelegung).

# Startet:

# 1. charuco_detector          -> publiziert TF camera_color_optical_frame -> charuco_board

# 2. easy_handeye2 calibrate.launch.py – Sample-Aufnahme + Lösung der AX=XB-Gleichung

# 3. joint_pose_gui           -> Gelenk-Slider, fährt Servos direkt (an MoveIt vorbei)


import os


from ament_index_python.packages import get_package_share_directory

from launch import LaunchDescription

from launch.actions import IncludeLaunchDescription

from launch.launch_description_sources import PythonLaunchDescriptionSource

from launch_ros.actions import Node


def generate_launch_description():

    pkg_share = get_package_share_directory('mycobot_calibration')

    # params_yaml: physische ChArUco-Board-Parameter (Feld-/Marker-Größe, Dictionary).

    params_yaml = os.path.join(pkg_share, 'config', 'charuco_params_gripper.yaml')


    # charuco_detector: erkennt das Board im Farbbild und publiziert die gemessene Pose

    # camera_color_optical_frame -> charuco_board als TF (Tracking-Seite der Kalibrierung).

    charuco_detector = Node(

        package='mycobot_calibration',

        executable='charuco_detector',

        name='charuco_detector',

        output='screen',

        parameters=[

            params_yaml,

            # Frame-/Topic-Verdrahtung: Eingänge vom bereits laufenden RealSense-Treiber

            # (image + camera_info), Ausgabe-TF-Kette camera -> board.

            # min_corners=8 verwirft instabile Posen bei teilweise verdecktem Board.
```

```

        {'camera_frame': 'camera_color_optical_frame',

        'board_frame': 'charuco_board',

        'image_topic': '/camera/color/image_raw',

        'camera_info_topic': '/camera/color/camera_info',

        'publish_rate_hz': 5.0,

        'min_corners': 8},

    ],

)

# easy_handeye2_calibrate: rqt-Panel zum Sammeln der Pose-Paare und Berechnen der
# eye-on-base-Transformation (Kamera -> Roboterbasis).

easy_handeye2_calibrate = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(
        os.path.join(
            get_package_share_directory('easy_handeye2'),
            'launch', 'calibrate.launch.py',
        )
    ),
    launch_arguments={
        # name: Kennung, unter der das Ergebnis gespeichert/geladen wird.
        'name': 'mycobot_d435i_eob',

        # eye_on_base: Kamera fest, Board am Effektor – passt zur Standmontage der D435i.
        'calibration_type': 'eye_on_base',

        # Tracking-Seite (Kamera sieht Board) – identisch zur charuco_detector-TF-Kette.
        'tracking_base_frame': 'camera_color_optical_frame',
        'tracking_marker_frame': 'charuco_board',

        # Roboter-Seite (aus der laufenden demo.launch.py: robot_base -> tcp).
        'robot_base_frame': 'robot_base',
        'robot_effector_frame': 'tcp',
    }.items(),
)

# joint_pose_gui: Gelenk-Slider-GUI; der Bediener fährt jedes Gelenk direkt zum
# Controller (an MoveIt vorbei). So lässt sich eine Pose präzise einstellen und dann ein
# Sample nehmen, während die Servos gesperrt bleiben.

joint_pose_gui = Node(
    package='mycobot_calibration',

```

```
    executable='joint_pose_gui',  
    name='joint_pose_gui',  
    output='screen',  
)  
  
return LaunchDescription([charuco_detector, easy_handeye2_calibrate, joint_pose_gui])
```

src/mycobot_calibration/launch/calibrate_full.launch.py

```
# Phase 5 – Hand-eye-Kalibrierung kompletter Stack.

#

# Was es startet:

# 1. robot_state_publisher (URDF -> TF)

# 2. mycobot_bridge (liest seriell, publiziert /joint_states,

#                   hört auf /joint_commands und führt send_radians aus)

# 3. joint_state_publisher_gui (Slider-GUI -> /joint_commands remap)

# 4. realsense2_camera (D435i live stream)

# 5. charuco_detector (image -> camera->board TF)

# 6. easy_handeye2_calibrate (rqt sampling + compute UI)

# 7. RViz (vorhandene world.rviz config)

#

# Verwendung:

# export LC_ALL=C

# source /opt/ros/galactic/setup.bash

# source ~/ros2_ws/install/setup.bash

# ros2 launch mycobot_calibration calibrate_full.launch.py

#

# Die Servos bleiben IMMER GESPERRT. Gib per Slider eine Pose, der Roboter fährt dorthin+hält,

# klicke im rqt-Panel "Take sample", sammle mindestens 15 verschiedene Posen.

#

# WARNUNG: dieser Launch ERSETZT Phase 6 NICHT – nur ein minimaler Stack für die

# Phase-5-Kalibrierungssitzung. Nicht gleichzeitig mit demo.launch.py ausführen (Port-Konflikt).


import os

from ament_index_python.packages import get_package_share_directory

from launch import LaunchDescription

from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription

from launch.conditions import IfCondition

from launch.launch_description_sources import PythonLaunchDescriptionSource

from launch.substitutions import Command, LaunchConfiguration, PathJoinSubstitution

from launch_ros.actions import Node

from launch_ros.parameter_descriptions import ParameterValue

from launch_ros.substitutions import FindPackageShare
```

```

def generate_launch_description():

    pkg_cal = get_package_share_directory('mycobot_calibration')

    pkg_world = FindPackageShare('mycobot_world')

    pkg_moveit = FindPackageShare('mycobot_moveit_config')

    xacro_path = PathJoinSubstitution([pkg_world, 'urdf', 'mycobot_world.urdf.xacro'])

    # world.rviz – minimale Config ohne MoveIt (kein move_group, ruft das MotionPlanning-Panel
    # nicht auf). RobotModel + TF + Image Displays genügen.

    rviz_cfg = PathJoinSubstitution([pkg_world, 'rviz', 'world.rviz'])

    robot_description = {

        'robot_description': ParameterValue(

            Command(['xacro ', xacro_path]),

            value_type=str,

        ),

    }

    # robot_state_publisher: liest die URDF aus dem 'robot_description'-Parameter und
    # publiziert daraus den kompletten statischen + gelenkabhängigen TF-Baum. Er
    # abonniert /joint_states (von der Bridge) und wandelt jeden Gelenkwinkel in eine
    # TF-Transformation um. Für die Hand-Auge-Kalibrierung ist dies die Quelle der
    # robot_base -> tcp Kette, die easy_handeye2 als "Roboter-Seite" der AX=XB-Gleichung
    # verwendet.

    rsp = Node(

        package='robot_state_publisher',

        executable='robot_state_publisher',

        name='robot_state_publisher',

        output='screen',

        parameters=[robot_description],

    )

    # mycobot_bridge: die Hardware-Schnittstelle zum echten Roboter. Sie liest über die
    # serielle Schnittstelle die realen Encoder-Winkel und publiziert sie auf /joint_states;
    # gleichzeitig hört sie auf /joint_commands und sendet Zielposen per send_radians an
    # die Servos (Servos bleiben dabei bestromt/gesperrt).

```



```

bridge = Node(
    package='mycobot_calibration',
    executable='mycobot_bridge',
    name='mycobot_bridge',
    output='screen',
    parameters=[{
        # port/baudrate: der Jetson-UART /dev/ttyTHS1 bei 1 Mbaud – die von der
        # myCobot-Firmware erwartete Standard-Baudrate.
        'port': '/dev/ttyTHS1',
        'baudrate': 1000000,
        # rate_hz: Frequenz, mit der die Bridge /joint_states publiziert (10 Hz reicht
        # für die Kalibrierung, da nur bei stehendem Roboter Samples genommen werden).
        'rate_hz': 10.0,
        # command_speed: Servo-Fahrgeschwindigkeit im Freiraum (URDF-unabhängiger
        # Firmware-Wert 0..100). 30 ist ein ruhiger, aber nicht träger Kompromiss.
        'command_speed': 30,
        # min_command_interval_s: Mindestabstand zwischen zwei Fahrbefehlen (0.2 s), um
        # den seriellen Bus nicht zu überfluten und Firmware-Aussetzer zu vermeiden.
        'min_command_interval_s': 0.20,
    }],
)

# joint_state_publisher_gui: Slider-Fenster zum manuellen Einstellen jeder Gelenkpose.
# Es publiziert normalerweise auf /joint_states – per Remap wird die Ausgabe stattdessen
# auf /joint_commands umgeleitet, also genau das Topic, auf das die Bridge hört. So
# bewegt ein Slider-Zug den echten Roboter, während die Bridge die realen Encoder-Werte
# unabhängig auf /joint_states zurückmeldet.
jsp_gui = Node(
    package='joint_state_publisher_gui',
    executable='joint_state_publisher_gui',
    name='joint_state_publisher_gui',
    output='screen',
    parameters=[robot_description],
    remappings=[('/joint_states', '/joint_commands')],
)

# realsense2_camera: startet den D435i-Kameratreiber (inkludiert dessen rs_launch.py).

```

```

# Er publiziert den Farb-Stream auf /camera/color/image_raw und die Intrinsics auf
# /camera/color/camera_info – die Eingänge des charuco_detector.

realsense = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(
        os.path.join(
            get_package_share_directory('realsense2_camera'),
            'launch', 'rs_launch.py',
        )
    ),
    launch_arguments={
        'enable_color': 'true',

        # Die Kalibrierung benötigt nur den Farb-Stream. Der Tiefen-Stream ist reiner
        # Overhead (CPU/USB/RAM) und trug zum Einfrieren des Jetson bei – daher aus.
        'enable_depth': 'false',
        'pointcloud.enable': 'false',
    }.items(),
)

# charuco_detector: erkennt das ChArUco-Board im Farbbild und publiziert die gemessene
# Transformation camera_color_optical_frame -> charuco_board als TF. Das ist die
# "Tracking-Seite" der Kalibrierung. Die YAML enthält die physischen Board-Maße
# (Feldgröße, Marker-Größe, Dictionary); die Inline-Parameter verdrahten die Frames
# und Topics.

charuco = Node(
    package='mycobot_calibration',
    executable='charuco_detector',
    name='charuco_detector',
    output='screen',
    parameters=[
        os.path.join(pkg_cal, 'config', 'charuco_params_gripper.yaml'),
        # camera_frame/board_frame: Namen der zu publizierenden TF-Kette.
        # image_topic/camera_info_topic: Eingänge vom RealSense-Treiber.
        # publish_rate_hz: 5 Hz genügt bei statischer Sampling-Aufnahme.
        # min_corners: mind. 8 erkannte Board-Ecken, sonst wird die Pose verworfen
        # (verhindert instabile Schätzungen bei teilweise verdecktem Board).
        {'camera_frame': 'camera_color_optical_frame',
         'board_frame': 'charuco_board',

```

```

        'image_topic': '/camera/color/image_raw',

        'camera_info_topic': '/camera/color/camera_info',

        'publish_rate_hz': 5.0,

        'min_corners': 8},

    ],

)

# easy_handeye2 calibrate: startet die eigentliche Kalibrier-Pipeline (rqt-Panel zum
# Sample-Sammeln + Berechnen). Sie sammelt Paare aus Roboter-Pose (robot_base -> tcp) und
# Tracking-Pose (camera -> board) und löst die AX=XB Hand-Auge-Gleichung.
handeye = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(
        os.path.join(
            get_package_share_directory('easy_handeye2'),
            'launch', 'calibrate.launch.py',
        )
    ),
    launch_arguments={
        # name: eindeutige Kennung, unter der das Kalibrierergebnis gespeichert wird.
        'name': 'mycobot_d435i_eob',

        # calibration_type eye_on_base: Kamera ist ORTSFEST montiert (eye-to-hand), das
        # Board sitzt am Greifer/TCP – passend zur Standmontage der D435i in diesem Setup.
        'calibration_type': 'eye_on_base',

        # tracking_base_frame / tracking_marker_frame: die Tracking-Seite (Kamera sieht
        # Board), identisch mit der vom charuco_detector publizierten TF-Kette.
        'tracking_base_frame': 'camera_color_optical_frame',
        'tracking_marker_frame': 'charuco_board',

        # robot_base_frame / robot_effector_frame: die Roboter-Seite (robot_state_publisher
        # liefert robot_base -> tcp aus der URDF + /joint_states).
        'robot_base_frame': 'robot_base',
        'robot_effector_frame': 'tcp',
    }.items(),
)

# RViz ist OPTIONAL – auf dem Jetson Nano sättigt es (zusammen mit dem restlichen Stack)
# RAM/CPU und friert den Desktop ein. Standardmäßig aus; nur auf einer Maschine mit
# Reserven einschalten: use_rviz:=true

```

```

use_rviz = LaunchConfiguration('use_rviz')

# RViz-Node mit der minimalen world.rviz-Config; nur aktiv, wenn use_rviz:=true

# (IfCondition wertet die LaunchConfiguration aus).

rviz = Node(
    package='rviz2',
    executable='rviz2',
    name='rviz2',
    output='screen',
    arguments=['-d', rviz_cfg],
    condition=IfCondition(use_rviz),
)

# Reihenfolge im LaunchDescription: zuerst das use_rviz-Argument deklarieren, dann alle
# Nodes/Includes starten. ROS 2 startet sie parallel; der TF-Baum + Kamera-Stream sind
# kurz nach dem Start verfügbar, sodass easy_handeye2 direkt Samples nehmen kann.

return LaunchDescription([
    DeclareLaunchArgument('use_rviz', default_value='false'),
    rsp, bridge, jsp_gui, realsense, charuco, handeye, rviz,
])

```

src/mycobot_calibration/launch/calibrate_session.launch.py

```
# Phase-5-Kalibriersitzung – schlanker Stack OHNE easy_handeye2.

#

# Zweck: Vorbereitungs-/Prüf-Sitzung, in der der Roboter, die Kamera und der

# ChArUco-Detektor zusammen laufen und in RViz visuell kontrolliert werden, BEVOR die

# eigentliche easy_handeye2-Sample-Aufnahme (siehe calibrate_full.launch.py) gestartet

# wird. Hier läuft der Tiefen-Stream mit, damit man die Board-Sicht und den TF-Baum

# vollständig verifizieren kann.

#

# Startet: robot_state_publisher, mycobot_bridge, joint_state_publisher_gui (Slider),

# realsense2_camera, charuco_detector und RViz mit der world.rviz-Config.


import os


from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import Command, PathJoinSubstitution
from launch_ros.actions import Node
from launch_ros.parameter_descriptions import ParameterValue
from launch_ros.substitutions import FindPackageShare


def generate_launch_description():

    pkg_cal = get_package_share_directory('mycobot_calibration')

    pkg_world = FindPackageShare('mycobot_world')


    # xacro_path: die Welt-URDF (Roboter + Greifer + fest montierte Kamera). Sie wird beim

    # Start durch xacro zu vollständigem URDF-XML expandiert.

    xacro_path = PathJoinSubstitution([pkg_world, 'urdf', 'mycobot_world.urdf.xacro'])

    # rviz_cfg: minimale RViz-Config ohne MoveIt-Panel (RobotModel + TF + Image genügen).

    rviz_cfg = PathJoinSubstitution([pkg_world, 'rviz', 'world.rviz'])


    # robot_description: URDF als String-Parameter, den mehrere Nodes (rsp, jsp_gui) teilen.

    robot_description = {
```

```

    'robot_description': ParameterValue(
        Command(['xacro ', xacro_path]),
        value_type=str,
    ),
}

# robot_state_publisher: erzeugt aus URDF + /joint_states den TF-Baum (robot_base -> tcp).
rsp = Node(
    package='robot_state_publisher',
    executable='robot_state_publisher',
    name='robot_state_publisher',
    output='screen',
    parameters=[robot_description],
)

# mycobot_bridge: serielle Hardware-Schnittstelle; publiziert echte Encoder-Winkel auf
# /joint_states und führt Fahrbefehle von /joint_commands aus.
bridge = Node(
    package='mycobot_calibration',
    executable='mycobot_bridge',
    name='mycobot_bridge',
    output='screen',
    parameters=[{
        # port/baudrate: Jetson-UART /dev/ttyTHS1 bei 1 Mbaud (Firmware-Standard).
        'port': '/dev/ttyTHS1',
        'baudrate': 1000000,
        # rate_hz: 10 Hz Publikationsrate der Gelenkzustände.
        'rate_hz': 10.0,
        # command_speed: ruhige Fahrgeschwindigkeit der Servos (0..100).
        'command_speed': 30,
        # min_command_interval_s: Drosselung der Fahrbefehle, schützt den seriellen Bus.
        'min_command_interval_s': 0.20,
        # freehand=False: Servos bleiben bestromt/gesperrt (kein Handführ-Modus), damit
        # eine per Slider angefahrene Pose für die Sample-Aufnahme stabil gehalten wird.
        'freehand': False,
    }],
)

```

```
# joint_state_publisher_gui: Slider-GUI; Ausgabe auf /joint_commands remappt, sodass ein  
# Slider-Zug den echten Roboter fährt (die Bridge meldet die realen Werte auf  
# /joint_states zurück).
```

```
jsp_gui = Node(  
    package='joint_state_publisher_gui',  
    executable='joint_state_publisher_gui',  
    name='joint_state_publisher_gui',  
    output='screen',  
    parameters=[robot_description],  
    remappings=[('/joint_states', '/joint_commands')],  
)
```

```
# realsense2_camera: D435i-Treiber. Hier MIT Tiefen-Stream (enable_depth=true), damit die  
# Board-Distanz und Kamera-Sicht in dieser Prüfsitzung vollständig kontrollierbar sind.
```

```
realsense = IncludeLaunchDescription(  
    PythonLaunchDescriptionSource(  
        os.path.join(  
            get_package_share_directory('realsense2_camera'),  
            'launch', 'rs_launch.py',  
        )  
    ),  
    launch_arguments={  
        'enable_color': 'true',  
        'enable_depth': 'true',  
        'pointcloud.enable': 'false',  
    }.items(),  
)
```

```
# charuco_detector: erkennt das ChArUco-Board im Farbbild und publiziert  
# camera_color_optical_frame -> charuco_board als TF (Tracking-Seite).
```

```
charuco = Node(  
    package='mycobot_calibration',  
    executable='charuco_detector',  
    name='charuco_detector',  
    output='screen',  
    parameters=[
```

```

    os.path.join(pkg_cal, 'config', 'charuco_params_gripper.yaml'),

    # Frame-/Topic-Verdrahtung wie in calibrate_full.launch.py; min_corners=8 verwirft

    # instabile Board-Posen bei Teilverdeckung.

    {'camera_frame': 'camera_color_optical_frame',

     'board_frame': 'charuco_board',

     'image_topic': '/camera/color/image_raw',

     'camera_info_topic': '/camera/color/camera_info',

     'publish_rate_hz': 5.0,

     'min_corners': 8},

    ],

)

# RViz: hier IMMER an (im Gegensatz zu calibrate_full), da diese Sitzung gerade der

# visuellen Kontrolle dient. Lädt die world.rviz-Config.

rviz = Node(

    package='rviz2',

    executable='rviz2',

    name='rviz2',

    output='screen',

    arguments=['-d', rviz_cfg],

)

return LaunchDescription([rsp, bridge, jsp_gui, realsense, charuco, rviz])

```


src/mycobot_calibration/launch/calibrate_standalone.launch.py

```
# Eigenständiger Basis-Stack – Roboter + Slider-GUI + D435i-Kamera OHNE Detektor/handeye.

#

# Zweck: nacktes Grundgerüst zum manuellen Fahren des Roboters und zur Kamera-Vorschau.

# Nützlich zum Testen der Hardware (serieller Bus, Kamera-Stream, TF-Baum), bevor die

# eigentlichen Kalibrier-Nodes hinzugeschaltet werden. Weder charuco_detector noch

# easy_handeye2 laufen hier.

#

# Startet: robot_state_publisher, mycobot_bridge, joint_state_publisher_gui, realsense2_camera.


import os


from ament_index_python.packages import get_package_share_directory

from launch import LaunchDescription

from launch.actions import IncludeLaunchDescription

from launch.launch_description_sources import PythonLaunchDescriptionSource

from launch.substitutions import Command, PathJoinSubstitution

from launch_ros.actions import Node

from launch_ros.parameter_descriptions import ParameterValue

from launch_ros.substitutions import FindPackageShare


def generate_launch_description():

    pkg_world = FindPackageShare('mycobot_world')

    # xacro_path: Welt-URDF (Roboter + Greifer + fest montierte Kamera), zur Laufzeit expandiert.
    xacro_path = PathJoinSubstitution([pkg_world, 'urdf', 'mycobot_world.urdf.xacro'])

    # robot_description: expandiertes URDF als geteilter String-Parameter (rsp + jsp_gui).
    robot_description = {
        'robot_description': ParameterValue(
            Command(['xacro ', xacro_path]),
            value_type=str,
        ),
    }
}
```

```

# robot_state_publisher: baut aus URDF + /joint_states den TF-Baum.

rsp = Node(
    package='robot_state_publisher',
    executable='robot_state_publisher',
    name='robot_state_publisher',
    output='screen',
    parameters=[robot_description],
)

# mycobot_bridge: serielle Hardware-Schnittstelle; publiziert /joint_states und führt
# Fahrbefehle von /joint_commands aus.

bridge = Node(
    package='mycobot_calibration',
    executable='mycobot_bridge',
    name='mycobot_bridge',
    output='screen',
    parameters=[{
        # port/baudrate: Jetson-UART /dev/ttyTHS1 bei 1 Mbaud (Firmware-Standard).
        'port': '/dev/ttyTHS1',
        'baudrate': 1000000,
        # rate_hz: 10 Hz Publikationsrate der Gelenkzustände.
        'rate_hz': 10.0,
        # command_speed: ruhige Servo-Fahrgeschwindigkeit (0..100).
        'command_speed': 30,
        # min_command_interval_s: Drosselung der Fahrbefehle, schützt den seriellen Bus.
        'min_command_interval_s': 0.20,
    }],
)

# joint_state_publisher_gui: Slider-GUI; Ausgabe auf /joint_commands remappt, sodass ein
# Slider-Zug den echten Roboter fährt.

jsp_gui = Node(
    package='joint_state_publisher_gui',
    executable='joint_state_publisher_gui',
    name='joint_state_publisher_gui',
    output='screen',
    parameters=[robot_description],
)

```

```

    remappings=[('/joint_states', '/joint_commands')],
)

# realsense2_camera: D435i-Treiber. Hier wird zusätzlich ein festes Farbprofil erzwungen
# (640x480x30) – höhere Auflösung als der Jetson-Sparbetrieb, da dieser Standalone-Stack
# ohne Detektor/handeye leichter ist und die USB-/CPU-Reserven für die Vorschau reichen.
realsense = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(
        os.path.join(
            get_package_share_directory('realsense2_camera'),
            'launch', 'rs_launch.py',
        )
    ),
    launch_arguments={
        'enable_color': 'true',
        'enable_depth': 'true',
        'pointcloud.enable': 'false',
        'color_module.profile': '640x480x30',
    }.items(),
)

return LaunchDescription([rsp, bridge, jsp_gui, realsense])

```

```
| src/mycobot_calibration/mycobot_calibration/__init__.py
```

src/mycobot_calibration/mycobot_calibration/charuco_detector.py

```
#!/usr/bin/env python3

# ChArUco-Board-Detektor – publiziert die Transformation camera_optical_frame -> charuco_board als TF.

# Wird von easy_handeye2 als tracking_marker-Quelle verwendet: aus dem laufenden

# Kamerabild wird die 6-DoF-Pose des ChArUco-Boards geschätzt und im TF-Baum

# bereitgestellt, sodass die Hand-Auge-Kalibrierung Samples aufnehmen kann.

#

# cv_bridge wird bewusst NICHT verwendet: das System-cv_bridge auf diesem Jetson

# ist gegen das aktualisierte OpenCV (4.8) inkompatibel/defekt. Daher konvertieren

# wir sensor_msgs/Image direkt in ein numpy-Array (siehe image_msg_to_bgr).


import math

import time

from typing import Optional, Tuple


import cv2

import cv2.aruco as aruco

import numpy as np


import rclpy

from rclpy.node import Node

from rclpy.duration import Duration

from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy, qos_profile_sensor_data


from sensor_msgs.msg import Image, CameraInfo

from std_msgs.msg import Int32

from geometry_msgs.msg import TransformStamped

from tf2_ros import StaticTransformBroadcaster


_ARUCO_DICT_BY_NAME = {

    'DICT_4X4_50': aruco.DICT_4X4_50,

    'DICT_4X4_100': aruco.DICT_4X4_100,

    'DICT_4X4_250': aruco.DICT_4X4_250,

    'DICT_5X5_50': aruco.DICT_5X5_50,

    'DICT_5X5_100': aruco.DICT_5X5_100,
```

```

'DICT_5X5_250': aruco.DICT_5X5_250,
'DICT_6X6_50': aruco.DICT_6X6_50,
'DICT_6X6_100': aruco.DICT_6X6_100,
'DICT_6X6_250': aruco.DICT_6X6_250,
}

```

```
def image_msg_to_bgr(msg: Image) -> np.ndarray:
```

```
    """Wandelt eine sensor_msgs/Image direkt in ein BGR-numpy-Bild um.
```

```

    Ersatz für cv_bridge (auf diesem Jetson defekt). Der rohe Byte-Puffer der
    Nachricht wird ohne Kopie als uint8-Array interpretiert und je nach Encoding
    in die von OpenCV erwartete BGR-Reihenfolge gebracht. Unterstützt werden die
    von der RealSense D435i gelieferten Encodings bgr8, rgb8 und mono8.

```

```
    """
```

```
    arr = np.frombuffer(msg.data, dtype=np.uint8)
```

```
    enc = msg.encoding.lower()
```

```
    if enc == 'bgr8':
```

```
        return arr.reshape((msg.height, msg.width, 3)).copy()
```

```
    if enc == 'rgb8':
```

```
        return cv2.cvtColor(
```

```
            arr.reshape((msg.height, msg.width, 3)), cv2.COLOR_RGB2BGR
```

```
        )
```

```
    if enc == 'mono8':
```

```
        return cv2.cvtColor(arr.reshape((msg.height, msg.width)), cv2.COLOR_GRAY2BGR)
```

```
    raise ValueError(f'Unsupported image encoding: {msg.encoding}')
```

```
def bgr_to_image_msg(bgr: np.ndarray, header) -> Image:
```

```
    """Verpackt ein BGR-numpy-Bild wieder in eine sensor_msgs/Image.
```

```

    Gegenrichtung zu image_msg_to_bgr: dient zum Publizieren des annotierten
    Debug-Bildes (mit gezeichneten Ecken, Achsen und Statusbanner). Der
    übergebene header (Zeitstempel/frame_id des Originalbildes) wird übernommen,
    damit RViz das annotierte Bild korrekt zuordnet.

```

```
    """
```

```
    out = Image()
```

```

out.header = header

out.height = bgr.shape[0]

out.width = bgr.shape[1]

out.encoding = 'bgr8'

out.is_bigendian = 0

out.step = bgr.shape[1] * 3

out.data = bgr.tobytes()

return out

```

```

def rvec_tvec_to_quat_xyz(
    rvec: np.ndarray, tvec: np.ndarray
) -> Tuple[float, float, float, float, float, float, float]:
    """Wandelt das solvePnP-Ergebnis (rvec, tvec) in Translation + Quaternion um.

    solvePnP liefert die Orientierung als Rodrigues-Rotationsvektor rvec und die
    Position als Translationsvektor tvec. Für eine TF/geometry_msgs-Transformation
    wird jedoch ein Quaternion benötigt. rvec wird per cv2.Rodrigues in eine
    3x3-Rotationsmatrix R überführt und daraus mit Shepperds Methode ein
    numerisch stabiles Quaternion (qx, qy, qz, qw) berechnet. Rückgabe:
    (tx, ty, tz, qx, qy, qz, qw).
    """
    R, _ = cv2.Rodrigues(rvec)

    # OpenCV-Rotationsmatrix -> Quaternion (w, x, y, z) mit Shepperds Methode.
    # Es wird der Zweig mit der größten Diagonal-/Spurkomponente gewählt, um
    # eine Division durch (nahe) null und damit numerische Instabilität zu vermeiden.
    trace = R[0, 0] + R[1, 1] + R[2, 2]

    if trace > 0.0:
        s = 0.5 / math.sqrt(trace + 1.0)
        qw = 0.25 / s
        qx = (R[2, 1] - R[1, 2]) * s
        qy = (R[0, 2] - R[2, 0]) * s
        qz = (R[1, 0] - R[0, 1]) * s
    elif R[0, 0] > R[1, 1] and R[0, 0] > R[2, 2]:
        s = 2.0 * math.sqrt(1.0 + R[0, 0] - R[1, 1] - R[2, 2])
        qw = (R[2, 1] - R[1, 2]) / s
        qx = 0.25 * s

```

```

    qy = (R[0, 1] + R[1, 0]) / s
    qz = (R[0, 2] + R[2, 0]) / s
elif R[1, 1] > R[2, 2]:
    s = 2.0 * math.sqrt(1.0 + R[1, 1] - R[0, 0] - R[2, 2])
    qw = (R[0, 2] - R[2, 0]) / s
    qx = (R[0, 1] + R[1, 0]) / s
    qy = 0.25 * s
    qz = (R[1, 2] + R[2, 1]) / s
else:
    s = 2.0 * math.sqrt(1.0 + R[2, 2] - R[0, 0] - R[1, 1])
    qw = (R[1, 0] - R[0, 1]) / s
    qx = (R[0, 2] + R[2, 0]) / s
    qy = (R[1, 2] + R[2, 1]) / s
    qz = 0.25 * s
tx, ty, tz = float(tvec[0]), float(tvec[1]), float(tvec[2])
return tx, ty, tz, qx, qy, qz, qw

```

```

class CharucoDetector(Node):

```

```

    """ROS-2-Node zur ChArUco-Board-Detektion und TF-Publikation.

```

```

    Ablauf: abonniert das Farbbild und die CameraInfo der RealSense D435i,
    detektiert das ChArUco-Board im Bild, schätzt dessen Pose per solvePnP und
    publiziert die Transformation camera_frame -> board_frame als (statische) TF.
    Zusätzlich wird ein annotiertes Debug-Bild sowie die Anzahl erkannter Ecken
    veröffentlicht. Ein HOLD-Mechanismus republiziert die letzte gültige Pose
    mit frischem Zeitstempel, damit easy_handeye2 trotz der niedrigen
    Detektionsrate auf dem Jetson stabil Samples aufnehmen kann.

```

```

    """

```

```

    def __init__(self) -> None:

```

```

        super().__init__('charuco_detector')

```

```

        # Board-Geometrie – Standardwerte entsprechen config/charuco_params.yaml.

```

```

        # Werden zur Laufzeit über ROS-Parameter (bzw. die YAML-Datei) überschrieben.

```

```

        self.declare_parameter('squares_x', 5)

```

```

        self.declare_parameter('squares_y', 7)

```



```

self.declare_parameter('square_length_m', 0.030)

self.declare_parameter('marker_length_m', 0.022)

self.declare_parameter('aruco_dict', 'DICT_5X5_100')


# Frames + Topics: Namen der Koordinatensysteme (Kamera-optisch, Board)
# sowie der abonnierten Bild- und CameraInfo-Topics der RealSense.

self.declare_parameter('camera_frame', 'camera_color_optical_frame')

self.declare_parameter('board_frame', 'charuco_board')

self.declare_parameter('image_topic', '/camera/color/image_raw')

self.declare_parameter('camera_info_topic', '/camera/color/camera_info')


# Detektions-Drosselung (Hz) – die D435i streamt mit 30 Hz, für die
# Kalibrierung genügen aber wenige Hz an stabilen Detektionen. Begrenzt
# zugleich die CPU-Last auf dem Jetson.

self.declare_parameter('publish_rate_hz', 5.0)

# Mindestanzahl erkannter ChArUco-Ecken, bevor einer Pose vertraut wird.
# Zu wenige Ecken -> schlecht konditionierte, unzuverlässige PnP-Lösung.

self.declare_parameter('min_corners', 8)


# HOLD: Der Detektor-Durchsatz liegt auf dem Jetson unter Volllast bei
# ~0.7 fps. Dadurch veraltet die camera->board-TF zwischen zwei Detektionen
# und der "Take Sample"-Button in easy_handeye2 flackert an/aus. Solange das
# Board stillsteht (Servos während der Kalibrierung gesperrt), bleibt die
# letzte Pose gültig, daher republishieren wir sie mit gleichmäßiger Rate
# und frischem Zeitstempel. Trifft innerhalb von hold_timeout_s keine neue
# Detektion ein, stoppt das Republishieren – so kann ein aus dem Bild
# verschwundenes Board nicht fälschlich als veraltete Pose gesampelt werden.

self.declare_parameter('hold_timeout_s', 2.0)

self.declare_parameter('republish_rate_hz', 15.0)

# Zeitstempel um diesen Betrag in die Zukunft datieren. handeye_server
# schlägt die Transformation etwa bei now() nach; die frischeste Detektion
# ist aber bis zu eine Republish-Periode alt, sodass ein Lookup bei now()
# in die Zukunft extrapolieren müsste und tf2 dies ablehnt. Ein
# vorgezogener Stempel garantiert, dass der Puffer now() stets abdeckt.
# Unschädlich, solange das Board stillsteht (während des Samplings).

self.declare_parameter('stamp_lead_s', 0.3)

```



```

# STATISCHER Broadcaster: tf2 behandelt statische Transformationen als für
# ALLE Zeitpunkte gültig, sodass der exact-time-Lookup von handeye_server
# stets gelingt, obwohl der Detektor auf dem ausgelasteten Jetson nur
# ~0.6 fps liefert. Hier korrekt, weil Board + Roboter während des
# Samplings stillstehen.

self._tf_pub = StaticTransformBroadcaster(self)

self._corners_pub = self.create_publisher(Int32, '~/corners_detected', 10)


# Annotiertes Bild: grüne Ecken/Marker + Pose-Achsen + Bereit-zum-Aufnehmen-
# Banner. In RViz per Image-Display auf /charuco_detector/image_annotated
# abonnierbar (visuelle Kontrolle der Detektion).

self._annotated_pub = self.create_publisher(
    Image, '~/image_annotated', 10
)


# CameraInfo: latched-äquivalente QoS – RELIABLE, KEEP_LAST, Tiefe 1.
# Die Intrinsics ändern sich nicht, daher genügt die jeweils letzte Nachricht.
qos = QoSProfile(
    reliability=ReliabilityPolicy.RELIABLE,
    history=HistoryPolicy.KEEP_LAST,
    depth=1,
)

self.create_subscription(
    CameraInfo,
    self.get_parameter('camera_info_topic').value,
    self._on_camera_info,
    qos,
)


# RealSense2 RGB stream wird RELIABLE publiziert. Das sensor_data Preset
# (BEST_EFFORT) ist mit einem RELIABLE Publisher inkompatibel – DDS verbindet nicht.
# SystemDefault (RELIABLE/KEEP_LAST/10) passt.

self.create_subscription(
    Image,
    self.get_parameter('image_topic').value,
    self._on_image,
    10,

```

```

)

# Zwei Timer: periodische Statistik-Ausgabe und der HOLD-Republish-Tick.
self.create_timer(5.0, self._log_stats)

self.create_timer(1.0 / republish_rate, self._on_hold_tick)

self.get_logger().info(
    f'ChArUco {sx}x{sy} square={sq*1000:.1f}mm marker={mk*1000:.1f}mm '
    f'dict={dict_name} -> {self._camera_frame} -> {self._board_frame}'
)

def _on_camera_info(self, msg: CameraInfo) -> None:
    """Callback für CameraInfo: speichert die Kamera-Intrinsics K und D.

    Ohne K/D ist keine PnP-Poseschätzung möglich, daher wartet _on_image,
    bis diese Werte einmal empfangen wurden. Zusätzlich wird der optische
    Frame aus dem Nachrichten-Header übernommen (maßgeblich gegenüber dem Parameter).
    """
    self._K = np.array(msg.k, dtype=np.float64).reshape(3, 3)
    self._D = np.array(msg.d, dtype=np.float64)

    # Den OPTISCHEN Frame aus dem Nachrichten-Header verwenden – überschreibt
    # bei Bedarf den gesetzten Parameter, damit die TF im korrekten Frame hängt.
    if msg.header.frame_id and msg.header.frame_id != self._camera_frame:
        self.get_logger().info(
            f'camera_info frame_id="{msg.header.frame_id}" '
            f'overrides param "{self._camera_frame}"'
        )
        self._camera_frame = msg.header.frame_id

def _on_image(self, msg: Image) -> None:
    """Callback für jedes Kamerabild: Detektion, Poseschätzung, Publikation.

    Kernablauf des Nodes:

    1) Rate begrenzen (publish_rate_hz), Bild in BGR wandeln, in Graustufen konvertieren.
    2) Mit dem CharucoDetector die Board-Ecken (charuco_corners/ids) und die
       zugrunde liegenden ArUco-Marker detektieren.
    3) Genügend Ecken vorausgesetzt, per matchImagePoints 2D->3D-Korrespondenzen

```

```

        bilden und mit cv2.solvePnP (SOLVEPNP_ITERATIVE) die Board-Pose lösen.

4) Ein annotiertes Debug-Bild mit Markern, Ecken, Pose-Achsen und farbigem
    Statusbanner zeichnen und publizieren; Eckenzahl publizieren.

5) Bei erfolgreicher Pose die TF broadcasten und als HOLD-Pose merken.
"""

self._frames_seen += 1

now = time.monotonic()

# Ratenbegrenzung: nur alle _min_period Sekunden verarbeiten (spart CPU).
if (now - self._last_pub) < self._min_period:

    return

try:

    bgr = image_msg_to_bgr(msg)

except ValueError as e:

    self.get_logger().warn(str(e), throttle_duration_sec=5.0)

    return

gray = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)

status = 'WAITING FOR CAMERA INFO'

ok = False

rvec = tvec = None

charuco_corners = charuco_ids = None

marker_corners = marker_ids = None

# Nur schätzen, wenn die Kamera-Intrinsics bereits vorliegen.

if self._K is not None:

    # ChArUco-Board im Graustufenbild detektieren: liefert die inneren
    # Schachbrettecken (subpixelgenau) samt IDs sowie die ArUco-Marker.

    charuco_corners, charuco_ids, marker_corners, marker_ids = \

        self._detector.detectBoard(gray)

    n_corners = 0 if charuco_corners is None else len(charuco_corners)

    if n_corners < self._min_corners:

        # Zu wenige Ecken -> keine vertrauenswürdige Pose möglich.

        status = f'NO BOARD (corners {n_corners}/{self._min_corners})'

    else:

        # 2D-Bildpunkte den 3D-Board-Modellpunkten zuordnen (Korrespondenzen).

```

```

obj_points, img_points = self._board.matchImagePoints(
    charuco_corners, charuco_ids
)

if obj_points is None or len(obj_points) < 4:
    # solvePnP benötigt mindestens 4 Korrespondenzen.
    status = f'PARTIAL (corners {n_corners})'
else:
    # Perspective-n-Point lösen: liefert Rotations- (rvec) und
    # Translationsvektor (tvec) des Boards im Kamera-Frame.
    ok, rvec, tvec = cv2.solvePnP(
        obj_points, img_points, self._K, self._D,
        flags=cv2.SOLVEPNP_ITERATIVE,
    )

    if ok:
        status = (
            f'READY TO SAMPLE (corners {n_corners}) '
            f'-> easy_handeye2 rqt: "Take Sample"'
        )
    else:
        status = 'POSE SOLVE FAILED'

# Overlays zeichnen – grüne Marker, grüne Ecken, RGB-Pose-Achsen.
if marker_corners is not None and len(marker_corners) > 0:
    aruco.drawDetectedMarkers(
        bgr, marker_corners, marker_ids, borderColor=(0, 255, 0)
    )

if charuco_corners is not None and len(charuco_corners) > 0:
    aruco.drawDetectedCornersCharuco(
        bgr, charuco_corners, charuco_ids, cornerColor=(0, 255, 0)
    )

if ok:
    # Koordinatenachsen der geschätzten Board-Pose einzeichnen (Achsenlänge
    # = doppelte Feldkantenlänge), zur Sichtkontrolle der Orientierung.
    axis_len = float(self.get_parameter('square_length_m').value) * 2.0
    cv2.drawFrameAxes(bgr, self._K, self._D, rvec, tvec, axis_len, 3)

# Statusbanner – grün wenn bereit, gelb bei Teil-Detektion, sonst rot.

```

```

ready = ok

partial = (not ok) and (charuco_corners is not None) and len(charuco_corners) > 0

color = (0, 200, 0) if ready else ((0, 200, 200) if partial else (0, 0, 220))

h, w = bgr.shape[:2]

cv2.rectangle(bgr, (0, 0), (w, 36), color, thickness=-1)

cv2.putText(
    bgr, status, (10, 25), cv2.FONT_HERSHEY_SIMPLEX,
    0.7, (255, 255, 255), 2, cv2.LINE_AA,
)

# Annotiertes Debug-Bild und Anzahl erkannter Ecken publizieren.
self._annotated_pub.publish(bgr_to_image_msg(bgr, msg.header))

corners_msg = Int32()
corners_msg.data = n_corners if self._K is not None else 0
self._corners_pub.publish(corners_msg)

if ok:
    # Erfolgreiche Pose in Translation+Quaternion umrechnen, als letzte
    # gültige Pose (HOLD) merken, sofort broadcasten und mitzählen.
    self._last_tf = rvec_tvec_to_quat_xyz(
        rvec.flatten(), tvec.flatten()
    )
    self._last_detect_t = time.monotonic()
    self._broadcast_last()
    self._detections += 1

self._last_pub = now

def _broadcast_last(self) -> None:
    """Publiziert die gespeicherte Board-Pose als TransformStamped (TF).

    Der Zeitstempel wird bewusst mit der FRISCHEN Node-Clock gesetzt und NICHT
    aus dem Bild-Header übernommen: die RealSense-Hardware-Uhr driftet auf dem
    Jetson gegenüber der Systemuhr. handeye_server schlägt bei now()-200ms nach,
    daher muss im selben Uhren-Domain gestempelt werden, sonst schlägt der
    TF-Lookup fehl. Baut die Transformation camera_frame -> board_frame auf.

```

```

"""

if self._last_tf is None:

    return

tx, ty, tz, qx, qy, qz, qw = self._last_tf

t = TransformStamped()

t.header.stamp = self.get_clock().now().to_msg()

t.header.frame_id = self._camera_frame

t.child_frame_id = self._board_frame

t.transform.translation.x = tx

t.transform.translation.y = ty

t.transform.translation.z = tz

t.transform.rotation.x = qx

t.transform.rotation.y = qy

t.transform.rotation.z = qz

t.transform.rotation.w = qw

self._tf_pub.sendTransform(t)

def _on_hold_tick(self) -> None:

    """HOLD-Timer: hält die camera->board-TF zwischen seltenen Detektionen frisch.

    Republiziert die letzte gültige Pose mit neuem Zeitstempel, damit der
    "Take Sample"-Button in easy_handeye2 stabil bleibt, solange das Board
    stillgehalten wird. Sicherheitsabbruch: liegt die letzte Detektion länger
    als hold_timeout_s zurück (Board evtl. aus dem Bild), wird NICHT weiter
    republiziert – so kann keine veraltete Pose fälschlich gesampelt werden.
    """

    if self._last_tf is None:

        return

    if (time.monotonic() - self._last_detect_t) > self._hold_timeout:

        return

    self._broadcast_last()

def _log_stats(self) -> None:

    """Periodische Diagnose-Ausgabe: gesehene Frames, Detektionen, K geladen?"""

    self.get_logger().info(

        f'frames={self._frames_seen}  detections={self._detections} '

        f'K_loaded={self._K is not None}'

```


)

```
def main() -> None:
```

```
    """Einstiegspunkt: rclpy initialisieren, Node erzeugen und spinnen lassen.
```

```
    Bei Ctrl-C (KeyboardInterrupt) wird sauber heruntergefahren: Node zerstören  
    und rclpy beenden, damit keine hängenden Ressourcen zurückbleiben.
```

```
    """
```

```
    rclpy.init()
```

```
    node = CharucoDetector()
```

```
    try:
```

```
        rclpy.spin(node)
```

```
    except KeyboardInterrupt:
```

```
        pass
```

```
    finally:
```

```
        node.destroy_node()
```

```
        rclpy.shutdown()
```

```
if __name__ == '__main__':
```

```
    main()
```

src/mycobot_calibration/mycobot_calibration/estop_button.py

```
#!/usr/bin/env python3

# NOT-AUS Taster – eigenständiges, immer-im-Vordergrund Fenster.

#

# Großer roter Knopf -> publiziert /estop=true (STOP). Der Bridge hält sofort

# jede Bewegung an und blockiert weitere Kommandos. Grüner Knopf -> /estop=false

# (FREIGABE). Läuft UNABHAENGIG von RViz – bleibt aktiv, auch wenn RViz auf dem

# Jetson einfriert/abstürzt (deshalb eigenes Fenster statt nur RViz-Panel).

#

# /estop ist transient-local (gelatcht): ein später gestarteter Bridge übernimmt

# sofort den letzten Zustand.

#

# KETTEN-RUECKMELDUNG (Vorfall 2026-07-16): der Taster ZEIGT jetzt, ob der Stopp

# wirklich beim Bridge angekommen ist. estop_relay publiziert /estop_ack, sobald

# das Kommando auf dem Bridge-Socket zugestellt wurde. Solange kein Ack kommt,

# zeigt das Fenster "KEINE BRIDGE-ANTWORT" – dann sofort NETZSCHALTER benutzen!

# Merke: waehrend des Skript-AUFRAEUMENS (nach Ctrl+C) sind relay+bridge tot –

# in dieser Phase kann KEIN Software-Not-Aus wirken.

#

# Alle Meldungen werden zusaetzlich mit print(flush=True) geschrieben – die

# rcutils-stdout-Pufferung hatte die Log-Datei sonst leer gelassen (Forensik!).

#

# Start:  ros2 run mycobot_calibration estop_button

# Terminal-Alternative (immer verfügbar):

#  ros2 topic pub --once /estop std_msgs/msg/Bool "{data: true}"


import ctypes

import time

import tkinter as tk


import rclpy

from rclpy.node import Node

from rclpy.qos import QoSProfile, DurabilityPolicy, ReliabilityPolicy

from std_msgs.msg import Bool
```

```

def _mlock_current():
    """Bereits belegten Prozess-Speicher sperren (best effort) – unter RAM-Druck/
    Swap-Thrash auf dem Nano bleibt das Not-Aus-Fenster sonst sekundenlang
    eingefroren. NUR MCL_CURRENT (=1)! MCL_FUTURE liesse pthread_create der
    DDS-Threads mit EAGAIN scheitern (Test 2026-07-16: Publisher war dann tot) –
    deshalb erst NACH rclpy.init/Node-Aufbau aufrufen und Zukunft nicht sperren."""
    try:
        libc = ctypes.CDLL("libc.so.6", use_errno=True)

        if libc.mlockall(1) != 0:

            print("NOT-AUS: mlockall nicht moeglich (Limit/Rechte) – weiter ohne", flush=True)
    except Exception as e: # noqa: BLE001

        print(f"NOT-AUS: mlockall Fehler: {e} – weiter ohne", flush=True)


class EstopPublisher(Node):
    def __init__(self) -> None:
        super().__init__('estop_button')

        q = QoSProfile(depth=1)

        q.durability = DurabilityPolicy.TRANSIENT_LOCAL

        q.reliability = ReliabilityPolicy.RELIABLE

        self._pub = self.create_publisher(Bool, '/estop', q)

        # Ack vom estop_relay: Kommando wurde WIRKLICH auf den Bridge-Socket geschrieben.

        self.ack_time = 0.0

        self.ack_state = None

        self.create_subscription(Bool, '/estop_ack', self._on_ack, q)

    def _on_ack(self, msg: Bool) -> None:
        self.ack_time = time.monotonic()

        self.ack_state = bool(msg.data)

    def send(self, active: bool) -> None:
        self._pub.publish(Bool(data=active))

        txt = 'NOT-AUS -> STOP' if active else 'NOT-AUS -> FREIGABE'

        self.get_logger().warn(txt)

        print(txt, flush=True) # rcutils-stdout ist gepuffert; das hier landet sofort in der Log-Datei

```

```

def main() -> None:

    rclpy.init()

    node = EstopPublisher()

    _mlock_current()    # NACH dem DDS-Aufbau (siehe Docstring: MCL_FUTURE-Falle)

    print('NOT-AUS-Fenster BEREIT', flush=True)


    root = tk.Tk()

    root.title('NOT-AUS')

    root.attributes('-topmost', True)    # immer im Vordergrund

    root.resizable(False, False)

    root.configure(bg='black')


    status = tk.Label(root, text='BEREIT', font=('DejaVu Sans', 16, 'bold'),

                        fg='white', bg='black')

    # Ketten-Status: kam das Ack des Relays an? (leer solange nichts gesendet wurde)

    chain = tk.Label(root, text='', font=('DejaVu Sans', 11),

                      fg='gray', bg='black')


    state = {'sent': None, 'sent_time': 0.0}


    def do_stop() -> None:

        state['sent'] = True

        state['sent_time'] = time.monotonic()

        node.send(True)

        status.config(text='!!! GESTOPPT !!!', fg='red')

        chain.config(text='warte auf Bridge-Bestaetigung...', fg='yellow')


    def do_reset() -> None:

        state['sent'] = False

        state['sent_time'] = time.monotonic()

        node.send(False)

        status.config(text='FREIGEgeben', fg='lime')

        chain.config(text='warte auf Bridge-Bestaetigung...', fg='yellow')


    stop_btn = tk.Button(root, text='NOT-AUS\nSTOP', command=do_stop,

                          font=('DejaVu Sans', 34, 'bold'),

                          bg='red', fg='white', activebackground='#aa0000',

```

```

        activeforeground='white', width=12, height=3, bd=6)

reset_btn = tk.Button(root, text='FREIGABE (RESET)', command=do_reset,

                        font=('DejaVu Sans', 14, 'bold'),

                        bg='#004400', fg='lime', activebackground='#006600',

                        activeforeground='white', width=22, height=1, bd=3)

stop_btn.pack(padx=16, pady=(16, 8))

reset_btn.pack(padx=16, pady=(0, 8))

status.pack(padx=16, pady=(0, 4))

chain.pack(padx=16, pady=(0, 12))

# Leertaste / Escape als Tastatur-Not-Aus.
root.bind('<space>', lambda e: do_stop())

root.bind('<Escape>', lambda e: do_stop())

# rclpy einfädig (kein Hintergrund-Thread – Galactic pybind/thread-Falle vermeiden)
# über die Tk-Ereignisschleife bedienen.
def spin() -> None:
    rclpy.spin_once(node, timeout_sec=0.0)

    # Ketten-Rueckmeldung auswerten: Ack nach dem letzten Senden eingetroffen?
    if state['sent'] is not None:
        if node.ack_time >= state['sent_time'] and node.ack_state == state['sent']:
            chain.config(text='BRIDGE BESTAETIGT – Kommando zugestellt', fg='lime')
        elif time.monotonic() - state['sent_time'] > 2.0:
            chain.config(text='KEINE BRIDGE-ANTWORT – NETZSCHALTER BENUTZEN!',
                          fg='red')

    root.after(50, spin)

root.after(50, spin)

try:
    root.mainloop()
finally:
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':

```

| main()

src/mycobot_calibration/mycobot_calibration/joint_encoder_publisher.py

```
#!/usr/bin/env python3

# myCobot 280 JN – bidirectional serial bridge for hand-eye calibration.

#

# Ein einziger Prozess hält den Port /dev/ttyTHS1:

# * liest Encoder, publiziert /joint_states -> robot_state_publisher aktualisiert TF

# * hört auf das Topic /joint_commands, sendet mit send_radians an den Roboter

#

# Die Slider-GUI (joint_state_publisher_gui) publiziert auf /joint_commands

# (Remap in der Launch-Datei). Die Servos bleiben IMMER GESPERRT – wir geben sie

# nicht frei (Drift-Problem). Der Benutzer stellt mit den Slidern die Pose ein,

# hält die Servos gesperrt, easy_handeye2 nimmt das Sample.

#

# send_coords wird NIEMALS aufgerufen. send_radians wird nur für das vom Slider

# kommende Gelenkziel verwendet (Phase 5 Sonderausnahme – im DEVLOG vermerkt).


import threading

import time

import rclpy

from rclpy.node import Node

from rclpy.qos import QoSProfile, DurabilityPolicy, ReliabilityPolicy

from sensor_msgs.msg import JointState

from std_msgs.msg import Bool

from pymycobot import MyCobot280

# Gleiche Index-Reihenfolge wie pymycobot.get_radians().

URDF_ARM_JOINTS = [

    'joint2_to_joint1',

    'joint3_to_joint2',

    'joint4_to_joint3',

    'joint5_to_joint4',

    'joint6_to_joint5',

    'joint6output_to_joint6',
```

```

]

# Greifer-Gelenknamen – werden während der Kalibrierung auf 0 (geschlossen) gehalten.

# Die mimic-Gelenke folgen dem Hauptgelenk per URDF mimic, aber in Galactic

# erwartet robot_state_publisher alle Gelenke in /joint_states – daher

# publizieren wir alle explizit.

URDF_GRIPPER_JOINTS = [

    'gripper_controller',

    'gripper_base_to_gripper_left2',

    'gripper_left3_to_gripper_left1',

    'gripper_base_to_gripper_right3',

    'gripper_base_to_gripper_right2',

    'gripper_right3_to_gripper_right1',

]

# Bei Problemen mit der Encoder-Leserichtung auf -1 setzen (nach Sanity-Check in RViz).

ENCODER_SIGNS = [1, 1, 1, 1, 1, 1]

# send_radians Richtung hat dasselbe Problem – falls beim korrekten Slider-Senden ein Flip nötig ist, -1.

COMMAND_SIGNS = [1, 1, 1, 1, 1, 1]


class MyCobotBridge(Node):

    def __init__(self) -> None:

        super().__init__('mycobot_bridge')

        self.declare_parameter('port', '/dev/ttyTHS1')

        self.declare_parameter('baudrate', 1000000)

        self.declare_parameter('rate_hz', 10.0)

        self.declare_parameter('command_speed', 30) # 0-100 pymycobot speed

        self.declare_parameter('min_command_interval_s', 0.20) # 5 Hz max

        self.declare_parameter('freehand', False)

        port = self.get_parameter('port').value

        baud = int(self.get_parameter('baudrate').value)

        rate = float(self.get_parameter('rate_hz').value)

        self._speed = int(self.get_parameter('command_speed').value)

        self._min_cmd_dt = float(self.get_parameter('min_command_interval_s').value)

```



```

self._freehand = bool(self.get_parameter('freehand').value)

self.get_logger().info(f'Opening serial: {port} @ {baud}')
self._mc = MyCobot280(port, baud)
time.sleep(0.5)

if self._freehand:
    self._mc.release_all_servos()
    time.sleep(0.3)
    self.get_logger().info('FREEHAND MODE – Servos frei, es wird nur der Encoder gelesen')

radians = self._mc.get_radians()
if not isinstance(radians, list) or len(radians) != 6:
    raise RuntimeError(f'Unexpected get_radians(): {radians!r}')
self.get_logger().info(
    'Connected. Initial joints (rad): '
    + ', '.join(f'{r:+.3f}' for r in radians)
)

self._lock = threading.Lock()
self._last_cmd_t = 0.0
self._pending_cmd = None
self._last_sent_cmd = None
self._cmd_eps = 0.002

self._pub = self.create_publisher(JointState, '/joint_states', 10)
self.create_timer(1.0 / rate, self._on_read_tick)
self.create_timer(5.0, self._log_stats)

if not self._freehand:
    self.create_subscription(
        JointState, '/joint_commands', self._on_command, 10
    )
    self.create_timer(self._min_cmd_dt, self._on_command_tick)

self._reads = 0
self._fails = 0

```

```

self._cmds = 0

# --- NOT-AUS (Emergency Stop) ---

# /estop (Bool): true=STOP (hält jede Bewegung an + blockiert Kommandos),
# false=FREIGABE. Transient-local, damit ein spät gestarteter Bridge sofort
# den aktuellen Zustand kennt. stop() hält an OHNE Servos freizugeben
# (release_all_servos würde den Arm fallen lassen).

self._estop = False

_eq = QoSProfile(depth=1)

_eq.durability = DurabilityPolicy.TRANSIENT_LOCAL

_eq.reliability = ReliabilityPolicy.RELIABLE

self.create_subscription(Bool, '/estop', self._on_estop, _eq)

self.get_logger().info('NOT-AUS bereit: /estop (true=STOP, false=FREIGABE)')

def _on_estop(self, msg: Bool) -> None:

    if msg.data:

        if not self._estop:

            self.get_logger().warn(

                '*** NOT-AUS AKTIV *** – jede Bewegung gestoppt, Kommandos blockiert'

            )

            self._estop = True

            self._pending_cmd = None

            with self._lock:

                for _ in range(3):

                    try:

                        self._mc.stop()

                    except Exception as e:

                        self.get_logger().warn(f'stop() failed: {e}')

                        time.sleep(0.02)

        else:

            if self._estop:

                self.get_logger().info(

                    'NOT-AUS FREIGABE – Kommandos wieder aktiv (neue Pose senden)'

                )

                self._estop = False

                self._last_sent_cmd = None # erzwingt Senden des nächsten Kommandos

```

```

def _on_read_tick(self) -> None:

    with self._lock:

        try:

            radians = self._mc.get_radians()

            except Exception as e:

                self._fails += 1

                self.get_logger().warn(f'read failed: {e}', throttle_duration_sec=2.0)

            return

        if radians is None or isinstance(radians, int) or len(radians) != 6:

            self._fails += 1

            return

    msg = JointState()

    msg.header.stamp = self.get_clock().now().to_msg()

    msg.name = list(URDF_ARM_JOINTS) + list(URDF_GRIPPER_JOINTS)

    msg.position = (

        [float(r) * ENCODER_SIGNS[i] for i, r in enumerate(radians)]

        + [0.0] * len(URDF_GRIPPER_JOINTS)

    )

    self._pub.publish(msg)

    self._reads += 1

def _on_command(self, msg: JointState) -> None:

    if self._estop:

        return # NOT-AUS aktiv: Kommandos ignorieren

    # Im vom Slider-GUI gesendeten JointState können sowohl Arm- als auch Greifer-Gelenke sein;

    # nur die 6 Arm-Gelenke extrahieren.

    name_to_pos = dict(zip(msg.name, msg.position))

    cmd = []

    for j in URDF_ARM_JOINTS:

        if j not in name_to_pos:

            return # fehlendes Gelenk -> nicht senden

        cmd.append(name_to_pos[j])

    # Letztes Ziel speichern; der Timer fasst es zusammen und sendet (verhindert Slider-Spam).

    self._pending_cmd = [cmd[i] * COMMAND_SIGNS[i] for i in range(6)]

```

```

def _on_command_tick(self) -> None:

    if self._estop:

        self._pending_cmd = None # NOT-AUS aktiv: nichts senden

        return

    if self._pending_cmd is None:

        return

    now = time.monotonic()

    if (now - self._last_cmd_t) < self._min_cmd_dt:

        return

    cmd = self._pending_cmd

    self._pending_cmd = None

    # Wenn die Abweichung vom zuvor gesendeten Kommando kleiner als eps ist, überspringen –
    # damit bei konstant gehaltenem Slider kein dauerndes send_radians-Spam entsteht.

    if self._last_sent_cmd is not None:

        delta = max(abs(cmd[i] - self._last_sent_cmd[i]) for i in range(6))

        if delta < self._cmd_eps:

            return

    with self._lock:

        try:

            self._mc.send_radians(cmd, self._speed)

        except Exception as e:

            self.get_logger().warn(f'send_radians failed: {e}', throttle_duration_sec=2.0)

            return

    self._last_cmd_t = now

    self._last_sent_cmd = list(cmd)

    self._cmds += 1

    self.get_logger().info(

        'cmd -> ' + ', '.join(f'{c:+.3f}' for c in cmd)

    )

def _log_stats(self) -> None:

    self.get_logger().info(

        f'reads={self._reads} fails={self._fails} cmds_sent={self._cmds}'

    )

```

```
def main() -> None:

    rclpy.init()

    node = MyCobotBridge()

    try:

        rclpy.spin(node)

    except KeyboardInterrupt:

        pass

    finally:

        node.destroy_node()

        rclpy.shutdown()


if __name__ == '__main__':

    main()
```

src/mycobot_calibration/mycobot_calibration/joint_pose_gui.py

```
#!/usr/bin/env python3
```

```
"""Tkinter-GUI mit Gelenk-Schiebereglern für den myCobot 280 JN.
```

Zweck

Dieses Werkzeug umgeht MoveIt vollständig: Jeder Klick auf "Send" schickt ein FollowJointTrajectory-Goal direkt an den ``/arm\_controller``. Dadurch entfällt die gesamte Bewegungsplanung (Kollisionsprüfung, OMPL usw.), und der Roboter fährt exakt die per Regler eingestellten Gelenkwinkel an.

Motivation (Hand-Auge-Kalibrierung)

Für die Hand-Auge-Kalibrierung (eye-to-hand) muss der Roboter in vielen verschiedenen, präzise reproduzierbaren Posen stehen, während jeweils ein Kalibriermuster (z. B. ArUco/Charuco) von der Kamera erfasst wird. Ein direkter Trajektorien-Befehl hält die Servos anschließend unter Drehmoment (verriegelt), sodass der Arm während der Bildaufnahme absolut still steht. Das ist entscheidend, weil jede Restbewegung die Rotations-/Translations-Schätzung der Kalibrierung verfälschen würde.

Aufruf

```
ros2 run mycobot_calibration joint_pose_gui
```

(oder)

```
python3 joint_pose_gui.py
```

```
"""
```

```
import math
```

```
import threading # ROS-Spin läuft in eigenem Thread, damit die Tk-GUI nicht blockiert
```

```
import time
```

```
import tkinter as tk
```

```
from tkinter import ttk
```

```
import rclpy
```

```
from rclpy.action import ActionClient
```

```

from rclpy.node import Node

from builtin_interfaces.msg import Duration

from control_msgs.action import FollowJointTrajectory

from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint

from sensor_msgs.msg import JointState


# Gelenknamen in der Reihenfolge J1..J6, exakt wie sie im URDF/Controller
# definiert sind. Diese Strings MUESSEN mit den Namen im FollowJointTrajectory-
# Goal und in den /joint_states übereinstimmen, sonst ordnet der Controller
# die Positionen falsch zu.
ARM_JOINT_NAMES = [
    'joint2_to_joint1',
    'joint3_to_joint2',
    'joint4_to_joint3',
    'joint5_to_joint4',
    'joint6_to_joint5',
    'joint6output_to_joint6',
]

# Konservative, symmetrische Reglergrenze (+/- pi) für alle Gelenke.
# Bewusst grob gehalten: Die exakten (asymmetrischen) URDF-Grenzen werden hier
# NICHT durchgesetzt – der Anwender ist für sinnvolle Posen verantwortlich, da
# ohne MoveIt keine Grenzwert-/Kollisionsprüfung stattfindet.
JOINT_LIMIT_RAD = math.pi


class JointPoseNode(Node):
    """ROS-2-Knoten: Kapselt die Kommunikation mit dem Arm-Controller.

    Verantwortlichkeiten:
    * Halten eines Action-Clients für FollowJointTrajectory, um Ziel-Posen
      zu senden.
    * Abonnieren von ``/joint_states``, um die aktuelle Ist-Pose des Roboters
      zu kennen (für die Anzeige und die "Sync"-Funktion der GUI).

    Der Knoten enthält bewusst KEINE Tkinter-Logik – Trennung von ROS-Schicht

```

(dieser Knoten) und Präsentationsschicht (Klasse ``App``).

"""

```
def __init__(self):
    super().__init__('joint_pose_gui')

    # Action-Client auf den Standard-Endpunkt des JointTrajectoryController.
    self.action_client = ActionClient(
        self, FollowJointTrajectory, '/arm_controller/follow_joint_trajectory'
    )

    # Zwischenspeicher der zuletzt empfangenen Ist-Gelenkwinkel (J1..J6).
    self.current_positions = [0.0] * 6

    # Flag: True, sobald mindestens eine /joint_states-Nachricht kam.
    # Verhindert, dass die GUI mit undefinierten Nullwerten synchronisiert.
    self.joint_states_received = False

    self.create_subscription(
        JointState, '/joint_states', self._on_joint_states, 10
    )
```

```
def _on_joint_states(self, msg: JointState):
    """Callback für ``/joint_states``: aktualisiert die Ist-Positionen.

    Die Reihenfolge der Gelenke in der Nachricht ist NICHT garantiert
    (sie kann Greifer-Gelenke enthalten oder anders sortiert sein), daher
    wird die Zuordnung über den Gelenknamen vorgenommen und nicht über
    den Index.
```

"""

```
# Zuordnung über den Namen (Reihenfolge kann variieren)
for i, name in enumerate(ARM_JOINT_NAMES):
    if name in msg.name:
        idx = msg.name.index(name)

        # Schutz gegen inkonsistente Nachrichten, bei denen das
        # position-Array kürzer ist als das name-Array.
        if idx < len(msg.position):
            self.current_positions[i] = msg.position[idx]

self.joint_states_received = True
```

```
def send_goal(self, positions, duration_s):
```



```
"""Sendet eine Ein-Punkt-Trajektorie als Ziel an den Controller.
```

Es wird ein einziger JointTrajectoryPoint erzeugt: der Controller interpoliert selbst von der aktuellen Pose zum Ziel innerhalb der vorgegebenen Dauer ``duration\_s``. Eine längere Dauer bedeutet also eine langsamere, sanftere Bewegung – wichtig, um die relativ schwachen Servos des myCobot 280 JN nicht zu überlasten.

```
:param positions: Iterable mit 6 Zielwinkeln [rad] in Reihenfolge J1..J6.
```

```
:param duration_s: Fahrzeit [s] von der Ist- zur Zielpose.
```

```
:return: True bei erfolgreichem Absenden, False falls der Action-Server  
        nicht erreichbar ist.
```

```
"""
```

```
# Kurzes Warten auf den Action-Server; schlägt fehl, wenn der
```

```
# Controller nicht läuft (z. B. bridge/ros2_control nicht gestartet).
```

```
if not self.action_client.wait_for_server(timeout_sec=2.0):
```

```
    self.get_logger().error('arm_controller action server not available')
```

```
    return False
```

```
traj = JointTrajectory()
```

```
traj.joint_names = list(ARM_JOINT_NAMES)
```

```
pt = JointTrajectoryPoint()
```

```
# Explizite float-Konvertierung: Tk-DoubleVar/Slider können numpy- oder
```

```
# int-artige Werte liefern, die die ROS-Serialisierung ablehnen würde.
```

```
pt.positions = [float(p) for p in positions]
```

```
# time_from_start = absoluter Zeitpunkt (relativ zum Trajektorienstart),
```

```
# zu dem dieser Punkt erreicht sein soll. Aufteilung in ganze Sekunden
```

```
# und Nanosekunden-Rest, wie es die builtin_interfaces/Duration verlangt.
```

```
pt.time_from_start = Duration(
```

```
    sec=int(duration_s),
```

```
    nanosec=int((duration_s - int(duration_s)) * 1e9),
```

```
)
```

```
traj.points = [pt]
```

```
goal = FollowJointTrajectory.Goal()
```

```
goal.trajectory = traj
```

```
# Asynchrones Senden: Wir warten NICHT auf das Ergebnis, damit die GUI
```

```
# reaktiv bleibt. Ein Feedback-/Result-Handling ist für diesen
```

```
# Kalibrier-Anwendungsfall nicht nötig.
```

```

self.action_client.send_goal_async(goal)

return True

```

```

class App:

```

```

    """Tkinter-Präsentationsschicht: baut die GUI und verknüpft sie mit dem Knoten.

```

```

    Aufbau der Oberfläche (von oben nach unten):

```

1. Spinbox für die Fahrzeit (Move duration).
2. Sechs Zeilen mit je einem Schieberegler pro Gelenk, dem Sollwert-Label und dem Istwert-Label.
3. Aktions-Buttons (Senden, Nullstellung, Synchronisieren).
4. Eine Statuszeile für Rückmeldungen.

```

    """

```

```

def __init__(self, root: tk.Tk, node: JointPoseNode):

    self.root = root

    self.node = node

    self.root.title('myCobot Joint Pose')

    # Parallele Listen, indiziert nach Gelenk J1..J6:

    self.sliders = []          # Tk-DoubleVars der Schieberegler (Sollwerte)
    self.value_labels = []     # Labels, die den Reglerwert numerisch zeigen
    self.current_labels = []   # Labels, die den Istwert aus /joint_states zeigen


    # --- Spinbox für die Fahrzeit -----
    # Bestimmt, wie lange die Bewegung zur Zielpose dauert (s. send_goal).
    top = ttk.Frame(root, padding=8)
    top.pack(fill='x')

    ttk.Label(top, text='Move duration (s):').pack(side='left')

    self.duration_var = tk.DoubleVar(value=3.0)

    ttk.Spinbox(top, from_=0.5, to=20.0, increment=0.5,
                textvariable=self.duration_var, width=6).pack(side='left', padx=4)


    # --- Ein Schieberegler pro Gelenk -----
    for i, name in enumerate(ARM_JOINT_NAMES):

        frame = ttk.Frame(root, padding=4)

        frame.pack(fill='x')

```

```

ttk.Label(frame, text=f'J{i+1} ({name})', width=30, anchor='w').pack(side='left')

var = tk.DoubleVar(value=0.0)

slider = ttk.Scale(frame, from_=-JOINT_LIMIT_RAD, to=JOINT_LIMIT_RAD,
                    variable=var, orient='horizontal', length=300,
                    # idx=i bindet den aktuellen Schleifenindex fest
                    # (Late-Binding-Falle bei Lambdas in Schleifen).
                    command=lambda v, idx=i: self._on_slider_change(idx, float(v)))

slider.pack(side='left', padx=4)

self.sliders.append(var)

# Zeigt den eingestellten SOLL-Wert des Reglers an.
value_label = ttk.Label(frame, text='+0.000 rad', width=14, anchor='e')
value_label.pack(side='left')

self.value_labels.append(value_label)

# Zeigt den IST-Wert (grau, bis Daten vorliegen; siehe _refresh_current).
current_label = ttk.Label(frame, text='cur: --', width=14,
                           anchor='e', foreground='gray')

current_label.pack(side='left')

self.current_labels.append(current_label)

# --- Aktions-Buttons -----
btn_frame = ttk.Frame(root, padding=8)

btn_frame.pack(fill='x')

ttk.Button(btn_frame, text='Send to robot',
            command=self.send).pack(side='left', padx=4)

ttk.Button(btn_frame, text='All zero',
            command=self.zero).pack(side='left', padx=4)

ttk.Button(btn_frame, text='Sync sliders ← current',
            command=self.sync_from_current).pack(side='left', padx=4)

self.status = ttk.Label(root, text='ready', padding=8, foreground='gray')

self.status.pack(fill='x')

# Zyklische Aktualisierung der Ist-Positions-Labels über den

```

```

# Tk-Event-Loop (kein eigener Thread nötig -> Tk ist nicht thread-safe).

self.root.after(200, self._refresh_current)

def _on_slider_change(self, idx, val):
    """Callback beim Verschieben eines Reglers: aktualisiert das Soll-Label."""
    self.value_labels[idx].config(text=f'{val:+.3f} rad')

def _refresh_current(self):
    """Periodischer Tk-Timer (alle 200 ms): schreibt die Ist-Winkel in die GUI.

    Liest die vom ROS-Thread gefüllten ``current_positions`` und stellt sie
    dar. Sobald echte Daten vorliegen, wechselt die Schriftfarbe von grau
    auf schwarz. Der Aufruf plant sich am Ende selbst neu ein (Tk-Idiom für
    wiederkehrende Aufgaben).
    """
    if self.node.joint_states_received:
        for i, v in enumerate(self.node.current_positions):
            self.current_labels[i].config(
                text=f'cur: {v:+.3f}', foreground='black'
            )
        self.root.after(200, self._refresh_current)

def send(self):
    """Liest alle Regler aus und sendet die Zielpose an den Roboter."""
    positions = [v.get() for v in self.sliders]

    # Untergrenze von 0.5 s erzwingen: zu kurze Fahrzeiten führen zu
    # ruckartigen, für die Servos schädlichen Bewegungen bzw. zu vom
    # Controller verworfenen Trajektorien.
    duration = max(0.5, float(self.duration_var.get()))

    if self.node.send_goal(positions, duration):
        self.status.config(
            text=f'Sent: [{", ".join(f'{p:+.3f}' for p in positions)}] in {duration:.1f}s',
            foreground='blue',
        )
    else:
        self.status.config(text='ERROR: action server not available',
                           foreground='red')

```

```

def zero(self):
    """Setzt alle Regler auf 0 rad und fährt die Nullstellung an."""
    for v in self.sliders:
        v.set(0.0)
    for i, lbl in enumerate(self.value_labels):
        lbl.config(text='+0.000 rad')
    self.send()

def sync_from_current(self):
    """Übernimmt die aktuelle Ist-Pose des Roboters in die Regler.

    Nützlich, um von der realen Stellung des Arms aus weiterzuarbeiten
    (statt von 0), ohne dass beim nächsten "Send" ein unerwarteter Sprung
    entsteht. Bricht ab, falls noch keine /joint_states empfangen wurden.
    """
    if not self.node.joint_states_received:
        self.status.config(text='no joint_states yet', foreground='red')
        return
    for i, v in enumerate(self.node.current_positions):
        self.sliders[i].set(v)
        self.value_labels[i].config(text=f'{v:+.3f} rad')
    self.status.config(text='sliders ← current pose', foreground='blue')

```

```

def ros_spin(node):
    """Ziel-Funktion des Hintergrund-Threads: verarbeitet ROS-Callbacks.

    ``rclpy.spin`` blockiert, solange der Knoten lebt. Es läuft deshalb in
    einem eigenen Thread, damit der Haupt-Thread ungestört den Tkinter-
    Event-Loop (``mainloop``) betreiben kann.
    """
    rclpy.spin(node)

```

```

def main():
    """Einstiegspunkt: initialisiert ROS, startet Spin-Thread und die GUI."""

```

```

rclpy.init()

node = JointPoseNode()

# ROS-Spin als Daemon-Thread: endet automatisch mit dem Hauptprogramm,
# ohne dass ein explizites Join nötig ist.

spin_thread = threading.Thread(target=ros_spin, args=(node,), daemon=True)

spin_thread.start()


# Kurz warten, bis die ersten /joint_states eintreffen, damit die GUI
# direkt mit der realen Roboter-Pose starten kann (siehe unten).

deadline = time.time() + 2.0

while not node.joint_states_received and time.time() < deadline:

    time.sleep(0.1)


root = tk.Tk()

app = App(root, node)

# Falls Ist-Daten vorliegen: Regler initial auf die aktuelle Pose setzen,
# um einen Sprung beim ersten "Send" zu vermeiden.

if node.joint_states_received:

    app.sync_from_current()

try:

    # Blockiert bis zum Schließen des Fensters.

    root.mainloop()

finally:

    # Sauberes Herunterfahren des ROS-Knotens, egal wie die GUI endet

    # (verhindert hängende Knoten / Ressourcenlecks).

    node.destroy_node()

    rclpy.shutdown()


if __name__ == '__main__':

    main()

```

src/mycobot_calibration/mycobot_calibration/numerical_ik.py

"""

myCobot 280 JN – Multi-Seed-Wrapper um MoveIts compute_ik-Service.

Grundidee

Die inverse Kinematik (IK) beantwortet die Frage: "Welche Gelenkwinkel $q = (q_1..q_6)$ führen den Endeffektor (TCP) an eine gewünschte kartesische Pose (Position + Orientierung)?" Für einen 6-DOF-Arm ist dieses Problem im Allgemeinen nichtlinear und kann mehrere, genau eine oder gar keine Lösung besitzen (Mehrdeutigkeit durch Ellenbogen-oben/-unten, Handgelenk-Flip usw.).

Statt eine eigene analytische/numerische IK zu implementieren, delegiert dieses Modul die Lösung an MoveIts ```/compute_ik```-Service (der intern einen IK-Solver wie KDL oder TRAC-IK verwendet). Der Solver ist jedoch nur ein lokaler, gradientenbasierter Löser: Er startet von EINEM Anfangswert (Seed) und konvergiert zur nächstgelegenen Lösung – oder scheitert, wenn er in einem schlechten Startbereich gefangen ist.

Multi-Seed-Strategie

Um die Erfolgsquote und die Lösungsqualität zu erhöhen, ruft dieser Wrapper den Solver mit MEHREREN, unterschiedlichen Startwerten (Seeds) auf und wählt anschließend die beste zurückgelieferte Lösung anhand einer Kostenfunktion (siehe ```solve_multi_seed```). So werden lokale Minima umgangen und eine gutmütige, zur aktuellen Pose passende Lösung bevorzugt.

Weitere Eigenschaften

- * Die Greifer-Orientierung wird fest auf "senkrecht nach unten" erzwungen (top-down grasp), was für das Aufnehmen von Objekten auf einer Ebene (Tisch/Stand) sinnvoll ist und einen Rotations-Freiheitsgrad festlegt.
- * FK- und IK-Konsistenz ist garantiert, da beide vom selben MoveIt-Modell stammen (identisches URDF/kinematisches Modell).

"""

```

import numpy as np

import rclpy

from rclpy.node import Node

from moveit_msgs.srv import GetPositionIK, GetPositionFK

from geometry_msgs.msg import PoseStamped

from sensor_msgs.msg import JointState


# Gelenknamen J1..J6 in kinematischer Reihenfolge (Basis -> TCP).
# Müssen mit dem URDF und der MoveIt-Gruppe 'arm' übereinstimmen, damit die
# vom Service zurückgegebenen Positionen korrekt zugeordnet werden können.
ARM_JOINTS = [
    'joint2_to_joint1', 'joint3_to_joint2', 'joint4_to_joint3',
    'joint5_to_joint4', 'joint6_to_joint5', 'joint6output_to_joint6',
]


# Untere/obere Gelenkgrenzen [rad] pro Achse J1..J6, entnommen aus dem URDF.
# Beachte: Diese Grenzen sind teils ASYMMETRISCH (z. B. J5). Sie werden hier
# zur Erzeugung plausibler Zufalls-Seeds genutzt; die eigentliche Einhaltung
# der Grenzen übernimmt der MoveIt-Solver selbst.
JOINT_LIMITS = [
    (-2.9321, 2.9321),
    (-2.4434, 2.4434),
    (-2.6179, 2.6179),
    (-2.6179, 2.6179),
    (-2.7052, 2.7925),
    (-3.14159, 3.14159),
]


# Ziel-Quaternion für die Greifer-Orientierung "senkrecht nach unten".
# KORRIGIERT 2026-07-16: der alte Wert (-0.7071, 0.7071, 0, 0) stammte per FK
# aus der Gelenkstellung [0,-1.57,0,-1.57,0,0] eines FRÜHEREN URDF-Stands.
# Nach den Greifer/tcp-Änderungen zeigt an dieser Referenzstellung KEINE
# tcp-Achse mehr nach unten (~48° gekippt) → /compute_ik lieferte für JEDE
# Position error -31 (NO_IK_SOLUTION), beide Klick/Jog-Werkzeuge wirkten kaputt.
# Aktuelles Modell: tcp liegt 110 mm in +Y von gripper_base – die ZEIGE-Richtung
# des Greifers ist die tcp-Y-Achse. "Senkrecht nach unten" = tcp-Y-Achse zeigt

```



```
# zum Boden = Rotation um X um -90°: (x,y,z,w) = (-sin45°, 0, 0, cos45°).

# Live verifiziert (TRAC-IK): löst am nahen Arbeitsbereich für ALLE Yaw-Winkel.

GRIPPER_DOWN_QUAT = (-0.70711, 0.0, 0.0, 0.70711) # x, y, z, w
```

```
class NumericalIK:
```

```
    """Dünner Wrapper um MoveIts IK-/FK-Services mit Multi-Seed-Auswahl.
```

```
    Hält persistente Service-Clients auf ``/compute_ik`` (inverse Kinematik)
    und ``/compute_fk`` (vorwärts Kinematik). Der zentrale Mehrwert liegt in
    ``solve_multi_seed``, das den lokalen Solver mit vielen Startwerten aufruft
    und die günstigste Lösung auswählt.
    """
```

```
def __init__(self, node: Node):
```

```
    # Referenz auf den aufrufenden ROS-Knoten (für Logging/Clients).

    self._node = node

    # Persistente Clients: einmal anlegen, mehrfach wiederverwenden, um
    # den Overhead wiederholter Client-Erzeugung zu vermeiden.

    self._ik_client = node.create_client(GetPositionIK, '/compute_ik')
    self._fk_client = node.create_client(GetPositionFK, '/compute_fk')
```

```
def _call_ik(self, target_xyz, seed_joints=None):
```

```
    """Einzelner IK-Aufruf: eine Zielposition + optionaler Seed -> Gelenkwinkel.

    Baut die GetPositionIK-Anfrage auf, sendet sie synchronisiert (blockierend)
    an MoveIt und dekodiert die Antwort in eine Liste [q1..q6].

    :param target_xyz: Zielposition (x, y, z) des TCP im Frame 'robot_base' [m].
    :param seed_joints: Optionaler Startwert (6 Gelenkwinkel), von dem aus der
                        lokale Solver iteriert. Ein guter Seed erhöht die
                        Wahrscheinlichkeit einer Lösung erheblich.

    :return: Liste von 6 Gelenkwinkeln [rad] bei Erfolg, sonst None
            (Service nicht verfügbar, Timeout oder IK-Fehlercode).
    """
```

```
    # Sicherstellen, dass der Service läuft (MoveIt gestartet?).

    if not self._ik_client.wait_for_service(timeout_sec=2.0):
```

```

    return None

req = GetPositionIK.Request()

req.ik_request.group_name = 'arm'      # Planungsgruppe aus der SRDF
req.ik_request.ik_link_name = 'tcp'    # Ziel-Link = Werkzeugspitze (TCP)

# Kollisionen ignorieren: Wir wollen eine reine kinematische Lösung.
# Die Kollisionsprüfung erfolgt später separat (z. B. bei der Planung).
req.ik_request.avoid_collisions = False

req.ik_request.timeout.sec = 1         # Rechenzeit-Budget des Solvers [s]

# Ziel-Pose zusammensetzen: Position aus dem Argument, Orientierung fest
# auf "senkrecht nach unten" (siehe GRIPPER_DOWN_QUAT).
pose = PoseStamped()

pose.header.frame_id = 'robot_base'    # Referenzrahmen der Zielkoordinaten
pose.pose.position.x = target_xyz[0]
pose.pose.position.y = target_xyz[1]
pose.pose.position.z = target_xyz[2]
pose.pose.orientation.x = GRIPPER_DOWN_QUAT[0]
pose.pose.orientation.y = GRIPPER_DOWN_QUAT[1]
pose.pose.orientation.z = GRIPPER_DOWN_QUAT[2]
pose.pose.orientation.w = GRIPPER_DOWN_QUAT[3]
req.ik_request.pose_stamped = pose

# Optionalen Seed als Roboterzustand übergeben. Der Greifer wird als
# zusätzliches Gelenk mitgeliefert (Position 0.0), damit der übergebene
# RobotState vollständig und für den Solver konsistent ist.
if seed_joints is not None:
    js = JointState()
    js.name = ARM_JOINTS + ['gripper_controller']
    js.position = list(seed_joints) + [0.0]
    req.ik_request.robot_state.joint_state = js

future = self._ik_client.call_async(req)

# Blockierendes Warten auf das Ergebnis. Dies ist thread-safe, weil der
# eigentliche ROS-Spin (der das Future erfüllt) in einem SEPARATEN
# Thread läuft; hier wird nur gepollt. Harte Obergrenze von 3 s

```

```

# verhindert ein Hängenbleiben, falls der Service nie antwortet.

import time

t0 = time.monotonic()

while not future.done() and (time.monotonic() - t0) < 3.0:

    time.sleep(0.01)

# Kein Ergebnis (Timeout) oder Aufruf fehlgeschlagen -> keine Lösung.

if not future.done() or future.result() is None:

    return None

res = future.result()

# error_code.val == 1 entspricht moveit_msgs::MoveItErrorCodes::SUCCESS.

# Jeder andere Wert (z. B. NO_IK_SOLUTION) bedeutet: keine Lösung.

if res.error_code.val != 1:

    return None


# Der Solver liefert die Gelenke evtl. in anderer Reihenfolge oder mit
# Zusatzgelenken zurück. Über ein Name->Position-Mapping extrahieren
# wir genau die sechs Armgelenke in der kanonischen Reihenfolge.

name_to_pos = dict(zip(res.solution.joint_state.name,

                       res.solution.joint_state.position))

return [name_to_pos.get(j, 0.0) for j in ARM_JOINTS]


def solve_multi_seed(self, target_xyz, gripper_down=True, current_joints=None,

                    n_seeds=10):

    """Löst die IK robust über mehrere Startwerte und wählt die beste Lösung.

    Ablauf:

    1. Analytische Schätzung für J1 (Basisdrehung) aus der Zielrichtung.

    2. Zusammenstellen einer Seed-Liste aus (a) handverlesenen, bewährten
       Posen, (b) optional der aktuellen Pose und (c) Zufalls-Seeds.

    3. Für jeden Seed den lokalen IK-Solver aufrufen.

    4. Jede gefundene Lösung mit einer Kostenfunktion bewerten und die
       günstigste zurückgeben.

    :param target_xyz: Zielposition (x, y, z) des TCP [m].

    :param gripper_down: Beibehalten aus Kompatibilitätsgründen; die
       Orientierung ist ohnehin fest nach unten erzwungen.

```

```

:param current_joints: Optionale aktuelle Gelenkstellung. Wird sowohl als
                        (guter) Seed genutzt als auch in der Kostenfunktion
                        bevorzugt, um kleine Bewegungen zu belohnen.

:param n_seeds: Mindestanzahl der auszuprobierenden Startwerte.

:return: Beste Gelenklösung [q1..q6] oder None, falls kein Seed konvergiert.
"""

# J1 (Basisdrehung) zeigt in Richtung der Ziel-Projektion in der
# XY-Ebene. atan2 liefert den Winkel des Zielvektors und damit einen
# sehr guten analytischen Startwert für das erste Gelenk.
j1_guess = np.arctan2(target_xyz[1], target_xyz[0])

# Handverlesene Start-Posen: mit dem geschätzten J1 kombiniert und
# typischen "Ellenbogen"-Konfigurationen (J2/J3/J4), die für top-down-
# Griffe auf dem Stand erfahrungsgemäß gut konvergieren. Der letzte
# Seed (Nullstellung) dient als neutraler Rückfall.
seeds = [
    [j1_guess, -0.5, -1.0, 0.0, 0.0, 0.0],
    [j1_guess, -1.0, -0.5, -0.5, 0.0, 0.0],
    [j1_guess, -0.3, -1.5, 0.5, 0.0, 0.0],
    [0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
]

# Die aktuelle Pose ganz vorne einfügen: Sie ist oft der beste Seed,
# weil die nächstgelegene Lösung meist auch die gewünschte ist.
if current_joints is not None:
    seeds.insert(0, current_joints)

# Auffüllen mit Zufalls-Seeds, bis n_seeds erreicht ist. Dies erhöht
# die Chance, auch schwierige Ziele zu lösen, indem der Lösungsraum
# breiter abgetastet wird (Umgehung lokaler Minima).
rng = np.random.default_rng()
while len(seeds) < n_seeds:
    # J1 nah an der analytischen Schätzung streuen (+/- 0.3 rad).
    seed = [j1_guess + rng.uniform(-0.3, 0.3)]

    # Restliche Gelenke innerhalb 40 % ihrer Grenzen zufällig wählen:
    # bewusst konservativ, um unrealistische, gestreckte Posen (die
    # selten konvergieren) zu vermeiden.

```

```

for lo, hi in JOINT_LIMITS[1:]:

    seed.append(rng.uniform(lo * 0.4, hi * 0.4))

seeds.append(seed)

best = None

best_cost = float('inf')

# Alle Seeds durchprobieren und die beste (kostenminimale) Lösung merken.
for seed in seeds:

    result = self._call_ik(target_xyz, seed_joints=seed)

    if result is not None:

        # Kostenterm 1: Abweichung von J1 zur analytischen Schätzung.

        # Auf das kürzere Winkelintervall normieren (Wrap-around bei pi),

        # damit z. B. +179 Grad und -179 Grad als "nah" gelten.

        j1_diff = abs(result[0] - j1_guess)

        if j1_diff > np.pi:

            j1_diff = 2 * np.pi - j1_diff

        # Gewichtete Gesamtkosten:

        # * j1_diff stark gewichtet (5.0): eine unerwartete

        #   Basisdrehung ist besonders unerwünscht.

        # * Summe der Beträge aller Gelenke schwach gewichtet (0.1):

        #   bevorzugt "kompakte", nicht ausladende Konfigurationen.

        cost = j1_diff * 5.0 + sum(abs(r) for r in result) * 0.1

        # Kostenterm 3 (nur falls Ist-Pose bekannt): Nähe zur aktuellen

        # Stellung belohnen -> minimiert die nötige Roboterbewegung und

        # vermeidet große, ruckartige Umkonfigurationen.

        if current_joints is not None:

            diff = sum(abs(a - b) for a, b in zip(result, current_joints))

            cost += diff * 0.3

        if cost < best_cost:

            best_cost = cost

            best = result

return best

```

src/mycobot_calibration/package.xml

```
<?xml version="1.0"?>

<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>

<package format="3">

  <name>mycobot_calibration</name>

  <version>0.1.0</version>

  <description>Hand-eye calibration helpers for the myCobot 280 JN + RealSense D435i (FAZ 5).</description>

  <maintainer email="ekinciorhandijvar@gmail.com">er</maintainer>

  <license>BSD-3-Clause</license>


  <buildtool_depend>ament_python</buildtool_depend>


  <exec_depend>roscpp</exec_depend>

  <exec_depend>tf2_ros</exec_depend>

  <exec_depend>sensor_msgs</exec_depend>

  <exec_depend>geometry_msgs</exec_depend>

  <exec_depend>std_msgs</exec_depend>

  <exec_depend>python3-opencv</exec_depend>

  <exec_depend>python3-numpy</exec_depend>

  <exec_depend>easy_handeye2</exec_depend>


  <export>

    <build_type>ament_python</build_type>

  </export>

</package>
```

src/mycobot_calibration/scripts/calibrate_handeye.py

```
#!/usr/bin/env python3
```

```
"""
```

Terminal-based hand-eye calibration sampler for myCobot 280 JN + D435i.

Board-Erkennung erfolgt durch charuco_detector Node (grünes Overlay in RViz).

Dieses Skript liest nur Posen aus TF, sammelt Samples und berechnet die Kalibrierung.

Usage:

Terminal 1: `ros2 launch mycobot_calibration calibrate_session.launch.py`

Terminal 2: `python3 ~/ros2_ws/src/mycobot_calibration/scripts/calibrate_handeye.py`

Keys (im Terminal):

- S – Sample nehmen (wenn Board vollständig erkannt)
- C – Kalibrierung berechnen (min 3 Samples, 10+ empfohlen)
- W – Ergebnis in YAML speichern
- D – letztes Sample löschen
- Q – beenden

Eye-on-base: Kamera fest, ChArUco Board am Roboterende.

```
"""
```

```
import math
```

```
import os
```

```
import select
```

```
import sys
```

```
import termios
```

```
import threading
```

```
import time
```

```
import tty
```

```
from typing import List, Optional, Tuple
```

```
import cv2
```

```
import numpy as np
```

```
import yaml
```

```

import rclpy

from rclpy.node import Node

from std_msgs.msg import Int32

from geometry_msgs.msg import TransformStamped

import tf2_ros


ROBOT_BASE_FRAME = 'robot_base'

ROBOT_EE_FRAME = 'tcp'

CAMERA_FRAME = 'camera_color_optical_frame'

BOARD_FRAME = 'charuco_board'


# 6x3 Greifer-ChArUco-Board -> (6-1)*(3-1) = 10 innere Ecken.

# 2026-08-30: Konstanten stammten noch vom alten 5x5-Board (16/14) – damit wurde

# JEDES Sample abgelehnt, weil das 6x3-Board maximal 10 Ecken liefern kann.

TOTAL_CORNERS = 10

MIN_CORNERS_FOR_SAMPLE = 9


OUTPUT_YAML = os.path.expanduser(

    '~/ros/easy_handeye/mycobot_handeye_eye_on_base.yaml'

)


HANDEYE_METHODS = {

    'tsai': cv2.CALIB_HAND_EYE_TSAI,

    'park': cv2.CALIB_HAND_EYE_PARK,

    'horaud': cv2.CALIB_HAND_EYE_HORAUD,

    'andreff': cv2.CALIB_HAND_EYE_ANDREFF,

    'daniilidis': cv2.CALIB_HAND_EYE_DANIILIDIS,

}


def tf_to_R_t(tf_stamped: TransformStamped) -> Tuple[np.ndarray, np.ndarray]:

    t = tf_stamped.transform.translation

    r = tf_stamped.transform.rotation

    tvec = np.array([t.x, t.y, t.z], dtype=np.float64)

    qx, qy, qz, qw = r.x, r.y, r.z, r.w

    R = np.array([

        [1 - 2*(qy*qy + qz*qz), 2*(qx*qy - qz*qw),      2*(qx*qz + qy*qw)],

```



```

    [2*(qx*qy + qz*qw),      1 - 2*(qx*qx + qz*qz), 2*(qy*qz - qx*qw)],
    [2*(qx*qz - qy*qw),      2*(qy*qz + qx*qw),      1 - 2*(qx*qx + qy*qy)],
], dtype=np.float64)

return R, tvec

```

```

def R_t_to_quaternion(R: np.ndarray, t: np.ndarray) -> dict:

    trace = R[0, 0] + R[1, 1] + R[2, 2]

    if trace > 0:

        s = 0.5 / math.sqrt(trace + 1.0)

        qw = 0.25 / s

        qx = (R[2, 1] - R[1, 2]) * s
        qy = (R[0, 2] - R[2, 0]) * s
        qz = (R[1, 0] - R[0, 1]) * s

    elif R[0, 0] > R[1, 1] and R[0, 0] > R[2, 2]:

        s = 2.0 * math.sqrt(1.0 + R[0, 0] - R[1, 1] - R[2, 2])

        qw = (R[2, 1] - R[1, 2]) / s

        qx = 0.25 * s
        qy = (R[0, 1] + R[1, 0]) / s
        qz = (R[0, 2] + R[2, 0]) / s

    elif R[1, 1] > R[2, 2]:

        s = 2.0 * math.sqrt(1.0 + R[1, 1] - R[0, 0] - R[2, 2])

        qw = (R[0, 2] - R[2, 0]) / s

        qx = (R[0, 1] + R[1, 0]) / s
        qy = 0.25 * s
        qz = (R[1, 2] + R[2, 1]) / s

    else:

        s = 2.0 * math.sqrt(1.0 + R[2, 2] - R[0, 0] - R[1, 1])

        qw = (R[1, 0] - R[0, 1]) / s

        qx = (R[0, 2] + R[2, 0]) / s
        qy = (R[1, 2] + R[2, 1]) / s
        qz = 0.25 * s

    return {

        'x': float(t[0]), 'y': float(t[1]), 'z': float(t[2]),

        'qx': float(qx), 'qy': float(qy), 'qz': float(qz), 'qw': float(qw),

    }

```

```

class CalibrationSampler(Node):
    def __init__(self):
        super().__init__('handeye_sampler')

        self._tf_buffer = tf2_ros.Buffer()
        self._tf_listener = tf2_ros.TransformListener(self._tf_buffer, self)

        self._corners_count = 0
        self._corners_lock = threading.Lock()
        self.create_subscription(
            Int32, '/charuco_detector/corners_detected',
            self._on_corners, 10
        )

        self._R_g2b: List[np.ndarray] = []
        self._t_g2b: List[np.ndarray] = []
        self._R_t2c: List[np.ndarray] = []
        self._t_t2c: List[np.ndarray] = []

        self._result_R: Optional[np.ndarray] = None
        self._result_t: Optional[np.ndarray] = None

    def _on_corners(self, msg: Int32):
        with self._corners_lock:
            self._corners_count = msg.data

    @property
    def corners(self) -> int:
        with self._corners_lock:
            return self._corners_count

    def take_sample(self) -> Tuple[bool, str]:
        corners = self.corners

        if corners < MIN_CORNERS_FOR_SAMPLE:
            return False, (
                f'ABGELEHNT – Board nicht vollstaendig erkannt '

```

```

        f'({corners}/{TOTAL_CORNERS} Ecken). '

        f'Board so ausrichten, dass es der Kamera vollstaendig gezeigt wird.'

    )

try:

    tf_board = self._tf_buffer.lookup_transform(

        CAMERA_FRAME, BOARD_FRAME,

        rclpy.time.Time(), rclpy.time.Duration(seconds=1.0)

    )

except Exception as e:

    return False, f'ABGELEHNT – Board TF nicht erhalten: {e}'

board_age = (

    self.get_clock().now() -

    rclpy.time.Time.from_msg(tf_board.header.stamp)

).nanoseconds / 1e9

if board_age > 1.0:

    return False, (

        f'ABGELEHNT – Board TF veraltet ({board_age:.1f}s). '

        f'Board ist moeglicherweise gerade nicht in der Kamera sichtbar.'

    )

try:

    tf_robot = self._tf_buffer.lookup_transform(

        ROBOT_EE_FRAME, ROBOT_BASE_FRAME,

        rclpy.time.Time(), rclpy.time.Duration(seconds=1.0)

    )

except Exception as e:

    return False, f'ABGELEHNT – Roboter TF nicht erhalten: {e}'

R_robot, t_robot = tf_to_R_t(tf_robot)

R_board, t_board = tf_to_R_t(tf_board)

self._R_g2b.append(R_robot)

self._t_g2b.append(t_robot.reshape(3, 1))

self._R_t2c.append(R_board)

self._t_t2c.append(t_board.reshape(3, 1))

```

```

n = len(self._R_g2b)

return True, (

    f'GESPEICHERT – Sample #{n} '

    f'[corners: {corners}/{TOTAL_CORNERS}] '

    f'Robot: [{t_robot[0]:+.3f}, {t_robot[1]:+.3f}, {t_robot[2]:+.3f}] '

    f'Board: [{t_board[0]:+.3f}, {t_board[1]:+.3f}, {t_board[2]:+.3f}]'

)

```

```

def delete_last(self) -> str:

    if not self._R_g2b:

        return 'Kein Sample zum Loeschen.'

    self._R_g2b.pop()

    self._t_g2b.pop()

    self._R_t2c.pop()

    self._t_t2c.pop()

    return f'Letztes Sample geloescht. Verbleibend: {len(self._R_g2b)}'

```

```

def compute(self) -> Tuple[bool, str]:

    n = len(self._R_g2b)

    if n < 3:

        return False, f'Mindestens 3 Samples noetig, aktuell {n} vorhanden.'

```

```

lines = [f'Kalibrierung wird berechnet ({n} Samples)...\\n']

lines.append(f'  {"Method":12s}  {"x":>8s}  {"y":>8s}  {"z":>8s}')

lines.append(f'  {"-"*12}  {"-"*8}  {"-"*8}  {"-"*8}')

```

```

best_method = None

```

```

best_R = None

```

```

best_t = None

```

```

for name, flag in HANDEYE_METHODS.items():

```

```

    try:

```

```

        R, t = cv2.calibrateHandEye(

            self._R_g2b, self._t_g2b,

            self._R_t2c, self._t_t2c,

            method=flag,

```

```

    )

    tv = t.flatten()

    lines.append(

        f' {name:12s} {tv[0]:+8.4f} {tv[1]:+8.4f} {tv[2]:+8.4f}'

    )

    if best_method is None:

        best_method = name

        best_R = R

        best_t = tv

except Exception as e:

    lines.append(f' {name:12s} FEHLGESCHLAGEN - {e}')

if best_R is not None:

    self._result_R = best_R

    self._result_t = best_t

    q = R_t_to_quaternion(best_R, best_t)

    lines.append(f'\nGewaehlte Methode: {best_method}')

    lines.append(

        f' t=[{q["x"]:.4f}, {q["y"]:.4f}, {q["z"]:.4f}] '

        f'q=[{q["qx"]:.4f}, {q["qy"]:.4f}, {q["qz"]:.4f}, {q["qw"]:.4f}]'

    )

return best_R is not None, '\n'.join(lines)

def save(self) -> Tuple[bool, str]:

    if self._result_R is None:

        return False, 'Zuerst mit C die Kalibrierung berechnen.'

    q = R_t_to_quaternion(self._result_R, self._result_t)

    data = {

        'parameters': {

            'eye_on_hand': False,

            'robot_base_frame': ROBOT_BASE_FRAME,

            'robot_effector_frame': ROBOT_EE_FRAME,

            'tracking_base_frame': CAMERA_FRAME,

            'tracking_marker_frame': BOARD_FRAME,

        },

```

```

        'transformation': {
            'x': q['x'], 'y': q['y'], 'z': q['z'],
            'qx': q['qx'], 'qy': q['qy'], 'qz': q['qz'], 'qw': q['qw'],
        },
    }
}

```

```

os.makedirs(os.path.dirname(OUTPUT_YAML), exist_ok=True)

```

```

if os.path.exists(OUTPUT_YAML):

```

```

    ts = time.strftime('%Y%m%d_%H%M%S')

```

```

    backup = OUTPUT_YAML.replace('.yaml', f'_backup_{ts}.yaml')

```

```

    os.rename(OUTPUT_YAML, backup)

```

```

with open(OUTPUT_YAML, 'w') as f:

```

```

    yaml.dump(data, f, default_flow_style=False)

```

```

return True, (

```

```

    f'GESPEICHERT -> {OUTPUT_YAML}\n'

```

```

    f'  t=[{q["x"]:.4f}, {q["y"]:.4f}, {q["z"]:.4f}]\n'

```

```

    f'  q=[{q["qx"]:.4f}, {q["qy"]:.4f}, {q["qz"]:.4f}, {q["qw"]:.4f}]'

```

```

)

```

```

@property

```

```

def sample_count(self) -> int:

```

```

    return len(self._R_g2b)

```

```

def get_key():

```

```

    """Single keypress read (no Enter required)."""

```

```

    if select.select([sys.stdin], [], [], 0.1)[0]:

```

```

        return sys.stdin.read(1)

```

```

    return None

```

```

def main():

```

```

    rclpy.init()

```

```

    node = CalibrationSampler()

```

```

spin_thread = threading.Thread(
    target=lambda: rclpy.spin(node), daemon=True
)

spin_thread.start()

old_settings = termios.tcgetattr(sys.stdin)

print('\n' + '=' * 50)
print('  HAND-EYE KALIBRIERUNG  (eye-on-base)')
print('=' * 50)
print('  S – Sample nehmen (wenn Board vollstaendig sichtbar)')
print('  C – Kalibrierung berechnen')
print('  W – Ergebnis speichern')
print('  D – letztes Sample loeschen')
print('  Q – beenden')
print('=' * 50)
print()
print('Mit der Slider-GUI den Roboter in verschiedene Posen bringen,')
print('wenn das Board vollstaendig in der Kamera sichtbar ist, S druecken.')
print()

try:
    tty.setraw(sys.stdin.fileno())

    while rclpy.ok():
        key = get_key()

        if key is None:
            continue

        k = key.lower()

        if k == 's':
            ok, msg = node.take_sample()

            sym = '>>>' if ok else '!!!'

            print(f'\r{sym} {msg}\r\n', end='', flush=True)

            if ok:
                print(

```

```

        f'\r    Samples gesamt: {node.sample_count}\r\n',
        end='', flush=True
    )

elif k == 'c':

    print(f'\r--- Wird berechnet... ---\r\n', end='', flush=True)

    ok, msg = node.compute()

    for line in msg.split('\n'):

        print(f'\r{line}\r\n', end='', flush=True)

elif k == 'w':

    ok, msg = node.save()

    sym = '>>>' if ok else '!!!'

    for line in msg.split('\n'):

        print(f'\r{sym} {line}\r\n', end='', flush=True)

elif k == 'd':

    msg = node.delete_last()

    print(f'\r--- {msg}\r\n', end='', flush=True)

elif k == 'q':

    print(f'\r\nBeenden...\r\n', end='', flush=True)

    break

except KeyboardInterrupt:

    pass

finally:

    termios.tcsetattr(sys.stdin, termios.TCSADRAIN, old_settings)

    node.destroy_node()

    rclpy.shutdown()

if __name__ == '__main__':

    main()

```


src/mycobot_calibration/scripts/camera_point_marker.py

```
#!/usr/bin/env python3
```

```
"""
```

Kamera-Bild anklicken -> in RViz als 3D-Marker anzeigen.

Roboter bewegt sich nicht, zeigt nur die 3D-Position des angeklickten

Punkts im robot_base Frame als Kugel in RViz an.

Verwendung:

Terminal 1: System läuft (start_mycobot.sh + Kamera)

Terminal 2: python3 ~/ros2_ws/src/mycobot_calibration/scripts/camera_point_marker.py

Mouse:

Links klicken – Punkt wählen (gelbe Kugel in RViz sichtbar)

Rechts klicken – löschen

Keys:

Q – beenden

```
"""
```

```
import threading
```

```
from typing import Optional, Tuple
```

```
import cv2
```

```
import numpy as np
```

```
import rclpy
```

```
from rclpy.node import Node
```

```
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy
```

```
from sensor_msgs.msg import Image, CameraInfo
```

```
from geometry_msgs.msg import PointStamped
```

```
from visualization_msgs.msg import Marker
```

```
import tf2_ros
```

```
import tf2_geometry_msgs # noqa: F401
```

```
CAMERA_FRAME = 'camera_color_optical_frame'
```

```
ROBOT_BASE_FRAME = 'robot_base'
```

```
class CameraPointMarker(Node):
```

```
    def __init__(self):
```

```
        super().__init__('camera_point_marker')
```

```
        self._K: Optional[np.ndarray] = None
```

```
        self._latest_bgr: Optional[np.ndarray] = None
```

```
        self._latest_depth: Optional[np.ndarray] = None
```

```
        self._lock = threading.Lock()
```

```
        self._click_px: Optional[Tuple[int, int]] = None
```

```
        self._target_3d: Optional[np.ndarray] = None
```

```
        self._status = 'Warte auf Kamera...'
```

```
        self._marker_id = 0
```

```
        self._tf_buffer = tf2_ros.Buffer()
```

```
        self._tf_listener = tf2_ros.TransformListener(self._tf_buffer, self)
```

```
        self._marker_pub = self.create_publisher(
```

```
            Marker, '/camera_click_marker', 10
```

```
)
```

```
        qos_ci = QoSProfile(
```

```
            reliability=ReliabilityPolicy.RELIABLE,
```

```
            history=HistoryPolicy.KEEP_LAST, depth=1,
```

```
)
```

```
        self.create_subscription(
```

```
            CameraInfo, '/camera/color/camera_info', self._on_info, qos_ci
```

```
)
```

```
        self.create_subscription(
```

```
            Image, '/camera/color/image_raw', self._on_color, 10
```

```
)
```

```
        self.create_subscription(
```

```
            Image, '/camera/aligned_depth_to_color/image_raw',
```

```
            self._on_depth, 10
```

```
)
```

```
def _on_info(self, msg: CameraInfo):
```

```
    self._K = np.array(msg.k, dtype=np.float64).reshape(3, 3)
```

```
def _on_color(self, msg: Image):
```

```
    try:
```

```
        arr = np.frombuffer(msg.data, dtype=np.uint8)
```

```
        enc = msg.encoding.lower()
```

```
        if enc == 'rgb8':
```

```
            bgr = cv2.cvtColor(
```

```
                arr.reshape((msg.height, msg.width, 3)), cv2.COLOR_RGB2BGR
```

```
            )
```

```
        else:
```

```
            bgr = arr.reshape((msg.height, msg.width, 3)).copy()
```

```
        with self._lock:
```

```
            self._latest_bgr = bgr
```

```
            if self._status == 'Warte auf Kamera...':
```

```
                self._status = 'Bereit – klicken!'
```

```
    except Exception:
```

```
        pass
```

```
def _on_depth(self, msg: Image):
```

```
    try:
```

```
        if msg.encoding == '16UC1':
```

```
            d = np.frombuffer(msg.data, dtype=np.uint16).reshape(
```

```
                (msg.height, msg.width)
```

```
            )
```

```
        elif msg.encoding == '32FC1':
```

```
            d = np.frombuffer(msg.data, dtype=np.float32).reshape(
```

```
                (msg.height, msg.width)
```

```
            )
```

```
        else:
```

```
            return
```

```
        with self._lock:
```

```
            self._latest_depth = d
```

```
    except Exception:
```

pass

```
def pixel_to_3d_base(self, u: int, v: int) -> Optional[np.ndarray]:
```

```
    if self._K is None:
```

```
        self._status = 'Camera Info noch nicht da'
```

```
        return None
```

```
    with self._lock:
```

```
        depth = self._latest_depth
```

```
    if depth is None:
```

```
        self._status = 'Keine Tiefen-Daten'
```

```
        return None
```

```
    h, w = depth.shape
```

```
    if not (0 <= v < h and 0 <= u < w):
```

```
        return None
```

```
    region = depth[max(0, v - 3):v + 4, max(0, u - 3):u + 4]
```

```
    valid = region[region > 0]
```

```
    if len(valid) == 0:
```

```
        self._status = f'Keine Tiefe ({u},{v})'
```

```
        return None
```

```
    z_raw = float(np.median(valid))
```

```
    z = z_raw / 1000.0 if z_raw > 100 else z_raw
```

```
    fx, fy = self._K[0, 0], self._K[1, 1]
```

```
    cx, cy = self._K[0, 2], self._K[1, 2]
```

```
    x_cam = (u - cx) * z / fx
```

```
    y_cam = (v - cy) * z / fy
```

```
    pt_cam = PointStamped()
```

```
    pt_cam.header.frame_id = CAMERA_FRAME
```

```
    pt_cam.header.stamp = self.get_clock().now().to_msg()
```

```
    pt_cam.point.x = float(x_cam)
```

```
    pt_cam.point.y = float(y_cam)
```

```
    pt_cam.point.z = float(z)
```

```

try:

    pt_base = self._tf_buffer.transform(

        pt_cam, ROBOT_BASE_FRAME,

        timeout=rclpy.time.Duration(seconds=2.0)

    )

except Exception as e:

    self._status = f'TF-Fehler: {e}'

    return None


return np.array([

    pt_base.point.x, pt_base.point.y, pt_base.point.z

], dtype=np.float64)


def on_click(self, u: int, v: int):

    p = self.pixel_to_3d_base(u, v)

    if p is None:

        self._click_px = (u, v)

        self._target_3d = None

        return


    self._click_px = (u, v)

    self._target_3d = p

    self._status = (

        f'robot_base: [{p[0]:.3f}, {p[1]:.3f}, {p[2]:.3f}] m'

    )

    self.get_logger().info(self._status)

    self._publish_marker(p)


def _publish_marker(self, p: np.ndarray):

    m = Marker()

    m.header.frame_id = ROBOT_BASE_FRAME

    m.header.stamp = self.get_clock().now().to_msg()

    m.ns = 'camera_click'

    m.id = self._marker_id

    self._marker_id += 1

    m.type = Marker.SPHERE

    m.action = Marker.ADD

```

```

        m.pose.position.x = float(p[0])

        m.pose.position.y = float(p[1])

        m.pose.position.z = float(p[2])

        m.pose.orientation.w = 1.0

        m.scale.x = 0.015

        m.scale.y = 0.015

        m.scale.z = 0.015

        m.color.r = 1.0

        m.color.g = 1.0

        m.color.b = 0.0

        m.color.a = 0.9

        m.lifetime.sec = 300

        self._marker_pub.publish(m)

def clear_markers(self):

    m = Marker()

    m.header.frame_id = ROBOT_BASE_FRAME

    m.header.stamp = self.get_clock().now().to_msg()

    m.ns = 'camera_click'

    m.action = Marker.DELETEALL

    self._marker_pub.publish(m)

    self._click_px = None

    self._target_3d = None

    self._marker_id = 0

    self._status = 'Geloescht – klicken!'

def get_display_frame(self) -> Optional[np.ndarray]:

    with self._lock:

        if self._latest_bgr is None:

            return None

        frame = self._latest_bgr.copy()

    h, w = frame.shape[:2]

    if self._click_px is not None:

        u, v = self._click_px

        color = (0, 255, 255)

```

```

cv2.circle(frame, (u, v), 8, color, 2)

cv2.drawMarker(frame, (u, v), color, cv2.MARKER_CROSS, 16, 2)

if self._target_3d is not None:
    p = self._target_3d
    text = f'[{p[0]:.3f}, {p[1]:.3f}, {p[2]:.3f}]'
    cv2.putText(frame, text, (u + 12, v - 12),
                 cv2.FONT_HERSHEY_SIMPLEX, 0.45,
                 color, 1, cv2.LINE_AA)

if self._status:
    cv2.rectangle(frame, (0, 0), (w, 26), (40, 40, 40), -1)
    cv2.putText(frame, self._status, (6, 18),
                 cv2.FONT_HERSHEY_SIMPLEX, 0.4,
                 (255, 255, 255), 1, cv2.LINE_AA)

cv2.rectangle(frame, (0, h - 20), (w, h), (40, 40, 40), -1)
cv2.putText(frame, 'Links: Punkt waehlen | Rechts: loeschen | Q: beenden',
            (4, h - 5), cv2.FONT_HERSHEY_SIMPLEX, 0.3,
            (180, 180, 180), 1, cv2.LINE_AA)

return frame

```

```

def mouse_cb(event, x, y, flags, param):

```

```

    node = param

```

```

    if event == cv2.EVENT_LBUTTONDOWN:

```

```

        node.on_click(x, y)

```

```

    elif event == cv2.EVENT_RBUTTONDOWN:

```

```

        node.clear_markers()

```

```

def main():

```

```

    rclpy.init()

```

```

    node = CameraPointMarker()

```

```

    spin_thread = threading.Thread(

```

```

        target=lambda: rclpy.spin(node), daemon=True
    )
    spin_thread.start()

cv2.namedWindow('Camera → RViz Marker', cv2.WINDOW_NORMAL)
cv2.resizeWindow('Camera → RViz Marker', 640, 380)
cv2.setMouseCallback('Camera → RViz Marker', mouse_cb, node)

print('\n' + '=' * 50)
print(' Camera Point Marker')
print('=' * 50)
print(' Links klicken : Punkt in Kamera waehlen -> in RViz sehen')
print(' Rechts klicken: alle Marker loeschen')
print(' Q           : beenden')
print('=' * 50 + '\n')

try:
    while rclpy.ok():
        frame = node.get_display_frame()

        if frame is not None:
            cv2.imshow('Camera → RViz Marker', frame)

            key = cv2.waitKey(50) & 0xFF
            if key == ord('q') or key == ord('Q'):
                break
except KeyboardInterrupt:
    pass
finally:
    cv2.destroyAllWindows()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```


src/mycobot_calibration/scripts/cartesian_jog.py

```
#!/usr/bin/env python3

"""Kartesischer Jog GUI – myCobot 280 JN.

Bewegt tcp im base frame in X/Y/Z-Schritten, während der Greifer SENKRECHT
(top-down) fixiert bleibt. Jeder Schritt: Ziel = aktuelles_tcp + delta -> numerical IK (gripper_down)
-> FollowJointTrajectory /arm_controller. KEIN send_coords; nutzt MoveIt compute_ik.

Für senkrechtes Berühren / präzise Positionierung.

Verwendung:

    python3 ~/ros2_ws/src/mycobot_calibration/scripts/cartesian_jog.py
"""

import threading

import time

import tkinter as tk

from tkinter import ttk

import rclpy

from rclpy.action import ActionClient

from rclpy.node import Node

from builtin_interfaces.msg import Duration

from control_msgs.action import FollowJointTrajectory

from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint

from sensor_msgs.msg import JointState

import tf2_ros

from mycobot_calibration.numerical_ik import NumericalIK, ARM_JOINTS


class CartJogNode(Node):

    def __init__(self):

        super().__init__('cartesian_jog')

        self.action_client = ActionClient(

            self, FollowJointTrajectory, '/arm_controller/follow_joint_trajectory')

        self.current_joints = [0.0] * 6
```

```

self.joints_ok = False

self.create_subscription(JointState, '/joint_states', self._on_js, 10)

self.tf_buffer = tf2_ros.Buffer()

tf2_ros.TransformListener(self.tf_buffer, self)

self.ik = NumericalIK(self)

def _on_js(self, msg: JointState):
    for i, n in enumerate(ARM_JOINTS):
        if n in msg.name:
            idx = msg.name.index(n)
            if idx < len(msg.position):
                self.current_joints[i] = msg.position[idx]
    self.joints_ok = True

def tcp_xyz(self):
    try:
        tf = self.tf_buffer.lookup_transform('robot_base', 'tcp', rclpy.time.Time())
        t = tf.transform.translation
        return [t.x, t.y, t.z]
    except Exception:
        return None

def send_joints(self, positions, duration_s):
    if not self.action_client.wait_for_server(timeout_sec=2.0):
        return False

    traj = JointTrajectory()
    traj.joint_names = list(ARM_JOINTS)
    pt = JointTrajectoryPoint()
    pt.positions = [float(p) for p in positions]
    pt.time_from_start = Duration(
        sec=int(duration_s),
        nanosec=int((duration_s - int(duration_s)) * 1e9))
    traj.points = [pt]
    goal = FollowJointTrajectory.Goal()
    goal.trajectory = traj
    self.action_client.send_goal_async(goal)

    return True

```

```

class App:
    def __init__(self, root, node: CartJogNode):
        self.root = root

        self.node = node

        self.busy = False

        root.title('myCobot Kartesischer Jog (SENKRECHT)')

        top = ttk.Frame(root, padding=8); top.pack(fill='x')

        ttk.Label(top, text='Schritt (mm):').pack(side='left')

        self.step_var = tk.DoubleVar(value=5.0)

        ttk.Spinbox(top, from_=0.5, to=50.0, increment=0.5, width=6,
                    textvariable=self.step_var).pack(side='left', padx=4)

        ttk.Label(top, text='Dauer (s):').pack(side='left', padx=(12, 0))

        self.dur_var = tk.DoubleVar(value=2.5)

        ttk.Spinbox(top, from_=0.5, to=15.0, increment=0.5, width=6,
                    textvariable=self.dur_var).pack(side='left', padx=4)

        cur = ttk.Frame(root, padding=8); cur.pack(fill='x')

        self.cur_lbl = ttk.Label(cur, text='TCP: --', font=('TkDefaultFont', 11, 'bold'))

        self.cur_lbl.pack(side='left')

        # Jog-Buttons: +X/-X vorwärts/rückwärts, +Y/-Y links/rechts, +Z/-Z hoch/runter

        grid = ttk.Frame(root, padding=8); grid.pack()

        def jbtn(txt, ax, sgn, r, c, col='black'):
            b = tk.Button(grid, text=txt, width=10, height=2,
                          command=lambda: self.jog(ax, sgn))

            b.grid(row=r, column=c, padx=4, pady=4)

        jbtn('+X (vorwaerts)', 0, +1, 0, 1)

        jbtn('-X (rueckwaerts)', 0, -1, 2, 1)

        jbtn('+Y (links)', 1, +1, 1, 0)

        jbtn('-Y (rechts)', 1, -1, 1, 2)

        jbtn('+Z (hoch)', 2, +1, 0, 3)

        jbtn('-Z (runter)', 2, -1, 2, 3)

        # Absolut anfahren

```

```

ab = ttk.Frame(root, padding=8); ab.pack(fill='x')

ttk.Label(ab, text='Absolut anfahren (mm) x:').pack(side='left')

self.gx = tk.StringVar(); ttk.Entry(ab, textvariable=self.gx, width=6).pack(side='left')

ttk.Label(ab, text='y:').pack(side='left')

self.gy = tk.StringVar(); ttk.Entry(ab, textvariable=self.gy, width=6).pack(side='left')

ttk.Label(ab, text='z:').pack(side='left')

self.gz = tk.StringVar(); ttk.Entry(ab, textvariable=self.gz, width=6).pack(side='left')

ttk.Button(ab, text='FAHRE', command=self.goto_abs).pack(side='left', padx=6)


self.status = ttk.Label(root, text='bereit', padding=8, foreground='gray')

self.status.pack(fill='x')

self.root.after(200, self._refresh)


def _refresh(self):

    p = self.node.tcp_xyz()

    if p:

        self.cur_lbl.config(

            text=f'TCP: x={p[0]*1000:.0f} y={p[1]*1000:.0f} z={p[2]*1000:.0f} mm')

        self.root.after(200, self._refresh)


def jog(self, axis, sgn):

    p = self.node.tcp_xyz()

    if p is None:

        self.status.config(text='kein TF', foreground='red'); return

    step = max(0.5, float(self.step_var.get())) / 1000.0 * sgn

    target = list(p); target[axis] += step

    self._move(target, f'jog {"XYZ"[axis]}{"+" if sgn>0 else "-"}')


def goto_abs(self):

    try:

        target = [float(self.gx.get())/1000.0, float(self.gy.get())/1000.0,

                  float(self.gz.get())/1000.0]

    except ValueError:

        self.status.config(text='ungueltige xyz', foreground='red'); return

    self._move(target, 'absolut anfahren')


def _move(self, target, label):

```

```

if self.busy:

    self.status.config(text='beschaeftigt, warten', foreground='orange'); return

self.busy = True

self.status.config(

    text=f'{label}: IK wird geloest -> ({target[0]*1000:.0f},{target[1]*1000:.0f},{target[2]*1000:.0f})',

    foreground='blue')

dur = max(0.5, float(self.dur_var.get()))

threading.Thread(target=self._solve_and_send,

                  args=(target, dur, label), daemon=True).start()

def _solve_and_send(self, target, dur, label):

    sol = self.node.ik.solve_multi_seed(

        target, gripper_down=True, current_joints=list(self.node.current_joints))

    if sol is None:

        self._status(f'{label}: KEINE IK-LOESUNG (ausser Reichweite?)', 'red')

        self.busy = False; return

    ok = self.node.send_joints(sol, dur)

    if ok:

        self._status(

            f'{label}: GESENDET -> ({target[0]*1000:.0f},{target[1]*1000:.0f},{target[2]*1000:.0f}) mm,

            {dur:.1f}s', 'green')

    else:

        self._status(f'{label}: kein action server', 'red')

    time.sleep(dur)

    self.busy = False

def _status(self, txt, color):

    self.root.after(0, lambda: self.status.config(text=txt, foreground=color))

def main():

    rclpy.init()

    node = CartJogNode()

    threading.Thread(target=rclpy.spin, args=(node,), daemon=True).start()

    t0 = time.time()

    while not node.joints_ok and time.time() - t0 < 3.0:

        time.sleep(0.1)

```

```
    root = tk.Tk()

    App(root, node)

    try:

        root.mainloop()

    finally:

        node.destroy_node()

        rclpy.shutdown()

if __name__ == '__main__':

    main()
```

src/mycobot_calibration/scripts/click_arc.py

```
#!/usr/bin/env python3
```

```
"""
```

3 Punkte ins Kamera-Bild klicken -> der hindurchgehende KREIS wird bestimmt ->

Roboter zeichnet den VIERTELKREIS-Bogen durch diese Punkte (~3cm über der

Plattform, Greifer senkrecht-nach-unten). Vorschau wird über das Bild gelegt.

Pipeline:

3x Pixel -> aligned depth -> 3D Kamera -> TF robot_base (XY)

-> Z konstant (Plattform+3cm) -> circumcircle fit -> Bogen-Waypoints

-> eigene FK/IK (Greifer nach unten, an Hardware verifiziert) -> Limit/Reichweite/

Hindernis/Höhe-Prüfung -> Bridge-Socket mit sequenziellem send_radians.

Verwendung:

Terminal 1: ~/ros2_ws/start_mycobot.sh (oder demo.launch use_camera:=true)

Terminal 2: python3 ../click_arc.py

Mouse: Links = Punkt hinzufügen (max 3) Rechts = alle löschen

Keys : E = Bogen zeichnen (Roboter bewegt sich) H = home(0) C = löschen Q = beenden

```
"""
```

```
import math
```

```
import threading
```

```
import time
```

```
from typing import List, Optional, Tuple
```

```
import cv2
```

```
import numpy as np
```

```
import rclpy
```

```
from rclpy.node import Node
```

```
from rclpy.action import ActionClient
```

```
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy
```

```
from sensor_msgs.msg import Image, CameraInfo, JointState
```

```
from geometry_msgs.msg import PointStamped
```

```

from visualization_msgs.msg import Marker, MarkerArray

from control_msgs.action import FollowJointTrajectory

from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint

from builtin_interfaces.msg import Duration

import tf2_ros

import tf2_geometry_msgs # noqa: F401

CAMERA_FRAME = 'camera_color_optical_frame'

ROBOT_BASE_FRAME = 'robot_base'

ARM_ACTION = '/arm_controller/follow_joint_trajectory'

# Trajektorien-Geschwindigkeit (rad/s): freie Bewegung + langsamer Abstieg
VEL_FREE = 1.4

VEL_DESCEND = 0.4

MIN_SEG = 0.12 # minimale Dauer pro Segment (s) – verhindert künstliche Langsamkeit

ARM_JOINTS = [
    'joint2_to_joint1', 'joint3_to_joint2', 'joint4_to_joint3',
    'joint5_to_joint4', 'joint6_to_joint5', 'joint6output_to_joint6',
]

ZC = 0.045 # Bogen tcp Höhe (m) -> unterster Greifer ~3cm über Plattform
HOVER_Z = 0.12 # sichere Hover-Höhe für Abstieg/Aufstieg (m)
N_PTS = 5 # Anzahl zu sammelnder Punkte (für Verlaufs-Schätzung)
N_ARC = 18 # Anzahl Waypoints entlang des angeklickten Spans
EXTRA_FRAC = 0.30 # "Schätzung" (Extrapolation) Anteil nach dem letzten Punkt
N_EXTRA = 6 # Anzahl Waypoints im Schätz-Bereich
MIN_CLEAR = 0.030 # minimale Höhe über Plattform (m)

# ---- URDF mycobot_280_jn kinematische Kette (robot_base Frame) ----
_LIMU = np.array([2.9321, 2.4434, 2.6179, 2.6179, 2.7925, 3.14159])
_LIML = -np.array([2.9321, 2.4434, 2.6179, 2.6179, 2.7052, 3.14159])
_ORIG = [
    ((0, 0, 0.15756), (0, 0, 0)),
    ((0, 0, -0.001), (0, 1.5708, -1.5708)),
    ((-0.1104, 0, 0), (0, 0, 0)),

```



```

((-0.096, 0, 0.06462), (0, 0, -1.5708)),
((0, -0.07318, 0), (1.5708, -1.5708, 0)),
((0, 0.0456, 0), (-1.5708, 0, 0)),
]

def _rpy(r, p, y):
    cr, sr = np.cos(r), np.sin(r)
    cp, sp = np.cos(p), np.sin(p)
    cy, sy = np.cos(y), np.sin(y)
    return (np.array([[cy, -sy, 0], [sy, cy, 0], [0, 0, 1]])
            @ np.array([[cp, 0, sp], [0, 1, 0], [-sp, 0, cp]])
            @ np.array([[1, 0, 0], [0, cr, -sr], [0, sr, cr]]))

def _T(xyz, r, p, yw):
    M = np.eye(4)
    M[:3, :3] = _rpy(r, p, yw)
    M[:3, 3] = xyz
    return M

def _Tz(q):
    M = np.eye(4)
    c, s = np.cos(q), np.sin(q)
    M[:3, :3] = [[c, -s, 0], [s, c, 0], [0, 0, 1]]
    return M

_FGB = _T((0, 0, 0.034), 1.579, 0, 0.8406) # joint6output->gripper_base
_FTCP = _T((0, 0.110, -0.01), 0, 0, 0) # gripper_base->tcp

def fk(q):
    """robot_base FK -> (frame_z_Liste, gripper_base_R, tcp_xyz)."""
    M = np.eye(4)
    zs = []

```

```

for i, (xyz, rr) in enumerate(_ORIG):

    M = M @ _T(xyz, *rr) @ _Tz(q[i])

    zs.append(M[2, 3])

Mgb = M @ _FGB

Mt = Mgb @ _FTCP

zs += [Mgb[2, 3], Mt[2, 3]]

return zs, Mgb[:3, :3], Mt[:3, 3]

def _err(q, tgt):

    _, Rgb, tcp = fk(q)

    ydir = Rgb @ np.array([0, 1.0, 0])          # gripper_base +Y -> tcp Richtung

    return np.concatenate([tcp - tgt, np.cross(ydir, [0, 0, -1.0])])

def ik_down(tgt, seed, max_iter=150):

    """IK für Greifer senkrecht-nach-unten + tcp=tgt. -> (q, residual)."""

    q = np.array(seed, float)

    for _ in range(max_iter):

        e = _err(q, tgt)

        if np.linalg.norm(e) < 1e-5:

            break

        J = np.zeros((6, 6))

        d = 1e-6

        for k in range(6):

            qq = q.copy()

            qq[k] += d

            J[:, k] = (_err(qq, tgt) - e) / d

        q = np.clip(q + np.linalg.solve(J.T @ J + 0.01 * np.eye(6), -J.T @ e),

                    _LIML, _LIMU)

    return q, float(np.linalg.norm(_err(q, tgt)))

def ik_multi(tgt, seeds, max_iter=150):

    best, bres = None, 9.9

    for s in seeds:

        q, r = ik_down(tgt, s, max_iter)

```

```

        if r < bres:

            best, bres = q, r

        if bres < 1e-4:            # gute Lösung gefunden, andere nicht testen

            break

    return best, bres

# Hindernis XY-Boxen (robot_base): column + clamp -> Abstand zur Waagerechten halten
_OBST = [

    (0.335, -0.125, 0.0125 + 0.02, 0.0175 + 0.02),    # column +Zugabe

    (0.3225, -0.125, 0.025 + 0.02, 0.055 + 0.02),    # clamp +Zugabe

]

def obstacle_hit(x, y):

    for ox, oy, hx, hy in _OBST:

        if abs(x - ox) < hx and abs(y - oy) < hy:

            return True

    return False

SEEDS = [

    [0, 0.4, -2.2, 0.3, 0, 0.0],

    [0.6, 0.5, -2.3, 0.2, 0, 0.0],

    [-0.6, 0.5, -2.3, 0.2, 0, 0.0],

    [0, 0.2, -1.8, 0.6, 0, 0.0],

]

def circumcircle(p1, p2, p3):

    """3 Punkte(XY) -> (cx, cy, R). Wenn kollinear -> None."""

    ax, ay = p1

    bx, by = p2

    cx, cy = p3

    d = 2.0 * (ax * (by - cy) + bx * (cy - ay) + cx * (ay - by))

    if abs(d) < 1e-9:

        return None

```

```

a2, b2, c2 = ax * ax + ay * ay, bx * bx + by * by, cx * cx + cy * cy

ux = (a2 * (by - cy) + b2 * (cy - ay) + c2 * (ay - by)) / d

uy = (a2 * (cx - bx) + b2 * (ax - cx) + c2 * (bx - ax)) / d

R = math.hypot(ax - ux, ay - uy)

return ux, uy, R

```

```
def fit_circle(pts):
```

```
    """n>=3 Punkte(XY) -> beste Anpassung (least-squares) (cx, cy, R). Wenn schlecht -> None."""
```

```
    P = np.array(pts, float)
```

```
    x, y = P[:, 0], P[:, 1]
```

```
    A = np.column_stack([2 * x, 2 * y, np.ones(len(P))])
```

```
    b = x * x + y * y
```

```
    try:
```

```
        sol, *_ = np.linalg.lstsq(A, b, rcond=None)
```

```
    except Exception:
```

```
        return None
```

```
    cx, cy, c = sol
```

```
    rr = c + cx * cx + cy * cy
```

```
    if rr <= 1e-9:
```

```
        return None
```

```
    return float(cx), float(cy), float(math.sqrt(rr))
```

```
def unwrap_angles(center, pts):
```

```
    """Winkel der angeklickten Punkte um das Zentrum, als monotone (unwrap)
```

```
    Folge, die die Klick-Reihenfolge bewahrt."""
```

```
    cx, cy = center
```

```
    raw = [math.atan2(p[1] - cy, p[0] - cx) for p in pts]
```

```
    out = [raw[0]]
```

```
    for a in raw[1:]:
```

```
        d = (a - out[-1] + math.pi) % (2 * math.pi) - math.pi
```

```
        out.append(out[-1] + d)
```

```
    return out
```

```
def arc_thetas(center, p1, p2, p3, n):
```

```

"""n Winkel entlang des Bogens p1->p3, der durch p2 geht, erzeugen."""

cx, cy = center

t1 = math.atan2(p1[1] - cy, p1[0] - cx)

t2 = math.atan2(p2[1] - cy, p2[0] - cx)

t3 = math.atan2(p3[1] - cy, p3[0] - cx)

tau = 2 * math.pi

ccw_span = (t3 - t1) % tau

ccw_mid = (t2 - t1) % tau

if ccw_mid <= ccw_span:          # CCW-Richtung schließt p2 ein
    direction, span = 1.0, ccw_span
else:
    direction, span = -1.0, tau - ccw_span

return [t1 + direction * span * k / (n - 1) for k in range(n)]


def img_to_bgr(msg):
    arr = np.frombuffer(msg.data, dtype=np.uint8)

    if msg.encoding.lower() == 'rgb8':
        return cv2.cvtColor(arr.reshape((msg.height, msg.width, 3)),
                             cv2.COLOR_RGB2BGR)

    return arr.reshape((msg.height, msg.width, 3)).copy()


def depth_to_arr(msg):
    if msg.encoding == '16UC1':
        return np.frombuffer(msg.data, np.uint16).reshape((msg.height, msg.width))

    if msg.encoding == '32FC1':
        return np.frombuffer(msg.data, np.float32).reshape((msg.height, msg.width))

    raise ValueError(f'depth encoding: {msg.encoding}')


class ClickArc(Node):
    def __init__(self):
        super().__init__('click_arc')

        self._K = None

        self._bgr = None

        self._depth = None

```

```

self._joints = None

self._lock = threading.Lock()


self._pts: List[np.ndarray] = []      # robot_base 3D (z=ZC)
self._px: List[Tuple[int, int]] = []  # Pixel
self._arc_base = None                 # Nx3 Bogen-Punkte (base)
self._arc_q = None                    # 6-Gelenke pro Waypoint
self._n_through = 0                   # Anzahl Waypoints im angeklickten Span
self._hover_first = None
self._hover_last = None
self._valid = False
self._busy = False
self._status = 'Warte auf Kamera...'


self._tfbuf = tf2_ros.Buffer()
self._tf = tf2_ros.TransformListener(self._tfbuf, self)
self._mk = self.create_publisher(MarkerArray, '/click_arc/markers', 5)
self._arm = ActionClient(self, FollowJointTrajectory, ARM_ACTION)


qos = QoSProfile(reliability=ReliabilityPolicy.RELIABLE,
                  history=HistoryPolicy.KEEP_LAST, depth=1)
self.create_subscription(CameraInfo, '/camera/color/camera_info',
                          self._on_info, qos)
self.create_subscription(Image, '/camera/color/image_raw',
                          self._on_color, 10)
self.create_subscription(Image,
                          '/camera/aligned_depth_to_color/image_raw',
                          self._on_depth, 10)
self.create_subscription(JointState, '/joint_states',
                          self._on_joints, 10)


# ---- subs ----

def _on_info(self, m):
    self._K = np.array(m.k, np.float64).reshape(3, 3)


def _on_color(self, m):
    try:

```

```

        with self._lock:

            self._bgr = img_to_bgr(m)

            if self._status == 'Warte auf Kamera...':

                self._status = 'Bereit – 3 Punkte anklicken'

except Exception:

    pass


def _on_depth(self, m):

    try:

        with self._lock:

            self._depth = depth_to_arr(m)

    except Exception:

        pass


def _on_joints(self, m):

    self._joints = m


def _cur_joints(self):

    if self._joints is None:

        return None

    d = dict(zip(self._joints.name, self._joints.position))

    if not all(j in d for j in ARM_JOINTS):

        return None

    return [d[j] for j in ARM_JOINTS]


# ---- Pixel -> base ----

def pixel_to_base(self, u, v):

    if self._K is None:

        return None

    with self._lock:

        depth = self._depth

    if depth is None:

        return None

    h, w = depth.shape

    if not (0 <= v < h and 0 <= u < w):

        return None

    reg = depth[max(0, v - 3):v + 4, max(0, u - 3):u + 4]

```

```

valid = reg[reg > 0]

if len(valid) == 0:

    return None

zr = float(np.median(valid))

z = zr / 1000.0 if zr > 100 else zr

fx, fy = self._K[0, 0], self._K[1, 1]

cx, cy = self._K[0, 2], self._K[1, 2]

pc = PointStamped()

pc.header.frame_id = CAMERA_FRAME

pc.header.stamp = self.get_clock().now().to_msg()

pc.point.x = (u - cx) * z / fx

pc.point.y = (v - cy) * z / fy

pc.point.z = z

try:

    pb = self._tfbuf.transform(pc, ROBOT_BASE_FRAME,

                               timeout=rclpy.time.Duration(seconds=2.0))

except Exception as e:

    self.get_logger().warn(f'TF: {e}')

    return None

return np.array([pb.point.x, pb.point.y, ZC])

# ---- base -> Pixel (overlay) ----

def _base_to_cam(self):

    if self._K is None:

        return None

    try:

        tf = self._tfbuf.lookup_transform(CAMERA_FRAME, ROBOT_BASE_FRAME,

                                           rclpy.time.Time())

    except Exception:

        return None

    q = tf.transform.rotation

    t = tf.transform.translation

    x, y, z, w = q.x, q.y, q.z, q.w

    R = np.array([

        [1 - 2 * (y * y + z * z), 2 * (x * y - z * w), 2 * (x * z + y * w)],

        [2 * (x * y + z * w), 1 - 2 * (x * x + z * z), 2 * (y * z - x * w)],

        [2 * (x * z - y * w), 2 * (y * z + x * w), 1 - 2 * (x * x + y * y)]]

```



```

        return R, np.array([t.x, t.y, t.z])

def _proj(self, pb, R, tv):
    pc = R @ pb + tv
    if pc[2] <= 1e-3:
        return None
    fx, fy = self._K[0, 0], self._K[1, 1]
    cx, cy = self._K[0, 2], self._K[1, 2]
    return (int(round(fx * pc[0] / pc[2] + cx)),
            int(round(fy * pc[1] / pc[2] + cy)))

# ---- click ----
def on_click(self, u, v):
    if self._busy:
        return
    if len(self._pts) >= N_PTS:
        self._status = f'{N_PTS} Punkte voll – mit C loeschen'
        return
    pb = self.pixel_to_base(u, v)
    if pb is None:
        self._status = f'Keine Tiefe ({u},{v}) – woanders klicken'
        return
    self._pts.append(pb)
    self._px.append((u, v))
    self._status = (f'{len(self._pts)}/{N_PTS} Punkte '
                   f'[{pb[0]:.3f},{pb[1]:.3f}] r={math.hypot(pb[0],pb[1])*1000:.0f}mm'
                   + (' (P: fertig)' if len(self._pts) >= 3 else ''))
    self.get_logger().info(self._status)
    if len(self._pts) == N_PTS:
        self.finalize()

def finalize(self):
    """Mit >=3 Punkten Verlaufs-Schätzung berechnen."""
    if self._busy or len(self._pts) < 3:
        if len(self._pts) < 3:
            self._status = 'Mindestens 3 Punkte noetig'
        return

```

```

self._busy = True

threading.Thread(target=self._compute_arc, daemon=True).start()

def clear(self):

    self._pts, self._px = [], []

    self._arc_base = self._arc_q = None

    self._n_through = 0

    self._valid = False

    self._status = f'Geloescht – {N_PTS} Punkte anklicken (P: frueh fertig)'

    self._mk.publish(MarkerArray())

# ---- Verlaufs-Schätzung + prüfen (Hintergrund-Thread) ----

def _wp_ok(self, wp, seed):

    """IK + Gültigkeit für einen Waypoint. -> (q, valid, Grund)."""

    if seed is None:

        q, res = ik_multi(wp, SEEDS)

    else:

        q, res = ik_down(wp, seed, max_iter=120)

        if res > 1.5e-3:

            q, res = ik_multi(wp, SEEDS + [seed])

    zs, _, _ = fk(q)

    minz = min(zs)

    inlim = bool(np.all(q >= _LIML - 1e-6) and np.all(q <= _LIMU + 1e-6))

    if res > 1.5e-3:

        return q, False, 'IK'

    if not inlim:

        return q, False, 'limit'

    if minz < MIN_CLEAR - 1e-3:

        return q, False, f'z={minz*100:.1f}cm'

    if obstacle_hit(wp[0], wp[1]):

        return q, False, 'Hindernis'

    return q, True, ''

def _compute_arc(self):

    try:

        self._status = f'{len(self._pts)} Punkte – Verlauf wird geschaeztz (IK)...'

        p = [(pt[0], pt[1]) for pt in self._pts]

```

```

cc = fit_circle(p)

line_mode = cc is None or cc[2] > 1.5      # sehr großes R -> Gerade

if not line_mode:

    cx, cy, R = cc

    ang = unwrap_angles((cx, cy), p)

    span = ang[-1] - ang[0]

    if abs(span) < 1e-3:

        line_mode = True

if line_mode:

    # linear: erster->letzter Punkt + Schätz-Verlängerung

    a = np.array(p[0]); b = np.array(p[-1])

    seg = b - a

    thru = [a + seg * (k / (N_ARC - 1)) for k in range(N_ARC)]

    extra = [b + seg * (EXTRA_FRAC * (k + 1) / N_EXTRA)

              for k in range(N_EXTRA)]

    shape = f'GERADE L={np.linalg.norm(seg)*1000:.0f}mm'

else:

    thru_a = [ang[0] + span * k / (N_ARC - 1) for k in range(N_ARC)]

    extra_a = [ang[-1] + span * EXTRA_FRAC * (k + 1) / N_EXTRA

               for k in range(N_EXTRA)]

    thru = [np.array([cx + R * math.cos(t), cy + R * math.sin(t)])

            for t in thru_a]

    extra = [np.array([cx + R * math.cos(t), cy + R * math.sin(t)])

            for t in extra_a]

    shape = f'BOGEN R={R*1000:.0f}mm {math.degrees(abs(span)):.0f}°'

n_through = len(thru)

allxy = thru + extra

base_all = np.array([[w[0], w[1], ZC] for w in allxy])

qs, seed = [], None

bad_through = None

kept = 0

for i, wp in enumerate(base_all):

    q, ok, why = self._wp_ok(wp, seed)

    if not ok:

        if i < n_through:          # angeklickter Punkt nicht erreichbar

            bad_through = (i, why)

```

```

        break

    else:
        # Schätz-Bereich: hier abschneiden

        break

    seed = q

    qs.append(q)

    kept = i + 1

if bad_through is not None or kept < n_through:

    self._valid = False

    if bad_through:

        self._status = f'UNGUELTIG wp{bad_through[0]}:{bad_through[1]} – C löschen'

    else:

        self._status = 'UNGUELTIG: angeklickter Span nicht erreichbar – C löschen'

    self._arc_base = base_all[:max(1, kept)]

    self._n_through = n_through

    self._publish_markers(self._arc_base, ok=False)

    self.get_logger().error(self._status)

    return

# gültig: kept >= n_through (um den Schätz-Teil verlängert)

base = base_all[:kept]

hf, rf = ik_multi(np.array([base[0][0], base[0][1], HOVER_Z]),

                    SEEDS + [qs[0]])

hl, rl = ik_multi(np.array([base[-1][0], base[-1][1], HOVER_Z]),

                    SEEDS + [qs[-1]])

self._arc_base = base

self._arc_q = qs

self._n_through = n_through

self._hover_first, self._hover_last = hf, hl

self._valid = True

self._publish_markers(base, ok=True)

extra_kept = kept - n_through

self._status = (f'BEREIT {shape} Schätzung: +{extra_kept}wp '

                f'z>={MIN_CLEAR*100:.0f}cm – E: zeichnen')

self.get_logger().info(self._status)

finally:

    self._busy = False

```

```

def _publish_markers(self, base, ok):

```

```

arr = MarkerArray()

ls = Marker()

ls.header.frame_id = ROBOT_BASE_FRAME

ls.header.stamp = self.get_clock().now().to_msg()

ls.ns = 'arc'

ls.id = 0

ls.type = Marker.LINE_STRIP

ls.action = Marker.ADD

ls.scale.x = 0.004

ls.color.r = 0.0 if ok else 1.0

ls.color.g = 1.0 if ok else 0.0

ls.color.b = 0.0

ls.color.a = 0.9

ls.pose.orientation.w = 1.0

from geometry_msgs.msg import Point

for wp in base:

    ls.points.append(Point(x=float(wp[0]), y=float(wp[1]), z=float(wp[2])))

arr.markers.append(ls)

for i, pt in enumerate(self._pts):

    sp = Marker()

    sp.header.frame_id = ROBOT_BASE_FRAME

    sp.header.stamp = ls.header.stamp

    sp.ns = 'pts'

    sp.id = i + 1

    sp.type = Marker.SPHERE

    sp.action = Marker.ADD

    sp.pose.position.x = float(pt[0])

    sp.pose.position.y = float(pt[1])

    sp.pose.position.z = float(pt[2])

    sp.pose.orientation.w = 1.0

    sp.scale.x = sp.scale.y = sp.scale.z = 0.02

    sp.color.b = 1.0

    sp.color.a = 1.0

    arr.markers.append(sp)

self._mk.publish(arr)

```

```
# ---- execute (FollowJointTrajectory action) ----
```

```

def execute(self):

    if not self._valid or self._arc_q is None:

        self._status = 'Erst gueltigen 3-Punkte-Bogen erstellen!'

        return

    if self._busy:

        return

    self._busy = True

    threading.Thread(target=self._exec_thread, daemon=True).start()


@staticmethod
def _dur(sec):

    sec = max(0.001, sec)

    return Duration(sec=int(sec), nanosec=int((sec % 1) * 1e9))


def _build_traj(self, waypoints, vels):

    """waypoints: Liste von 6-Gelenke; vels: rad/s pro Segment.

    Erzeugt streng steigende time_from_start ab der aktuellen Pose."""

    cur = self._cur_joints() or [0.0] * 6

    traj = JointTrajectory()

    traj.joint_names = ARM_JOINTS

    t = 0.0

    prev = cur

    for wp, v in zip(waypoints, vels):

        dmax = max(abs(a - b) for a, b in zip(wp, prev))

        t += max(MIN_SEG, dmax / v)

        p = JointTrajectoryPoint()

        p.positions = [float(x) for x in wp]

        p.velocities = [0.0] * 6

        p.time_from_start = self._dur(t)

        traj.points.append(p)

        prev = wp

    return traj


def _run_traj(self, traj, label):

    self._status = label

    self.get_logger().info(label)

    if not self._arm.wait_for_server(timeout_sec=4.0):

```

```

        self._status = 'arm_controller Action fehlt!'

        self.get_logger().error(self._status)

        return False

goal = FollowJointTrajectory.Goal()

goal.trajectory = traj

fut = self._arm.send_goal_async(goal)

# Node spinnt in separatem Thread; Futures manuell abwarten

t0 = time.time()

while not fut.done() and time.time() - t0 < 6.0:

    time.sleep(0.05)

gh = fut.result()

if gh is None or not gh.accepted:

    self._status = 'Trajektorie ABGELEHNT'

    self.get_logger().error(self._status)

    return False

rf = gh.get_result_async()

total = traj.points[-1].time_from_start.sec + 3.0

t0 = time.time()

while not rf.done() and time.time() - t0 < total + 5.0:

    time.sleep(0.1)

return True

def _exec_thread(self):

    try:

        # Eine Trajektorie: hover -> Abstieg -> Bogen(15) -> Aufstieg -> home

        wps = ([self._hover_first, self._arc_q[0]]

               + list(self._arc_q) + [self._hover_last, [0.0] * 6])

        vels = ([VEL_FREE, VEL_DESCEND]

               + [VEL_FREE] * len(self._arc_q) + [VEL_FREE, VEL_FREE])

        traj = self._build_traj(wps, vels)

        self._run_traj(traj, f'BOGEN wird gezeichnet ({len(self._arc_q)}wp) – Roboter bewegt sich...')

        self._status = 'FERTIG – Bogen gezeichnet, Roboter in Null. C: neu'

        self.get_logger().info(self._status)

    finally:

        self._busy = False

def home(self):

```

```

if self._busy:
    return

self._busy = True

try:
    traj = self._build_traj([[0.0] * 6], [VEL_FREE])

    self._run_traj(traj, 'HOME (Null)...')

finally:
    self._busy = False

# ---- overlay ----

def frame(self):
    with self._lock:
        if self._bgr is None:
            return None

        f = self._bgr.copy()

    h, w = f.shape[:2]

    rt = self._base_to_cam()

    # Bogen-Polyline (back-project): angeklickter Span=grün, Schätzung=orange

    if self._arc_base is not None and rt is not None:
        R, tv = rt

        nt = self._n_through

        if not self._valid:
            pix = [p for p in (self._proj(w, R, tv) for w in self._arc_base)
                    if p is not None]

            if len(pix) >= 2:
                cv2.polylines(f, [np.array(pix, np.int32)], False,
                              (0, 0, 255), 2, cv2.LINE_AA)

        else:
            thru = [p for p in (self._proj(w, R, tv)
                                for w in self._arc_base[:nt]) if p is not None]

            pred = [p for p in (self._proj(w, R, tv)
                                for w in self._arc_base[max(0, nt - 1):]) if p is not None]

            if len(thru) >= 2:
                cv2.polylines(f, [np.array(thru, np.int32)], False,
                              (0, 220, 0), 2, cv2.LINE_AA)

            if len(pred) >= 2:
                cv2.polylines(f, [np.array(pred, np.int32)], False,

```



```

        (0, 160, 255), 2, cv2.LINE_AA)

# angeklickte Punkte

for i, (u, v) in enumerate(self._px):

    cv2.drawMarker(f, (u, v), (255, 200, 0), cv2.MARKER_CROSS, 18, 2)

    cv2.circle(f, (u, v), 9, (255, 200, 0), 2)

    cv2.putText(f, str(i + 1), (u + 10, v - 8),

                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 200, 0), 2,

                cv2.LINE_AA)

cv2.rectangle(f, (0, 0), (w, 28), (40, 40, 40), -1)

cv2.putText(f, self._status, (8, 20), cv2.FONT_HERSHEY_SIMPLEX, 0.45,

            (255, 255, 255), 1, cv2.LINE_AA)

cv2.rectangle(f, (0, h - 22), (w, h), (40, 40, 40), -1)

cv2.putText(f, f'Links:Punkt({N_PTS}) P:fertig E:zeichnen H:home C:loeschen Q:beenden'

            '[gruen=angeklickt orange=Schaetzung]',

            (4, h - 6), cv2.FONT_HERSHEY_SIMPLEX, 0.4, (180, 180, 180),

            1, cv2.LINE_AA)

return f


def mouse_cb(event, x, y, flags, node):

    if event == cv2.EVENT_LBUTTONDOWN:

        node.on_click(x, y)

    elif event == cv2.EVENT_RBUTTONDOWN:

        node.clear()


def main():

    rclpy.init()

    node = ClickArc()

    threading.Thread(target=lambda: rclpy.spin(node), daemon=True).start()

    cv2.namedWindow('Click Arc', cv2.WINDOW_NORMAL)

    cv2.resizeWindow('Click Arc', 1280, 720)

    cv2.setMouseCallback('Click Arc', mouse_cb, node)

    print(f'\n CLICK ARC – {N_PTS} Punkte anklicken; Verlauf wird geschaetzt; '

          'mit E zeichnen (P=frueh fertig)\n')

    try:

        while rclpy.ok():

```

```

        fr = node.frame()

        if fr is not None:

            cv2.imshow('Click Arc', fr)

            k = cv2.waitKey(50) & 0xFF

            if k in (ord('e'), ord('E')):

                node.execute()

            elif k in (ord('p'), ord('P')):

                node.finalize()

            elif k in (ord('h'), ord('H')):

                node.home()

            elif k in (ord('c'), ord('C')):

                node.clear()

            elif k in (ord('q'), ord('Q')):

                break

    except KeyboardInterrupt:

        pass

finally:

    cv2.destroyAllWindows()

    node.destroy_node()

    rclpy.shutdown()

if __name__ == '__main__':

    main()

```

src/mycobot_calibration/scripts/click_place.py

```
#!/usr/bin/env python3

"""

Ein-Klick PLACE: Ablage-Punkt in der Kamera anklicken -> Roboter dreht die
WAAGERECHT zwischen den Fingern gehaltene Schraube SENKRECHT (Anflug waagerecht)
und führt sie über diesen Punkt -> Sicht-Kontrolle (mit Roll aufrichten) -> ablegen.

Einrichtung:

    Terminal 1: ros2 launch <vo_rsp> (TF: camera->robot_base, statisch eye-to-hand)
                ros2 launch realsense2_camera rs_launch.py (1280x720 + aligned depth)

    Terminal 2: python3 click_place.py (Bewegung mit pymycobot; öffnet DEN Port)

Mouse: Links = Ablage-Punkt wählen    Rechts = löschen

Keys:

    O / C = Greifer auf / zu (Schraube waagerecht laden, dann mit C greifen)

    G      = los: Schraube SENKRECHT halten, über das Ziel (langsam) [prüft erst IK]

    A      = Schrauben-Achse wechseln (kommt Schraube nicht senkrecht: Z->X->Y testen)

    R      = ablegen: Greifer auf -> gerade nach oben zurück -> Home

    Z      = Home (Null)

    Q      = beenden

"""

import math, threading, time

from typing import Optional, Tuple

import cv2, numpy as np

import rclpy

from rclpy.node import Node

from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy

from sensor_msgs.msg import Image, CameraInfo

from geometry_msgs.msg import PointStamped

from visualization_msgs.msg import Marker

import tf2_ros, tf2_geometry_msgs # noqa

CAMERA_FRAME='camera_color_optical_frame'; ROBOT_BASE='robot_base'

PORT='/dev/ttyTHS1'; BAUD=1000000 # serieller Port + Baudrate zum myCobot (Jetson UART)

HOVER_OFF=0.06 # TCP so weit über dem angeklickten Punkt (m)

RELEASE_UP=0.12 # nach dem Ablegen gerade nach oben zurück (m)
```

```

# ---- URDF FK (mycobot_280_jn) ----

# Eigenständige Vorwärtskinematik (Forward Kinematics), aus der URDF von Hand

# übernommen, damit dieses Skript ohne MoveIt-Dienste FK/IK rechnen kann. Die

# festen Joint-Origins (_0) und die Greifer-/TCP-Offsets (_FGB, _FTCP) entsprechen

# exakt den <origin>-Angaben der URDF-Joints.

def rpy(r,p,y):

    """Roll-Pitch-Yaw (URDF-Konvention) -> 3x3-Rotationsmatrix.

    Reihenfolge wie in URDF: R = Rz(yaw) @ Ry(pitch) @ Rx(roll) (extrinsisch xyz).

    """

    cr,sr=np.cos(r),np.sin(r);cp,sp=np.cos(p),np.sin(p);cy,sy=np.cos(y),np.sin(y)

    return np.array([[cy,-sy,0],[sy,cy,0],[0,0,1]])@np.array([[cp,0,sp],[0,1,0],[-sp,0,cp]])@np.array([[1,0,0],
[0,cr,-sr],[0,sr,cr]])

def Tt(xyz,rr):

    """Baut eine 4x4-Homogen-Transformation aus Translation xyz und rpy-Rotation rr."""

    M=np.eye(4);M[:3,:3]=rpy(*rr);M[:3,3]=xyz;return M

def Tz(q):

    """4x4-Rotation um die lokale Z-Achse um Winkel q (Gelenkbewegung eines Drehgelenks)."""

    M=np.eye(4);c,s=np.cos(q),np.sin(q);M[:3,:3]=[[c,-s,0],[s,c,0],[0,0,1]];return M

# _0: konstante Joint-Origins (Translation, rpy) der 6 Armgelenke aus der URDF.

_0=[((0,0,0.15756),(0,0,0)),((0,0,-0.001),(0,1.5708,-1.5708)),((-0.1104,0,0),(0,0,0)),

    ((-0.096,0,0.06462),(0,0,-1.5708)),((0,-0.07318,0),(1.5708,-1.5708,0)),((0,0.0456,0),(-1.5708,0,0))]

# _0C: dieselben Origins als vorberechnete 4x4-Matrizen. _FGB: fester Offset Flansch->

# gripper_base. _FTCP: Offset gripper_base->TCP (Werkzeugspitze zwischen den Fingern).

_0C=[Tt(*o) for o in _0]; _FGB=Tt((0,0,0.034),(1.579,0,0.8406)); _FTCP=Tt((0,0.110,-0.01),(0,0,0))

# _LU/_LL: obere/untere Gelenkwinkel-Grenzen (rad) aus der URDF; IK-Lösungen werden hierauf geklippt.

_LU=np.array([2.9321,2.4434,2.6179,2.6179,2.7925,3.14159]); _LL=-
np.array([2.9321,2.4434,2.6179,2.6179,2.7052,3.14159])

def fk(q):

    """Vorwärtskinematik: Gelenkwinkel q(6) -> (TCP-Position, gripper_base-Rotation, z-Höhen).

    Verkettet die 6 Joint-Origins mit den jeweiligen Z-Drehungen, hängt den Greifer-

    Offset an und gibt zurück: TCP-Position (3), Rotationsmatrix von gripper_base (3x3,

    für die Achsen-Ausrichtung), sowie die z-Höhen aller Gelenk-Origins + gripper_base

    (für Kollisions-/Boden-Abstandsprüfungen).

    """

    M=np.eye(4); zs=[]

```

```

for i in range(6): M=M@_OC[i]@Tz(q[i]); zs.append(M[2,3])

Mgb=M@_FGB; Mt=Mgb@_FTCP

return Mt[:3,3], Mgb[:3,:3], zs+[Mgb[2,3]]

_DOWN=np.array([0,0,-1.0]) # Ziel-Richtung "senkrecht nach unten" im robot_base-Frame

def err_align(q,p,axis):

    """pos(3) + Ausrichtung der gewählten Greifer-Achse nach UNTEN (Azimut frei).

    6er-Fehlervektor für die IK: die ersten 3 Komponenten sind der TCP-Positionsfehler
    (tcp - Ziel p). Die letzten 3 sind das Kreuzprodukt der gewählten Greifer-Achse mit
    der Nach-unten-Richtung; es wird 0, wenn die Achse exakt nach unten zeigt. Der
    Azimut (Drehung um die Vertikale) bleibt bewusst frei -> mehr Lösungen erreichbar.
    """

    tcp,Rc,_=fk(q)

    return np.concatenate([tcp-p, np.cross(Rc[:,axis],_DOWN)])

def ik_align(p,axis,seed,it=120):

    """Numerische inverse Kinematik (Gauß-Newton / Levenberg-Marquardt-Dämpfung).

    Sucht Gelenkwinkel q, sodass err_align klein wird. Pro Iteration wird die Jacobi-
    Matrix J per finiter Differenz (Störung 1e-6) gebildet und der gedämpfte
    Newton-Schritt  $dq = -(J^T J + 0.01 I)^{-1} J^T e$  angewandt; das Ergebnis wird auf die
    Gelenkgrenzen geklippt. Rückgabe: geklippte Lösung q und der Restfehler (Norm).
    """

    q=np.array(seed,float)

    for _ in range(it):

        e=err_align(q,p,axis)

        if np.linalg.norm(e)<1e-4: break

        J=np.zeros((6,6))

        for k in range(6):

            qq=q.copy();qq[k]+=1e-6;J[:,k]=(err_align(qq,p,axis)-e)/1e-6

        q=np.clip(q+np.linalg.solve(J.T@J+0.01*np.eye(6),-J.T@e),_LL,_LU)

    return q,float(np.linalg.norm(err_align(q,p,axis)))

# SEEDS: mehrere Start-Gelenkkonfigurationen. Weil die IK lokal konvergiert, wird sie mit
# jedem Seed gestartet und die beste erreichbare Lösung gewählt (Multi-Start gegen lokale Minima).

SEEDS=[[0,0.4,-2.0,0.3,0,0],[-1.0,0,-1.5,1.0,0,0],[0,-0.3,-1.0,0.8,0.8,0],[-0.5,0.3,-1.8,1.2,-0.5,0.5],

        [0.8,0.3,-1.8,0.5,0,0],[-0.3,-0.5,-0.8,1.0,1.2,0],[0.3,0.5,-2.2,0.0,-1.0,0.5],

        [-0.8,0.8,-2.2,0.3,0.5,-0.5],[0,0.8,-1.5,-0.5,1.0,0],[-1.2,0.3,-1.2,1.2,0.8,0.5]]

```

```

def img_bgr(m):
    """ROS-Farbbild -> OpenCV-BGR (rgb8 wird kanalgetauscht, sonst Kopie). Siehe click_to_go."""
    a=np.frombuffer(m.data,np.uint8)
    if m.encoding.lower()=='rgb8': return cv2.cvtColor(a.reshape((m.height,m.width,3)),cv2.COLOR_RGB2BGR)
    return a.reshape((m.height,m.width,3)).copy()

def depth_arr(m):
    """ROS-Tiefenbild -> 2D-Array. 16UC1=mm (uint16), 32FC1=m (float32); sonst Fehler."""
    if m.encoding=='16UC1': return np.frombuffer(m.data,np.uint16).reshape((m.height,m.width))
    if m.encoding=='32FC1': return np.frombuffer(m.data,np.float32).reshape((m.height,m.width))
    raise ValueError(m.encoding)

class ClickPlace(Node):
    """Ein-Klick-PLACE OHNE MoveIt: IK-Ziel finden und direkt per pymycobot anfahren.

    Ablauf: Punkt anklicken -> pixel_to_base liefert 3D-Ziel im robot_base-Frame ->
    _solve_place sucht eine Pose, in der die Schraube SENKRECHT gehalten wird (gewählte
    Greifer-Achse zeigt nach unten) -> 'G' fährt mit send_radians dorthin -> Sichtkontrolle
    -> 'R' öffnet den Greifer und fährt gerade nach oben zurück zu Home.

    ACHTUNG: Diese Variante plant KEINEN kollisionsfreien Weg (daher gibt es die sichere
    MoveIt-Variante click_place_moveit.py). Die GUI läuft im Haupt-Thread, rclpy.spin im
    Daemon-Thread; _busy serialisiert die pymycobot-Bewegungen.

    """
    def __init__(self):
        super().__init__('click_place')

        # Kamera-Intrinsik + letzte Bild-/Tiefenpuffer (spin-Thread schreibt, GUI liest; Lock schützt).
        self._K=None;self._bgr=None;self._depth=None;self._lock=threading.Lock()

        self._px=None;self._tgt=None      # angeklicktes Pixel bzw. dazugehöriger 3D-Punkt (robot_base)

        self._screw_axis=2                # welche Achse die Schraube in gripper_base ist (2=Z Annahme)

        self._place_q=None                # gefundene Zielpose (6 Gelenkwinkel), None solange keine gültige IK

        self._status='Warte auf Kamera...'

        # TF-Buffer/Listener: statische Kamera->Basis-Transformation (eye-to-hand-Kalibrierung).
        self._tf=tf2_ros.Buffer(); tf2_ros.TransformListener(self._tf,self)

        self._mk=self.create_publisher(Marker,'/click_place/marker',5) # Zielpunkt als RViz-Kugel

        # CameraInfo RELIABLE/depth=1: Intrinsik ist konstant, letzter Wert genügt.
        qos=QoSProfile(reliability=ReliabilityPolicy.RELIABLE,history=HistoryPolicy.KEEP_LAST,depth=1)

        self.create_subscription(CameraInfo,'/camera/color/camera_info',self._oninfo,qos)

```

```

self.create_subscription(Image, '/camera/color/image_raw', self._oncolor, 10)

self.create_subscription(Image, '/camera/aligned_depth_to_color/image_raw', self._ondepth, 10)

# pymycobot öffnet HIER den seriellen Port (nur dieses Skript darf ihn belegen).

# Bewegungen werden ausschließlich aus dem Haupt-Thread aufgerufen (serielle Kommunikation
# ist nicht thread-sicher). time.sleep gibt der Firmware Zeit, power_on zu quittieren.

from pymycobot import MyCobot280

self.mc=MyCobot280(PORT,BAUD); time.sleep(0.8)

try: self.mc.power_on()

except Exception: pass

self._busy=False # True während einer laufenden Bewegung -> blockiert neue Kommandos

def _oninfo(self,m): self._K=np.array(m.k,np.float64).reshape(3,3) # 3x3-Intrinsik zwischenspeichern
def _oncolor(self,m):

    try:

        with self._lock:

            self._bgr=img_bgr(m)

            if self._status=='Warte auf Kamera...': self._status='Bereit – Ablage-Punkt anklicken'

    except Exception: pass

def _ondepth(self,m):

    try:

        with self._lock: self._depth=depth_arr(m)

    except Exception: pass

def pixel_to_base(self,u,v):

    """Angeklicktes Pixel (u,v) -> 3D-Punkt im robot_base-Frame (identisch zu click_to_go).

    7x7-Median-Tiefe (robust gegen Rauschen/Löcher), mm->m, Lochkamera-Rückprojektion
    in den Kamera-optical-Frame, dann TF nach robot_base. None bei fehlender Intrinsik/
    Tiefe, Klick außerhalb des Bildes oder ungültigem Tiefenfenster.

    """

    if self._K is None: return None

    with self._lock: depth=self._depth

    if depth is None: return None

    h,w=depth.shape

    if not(0<=v<h and 0<=u<w): return None

    reg=depth[max(0,v-3):v+4,max(0,u-3):u+4]; val=reg[reg>0] # nur gültige Tiefen (>0)

    if len(val)==0: return None

```

```

z=float(np.median(val)); z=z/1000.0 if z>100 else z      # Median, mm->m falls nötig
fx,fy=self._K[0,0],self._K[1,1];cx,cy=self._K[0,2],self._K[1,2]
pc=PointStamped();pc.header.frame_id=CAMERA_FRAME;pc.header.stamp=self.get_clock().now().to_msg()
pc.point.x=(u-cx)*z/fx;pc.point.y=(v-cy)*z/fy;pc.point.z=z # inverse Lochkamera-Projektion
try:
    pb=self._tf.transform(pc,ROBOT_BASE,timeout=rclpy.time.Duration(seconds=2.0))
except Exception as e:
    self.get_logger().warn(f'TF: {e}'); return None
return np.array([pb.point.x,pb.point.y,pb.point.z])

def on_click(self,u,v):
    """Linksklick: Zielpunkt setzen, RViz-Marker publizieren, vorherige Lösung verwerfen.

    Während einer laufenden Bewegung (_busy) werden Klicks ignoriert. r = horizontaler
    Abstand von der Basisachse (Anzeige in mm).
    """
    if self._busy: return
    pb=self.pixel_to_base(u,v)
    if pb is None: self._status=f'Keine Tiefe ({u},{v})'; return
    self._px=(u,v); self._tgt=pb; self._place_q=None
    r=math.hypot(pb[0],pb[1])
    self._status=f'Ziel [{pb[0]:.3f},{pb[1]:.3f},{pb[2]:.3f}] r={r*1000:.0f}mm - G: los'
    self.get_logger().info(self._status)
    m=Marker();m.header.frame_id=ROBOT_BASE;m.header.stamp=self.get_clock().now().to_msg()
    m.ns='place';m.id=0;m.type=Marker.SPHERE;m.action=Marker.ADD
    m.pose.position.x=float(pb[0]);m.pose.position.y=float(pb[1]);m.pose.position.z=float(pb[2])
    m.pose.orientation.w=1.0;m.scale.x=m.scale.y=m.scale.z=0.02;m.color.b=1.0;m.color.a=1.0
    self._mk.publish(m)

def _solve_place(self):
    """Pose über Ziel, Schraube SENKRECHT (Anflug waagrecht). Bester Azimut wird gesucht.

    Zielpunkt wird um HOVER_OFF angehoben (TCP schwebt über der Ablage). Für jeden Seed
    wird die Achsen-Ausrichtungs-IK gelöst und die Lösung nur akzeptiert, wenn:
    - Restfehler klein (res<1.5e-3) und alle Gelenke innerhalb der Grenzen,
    - die gewählte Greifer-Achse tatsächlich nach UNTEN zeigt (Rc[2,ax]<-0.95),
    - der niedrigste Punkt der Kette über dem Boden bleibt (mz>0.02 m).
    """

```


Unter den gültigen Lösungen wird die mit dem größten Bodenabstand (mz) gewählt (sicherste Pose). Kein Treffer -> Meldung 'UNERREICHBAR' (dann A-Achse wechseln).

"""

```
if self._tgt is None: self._status='Erst einen Punkt waehlen!'; return False
```

```
p=np.array([self._tgt[0],self._tgt[1],self._tgt[2]+HOVER_OFF])
```

```
ax=self._screw_axis; best=None
```

```
for s in SEEDS:
```

```
    q,res=ik_align(p,ax,s)
```

```
    if res<1.5e-3 and np.all(q>=_LL-1e-6)and np.all(q<=_LU+1e-6):
```

```
        _,Rc,zs=fk(q); mz=min(zs)
```

```
        if Rc[2,ax]<-0.95 and mz>0.02 and (best is None or mz>best[1]): # Achse nach UNTEN + Bodenabstand
```

```
            best=(q,mz)
```

```
if best is None:
```

```
    self._place_q=None
```

```
    self._status=f'UNERREICHBAR (keine Schraube-senkrecht Pose, Achse={["X","Y","Z"][ax]}) – mit A Achse  
wechseln / anderer Punkt'
```

```
    return False
```

```
self._place_q,mz=best
```

```
self._status=f'Pose BEREIT (Schrauben-Achse={["X","Y","Z"][ax]}, Abstand {mz*1000:.0f}mm) – faehrt...'
```

```
return True
```

```
# ---- pymycobot Bewegung (Haupt-Thread) ----
```

```
def _go_radians(self,q,speed=20,wait=12):
```

```
    """Blockierendes Anfahren einer Gelenkkonfiguration q (rad) mit Geschwindigkeit speed.
```

```
Sendet send_radians und wartet, bis der Roboter still steht (is_moving()==0) oder
```

```
das Zeitlimit (wait Sekunden) erreicht ist. Das Blockieren stellt sicher, dass die
```

```
nächste Aktion erst nach abgeschlossener Bewegung startet.
```

```
"""
```

```
self.mc.send_radians([float(x) for x in q],speed)
```

```
for _ in range(wait):
```

```
    time.sleep(1.0)
```

```
    try:
```

```
        if self.mc.is_moving()==0: break
```

```
    except Exception: pass
```

```
def go(self):
```

```

        """'G': Schraube-senkrecht-Pose lösen und (langsam) anfahren. _busy sperrt Reentranz."""
        if self._busy: return
        self._busy=True
        try:
            if not self._solve_place(): return
            self.get_logger().info('G0: faehrt zu Schraube-senkrecht Pose (langsam)')
            self._go_radians(self._place_q,18)
            self._status='Angekommen – Schraube SENKRECHT? [ ] mit Roll justieren, R ablegen'
        finally: self._busy=False

def toggle_axis(self):
    """Wenn Schraube im Greifer auf anderer Achse (nicht senkrecht): Achse wechseln + erneut fahren.

    Wechselt zyklisch die als "Schraubenachse" angenommene gripper_base-Achse (Z->X->Y)
    und plant/fährt neu. Nötig, weil die Schraube je nach Ladeorientierung auf einer
    anderen lokalen Achse liegt und nur die richtige Achse "senkrecht nach unten" ergibt.
    """
    if self._busy or self._tgt is None: return
    self._screw_axis=(self._screw_axis+1)%3 # Z->X->Y Zyklus
    self._busy=True
    try:
        if self._solve_place(): self._go_radians(self._place_q,15)
        self._status=f'Schrauben-Achse={["X","Y","Z"][self._screw_axis]} – Schraube senkrecht? wenn nicht A,
wenn ja R'
    finally: self._busy=False

def grip(self,opn):
    """'O'/'C': Greifer öffnen (opn=True, Wert 100) bzw. schließen (Wert 0). Kraft 50, Modus 1."""
    if self._busy: return
    try: self.mc.set_gripper_value(100 if opn else 0,50,1)
    except Exception as e: self.get_logger().warn(str(e))
    self._status=('Greifer AUF' if opn else 'Greifer ZU')+ ' '

def release(self):
    """'R': Ablegen. Greifer öffnen, gerade nach oben zurückziehen, dann Home anfahren.

    Nach dem Öffnen wird per FK die aktuelle TCP-Pose bestimmt und ein Punkt RELEASE_UP

```

darüber angesteuert (gleiche Orientierung), damit der Greifer die abgelegte Schraube senkrecht verlässt und nicht seitlich streift. Danach zurück auf Null (Home).

HINWEIS: ik6(...) ist hier nicht definiert -> dieser Zweig würde einen NameError werfen; die sichere/gepflegte Ablage-Logik steckt in click_place_moveit.py.

```
"""
```

```
if self._busy or self._place_q is None: return
```

```
self._busy=True
```

```
try:
```

```
    self.get_logger().info('ABLEGEN: Greifer auf')
```

```
    self.mc.set_gripper_value(100,50,1); time.sleep(1.5)    # Greifer öffnen -> Schraube fällt/liegt ab
```

```
    tcp,Rc,_=fk(self._place_q)
```

```
    up=np.array([tcp[0],tcp[1],tcp[2]+RELEASE_UP])
```

```
    qup,res=ik6(up,Rc,self._place_q)    # gerade nach oben (gleiche Orientierung)
```

```
    if res<2e-3: self.get_logger().info('zurueck (nach oben)'); self._go_radians(qup,18)
```

```
    self.get_logger().info('home'); self._go_radians([0.0]*6,25)
```

```
    self.mc.set_gripper_value(100,50,1)
```

```
    self._status='ABGELEGT – Roboter in Null, Greifer auf'
```

```
finally: self._busy=False
```

```
def home(self):
```

```
    """'Z': alle Gelenke auf Null (Home) fahren."""
```

```
    if self._busy: return
```

```
    self._busy=True
```

```
    try: self._go_radians([0.0]*6,25)
```

```
    finally: self._busy=False
```

```
    self._status='HOME (Null)'
```

```
def frame(self):
```

```
    """Anzeige-Bild bauen: Klick-Marker (grün=Pose bereit, gelb=nur gewählt) + Status/Legende."""
```

```
    with self._lock:
```

```
        if self._bgr is None: return None
```

```
        f=self._bgr.copy()
```

```
    h,w=f.shape[:2]
```

```
    if self._px is not None:
```

```
        u,v=self._px; col=(0,255,0) if self._place_q is not None else (0,255,255)
```

```
        cv2.drawMarker(f,(u,v),col,cv2.MARKER_CROSS,22,2); cv2.circle(f,(u,v),10,col,2)
```

```
    cv2.rectangle(f,(0,0),(w,28),(40,40,40),-1)
```

```

cv2.putText(f,self._status,(8,20),cv2.FONT_HERSHEY_SIMPLEX,0.5,(255,255,255),1,cv2.LINE_AA)

cv2.rectangle(f,(0,h-22),(w,h),(40,40,40),-1)

cv2.putText(f,'Links:Punkt  O/C:Greifer  G:los  A:Schrauben-Achse  R:ablegen  Z:home  Q:beenden',
            (4,h-6),cv2.FONT_HERSHEY_SIMPLEX,0.45,(180,180,180),1,cv2.LINE_AA)

return f

```

```

def mouse(ev,x,y,fl,node):

```

```

    # Maus-Callback: Links = Ziel wählen, Rechts = Ziel/Lösung/Marker zurücksetzen.

    if ev==cv2.EVENT_LBUTTONDOWN: node.on_click(x,y)

    elif ev==cv2.EVENT_RBUTTONDOWN: node._px=None;node._tgt=None;node._place_q=None;node._status='geloescht'

```

```

def main():

```

```

    # ROS + Knoten starten, spin in Daemon-Thread, OpenCV-GUI im Haupt-Thread.

    rclpy.init(); node=ClickPlace()

    threading.Thread(target=lambda: rclpy.spin(node),daemon=True).start()

    cv2.namedWindow('Click Place',cv2.WINDOW_NORMAL); cv2.resizeWindow('Click Place',1280,720)

    cv2.setMouseCallback('Click Place',mouse,node)

    print('\n  CLICK PLACE – Punkt anklicken, O/C laden, G los, A Schrauben-Achse, R ablegen\n')

    # GUI-Hauptschleife (~20 Hz): Bild zeigen, Tasten auf Knoten-Aktionen abbilden.

```

```

    try:

```

```

        while rclpy.ok():

            fr=node.frame()

            if fr is not None: cv2.imshow('Click Place',fr)

            k=cv2.waitKey(50)&0xFF

            if k in (ord('o'),ord('O')): node.grip(True)

            elif k in (ord('c'),ord('C')): node.grip(False)

            elif k in (ord('g'),ord('G')): node.go()

            elif k in (ord('a'),ord('A')): node.toggle_axis()

            elif k in (ord('r'),ord('R')): node.release()

            elif k in (ord('z'),ord('Z')): node.home()

            elif k in (ord('q'),ord('Q')): break

```

```

    except KeyboardInterrupt: pass

```

```

    finally:

```

```

        cv2.destroyAllWindows(); node.destroy_node(); rclpy.shutdown()

```

```
| if __name__ == '__main__': main()
```

src/mycobot_calibration/scripts/click_place_moveit.py

```
#!/usr/bin/env python3
```

```
"""
```

Ein-Klick PLACE – MoveIt KOLLISIONSFREIER Plan + RViz-Vorschau + BESTÄTIGEN-ausführen.

Sichere Version von click_place (Kollisions-Problem: send_radians direkt, keine Wegplanung): Zielpose wird mit Achsen-Ausrichtungs-IK gefunden, MoveIt move_group plant kollisionsfreien WEG aus der ECHTEN Startpose (Plattform/Base im Modell), Vorschau in RViz, mit BESTÄTIGUNG (E) FOLGT pymycobot der geplanten Trajektorie.

Einrichtung:

```
demo.launch use_fake_hardware:=true use_rviz:=true (move_group + RViz + Modell)

rs_launch 1280x720 + aligned depth (Kamera)

python3 click_place_moveit.py (pymycobot Port wird HIER geöffnet)
```

Mouse: Links = Ablage-Punkt Rechts = löschen

Keys:

```
O / C = Greifer auf / zu (Schraube waagerecht laden, mit C greifen)

P      = PLANEN: Schraube-senkrecht Pose -> kollisionsfreier Weg -> RViz-Vorschau

E      = bestätigen & AUSFÜHREN (folgt dem geplanten Weg)

A      = Schrauben-Achse wechseln (kommt Schraube nicht senkrecht: Z->X->Y)

Pfeile / 8/2/4/6 = Ziel 1cm verschieben (xy) ; +/- = z 1cm (oben==X links==+Y)

S      = aktuellen Offset als PERMANENTEN Standard speichern (.place_offset.json)

H      = PLANEN home (Null) ; danach mit E ausführen

R      = ABLEGEN (Greifer auf)

Q      = beenden
```

```
"""
```

```
import math, os, json, threading, time
```

```
import cv2, numpy as np
```

```
import rclpy
```

```
from rclpy.node import Node
```

```
from rclpy.action import ActionClient
```

```
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy, DurabilityPolicy
```

```
from std_msgs.msg import Bool
```

```
from sensor_msgs.msg import Image, CameraInfo, JointState
```

```
from geometry_msgs.msg import PointStamped
```

```

from visualization_msgs.msg import Marker

from moveit_msgs.action import MoveGroup

from moveit_msgs.msg import Constraints, JointConstraint, DisplayTrajectory, RobotState

import tf2_ros, tf2_geometry_msgs # noqa

CAMERA_FRAME='camera_color_optical_frame'; ROBOT_BASE='robot_base'

PORT='/dev/ttyTHS1'; BAUD=1000000; GROUP='arm' # serieller Port/Baud + MoveIt-Planungsgruppe

# ARM_JOINTS: Reihenfolge der 6 Armgelenke exakt wie in URDF/MoveIt. Vertrag für
# JointConstraints, RobotState-Startpose und Trajektorien-Indizierung.

ARM_JOINTS=['joint2_to_joint1','joint3_to_joint2','joint4_to_joint3','joint5_to_joint4','joint6_to_joint5','joint6o
utput_to_joint6']

HOVER_OFF=0.13; PLAYBACK_VEL=0.5 # TCP-Schwebhöhe (m) über Ziel; rad/s Trajektorien-Wiedergabe (langsam/sicher)

# Permanente XY(-Z) Offset-Korrektur (TCP ~2.5cm + Seiten-Griff-Geometrie Band-Aid).

# wird im robot_base Frame als konstant angenommen; mit S hierhin geschrieben, beim Start geladen.

# Grund: der reale TCP weicht ~2.5cm von der Modellspitze ab (Seiten-Griff-Geometrie); statt

# das Modell zu korrigieren, wird ein konstanter Offset auf das Ziel addiert (dokumentierter Band-Aid).

OFFSET_FILE=os.path.expanduser('~/.ros2_ws/.place_offset.json')

def load_default_off():

    """Gespeicherten Standard-Offset (x,y,z in m) aus der JSON-Datei laden; sonst Null-Vektor."""

    try:

        with open(OFFSET_FILE) as f: d=json.load(f)

        return np.array([float(d['x']),float(d['y']),float(d['z'])])

    except Exception:

        return np.zeros(3)

# ---- FK + Achsen-Ausrichtungs-IK (mycobot_280_jn) ----

# Identische eigenständige FK/IK wie in click_place.py; hier dient sie NUR dazu, ein

# GELENK-Ziel (Schraube senkrecht) zu finden. Den kollisionsfreien WEG dorthin plant

# danach MoveIt (siehe _plan_thread). Zusätzlich gibt es hier fk_pts + in_base_box, um

# Kandidaten vorab auszusortieren, die in die Roboter-Basiskiste ragen würden.

def rpy(r,p,y):

    """Roll-Pitch-Yaw (URDF) -> 3x3-Rotation: Rz(y) @ Ry(p) @ Rx(r)."""

    cr,sr=np.cos(r),np.sin(r);cp,sp=np.cos(p),np.sin(p);cy,sy=np.cos(y),np.sin(y)

    return np.array([[cy,-sy,0],[sy,cy,0],[0,0,1]])@np.array([[cp,0,sp],[0,1,0],[-sp,0,cp]])@np.array([[1,0,0],
[0,cr,-sr],[0,sr,cr]])

def Tt(xyz,rr):

    """4x4-Homogen-Transformation aus Translation xyz und rpy-Rotation rr."""

    M=np.eye(4);M[:3,:3]=rpy(*rr);M[:3,3]=xyz;return M

```

```

def Tz(q):
    """4x4-Rotation um lokale Z-Achse um Gelenkwinkel q."""
    M=np.eye(4);c,s=np.cos(q),np.sin(q);M[:3,:3]=[c,-s,0],[s,c,0],[0,0,1];return M
# _0: Joint-Origins aus URDF; _OC: als Matrizen; _FGB: Flansch->gripper_base; _FTCP: gripper_base->TCP.
_0=[((0,0,0.15756),(0,0,0)),((0,0,-0.001),(0,1.5708,-1.5708)),((-0.1104,0,0),(0,0,0)),
    ((-0.096,0,0.06462),(0,0,-1.5708)),((0,-0.07318,0),(1.5708,-1.5708,0)),((0,0.0456,0),(-1.5708,0,0))]
_OC=[Tt(*o) for o in _0]; _FGB=Tt((0,0,0.034),(1.579,0,0.8406)); _FTCP=Tt((0,0.110,-0.01),(0,0,0))
# _LU/_LL: obere/untere Gelenkgrenzen (rad) aus URDF.
_LU=np.array([2.9321,2.4434,2.6179,2.6179,2.7925,3.14159]); _LL=-
np.array([2.9321,2.4434,2.6179,2.6179,2.7052,3.14159])
_DOWN=np.array([0,0,-1.0]) # "senkrecht nach unten" im robot_base-Frame
def fk(q):
    """FK: Gelenkwinkel q(6) -> (TCP-Position, gripper_base-Rotation, z-Höhen). Siehe click_place.fk."""
    M=np.eye(4); zs=[]
    for i in range(6): M=M@_OC[i]@Tz(q[i]); zs.append(M[2,3])
    Mgb=M@_FGB; return (Mgb@_FTCP)[:3,3], Mgb[:3,:3], zs+[Mgb[2,3]]
def fk_pts(q):
    """alle Gelenk-Origins + gripper_base Positionen (Base-Box Kollisions-Check).
    Liefert die 3D-Positionen aller Gelenk-Ursprungspunkte und des gripper_base. Damit
    prüft _place_joint_goal, ob eine IK-Lösung mit irgendeinem Kettenpunkt in die
    Basiskiste ragt (self-collision mit dem eigenen Sockel).
    """
    M=np.eye(4); P=[]
    for i in range(6): M=M@_OC[i]@Tz(q[i]); P.append(M[:3,3].copy())
    P.append((M@_FGB)[:3,3].copy()); return P
# robot_base Box: 0.15x0.11x0.11, z 0..0.11 (mycobot_world.urdf.xacro)
def in_base_box(p,pad=0.02):
    """True, wenn Punkt p (robot_base) innerhalb der Basiskiste (+ Sicherheitsrand pad) liegt.
    Die reale Basis ist ein Kasten von 0.15x0.11x0.11 m um den Ursprung; ein Kettenpunkt
    darin bedeutet Selbstkollision. pad=2cm gibt einen Sicherheitsabstand.
    """
    return abs(p[0])<0.075+pad and abs(p[1])<0.055+pad and -pad<p[2]<0.11+pad
def err_align(q,p,ax):
    """IK-Fehler: TCP-Positionsfehler (3) + Ausrichtung der Achse ax nach unten (3). Siehe click_place."""
    tcp,Rc,_=fk(q); return np.concatenate([tcp-p, np.cross(Rc[:,ax],_DOWN)])

```



```

def ik_align(p,ax,seed,it=120):

    """Gedämpfte Newton-IK (Levenberg-Marquardt), finite-Differenzen-Jacobi. Siehe click_place.ik_align."""

    q=np.array(seed,float)

    for _ in range(it):

        e=err_align(q,p,ax)

        if np.linalg.norm(e)<1e-4: break

        J=np.zeros((6,6))

        for k in range(6):

            qq=q.copy();qq[k]+=1e-6;J[:,k]=(err_align(qq,p,ax)-e)/1e-6

        q=np.clip(q+np.linalg.solve(J.T@J+0.01*np.eye(6),-J.T@e),_LL,_LU)

    return q,float(np.linalg.norm(err_align(q,p,ax)))

SEEDS=[[0,0.4,-2.0,0.3,0,0],[-1.0,0,-1.5,1.0,0,0],[0,-0.3,-1.0,0.8,0.8,0],[-0.5,0.3,-1.8,1.2,-0.5,0.5],

        [0.8,0.3,-1.8,0.5,0,0],[-0.3,-0.5,-0.8,1.0,1.2,0],[0.3,0.5,-2.2,0.0,-1.0,0.5],

        [-0.8,0.8,-2.2,0.3,0.5,-0.5],[0,0.8,-1.5,-0.5,1.0,0],[-1.2,0.3,-1.2,1.2,0.8,0.5]]

def img_bgr(m):

    """ROS-Farbbild -> OpenCV-BGR (rgb8 kanalgetauscht, sonst Kopie)."""

    a=np.frombuffer(m.data,np.uint8)

    if m.encoding.lower()=='rgb8': return cv2.cvtColor(a.reshape((m.height,m.width,3)),cv2.COLOR_RGB2BGR)

    return a.reshape((m.height,m.width,3)).copy()

def depth_arr(m):

    """ROS-Tiefenbild -> 2D-Array (16UC1=mm, 32FC1=m); unbekannte Kodierung -> Fehler."""

    if m.encoding=='16UC1': return np.frombuffer(m.data,np.uint16).reshape((m.height,m.width))

    if m.encoding=='32FC1': return np.frombuffer(m.data,np.float32).reshape((m.height,m.width))

    raise ValueError(m.encoding)

class ClickPlaceMoveit(Node):

    """SICHERE Ein-Klick-PLACE-Variante: eigenes IK-Gelenkziel + MoveIt-Wegplanung + Bestätigung.

    Unterschied zu click_place.py: statt direkt per send_radians zu fahren, wird das
    Schraube-senkrecht-Gelenkziel an MoveIt übergeben, das aus der ECHTEN Startpose
    (per pymycobot gelesen) einen KOLLISIONSFREIEN Weg plant und ihn in RViz als Ghost
    zeigt. Erst 'E' lässt pymycobot der geplanten Trajektorie (abgetastet) folgen.
    Zusätzlich: verschiebbarer Ziel-Offset ('nudge', Pfeile/8246/+-) mit persistentem
    Standard ('S' -> .place_offset.json), um TCP-Abweichung/Griff-Geometrie zu kompensieren.

    """

```

```

def __init__(self):
    super().__init__('click_place_moveit')

    # Kamera-Intrinsik + Bild/Tiefe (spin-Thread schreibt, GUI liest; Lock schützt).
    self._K=None;self._bgr=None;self._depth=None;self._lock=threading.Lock()

    self._def_off=load_default_off()    # persistenter Standard-Offset (beim Start geladen)

    # _off: aktueller kumulativer Offset (startet auf Standard); _screw_axis: angenommene Schraubenachse.
    self._px=None;self._tgt=None;self._screw_axis=2;self._off=self._def_off.copy()

    self._plan=None;self._plan_kind=''    # zuletzt geplante Trajektorie + Art ('place' / 'home')

    self._planning=False;self._busy=False    # Reentranz-Flags: Planung läuft / Ausführung läuft

    self._status='Warte auf Kamera/MoveGroup...'

    # TF: statische Kamera->Basis-Transformation (eye-to-hand-Kalibrierung).
    self._tf=tf2_ros.Buffer(); tf2_ros.TransformListener(self._tf,self)

    self._mk=self.create_publisher(Marker,'/click_place/marker',5)        # Zielpunkt-Marker (RViz)

    self._disp=self.create_publisher(DisplayTrajectory,'/display_planned_path',5)    # geplanter Ghost (RViz)

    self._mg=ActionClient(self,MoveGroup,'/move_action')    # MoveIt-Wegplanung

    qos=QoSProfile(reliability=ReliabilityPolicy.RELIABLE,history=HistoryPolicy.KEEP_LAST,depth=1)

    self.create_subscription(CameraInfo,'/camera/color/camera_info',self._oninfo,qos)

    self.create_subscription(Image,'/camera/color/image_raw',self._oncolor,10)

    self.create_subscription(Image,'/camera/aligned_depth_to_color/image_raw',self._ondepth,10)

    # pymycobot: seriellen Port öffnen; hier NUR für Startpose-Lesung und Trajektorien-Wiedergabe.

    from pymycobot import MyCobot280

    self.mc=MyCobot280(PORT,BAUD); time.sleep(0.8)

    try: self.mc.power_on()

    except Exception: pass

    # NOT-AUS: dieses Werkzeug schreibt per pymycobot DIREKT auf den Port – der

    # Bridge-Enforcement-Weg greift hier nicht. Darum eigener /estop-Abonnent

    # (gelatcht, gleiche QoS wie estop_button): STOP -> sofort mc.stop() aus dem

    # Spin-Thread (pymycobot serialisiert intern per thread_lock) + Abbruch der

    # Wiedergabe-Schleife in _exec_thread über das Flag.

    eq=QoSProfile(depth=1)

    eq.durability=DurabilityPolicy.TRANSIENT_LOCAL

    eq.reliability=ReliabilityPolicy.RELIABLE

    self._estop=False

    self.create_subscription(Bool,'/estop',self._on_estop,eq)

def _on_estop(self,m):
    """NOT-AUS-Callback: Flag setzen; bei STOP sofort mc.stop() – die Servos behalten

```

```

ihr Drehmoment (der Arm fällt NICHT), nur die Bewegung wird angehalten. Der
alte Plan wird verworfen (Startpose stimmt nach dem Abbruch nicht mehr)."""
was=self._estop; self._estop=bool(m.data)

if self._estop:
    self._plan=None

    for _ in range(3):
        try: self.mc.stop()

        except Exception as e: self.get_logger().warn(f'NOT-AUS stop(): {e}')

        time.sleep(0.02)

    self._status='*** NOT-AUS *** – Bewegung gestoppt'

    self.get_logger().warn(self._status)

elif was:
    self._status='NOT-AUS freigegeben – mit P/H neu planen'

    self.get_logger().info(self._status)

def _oninfo(self,m): self._K=np.array(m.k,np.float64).reshape(3,3) # 3x3-Intrinsik puffern
def _oncolor(self,m):
    try:
        with self._lock:
            self._bgr=img_bgr(m)

            if self._status.startswith('Warte'): self._status='Bereit – Ablage-Punkt anklicken'
    except Exception: pass

def _ondepth(self,m):
    try:
        with self._lock: self._depth=depth_arr(m)
    except Exception: pass

def _cur_real(self):
    """Aktuelle ECHTE Gelenkstellung (rad) vom Roboter lesen (pymycobot get_radians).

    Wird als MoveIt-Startpose (RobotState) benutzt, damit der Planer aus der tatsächlichen
    Ist-Lage plant und nicht aus einer fake /joint_states. Fallback [0]*6 bei Lesefehler.
    """
    try:
        a=self.mc.get_radians()

        if a and len(a)==6: return [float(x) for x in a]
    except Exception: pass

```

```
return [0.0]*6
```

```
def pixel_to_base(self,u,v):
```

```
    """Pixel (u,v) -> 3D-Punkt im robot_base-Frame (Median-Tiefe, Rückprojektion, TF). Siehe click_place."""
```

```
    if self._K is None: return None
```

```
    with self._lock: depth=self._depth
```

```
    if depth is None: return None
```

```
    h,w=depth.shape
```

```
    if not(0<=v<h and 0<=u<w): return None
```

```
    reg=depth[max(0,v-3):v+4,max(0,u-3):u+4]; val=reg[reg>0]
```

```
    if len(val)==0: return None
```

```
    z=float(np.median(val)); z=z/1000.0 if z>100 else z
```

```
    fx,fy=self._K[0,0],self._K[1,1];cx,cy=self._K[0,2],self._K[1,2]
```

```
    pc=PointStamped();pc.header.frame_id=CAMERA_FRAME;pc.header.stamp=self.get_clock().now().to_msg()
```

```
    pc.point.x=(u-cx)*z/fx;pc.point.y=(v-cy)*z/fy;pc.point.z=z
```

```
    try: pb=self._tf.transform(pc,ROBOT_BASE,timeout=rclpy.time.Duration(seconds=2.0))
```

```
    except Exception as e: self.get_logger().warn(f'TF:{e}'); return None
```

```
    return np.array([pb.point.x,pb.point.y,pb.point.z])
```

```
def on_click(self,u,v):
```

```
    """Linksklick: Ziel setzen, Offset auf Standard zurücksetzen, RViz-Marker + Status. Ignoriert während Bewegung/Planung."""
```

```
    if self._busy or self._planning: return
```

```
    pb=self.pixel_to_base(u,v)
```

```
    if pb is None: self._status=f'Keine Tiefe ({u},{v})'; return
```

```
    self._px=(u,v);self._tgt=pb;self._plan=None;self._off=self._def_off.copy()
```

```
    self._status=f'Ziel [{pb[0]:.3f},{pb[1]:.3f},{pb[2]:.3f}] r={math.hypot(pb[0],pb[1])*1000:.0f}mm - P:  
planen'
```

```
    self.get_logger().info(self._status)
```

```
    m=Marker();m.header.frame_id=ROBOT_BASE;m.header.stamp=self.get_clock().now().to_msg()
```

```
    m.ns='place';m.id=0;m.type=Marker.SPHERE;m.action=Marker.ADD
```

```
    m.pose.position.x=float(pb[0]);m.pose.position.y=float(pb[1]);m.pose.position.z=float(pb[2])
```

```
    m.pose.orientation.w=1.0;m.scale.x=m.scale.y=m.scale.z=0.02;m.color.b=1.0;m.color.a=1.0
```

```
    self._mk.publish(m)
```

```
def _place_joint_goal(self):
```

```
    """Gelenk-Zielkonfiguration für die aktuelle Ablage suchen (Schraube senkrecht, Base-Box-frei).
```

```

Ziel = angeklickter Punkt + kumulativer Offset + HOVER_OFF in z (Schwebhöhe). Für
jeden Seed wird ik_align gelöst und die Lösung akzeptiert, wenn: Restfehler klein,
Gelenkgrenzen eingehalten, gewählte Greifer-Achse zeigt nach unten ( $Rc[2,ax] < -0.95$ ),
Bodenabstand  $mz > 0.02$  UND kein Kettenpunkt in der Basiskiste (in_base_box). Gewählt
wird die Lösung mit größtem Bodenabstand. Rückgabe: (Gelenkliste, Info-Text) oder
(None, Fehlertext).

"""

if self._tgt is None: return None, 'Erst einen Punkt waehlen!'

p=np.array([self._tgt[0]+self._off[0],self._tgt[1]+self._off[1],self._tgt[2]+self._off[2]+HOVER_OFF]);
ax=self._screw_axis; best=None

for s in SEEDS:

    q,res=ik_align(p,ax,s)

    if res<1.5e-3 and np.all(q>=_LL-1e-6)and np.all(q<=_LU+1e-6):

        _,Rc,zs=fk(q); mz=min(zs)

        if (Rc[2,ax]<-0.95 and mz>0.02

            and not any(in_base_box(pt) for pt in fk_pts(q)) # nicht in die Base-Box ragen

            and (best is None or mz>best[1])): best=(q,mz)

        if best is None: return None,f'Keine IK (keine Schraube-senkrecht + Base-Box-freie Pose,
Achse=["X","Y","Z"][ax]) – ferner/hoeherer Punkt oder A'

        return list(best[0]),f'Schrauben-Achse=["X","Y","Z"][ax], Abstand {best[1]*1000:.0f}mm'

def plan_place(self):

    """'P': Ablage-Gelenkziel suchen und (falls gefunden) MoveIt-Wegplanung dafür anstoßen."""

    if self._busy or self._planning: return

    goal,msg=self._place_joint_goal()

    if goal is None: self._status=msg; return

    self._plan_kind='place'; self._do_plan(goal,f'WIRD GEPLANT (place: {msg})')

def plan_home(self):

    """'H': Weg zur Null-Stellung (Home) planen (danach mit 'E' ausführen)."""

    if self._busy or self._planning: return

    self._plan_kind='home'; self._do_plan([0.0]*6,'WIRD GEPLANT (home)')

def _do_plan(self,joint_goal,msg):

    """Planung im Hintergrund-Thread starten (GUI bleibt reaktiv). Setzt _planning."""

    self._planning=True;self._plan=None;self._status=msg;self.get_logger().info(msg)

    threading.Thread(target=self._plan_thread,args=(joint_goal,),daemon=True).start()

```

```

def _plan_thread(self, joint_goal):

    """MoveIt-Wegplanung (plan_only) für ein Gelenk-Ziel; synchron per Polling der Futures.

    Baut ein MoveGroup-Goal mit: Gruppe, Planungsversuchen/-zeit, konservativem
    Geschwindigkeits-/Beschleunigungs-Scaling (0.25). Wichtig: als Startpose wird die
    ECHTE, per pymycobot gelesene Ist-Lage gesetzt (statt der fake /joint_states aus
    use_fake_hardware) – nur so plant MoveIt kollisionsfrei relativ zur wahren Roboterlage.
    Das Ziel wird als JointConstraints (enge Toleranz 0.03) übergeben, plan_only=True.
    Bei Erfolg (error_code==1) wird die Trajektorie gespeichert und als DisplayTrajectory
    für die RViz-Vorschau publiziert; Fehler/Kollision -> Statusmeldung, kein Plan.
    """

    if not self._mg.wait_for_server(timeout_sec=4.0):

        self._status='move_group FEHLT!';self._planning=False;return

    goal=MoveGroup.Goal();req=goal.request;req.group_name=GROUP

    req.num_planning_attempts=10;req.allowed_planning_time=5.0

    req.max_velocity_scaling_factor=0.25;req.max_acceleration_scaling_factor=0.25

    # ECHTE Startpose (pymycobot-Lesung statt fake /joint_states)

    st=RobotState();js=JointState();js.name=ARM_JOINTS;js.position=self._cur_real()

    st.joint_state=js;st.is_diff=False;req.start_state=st

    # Arbeitsraum-Grenzen (+/-1 m) als Suchraum-Begrenzung.

    wp=req.workspace_parameters;wp.header.frame_id=ROBOT_BASE

    wp.min_corner.x=wp.min_corner.y=wp.min_corner.z=-1.0

    wp.max_corner.x=wp.max_corner.y=wp.max_corner.z=1.0

    # Gelenk-Ziel als JointConstraints (eng, damit MoveIt genau unsere IK-Lösung ansteuert).

    c=Constraints()

    for n,v in zip(ARM_JOINTS,joint_goal):

        jc=JointConstraint();jc.joint_name=n;jc.position=float(v)

        jc.tolerance_above=0.03;jc.tolerance_below=0.03;jc.weight=1.0

        c.joint_constraints.append(jc)

    req.goal_constraints.append(c)

    goal.planning_options.plan_only=True # nur planen, nicht ausführen (Vorschau + 'E'-Bestätigung)

    # Goal senden und auf Annahme pollen (max 6 s), da dieser Thread synchron arbeitet.

    fut=self._mg.send_goal_async(goal)

    t0=time.time()

    while not fut.done() and time.time()-t0<6: time.sleep(0.05)

    gh=fut.result()

    if gh is None or not gh.accepted: self._status='Plan ABGELEHNT';self._planning=False;return

```

```

# Auf das eigentliche Planungsergebnis warten (max 10 s).

rf=gh.get_result_async();t0=time.time()

while not rf.done() and time.time()-t0<10: time.sleep(0.05)

res=rf.result().result if rf.done() else None

if res is None or res.error_code.val!=1:

    code=res.error_code.val if res else '??'

    self._status=f'Plan FEHLGESCHLAGEN (code={code}) – evtl. Kollision/nicht
erreichbar';self._plan=None;self._planning=False;return

self._plan=res.planned_trajectory

# Ghost in RViz anzeigen.

d=DisplayTrajectory();d.trajectory_start=res.trajectory_start;d.trajectory.append(res.planned_trajectory)

self._disp.publish(d)

n=len(res.planned_trajectory.joint_trajectory.points)

self._status=f'PLAN BEREIT ({n} Punkte) – RViz-Vorschau, E: bestaetigen&ausfuehren'

self.get_logger().info(self._status);self._planning=False


def execute(self):

    """'E': geplante Trajektorie ausführen (im Hintergrund-Thread). Nur wenn ein Plan vorliegt."""

    if self._estop:

        self._status='NOT-AUS aktiv – erst freigeben!'; return

    if self._busy or self._planning or self._plan is None:

        if self._plan is None: self._status='Erst mit P/H planen!'

        return

    self._busy=True;threading.Thread(target=self._exec_thread,daemon=True).start()


def _exec_thread(self):

    """Der geplanten MoveIt-Trajektorie folgen, indem abgetastete Wegpunkte per send_radians angefahren werden.

    Die MoveIt-Trajektorie kann viele Punkte haben; sie wird auf ~18 Stützstellen
    heruntergerechnet (step), der Endpunkt aber immer angehängt, damit das exakte Ziel
    erreicht wird. names.index bildet die Trajektorien-Gelenkreihenfolge auf ARM_JOINTS ab
    (die Reihenfolge muss nicht übereinstimmen). Pro Wegpunkt wird die Wartezeit aus dem
    größten Gelenk-Delta und PLAYBACK_VEL (rad/s) berechnet -> gleichmäßig langsame,
    sichere Wiedergabe. Am Ende wird auf Stillstand gewartet (is_moving).

    """

    try:

        jt=self._plan.joint_trajectory;names=jt.joint_names

```

```

idx=[names.index(j) for j in ARM_JOINTS] # Trajektorien-Spalten -> ARM_JOINTS-Reihenfolge

pts=jt.points

step=max(1,len(pts)//18); sel=list(pts[:,step]) # auf ~18 Stützstellen abtasten

if sel[-1] is not pts[-1]: sel.append(pts[-1]) # Endpunkt garantiert mitnehmen

self._status=f'LAEUFT (folgt geplantem Weg, {len(sel)} Punkte)...'

self.get_logger().info(self._status)

prev=self._cur_real()

for p in sel:

    # NOT-AUS: Wiedergabe sofort abbrechen (mc.stop() hat der Callback

    # bereits gesendet); finally setzt _busy zurück.

    if self._estop:

        self._status='*** NOT-AUS *** – Wiedergabe abgebrochen'

        self.get_logger().warn(self._status)

        return

    q=[p.positions[i] for i in idx]

    dmax=max(abs(a-b) for a,b in zip(q,prev)) # größte Gelenkänderung zum Vorpunkt

    self.mc.send_radians([float(x) for x in q],30)

    # Wartezeit ~ Weg/Geschwindigkeit – in 50-ms-Schritten, damit ein

    # NOT-AUS auch WAEREND eines langen Wegpunkt-Schritts die Schleife beendet.

    t_wait=max(0.08,dmax/PLAYBACK_VEL); t0=time.time()

    while time.time()-t0<t_wait and not self._estop:

        time.sleep(0.05)

    prev=q

# Falls NOT-AUS genau beim letzten Wegpunkt kam: nicht "fertig" melden.

if self._estop:

    self._status='*** NOT-AUS *** – Wiedergabe abgebrochen'

    self.get_logger().warn(self._status)

    return

for _ in range(8):

    time.sleep(0.5)

    try:

        if self.mc.is_moving()==0: break

    except Exception: pass

self._plan=None # Plan verbraucht -> kein versehentliches erneutes Ausführen

self._status=('PLATZIERT – mit R ablegen' if self._plan_kind=='place' else 'HOME fertig')

self.get_logger().info(self._status)

finally: self._busy=False

```



```

def grip(self,opn):
    """'O'/'C': Greifer öffnen (Wert 100) / schließen (Wert 0), Kraft 50, Modus 1."""
    if self._busy or self._estop: return
    try: self.mc.set_gripper_value(100 if opn else 0,50,1)
    except Exception as e: self.get_logger().warn(str(e))
    self._status='Greifer '+'(AUF' if opn else 'ZU')

def release(self):
    """'R': Greifer öffnen (Schraube ablegen). Danach H (home planen) + E (ausführen) zum Zurückfahren."""
    if self._busy or self._estop: return
    try: self.mc.set_gripper_value(100,50,1)
    except Exception as e: self.get_logger().warn(str(e))
    self._status='ABGELEGT (Greifer auf) – H dann E fuer home'

def toggle_axis(self):
    """'A': angenommene Schraubenachse zyklisch wechseln (Z->X->Y); alten Plan verwerfen, neu planen nötig."""
    if self._busy or self._planning: return
    self._screw_axis=(self._screw_axis+1)%3;self._plan=None
    self._status=f'Schrauben-Achse=[{"X","Y","Z"}[self._screw_axis]] – mit P erneut planen'

def nudge(self,dx,dy,dz):
    """Ziel in robot_base lcm verschieben + erneut planen (Greifer über das Loch bringen)."""
    if self._busy or self._planning or self._tgt is None: return
    self._off=self._off+np.array([dx,dy,dz])
    self._status=f'offset x={self._off[0]*100:+.0f} y={self._off[1]*100:+.0f} z={self._off[2]*100:+.0f}cm ->
wird geplant'
    self.get_logger().info(f'>> NUDGE kumulativer offset (robot_base): x={self._off[0]*100:+.1f}
y={self._off[1]*100:+.1f} z={self._off[2]*100:+.1f} cm (mit S permanent speichern)')
    self.plan_place()

def save_offset(self):
    """Aktuellen kumulativen Offset als permanenten Standard auf Platte schreiben."""
    try:
        with open(OFFSET_FILE,'w') as f:
            json.dump({'x':float(self._off[0]),'y':float(self._off[1]),'z':float(self._off[2])},f,indent=2)
            self._def_off=self._off.copy()
            self._status=f'OFFSET GESPEICHERT x={self._off[0]*100:+.1f} y={self._off[1]*100:+.1f}
z={self._off[2]*100:+.1f}cm -> {OFFSET_FILE}'
            self.get_logger().info(self._status)

```

```

except Exception as e:

    self._status=f'offset Speicher-FEHLER: {e}'; self.get_logger().error(self._status)

def frame(self):

    """Anzeige-Bild bauen: Klick-Marker (grün=Plan bereit, gelb=nur gewählt), aktueller + Standard-Offset,
    Status, Legende."""

    with self._lock:

        if self._bgr is None: return None

        f=self._bgr.copy()

        h,w=f.shape[:2]

        if self._px is not None:

            u,v=self._px;col=(0,255,0) if self._plan is not None else (0,255,255)

            cv2.drawMarker(f,(u,v),col,cv2.MARKER_CROSS,22,2);cv2.circle(f,(u,v),10,col,2)

            cv2.rectangle(f,(0,0),(w,50),(40,40,40),-1)

            cv2.putText(f,self._status,(8,20),cv2.FONT_HERSHEY_SIMPLEX,0.5,(255,255,255),1,cv2.LINE_AA)

            off_txt=(f'offset x={self._off[0]*100:+.1f} y={self._off[1]*100:+.1f} z={self._off[2]*100:+.1f}cm'

                    f' | Standard x={self._def_off[0]*100:+.1f} y={self._def_off[1]*100:+.1f}

                    z={self._def_off[2]*100:+.1f}')

            cv2.putText(f,off_txt,(8,42),cv2.FONT_HERSHEY_SIMPLEX,0.42,(0,220,255),1,cv2.LINE_AA)

            cv2.rectangle(f,(0,h-22),(w,h),(40,40,40),-1)

            cv2.putText(f,'Links:Punkt O/C:Greifer P:Plan E:ausfuehren Pfeile|8/2/4/6:verschieben(xy) +/-:z A:Achse

            S:speichern H:home R:ablegen Q:beenden',

                    (4,h-6),cv2.FONT_HERSHEY_SIMPLEX,0.4,(180,180,180),1,cv2.LINE_AA)

        return f

# Pfeiltasten -> nudge (NumLock-unabhängig). waitKeyEx gibt VOLLES keysym zurück.

# ACHTUNG (2026-07-16, live beobachtet): dieser OpenCV/GTK-Build liefert Keysyms mit

# 0x100000-Präfix (KP_2 -> 1114034 = 0x10FFB2) – deshalb wird in der Hauptschleife

# mit kx&0xFFFF auf das nackte X11-Keysym normalisiert, sonst trifft KEIN Eintrag.

# Sowohl normale Pfeile (X11 0xFF51-54), Numpad-Pfeile (NumLock aus, KP_* 0xFF96-99)

# als auch Numpad-ZIFFERN (NumLock an, KP_0.. 0xFFB0-B9) und KP_+/- sind gebunden.

# Richtung = wie Numpad 8/2/4/6: oben=+X, unten=-X, links=+Y, rechts=-Y.

ARROW_KEYS={

    65362:(0.01,0,0), 65431:(0.01,0,0), # Up / KP_Up

    65364:(-0.01,0,0), 65433:(-0.01,0,0), # Down / KP_Down

    65361:(0,0.01,0), 65430:(0,0.01,0), # Left / KP_Left

    65363:(0,-0.01,0), 65432:(0,-0.01,0), # Right / KP_Right

```

```

65464:(0.01,0,0),          # KP_8 (NumLock an)
65458:(-0.01,0,0),        # KP_2 (NumLock an)
65460:(0,0.01,0),        # KP_4 (NumLock an)
65462:(0,-0.01,0),        # KP_6 (NumLock an)
65451:(0,0,0.01),        # KP_Add      (+z)
65453:(0,0,-0.01),       # KP_Subtract (-z)
}

def mouse(ev,x,y,fl,node):

    # Maus-Callback: Links = Ziel wählen, Rechts = Ziel/Plan/Marker zurücksetzen.

    if ev==cv2.EVENT_LBUTTONDOWN: node.on_click(x,y)

    elif ev==cv2.EVENT_RBUTTONDOWN: node._px=None;node._tgt=None;node._plan=None;node._status='geloescht'

def main():

    # ROS + Knoten starten, spin in Daemon-Thread, OpenCV-GUI im Haupt-Thread.

    rclpy.init();node=ClickPlaceMoveit()

    threading.Thread(target=lambda: rclpy.spin(node),daemon=True).start()

    cv2.namedWindow('Click Place (MoveIt)',cv2.WINDOW_NORMAL);cv2.resizeWindow('Click Place (MoveIt)',1280,720)

    cv2.setMouseCallback('Click Place (MoveIt)',mouse,node)

    print('\n CLICK PLACE (MoveIt) – Punkt anklicken, O/C laden, P planen(RViz), E ausfuehren\n')

    # GUI-Hauptschleife (~20 Hz). waitKeyEx liefert den VOLLEN Tastencode (für Pfeiltasten);

    # k=kx&0xFF ist der ASCII-Anteil für normale Buchstaben-/Zeichentasten.

    try:

        while rclpy.ok():

            fr=node.frame()

            if fr is not None: cv2.imshow('Click Place (MoveIt)',fr)

            kx=cv2.waitKeyEx(50); k=kx&0xFF

            # ksym: nacktes X11-Keysym (GTK-Build liefert Sondertasten mit 0x100000-Präfix)

            ksym=kx&0xFFFF

            if kx>255 and ksym in ARROW_KEYS: node.nudge(*ARROW_KEYS[ksym])

            elif k in (ord('o'),ord('O')): node.grip(True)

            elif k in (ord('c'),ord('C')): node.grip(False)

            elif k in (ord('p'),ord('P')): node.plan_place()

            elif k in (ord('e'),ord('E')): node.execute()

            elif k in (ord('a'),ord('A')): node.toggle_axis()

            elif k in (ord('s'),ord('S')): node.save_offset()

            elif k==ord('8'): node.nudge(0.01,0,0)

```

```

        elif k==ord('2'): node.nudge(-0.01,0,0)

        elif k==ord('4'): node.nudge(0,0.01,0)

        elif k==ord('6'): node.nudge(0,-0.01,0)

        elif k==ord('+'): node.nudge(0,0,0.01)

        elif k==ord('-'): node.nudge(0,0,-0.01)

        elif k in (ord('h'),ord('H')): node.plan_home()

        elif k in (ord('r'),ord('R')): node.release()

        elif k in (ord('q'),ord('Q')): break

        elif kx>255: node.get_logger().info(f'[Taste] nicht zugeordneter Sonder-Tastencode={kx} (falls
Pfeiltaste, melde es mir, ich binde sie)')

    except KeyboardInterrupt: pass

    finally:

        cv2.destroyAllWindows();node.destroy_node();rclpy.shutdown()

if __name__=='__main__': main()

```

src/mycobot_calibration/scripts/click_to_go.py

```
#!/usr/bin/env python3
```

```
"""
```

Kamera-Bild anklicken -> MoveIt Plan -> in RViz sehen -> ausführen.

Pipeline:

Pixel -> aligned depth -> 3D Kamera-Punkt -> TF robot_base

-> MoveIt Plan (oranger Ghost in RViz) -> Benutzer-Bestätigung -> ausführen

Verwendung:

Terminal 1: ~/ros2_ws/start_mycobot.sh

Terminal 2: python3 ~/ros2_ws/src/mycobot_calibration/scripts/click_to_go.py

Mouse:

Links klicken – Zielpunkt wählen (gelber Marker)

Rechts klicken – löschen

Keys:

P – Plan! Mit MoveIt Weg planen (in RViz sichtbar)

E – ausführen! Plan ausführen (Roboter bewegt sich)

H – Home (alle Gelenke 0)

Q – beenden

```
"""
```

```
import os
```

```
import threading
```

```
from typing import Optional, Tuple
```

```
import cv2
```

```
import numpy as np
```

```
import rclpy
```

```
from rclpy.node import Node
```

```
from rclpy.action import ActionClient
```

```
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy
```

```
from sensor_msgs.msg import Image, CameraInfo, JointState
```

```

from geometry_msgs.msg import PoseStamped, PointStamped

from visualization_msgs.msg import Marker

from control_msgs.action import FollowJointTrajectory

from trajectory_msgs.msg import JointTrajectoryPoint

from builtin_interfaces.msg import Duration

from moveit_msgs.action import MoveGroup

from moveit_msgs.msg import (
    Constraints, JointConstraint, DisplayTrajectory, RobotTrajectory,
)

import tf2_ros

import tf2_geometry_msgs # noqa: F401

from mycobot_calibration.numerical_ik import NumericalIK

# --- Feste Frame- und Gruppennamen (müssen mit URDF/SRDF/MoveIt-Config übereinstimmen) ---
# CAMERA_FRAME: optischer Rahmen der RealSense-Farbkamera. In diesem Frame liefert die
# Kamera-Rückprojektion (Pixel -> 3D) ihre Punkte. Achsenkonvention "optical": z nach
# vorne (in die Szene), x nach rechts, y nach unten.
# ROBOT_BASE_FRAME: Roboter-Basis-Frame. Ziel-Frame nach der TF-Transformation; hier plant
# MoveIt und hier wird der Reichweiten-Kreis definiert. Die statische eye-to-hand-Kalibrierung
# liefert die konstante Transformation camera_color_optical_frame -> robot_base.
# TCP_FRAME: Tool Center Point (Werkzeugmittelpunkt) des Greifers.
# PLANNING_GROUP: Name der MoveIt-Planungsgruppe (die 6 Armgelenke), aus der SRDF.
CAMERA_FRAME = 'camera_color_optical_frame'
ROBOT_BASE_FRAME = 'robot_base'

# Schweben-Aufschlag (m) über dem angeklickten Punkt, per Umgebungsvariable CLICK_HOVER_M
# (z.B. 0.04 = 4 cm). 0.0 = Ziel exakt auf angeklickter Höhe (bisheriges Verhalten).
CLICK_HOVER_M = float(os.environ.get('CLICK_HOVER_M', '0.0'))

TCP_FRAME = 'tcp'

PLANNING_GROUP = 'arm'

# Reihenfolge der 6 Armgelenke exakt wie in der URDF/MoveIt-Config. Diese Reihenfolge
# ist ein Vertrag: JointConstraints und Trajektorien-Punkte werden nach diesen Namen
# indiziert; eine Umsortierung würde die falschen Gelenke ansteuern.
ARM_JOINTS = [
    'joint2_to_joint1',

```

```

'joint3_to_joint2',
'joint4_to_joint3',
'joint5_to_joint4',
'joint6_to_joint5',
'joint6output_to_joint6',
]

HOME_RADIANS = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]

def image_msg_to_bgr(msg: Image) -> np.ndarray:
    """Wandelt eine ROS sensor_msgs/Image (Farbe) in ein OpenCV-BGR-Array um.

    Der rohe Byte-Puffer der Nachricht wird ohne Kopie als uint8 interpretiert und
    in (Höhe, Breite, 3) umgeformt. RealSense kann je nach Konfiguration 'rgb8'
    liefern; da OpenCV intern mit BGR arbeitet, müssen in diesem Fall die Kanäle
    getauscht werden. Bei bereits vorliegendem BGR wird eine echte Kopie
    zurückgegeben, damit späteres Einzeichnen (Marker, Overlay) den zugrunde
    liegenden ROS-Nachrichtenpuffer nicht verändert.

    """
    arr = np.frombuffer(msg.data, dtype=np.uint8)
    enc = msg.encoding.lower()
    if enc == 'rgb8':
        return cv2.cvtColor(
            arr.reshape((msg.height, msg.width, 3)), cv2.COLOR_RGB2BGR
        )
    return arr.reshape((msg.height, msg.width, 3)).copy()

def depth_msg_to_array(msg: Image) -> np.ndarray:
    """Wandelt eine ROS-Tiefenbild-Nachricht in ein 2D-numpy-Array um.

    Zwei bei der RealSense übliche Kodierungen werden unterstützt:

    - '16UC1': Tiefe als vorzeichenlose 16-bit-Ganzzahl in Millimetern.
    - '32FC1': Tiefe als 32-bit-Gleitkomma in Metern.

    Die spätere Rückprojektion normiert beide Fälle auf Meter. Eine unbekannte
    Kodierung wird als Fehler gemeldet, statt stillschweigend falsche Werte zu liefern.

    """

```

```

if msg.encoding == '16UC1':
    return np.frombuffer(msg.data, dtype=np.uint16).reshape(
        (msg.height, msg.width)
    )

if msg.encoding == '32FC1':
    return np.frombuffer(msg.data, dtype=np.float32).reshape(
        (msg.height, msg.width)
    )

raise ValueError(f'Unsupported depth encoding: {msg.encoding}')

```

```

class ClickToGo(Node):

```

```

    """ROS-2-Knoten für den Ablauf Kamera-Klick -> MoveIt-Plan -> Ausführung.

```

```

    Aufgabenteilung des Knotens:

```

- * Abonniert Farbbild, Tiefenbild, CameraInfo und /joint_states.
- * Ein Mausklick im OpenCV-Fenster erzeugt aus Pixel+Tiefe einen 3D-Punkt im robot_base-Frame (Rückprojektion + TF).
- * 'P' löst die numerische IK (senkrechter Greifer) und lässt MoveIt einen kollisionsfreien Weg planen; das Ergebnis wird als DisplayTrajectory in RViz angezeigt (oranger Ghost), aber noch NICHT gefahren.
- * 'E' führt den bestätigten Plan aus, indem das Gelenkziel über die Bridge-Kommando-Socket an den echten Roboter gesendet wird.

```

    Die GUI läuft im Haupt-Thread (OpenCV), rclpy.spin in einem Daemon-Thread;

```

```

    daher schützt ein Lock die von beiden Threads berührten Bildpuffer.

```

```

    """

```

```

    def __init__(self):

```

```

        super().__init__('click_to_go')

```

```

        # Kamera-Intrinsik (3x3), letzte Bild-/Tiefen-/Gelenkzustände. Werden von

```

```

        # den Subscriber-Callbacks (spin-Thread) geschrieben und von der GUI gelesen.

```

```

        self._K: Optional[np.ndarray] = None

```

```

        self._latest_bgr: Optional[np.ndarray] = None

```

```

        self._latest_depth: Optional[np.ndarray] = None

```

```

        self._latest_joints: Optional[JointState] = None

```

```

        self._lock = threading.Lock() # schützt _latest_bgr/_latest_depth zwischen spin- und GUI-Thread

```



```

# Zustand des Klick-/Plan-Ablaufs: angeklicktes Pixel, zugehöriger 3D-Zielpunkt

# im robot_base-Frame und die zuletzt geplante (noch nicht ausgeführte) Trajektorie.

self._click_px: Optional[Tuple[int, int]] = None

self._target_3d_base: Optional[np.ndarray] = None

self._planned_trajectory = None

self._status = 'Warte auf Kamera...'


# Reichweiten-Kreis (senkrechter Griff): grün=sicher, amber=kinematisches Limit.

# in robot_base XY-Ebene, auf Plattformhöhe (reach_z). Quelle: URDF FK.

# reach_radius: Radius (m), innerhalb dessen der myCobot 280 mit senkrechtem Griff

# zuverlässig planen kann. reach_radius_max: äußeres Limit, ab dem die

# Kinematik unsicher wird. reach_z: Höhe der XY-Ebene über der Basis.

self.reach_radius = 0.22

self.reach_radius_max = 0.30

self.reach_z = 0.008


# TF-Puffer + Listener empfangen die statische Kamera->Basis-Transformation

# (eye-to-hand-Kalibrierung) und stellen sie für Punkt-Transformationen bereit.

self._tf_buffer = tf2_ros.Buffer()

self._tf_listener = tf2_ros.TransformListener(self._tf_buffer, self)


# Numerischer IK-Löser (nutzt den MoveIt-Dienst compute_ik). Liefert für eine

# kartesische Zielposition eine Gelenk-Lösung mit senkrecht nach unten zeigendem Greifer.

self._nik = NumericalIK(self)

self.get_logger().info('Numerical IK Solver geladen (MoveIt compute_ik)')


# Publisher: Zielpunkt-Marker (RViz-Kugel) und geplante Trajektorie (oranger Ghost).

self._marker_pub = self.create_publisher(

    Marker, '/click_to_go/target_marker', 10

)

self._traj_pub = self.create_publisher(

    DisplayTrajectory, '/display_planned_path', 10

)


# Action-Clients: MoveGroup für die Wegplanung, sowie die Controller-Actions

# für Arm und Greifer (Trajektorien-Ausführung über ros2_control).

self._move_group_client = ActionClient(

```

```

        self, MoveGroup, '/move_action'
    )

    self._arm_client = ActionClient(
        self, FollowJointTrajectory,
        '/arm_controller/follow_joint_trajectory'
    )

    self._gripper_client = ActionClient(
        self, FollowJointTrajectory,
        '/gripper_controller/follow_joint_trajectory'
    )

    # CameraInfo wird mit RELIABLE-QoS abonniert (latched-ähnlich, depth=1): die
    # Intrinsik ändert sich nicht, es genügt der letzte gültige Wert. Bild- und
    # Tiefen-Topics laufen dagegen mit dem Default-Sensor-Datenstrom (Tiefe 10).
    qos_ci = QoSProfile(
        reliability=ReliabilityPolicy.RELIABLE,
        history=HistoryPolicy.KEEP_LAST, depth=1,
    )

    self.create_subscription(
        CameraInfo, '/camera/color/camera_info', self._on_info, qos_ci
    )

    self.create_subscription(
        Image, '/camera/color/image_raw', self._on_color, 10
    )

    self.create_subscription(
        Image, '/camera/aligned_depth_to_color/image_raw',
        self._on_depth, 10
    )

    self.create_subscription(
        JointState, '/joint_states', self._on_joints, 10
    )

def _on_info(self, msg: CameraInfo):
    # msg.k ist die 3x3-Intrinsik-Matrix (fx, fy, cx, cy) zeilenweise als 9er-Vektor.
    self._K = np.array(msg.k, dtype=np.float64).reshape(3, 3)

def _on_color(self, msg: Image):

```

```

# Callback im spin-Thread: neuestes Farbbild puffern; beim ersten Bild wird der
# Startstatus auf "Bereit" gesetzt. Fehler werden verschluckt, damit ein einzelner
# defekter Frame den Knoten nicht abbricht.

try:
    with self._lock:
        self._latest_bgr = image_msg_to_bgr(msg)
        if self._status == 'Warte auf Kamera...':
            self._status = 'Bereit – klicken!'
except Exception:
    pass

def _on_depth(self, msg: Image):
    try:
        with self._lock:
            self._latest_depth = depth_msg_to_array(msg)
    except Exception:
        pass

def _on_joints(self, msg: JointState):
    # Aktuelle Gelenkstellungen; dienen später als IK-Seed (Startschätzung).
    self._latest_joints = msg

def pixel_to_3d_base(self, u: int, v: int) -> Optional[np.ndarray]:
    """Rückprojektion eines angeklickten Pixels (u, v) in einen 3D-Punkt im robot_base-Frame.

    Ablauf:

    1. Tiefe am Klickpunkt aus dem (auf Farbe ausgerichteten) Tiefenbild lesen.
       Statt eines einzelnen Pixels wird der Median eines 7x7-Fensters genommen,
       um Rauschen und Tiefen-Löcher (Wert 0) robust zu überbrücken.

    2. Millimeter->Meter normieren (16UC1 liefert mm, 32FC1 bereits m).

    3. Mit dem Lochkamera-Modell (fx, fy, cx, cy) das Pixel in einen 3D-Punkt
       im Kamera-optical-Frame zurückprojizieren:  $x=(u-cx)*z/fx$ ,  $y=(v-cy)*z/fy$ .

    4. Diesen Punkt per TF in den robot_base-Frame transformieren.

    Rückgabe None, sobald Intrinsik/Tiefe fehlen, der Klick außerhalb des Bildes
    liegt oder das Fenster keine gültige Tiefe enthält.

    """

    if self._K is None:

```

```

        return None

    with self._lock:

        depth = self._latest_depth

    if depth is None:

        return None

    h, w = depth.shape

    if not (0 <= v < h and 0 <= u < w):

        return None

    # 7x7-Nachbarschaft um den Klick; nur Werte > 0 sind gültige Tiefen.

    region = depth[max(0, v - 3):v + 4, max(0, u - 3):u + 4]

    valid = region[region > 0]

    if len(valid) == 0:

        return None

    # Median unterdrückt Ausreißer; >100 => Wert ist in Millimetern (16UC1) -> in Meter.

    z_raw = float(np.median(valid))

    z = z_raw / 1000.0 if z_raw > 100 else z_raw

    # Inverse Lochkamera-Projektion in den Kamera-optical-Frame.

    fx, fy = self._K[0, 0], self._K[1, 1]

    cx, cy = self._K[0, 2], self._K[1, 2]

    x_cam = (u - cx) * z / fx

    y_cam = (v - cy) * z / fy

    pt_cam = PointStamped()

    pt_cam.header.frame_id = CAMERA_FRAME

    pt_cam.header.stamp = self.get_clock().now().to_msg()

    pt_cam.point.x = x_cam

    pt_cam.point.y = y_cam

    pt_cam.point.z = z

    # TF: Kamera-optical -> robot_base. Timeout gibt dem Buffer Zeit, die (statische)

    # Kalibrier-Transformation zu erhalten, falls sie noch nicht vorliegt.

    try:

        pt_base = self._tf_buffer.transform(

```

```

        pt_cam, ROBOT_BASE_FRAME,

        timeout=rclpy.time.Duration(seconds=2.0)

    )

except Exception as e:

    self.get_logger().warn(f'TF-Transformation fehlgeschlagen: {e}')

    return None


# Untergrenze für z: verhindert Ziele unter/auf der Tischplatte (Tiefenrauschen);
# der Greifer soll nicht in die Plattform hineinfahren.

z = pt_base.point.z

if z < 0.005:

    z = 0.005

# Optionaler Schweben-Aufschlag (CLICK_HOVER_M, m): Ziel N cm ÜBER dem angeklickten
# Punkt – für Kalibrier-Messungen (Feinausrichtung danach per cartesian_jog),
# damit der Greifer bei XY-Abweichung nicht seitlich gegen die Schraube fährt.
# Default 0.0 = bisheriges Verhalten unverändert.

z += CLICK_HOVER_M

return np.array([

    pt_base.point.x, pt_base.point.y, z

], dtype=np.float64)


# ---- Reichweiten-Kreis (base XY-Ebene -> Pixel-Projektion) ----

# Diese drei Methoden projizieren den (im robot_base definierten) Arbeitsraum-Kreis
# zurück ins Kamerabild, damit der Benutzer VOR dem Klick sieht, welche Punkte der
# Roboter mit senkrechtem Griff sicher (grün) bzw. am kinematischen Limit (amber)
# erreicht. Es ist die inverse Richtung zu pixel_to_3d_base: base-Punkt -> Pixel.
def _base_to_cam_RT(self):

    """robot_base -> camera_optical (R 3x3, t 3). EIN TF-Lookup. Kann None sein.

    Holt die Transformation als Quaternion+Translation und baut daraus die
    Rotationsmatrix R und den Translationsvektor t, sodass ein base-Punkt p_base
    via p_cam = R @ p_base + t in den Kamera-optical-Frame überführt wird.

    Ein einziger Lookup wird pro Frame gemacht und an alle Projektionen weitergereicht.

    """

    if self._K is None:

        return None

    try:

```

```

        tf = self._tf_buffer.lookup_transform(

            CAMERA_FRAME, ROBOT_BASE_FRAME, rclpy.time.Time())

except Exception:

    return None

# Quaternion (x,y,z,w) -> Rotationsmatrix (Standard-Formel), t = Translation.

q = tf.transform.rotation

t = tf.transform.translation

x, y, z, w = q.x, q.y, q.z, q.w

R = np.array([

    [1 - 2 * (y * y + z * z), 2 * (x * y - z * w), 2 * (x * z + y * w)],

    [2 * (x * y + z * w), 1 - 2 * (x * x + z * z), 2 * (y * z - x * w)],

    [2 * (x * z - y * w), 2 * (y * z + x * w), 1 - 2 * (x * x + y * y)],

])

return R, np.array([t.x, t.y, t.z])

def _proj(self, pb, R, tv):

    """base Punkt (3,) -> Pixel (u,v). Hinter der Kamera -> None.

    Transformiert den Punkt in den Kamera-Frame und wendet dann das Lochkamera-Modell
    an ( $u = f_x * x / z + c_x$ ,  $v = f_y * y / z + c_y$ ). Punkte mit  $z \leq 0$  liegen hinter/auf der Bildebene und
    können nicht projiziert werden -> None (verhindert Division durch  $\sim 0$  und Spiegelartefakte).

    """

    pc = R @ pb + tv

    if pc[2] <= 1e-3:

        return None

    fx, fy = self._K[0, 0], self._K[1, 1]

    cx, cy = self._K[0, 2], self._K[1, 2]

    return (int(round(fx * pc[0] / pc[2] + cx)),

            int(round(fy * pc[1] / pc[2] + cy)))

def _draw_reach(self, frame):

    """Reichweiten-Kreise (grün sicher + amber Limit) + Roboter-Zentrum zeichnen.

    Für jeden Radius werden 72 Punkte auf einem Kreis in der robot_base-XY-Ebene
    (auf Plattformhöhe reach_z) erzeugt, einzeln ins Bild projiziert und als
    Polygonzug gezeichnet. Der Kreis wird nur dann geschlossen gezeichnet, wenn ALLE
    Punkte sichtbar sind; ragt er teilweise aus dem Bild/hinter die Kamera, bleibt er

```

offen. So bleibt die Anzeige auch bei starker Perspektive korrekt.

"""

```
rt = self._base_to_cam_RT()
```

```
if rt is None:
```

```
    return
```

```
R, tv = rt
```

```
AMBER, GREEN = (0, 200, 255), (0, 200, 0)
```

```
# Zuerst den äußeren (amber) Limit-Kreis, dann den inneren (grünen) Sicherheitskreis.
```

```
for radius, col, th in ((self.reach_radius_max, AMBER, 1),
```

```
                        (self.reach_radius, GREEN, 2)):
```

```
    if radius <= 0:
```

```
        continue
```

```
    pts = [self._proj(np.array([radius * np.cos(a), radius * np.sin(a), self.reach_z]),
```

```
                      R, tv)
```

```
            for a in (2 * np.pi * i / 72 for i in range(72))]
```

```
    valid = [p for p in pts if p is not None]
```

```
    if len(valid) >= 2:
```

```
        closed = all(p is not None for p in pts)
```

```
        cv2.polylines(frame, [np.array(valid, np.int32)], closed,
```

```
                        col, th, cv2.LINE_AA)
```

```
# Mittelpunkt (Roboter-Basis) als Kreuz + Legende mit den Radien in mm markieren.
```

```
c = self._proj(np.array([0.0, 0.0, self.reach_z]), R, tv)
```

```
if c is not None:
```

```
    cv2.drawMarker(frame, c, AMBER, cv2.MARKER_TILTED_CROSS, 16, 2)
```

```
    cv2.putText(frame, f'ROBOTER  g<{self.reach_radius * 1000:.0f} lim<{self.reach_radius_max *  
1000:.0f}mm',
```

```
                (c[0] + 8, c[1] + 4), cv2.FONT_HERSHEY_SIMPLEX,
```

```
                0.4, AMBER, 1, cv2.LINE_AA)
```

```
def on_click(self, u: int, v: int):
```

```
    """Linksklick-Handler: Pixel in 3D-Zielpunkt umrechnen und als gelben Marker anzeigen.
```

Schlägt die Rückprojektion fehl (keine gültige Tiefe), wird der Klick nur als

Kreuz gemerkt und eine Fehlermeldung gesetzt; ein vorheriges Ziel/Plan wird verworfen.

Bei Erfolg wird der Zielpunkt (robot_base) gespeichert und der Status auf 'P: planen'
gesetzt.

"""

```

p_base = self.pixel_to_3d_base(u, v)

if p_base is None:

    self._click_px = (u, v)

    self._target_3d_base = None

    self._planned_trajectory = None

    self._status = f'Keine Tiefe ({u},{v})'

    self.get_logger().warn(self._status)

    return

self._click_px = (u, v)

self._target_3d_base = p_base

self._planned_trajectory = None

self._status = (

    f'Ziel: [{p_base[0]:.3f}, {p_base[1]:.3f}, {p_base[2]:.3f}] '

    f'— P: planen'

)

self.get_logger().info(self._status)

self._publish_marker(p_base, r=1.0, g=1.0, b=0.0)

def _publish_marker(self, p_base: np.ndarray, r=1.0, g=1.0, b=0.0):

    # Veröffentlicht eine RViz-Kugel am Zielpunkt. Farbcodierung des Ablaufs:

    # gelb (r,g)=noch nicht geplant, grün=Plan bereit, rot=IK/Plan fehlgeschlagen.

    m = Marker()

    m.header.frame_id = ROBOT_BASE_FRAME

    m.header.stamp = self.get_clock().now().to_msg()

    m.ns = 'click_to_go'

    m.id = 0

    m.type = Marker.SPHERE

    m.action = Marker.ADD

    m.pose.position.x = float(p_base[0])

    m.pose.position.y = float(p_base[1])

    m.pose.position.z = float(p_base[2])

    m.pose.orientation.w = 1.0

    m.scale.x = 0.03

    m.scale.y = 0.03

    m.scale.z = 0.03

    m.color.r = r

```



```

m.color.g = g

m.color.b = b

m.color.a = 0.9

m.lifetime.sec = 60

self._marker_pub.publish(m)

```

```

def _delete_marker(self):

    # Entfernt den Zielpunkt-Marker in RViz (bei Rechtsklick/Löschen).

    m = Marker()

    m.header.frame_id = ROBOT_BASE_FRAME

    m.header.stamp = self.get_clock().now().to_msg()

    m.ns = 'click_to_go'

    m.id = 0

    m.action = Marker.DELETE

    self._marker_pub.publish(m)

```

```

def plan(self):

    """'P'-Taste: Planung anstoßen (im Hintergrund-Thread, damit die GUI nicht blockiert).

    Es muss ein Ziel gewählt sein und es darf nicht bereits eine Planung laufen
    (Wiedereintritts-Schutz über _planning).

    """

    if self._target_3d_base is None:

        self._status = 'Erst einen Punkt waehlen!'

        return

    if hasattr(self, '_planning') and self._planning:

        self._status = 'Wird bereits geplant...'

        return

    self._planning = True

    threading.Thread(target=self._plan_thread, daemon=True).start()

```

```

def _plan_thread(self):

    """Hintergrund-Thread: IK lösen und daraus einen MoveIt-Plan als Joint-Goal anfordern.

    Schritt 1 (hier): kartesisches Ziel -> Gelenkwinkel via numerischer IK
    (senkrechter Greifer). Schritt 2: die IK-Lösung als JointConstraints an MoveGroup
    senden -> MoveIt plant den kollisionsfreien Weg. Die Antwort kommt asynchron über

```

```

add_done_callback (_plan_response_cb / _plan_result_cb).

"""

tx = float(self._target_3d_base[0])
ty = float(self._target_3d_base[1])
tz = float(self._target_3d_base[2])

if tz < 0.005:

    tz = 0.005

self._status = f'IK + Plan [{tx:.3f},{ty:.3f},{tz:.3f}]...'
self.get_logger().info(self._status)

# Aktuelle Gelenk-Positionen als Seed verwenden: die IK startet nahe der Ist-Lage,
# was Konvergenz beschleunigt und unnötig große Gelenk-Sprünge vermeidet.
# Namen->Position-Abbildung, da /joint_states nicht zwingend in ARM_JOINTS-Reihenfolge kommt.
current = None

if self._latest_joints is not None:

    name_to_pos = dict(zip(

        self._latest_joints.name, self._latest_joints.position

    ))

    current = [name_to_pos.get(j, 0.0) for j in ARM_JOINTS]

# Multi-Seed-IK: probiert mehrere Startschätzungen, um lokale Minima zu umgehen.
# gripper_down=True erzwingt eine senkrecht nach unten zeigende Werkzeug-Orientierung.
target_radians = self._nik.solve_multi_seed(

    [tx, ty, tz], gripper_down=True, current_joints=current, n_seeds=5

)

if target_radians is None:

    self._status = f'IK FEHLGESCHLAGEN – Punkt nicht erreichbar'

    self._planning = False

    self._publish_marker(self._target_3d_base, r=1.0, g=0.0, b=0.0)

    return

self.get_logger().info(

    f'IK OK: joints={[round(r,3) for r in target_radians]}'

)

```

```

# 2) An MoveGroup als Joint-Goal übergeben – Wegplanung + Kollisions-Check

if not self._move_group_client.wait_for_server(timeout_sec=3.0):

    self._status = 'MoveGroup nicht gefunden!'

    return


# MoveGroup-Ziel zusammenstellen. num_planning_attempts+allowed_planning_time geben
# dem Planer mehrere Versuche/Zeit; velocity/acceleration_scaling begrenzen die
# Ausführungsgeschwindigkeit (0.3/0.2 = sicher/langsam für den echten Roboter).
goal = MoveGroup.Goal()

req = goal.request

req.group_name = PLANNING_GROUP

req.num_planning_attempts = 10

req.allowed_planning_time = 5.0

req.max_velocity_scaling_factor = 0.3

req.max_acceleration_scaling_factor = 0.2


# Arbeitsraum-Grenzen (+/-1 m um die Basis): Suchraum-Begrenzung für den Planer.
req.workspace_parameters.header.frame_id = ROBOT_BASE_FRAME

req.workspace_parameters.min_corner.x = -1.0

req.workspace_parameters.min_corner.y = -1.0

req.workspace_parameters.min_corner.z = -1.0

req.workspace_parameters.max_corner.x = 1.0

req.workspace_parameters.max_corner.y = 1.0

req.workspace_parameters.max_corner.z = 1.0


# Das IK-Ergebnis wird NICHT als kartesisches Pose-Ziel, sondern als Gelenk-Ziel
# (JointConstraint pro Achse) übergeben. So plant MoveIt exakt zu der von uns
# gewählten IK-Lösung (senkrechter Griff) und wählt nicht selbst eine andere.
constraints = Constraints()

for jname, jval in zip(ARM_JOINTS, target_radians):

    jc = JointConstraint()

    jc.joint_name = jname

    jc.position = jval

    jc.tolerance_above = 0.05

    jc.tolerance_below = 0.05

    jc.weight = 1.0

    constraints.joint_constraints.append(jc)

```

```

req.goal_constraints.append(constraints)

# plan_only=True: nur planen, NICHT ausführen (Vorschau in RViz, Bestätigung über 'E').
goal.planning_options.plan_only = True
goal.planning_options.look_around = False
goal.planning_options.replan = False

# Asynchron senden; die Ergebnis-Callbacks laufen im spin-Thread.
self._status = 'MoveIt plant Weg...'

future = self._move_group_client.send_goal_async(goal)
future.add_done_callback(self._plan_response_cb)

def _plan_response_cb(self, future):
    # Erste Antwort von MoveGroup: wurde das Planungsziel überhaupt angenommen?
    # Bei Ablehnung Ziel-Marker rot färben; sonst auf das eigentliche Ergebnis warten.
    goal_handle = future.result()

    if not goal_handle.accepted:
        self._status = 'Plan ABGELEHNT'

        if self._target_3d_base is not None:
            self._publish_marker(self._target_3d_base, r=1.0, g=0.0, b=0.0)

        return

    result_future = goal_handle.get_result_async()
    result_future.add_done_callback(self._plan_result_cb)

def _plan_result_cb(self, future):
    # Planungsergebnis auswerten. error_code.val==1 (SUCCESS) => Trajektorie liegt vor.
    result = future.result().result
    error_code = result.error_code.val

    if error_code != 1:
        self._planning = False
        self._status = f'Plan FEHLGESCHLAGEN (code={error_code})'
        self.get_logger().error(self._status)
        self._planned_trajectory = None

        if self._target_3d_base is not None:
            self._publish_marker(self._target_3d_base, r=1.0, g=0.0, b=0.0)

    return

```

```

# Letzter Trajektorien-Punkt = Ziel-Gelenkstellung; die wird bei 'E' angefahren.

# Vollständige Trajektorie für die RViz-Vorschau merken.

traj = result.planned_trajectory.joint_trajectory

self._planned_joints = list(traj.points[-1].positions)

self._planned_trajectory = result.planned_trajectory


# DisplayTrajectory veröffentlichen -> RViz zeigt den geplanten Weg als orangen Ghost.

display = DisplayTrajectory()

display.trajectory_start = result.trajectory_start

display.trajectory.append(result.planned_trajectory)

self._traj_pub.publish(display)


n_points = len(traj.points)

self._planning = False

self._status = (
    f'Plan BEREIT ({n_points} Punkte) – '
    f'in RViz sehen, E: ausfuehren'
)

self.get_logger().info(self._status)

if self._target_3d_base is not None:
    self._publish_marker(self._target_3d_base, r=0.0, g=1.0, b=0.0)


def execute(self):
    """'E'-Taste: den bestätigten Plan ausführen.

    Statt die volle MoveIt-Trajektorie über den Controller zu spielen, wird hier das
    Ziel-Gelenktupel direkt über die Bridge-Socket an den echten Roboter gesendet
    (flüssige send_radians-Bewegung, siehe _send_bridge_cmd). Der Plan wird verworfen,
    damit ein versehentliches zweites 'E' nicht erneut fährt.
    """

    if self._planned_trajectory is None or not hasattr(self, '_planned_joints'):
        self._status = 'Erst mit P planen!'
        return

    target = self._planned_joints

    self._status = 'Roboter faehrt zum Ziel...'

```

```

self.get_logger().info(
    f'Execute: Ziel={[round(p,3) for p in target]}'
)

self._planned_trajectory = None

self._send_bridge_cmd(target, speed=20)

def _send_bridge_cmd(self, radians: list, speed: int = 20):
    """Ein send_radians an Bridge-Kommando-Socket senden – flüssige Bewegung.

    Der myCobot-Bridge-Prozess hält den seriellen Port offen und lauscht auf einem
    Unix-Domain-Socket. Wir schicken eine Textzeile im Wire-Protokoll
    'send_radians r1 r2 ... r6 speed'. Dieser Umweg (statt pymycobot direkt) vermeidet
    Port-Konkurrenz: nur die Bridge spricht mit der Hardware. Das Zeilenformat und der
    Socket-Pfad sind ein Laufzeit-Vertrag mit der Bridge und dürfen nicht geändert werden.
    """

    import socket as sock

    CMD_PATH = '/tmp/mycobot_bridge.sock.cmd'

    cmd = 'send_radians ' + ' '.join(f'{r:.6f}' for r in radians) + f' {speed}\n'

    try:
        s = sock.socket(sock.AF_UNIX, sock.SOCK_STREAM)

        s.connect(CMD_PATH)

        s.sendall(cmd.encode())

        s.close()

        self._status = 'Faehrt zum Ziel...'

        self.get_logger().info('Bridge cmd sent OK')

    except Exception as e:
        self._status = f'Bridge-Fehler: {e}'

        self.get_logger().error(self._status)

def go_named_pose(self, pose_name: str):
    """Direkt über Bridge zu benannter Pose fahren – flüssige Bewegung.

    Vordefinierte Gelenkkonfigurationen ohne Planung: 'home' (Null-Stellung) und
    'ready' (Arbeitsvorbereitungspose). Wird über dieselbe Bridge-Socket wie execute()
    gesendet.
    """

    NAMED_POSES = {

```

```

        'home': [0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
        'ready': [0.0, -0.5, -1.0, 0.0, 0.0, 0.0],
    }

    if pose_name not in NAMED_POSES:
        self._status = f'Unbekannte Pose: {pose_name}'
        return

    self._status = f'Faehrt zu Position {pose_name}...'
    self._planned_trajectory = None
    self._send_bridge_cmd(NAMED_POSES[pose_name], speed=15)

def gripper_control(self, state: str):
    """Greifer öffnen/schließen über die gripper_controller-Action.

    Anders als die Armbewegung läuft der Greifer hier über ros2_control (Action
    FollowJointTrajectory) statt über die Bridge. Die Positionen (rad) sind
    firmware-/URDF-kalibrierte Anschlagwerte für 'open'/'closed'.
    """
    GRIPPER_POSES = {'open': 0.15, 'closed': -0.74}

    if state not in GRIPPER_POSES:
        return

    self._status = f'Greifer {state}...'

    if not self._gripper_client.wait_for_server(timeout_sec=3.0):
        self._status = 'Greifer Action-Server nicht gefunden!'
        return

    goal = FollowJointTrajectory.Goal()
    goal.trajectory.joint_names = ['gripper_controller']
    pt = JointTrajectoryPoint()
    pt.positions = [GRIPPER_POSES[state]]
    pt.time_from_start = Duration(sec=2, nanosec=0)
    goal.trajectory.points = [pt]

    future = self._gripper_client.send_goal_async(goal)
    future.add_done_callback(lambda f: self._gripper_cb(f, state))

```

```

def _gripper_cb(self, future, state):

    goal_handle = future.result()

    if not goal_handle.accepted:

        self._status = f'Greifer {state} ABGELEHNT'

        return

    result_future = goal_handle.get_result_async()

    result_future.add_done_callback(

        lambda f: setattr(self, '_status', f'Greifer {state} OK')

    )

def get_display_frame(self) -> Optional[np.ndarray]:

    """Baut das Anzeige-Bild für das OpenCV-Fenster (wird im Haupt-Thread aufgerufen).

    Zeichnet in Reihenfolge: Reichweiten-Kreise, Klick-Marker (grün=geplant,
    gelb=nur gewählt) mit 3D-Koordinaten und Reichweiten-Tag (ERREICHBAR/GRENZE/
    UNERREICHBAR), Status-Zeile oben und Tastenlegende unten. Der Bildpuffer wird unter
    Lock kopiert, damit der spin-Thread ihn nicht während des Zeichnens überschreibt.
    """

    with self._lock:

        if self._latest_bgr is None:

            return None

        frame = self._latest_bgr.copy()

    h, w = frame.shape[:2]

    # Reichweiten-Kreise (Bereich, den der Roboter mit senkrechtem Griff erreicht)

    try:

        self._draw_reach(frame)

    except Exception:

        pass

    if self._click_px is not None:

        u, v = self._click_px

        color = (0, 255, 0) if self._planned_trajectory else (0, 255, 255)

        cv2.circle(frame, (u, v), 10, color, 2)

        cv2.drawMarker(frame, (u, v), color,

                       cv2.MARKER_CROSS, 20, 2)

```



```

if self._target_3d_base is not None:

    # r = horizontaler Abstand des Ziels von der Basisachse -> Reichweiten-Kategorie.

    p = self._target_3d_base

    r = float(np.hypot(p[0], p[1]))

    if r <= self.reach_radius:

        rcol, rtag = (0, 200, 0), 'ERREICHBAR'

    elif r <= self.reach_radius_max:

        rcol, rtag = (0, 200, 255), 'GRENZE'

    else:

        rcol, rtag = (0, 0, 255), 'UNERREICHBAR'

    text = f'[{p[0]:.3f}, {p[1]:.3f}, {p[2]:.3f}]  r={r*1000:.0f}mm {rtag}'

    cv2.putText(frame, text, (u + 15, v - 15),

                cv2.FONT_HERSHEY_SIMPLEX, 0.5,

                rcol, 1, cv2.LINE_AA)

if self._status:

    cv2.rectangle(frame, (0, 0), (w, 30), (40, 40, 40), -1)

    cv2.putText(frame, self._status, (8, 22),

                cv2.FONT_HERSHEY_SIMPLEX, 0.45,

                (255, 255, 255), 1, cv2.LINE_AA)

cv2.rectangle(frame, (0, h - 24), (w, h), (40, 40, 40), -1)

cv2.putText(frame, 'Klick:wählen P:Plan E:exe H:Home R:ready O:auf C:zu Q:beenden',

            (4, h - 6), cv2.FONT_HERSHEY_SIMPLEX, 0.3,

            (180, 180, 180), 1, cv2.LINE_AA)

return frame

def mouse_cb(event, x, y, flags, param):

    # OpenCV-Maus-Callback: Links = Ziel wählen, Rechts = Ziel/Plan/Marker löschen.

    node = param

    if event == cv2.EVENT_LBUTTONDOWN:

        node.on_click(x, y)

    elif event == cv2.EVENT_RBUTTONDOWN:

        node._click_px = None

```

```
node._target_3d_base = None

node._planned_trajectory = None

node._status = 'Bereit – klicken!'

node._delete_marker()
```

```
def main():
```

```
    # ROS initialisieren, Knoten anlegen und rclpy.spin in einen Daemon-Thread auslagern,
    # damit der Haupt-Thread die (blockierende) OpenCV-GUI-Schleife fahren kann.
```

```
    rclpy.init()
```

```
    node = ClickToGo()
```

```
    spin_thread = threading.Thread(
        target=lambda: rclpy.spin(node), daemon=True
    )
```

```
    spin_thread.start()
```

```
    cv2.namedWindow('Click to Go', cv2.WINDOW_NORMAL)
```

```
    cv2.resizeWindow('Click to Go', 640, 380)
```

```
    cv2.setMouseCallback('Click to Go', mouse_cb, node)
```

```
    print('\n' + '=' * 55)
```

```
    print('  CLICK TO GO – Plan & Execute')
```

```
    print('=' * 55)
```

```
    print('  Links klicken : Ziel waehlen (gelber Marker)')
```

```
    print('  Rechts klicken: loeschen')
```

```
    print('  P           : Mit MoveIt planen (in RViz sehen)')
```

```
    print('  E           : Plan ausfuehren (Roboter faehrt)')
```

```
    print('  H           : Home (alle Gelenke 0)')
```

```
    print('  R           : ready (Arbeitsposition)')
```

```
    print('  O           : Greifer oeffnen')
```

```
    print('  C           : Greifer schliessen')
```

```
    print('  Q           : beenden')
```

```
    print('=' * 55 + '\n')
```

```
    # GUI-Hauptschleife: Bild anzeigen und Tastendrücke auf Knoten-Aktionen abbilden.
```

```
    # cv2.waitKey(50) taktet die Schleife auf ~20 Hz und liefert den gedrückten Tastencode.
```

```

try:
    while rclpy.ok():
        frame = node.get_display_frame()

        if frame is not None:
            cv2.imshow('Click to Go', frame)

        key = cv2.waitKey(50) & 0xFF

        if key != 255:
            node.get_logger().info(f'Key pressed: {key} = {chr(key) if 32 <= key < 127 else "?"}')

            if key == ord('p') or key == ord('P'):
                node.plan()

            elif key == ord('e') or key == ord('E'):
                node.execute()

            elif key == ord('h') or key == ord('H'):
                node.go_named_pose('home')

            elif key == ord('r') or key == ord('R'):
                node.go_named_pose('ready')

            elif key == ord('o') or key == ord('O'):
                node.gripper_control('open')

            elif key == ord('c') or key == ord('C'):
                node.gripper_control('closed')

            elif key == ord('q') or key == ord('Q'):
                break

except KeyboardInterrupt:
    pass

finally:
    # Sauberes Herunterfahren: Fenster schließen, Knoten zerstören, ROS beenden.

    cv2.destroyAllWindows()

    node.destroy_node()

    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

src/mycobot_calibration/scripts/generate_charuco_board.py

```
#!/usr/bin/env python3

# ChArUco 5x7 board generator – Phase 5 hand-eye calibration.

# Prints at A4, 100% scale. Square size MUST be verified with a ruler after printing.


import sys

from pathlib import Path


import cv2

import cv2.aruco as aruco

import numpy as np

from PIL import Image, ImageDraw, ImageFont


# --- Board-Parameter (nach dem ersten Druck NIEMALS ändern) ---

SQUARES_X = 5          # Anzahl Spalten

SQUARES_Y = 7          # Anzahl Zeilen

SQUARE_LENGTH_MM = 30.0 # Kantenlänge eines Quadrats

MARKER_LENGTH_MM = 22.0 # ArUco Marker-Kante (im Quadrat, ~0.73x square)

DICT_ID = aruco.DICT_5X5_100

DICT_NAME = "DICT_5X5_100"


# --- A4 Druck ---

DPI = 300

A4_W_MM = 210.0

A4_H_MM = 297.0


def mm2px(mm: float) -> int:

    return int(round(mm * DPI / 25.4))


def main() -> int:

    # Script lives in <pkg>/scripts/ – write outputs to <pkg>/boards and <pkg>/config.

    pkg_root = Path(__file__).resolve().parent.parent

    boards_dir = pkg_root / "boards"

    cfg_dir = pkg_root / "config"
```

```

boards_dir.mkdir(parents=True, exist_ok=True)

cfg_dir.mkdir(parents=True, exist_ok=True)


dictionary = aruco.getPredefinedDictionary(DICT_ID)

board = aruco.CharucoBoard(
    (SQUARES_X, SQUARES_Y),
    SQUARE_LENGTH_MM / 1000.0,
    MARKER_LENGTH_MM / 1000.0,
    dictionary,
)

board_w_px = mm2px(SQUARES_X * SQUARE_LENGTH_MM)
board_h_px = mm2px(SQUARES_Y * SQUARE_LENGTH_MM)

board_img = board.generateImage((board_w_px, board_h_px), marginSize=0, borderBits=1)


a4_w_px = mm2px(A4_W_MM)
a4_h_px = mm2px(A4_H_MM)

canvas = Image.new("RGB", (a4_w_px, a4_h_px), "white")


# Board auf der Seite horizontal zentrieren, oben 20 mm Abstand lassen
off_x = (a4_w_px - board_w_px) // 2
off_y = mm2px(20)

canvas.paste(Image.fromarray(board_img).convert("RGB"), (off_x, off_y))


draw = ImageDraw.Draw(canvas)

try:
    font_md = ImageFont.truetype(
        "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf", 32
    )

    font_sm = ImageFont.truetype(
        "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf", 24
    )
except OSError:
    font_md = ImageFont.load_default()
    font_sm = font_md


text_y = off_y + board_h_px + mm2px(12)

```

```

lines = [

    f"ChArUco {SQUARES_X}x{SQUARES_Y}",

    f"square = {SQUARE_LENGTH_MM:.1f} mm    marker = {MARKER_LENGTH_MM:.1f} mm",

    f"dict    = {DICT_NAME}",

    "Print at 100 % scale (NO fit-to-page).",

]

for ln in lines:

    draw.text((mm2px(15), text_y), ln, fill="black", font=font_sm)

    text_y += mm2px(7)


# 50 mm Referenzlineal – nach dem Druck mit einem Lineal prüfen

ruler_y = text_y + mm2px(6)

ruler_x0 = mm2px(15)

ruler_x1 = ruler_x0 + mm2px(50)

draw.line([(ruler_x0, ruler_y), (ruler_x1, ruler_y)], fill="black", width=4)

for i in range(6):

    x = ruler_x0 + mm2px(10 * i)

    h = mm2px(5) if i % 5 == 0 else mm2px(3)

    draw.line([(x, ruler_y), (x, ruler_y + h)], fill="black", width=3)

draw.text(

    (ruler_x0, ruler_y + mm2px(7)),

    "Reference 0-50 mm (verify after print)",

    fill="black",

    font=font_sm,

)


png_path = boards_dir / f"charuco_{SQUARES_X}x{SQUARES_Y}_a4.png"
pdf_path = boards_dir / f"charuco_{SQUARES_X}x{SQUARES_Y}_a4.pdf"

canvas.save(png_path, dpi=(DPI, DPI))

canvas.save(pdf_path, "PDF", resolution=float(DPI))


yaml_path = cfg_dir / "charuco_params.yaml"

yaml_path.write_text(

    "# ChArUco board parameters – paired with the printed PDF.\n"

    "# Generated by generate_charuco_board.py – regenerate if any value changes.\n"

    f"squares_x: {SQUARES_X}\n"

    f"squares_y: {SQUARES_Y}\n"

```

```
f"square_length_m: {SQUARE_LENGTH_MM / 1000.0}\n"

f"marker_length_m: {MARKER_LENGTH_MM / 1000.0}\n"

f"aruco_dict: {DICT_NAME}\n"

f"aruco_dict_id: {int(DICT_ID)}\n"

)

print(f"PNG : {png_path}")
print(f"PDF : {pdf_path}")
print(f"YAML: {yaml_path}")

print()

print("Mit 100% Skalierung drucken (fit-to-page AUS).")

print("Nach dem Druck mit dem Lineal vergleichen – ein Quadrat = 30.0 mm muss sein.")

return 0


if __name__ == "__main__":

    sys.exit(main())
```

src/mycobot_calibration/scripts/generate_charuco_gripper.py

```
#!/usr/bin/env python3

# ChArUco 6x3 Greifer-Marker-Generator – für die Neukalibrierung (eye-to-hand).

# Kleines Board zum Aufkleben auf den Greifer: Außenmaß 96x48 mm (passt in ~10x5 cm).

# Mit 100% Skalierung auf A4 drucken; NACH dem Druck ein Quadrat mit dem Lineal prüfen (muss 16.0 mm sein).

#

# Kompatibel mit charuco_detector.py: gleiches DICT_4X4_50 Wörterbuch.

# Aktive config/charuco_params.yaml wird NICHT überschrieben – neue Parameter werden in eine separate Datei
geschrieben.

# Nach dem Drucken und Anbringen am Greifer den Detector mit dieser neuen Datei starten.


import sys

from pathlib import Path

import cv2.aruco as aruco
import numpy as np

from PIL import Image, ImageDraw, ImageFont

# --- Board-Parameter (nach dem ersten Druck NIEMALS ändern) ---

SQUARES_X = 6          # Anzahl Spalten (Breitenrichtung) -> 6 * 16 = 96 mm
SQUARES_Y = 3          # Anzahl Zeilen (Höhenrichtung) -> 3 * 16 = 48 mm
SQUARE_LENGTH_MM = 16.0 # Kantenlänge eines Quadrats
MARKER_LENGTH_MM = 12.0 # ArUco Marker-Kante (im Quadrat, 0.75x square)
DICT_ID = aruco.DICT_4X4_50
DICT_NAME = "DICT_4X4_50"

# --- A4 Druck ---

DPI = 300

A4_W_MM = 210.0
A4_H_MM = 297.0

def mm2px(mm: float) -> int:

    return int(round(mm * DPI / 25.4))

def main() -> int:
```



```

pkg_root = Path(__file__).resolve().parent.parent

boards_dir = pkg_root / "boards"

cfg_dir = pkg_root / "config"

boards_dir.mkdir(parents=True, exist_ok=True)

cfg_dir.mkdir(parents=True, exist_ok=True)


dictionary = aruco.getPredefinedDictionary(DICT_ID)

board = aruco.CharucoBoard(
    (SQUARES_X, SQUARES_Y),
    SQUARE_LENGTH_MM / 1000.0,
    MARKER_LENGTH_MM / 1000.0,
    dictionary,
)


board_w_px = mm2px(SQUARES_X * SQUARE_LENGTH_MM)
board_h_px = mm2px(SQUARES_Y * SQUARE_LENGTH_MM)
board_img = board.generateImage((board_w_px, board_h_px), marginSize=0, borderBits=1)


a4_w_px = mm2px(A4_W_MM)
a4_h_px = mm2px(A4_H_MM)
canvas = Image.new("RGB", (a4_w_px, a4_h_px), "white")


off_x = (a4_w_px - board_w_px) // 2
off_y = mm2px(25)
canvas.paste(Image.fromarray(board_img).convert("RGB"), (off_x, off_y))


draw = ImageDraw.Draw(canvas)

try:
    font_sm = ImageFont.truetype(
        "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf", 24
    )
except OSError:
    font_sm = ImageFont.load_default()

text_y = off_y + board_h_px + mm2px(12)

lines = [
    f"ChArUco {SQUARES_X}x{SQUARES_Y} (gripper marker)",

```

```

f"square = {SQUARE_LENGTH_MM:.1f} mm    marker = {MARKER_LENGTH_MM:.1f} mm",

f"dict    = {DICT_NAME}",

f"board    = {SQUARES_X*SQUARE_LENGTH_MM:.0f} x {SQUARES_Y*SQUARE_LENGTH_MM:.0f} mm",

"Print at 100 % scale (NO fit-to-page).",

]

for ln in lines:

    draw.text((mm2px(15), text_y), ln, fill="black", font=font_sm)

    text_y += mm2px(7)

# 50 mm Referenzlineal – nach dem Druck mit einem Lineal prüfen

ruler_y = text_y + mm2px(6)

ruler_x0 = mm2px(15)

ruler_x1 = ruler_x0 + mm2px(50)

draw.line([(ruler_x0, ruler_y), (ruler_x1, ruler_y)], fill="black", width=4)

for i in range(6):

    x = ruler_x0 + mm2px(10 * i)

    h = mm2px(5) if i % 5 == 0 else mm2px(3)

    draw.line([(x, ruler_y), (x, ruler_y + h)], fill="black", width=3)

draw.text(

    (ruler_x0, ruler_y + mm2px(7)),

    "Reference 0-50 mm (verify after print)",

    fill="black",

    font=font_sm,

)

png_path = boards_dir / f"charuco_{SQUARES_X}x{SQUARES_Y}_gripper.png"

pdf_path = boards_dir / f"charuco_{SQUARES_X}x{SQUARES_Y}_gripper.pdf"

canvas.save(png_path, dpi=(DPI, DPI))

canvas.save(pdf_path, "PDF", resolution=float(DPI))

# Aktive charuco_params.yaml NICHT überschreiben – in separate Datei schreiben.

yaml_path = cfg_dir / "charuco_params_gripper.yaml"

yaml_path.write_text(

    "# ChArUco 6x3 gripper board parameters – passt zum gedruckten PDF.\n"

    "# Erzeugt von generate_charuco_gripper.py. Bei jeder Wertaenderung neu erzeugen.\n"

    "# Bei der Neukalibrierung den Detector mit DIESER Datei starten (statt dem alten 5x5).\n"

    "/*:\n"

```

```

    "  ros__parameters:\n"

    f"    squares_x: {SQUARES_X}\n"

    f"    squares_y: {SQUARES_Y}\n"

    f"    square_length_m: {SQUARE_LENGTH_MM / 1000.0}\n"

    f"    marker_length_m: {MARKER_LENGTH_MM / 1000.0}\n"

    f"    aruco_dict: {DICT_NAME}\n"

)

print(f"PNG : {png_path}")

print(f"PDF : {pdf_path}")

print(f"YAML: {yaml_path}")

print()

print("Mit 100% Skalierung drucken (fit-to-page AUS).")

print(f"Nach dem Druck: ein Quadrat = {SQUARE_LENGTH_MM:.1f} mm muss sein (mit dem Lineal pruefen).")

return 0


if __name__ == "__main__":

    sys.exit(main())

```

src/mycobot_calibration/setup.cfg

```
[develop]
```

```
script_dir=$base/lib/mycobot_calibration
```

```
[install]
```

```
install_scripts=$base/lib/mycobot_calibration
```

src/mycobot_calibration/setup.py

```
from glob import glob

from setuptools import setup

package_name = 'mycobot_calibration'

setup(
    name=package_name,
    version='0.1.0',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages', ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        ('share/' + package_name + '/launch', glob('launch/*.launch.py')),
        ('share/' + package_name + '/config', glob('config/*.yaml')),
        ('share/' + package_name + '/boards', glob('boards/*')),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='er',
    maintainer_email='ekinciorhandijvar@gmail.com',
    description='Hand-Auge-Kalibrierungswerkzeuge fuer myCobot 280 JN + D435i.',
    license='BSD-3-Clause',
    entry_points={
        'console_scripts': [
            'charuco_detector = mycobot_calibration.charuco_detector:main',
            'mycobot_bridge = mycobot_calibration.joint_encoder_publisher:main',
            'joint_pose_gui = mycobot_calibration.joint_pose_gui:main',
            'estop_button = mycobot_calibration.estop_button:main',
        ],
    },
)
```

src/mycobot_demo/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)

project(mycobot_demo)

if(NOT CMAKE_CXX_STANDARD)
    set(CMAKE_CXX_STANDARD 17)
endif()

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
    add_compile_options(-Wall -Wextra -Wpedantic)
endif()

find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(moveit_ros_planning_interface REQUIRED)
find_package(moveit_msgs REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(tf2_geometry_msgs REQUIRED)
find_package(visualization_msgs REQUIRED)
find_package(rclcpp_action REQUIRED)
find_package(control_msgs REQUIRED)
find_package(trajjectory_msgs REQUIRED)
find_package(std_msgs REQUIRED)

add_executable(pick_place_cartesian src/pick_place_cartesian.cpp)

ament_target_dependencies(pick_place_cartesian
    rclcpp
    rclcpp_action
    moveit_ros_planning_interface
    moveit_msgs
    geometry_msgs
    tf2_geometry_msgs
    control_msgs
    trajectory_msgs
)
```

```
add_executable(approach_target src/approach_target.cpp)
```

```
ament_target_dependencies(approach_target
```

```
  rclcpp
```

```
  rclcpp_action
```

```
  moveit_ros_planning_interface
```

```
  moveit_msgs
```

```
  geometry_msgs
```

```
  tf2_geometry_msgs
```

```
  visualization_msgs
```

```
  control_msgs
```

```
  trajectory_msgs
```

```
)
```

```
add_executable(pick_tilt src/pick_tilt.cpp)
```

```
ament_target_dependencies(pick_tilt
```

```
  rclcpp
```

```
  rclcpp_action
```

```
  moveit_ros_planning_interface
```

```
  moveit_msgs
```

```
  geometry_msgs
```

```
  tf2_geometry_msgs
```

```
  visualization_msgs
```

```
  control_msgs
```

```
  trajectory_msgs
```

```
  std_msgs
```

```
)
```

```
add_executable(goto_clicked_point src/goto_clicked_point.cpp)
```

```
ament_target_dependencies(goto_clicked_point
```

```
  rclcpp
```

```
  moveit_ros_planning_interface
```

```
  moveit_msgs
```

```
  geometry_msgs
```

```
  tf2_geometry_msgs
```

```
  visualization_msgs
```

```
)
```

```
install(TARGETS pick_place_cartesian approach_target pick_tilt goto_clicked_point
  DESTINATION lib/${PROJECT_NAME}
)

install(DIRECTORY launch rviz
  DESTINATION share/${PROJECT_NAME}
)

ament_package()
```


src/mycobot_demo/launch/approach_target.launch.py

```
# Ziel-Annäherungs-Demo (nur pre-grasp): 0-Punkt → 10cm über dem Ziel, 90° senkrecht, Greifer öffnen.

# Zuerst muss demo.launch.py laufen (move_group + controllers + RViz).

# Für MoveGroupInterface laden wir die robot_description + SRDF + kinematics Parameter.


import os

from pathlib import Path

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node
from moveit_configs_utils import MoveItConfigsBuilder

def generate_launch_description():

    world_share = get_package_share_directory("mycobot_world")

    xacro_path = str(Path(world_share) / "urdf" / "mycobot_world.urdf.xacro")

    use_fake_hw = os.environ.get("USE_FAKE_HARDWARE", "false")

    robot_port = os.environ.get("MYCOBOT_PORT", "/dev/ttyTHS1")

    moveit_config = (
        MoveItConfigsBuilder("mycobot")
        .robot_description(
            file_path=xacro_path,
            mappings={
                "use_fake_hardware": use_fake_hw,
                "robot_port": robot_port,
            },
        )
        .robot_description_semantic(file_path="config/mycobot.srdf")
        .robot_description_kinematics(file_path="config/kinematics.yaml")
        .to_moveit_configs()
    )

    moveit_config.robot_description_kinematics = {
```

```

"robot_description_kinematics": {
    "arm": {
        "kinematics_solver": "trac_ik_kinematics_plugin/TRAC_IKKinematicsPlugin",
        "kinematics_solver_search_resolution": float(0.005),
        "kinematics_solver_timeout": float(0.5),
        "position_only_ik": False,
        "solve_type": "Distance",
        "epsilon": float(1e-5),
    }
}

approach_params = {
    "approach_height": float(os.environ.get("APPROACH_HEIGHT", "0.10")),
}

return LaunchDescription([
    Node(
        package="mycobot_demo",
        executable="approach_target",
        name="approach_target",
        output="screen",
        parameters=[
            moveit_config.robot_description,
            moveit_config.robot_description_semantic,
            moveit_config.robot_description_kinematics,
            approach_params,
            {"use_sim_time": False},
        ],
    ),
])

```

src/mycobot_demo/launch/goto_clicked_point.launch.py

```
# Kalibrierungs-Verifikation: RViz-Klick → Roboter hover.  
  
# Zuerst muss demo.launch.py laufen (move_group + RViz + controllers + Kamera).  
  
# Dieser Launch startet nur das goto_clicked_point Executable.  
  
# MoveGroupInterface benötigt die robot_description + SRDF + kinematics Parameter,  
# daher laden wir sie auch hier mit MoveItConfigsBuilder und übergeben sie an den Node.
```

```
import os  
  
from pathlib import Path  
  
  
from ament_index_python.packages import get_package_share_directory  
from launch import LaunchDescription  
from launch_ros.actions import Node  
from moveit_configs_utils import MoveItConfigsBuilder  
  
  
def generate_launch_description():  
    world_share = get_package_share_directory("mycobot_world")  
    xacro_path = str(Path(world_share) / "urdf" / "mycobot_world.urdf.xacro")  
  
    use_fake_hw = os.environ.get("USE_FAKE_HARDWARE", "false")  
    robot_port = os.environ.get("MYCOBOT_PORT", "/dev/ttyTHS1")  
  
    moveit_config = (  
        MoveItConfigsBuilder("mycobot")  
        .robot_description(  
            file_path=xacro_path,  
            mappings={  
                "use_fake_hardware": use_fake_hw,  
                "robot_port": robot_port,  
            },  
        )  
        .robot_description_semantic(file_path="config/mycobot.srdf")  
        .robot_description_kinematics(file_path="config/kinematics.yaml")  
        .to_moveit_configs()  
    )
```

```

moveit_config.robot_description_kinematics = {
    "robot_description_kinematics": {
        "arm": {
            "kinematics_solver": "trac_ik_kinematics_plugin/TRAC_IKKinematicsPlugin",
            "kinematics_solver_search_resolution": float(0.005),
            "kinematics_solver_timeout": float(0.5),
            "position_only_ik": False,
            "solve_type": "Distance",
            "epsilon": float(1e-5),
        }
    }
}

return LaunchDescription([
    Node(
        package="mycobot_demo",
        executable="goto_clicked_point",
        name="goto_clicked_point",
        output="screen",
        parameters=[
            moveit_config.robot_description,
            moveit_config.robot_description_semantic,
            moveit_config.robot_description_kinematics,
            {"use_sim_time": False},
        ],
    ),
])

```

src/mycobot_demo/launch/pick_place.launch.py

```
# Führt die Pick & Place Demo aus.

# Zuerst muss demo.launch.py laufen (move_group + RViz + controllers).

# Für MoveGroupInterface laden wir die robot_description + SRDF + kinematics Parameter.


import os

from pathlib import Path

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node
from moveit_configs_utils import MoveItConfigsBuilder

def generate_launch_description():

    world_share = get_package_share_directory("mycobot_world")

    xacro_path = str(Path(world_share) / "urdf" / "mycobot_world.urdf.xacro")

    use_fake_hw = os.environ.get("USE_FAKE_HARDWARE", "false")

    robot_port = os.environ.get("MYCOBOT_PORT", "/dev/ttyTHS1")

    moveit_config = (
        MoveItConfigsBuilder("mycobot")
        .robot_description(
            file_path=xacro_path,
            mappings={
                "use_fake_hardware": use_fake_hw,
                "robot_port": robot_port,
            },
        )
        .robot_description_semantic(file_path="config/mycobot.srdf")
        .robot_description_kinematics(file_path="config/kinematics.yaml")
        .to_moveit_configs()
    )

    moveit_config.robot_description_kinematics = {
```

```

"robot_description_kinematics": {
    "arm": {
        "kinematics_solver": "trac_ik_kinematics_plugin/TRAC_IKKinematicsPlugin",
        "kinematics_solver_search_resolution": float(0.005),
        "kinematics_solver_timeout": float(0.5),
        "position_only_ik": True,
        "solve_type": "Distance",
        "epsilon": float(1e-5),
    }
}
}

# place-Ziel + grasp z-offset können per env überschrieben werden (run_pick_test.sh).
pick_params = {
    "place_x": float(os.environ.get("PLACE_X", "0.15")),
    "place_y": float(os.environ.get("PLACE_Y", "0.15")),
    "place_z": float(os.environ.get("PLACE_Z", "0.05")),
    "grasp_z_offset": float(os.environ.get("GRASP_Z_OFFSET", "0.0")),
}

return LaunchDescription([
    Node(
        package="mycobot_demo",
        executable="pick_place_cartesian",
        name="pick_place_cartesian",
        output="screen",
        parameters=[
            moveit_config.robot_description,
            moveit_config.robot_description_semantic,
            moveit_config.robot_description_kinematics,
            pick_params,
            {"use_sim_time": False},
        ],
    ),
])

```

src/mycobot_demo/launch/pick_tilt.launch.py

```
# Tilt-bewusster PICK (Sitzung 25): home → hover → Bestätigung → paralleler Abstieg → greifen → heben → halten.  
# Zuerst muss demo.launch.py (move_group + controllers) laufen.  
# Für MoveGroupInterface werden robot_description + SRDF + kinematics geladen.  
# position_only_ik:False – Greifer-Ausrichtung (tilt-ausgerichteter Grasp) MUSS erzwungen werden.
```

```
import os  
  
from pathlib import Path  
  
  
from ament_index_python.packages import get_package_share_directory  
from launch import LaunchDescription  
from launch_ros.actions import Node  
from moveit_configs_utils import MoveItConfigsBuilder  
  
  
def generate_launch_description():  
    world_share = get_package_share_directory("mycobot_world")  
    xacro_path = str(Path(world_share) / "urdf" / "mycobot_world.urdf.xacro")  
  
    use_fake_hw = os.environ.get("USE_FAKE_HARDWARE", "false")  
    robot_port = os.environ.get("MYCOBOT_PORT", "/dev/ttyTHS1")  
  
    moveit_config = (  
        MoveItConfigsBuilder("mycobot")  
        .robot_description(  
            file_path=xacro_path,  
            mappings={  
                "use_fake_hardware": use_fake_hw,  
                "robot_port": robot_port,  
            },  
        )  
        .robot_description_semantic(file_path="config/mycobot.srdf")  
        .robot_description_kinematics(file_path="config/kinematics.yaml")  
        .to_moveit_configs()  
    )
```

```

moveit_config.robot_description_kinematics = {
    "robot_description_kinematics": {
        "arm": {
            "kinematics_solver": "trac_ik_kinematics_plugin/TRAC_IKKinematicsPlugin",
            "kinematics_solver_search_resolution": float(0.005),
            "kinematics_solver_timeout": float(0.5),
            "position_only_ik": False,
            "solve_type": "Distance",
            "epsilon": float(1e-5),
        }
    }
}

pick_params = {
    "approach_height": float(os.environ.get("APPROACH_HEIGHT", "0.10")),
    "grasp_z_offset": float(os.environ.get("GRASP_Z_OFFSET", "0.0")),
    # PREVIEW_CONFIRM=1 → jede Bewegung (home/hover/Abstieg/Heben) wird zuerst in
    # RViz vorab angezeigt und NICHT ausgeführt, bis /pick/confirm (ENTER) kommt
    # (echter Roboter wird zuerst beobachtet). 0 → altes Verhalten (nur Abstieg+Greif-Bestätigung).
    "preview_confirm": os.environ.get("PREVIEW_CONFIRM", "0") in ("1", "true", "True"),
    # Der Abstiegs-Cartesian wird mit dieser (niedrigeren) Geschwindigkeit parametrisiert – präziser Grasp.
    "descend_vel_scale": float(os.environ.get("DESCEND_VEL_SCALE", "0.10")),
    # Freiraum (home/hover) Eilgang-Geschwindigkeit – Übergänge ohne Schrauben-Kontakt.
    # Abstieg/Greifen/Heben sind nicht betroffen. Entspricht CNC G00.
    "rapid_vel_scale": float(os.environ.get("RAPID_VEL_SCALE", "0.40")),
    # PLACE (Sortierung nach Farbe): nach dem Greifen und Rückkehr zu home die Schraube aus der Luft
    # in die ihrer FARBE zugehörige Box ablegen. Box-Mittelpunkte kommen aus box_map.yaml (teach_boxes.py).
    "place_enabled": os.environ.get("PLACE_ENABLED", "1") in ("1", "true", "True"),
    # ABSOLUTES z (von robot_base/Plattform): Boxen an der Reichweitengrenze → nicht das
    # gespeicherte z der Box, sondern feste Höhe. Ausrichtung wird per yaw+tilt-Scan gefunden.
    "place_hover_z": float(os.environ.get("PLACE_HOVER_Z", "0.12")),
    "place_drop_z": float(os.environ.get("PLACE_DROP_Z", "0.08")),
}

# Farbe→Box Zuordnung (ros params YAML). teach_boxes.py schreibt sie; falls vorhanden, wird sie geladen.
# Zum Aktualisieren der Koordinaten ist KEIN REBUILD nötig – neu einlernen/ausführen genügt.

box_map = os.environ.get("BOX_MAP", os.path.expanduser("~/ros2_ws/box_map.yaml"))

```



```
node_params = [  
    moveit_config.robot_description,  
    moveit_config.robot_description_semantic,  
    moveit_config.robot_description_kinematics,  
    pick_params,  
    {"use_sim_time": False},  
]  
  
if os.path.exists(box_map):  
    node_params.append(box_map)  
  
return LaunchDescription([  
    Node(  
        package="mycobot_demo",  
        executable="pick_tilt",  
        name="pick_tilt",  
        output="screen",  
        parameters=node_params,  
    ),  
])
```

src/mycobot_demo/package.xml

```
<?xml version="1.0"?>

<package format="3">

  <name>mycobot_demo</name>

  <version>0.1.0</version>

  <description>

    MoveIt-2-basierte Demo-Nodes fuer den myCobot 280 JN.

    Erste Demo: Pick & Place – mit kartesischem Anfahren (Approach) und Rueckzug (Retreat).

  </description>

  <maintainer email="ekinciorhandijvar@gmail.com">er</maintainer>

  <license>MIT</license>


  <buildtool_depend>ament_cmake</buildtool_depend>


  <depend>rclcpp</depend>

  <depend>moveit_ros_planning_interface</depend>

  <depend>moveit_msgs</depend>

  <depend>geometry_msgs</depend>

  <depend>tf2_geometry_msgs</depend>

  <depend>visualization_msgs</depend>

  <depend>rclcpp_action</depend>

  <depend>control_msgs</depend>

  <depend>trajectory_msgs</depend>

  <depend>std_msgs</depend>


  <exec_depend>mycobot_world</exec_depend>

  <exec_depend>mycobot_moveit_config</exec_depend>

  <exec_depend>moveit_configs_utils</exec_depend>


  <export>

    <build_type>ament_cmake</build_type>

  </export>

</package>
```

src/mycobot_demo/rviz/approach.rviz

Panels:

- Class: rviz_common/Displays

Help Height: 78

Name: Displays

Property Tree Widget:

Expanded: ~

Splitter Ratio: 0.5

Tree Height: 78

Visualization Manager:

Class: ""

Displays:

- Alpha: 0.5

Cell Size: 0.10000000149011612

Class: rviz_default_plugins/Grid

Color: 160; 160; 164

Enabled: true

Line Style:

Line Width: 0.029999999329447746

Value: Lines

Name: Grid

Normal Cell Count: 0

Offset:

X: 0

Y: 0

Z: 0

Plane: XY

Plane Cell Count: 10

Reference Frame: <Fixed Frame>

Value: true

- Acceleration_Scaling_Factor: 0.1

Class: moveit_rviz_plugin/MotionPlanning

Enabled: true

Move Group Namespace: ""

MoveIt_Allow_Approximate_IK: false

MoveIt_Allow_External_Program: false

MoveIt_Allow_Replanning: false

MoveIt_Allow_Sensor_Positioning: false

MoveIt_Planning_Attempts: 10

MoveIt_Planning_Time: 5

MoveIt_Use_Cartesian_Path: false

MoveIt_Use_Constraint_Aware_IK: true

MoveIt_Workspace:

Center:

X: 0

Y: 0

Z: 0

Size:

X: 2

Y: 2

Z: 2

Name: MotionPlanning

Planned Path:

Color Enabled: false

Interrupt Display: false

Links:

All Links Enabled: true

Expand Joint Details: false

Expand Link Details: false

Expand Tree: false

Link Tree Style: Links in Alphabetic Order

Loop Animation: false

Robot Alpha: 0.5

Robot Color: 150; 50; 150

Show Robot Collision: false

Show Robot Visual: true

Show Trail: false

State Display Time: 0.05 s

Trail Step Size: 1

Trajectory Topic: display_planned_path

Planning Metrics:

Payload: 1

Show Joint Torques: false

Show Manipulability: false

Show Manipulability Index: false

Show Weight Limit: false

TextHeight: 0.07999999821186066

Planning Request:

Colliding Link Color: 255; 0; 0

Goal State Alpha: 1

Goal State Color: 250; 128; 0

Interactive Marker Size: 0.029999999329447746

Joint Violation Color: 255; 0; 255

Planning Group: arm

Query Goal State: false

Query Start State: false

Show Workspace: false

Start State Alpha: 1

Start State Color: 0; 255; 0

Planning Scene Topic: monitored_planning_scene

Robot Description: robot_description

Scene Geometry:

Scene Alpha: 0.8999999761581421

Scene Color: 50; 230; 50

Scene Display Time: 9.999999747378752e-05

Show Scene Geometry: true

Voxel Coloring: Z-Axis

Voxel Rendering: Occupied Voxels

Scene Robot:

Attached Body Color: 150; 50; 150

Links:

All Links Enabled: true

Expand Joint Details: false

Expand Link Details: false

Expand Tree: false

Link Tree Style: Links in Alphabetic Order

Robot Alpha: 1

Show Robot Collision: false

Show Robot Visual: true

Value: true

Velocity_Scaling_Factor: 0.1

- Class: rviz_default_plugins/Marker

Enabled: true

Name: Hedef_Secilen

Namespaces:

{}

Topic:

Depth: 5

Durability Policy: Transient Local

History Policy: Keep Last

Reliability Policy: Reliable

Value: /approach/target_marker

Value: true

- Class: rviz_default_plugins/Marker

Enabled: true

Name: Hedef_Detector

Namespaces:

{}

Topic:

Depth: 5

Durability Policy: Volatile

History Policy: Keep Last

Reliability Policy: Reliable

Value: /vida/target_marker

Value: true

Enabled: true

Global Options:

Background Color: 48; 48; 48

Fixed Frame: robot_base

Frame Rate: 30

Name: root

Tools:

- Class: rviz_default_plugins/Interact

Hide Inactive Objects: true

- Class: rviz_default_plugins/MoveCamera

- Class: rviz_default_plugins/Select

- Class: rviz_default_plugins/FocusCamera

Transformation:

Current:

Class: rviz_default_plugins/TF

Value: true

Views:

Current:

Class: rviz_default_plugins/Orbit

Distance: 0.8

Enable Stereo Rendering:

Value: false

Focal Point:

X: 0.1

Y: 0

Z: 0.2

Focal Shape Fixed Size: true

Focal Shape Size: 0.05000000074505806

Invert Z Axis: false

Name: Current View

Near Clip Distance: 0.009999999776482582

Pitch: 0.6

Target Frame: <Fixed Frame>

Value: Orbit (rviz_default_plugins)

Yaw: 5.6

Saved: ~

Window Geometry:

Displays:

collapsed: false

Height: 700

Hide Left Dock: false

Hide Right Dock: true

Width: 1024

X: 0

Y: 24

src/mycobot_demo/src/approach_target.cpp

```
// myCobot 280 JN – Schrauben-/Ziel-ANNAEHERUNG (nur pre-grasp, KEIN descend/greifen).

//

// Benutzerwunsch (2026-06-07):

// 1) Roboter startet IMMER vom Punkt 0 (SRDF "home" = alle Gelenke 0).

// 2) Genau über die vom YOLO-Detector gewählte Schraube/Ziel, Greifer 90° SENKRECHT

// (top-down), 10 cm über dem Ziel anhalten und den Greifer OEFFNEN.

// Hier wird angehalten – KEIN Abstieg / Schließen / Place. (pick_place_cartesian macht das.)

//

// RViz: Ziel wird mit /approach/target_marker (grüne Kugel) + dem /vida/target_marker

// des Detectors angezeigt; die Planungs-Vorschau ist über /display_planned_path sichtbar.

//

// Frame: robot_base (planning), tcp (end-effector). Greifer direkt über Controller.


#include <atomic>

#include <chrono>

#include <cmath>

#include <thread>

#include <vector>


#include <rclcpp/rclcpp.hpp>

#include <rclcpp_action/rclcpp_action.hpp>

#include <moveit/move_group_interface/move_group_interface.h>

#include <geometry_msgs/msg/pose.hpp>

#include <geometry_msgs/msg/pose_stamped.hpp>

#include <geometry_msgs/msg/point.hpp>

#include <visualization_msgs/msg/marker.hpp>

#include <control_msgs/action/follow_joint_trajectory.hpp>

#include <trajectory_msgs/msg/joint_trajectory_point.hpp>

#include <tf2/LinearMath/Quaternion.h>

#include <tf2_geometry_msgs/tf2_geometry_msgs.h>


using moveit::planning_interface::MoveGroupInterface;

using Pose = geometry_msgs::msg::Pose;

using PoseStamped = geometry_msgs::msg::PoseStamped;

using Marker = visualization_msgs::msg::Marker;
```



```

using FollowJointTrajectory = control_msgs::action::FollowJointTrajectory;

using GripperClient = rclcpp_action::Client<FollowJointTrajectory>;

namespace
{
constexpr double EEf_STEP          = 0.005;
constexpr double VEL_SCALE         = 0.20;    // Sicherheit: 20% Geschwindigkeit
constexpr double ACC_SCALE         = 0.20;
constexpr double TARGET_WAIT_SEC = 30.0;    // Warten auf /vida/target
}

// Greifer schaut nach unten: tcp +Y -> Welt -Z. yaw = Azimut, tilt = Abweichung von der Senkrechten.
// tilt=0 -> direkt nach unten (90° senkrecht). (exakt gleiche Konvention wie pick_place_cartesian.)
static geometry_msgs::msg::Quaternion gripperOrientation(double yaw, double tilt)
{
    tf2::Quaternion q;
    q.setRPY(-M_PI_2 + tilt, 0.0, yaw);
    q.normalize();
    return tf2::toMsg(q);
}

// Versucht die Pre-grasp Pose mit einem yaw-Raster (tilt=0, direkt nach unten); wendet den ersten planbaren+
// ausführbaren an. Wenn die Schraube armnah/fern ist, ist die reine senkrechte Einzel-Pose in der IK evtl. nicht
// lösbar;

// durch yaw-Drehen suchen wir eine erreichbare Konfiguration (KEIN descend – Ausrichtung wird nicht
// weitergegeben).

static bool approachTopDown(
    MoveGroupInterface& arm, const Pose& pre_pos,
    const rclcpp::Logger& log)
{
    const double yaw_out = std::atan2(pre_pos.position.y, pre_pos.position.x);
    const std::vector<double> yaws{
        yaw_out, yaw_out + M_PI_2, yaw_out - M_PI_2,
        yaw_out + M_PI, yaw_out + M_PI_4, yaw_out - M_PI_4,
    };

    Pose target = pre_pos;
    MoveGroupInterface::Plan plan;

```

```

for (double yaw : yaws) {

    target.orientation = gripperOrientation(yaw, 0.0);

    arm.setPoseTarget(target);

    if (arm.plan(plan) == moveit::planning_interface::MoveItErrorCode::SUCCESS) {

        RCLCPP_INFO(log, "[pre-grasp] geloest: yaw=%+.0f° tilt=0° (senkrecht)",
                    yaw * 180.0 / M_PI);

        if (!arm.execute(plan)) {

            RCLCPP_ERROR(log, "[pre-grasp] execute() fehlgeschlagen");

            return false;

        }

        return true;

    }

    RCLCPP_INFO(log, " [pre-grasp] yaw=%+.0f° nicht planbar, naechste", yaw * 180.0 / M_PI);

}

RCLCPP_ERROR(log, "[pre-grasp] kein yaw planbar – Ziel unerreichbar "
                "(in den gruenen Kreis legen)");

return false;

}

// Greifer DIREKT an den Controller (FollowJointTrajectory) senden (open=0.15).

static bool setGripper(

    const std::shared_ptr<GripperClient>& client,

    const std::string& named_state, const rclcpp::Logger& log)

{

    const double pos = (named_state == "open") ? 0.15 : -0.74;

    if (!client->wait_for_action_server(std::chrono::seconds(3))) {

        RCLCPP_ERROR(log, "kein Greifer-Action-Server");

        return false;

    }

    FollowJointTrajectory::Goal goal;

    goal.trajectory.joint_names = {"gripper_controller"};

    trajectory_msgs::msg::JointTrajectoryPoint pt;

    pt.positions = {pos};

    pt.time_from_start = rclcpp::Duration::from_seconds(1.0);

    goal.trajectory.points = {pt};

    auto goal_fut = client->async_send_goal(goal);

```

```

    if (goal_fut.wait_for(std::chrono::seconds(3)) != std::future_status::ready) {
        RCLCPP_ERROR(log, "Greifer-Goal konnte nicht gesendet werden"); return false;
    }

    auto handle = goal_fut.get();

    if (!handle) { RCLCPP_ERROR(log, "Greifer-Goal abgelehnt"); return false; }

    auto res_fut = client->async_get_result(handle);

    if (res_fut.wait_for(std::chrono::seconds(5)) != std::future_status::ready) {
        RCLCPP_WARN(log, "Greifer-Ergebnis Zeitueberschreitung (trotzdem weiter)");
    }

    std::this_thread::sleep_for(std::chrono::milliseconds(300));

    RCLCPP_INFO(log, "Greifer -> %s (pos=%.2f) OK", named_state.c_str(), pos);

    return true;
}

static Marker makeTargetMarker(const geometry_msgs::msg::Point& p)
{
    Marker m;

    m.header.frame_id = "robot_base";

    m.ns = "approach_target";

    m.id = 0;

    m.type = Marker::SPHERE;

    m.action = Marker::ADD;

    m.pose.position = p;

    m.pose.orientation.w = 1.0;

    m.scale.x = m.scale.y = m.scale.z = 0.02; // 2 cm Kugel

    m.color.g = 1.0; m.color.a = 0.9; // grün

    return m;
}

int main(int argc, char** argv)
{
    rclcpp::init(argc, argv);

    auto node = rclcpp::Node::make_shared(
        "approach_target",
        rclcpp::NodeOptions().automatically_declare_parameters_from_overrides(true));

    auto log = node->get_logger();

```

```

auto param_d = [&](const std::string& name, double def) {
    if (!node->has_parameter(name)) node->declare_parameter<double>(name, def);
    return node->get_parameter(name).as_double();
};

const double approach_height = param_d("approach_height", 0.10); // 10 cm darüber

rclcpp::executors::SingleThreadedExecutor executor;
executor.add_node(node);
std::thread spinner([&]() { executor.spin(); });

auto marker_pub = node->create_publisher<Marker>("/approach/target_marker",
                                                rclcpp::QoS(1).transient_local());

// --- Zielposition vom Detector holen (/vida/target, robot_base Frame) ---
geometry_msgs::msg::Point screw;
std::atomic<bool> have_target{false};
auto target_sub = node->create_subscription<PoseStamped>(
    "/vida/target", rclcpp::QoS(1),
    [&](PoseStamped::SharedPtr m) {
        screw = m->pose.position; have_target = true; // immer das AKTUELLSTE Ziel
    });

MoveGroupInterface arm(node, "arm");
auto gripper_client = rclcpp_action::create_client<FollowJointTrajectory>(
    node, "/gripper_controller/follow_joint_trajectory");
arm.setPoseReferenceFrame("robot_base");
arm.setEndEffectorLink("tcp");
arm.setPlanningTime(5.0);
arm.setNumPlanningAttempts(10);
arm.setMaxVelocityScalingFactor(VEL_SCALE);
arm.setMaxAccelerationScalingFactor(ACC_SCALE);
(void)EEF_STEP;

// --- SCHRITT 1: Roboter zum Punkt 0 (SRDF home = alle Gelenke 0) ---
RCLCPP_INFO(log, ">> SCHRITT 1: zum Punkt 0 fahren (home, alle Gelenke 0)");
arm.setNamedTarget("home");

if (!arm.move()) {

```

```

    RCLCPP_ERROR(log, "konnte nicht zum Punkt 0 fahren"); rclcpp::shutdown(); spinner.join(); return 1;
}

// --- Auf erstes Ziel warten ---
RCLCPP_INFO(log, "warte auf /vida/target (Detector muss laufen)...");
{
    auto t0 = node->now();

    while (rclcpp::ok() && !have_target &&
           (node->now() - t0).seconds() < TARGET_WAIT_SEC) {
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
    }
}

if (!have_target) {
    RCLCPP_ERROR(log, "innerhalb von %.0f s kein /vida/target gekommen – laeuft Detector/Kamera?",
                 TARGET_WAIT_SEC);

    rclcpp::shutdown(); spinner.join(); return 1;
}

// --- SCHRITT 2: erneut versuchen BIS ERREICHT (wenn Schraube fern ist, nähert der Benutzer sie an) ---
// Bei jedem Versuch wird das AKTUELLSTE Ziel gelesen; wenn unerreichbar, wartet der Roboter bei home und fährt
// automatisch, sobald die Schraube in die grüne Zone gebracht wird. Bei Erfolg öffnet er den Greifer und wartet
// dort.

bool reached = false;
while (rclcpp::ok() && !reached) {
    geometry_msgs::msg::Point t = screw;           // aktuellstes Ziel
    marker_pub->publish(makeTargetMarker(t));       // Ziel in Kamera/RViz anzeigen
    const double r = std::sqrt(t.x * t.x + t.y * t.y) * 1000.0;
    RCLCPP_INFO(log, "Ziel: x=%.3f y=%.3f z=%.3f (r=%.0fmm)", t.x, t.y, t.z, r);

    Pose pre;
    pre.position.x = t.x;
    pre.position.y = t.y;
    pre.position.z = t.z + approach_height;

    RCLCPP_INFO(log, ">> SCHRITT 2: %.0f cm ueber dem Ziel (z=%.3f), Greifer 90° senkrecht",
                 approach_height * 100.0, pre.position.z);

    if (approachTopDown(arm, pre, log)) {

```

```

RCLCPP_INFO(log, ">> Greifer oeffnen");

setGripper(gripper_client, "open", log);

RCLCPP_INFO(log, "    FERTIG – Roboter %.0fcm ueber dem Ziel, 90° senkrecht, Greifer offen. "
               "Wartet dort (mit Ctrl+C schliessen).", approach_height * 100.0);

reached = true;

break;
}

RCLCPP_WARN(log, "⚠ Ziel UNERREICHBAR (r=%.0fmm; ~265mm Senkrecht-Limit). Schraube NAEHER an den Roboter "
               "legen (in den GRUENEN Kreis in der Kamera, r<220mm). In 5s erneut...", r);

for (int k = 0; k < 50 && rclcpp::ok(); ++k) // 5 s warten, Marker am Leben halten
    std::this_thread::sleep_for(std::chrono::milliseconds(100));
}

// Nach Erfolg den Marker am Leben halten + den Node am Laufen halten. Beenden mit Ctrl+C.
rclcpp::Rate rate(2.0);
while (rclcpp::ok()) {
    marker_pub->publish(makeTargetMarker(screw));

    rate.sleep();
}

rclcpp::shutdown();
spinner.join();

return 0;
}

```

src/mycobot_demo/src/goto_clicked_point.cpp

```
// RViz-Klick → Greifer senkrecht 5 cm über den Punkt hovern.

//

// Ablauf:

// 1) In RViz mit dem "Publish Point" Tool auf einen Punkt der Pointcloud klicken
// 2) Wenn /clicked_point kommt, Marker veröffentlichen + Sicherheits-/Reichweiten-Prüfung
// 3) Annäherungs-Pose: Punkt + 15 cm Z, Greifer schaut nach unten (tcp Y = world -Z)
//     Yaw frei – zuerst atan2(y,x), bei Fehlschlag werden andere Winkel probiert
// 4) Cartesian Abstieg: 15 cm → 5 cm Hover

//

// Wichtig:

// - Fixed Frame robot_base (RViz Publish Point veröffentlicht in diesem Frame)
// - TCP Frame +Y Achse ist "fingers" Richtung (gripper_base_to_tcp xyz="0 0.105 -0.01")
// - position_only_ik = false (aus launch), TRAC-IK sucht 6DOF


#include <atomic>

#include <chrono>

#include <cmath>

#include <thread>

#include <vector>


#include <rclcpp/rclcpp.hpp>

#include <moveit/move_group_interface/move_group_interface.h>

#include <moveit_msgs/msg/robot_trajectory.hpp>

#include <geometry_msgs/msg/point_stamped.hpp>

#include <geometry_msgs/msg/pose.hpp>

#include <visualization_msgs/msg/marker.hpp>

#include <tf2/LinearMath/Quaternion.h>

#include <tf2_geometry_msgs/tf2_geometry_msgs.h>


using moveit::planning_interface::MoveGroupInterface;


namespace

{

constexpr double HOVER_HEIGHT    = 0.05;    // 5 cm darüber – Ziel-Hover

constexpr double APPROACH_HEIGHT = 0.15;    // 15 cm darüber – Annäherung
```

```
constexpr double MAX_REACH_XY    = 0.25;    // myCobot 280: 28 cm Reichweite – 3 cm Sicherheit
// Innere Totzone: Schulter bei z=0.158 m; an Punkten sehr nah an der Basis muss der Arm den
// Greifer nicht senkrecht halten können und den Ellbogen über sich selbst falten → Plan abort.
// Der echte Arbeitsbereich ist ein Ring: MIN_REACH_XY .. MAX_REACH_XY.
constexpr double MIN_REACH_XY    = 0.13;    // 13 cm – innerhalb dieses Radius unerreichbar
constexpr double VEL_SCALE       = 0.20;
constexpr double ACC_SCALE       = 0.20;
constexpr double EEK_STEP        = 0.005;   // 5 mm
constexpr double JUMP_THRESHOLD  = 0.0;     // 0 = deaktiviert
constexpr double MIN_FRACTION    = 0.90;
}
```

```
class GotoClickedPoint : public rclcpp::Node
{
public:
    GotoClickedPoint()
    : Node("goto_clicked_point",
          rclcpp::NodeOptions().automatically_declare_parameters_from_overrides(true))
    {
        marker_pub_ = create_publisher<visualization_msgs::msg::Marker>(
            "/clicked_target_marker", 10);

        boundary_pub_ = create_publisher<visualization_msgs::msg::Marker>(
            "/reach_boundary_marker", 10);

        click_sub_ = create_subscription<geometry_msgs::msg::PointStamped>(
            "/clicked_point", 10,
            std::bind(&GotoClickedPoint::onClick, this, std::placeholders::_1));

        // Reichweitengrenze periodisch veröffentlichen (nicht latched → alle 2 s aktualisieren).
        boundary_timer_ = create_wall_timer(
            std::chrono::seconds(2),
            std::bind(&GotoClickedPoint::publishReachBoundary, this));

        RCLCPP_INFO(get_logger(),
            "Bereit. In RViz mit dem 'Publish Point' Tool auf die Pointcloud klicken. "
            "Klickbarer Bereich ist ein RING: ZWISCHEN dem roten inneren Kreis und dem "
```



```

        "orangefarbenen aeusseren Kreis. Das Innere (Totzone) und Aeussere (ausser Reichweite) funktioniert
nicht.");
    }

    void setArm(std::shared_ptr<MoveGroupInterface> arm) { arm_ = std::move(arm); }

private:
    void onClick(const geometry_msgs::msg::PointStamped::SharedPtr msg)
    {
        const auto& p = msg->point;

        const std::string& frame = msg->header.frame_id;

        if (frame != "robot_base") {

            RCLCPP_WARN(get_logger(),

                "Geklickter Punkt im Frame '%s' – robot_base wurde erwartet. "

                "Setze RViz Fixed Frame auf robot_base.",

                frame.c_str());

            return;
        }

        const double dist_xy = std::hypot(p.x, p.y);

        const double dist_3d = std::sqrt(p.x * p.x + p.y * p.y + p.z * p.z);

        RCLCPP_INFO(get_logger(), "=====");
        RCLCPP_INFO(get_logger(), "Gewaehlter Punkt (robot_base Frame):");
        RCLCPP_INFO(get_logger(), "  x = %.4f m  (%.1f cm)", p.x, p.x * 100);
        RCLCPP_INFO(get_logger(), "  y = %.4f m  (%.1f cm)", p.y, p.y * 100);
        RCLCPP_INFO(get_logger(), "  z = %.4f m  (%.1f cm)", p.z, p.z * 100);
        RCLCPP_INFO(get_logger(), "Abstand zum Roboterzentrum:");
        RCLCPP_INFO(get_logger(), "  in XY-Ebene: %.4f m  (%.1f cm)", dist_xy, dist_xy * 100);
        RCLCPP_INFO(get_logger(), "  3D (gesamt) : %.4f m  (%.1f cm)", dist_3d, dist_3d * 100);
        RCLCPP_INFO(get_logger(), "=====");

        publishMarker(p);

        if (dist_xy > MAX_REACH_XY) {

            RCLCPP_WARN(get_logger(),

```

```

        "Punkt ausserhalb der Roboter-Reichweite (%.1f cm > %.1f cm). Bewegung abgebrochen.",
        dist_xy * 100, MAX_REACH_XY * 100);

    return;
}

if (dist_xy < MIN_REACH_XY) {
    RCLCPP_WARN(get_logger(),
        "Punkt zu nah am Roboter (%.1f cm < %.1f cm) – innere Totzone. "
        "Der Greifer kann senkrecht nach unten nicht so nah heran. "
        "Klicke auf den AEUSSEREN Teil des orangefarbenen Rings.",
        dist_xy * 100, MIN_REACH_XY * 100);

    return;
}

if (!arm_) {
    RCLCPP_WARN(get_logger(), "Arm nicht bereit, nur Marker veroeffentlicht.");
    return;
}

// plan()/execute() blockierend + verstopft den Executor (Action-Client-Deadlock).
// Den Subscription-Callback verlassen, Planung in eigenem Worker-Thread machen.
if (busy_.exchange(true)) {
    RCLCPP_WARN(get_logger(),
        "Vorherige Bewegung laeuft noch – dieser Klick wurde ignoriert.");

    return;
}

geometry_msgs::msg::Point pt = p;
std::thread([this, pt]() {
    moveToHover(pt);
    busy_.store(false);
}).detach();
}

// Der echte Arbeitsbereich ist ein RING (in der z=0 Ebene):
//   - äußerer orangefarbener Kreis = MAX_REACH_XY (außerhalb ist außer Reichweite)
//   - innerer roter Kreis   = MIN_REACH_XY (innerhalb ist Totzone)

```

```

// Der klickbare Bereich ist ZWISCHEN den beiden Kreisen.

void publishReachBoundary()
{
    publishCircle(MAX_REACH_XY, 0, 1.0f, 0.5f, 0.0f); // außen: orange
    publishCircle(MIN_REACH_XY, 1, 1.0f, 0.0f, 0.0f); // innen: rot
}

void publishCircle(double radius, int id, float r, float g, float b)
{
    visualization_msgs::msg::Marker m;

    m.header.frame_id = "robot_base";

    m.header.stamp = now();

    m.ns = "reach_boundary";

    m.id = id;

    m.type = visualization_msgs::msg::Marker::LINE_STRIP;

    m.action = visualization_msgs::msg::Marker::ADD;

    m.pose.orientation.w = 1.0;

    m.scale.x = 0.005; // 5 mm Linie

    m.color.r = r;

    m.color.g = g;

    m.color.b = b;

    m.color.a = 1.0f;

    m.lifetime = rclcpp::Duration::from_seconds(0);

    constexpr int N = 64;

    for (int i = 0; i <= N; ++i) {
        const double a = 2.0 * M_PI * static_cast<double>(i) / N;

        geometry_msgs::msg::Point pt;

        pt.x = radius * std::cos(a);

        pt.y = radius * std::sin(a);

        pt.z = 0.0;

        m.points.push_back(pt);
    }

    boundary_pub_>publish(m);
}

void publishMarker(const geometry_msgs::msg::Point& p)

```

```

{
    visualization_msgs::msg::Marker m;

    m.header.frame_id = "robot_base";

    m.header.stamp = now();

    m.ns = "clicked_target";

    m.id = 0;

    m.type = visualization_msgs::msg::Marker::SPHERE;

    m.action = visualization_msgs::msg::Marker::ADD;

    m.pose.position = p;

    m.pose.orientation.w = 1.0;

    m.scale.x = m.scale.y = m.scale.z = 0.02;

    m.color.r = 0.1f;

    m.color.g = 1.0f;

    m.color.b = 0.1f;

    m.color.a = 1.0f;

    m.lifetime = rclcpp::Duration::from_seconds(0);

    marker_pub_ -> publish(m);
}

// Greifer schaut nach unten (tcp Y -> -Z world). yaw = Azimut, tilt = Abweichung von der Senkrechten.
// tilt=0 → direkt nach unten; tilt>0 → Greifer neigt sich in yaw-Richtung (an nahen
// Punkten kann der Arm schräg von außen annähern statt den Ellbogen zu falten).
static geometry_msgs::msg::Quaternion gripperOrientation(double yaw, double tilt)
{
    tf2::Quaternion q;

    q.setRPY(-M_PI_2 + tilt, 0.0, yaw);

    q.normalize();

    return tf2::toMsg(q);
}

void moveToHover(const geometry_msgs::msg::Point& target)
{
    geometry_msgs::msg::Pose approach;

    approach.position.x = target.x;

    approach.position.y = target.y;

    approach.position.z = target.z + APPROACH_HEIGHT;
}

```

```

// Yaw frei: zuerst auf den Punkt gerichtet (atan2), dann  $\pm 90$ ,  $\pm 180$ ,  $\pm 45$  probieren.
const double yaw_out = std::atan2(target.y, target.x);

const std::vector<double> yaws{

    yaw_out,

    yaw_out + M_PI_2,

    yaw_out - M_PI_2,

    yaw_out + M_PI,

    yaw_out + M_PI_4,

    yaw_out - M_PI_4,

};

// Zuerst direkt nach unten (tilt=0); wenn nicht lösbar, Greifer stufenweise neigen.
// Die Neigung erleichtert das Ellbogen-Falten an armnahen/niedrigen Punkten.

const std::vector<double> tilts{

    0.0,

    20.0 * M_PI / 180.0,

    35.0 * M_PI / 180.0,

};

MoveGroupInterface::Plan approach_plan;

double chosen_yaw = 0.0;

double chosen_tilt = 0.0;

bool ok = false;

for (double tilt : tilts) {

    for (double yaw : yaws) {

        approach.orientation = gripperOrientation(yaw, tilt);

        arm_ -> setPoseTarget(approach);

        if (arm_ -> plan(approach_plan) == moveit::planning_interface::MoveItErrorCode::SUCCESS) {

            chosen_yaw = yaw;

            chosen_tilt = tilt;

            ok = true;

            break;

        }

        RCLCPP_INFO(get_logger(),

            " yaw=%+.0f° tilt=%+.0f° nicht planbar, naechste wird probiert",

            yaw * 180.0 / M_PI, tilt * 180.0 / M_PI);
    }
}

```

```

    }

    if (ok) break;
}

if (!ok) {
    RCLCPP_ERROR(get_logger(),
        "Annaeherungs-Plan fehlgeschlagen – bei keinem yaw/tilt wurde eine Loesung gefunden. "
        "Punkt ausser Reichweite oder Hindernis vorhanden.");

    return;
}

RCLCPP_INFO(get_logger(),
    "Annaeherung: 15 cm darueber, yaw=%+.0f° tilt=%+.0f°, wird ausgefuehrt",
    chosen_yaw * 180.0 / M_PI, chosen_tilt * 180.0 / M_PI);

if (!arm_->execute(approach_plan)) {
    RCLCPP_ERROR(get_logger(), "Annaeherungs-execute fehlgeschlagen.");
    return;
}

// Cartesian Abstieg 15cm → 5cm, Ausrichtung konstant (auch yaw konstant)
geometry_msgs::msg::Pose descent = approach;
descent.position.z = target.z + HOVER_HEIGHT;

std::vector<geometry_msgs::msg::Pose> waypoints{descent};
moveit_msgs::msg::RobotTrajectory traj;
const double fraction = arm_->computeCartesianPath(
    waypoints, EEF_STEP, JUMP_THRESHOLD, traj);

RCLCPP_INFO(get_logger(), "Cartesian Abstieg: %.0f%% Abdeckung", fraction * 100.0);
if (fraction < MIN_FRACTION) {
    RCLCPP_ERROR(get_logger(),
        "Cartesian Abstieg unzureichend (%.0f%% < %.0f%%). Abgebrochen.",
        fraction * 100.0, MIN_FRACTION * 100.0);

    return;
}

```

```

    if (!arm_->execute(traj)) {

        RCLCPP_ERROR(get_logger(), "Abstiegs-execute fehlgeschlagen.");

        return;
    }

    RCLCPP_INFO(get_logger(),

        "Roboter in Hover-Position (Punkt + 5 cm). Mit dem Lineal messen!");
}

rclcpp::Subscription<geometry_msgs::msg::PointStamped>::SharedPtr click_sub_;
rclcpp::Publisher<visualization_msgs::msg::Marker>::SharedPtr marker_pub_;
rclcpp::Publisher<visualization_msgs::msg::Marker>::SharedPtr boundary_pub_;
rclcpp::TimerBase::SharedPtr boundary_timer_;

std::shared_ptr<MoveGroupInterface> arm_;

std::atomic<bool> busy_{false};
};

int main(int argc, char** argv)
{
    rclcpp::init(argc, argv);

    auto node = std::make_shared<GotoClickedPoint>();

    // SingleThreaded – der goal-register/response Race in Galactic rclcpp_action
    // wird nur im Multi-Thread ausgelöst. Damit der Subscription-Callback nicht lange
    // blockiert, wird plan/execute ohnehin in einem eigenen std::thread gemacht (in onClick).

    rclcpp::executors::SingleThreadedExecutor executor;

    executor.add_node(node);

    std::thread spinner([&]() { executor.spin(); });

    auto arm = std::make_shared<MoveGroupInterface>(node, "arm");

    arm->setPoseReferenceFrame("robot_base");

    arm->setEndEffectorLink("tcp");

    arm->setPlanningTime(3.0);

    arm->setNumPlanningAttempts(5);

    arm->setMaxVelocityScalingFactor(VEL_SCALE);

    arm->setMaxAccelerationScalingFactor(ACC_SCALE);

```

```
node->setArm(arm);

RCLCPP_INFO(node->get_logger(), "MoveGroup bereit. Wartet.");


spinner.join();

rclcpp::shutdown();

return 0;

}
```


src/mycobot_demo/src/pick_place_cartesian.cpp

```
// myCobot 280 JN – Schrauben Pick & Place (Cartesian Annäherung + orientation constraint).

//

// Ablauf (exakt nach Herstellerempfehlung – 3-stufiges Greifen):

//   /vida/target (Detector) → Pick-Position

//   home → joint plan → pick_pre (10cm darüber)   ← PRE-GRASP (senkrechte Ziel-Pose)

//       ↓ cartesian descend                        ← LINEAR DESCENT (Ausrichtung konstant)

//       pick → gripper close                        ← CLOSE GRIPPER

//       ↑ cartesian retreat

//       pick_post → joint plan → place_pre (10cm darüber)

//       ↓ cartesian descend

//       place → gripper open

//       ↑ cartesian retreat → home

//

// Greifer senkrecht (90°) halten – Lösung OHNE orientation PATH CONSTRAINT:

// - die ZIEL-Pose von pre-grasp/place_pre ist bereits mit gripperDown() senkrecht definiert,

//   daher endet der joint plan in einer senkrechten Konfiguration.

// - Abstieg/Aufstieg werden mit computeCartesianPath gemacht; zwischen zwei Waypoints

//   wird die Ausrichtung ohnehin konstant (senkrecht) gehalten.

// - Deshalb wird auf die free-space joint plans KEIN orientation constraint gelegt:

//   RRTConnect + rejection-sampling beschränkte Planung (projected state space

//   nicht definiert) lief sekundenlang in timeout/abort (Notizen Sitzung 17b).

//   Ohne Constraint ist die Planung schnell und zuverlässig.

//

// Notizen:

// - Frame: robot_base (planning), tcp (end-effector).

// - SRDF named states: arm/home, arm/ready, gripper/open, gripper/closed.

// - TCP +Y = Annäherungsachse; wenn der Greifer nach unten schaut, +Y → Welt -Z.

//   Yaw (Drehung um die Schraube) frei; seed wird mit atan2 gegeben.


#include <chrono>

#include <thread>

#include <vector>


#include <rclcpp/rclcpp.hpp>

#include <rclcpp_action/rclcpp_action.hpp>
```

```

#include <moveit/move_group_interface/move_group_interface.h>

#include <moveit_msgs/msg/robot_trajectory.hpp>

#include <geometry_msgs/msg/pose.hpp>

#include <geometry_msgs/msg/pose_stamped.hpp>

#include <geometry_msgs/msg/point.hpp>

#include <control_msgs/action/follow_joint_trajectory.hpp>

#include <trajectory_msgs/msg/joint_trajectory_point.hpp>

#include <tf2/LinearMath/Quaternion.h>

#include <tf2_geometry_msgs/tf2_geometry_msgs.h>

using moveit::planning_interface::MoveGroupInterface;

using Pose = geometry_msgs::msg::Pose;

using PoseStamped = geometry_msgs::msg::PoseStamped;

using FollowJointTrajectory = control_msgs::action::FollowJointTrajectory;

using GripperClient = rclcpp_action::Client<FollowJointTrajectory>;

namespace
{
constexpr double APPROACH_HEIGHT = 0.10;    // 10 cm Annäherungshöhe

constexpr double EEf_STEP        = 0.005;   // 5 mm Interpolationsschritt

constexpr double JUMP_THRESHOLD  = 5.0;      // >0: Sprung im Gelenk-Raum ABLEHNEN
                                           // (war 0 → im descend wurde Handgelenk/Ellbogen
                                           //   teleportiert und machte unsinnige Bewegungen)

constexpr double MIN_FRACTION    = 0.95;     // unter 95% nicht akzeptiert

constexpr double VEL_SCALE       = 0.20;     // Sicherheit: 20% Geschwindigkeit

constexpr double ACC_SCALE       = 0.20;

constexpr double TARGET_WAIT_SEC = 10.0;     // Warten auf /vida/target
}

// Greifer schaut nach unten: tcp +Y -> Welt -Z. yaw = Azimut, tilt = Abweichung von der Senkrechten.
// tilt=0 -> direkt nach unten; tilt>0 -> Greifer neigt sich in yaw-Richtung. (exakt gleiche Konvention wie
// goto_clicked_point.cpp – NICHT literal (1,0,0,0).)

static geometry_msgs::msg::Quaternion gripperOrientation(double yaw, double tilt)
{
    tf2::Quaternion q;

    q.setRPY(-M_PI_2 + tilt, 0.0, yaw);

```

```

    q.normalize();

    return tf2::toMsg(q);
}

static Pose lift(const Pose& p, double dz)
{
    Pose out = p;

    out.position.z += dz;

    return out;
}

static bool cartesianMove(
    MoveGroupInterface& arm, const Pose& target,
    const rclcpp::Logger& log, const std::string& label)
{
    std::vector<Pose> waypoints{ target };

    moveit_msgs::msg::RobotTrajectory traj;

    double fraction = arm.computeCartesianPath(
        waypoints, EEK_STEP, JUMP_THRESHOLD, traj);

    RCLCPP_INFO(log, "[%s] cartesian fraction = %.3f", label.c_str(), fraction);

    if (fraction < MIN_FRACTION) {
        RCLCPP_ERROR(log, "[%s] cartesian Pfad unvollstaendig (%.2f < %.2f) – abgebrochen",
            label.c_str(), fraction, MIN_FRACTION);

        return false;
    }

    if (!arm.execute(traj)) {
        RCLCPP_ERROR(log, "[%s] execute() fehlgeschlagen", label.c_str());

        return false;
    }

    return true;
}

// Versucht die Pre-grasp (Annäherungs-)Pose mit einem yaw+tilt Raster; wendet den ersten planbaren+
// ausführbaren an und gibt die gewählte Ausrichtung über `chosen` zurück. So nutzen descend/retreat

```

```

// die GLEICHE Ausrichtung (die orientation des cartesian Abstiegs bleibt konstant).

//

// Warum: wenn die Schraube armnah/niedrig ist, ist die reine senkrecht-nach-unten (tilt=0) Einzel-Pose
// in der IK nicht kollisionsfrei/limit-innerhalb lösbar (OMPL "unable to sample valid goal states").
// Exakt gleicher stufenweiser Ansatz wie goto_clicked_point.cpp: zuerst direkt nach unten,
// sonst Greifer neigen und yaw drehen.

static bool approachWithFallback(
    MoveGroupInterface& arm, const Pose& pre_pos /*nur position wird genutzt*/,
    const rclcpp::Logger& log, const std::string& label,
    geometry_msgs::msg::Quaternion& chosen)
{
    const double yaw_out = std::atan2(pre_pos.position.y, pre_pos.position.x);
    const std::vector<double> yaws{
        yaw_out, yaw_out + M_PI_2, yaw_out - M_PI_2,
        yaw_out + M_PI, yaw_out + M_PI_4, yaw_out - M_PI_4,
    };

    // Für eine flach liegende Schraube NUR direkt-nach-unten (tilt=0) greifen. Schräge Annäherung
    // (alter 20°/35° Fallback) schob die Finger seitlich weg und verdarb den Griff;
    // außerdem fiel sie, als "Reichweiten-Kompromiss" gewählt, 1-2cm neben die Schraube.
    // Jetzt geht sie entweder mit tilt=0 gerade runter-hält, oder sagt sauber "unerreichbar".
    const std::vector<double> tilts{ 0.0 };

    Pose target = pre_pos;
    MoveGroupInterface::Plan plan;
    for (double tilt : tilts) {
        for (double yaw : yaws) {
            target.orientation = gripperOrientation(yaw, tilt);
            arm.setPoseTarget(target);
            if (arm.plan(plan) == moveit::planning_interface::MoveItErrorCode::SUCCESS) {
                RCLCPP_INFO(log, "[%s] gelöst: yaw=%+.0f° tilt=%+.0f°",
                    label.c_str(), yaw * 180.0 / M_PI, tilt * 180.0 / M_PI);
                if (!arm.execute(plan)) {
                    RCLCPP_ERROR(log, "[%s] execute() fehlgeschlagen", label.c_str());
                    return false;
                }
            }
            chosen = target.orientation;
        }
    }
    return true;
}

```

```

    }

    RCLCPP_INFO(log, " [%s] yaw=%+.0f° tilt=%+.0f° nicht planbar, naechste",

                label.c_str(), yaw * 180.0 / M_PI, tilt * 180.0 / M_PI);

    }

}

RCLCPP_ERROR(log, "[%s] keine yaw/tilt Kombination planbar – Ziel unerreichbar",

            label.c_str());

return false;

}

// Greifer DIREKT an den Controller (FollowJointTrajectory) senden – MoveIt
// collision-aware Plan NICHT verwenden. Grund: im abgesenkten Pose sind die Greiferfinger
// nahe an platform_top → der MoveGroup Greifer-Plan brach sofort mit "start state in collision"
// ab. Die Schließ-/Öffnungsbewegung braucht keine Planung; eine einzelne Gelenk-
// (gripper_controller) Position wird auf den joint_trajectory_controller geschrieben.
// SRDF: open=0.15, closed=-0.74 (URDF Gelenkbereich [-0.74, 0.15]).

static bool setGripper(

    const std::shared_ptr<GripperClient>& client,

    const std::string& named_state, const rclcpp::Logger& log)

{

    const double pos = (named_state == "open") ? 0.15 : -0.74;

    if (!client->wait_for_action_server(std::chrono::seconds(3))) {

        RCLCPP_ERROR(log, "kein Greifer-Action-Server "

                    "(/gripper_controller/follow_joint_trajectory)");

        return false;

    }

    FollowJointTrajectory::Goal goal;

    goal.trajectory.joint_names = {"gripper_controller"};

    trajectory_msgs::msg::JointTrajectoryPoint pt;

    pt.positions = {pos};

    pt.time_from_start = rclcpp::Duration::from_seconds(1.0);

    goal.trajectory.points = {pt};

    auto goal_fut = client->async_send_goal(goal);

    if (goal_fut.wait_for(std::chrono::seconds(3)) != std::future_status::ready) {

```

```

    RCLCPP_ERROR(log, "Greifer-Goal konnte nicht gesendet werden: %s", named_state.c_str());

    return false;
}

auto handle = goal_fut.get();

if (!handle) {

    RCLCPP_ERROR(log, "Greifer-Goal abgelehnt: %s", named_state.c_str());

    return false;
}

auto res_fut = client->async_get_result(handle);

if (res_fut.wait_for(std::chrono::seconds(5)) != std::future_status::ready) {

    RCLCPP_WARN(log, "Greifer-Ergebnis Zeitueberschreitung (trotzdem weiter): %s",

        named_state.c_str());

}

std::this_thread::sleep_for(std::chrono::milliseconds(300)); // mechanisches Settling

RCLCPP_INFO(log, "Greifer -> %s (pos=%.2f) OK", named_state.c_str(), pos);

return true;
}

int main(int argc, char** argv)

{

    rclcpp::init(argc, argv);

    auto node = rclcpp::Node::make_shared(

        "pick_place_cartesian",

        rclcpp::NodeOptions().automatically_declare_parameters_from_overrides(true));

    auto log = node->get_logger();

    // Optionale Parameter (ros2 run ... --ros-args -p grasp_z_offset:=0.005)

    // Da NodeOptions().automatically_declare_parameters_from_overrides(true) aktiv ist,

    // sind die per -p von der Kommandozeile übergebenen Parameter bereits deklariert; ein erneuter

    // declare_parameter Aufruf wirft ParameterAlreadyDeclaredException. Deshalb

    // nur deklarieren, wenn noch nicht deklariert, dann den Wert lesen.

    auto param_d = [&](const std::string& name, double def) {

        if (!node->has_parameter(name)) node->declare_parameter<double>(name, def);

        return node->get_parameter(name).as_double();

    };

    const double grasp_z_offset = param_d("grasp_z_offset", 0.0);

    const double place_x        = param_d("place_x", 0.15);

```

```

const double place_y      = param_d("place_y", 0.15);
const double place_z      = param_d("place_z", 0.05);

// Für die async callbacks von MoveGroupInterface ist ein eigener Executor nötig.
rclcpp::executors::SingleThreadedExecutor executor;

executor.add_node(node);

std::thread spinner([&]() { executor.spin(); });

// --- Schraubenposition vom Detector holen (/vida/target, robot_base Frame) ---
// Die orientation des Detectors ist nur Z-yaw; da wir yaw-frei arbeiten, nutzen wir
// NUR die Position, die Ausrichtung bauen wir selbst mit gripperDown().
geometry_msgs::msg::Point screw;

std::atomic<bool> have_target{false};

auto target_sub = node->create_subscription<PoseStamped>(
    "/vida/target", rclcpp::QoS(1),
    [&](PoseStamped::SharedPtr m) {
        screw = m->pose.position;
        have_target = true;
    });

RCLCPP_INFO(log, "warte auf /vida/target (Detector muss laufen)...");

{
    auto t0 = node->now();
    while (rclcpp::ok() && !have_target &&
        (node->now() - t0).seconds() < TARGET_WAIT_SEC) {
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
    }
}

if (!have_target) {
    RCLCPP_ERROR(log, "innerhalb von %.0f s kein /vida/target gekommen – laeuft der Detector? Abgebrochen.",
        TARGET_WAIT_SEC);
    rclcpp::shutdown(); spinner.join(); return 1;
}

RCLCPP_INFO(log, "Schraubenziel: x=%.3f y=%.3f z=%.3f (z_offset=%.3f)",
    screw.x, screw.y, screw.z, grasp_z_offset);

MoveGroupInterface arm(node, "arm");

```

```

// Der Greifer wird jetzt statt über MoveIt direkt über die Controller-Action gesteuert.
auto gripper_client = rclcpp_action::create_client<FollowJointTrajectory>(
    node, "/gripper_controller/follow_joint_trajectory");

arm.setPoseReferenceFrame("robot_base");
arm.setEndEffectorLink("tcp");
arm.setPlanningTime(5.0);           // constraint-freier joint plan ist schnell; 5s reicht reichlich
arm.setNumPlanningAttempts(10);
arm.setMaxVelocityScalingFactor(VEL_SCALE);
arm.setMaxAccelerationScalingFactor(ACC_SCALE);

RCLCPP_INFO(log, "Planning frame: %s | EE link: %s",
    arm.getPlanningFrame().c_str(),
    arm.getEndEffectorLink().c_str());

// --- Ziel-Posen (robot_base Frame, Meter) ---
// Die Ausrichtung wird hier NICHT FIXIERT: im pre-grasp Plan wird ein yaw+tilt Raster probiert
// (approachWithFallback) und die gewählte Ausrichtung auf descend/retreat übertragen.
Pose pick;
pick.position.x = screw.x;
pick.position.y = screw.y;
pick.position.z = screw.z + grasp_z_offset;

Pose place;
place.position.x = place_x;
place.position.y = place_y;
place.position.z = place_z;

// --- Sequenz ---
RCLCPP_INFO(log, ">> Zur Ready-Position fahren (constraint-frei)");
arm.setNamedTarget("ready");
if (!arm.move()) {
    RCLCPP_ERROR(log, "konnte nicht zu Ready fahren"); rclcpp::shutdown(); spinner.join(); return 1;
}

RCLCPP_INFO(log, ">> Gripper open");
if (!setGripper(gripper_client, "open", log)) { rclcpp::shutdown(); spinner.join(); return 1; }

```



```

// HINWEIS: auf die free-space joint plans wird KEIN orientation PATH CONSTRAINT gelegt.

// Die senkrechte Ausrichtung ist in der Ziel-Pose (gripperDown) bereits vorhanden; die cartesian
// Abstiegs-/Aufstiegs-Segmente halten die Ausrichtung konstant. (Siehe Erklärung am Dateianfang.)
RCLCPP_INFO(log, ">> [PICK] joint plan → pick_pre (yaw+tilt fallback)");

geometry_msgs::msg::Quaternion pick_orient;

if (!approachWithFallback(arm, lift(pick, APPROACH_HEIGHT), log, "pick_pre", pick_orient)) {
    rclcpp::shutdown(); spinner.join(); return 1;
}

pick.orientation = pick_orient; // descend/retreat nutzt die gleiche Ausrichtung

RCLCPP_INFO(log, ">> [PICK] cartesian descend");

if (!cartesianMove(arm, pick, log, "pick_descend")) {
    rclcpp::shutdown(); spinner.join(); return 1;
}

RCLCPP_INFO(log, ">> [PICK] gripper close");

if (!setGripper(gripper_client, "closed", log)) {
    rclcpp::shutdown(); spinner.join(); return 1;
}

RCLCPP_INFO(log, ">> [PICK] cartesian retreat");

if (!cartesianMove(arm, lift(pick, APPROACH_HEIGHT), log, "pick_lift")) {
    rclcpp::shutdown(); spinner.join(); return 1;
}

RCLCPP_INFO(log, ">> [PLACE] joint plan → place_pre (yaw+tilt fallback)");

geometry_msgs::msg::Quaternion place_orient;

if (!approachWithFallback(arm, lift(place, APPROACH_HEIGHT), log, "place_pre", place_orient)) {
    rclcpp::shutdown(); spinner.join(); return 1;
}

place.orientation = place_orient; // descend/retreat nutzt die gleiche Ausrichtung

RCLCPP_INFO(log, ">> [PLACE] cartesian descend");

if (!cartesianMove(arm, place, log, "place_descend")) {
    rclcpp::shutdown(); spinner.join(); return 1;
}

```

```
RCLCPP_INFO(log, ">> [PLACE] gripper open");

if (!setGripper(gripper_client,"open", log)) {

    rclcpp::shutdown(); spinner.join(); return 1;

}


RCLCPP_INFO(log, ">> [PLACE] cartesian retreat");

if (!cartesianMove(arm, lift(place, APPROACH_HEIGHT), log, "place_lift")) {

    rclcpp::shutdown(); spinner.join(); return 1;

}


RCLCPP_INFO(log, ">> Zu Home zurueck");

arm.setNamedTarget("home");

arm.move();


RCLCPP_INFO(log, "Pick & place abgeschlossen.");

rclcpp::shutdown();

spinner.join();

return 0;

}
```

src/mycobot_demo/src/pick_tilt.cpp

```
// myCobot 280 JN – Tilt-bewusster Schrauben-PICK (Hover + Bestätigung + paralleler Abstieg + greifen + heben).  
  
//  
  
// Ablauf (Sitzung 25 Plan – ~/.claude/plans/shimmering-wondering-bachman.md):  
  
//   home (0) → /vida/target warten (Detector gibt VOLLES Quaternion aus; ohne  
//           zuverlässigen Tilt gibt er gar nichts aus → Roboter wartet hier,  
//           Benutzer mischt die Schrauben)  
  
//   → HOVER: vom Greif-Pose entlang der ANNAEHERUNGSACHSE um approach_height zurück, Greifer in  
//           Greif-Ausrichtung, Finger offen. IK-Fallback: Detector q, sonst q•Ry( $\pi$ )  
//           (180° Finger-Flip – Annäherungsachse GLEICH, Griff symmetrisch). Wenn beides scheitert,  
//           "unerreichbar" → 5s warten, mit dem AKTUELLSTEN Ziel erneut versuchen.  
  
//   → /pick/confirm WARTEN (std_msgs/Empty; in cam_viewer 'g'). Ziel wird im Hover-Moment EINGEFROREN.  
//   → PARALLELER ABSTIEG: cartesianMove von hover→grasp (Ausrichtung konstant = parallele Annäherung).  
//   → Greifer SCHLIESSEN → senkrechtes LIFT → HALTEN + warten (Ctrl+C). Kein Place.  
  
//  
// Die /vida/target Orientation des Detectors ist Greif-Frame: tcp +Y = Annäherungsachse.  
// Gerade Schraube → Annäherung ~direkt-nach-unten (top-down); schiefe Schraube → achsenparalleler Abstieg.  
  
  
#include <atomic>  
  
#include <chrono>  
  
#include <functional>  
  
#include <thread>  
  
#include <vector>  
  
  
#include <rclcpp/rclcpp.hpp>  
  
#include <rclcpp_action/rclcpp_action.hpp>  
  
#include <moveit/move_group_interface/move_group_interface.h>  
  
#include <moveit/robot_state/conversions.h>  
  
#include <moveit_msgs/msg/robot_trajectory.hpp>  
  
#include <moveit_msgs/msg/display_trajectory.hpp>  
  
#include <geometry_msgs/msg/pose.hpp>  
  
#include <geometry_msgs/msg/pose_stamped.hpp>  
  
#include <control_msgs/action/follow_joint_trajectory.hpp>  
  
#include <trajectory_msgs/msg/joint_trajectory_point.hpp>  
  
#include <visualization_msgs/msg/marker.hpp>  
  
#include <std_msgs/msg/empty.hpp>
```

```

#include <std_msgs/msg/int32.hpp>

#include <tf2/LinearMath/Quaternion.h>

#include <tf2/LinearMath/Vector3.h>

#include <tf2_geometry_msgs/tf2_geometry_msgs.h>

using moveit::planning_interface::MoveGroupInterface;

using Pose = geometry_msgs::msg::Pose;

using PoseStamped = geometry_msgs::msg::PoseStamped;

using Marker = visualization_msgs::msg::Marker;

using FollowJointTrajectory = control_msgs::action::FollowJointTrajectory;

using GripperClient = rclcpp_action::Client<FollowJointTrajectory>;

namespace
{
constexpr double EEf_STEP          = 0.005; // 5 mm Interpolationsschritt
constexpr double JUMP_THRESHOLD    = 5.0;   // >0: Sprung im Gelenk-Raum ABLEHNEN
constexpr double MIN_FRACTION      = 0.95;   // unter 95% nicht akzeptiert
constexpr double VEL_SCALE         = 0.20;   // Sicherheit: 20% Geschwindigkeit
constexpr double ACC_SCALE         = 0.20;
constexpr double TARGET_WAIT_SEC   = 30.0;   // Warten auf das erste /vida/target

// EXAKT gleiche Reihenfolge wie detection.py CLASS_NAMES (class_id 0..4 = Farbe).
const std::vector<std::string> CLASS_NAMES{"sari", "beyaz", "siyah", "yesil", "kirmizi"};
}

// PLACE Ausrichtungs-Kandidaten (gleiche Logik wie goto_clicked_point): Greifer schaut nach unten,
// yaw = Azimut zum Punkt (+Varianten), tilt = Abweichung von der Senkrechten. Wenn die Boxen an der
// REICHWEITENGRENZE des Roboters sind, gibt das senkrechte Handgelenk (tilt=0) keine IK → stufenweise neigen
// (tilt 20..55°) + yaw drehen. tf2::setRPY(-pi/2+tilt, 0, yaw) = gripperOrientation. Der erste planbare wird
// gewählt.

static std::vector<geometry_msgs::msg::Quaternion> placeOrientations(double x, double y)
{
    const double yaw_out = std::atan2(y, x);

    const std::vector<double> yaws{
        yaw_out, yaw_out + M_PI_2, yaw_out - M_PI_2,
        yaw_out + M_PI, yaw_out + M_PI_4, yaw_out - M_PI_4};

    const std::vector<double> tilts{

```

```

    0.0, 20.0 * M_PI / 180.0, 35.0 * M_PI / 180.0,
    45.0 * M_PI / 180.0, 55.0 * M_PI / 180.0});

std::vector<geometry_msgs::msg::Quaternion> cands;

for (double tilt : tilts) {
    for (double yaw : yaws) {
        tf2::Quaternion q;

        q.setRPY(-M_PI_2 + tilt, 0.0, yaw);

        q.normalize();

        cands.push_back(tf2::toMsg(q));
    }
}

return cands;
}

// Richtung von tcp +Y (Annäherungsachse) in der base aus dem Greif-Quaternion holen.
static tf2::Vector3 approachAxis(const geometry_msgs::msg::Quaternion& q)
{
    tf2::Quaternion tq(q.x, q.y, q.z, q.w);

    return tf2::quatRotate(tq, tf2::Vector3(0.0, 1.0, 0.0));
}

// Greif-Ausrichtung um die tcp +Y (Annäherungs-)Achse um 180° drehen: Finger tauschen
// die Plätze, aber der Griff ist symmetrisch; die Annäherungsachse ÄNDERT SICH NICHT (IK-Alternative).
static geometry_msgs::msg::Quaternion flipAboutApproach(const geometry_msgs::msg::Quaternion& q)
{
    tf2::Quaternion tq(q.x, q.y, q.z, q.w);

    tf2::Quaternion roll; roll.setRotation(tf2::Vector3(0.0, 1.0, 0.0), M_PI);

    tf2::Quaternion out = tq * roll;    // Drehung im lokalen Frame (tcp)

    out.normalize();

    return tf2::toMsg(out);
}

static Pose lift(const Pose& p, double dz)
{
    Pose out = p;

    out.position.z += dz;

    return out;
}

```

```

}

using GateFn = std::function<bool(const std::string&)>;

using DispPub = rclcpp::Publisher<moveit_msgs::msg::DisplayTrajectory>::SharedPtr;

static bool cartesianMove(
    MoveGroupInterface& arm, const Pose& target,
    const rclcpp::Logger& log, const std::string& label,
    const GateFn& gate, const DispPub& disp_pub)
{
    std::vector<Pose> waypoints{ target };
    moveit_msgs::msg::RobotTrajectory traj;
    double fraction = arm.computeCartesianPath(waypoints, EEF_STEP, JUMP_THRESHOLD, traj);
    RCLCPP_INFO(log, "[%s] cartesian fraction = %.3f", label.c_str(), fraction);
    if (fraction < MIN_FRACTION) {
        RCLCPP_ERROR(log, "[%s] cartesian Pfad unvollstaendig (%.2f < %.2f) – abgebrochen",
            label.c_str(), fraction, MIN_FRACTION);
        return false;
    }

    // RViz Vorschau: cartesian Trajektorie als DisplayTrajectory veröffentlichen (move_group
    // gibt cartesian nicht automatisch auf display_planned_path aus – hover/home plan() gibt aus).
    if (disp_pub) {
        moveit_msgs::msg::DisplayTrajectory disp;
        disp.model_id = arm.getRobotModel()->getName();
        disp.trajectory.push_back(traj);
        auto cs = arm.getCurrentState();
        if (cs) moveit::core::robotStateToRobotStateMsg(*cs, disp.trajectory_start);
        disp_pub->publish(disp);
    }

    if (!gate(label)) return false;        // wenn preview_confirm an ist, VOR execute auf Bestätigung warten
    if (!arm.execute(traj)) {
        RCLCPP_ERROR(log, "[%s] execute() fehlgeschlagen", label.c_str());
        return false;
    }

    return true;
}

```

```

// Greifer DIREKT an den Controller senden (MoveIt collision-aware Plan NICHT verwenden).
// SRDF: open=0.15, closed=-0.74 (URDF Gelenkbereich [-0.74, 0.15]).

static bool setGripper(
    const std::shared_ptr<GripperClient>& client,
    const std::string& named_state, const rclcpp::Logger& log)
{
    const double pos = (named_state == "open") ? 0.15 : -0.74;

    if (!client->wait_for_action_server(std::chrono::seconds(3))) {
        RCLCPP_ERROR(log, "kein Greifer-Action-Server"); return false;
    }

    FollowJointTrajectory::Goal goal;

    goal.trajectory.joint_names = {"gripper_controller"};

    trajectory_msgs::msg::JointTrajectoryPoint pt;

    pt.positions = {pos};

    pt.time_from_start = rclcpp::Duration::from_seconds(1.0);

    goal.trajectory.points = {pt};

    auto goal_fut = client->async_send_goal(goal);

    if (goal_fut.wait_for(std::chrono::seconds(3)) != std::future_status::ready) {
        RCLCPP_ERROR(log, "Greifer-Goal konnte nicht gesendet werden"); return false;
    }

    auto handle = goal_fut.get();

    if (!handle) { RCLCPP_ERROR(log, "Greifer-Goal abgelehnt"); return false; }

    auto res_fut = client->async_get_result(handle);

    if (res_fut.wait_for(std::chrono::seconds(5)) != std::future_status::ready) {
        RCLCPP_WARN(log, "Greifer-Ergebnis Zeitueberschreitung (trotzdem weiter)");
    }

    std::this_thread::sleep_for(std::chrono::milliseconds(300));

    RCLCPP_INFO(log, "Greifer -> %s (pos=%.2f) OK", named_state.c_str(), pos);

    return true;
}

// Zum HOVER planen+gehen: in Greif-Ausrichtung, sonst 180° Finger-Flip. Bei Erfolg wird die gewählte
// Ausrichtung in `chosen` geschrieben (descend nutzt die GLEICHE Ausrichtung = paralleler Abstieg).

static bool moveToHover(
    MoveGroupInterface& arm, const Pose& hover,
    const geometry_msgs::msg::Quaternion& grasp_q,
    const rclcpp::Logger& log, geometry_msgs::msg::Quaternion& chosen,

```

```

    const GateFn& gate, const std::string& label = "hover")
{
    const std::vector<geometry_msgs::msg::Quaternion> cands{
        grasp_q, flipAboutApproach(grasp_q)
    };

    Pose target = hover;

    MoveGroupInterface::Plan plan;

    for (size_t i = 0; i < cands.size(); ++i) {
        target.orientation = cands[i];

        arm.setPoseTarget(target);

        // wenn plan() erfolgreich ist, veröffentlicht move_group die Trajektorie auf
        // /move_group/display_planned_path → RViz MotionPlanning zeigt sie → gate wartet vor execute auf Bestätigung.
        if (arm.plan(plan) == moveit::planning_interface::MoveItErrorCode::SUCCESS) {

            RCLCPP_INFO(log, "[%s] geloest (%s Ausrichtung)", label.c_str(), i == 0 ? "Haupt" : "180°-flip");

            if (!gate(label)) return false;          // wenn preview_confirm an ist, auf Bestätigung warten

            if (!arm.execute(plan)) {

                RCLCPP_ERROR(log, "[%s] execute() fehlgeschlagen", label.c_str()); return false;

            }

            chosen = cands[i];

            return true;

        }

        RCLCPP_INFO(log, " [%s] %s Ausrichtung nicht planbar, naechste",
                    label.c_str(), i == 0 ? "Haupt" : "180°-flip");

    }

    RCLCPP_ERROR(log, "[%s] IK nicht loesbar – Ziel unerreichbar", label.c_str());

    return false;
}

// Zum HOVER planen+gehen, KANDIDATEN-AUSRICHTUNGSLISTE der Reihe nach probieren (yaw+tilt
// Suche für PLACE). Der erste planbare wird ausgeführt, die gewählte Ausrichtung in `chosen` geschrieben.
static bool moveToHoverCands(
    MoveGroupInterface& arm, const Pose& hover,
    const std::vector<geometry_msgs::msg::Quaternion>& cands,
    const rclcpp::Logger& log, geometry_msgs::msg::Quaternion& chosen,
    const GateFn& gate, const std::string& label)
{
    Pose target = hover;

```



```

MoveGroupInterface::Plan plan;

for (size_t i = 0; i < cands.size(); ++i) {

    target.orientation = cands[i];

    arm.setPoseTarget(target);

    if (arm.plan(plan) == moveit::planning_interface::MoveItErrorCode::SUCCESS) {

        RCLCPP_INFO(log, "[%s] geloest (Kandidat %zu/%zu)", label.c_str(), i + 1, cands.size());

        if (!gate(label)) return false;

        if (!arm.execute(plan)) {

            RCLCPP_ERROR(log, "[%s] execute() fehlgeschlagen", label.c_str()); return false;

        }

        chosen = cands[i];

        return true;

    }

}

RCLCPP_ERROR(log, "[%s] keiner der %zu Kandidaten geloest – unerreichbar",

            label.c_str(), cands.size());

return false;

}

static Marker makeTargetMarker(const geometry_msgs::msg::Point& p)

{

    Marker m;

    m.header.frame_id = "robot_base";

    m.ns = "pick_tilt_target";

    m.id = 0;

    m.type = Marker::SPHERE;

    m.action = Marker::ADD;

    m.pose.position = p;

    m.pose.orientation.w = 1.0;

    m.scale.x = m.scale.y = m.scale.z = 0.02;

    m.color.g = 1.0; m.color.a = 0.9;

    return m;

}

int main(int argc, char** argv)

{

    rclcpp::init(argc, argv);

```

```

auto node = rclcpp::Node::make_shared(
    "pick_tilt",
    rclcpp::NodeOptions().automatically_declare_parameters_from_overrides(true));

auto log = node->get_logger();

auto param_d = [&](const std::string& name, double def) {
    if (!node->has_parameter(name)) node->declare_parameter<double>(name, def);
    return node->get_parameter(name).as_double();
};

const double approach_height = param_d("approach_height", 0.10); // 5-10 cm
const double grasp_z_offset = param_d("grasp_z_offset", 0.0);
const double descend_vel = param_d("descend_vel_scale", 0.10); // langsamer Abstieg
// rapid traverse: Freiraum-Übergänge OHNE Schraubenkontakt (home, hover) mit dieser
// höheren Geschwindigkeit. Pendant zu CNC G00. Abstieg/greifen/heben NICHT BETROFFEN (bleibt bei
// descend_vel und VEL_SCALE). Vernünftige Obergrenze für Open-Loop-Brücke + Sicherheit; bei 0 oder
// gleich VEL_SCALE altes Verhalten (alles 20%).
const double rapid_vel = param_d("rapid_vel_scale", 0.40);
auto param_b = [&](const std::string& name, bool def) {
    if (!node->has_parameter(name)) node->declare_parameter<bool>(name, def);
    return node->get_parameter(name).as_bool();
};

// preview_confirm: jede Bewegung (home/hover/Abstieg/heben) wird zuerst in RViz vorab angezeigt,
// und WIRD NICHT AUSGEFUEHRT, bis /pick/confirm kommt → der echte Roboter wird zuerst beobachtet.
const bool preview_confirm = param_b("preview_confirm", false);

// --- PLACE (Sortierung nach Farbe) Parameter ---
// place_enabled: nach dem Greifen + Rückkehr zu home die Schraube in die ihrer FARBE gehörende Box
// aus der Luft fallen lassen. Box-Zentren aus dem box_<farbe> Parameter (box_map.yaml, teach_boxes.py).
const bool place_enabled = param_b("place_enabled", true);
// place_hover_z / place_drop_z: ABSOLUTES z über der Box (robot_base, m). Da die Boxen
// an der Reichweitengrenze sind, wird nicht das gespeicherte z der Box, sondern eine feste Höhe über der
// Plattform
// genutzt (z.B. 8 cm Ablage). Ausrichtung wird über yaw+tilt-Suche gefunden (nicht top-down).
const double place_hover_z = param_d("place_hover_z", 0.12); // Annäherung über der Box (absolut)
const double place_drop_z = param_d("place_drop_z", 0.08); // Greifer-Öffnungshöhe (absolut)
// box_<farbe> Parameter lesen (robot_base x,y,z m). Wenn nicht vorhanden, ist diese Farbe nicht eingelernt →
// leer.
std::vector<std::vector<double>> box_xyz(CLASS_NAMES.size());

```

```

for (size_t i = 0; i < CLASS_NAMES.size(); ++i) {

    const std::string pname = "box_" + CLASS_NAMES[i];

    if (node->has_parameter(pname)) {

        auto v = node->get_parameter(pname).as_double_array();

        if (v.size() == 3) box_xyz[i] = v;

    }

}

rclcpp::executors::SingleThreadedExecutor executor;

executor.add_node(node);

std::thread spinner([&]() { executor.spin(); });

auto marker_pub = node->create_publisher<Marker>(

    "/approach/target_marker", rclcpp::QoS(1).transient_local());

// --- /vida/target (VOLLE Pose: Position + Greif-Ausrichtung) ---
PoseStamped target_msg;

std::atomic<bool> have_target{false};

auto target_sub = node->create_subscription<PoseStamped>(

    "/vida/target", rclcpp::QoS(1),

    [&](PoseStamped::SharedPtr m) { target_msg = *m; have_target = true; });

// --- /vida/target_color (Farbklassse der gegriffenen Schraube, synchron mit /vida/target) ---
std::atomic<int> latched_color{-1};

auto color_sub = node->create_subscription<std_msgs::msg::Int32>(

    "/vida/target_color", rclcpp::QoS(1),

    [&](std_msgs::msg::Int32::SharedPtr m) { latched_color = m->data; });

// --- /pick/confirm (Bestätigung im Hover) ---
std::atomic<bool> confirmed{false};

auto confirm_sub = node->create_subscription<std_msgs::msg::Empty>(

    "/pick/confirm", rclcpp::QoS(1),

    [&](std_msgs::msg::Empty::SharedPtr) { confirmed = true; });

// Cartesian Vorschau-Ausgabe (RViz MotionPlanning "Planned Path" liest von hier).
auto disp_pub = node->create_publisher<moveit_msgs::msg::DisplayTrajectory>(

    "/move_group/display_planned_path", rclcpp::QoS(1));

```

```

// wenn preview_confirm an ist: geplante Bewegung in RViz zeigen + auf ENTER-Bestätigung warten.

// wenn aus, sofort true zurückgeben (altes Verhalten). Ctrl+C → false (abbrechen).

GateFn gate = [&](const std::string& label) -> bool {

    if (!preview_confirm) return rclcpp::ok();

    confirmed = false;

    RCLCPP_INFO(log, ">> VORSCHAU [%s]: Trajektorie in RViz pruefen → ENTER = BESTAETIGUNG "
        "(ausfuehren). Ctrl+C = abbrechen.", label.c_str());

    while (rclcpp::ok() && !confirmed)

        std::this_thread::sleep_for(std::chrono::milliseconds(100));

    if (rclcpp::ok()) RCLCPP_INFO(log, "✓ [%s] bestaetigt – wird ausgefuehrt.", label.c_str());

    return rclcpp::ok();

};

MoveGroupInterface arm(node, "arm");

auto gripper_client = rclcpp_action::create_client<FollowJointTrajectory>(
    node, "/gripper_controller/follow_joint_trajectory");

arm.setPoseReferenceFrame("robot_base");

arm.setEndEffectorLink("tcp");

arm.setPlanningTime(5.0);

arm.setNumPlanningAttempts(10);

arm.setMaxVelocityScalingFactor(rapid_vel);    // home = Freiraum rapid traverse

arm.setMaxAccelerationScalingFactor(rapid_vel);

RCLCPP_INFO(log, "Geschwindigkeitsprofil: rapid(Freiraum)=%.0f%% | Abstieg=%.0f%% | greifen/heben=%.0f%%",
    rapid_vel * 100.0, descend_vel * 100.0, VEL_SCALE * 100.0);

// --- SCHRITT 1: home (0) – plan→(Vorschau+Bestätigung)→execute ---

RCLCPP_INFO(log, ">> SCHRITT 1: home (Punkt 0)");

arm.setNamedTarget("home");

{

    MoveGroupInterface::Plan home_plan;

    if (arm.plan(home_plan) != moveit::planning_interface::MoveItErrorCode::SUCCESS) {

        RCLCPP_ERROR(log, "home nicht planbar"); rclcpp::shutdown(); spinner.join(); return 1;

    }

    if (!gate("home")) { rclcpp::shutdown(); spinner.join(); return 0; }

    if (!arm.execute(home_plan)) {

        RCLCPP_ERROR(log, "konnte nicht zu home fahren"); rclcpp::shutdown(); spinner.join(); return 1;

    }

}

```

```

    }
}

RCLCPP_INFO(log, ">> Greifer oeffnen (pre-grasp)");
setGripper(gripper_client, "open", log);

// --- Auf erstes Ziel warten (UNBEGRENZT) ---
// Wenn der Detector keinen zuverlässigen Tilt findet, kommt /vida/target nie → Roboter wartet bei home,
// Benutzer mischt die Schrauben. Bei gewünschtem Beenden Ctrl+C. (TARGET_WAIT_SEC ist nur
// die Periode der "noch nicht gekommen" Warnung.)
RCLCPP_INFO(log, "warte auf /vida/target – Detector muss ZUVERLAESSIGEN Tilt finden "
               "(sonst gibt er nichts aus). Wenn kein Ziel, Schrauben mischen. Ctrl+C = Beenden.");
{
    auto t0 = node->now();
    while (rclcpp::ok() && !have_target) {
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        if ((node->now() - t0).seconds() >= TARGET_WAIT_SEC) {
            RCLCPP_WARN(log, "immer noch kein /vida/target – ist Detector/Kamera an? "
                          "fuer zuverlaessigen Tilt Schrauben deutlich schraeg legen/mischen.");
            t0 = node->now();
        }
    }
}

if (!rclcpp::ok()) { rclcpp::shutdown(); spinner.join(); return 0; }

// --- SCHRITT 2: hover probieren BIS ERREICHT; bei Erfolg auf Bestätigung warten, dann absteigen+greifen ---
bool picked = false;
int color_at_pick = -1; // Farbklasse des Gegriffenen (bei Hover-Erfolg eingefroren → PLACE)
int hover_fails = 0;    // bei jedem 5. aufeinanderfolgenden Fehlschlag zu home zurück, Pose auffrischen
int descend_fails = 0;  // bei 3 Abstiegs-Fehlschlägen aufgeben (Endlosschleife vermeiden)
while (rclcpp::ok() && !picked) {
    // hover + home-recovery = Freiraum → rapid traverse. (descend reduziert unten auf
    // descend_vel, vor lift zurück auf VEL_SCALE.)
    arm.setMaxVelocityScalingFactor(rapid_vel);
    arm.setMaxAccelerationScalingFactor(rapid_vel);
    PoseStamped t = target_msg;           // AKTUELLSTES Ziel (Snapshot)
    Pose grasp;

```

```

grasp.position = t.pose.position;

grasp.position.z += grasp_z_offset;

grasp.orientation = t.pose.orientation;    // Detector Greif-Frame-Ausrichtung

marker_pub->publish(makeTargetMarker(grasp.position));

const double r = std::hypot(grasp.position.x, grasp.position.y) * 1000.0;

// Hover = vom Greif-Pose entlang der Annäherungsachse um approach_height ZURUECK.
tf2::Vector3 ay = approachAxis(grasp.orientation);    // Einheit (Annäherungsrichtung)

Pose hover = grasp;

hover.position.x -= ay.x() * approach_height;
hover.position.y -= ay.y() * approach_height;
hover.position.z -= ay.z() * approach_height;

RCLCPP_INFO(log, ">> SCHRITT 2: HOVER (%.0fcm zurueck von grasp, entlang der Annaeherungsachse) "
              "| grasp r=%.0fmm z=%.3f", approach_height * 100.0, r, grasp.position.z);

geometry_msgs::msg::Quaternion chosen;

if (!moveToHover(arm, hover, grasp.orientation, log, chosen, gate)) {
    ++hover_fails;

    RCLCPP_WARN(log, "⚠ Ziel UNERREICHBAR (r=%.0fmm, %d. in Folge). Schraube NAEHER an den Roboter "
                  "legen (GRUENER Kreis in der Kamera). In 5s erneut mit AKTUELLSTEM Ziel...",
                r, hover_fails);

    if (hover_fails % 5 == 0) {
        RCLCPP_WARN(log, "5 aufeinanderfolgende Hover-Fehlschlaege – Rueckkehr zu home (Pose auffrischen)");
        arm.setNamedTarget("home"); arm.move();
    }

    for (int k = 0; k < 50 && rclcpp::ok(); ++k)
        std::this_thread::sleep_for(std::chrono::milliseconds(100));

    continue;
}

hover_fails = 0;

color_at_pick = latched_color.load();    // Farbe dieses Ziels einfrieren (PLACE nutzt das)

grasp.orientation = chosen;    // descend GLEICHE Ausrichtung = paralleler Abstieg

// --- SCHRITT 3: im HOVER auf Bestätigung warten – NUR wenn preview_confirm AUS ist.
// wenn preview_confirm an ist, wurde hover bereits über gate bestätigt, und der Abstieg unten
// über ein eigenes gate bestätigt → diese alte Einzel-Bestätigung wird übersprungen (keine Doppel-

```

Bestätigung).

```
    if (!preview_confirm) {

        confirmed = false;

        RCLCPP_INFO(log, ">> SCHRITT 3: im HOVER auf Bestaetigung warten – im cam_viewer-Fenster 'g' "
            "(oder: ros2 topic pub --once /pick/confirm std_msgs/msg/Empty '{}')");

        while (rclcpp::ok() && !confirmed) {

            marker_pub->publish(makeTargetMarker(grasp.position));

            std::this_thread::sleep_for(std::chrono::milliseconds(100));

        }

        if (!rclcpp::ok()) break;

        RCLCPP_INFO(log, "✓ Bestaetigung erhalten – Abstieg.");

    }

    // --- SCHRITT 4: paralleler Abstieg (Ausrichtung konstant) – LANGSAME Geschwindigkeit + (preview)
    // Vorschau+Bestätigung ---

    arm.setMaxVelocityScalingFactor(descend_vel);

    arm.setMaxAccelerationScalingFactor(descend_vel);

    bool descend_ok = cartesianMove(arm, grasp, log, "descend", gate, disp_pub);

    arm.setMaxVelocityScalingFactor(VEL_SCALE);

    arm.setMaxAccelerationScalingFactor(ACC_SCALE);

    if (!descend_ok) {

        if (!rclcpp::ok()) break;

        ++descend_fails;

        RCLCPP_ERROR(log, "Abstieg fehlgeschlagen (%d/3) – zu home zurueck, erneut versuchen", descend_fails);

        arm.setNamedTarget("home"); arm.move();

        if (descend_fails >= 3) {

            RCLCPP_FATAL(log, "3 Abstiegs-Fehlschlaege – AUFGEGEBEN. Schraube/Szene anpassen und "
                "Skript neu starten.");

            break;

        }

        continue;

    }

    // --- SCHRITT 5: greifen ---

    RCLCPP_INFO(log, ">> SCHRITT 5: Greifer schliessen (greifen)");

    setGripper(gripper_client, "closed", log);
```

```

// --- SCHRITT 5b: GREIF-BESTAETIGUNG (VOR dem Heben warten) ---

// Der Roboter STEHT mit gehaltener Schraube im Greif-Pose; der Benutzer prüft per Kamera, ob die Schraube
// wirklich gegriffen wurde, und lässt sie bei korrekt mit 'g' (/pick/confirm)
// heben. Wenn nicht gegriffen, Ctrl+C (hebt nicht). Dies ist die zweite, separate Bestätigung.
confirmed = false;

RCLCPP_INFO(log, ">> SCHRITT 5b: warte auf GREIF-Bestaetigung – wurde die Schraube gehalten? "
               "in cam_viewer 'g' = heben. Wenn nicht gegriffen, Ctrl+C.");

while (rclcpp::ok() && !confirmed) {
    marker_pub->publish(makeTargetMarker(grasp.position));

    std::this_thread::sleep_for(std::chrono::milliseconds(100));
}

if (!rclcpp::ok()) break;

RCLCPP_INFO(log, "✓ Greif-Bestaetigung erhalten – heben.");


// --- SCHRITT 6: senkrecht heben ---

RCLCPP_INFO(log, ">> SCHRITT 6: senkrecht heben (%.0fcm)", approach_height * 100.0);

if (!cartesianMove(arm, lift(grasp, approach_height), log, "lift", gate, disp_pub)) {
    RCLCPP_WARN(log, "Heben cartesian unvollstaendig/abgebrochen – trotzdem weiter halten");
}


// --- SCHRITT 7: Schraube HALTEND DIREKT + SCHNELL zum Null-(home-)Punkt zurück (rabbit) ---

// Freiraum, kein Schraubenkontakt → KEINE Bestätigung/Vorschau, direkt mit rapid Geschwindigkeit.
// (Bei langsamer Fahrt zittern die Servos durch stick-slip; schnell = glatt.)

RCLCPP_INFO(log, ">> SCHRITT 7: Schraube wird gehalten – schnell zum NULL-(home-)Punkt zurueck (%.0f%%)",
               rapid_vel * 100.0);

arm.setMaxVelocityScalingFactor(rapid_vel);
arm.setMaxAccelerationScalingFactor(rapid_vel);
arm.setNamedTarget("home");

{
    MoveGroupInterface::Plan home_plan;

    if (arm.plan(home_plan) == moveit::planning_interface::MoveItErrorCode::SUCCESS) {
        arm.execute(home_plan);    // direkt – kein gate
    } else {
        RCLCPP_WARN(log, "Rueckkehr zu Null nicht planbar – Schraube wird im Greif-Pose gehalten");
    }
}

RCLCPP_INFO(log, "✓ Schraube gegriffen, am NULL-Punkt – jetzt PLACE (Box nach Farbe).");

```



```

// --- SCHRITT 8: PLACE – Schraube über die ihrer FARBE gehörende Box bringen, aus der Luft fallen lassen ---

// Farbe = das beim Pick eingefrorene color_at_pick. Box-Zentrum = box_<farbe> (teach).

// "Aus der Luft fallen lassen": kein senkrecht Drehen/in-Loch-Setzen – über die Box absteigen, Greifer
öffnen.

const std::string color_name =

    (color_at_pick >= 0 && color_at_pick < (int)CLASS_NAMES.size())

    ? CLASS_NAMES[color_at_pick] : std::string("");

const bool have_box = (color_at_pick >= 0 &&

    color_at_pick < (int)box_xyz.size() &&

    !box_xyz[color_at_pick].empty());

if (!place_enabled) {

    RCLCPP_INFO(log, "place_enabled=false – Schraube wird bei NULL gehalten (Ctrl+C).");

} else if (!have_box) {

    RCLCPP_WARN(log, "⚠ PLACE uebersprungen: keine Box fuer Farbe='%s' (id=%d). "

        "Wurde box_map.yaml eingelernt (teach_boxes.py)? Schraube wird bei NULL gehalten.",

        color_name.c_str(), color_at_pick);

} else {

    const auto& b = box_xyz[color_at_pick];

    RCLCPP_INFO(log, ">> SCHRITT 8: PLACE – Box der Farbe='%s' (x=%.3f y=%.3f z=%.3f)",

        color_name.c_str(), b[0], b[1], b[2]);

    // HOVER über der Box (Freiraum, Schraube in der Luft → rapid). ABSOLUTES z, yaw+tilt-Suche

    // (Boxen an der Reichweitengrenze → top-down nicht lösbar, schräge Ausrichtung nötig).

    arm.setMaxVelocityScalingFactor(rapid_vel);

    arm.setMaxAccelerationScalingFactor(rapid_vel);

    Pose place_hover;

    place_hover.position.x = b[0];

    place_hover.position.y = b[1];

    place_hover.position.z = place_hover_z;          // ABSOLUT (ab Plattform)

    geometry_msgs::msg::Point bp; bp.x = b[0]; bp.y = b[1]; bp.z = place_drop_z;

    marker_pub->publish(makeTargetMarker(bp));

    const auto place_cands = placeOrientations(b[0], b[1]);

    geometry_msgs::msg::Quaternion place_q;

    if (!moveToHoverCands(arm, place_hover, place_cands, log, place_q, gate, "place-hover")) {

```

```

RCLCPP_WARN(log, "\u00a0 keine IK ueber der Box (kein yaw/tilt) – Schraube wird gehalten, zu NULL zurueck.");

arm.setNamedTarget("home"); arm.move();

} else {

    // Kurzer Abstieg in die Box hinein (noch in der Luft, kein Kontakt) – kontrollierte Geschwindigkeit.
    arm.setMaxVelocityScalingFactor(VEL_SCALE);
    arm.setMaxAccelerationScalingFactor(VEL_SCALE);
    Pose drop = place_hover;
    drop.position.z = place_drop_z;           // ABSOLUTE Ablagehöhe (z.B. 8cm)
    drop.orientation = place_q;
    if (!cartesianMove(arm, drop, log, "place-descend", gate, disp_pub)) {
        RCLCPP_WARN(log, "place Abstieg unvollstaendig/abgebrochen – wird aus aktueller Hoehe abgelegt");
        drop = place_hover;    // aus der Hover-Höhe ablegen
    }

    // ABLEGEN: Greifer öffnen → Schraube fällt in die Box.
    RCLCPP_INFO(log, ">> SCHRITT 8b: Greifer oeffnen – Schraube wird in Box '%s' abgelegt",
        color_name.c_str());
    setGripper(gripper_client, "open", log);

    // Senkrecht zurückziehen (rapid) – dann zu NULL zurück.
    arm.setMaxVelocityScalingFactor(rapid_vel);
    arm.setMaxAccelerationScalingFactor(rapid_vel);
    cartesianMove(arm, lift(drop, place_hover_z - place_drop_z),
        log, "place-retract", gate, disp_pub);
}

}

// --- Zu NULL zurück (Finger offen oder leer – freie Rückkehr) ---
arm.setMaxVelocityScalingFactor(rapid_vel);
arm.setMaxAccelerationScalingFactor(rapid_vel);
arm.setNamedTarget("home");
{
    MoveGroupInterface::Plan home_plan;
    if (arm.plan(home_plan) == moveit::planning_interface::MoveItErrorCode::SUCCESS)
        arm.execute(home_plan);
}

RCLCPP_INFO(log, "    FERTIG – Schraube in Box '%s' abgelegt, Roboter bei NULL (Ctrl+C).",

```

```
        color_name.c_str());

    picked = true;
}

// Halten + warten (bis Ctrl+C).
rclcpp::Rate rate(2.0);
while (rclcpp::ok()) {
    rate.sleep();
}

rclcpp::shutdown();
spinner.join();
return 0;
}
```

src/mycobot_hardware/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)

project(mycobot_hardware LANGUAGES CXX)

if(NOT CMAKE_CXX_STANDARD)
  set(CMAKE_CXX_STANDARD 17)
  set(CMAKE_CXX_STANDARD_REQUIRED ON)
endif()

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

find_package(ament_cmake REQUIRED)
find_package(ament_index_cpp REQUIRED)
find_package(hardware_interface REQUIRED)
find_package(pluginlib REQUIRED)
find_package(rclcpp REQUIRED)
find_package(rclcpp_lifecycle REQUIRED)

add_library(${PROJECT_NAME} SHARED
  src/mycobot_hardware.cpp
)

target_include_directories(${PROJECT_NAME} PUBLIC
  $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
  $<INSTALL_INTERFACE:include>
)

ament_target_dependencies(${PROJECT_NAME}
  ament_index_cpp
  hardware_interface
  pluginlib
  rclcpp
  rclcpp_lifecycle
)
```

```
pluginlib_export_plugin_description_file(hardware_interface mycobot_hardware_plugin.xml)
```

```
install(  
  DIRECTORY include/  
  DESTINATION include  
)
```

```
install(  
  TARGETS ${PROJECT_NAME}  
  EXPORT export_${PROJECT_NAME}  
  ARCHIVE DESTINATION lib  
  LIBRARY DESTINATION lib  
  RUNTIME DESTINATION bin  
)
```

```
install(FILES mycobot_hardware_plugin.xml  
  DESTINATION share/${PROJECT_NAME}  
)
```

```
# Bridge script – located by ament_index at runtime.
```

```
# estop_relay: NOT-AUS-Relay (/estop-Topic -> Bridge-Kommando-Socket).
```

```
install(PROGRAMS  
  scripts/mycobot_bridge.py  
  scripts/estop_relay.py  
  DESTINATION lib/${PROJECT_NAME}  
)
```

```
ament_export_include_directories(include)
```

```
ament_export_libraries(${PROJECT_NAME})
```

```
ament_export_dependencies(
```

```
  ament_index_cpp  
  hardware_interface  
  pluginlib  
  rclcpp  
  rclcpp_lifecycle
```

```
)
```

ament_package()

src/mycobot_hardware/include/mycobot_hardware/mycobot_hardware.hpp

```
// myCobot 280 JN ros2_control SystemInterface.

//

// We do NOT embed Python here – the Galactic pybind11 vendor (2.2.7) segfaults
// when worker threads share the GIL with the main thread (see
// feedback_pybind_galactic_threading memory). Instead we fork a Python
// subprocess (mycobot_bridge.py) that owns pymycobot, and talk to it over a
// Unix socket. The C++ side stays pure POSIX.

#pragma once

#include <atomic>

#include <memory>

#include <mutex>

#include <string>

#include <thread>

#include <vector>

#include <sys/types.h>

#include "hardware_interface/system_interface.hpp"

#include "hardware_interface/handle.hpp"

#include "hardware_interface/hardware_info.hpp"

#include "hardware_interface/types/hardware_interface_return_values.hpp"

#include "hardware_interface/types/hardware_interface_type_values.hpp"

#include "rclcpp/macros.hpp"

#include "rclcpp/time.hpp"

#include "rclcpp_lifecycle/state.hpp"

namespace mycobot_hardware

{

class MyCobotHardware : public hardware_interface::SystemInterface

{

public:

    RCLCPP_SHARED_PTR_DEFINITIONS(MyCobotHardware)
```

```

::CallbackReturn on_init(const hardware_interface::HardwareInfo & info) override;

::CallbackReturn on_configure(const rclcpp_lifecycle::State & previous_state) override;

::CallbackReturn on_activate(const rclcpp_lifecycle::State & previous_state) override;

::CallbackReturn on_deactivate(const rclcpp_lifecycle::State & previous_state) override;

::CallbackReturn on_cleanup(const rclcpp_lifecycle::State & previous_state) override;


std::vector<hardware_interface::StateInterface> export_state_interfaces() override;

std::vector<hardware_interface::CommandInterface> export_command_interfaces() override;


hardware_interface::return_type read() override;

hardware_interface::return_type write() override;


private:

    static constexpr size_t kArmDof = 6;

    static constexpr size_t kGripperIdx = 6;


    // ros2_control storage (7 joints: 6 arm + 1 gripper)

    std::vector<double> hw_positions_;

    std::vector<double> hw_velocities_;

    std::vector<double> hw_position_cmds_;

    std::vector<double> last_sent_cmds_;


    // Parameters

    std::string port_;

    int baud_ = 1000000;

    int command_speed_ = 30;

    double write_period_s_ = 0.05;

    double change_threshold_rad_ = 0.001;

    std::string socket_path_;

    std::string bridge_script_;    // absolute path to mycobot_bridge.py


    // Subprocess + socket

    pid_t bridge_pid_ = -1;

    int sock_fd_ = -1;

    std::atomic<bool> bridge_ready_{false};

```



```

// Read thread (just drains the socket, no Python in this process)

std::thread read_thread_;

std::atomic<bool> read_running_{false};

std::mutex state_mutex_;

std::vector<double> latest_arm_;

double latest_gripper_ = 0.0;

std::atomic<bool> latest_fresh_{false};


// Timing

rclcpp::Time last_write_;

rclcpp::Time last_read_;


// Helpers

bool start_bridge();

bool connect_socket();

void stop_bridge();

void read_loop();

bool send_line(const std::string & line);

};

} // namespace mycobot_hardware

```

src/mycobot_hardware/mycobot_hardware_plugin.xml

```
<library path="mycobot_hardware">

  <class

    name="mycobot_hardware/MyCobotHardware"

    type="mycobot_hardware::MyCobotHardware"

    base_class_type="hardware_interface::SystemInterface">

    <description>

      myCobot 280 JN ros2_control SystemInterface – pymycobot via pybind11.

    </description>

  </class>

</library>
```

src/mycobot_hardware/package.xml

```
<?xml version="1.0"?>

<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>

<package format="3">

  <name>mycobot_hardware</name>

  <version>0.1.0</version>

  <description>

    ros2_control SystemInterface for myCobot 280 JN.

    Bridges to pymycobot via embedded Python (pybind11).

  </description>

  <maintainer email="ekinciorhandijvar@gmail.com">Orhan</maintainer>

  <license>MIT</license>


  <buildtool_depend>ament_cmake</buildtool_depend>


  <depend>ament_index_cpp</depend>

  <depend>hardware_interface</depend>

  <depend>pluginlib</depend>

  <depend>rclcpp</depend>

  <depend>rclcpp_lifecycle</depend>


  <exec_depend>python3</exec_depend>

  <!-- pymycobot installed via pip; declared informally. -->

  <exec_depend>python3-pymycobot-pip</exec_depend>


  <export>

    <build_type>ament_cmake</build_type>

    <hardware_interface plugin="${prefix}/mycobot_hardware_plugin.xml"/>

  </export>

</package>
```

src/mycobot_hardware/scripts/estop_relay.py

```
#!/usr/bin/env python3

# NOT-AUS-Relay – übersetzt das ROS-Topic /estop in das "estop"-Wire-Kommando
# des mycobot_bridge-Prozesses (Unix-Kommando-Socket).

#

# Hintergrund: mycobot_bridge.py ist BEWUSST KEIN ROS-Node (pybind11 2.2.7 auf
# Galactic segfaultet bei geteiltem GIL mit Hintergrund-Threads, siehe Docstring
# dort). Damit der Not-Aus-Taster (estop_button, publiziert /estop) trotzdem den
# echten Roboter stoppt, abonniert DIESER kleine Node das Topic und leitet den
# Zustand als einzeliges Kommando an den Kommando-Socket des Bridge weiter.

#

# Eigenschaften:

# * /estop ist transient_local (gelatcht) – beim Start übernimmt der Relay
#   sofort den letzten Zustand (z.B. wenn der Taster früher gedrückt wurde).
# * Heartbeat: solange NOT-AUS aktiv ist, wird "estop 1" alle 2 s erneut
#   gesendet – ein neu gestarteter Bridge-Prozess (Stack-Neustart) fällt so
#   nicht in den freigegebenen Zustand zurück.
# * Läuft strikt einfädig (rclpy.spin im Hauptthread, kein Executor-Thread)
#   – vermeidet die Galactic-pybind11-Threading-Falle.

#

# Start (normalerweise automatisch über demo.launch.py bei echter Hardware):

#   ros2 run mycobot_hardware estop_relay.py


import os

import socket

import rclpy

from rclpy.node import Node

from rclpy.qos import QoSProfile, DurabilityPolicy, ReliabilityPolicy

from std_msgs.msg import Bool


# Kommando-Socket des Bridge – gleiche Konvention wie mycobot_bridge.py
# (--socket Basis-Pfad + ".cmd"). Per Umgebungsvariable überschreibbar.

CMD_SOCKET = os.environ.get('MYCOBOT_BRIDGE_SOCKET', '/tmp/mycobot_bridge.sock') + '.cmd'
```

```

class EstopRelay(Node):

    def __init__(self):

        super().__init__('estop_relay')

        # QoS MUSS zum estop_button passen: transient_local + reliable, depth 1.

        q = QoSProfile(depth=1)

        q.durability = DurabilityPolicy.TRANSIENT_LOCAL

        q.reliability = ReliabilityPolicy.RELIABLE

        self._active = False

        self._fail_logged = False    # Socket-Fehler nur einmal loggen (kein Log-Spam)

        self.create_subscription(Bool, '/estop', self._on_estop, q)

        # Ack an den Taster: das Kommando wurde WIRKLICH auf den Bridge-Socket

        # geschrieben (Vorfall 2026-07-16: Taster hatte keine Rueckmeldung, ob der

        # Stopp irgendwo ankommt). estop_button abonniert /estop_ack und zeigt es an.

        self._pub_ack = self.create_publisher(Bool, '/estop_ack', q)

        self.create_timer(2.0, self._tick)

        self.get_logger().info(f'NOT-AUS-Relay bereit: /estop -> {CMD_SOCKET}')

    def _send(self, active):

        """Zustand als Wire-Kommando an den Bridge-Socket schicken (einmalige Verbindung)."""

        try:

            s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)

            s.settimeout(1.0)

            s.connect(CMD_SOCKET)

            s.sendall(b'estop 1\n' if active else b'estop 0\n')

            s.close()

            self._fail_logged = False

            return True

        except OSError as e:

            # Bridge (noch) nicht da – z.B. fake hardware oder Stack startet gerade.

            if not self._fail_logged:

                self.get_logger().warn(f'Bridge-Socket nicht erreichbar ({e}) – es wird weiter versucht')

                self._fail_logged = True

            return False

    def _on_estop(self, msg):

        self._active = bool(msg.data)

        ok = self._send(self._active)

```

```

text = 'NOT-AUS -> STOP' if self._active else 'NOT-AUS -> FREIGABE'

if ok:

    self.get_logger().warn(text)

    self._pub_ack.publish(Bool(data=self._active))    # Zustellung bestaetigen

else:

    self.get_logger().error(text + ' - Bridge-Socket NICHT erreichbar!')


def _tick(self):

    # Heartbeat nur im STOP-Zustand - Freigabe wird nur bei Zustandswechsel gesendet.

    if self._active:

        self._send(True)


def main():

    rclpy.init()

    node = EstopRelay()

    try:

        rclpy.spin(node)

    except KeyboardInterrupt:

        pass

    node.destroy_node()

    rclpy.shutdown()


if __name__ == '__main__':

    main()

```

src/mycobot_hardware/scripts/mycobot_bridge.py

```
#!/usr/bin/env python3

"""myCobot 280 JN bridge – owns pymycobot, exposes state/commands over Unix socket.

Design: a single Python process holds the only `pymycobot.MyCobot280` instance.
All serial I/O happens here, in one worker thread. A separate accept loop
handles a single client (the C++ ros2_control plugin) via line-delimited JSON.

This isolates the C++ ros2_control_node from embedded-Python threading bugs
(pybind11 2.2.7 on Galactic segfaults when GIL is shared with bg threads).

Protocol – newline-delimited, space-separated, both directions:

bridge → client (state, ~20 Hz):

    state <ts> <r1> <r2> <r3> <r4> <r5> <r6> <grip|-1>

client → bridge (commands):

    send_radians <r1> <r2> <r3> <r4> <r5> <r6> <speed>
    set_gripper <value> <speed>
    power_on
    release_all
    shutdown
    estop <0|1>    # NOT-AUS: 1 = mc.stop() + Bewegungs-Kommandos blockieren, 0 = Freigabe
"""

import argparse
import os
import queue
import signal
import socket
import sys
import threading
import time

from pymycobot import MyCobot280
```

```

def log(msg):
    print(f'[mycobot_bridge] {msg}', file=sys.stderr, flush=True)

class Bridge:
    def __init__(self, mc, speed, rate_hz, socket_path):
        self.mc = mc
        self.default_speed = speed
        self.period = 1.0 / rate_hz
        self.socket_path = socket_path
        self.cmd_socket_path = socket_path + '.cmd'
        self.cmd_queue: queue.Queue = queue.Queue(maxsize=32)
        self.client_lock = threading.Lock()
        self.client_conn = None
        self.stop_event = threading.Event()
        # Greifer Open-Loop Reserve: wenn get_gripper_value() bei dieser Firmware oft
        # 255/-1 (ungültig) zurückgibt, melden wir den zuletzt kommandierten Wert.
        self.last_gripper_cmd = None
        self._grip_log_counter = 0
        # NOT-AUS-Zustand: solange True werden send_radians/set_gripper/release_all
        # verworfen. Gesetzt/gelöscht über das Wire-Kommando "estop <0|1>"
        # (kommt vom estop_relay-Node, der das /estop-Topic abonniert).
        self.estop = False

    def run(self):
        try:
            os.unlink(self.socket_path)
        except FileNotFoundError:
            pass

        worker = threading.Thread(target=self._worker, daemon=True)
        worker.start()

        # Kommando-Socket – jeder kann schreiben, legt in cmd_queue ab
        cmd_thread = threading.Thread(target=self._cmd_listener, daemon=True)
        cmd_thread.start()

```



```
srv = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)

srv.bind(self.socket_path)

os.chmod(self.socket_path, 0o666)

srv.listen(1)

srv.settimeout(0.5)

log(f'listening on {self.socket_path}')
```

```
try:

    while not self.stop_event.is_set():

        try:

            conn, _ = srv.accept()

        except socket.timeout:

            continue

        log('client connected')

        self._handle_client(conn)

        log('client disconnected')
```

```
except KeyboardInterrupt:
```

```
    pass
```

```
finally:
```

```
    self.stop_event.set()
```

```
    srv.close()
```

```
    try:
```

```
        os.unlink(self.socket_path)
```

```
    except FileNotFoundError:
```

```
        pass
```

```
    try:
```

```
        os.unlink(self.cmd_socket_path)
```

```
    except FileNotFoundError:
```

```
        pass
```

```
    worker.join(timeout=2.0)
```

```
    try:
```

```
        self.mc.release_all_servos()
```

```
    except Exception:
```

```
        pass
```

```
def _enqueue(self, line):
```

""Einheitlicher Kommando-Eingang (beide Sockets laufen hier durch).

NOT-AUS wird VORGEZOGEN: das Flag wird sofort im Netz-Thread gesetzt und die Warteschlange geleert, damit kein bereits eingereihtes Bewegungs-Kommando nach dem Stopp "nachläuft". Das eigentliche mc.stop() führt der Worker aus (einziger Serial-Thread, nächster Zyklus, <~100 ms). Während aktivem NOT-AUS werden Bewegungs-Kommandos hier verworfen.

"""

```
if not line:

    return

if line.startswith('estop'):

    parts = line.split()

    active = len(parts) > 1 and parts[1] not in ('0', 'false', 'False')

    self.estop = active

    # Warteschlange leeren – bei STOP darf nichts nachlaufen, bei
    # FREIGABE ist sie ohnehin nur mit Nicht-Bewegungs-Resten gefüllt.

    try:

        while True:

            self.cmd_queue.get_nowait()

    except queue.Empty:

        pass

    self.cmd_queue.put('estop 1' if active else 'estop 0')

    return

if self.estop and line.split()[0] in ('send_radians', 'set_gripper', 'release_all'):

    log(f'NOT-AUS aktiv – Kommando verworfen: {line.split()[0]}')

    return

try:

    self.cmd_queue.put_nowait(line)

except queue.Full:

    log('WARN cmd_queue full, dropping')

def _handle_client(self, conn):

    with self.client_lock:

        if self.client_conn is not None:

            try:

                self.client_conn.close()

            except OSError:
```

```

        pass

    self.client_conn = conn

buf = b''
conn.settimeout(0.5)
try:
    while not self.stop_event.is_set():
        try:
            chunk = conn.recv(4096)

        except socket.timeout:
            continue

        if not chunk:
            break

        buf += chunk

        while b'\n' in buf:
            line, buf = buf.split(b'\n', 1)

            line = line.strip()

            if not line:
                continue

            self._enqueue(line.decode('utf-8'))

except OSError as e:
    log(f'client recv error: {e}')

finally:
    with self.client_lock:
        if self.client_conn is conn:
            self.client_conn = None

        try:
            conn.close()

        except OSError:
            pass

def _cmd_listener(self):
    """Separater Kommando-Socket – empfängt Kommandos von einmaligen Verbindungen."""

    try:
        os.unlink(self.cmd_socket_path)

    except FileNotFoundError:
        pass

```

```
srv = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
```

```
srv.bind(self.cmd_socket_path)
```

```
os.chmod(self.cmd_socket_path, 0o666)
```

```
srv.listen(5)
```

```
srv.settimeout(1.0)
```

```
log(f'cmd listener on {self.cmd_socket_path}')
```

```
while not self.stop_event.is_set():
```

```
    try:
```

```
        conn, _ = srv.accept()
```

```
    except socket.timeout:
```

```
        continue
```

```
    try:
```

```
        data = conn.recv(4096).decode().strip()
```

```
        if data:
```

```
            log(f'cmd recv: {data}')
```

```
            for ln in data.splitlines():
```

```
                self._enqueue(ln.strip())
```

```
    except Exception:
```

```
        pass
```

```
    finally:
```

```
        conn.close()
```

```
srv.close()
```

```
def _worker(self):
```

```
    next_t = time.monotonic()
```

```
    grip_counter = 0
```

```
    last_grip = -1
```

```
    while not self.stop_event.is_set():
```

```
        # Drain command queue first – commands take priority over polling.
```

```
        while True:
```

```
            try:
```

```
                cmd = self.cmd_queue.get_nowait()
```

```
            except queue.Empty:
```

```
                break
```

```
            self._exec(cmd)
```

```
            if self.stop_event.is_set():
```

```
                return
```

```

# Poll robot state.

try:

    radians = self.mc.get_radians()

    if not isinstance(radians, list) or len(radians) < 6:

        continue

    # Greifer nur alle 10 Zyklen lesen (serielle Last reduzieren)

    grip_counter += 1

    if grip_counter >= 10:

        grip_counter = 0

        val = self._read_gripper()

        if val is not None:

            last_grip = val

        parts = ['state', f'{time.time():.6f}']

        parts.extend(f'{float(r):.6f}' for r in radians)

        parts.append(str(last_grip))

        self._broadcast(' '.join(parts) + '\n')

except Exception as e:

    log(f'WARN poll: {e}')


# Pace.

next_t += self.period

sleep = next_t - time.monotonic()

if sleep > 0:

    time.sleep(sleep)

else:

    next_t = time.monotonic()


def _read_gripper(self):

    """Gibt die Greifer-Position zurück – OPEN-LOOP.

```

Die Atom-Firmware dieses Roboters meldet die Position des adaptiven Greifers NICHT ZURUECK: `get_gripper_value()` (mit/ohne Argument, auch nach `init_gripper`) gibt immer 255 (0xFF = "keine Daten") zurück. Mit einem Hardware-Probe bestätigt; `is_gripper_moving` / `protect_current` funktionieren, nur die Positionslesung fehlt. Eine echte Encoder-Lesung ist daher nicht möglich – wir melden den zuletzt KOMMANDIERTEN Wert

(RViz zeigt die kommandierte Pose). Ein einziger Versuch wird trotzdem gemacht: falls die Firmware es künftig unterstützt, wird der echte Wert 0-100 automatisch verwendet.

```
"""
```

```
try:
```

```
    g = self.mc.get_gripper_value(1) # 1 = adaptive gripper
```

```
    if isinstance(g, int) and 0 <= g <= 100:
```

```
        self._grip_log(f'firmware reported value={g}')
```

```
        return g
```

```
    self._grip_log(f'no position from firmware (raw={g}); '
```

```
        f'open-loop cmd={self.last_gripper_cmd}')
```

```
except Exception as e:
```

```
    self._grip_log(f'read exc={e}; open-loop cmd={self.last_gripper_cmd}')
```

```
return self.last_gripper_cmd
```

```
def _grip_log(self, msg):
```

```
    """Die ersten paar Lesungen und danach jede 20. loggen (Spam vermeiden)."""
```

```
    self._grip_log_counter += 1
```

```
    if self._grip_log_counter <= 5 or self._grip_log_counter % 20 == 0:
```

```
        log(f'gripper read: {msg}')
```

```
def _exec(self, line):
```

```
    parts = line.split()
```

```
    if not parts:
```

```
        return
```

```
    name = parts[0]
```

```
    # Zweite Verteidigungslinie (Race beim Flag-Setzen): auch hier blockieren.
```

```
    if self.estop and name in ('send_radians', 'set_gripper', 'release_all'):
```

```
        log(f'NOT-AUS aktiv – Kommando verworfen: {name}')
```

```
        return
```

```
    try:
```

```
        if name == 'estop':
```

```
            if len(parts) > 1 and parts[1] not in ('0', 'false', 'False'):
```

```
                # Stopp DREIMAL senden (seriell ist gelegentlich verlustbehaftet);
```

```
                # stop() hält die Bewegung an, lässt die Servos aber unter
```

```
                # Drehmoment – der Arm fällt NICHT (kein release_all_servos!).
```

```
                for _ in range(3):
```

```

        try:

            self.mc.stop()

        except Exception as e:

            log(f'WARN estop stop(): {e}')

            time.sleep(0.02)

            log('*** NOT-AUS AKTIV *** – Bewegung gestoppt, Kommandos blockiert')

        else:

            log('NOT-AUS FREIGABE – Kommandos wieder aktiv')

    elif name == 'send_radians' and len(parts) >= 8:

        rads = [float(x) for x in parts[1:7]]

        speed = int(parts[7])

        self.mc.send_radians(rads, speed)

    elif name == 'set_gripper' and len(parts) >= 3:

        v = int(parts[1])

        self.mc.set_gripper_value(v, int(parts[2]))

        self.last_gripper_cmd = max(0, min(100, v))

    elif name == 'power_on':

        self.mc.power_on()

    elif name == 'release_all':

        self.mc.release_all_servos()

    elif name == 'shutdown':

        self.stop_event.set()

    else:

        log(f'WARN unknown cmd: {line!r}')

except Exception as e:

    log(f'WARN cmd {name}: {e}')

def _broadcast(self, line):

    payload = line.encode('utf-8')

    with self.client_lock:

        conn = self.client_conn

    if conn is None:

        return

    try:

        conn.sendall(payload)

    except (BrokenPipeError, OSError):

        pass # accept loop will reset on next reconnect

```

```

def main():

    ap = argparse.ArgumentParser()

    ap.add_argument('--port', default='/dev/ttyTHS1')

    ap.add_argument('--baud', type=int, default=1000000)

    ap.add_argument('--socket', default='/tmp/mycobot_bridge.sock')

    ap.add_argument('--rate', type=float, default=20.0,
                    help='polling Hz (serial round-trip ~50ms)')

    ap.add_argument('--speed', type=int, default=30, help='default send_radians speed 1-100')

    ap.add_argument('--no-power-on', action='store_true',
                    help='do not call power_on() at startup (leave robot in teach mode)')

    args = ap.parse_args()

    log(f'opening {args.port} @ {args.baud}')

    mc = MyCobot280(args.port, args.baud)

    time.sleep(0.2) # let serial settle

    if not args.no_power_on:

        try:

            mc.power_on()

            log('power_on OK')

        except Exception as e:

            log(f'WARN power_on: {e}')

    bridge = Bridge(mc, args.speed, args.rate, args.socket)

    # Clean shutdown on SIGINT/SIGTERM (parent kill).

    def _sigterm(*_):

        log('signal received, stopping')

        bridge.stop_event.set()

    signal.signal(signal.SIGTERM, _sigterm)

    signal.signal(signal.SIGINT, _sigterm)

    bridge.run()

    log('exiting')

```



```
if __name__ == '__main__':  
    main()
```

src/mycobot_hardware/src/mycobot_hardware.cpp

```
// myCobot 280 JN SystemInterface – talks to mycobot_bridge.py subprocess

// over a Unix socket. All pymycobot calls live in the bridge process.

#include "mycobot_hardware/mycobot_hardware.hpp"

#include <algorithm>
#include <chrono>
#include <cerrno>
#include <cmath>
#include <cstring>
#include <limits>
#include <sstream>

#include <fcntl.h>
#include <signal.h>
#include <spawn.h>
#include <sys/socket.h>
#include <sys/stat.h>
#include <sys/un.h>
#include <sys/wait.h>
#include <unistd.h>

#include "ament_index_cpp/get_package_prefix.hpp"
#include "rclcpp/rclcpp.hpp"
#include "pluginlib/class_list_macros.hpp"

extern char ** environ;

using namespace std::chrono_literals;
using hardware_interface::return_type;

namespace mycobot_hardware
{
{

static rclcpp::Logger logger()
{
```

```

    return rclcpp::get_logger("mycobot_hardware");
}

::CallbackReturn MyCobotHardware::on_init(const hardware_interface::HardwareInfo & info)
{
    if (SystemInterface::on_init(info) != ::CallbackReturn::SUCCESS) {
        return ::CallbackReturn::ERROR;
    }

    auto get_or = [&info](const std::string & key, const std::string & def) {
        auto it = info.hardware_parameters.find(key);
        return (it == info.hardware_parameters.end()) ? def : it->second;
    };

    port_ = get_or("port", "/dev/ttyTHS1");
    baud_ = std::stoi(get_or("baud", "1000000"));
    command_speed_ = std::stoi(get_or("command_speed", "30"));
    write_period_s_ = std::stod(get_or("write_period_s", "0.25"));
    change_threshold_rad_ = std::stod(get_or("change_threshold_rad", "0.001"));
    socket_path_ = get_or("socket_path", "/tmp/mycobot_bridge.sock");

    // Locate bridge script in install/share/...
    try {
        bridge_script_ = ament_index_cpp::get_package_prefix("mycobot_hardware")
            + "/lib/mycobot_hardware/mycobot_bridge.py";
    } catch (const std::exception & e) {
        RCLCPP_FATAL(logger(), "cannot locate mycobot_hardware prefix: %s", e.what());
        return ::CallbackReturn::ERROR;
    }

    const size_t n = info_.joints.size();
    if (n != kArmDof && n != (kArmDof + 1)) {
        RCLCPP_FATAL(logger(),
            "Expected 6 arm joints (optionally +1 gripper). Got %zu.", n);
        return ::CallbackReturn::ERROR;
    }
}

```

```

hw_positions_.assign(n, 0.0);

hw_velocities_.assign(n, 0.0);

hw_position_cmds_.assign(n, std::numeric_limits<double>::quiet_NaN());

last_sent_cmds_.assign(n, std::numeric_limits<double>::quiet_NaN());

latest_arm_.assign(kArmDof, 0.0);


RCLCPP_INFO(logger(),

    "init: port=%s baud=%d speed=%d joints=%zu socket=%s",

    port_.c_str(), baud_, command_speed_, n, socket_path_.c_str());

return ::CallbackReturn::SUCCESS;
}

bool MyCobotHardware::start_bridge()
{
    // Spawn: python3 <bridge_script_> --port <port_> --baud <baud_>

    //                                --socket <socket_path_> --rate 20

    std::string sport = port_;

    std::string sbaud = std::to_string(baud_);

    std::string srate = "20";

    std::string sspeed = std::to_string(command_speed_);


    std::vector<char *> argv = {

        const_cast<char *>("python3"),

        const_cast<char *>(bridge_script_.c_str()),

        const_cast<char *>("--port"),    const_cast<char *>(sport.c_str()),

        const_cast<char *>("--baud"),    const_cast<char *>(sbaud.c_str()),

        const_cast<char *>("--socket"), const_cast<char *>(socket_path_.c_str()),

        const_cast<char *>("--rate"),    const_cast<char *>(srate.c_str()),

        const_cast<char *>("--speed"),   const_cast<char *>(sspeed.c_str()),

        nullptr,

    };


    pid_t pid = 0;

    int rc = posix_spawn(&pid, "python3", nullptr, nullptr, argv.data(), environ);

    if (rc != 0) {

        RCLCPP_FATAL(logger(), "posix_spawn failed: %s", std::strerror(rc));

        return false;
    }
}

```

```

}

bridge_pid_ = pid;

RCLCPP_INFO(logger(), "bridge pid=%d (%s)", pid, bridge_script_.c_str());

return true;
}

bool MyCobotHardware::connect_socket()
{
    // Wait for bridge to create the socket, then connect.

    for (int attempt = 0; attempt < 60; ++attempt) { // 60 * 200ms = 12s

        struct stat st;

        if (::stat(socket_path_.c_str(), &st) == 0 && S_ISSOCK(st.st_mode)) {

            int fd = ::socket(AF_UNIX, SOCK_STREAM, 0);

            if (fd < 0) {

                RCLCPP_FATAL(logger(), "socket() failed: %s", std::strerror(errno));

                return false;

            }

            sockaddr_un addr{};

            addr.sun_family = AF_UNIX;

            std::strncpy(addr.sun_path, socket_path_.c_str(), sizeof(addr.sun_path) - 1);

            if (::connect(fd, reinterpret_cast<sockaddr *>(&addr), sizeof(addr)) == 0) {

                sock_fd_ = fd;

                RCLCPP_INFO(logger(), "connected to bridge socket");

                return true;

            }

            ::close(fd);

        }

        std::this_thread::sleep_for(200ms);

    }

    RCLCPP_FATAL(logger(), "bridge socket never appeared at %s", socket_path_.c_str());

    return false;
}

::CallbackReturn MyCobotHardware::on_configure(const rclcpp_lifecycle::State &)
{
    if (!start_bridge()) return ::CallbackReturn::ERROR;

    if (!connect_socket()) {

```

```

    stop_bridge();

    return ::CallbackReturn::ERROR;
}

// Start read thread now so we can prime hw_positions_ from incoming state.
read_running_ = true;
read_thread_ = std::thread(&MyCobotHardware::read_loop, this);

// Wait up to 2s for first state line.
for (int i = 0; i < 20 && !latest_fresh_; ++i) {
    std::this_thread::sleep_for(100ms);
}

if (latest_fresh_) {
    std::lock_guard<std::mutex> lk(state_mutex_);

    for (size_t i = 0; i < kArmDof; ++i) {
        hw_positions_[i] = latest_arm_[i];
        hw_position_cmds_[i] = latest_arm_[i];
    }

    if (info_.joints.size() > kArmDof) {
        hw_positions_[kGripperIdx] = latest_gripper_;
    }

    RCLCPP_INFO(logger(), "primed positions from bridge");
} else {
    RCLCPP_WARN(logger(), "no state from bridge yet – continuing");
}

return ::CallbackReturn::SUCCESS;
}

::CallbackReturn MyCobotHardware::on_activate(const rclcpp_lifecycle::State &)
{
    send_line("power_on\n");

    const auto now = rclcpp::Clock(RCL_STEADY_TIME).now();

    last_write_ = now;
    last_read_ = now;

    RCLCPP_INFO(logger(), "Activated.");

    return ::CallbackReturn::SUCCESS;
}

```

```

::CallbackReturn MyCobotHardware::on_deactivate(const rclcpp_lifecycle::State &)
{
    send_line("release_all\n");

    RCLCPP_INFO(logger(), "Deactivated - servos released.");

    return ::CallbackReturn::SUCCESS;
}

::CallbackReturn MyCobotHardware::on_cleanup(const rclcpp_lifecycle::State &)
{
    read_running_ = false;

    if (sock_fd_ >= 0) {
        send_line("shutdown\n");

        ::shutdown(sock_fd_, SHUT_RDWR);

        ::close(sock_fd_);

        sock_fd_ = -1;
    }

    if (read_thread_.joinable()) {
        read_thread_.join();
    }

    stop_bridge();

    return ::CallbackReturn::SUCCESS;
}

void MyCobotHardware::stop_bridge()
{
    if (bridge_pid_ <= 0) return;

    ::kill(bridge_pid_, SIGTERM);

    // Wait up to 3s
    for (int i = 0; i < 30; ++i) {
        int status = 0;

        pid_t r = ::waitpid(bridge_pid_, &status, WNOHANG);

        if (r == bridge_pid_) {
            bridge_pid_ = -1;

            return;
        }

        std::this_thread::sleep_for(100ms);
    }
}

```

```

}

::kill(bridge_pid_, SIGKILL);

::waitpid(bridge_pid_, nullptr, 0);

bridge_pid_ = -1;
}

bool MyCobotHardware::send_line(const std::string & line)
{
    if (sock_fd_ < 0) return false;

    const char * p = line.data();

    size_t left = line.size();

    while (left > 0) {
        ssize_t n = ::send(sock_fd_, p, left, MSG_NOSIGNAL);

        if (n < 0) {
            if (errno == EINTR) continue;

            RCLCPP_WARN_THROTTLE(
                logger(), *rclcpp::Clock::make_shared(RCL_STEADY_TIME).get(), 5000,
                "send failed: %s", std::strerror(errno));

            return false;
        }

        p += n;

        left -= n;
    }

    return true;
}

void MyCobotHardware::read_loop()
{
    std::string buf;

    buf.reserve(4096);

    char chunk[1024];

    while (read_running_) {
        ssize_t n = ::recv(sock_fd_, chunk, sizeof(chunk), 0);

        if (n == 0) {
            RCLCPP_WARN(logger(), "bridge closed socket");

            break;
        }
    }
}

```



```

if (n < 0) {

    if (errno == EINTR) continue;

    if (errno == EBADF) break; // closed during cleanup

    RCLCPP_WARN_THROTTLE(
        logger(), *rclcpp::Clock::make_shared(RCL_STEADY_TIME).get(), 5000,
        "recv failed: %s", std::strerror(errno));

    std::this_thread::sleep_for(100ms);

    continue;
}

buf.append(chunk, static_cast<size_t>(n));

// Process complete lines.

size_t pos;

while ((pos = buf.find('\n')) != std::string::npos) {

    std::string line = buf.substr(0, pos);

    buf.erase(0, pos + 1);

    if (line.empty()) continue;

    // "state <ts> r1..r6 grip"

    std::istringstream iss(line);

    std::string tag;

    iss >> tag;

    if (tag != "state") continue;

    double ts;

    iss >> ts;

    std::vector<double> r(kArmDof);

    bool ok = true;

    for (size_t i = 0; i < kArmDof; ++i) {

        if (!(iss >> r[i])) { ok = false; break; }

    }

    if (!ok) continue;

    int grip = -1;

    iss >> grip;

    {

        std::lock_guard<std::mutex> lk(state_mutex);

```

```

    latest_arm_ = r;

    // pymycobot returns 0-100; values like 255/-1 mean "servo not responding"

    // and must be rejected, otherwise 255 -> 1.785 rad rotates the mimic

    // finger links out of view. Hold the last valid value on a bad read.

    if (grip >= 0 && grip <= 100) {

        latest_gripper_ = (static_cast<double>(grip) / 100.0) * 0.7;

    }

}

latest_fresh_ = true;

}

}
}

```

```

std::vector<hardware_interface::StateInterface>

MyCobotHardware::export_state_interfaces()

{

    std::vector<hardware_interface::StateInterface> out;

    for (size_t i = 0; i < info_.joints.size(); ++i) {

        out.emplace_back(info_.joints[i].name, hardware_interface::HW_IF_POSITION,

            &hw_positions_[i]);

        out.emplace_back(info_.joints[i].name, hardware_interface::HW_IF_VELOCITY,

            &hw_velocities_[i]);

    }

    return out;

}

```

```

std::vector<hardware_interface::CommandInterface>

MyCobotHardware::export_command_interfaces()

{

    std::vector<hardware_interface::CommandInterface> out;

    for (size_t i = 0; i < info_.joints.size(); ++i) {

        out.emplace_back(info_.joints[i].name, hardware_interface::HW_IF_POSITION,

            &hw_position_cmds_[i]);

    }

    return out;

}

```

```

return_type MyCobotHardware::read()
{
    if (!latest_fresh_) return return_type::OK;

    const auto now = rclcpp::Clock(RCL_STEADY_TIME).now();

    const double dt = std::max((now - last_read_).seconds(), 1e-3);

    last_read_ = now;

    std::vector<double> arm(kArmDof);

    double gripper;

    {
        std::lock_guard<std::mutex> lk(state_mutex_);

        for (size_t i = 0; i < kArmDof; ++i) arm[i] = latest_arm_[i];

        gripper = latest_gripper_;
    }

    for (size_t i = 0; i < kArmDof; ++i) {
        hw_velocities_[i] = (arm[i] - hw_positions_[i]) / dt;

        hw_positions_[i] = arm[i];
    }

    if (info_.joints.size() > kArmDof) {
        hw_velocities_[kGripperIdx] = (gripper - hw_positions_[kGripperIdx]) / dt;

        hw_positions_[kGripperIdx] = gripper;
    }

    return return_type::OK;
}

return_type MyCobotHardware::write()
{
    const auto now = rclcpp::Clock(RCL_STEADY_TIME).now();

    if ((now - last_write_).seconds() < write_period_s_) {
        return return_type::OK;
    }

    std::vector<double> arm_cmd(kArmDof);

    bool any_change = false;

    for (size_t i = 0; i < kArmDof; ++i) {
        const double c = hw_position_cmds_[i];

        if (!std::isfinite(c)) {

```

```

        arm_cmd[i] = hw_positions_[i]; // hold

        continue;
    }

    arm_cmd[i] = c;

    if (!std::isfinite(last_sent_cmds_[i]) ||
        std::abs(c - last_sent_cmds_[i]) > change_threshold_rad_)
    {
        any_change = true;
    }
}

if (!any_change) return return_type::OK;

std::ostringstream oss;

oss.precision(6);

oss << std::fixed << "send_radians";

for (size_t i = 0; i < kArmDof; ++i) oss << ' ' << arm_cmd[i];

oss << ' ' << command_speed_ << '\n';

send_line(oss.str());

if (info_.joints.size() > kArmDof) {
    const double c = hw_position_cmds_[kGripperIdx];

    if (std::isfinite(c)) {
        int g = static_cast<int>(std::clamp(c / 0.7, 0.0, 1.0) * 100.0);

        std::ostringstream gg;

        gg << "set_gripper " << g << ' ' << command_speed_ << '\n';

        send_line(gg.str());
    }
}

for (size_t i = 0; i < kArmDof; ++i) last_sent_cmds_[i] = arm_cmd[i];

last_write_ = now;

return return_type::OK;
}

} // namespace mycobot_hardware

PLUGINLIB_EXPORT_CLASS(

```

```
| mycobot_hardware::MyCobotHardware, hardware_interface::SystemInterface)
```

src/mycobot_moveit_config/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.5)

project(mycobot_moveit_config)


find_package(ament_cmake REQUIRED)


install(DIRECTORY config launch rviz
  DESTINATION share/${PROJECT_NAME}
)

ament_package()
```

src/mycobot_moveit_config/config/cartesian_limits.yaml

```
# Kartesische Limits für den Pilz-Industrial-Motion-Planner (PTP/LIN/CIRC).

# Diese Grenzen begrenzen die Geschwindigkeit/Beschleunigung des End-Effektors
# im kartesischen Raum (Translation und Rotation), nicht in Gelenkkoordinaten.

# Auch wenn Pilz in diesem Projekt nicht aktiv verwendet wird, lädt
# MoveItConfigsBuilder.to_moveit_configs() diese Datei implizit – fehlt sie,
# schlägt der Konfigurationsaufbau fehl. Daher vorerst sinnvolle Standardwerte.

cartesian_limits:

  # max_trans_vel: maximale translatorische Geschwindigkeit des TCP in m/s.
  max_trans_vel: 1.0

  # max_trans_acc: maximale translatorische Beschleunigung in m/s^2.
  max_trans_acc: 2.25

  # max_trans_dec: maximale translatorische Verzögerung (Bremsen) in m/s^2;
  # negativ, da der Beschleunigungsvektor der Bewegung entgegengerichtet ist.
  max_trans_dec: -5.0

  # max_rot_vel: maximale rotatorische Geschwindigkeit der Orientierung in rad/s.
  max_rot_vel: 1.57
```

src/mycobot_moveit_config/config/joint_limits.yaml

```
# Gelenk-Geschwindigkeits- und Beschleunigungslimits für die Zeitparametrisierung.

# MoveIt nutzt diese Grenzen, um dem geometrisch geplanten Pfad ein zulässiges
# Geschwindigkeits-/Beschleunigungsprofil zu geben (siehe AddTimeOptimalParameterization).

# Hintergrund: Das <limit velocity="0"/> in der URDF wurde upstream auf 0 gelassen
# (für den echten Roboter nicht angegeben); ein Limit von 0 würde jede Bewegung
# unmöglich machen, daher überschreiben wir es hier. Werte ungefähr aus dem
# myCobot-280-Datenblatt. Einheiten: rad/s (Geschwindigkeit), rad/s^2 (Beschleunigung).

joint_limits:

  # joint2_to_joint1 bis joint6output_to_joint6: die sechs aktiven Armgelenke.
  # Alle mit identischen konservativen Limits belegt (1.5 rad/s, 2.0 rad/s^2),
  # um ruckfreie, sichere Bewegungen des kleinen myCobot 280 zu gewährleisten.

  joint2_to_joint1:

    has_velocity_limits: true

    max_velocity: 1.5          # rad/s

    has_acceleration_limits: true

    max_acceleration: 2.0      # rad/s^2

  joint3_to_joint2:

    has_velocity_limits: true

    max_velocity: 1.5

    has_acceleration_limits: true

    max_acceleration: 2.0

  joint4_to_joint3:

    has_velocity_limits: true

    max_velocity: 1.5

    has_acceleration_limits: true

    max_acceleration: 2.0

  joint5_to_joint4:

    has_velocity_limits: true

    max_velocity: 1.5

    has_acceleration_limits: true

    max_acceleration: 2.0

  joint6_to_joint5:

    has_velocity_limits: true

    max_velocity: 1.5

    has_acceleration_limits: true
```



```
    max_acceleration: 2.0

joint6output_to_joint6:

    has_velocity_limits: true

    max_velocity: 1.5

    has_acceleration_limits: true

    max_acceleration: 2.0

# gripper_controller: das aktive Greifergelenk. Etwas langsamere Geschwindigkeit
# (1.0 rad/s), da das Öffnen/Schließen keine schnelle Bewegung erfordert.
gripper_controller:

    has_velocity_limits: true

    max_velocity: 1.0          # rad/s

    has_acceleration_limits: true

    max_acceleration: 2.0      # rad/s^2
```

src/mycobot_moveit_config/config/kinematics.yaml

```
# Inverse-Kinematik-(IK-)Konfiguration für die Planungsgruppe "arm".

# MoveIt lädt hier den Kinematik-Solver, der aus einer kartesischen Ziel-Pose
# (Position + Orientierung des tcp) die zugehörigen Gelenkwinkel berechnet.

arm:

  # kinematics_solver: verwendeter IK-Plugin. TRAC-IK löst die IK numerisch
  # und ist robuster/schneller als der Standard-KDL-Solver, besonders nahe
  # Singularitäten. Für Galactic manuell portiert (siehe Projekt-Memory).

  kinematics_solver: trac_ik_kinematics_plugin/TRAC_IKKinematicsPlugin

  # kinematics_solver_search_resolution: Schrittweite der internen Suche (rad).
  # Kleinere Werte = feiner, aber langsamer.

  kinematics_solver_search_resolution: !!float 0.005

  # kinematics_solver_timeout: max. Rechenzeit pro IK-Aufruf in Sekunden.
  # Wird die Lösung nicht rechtzeitig gefunden, gilt der Aufruf als Fehlschlag.

  kinematics_solver_timeout: !!float 0.5

  # position_only_ik: false = Position UND Orientierung werden gelöst.
  # true würde nur die Position berücksichtigen (für diesen 6-DoF-Arm nicht gewünscht).

  position_only_ik: false

  # solve_type: TRAC-IK-Optimierungsmodus. "Distance" minimiert die Gelenk-
  # bewegung gegenüber dem Startzustand und liefert damit ruhige, natürliche
  # Lösungen (Alternativen wären z. B. "Speed" oder "Manip1").

  solve_type: Distance

  # epsilon: Konvergenztoleranz der numerischen Lösung. Kleiner = genauer.

  epsilon: !!float 1e-5
```

src/mycobot_moveit_config/config/moveit_controllers.yaml

```
# MoveIt → ros2_control Brücke.

# moveit_simple_controller_manager sendet die von MoveIt geplante Trajectory
# an den FollowJointTrajectory-Action-Server des joint_trajectory_controller.

# ros2_control läuft mit mock_components als fake execution.


moveit_simple_controller_manager:

  # controller_names: Liste aller Controller, die MoveIt ansteuern darf.

  controller_names:

    - arm_controller

    - gripper_controller


  # arm_controller: führt die geplante Armtrajektorie aus.

  #   type: FollowJointTrajectory-Interface.

  #   action_ns: Namespace des Action-Servers (voller Name: arm_controller/follow_joint_trajectory).

  #   default: true = Standard-Controller für die Armgruppe.

  #   joints: exakt die sechs Armgelenke in Trajektorien-Reihenfolge – muss mit
  #   den in ros2_controllers.yaml definierten Gelenken übereinstimmen.

  arm_controller:

    type: FollowJointTrajectory

    action_ns: follow_joint_trajectory

    default: true

    joints:

      - joint2_to_joint1

      - joint3_to_joint2

      - joint4_to_joint3

      - joint5_to_joint4

      - joint6_to_joint5

      - joint6output_to_joint6


  # gripper_controller: führt die Greiferbewegung aus (ebenfalls FollowJointTrajectory).

  gripper_controller:

    type: FollowJointTrajectory

    action_ns: follow_joint_trajectory

    default: true

    joints:
```

- gripper_controller

```
# moveit_controller_manager: wählt die Simple-Controller-Manager-Implementierung,  
# die obige controller_names auf ros2_control-Action-Server abbildet.  
moveit_controller_manager: moveit_simple_controller_manager/MoveItSimpleControllerManager  
  
# trajectory_execution: Toleranzen der Ausführungsüberwachung von MoveIt.  
trajectory_execution:  
  # allowed_execution_duration_scaling: erlaubter Faktor, um den die reale  
  # Ausführung länger dauern darf als geplant, bevor abgebrochen wird (1.2 = 20% Puffer).  
  allowed_execution_duration_scaling: 1.2  
  # allowed_goal_duration_margin: zusätzlicher absoluter Zeitpuffer (s) am Trajektorienende.  
  allowed_goal_duration_margin: 0.5  
  # allowed_start_tolerance: erlaubte Abweichung zwischen aktuellem Ist-Zustand und  
  # geplantem Startzustand. MUSS hier 0.0 sein – andernfalls verwirft der  
  # joint_trajectory_controller die Trajektorie mit "time not strictly increasing"  
  # (siehe Projekt-Memory "Trajectory timestamp fix").  
  allowed_start_tolerance: 0.0
```

src/mycobot_moveit_config/config/mycobot.srdf

```
<?xml version="1.0" encoding="UTF-8"?>

<!--
  SRDF für myCobot 280 JN + adaptive gripper.

  Planning groups: arm (6 DoF), gripper (1 active DoF + Mimics).

  End effector tip frame: tcp (Master-Plan vorgeschrieben).
-->

<robot name="mycobot_world">

  <!-- =====

    Planning groups

    ===== -->

  <!-- 6 DoF arm chain (robot_base → joint6_flange).

    Tip frame tcp (Greifer-Spitze) wird als Ziel genommen. -->

  <group name="arm">

    <chain base_link="robot_base" tip_link="tcp"/>

  </group>

  <!-- Greifer-Gruppe: aktives Gelenk + Mimic-Finger -->

  <group name="gripper">

    <joint name="gripper_controller"/>

    <joint name="gripper_base_to_gripper_left2"/>

    <joint name="gripper_left3_to_gripper_left1"/>

    <joint name="gripper_base_to_gripper_right3"/>

    <joint name="gripper_base_to_gripper_right2"/>

    <joint name="gripper_right3_to_gripper_right1"/>

  </group>

  <!-- =====

    End-Effector-Definition

    ===== -->

  <end_effector name="gripper_ee"

    parent_link="tcp"

    group="gripper"

    parent_group="arm"/>
```

```

<!-- =====
    Passive (Mimic-) Gelenke – markiert, damit der MoveIt-Planner
    sie nicht aktiv berechnet.
    ===== -->

<passive_joint name="gripper_base_to_gripper_left2"/>
<passive_joint name="gripper_left3_to_gripper_left1"/>
<passive_joint name="gripper_base_to_gripper_right3"/>
<passive_joint name="gripper_base_to_gripper_right2"/>
<passive_joint name="gripper_right3_to_gripper_right1"/>

<!-- =====
    Named states (vordefinierte Positionen)
    ===== -->

<group_state name="home" group="arm">
    <joint name="joint2_to_joint1" value="0"/>
    <joint name="joint3_to_joint2" value="0"/>
    <joint name="joint4_to_joint3" value="0"/>
    <joint name="joint5_to_joint4" value="0"/>
    <joint name="joint6_to_joint5" value="0"/>
    <joint name="joint6output_to_joint6" value="0"/>
</group_state>

<group_state name="ready" group="arm">
    <joint name="joint2_to_joint1" value="0"/>
    <joint name="joint3_to_joint2" value="-0.5"/>
    <joint name="joint4_to_joint3" value="-1.0"/>
    <joint name="joint5_to_joint4" value="0"/>
    <joint name="joint6_to_joint5" value="0"/>
    <joint name="joint6output_to_joint6" value="0"/>
</group_state>

<group_state name="open" group="gripper">
    <joint name="gripper_controller" value="0.15"/>
</group_state>

<group_state name="closed" group="gripper">

```

```

    <joint name="gripper_controller" value="-0.74"/>
</group_state>

<!-- =====

Komfort-Kollisions-Disables zwischen benachbarten Links.

Ohne Setup Assistant geben wir diese manuell an; im Baum
ist jedes direkt miteinander verbundene Link-Paar ohnehin
automatisch vom Kollisions-Check ausgenommen – hier nur
Mimic-Finger im Greifer und ferne Kamera-Arm-Paare.

===== -->

<!-- Nachbarschaft innerhalb des Arms (nicht zwingend, aber explizit notiert) -->
<disable_collisions link1="joint1" link2="joint2" reason="Adjacent"/>
<disable_collisions link1="joint2" link2="joint3" reason="Adjacent"/>
<disable_collisions link1="joint3" link2="joint4" reason="Adjacent"/>
<disable_collisions link1="joint4" link2="joint5" reason="Adjacent"/>
<disable_collisions link1="joint5" link2="joint6" reason="Adjacent"/>
<disable_collisions link1="joint6" link2="joint6_flange" reason="Adjacent"/>

<!-- Flange ↔ Greifer-Verbindung -->
<disable_collisions link1="joint6_flange" link2="gripper_base" reason="Adjacent"/>
<disable_collisions link1="joint6" link2="gripper_base" reason="Never"/>

<!-- joint6 und joint6_flange ↔ alle Greifer-Finger-Links – Finger unter joint6, keine Kollision -->
<disable_collisions link1="joint6" link2="gripper_left1" reason="Never"/>
<disable_collisions link1="joint6" link2="gripper_left2" reason="Never"/>
<disable_collisions link1="joint6" link2="gripper_left3" reason="Never"/>
<disable_collisions link1="joint6" link2="gripper_right1" reason="Never"/>
<disable_collisions link1="joint6" link2="gripper_right2" reason="Never"/>
<disable_collisions link1="joint6" link2="gripper_right3" reason="Never"/>
<disable_collisions link1="joint6_flange" link2="gripper_left1" reason="Never"/>
<disable_collisions link1="joint6_flange" link2="gripper_left2" reason="Never"/>
<disable_collisions link1="joint6_flange" link2="gripper_left3" reason="Never"/>
<disable_collisions link1="joint6_flange" link2="gripper_right1" reason="Never"/>
<disable_collisions link1="joint6_flange" link2="gripper_right2" reason="Never"/>
<disable_collisions link1="joint6_flange" link2="gripper_right3" reason="Never"/>

```

```

<!-- Mimic-Finger im Greifer – können sich ständig gegenseitig berühren -->
<disable_collisions link1="gripper_base" link2="gripper_left1" reason="Adjacent"/>
<disable_collisions link1="gripper_base" link2="gripper_left2" reason="Adjacent"/>
<disable_collisions link1="gripper_base" link2="gripper_left3" reason="Adjacent"/>
<disable_collisions link1="gripper_base" link2="gripper_right1" reason="Adjacent"/>
<disable_collisions link1="gripper_base" link2="gripper_right2" reason="Adjacent"/>
<disable_collisions link1="gripper_base" link2="gripper_right3" reason="Adjacent"/>
<disable_collisions link1="gripper_left3" link2="gripper_left1" reason="Adjacent"/>
<disable_collisions link1="gripper_right3" link2="gripper_right1" reason="Adjacent"/>
<disable_collisions link1="gripper_left1" link2="gripper_left2" reason="Never"/>
<disable_collisions link1="gripper_right1" link2="gripper_right2" reason="Never"/>
<!-- 4-Gelenk-Geschwister-Links: in OFFENER (0.15) Greifer-Pose überlappen die Kollisions-Boxen
konstruktionsbedingt (Sitzung-29-Sim-Befund: left2×left3-Berührung sperrte nach dem
Öffnen des Greifers die GESAMTE Planung mit "invalid start state"). -->
<disable_collisions link1="gripper_left2" link2="gripper_left3" reason="Never"/>
<disable_collisions link1="gripper_right2" link2="gripper_right3" reason="Never"/>
<disable_collisions link1="gripper_left1" link2="gripper_right1" reason="Never"/>
<disable_collisions link1="gripper_left2" link2="gripper_right2" reason="Never"/>
<disable_collisions link1="gripper_left3" link2="gripper_right3" reason="Never"/>
<disable_collisions link1="gripper_left1" link2="gripper_right2" reason="Never"/>
<disable_collisions link1="gripper_left2" link2="gripper_right1" reason="Never"/>
<disable_collisions link1="gripper_left1" link2="gripper_right3" reason="Never"/>
<disable_collisions link1="gripper_left3" link2="gripper_right1" reason="Never"/>
<disable_collisions link1="gripper_left2" link2="gripper_right3" reason="Never"/>
<disable_collisions link1="gripper_left3" link2="gripper_right2" reason="Never"/>

<!-- Kamera weit vom Roboter-Körper entfernt (eye-to-hand statisch) – Kollisions-Check unnötig -->
<disable_collisions link1="camera_link" link2="robot_base" reason="Never"/>
<disable_collisions link1="camera_link" link2="joint1" reason="Never"/>
<disable_collisions link1="camera_link" link2="joint2" reason="Never"/>
<disable_collisions link1="camera_link" link2="joint3" reason="Never"/>
<disable_collisions link1="camera_link" link2="joint4" reason="Never"/>
<disable_collisions link1="camera_link" link2="joint5" reason="Never"/>
<disable_collisions link1="camera_link" link2="joint6" reason="Never"/>
<disable_collisions link1="camera_link" link2="joint6_flange" reason="Never"/>
<disable_collisions link1="camera_link" link2="gripper_base" reason="Never"/>
<disable_collisions link1="camera_link" link2="gripper_left1" reason="Never"/>

```



```

<disable_collisions link1="camera_link" link2="gripper_left2" reason="Never"/>
<disable_collisions link1="camera_link" link2="gripper_left3" reason="Never"/>
<disable_collisions link1="camera_link" link2="gripper_right1" reason="Never"/>
<disable_collisions link1="camera_link" link2="gripper_right2" reason="Never"/>
<disable_collisions link1="camera_link" link2="gripper_right3" reason="Never"/>
<disable_collisions link1="camera_link" link2="tcp" reason="Never"/>

<!-- Greifer ↔ alle Arm-Links – myCobot 280 sehr klein, beim Falten des Arms ist naher Durchgang normal -->
<disable_collisions link1="gripper_base" link2="joint1" reason="Never"/>
<disable_collisions link1="gripper_base" link2="joint5" reason="Never"/>
<disable_collisions link1="gripper_base" link2="robot_base" reason="Never"/>
<disable_collisions link1="gripper_left1" link2="joint1" reason="Never"/>
<disable_collisions link1="gripper_left2" link2="joint1" reason="Never"/>
<disable_collisions link1="gripper_left3" link2="joint1" reason="Never"/>
<disable_collisions link1="gripper_right1" link2="joint1" reason="Never"/>
<disable_collisions link1="gripper_right2" link2="joint1" reason="Never"/>
<disable_collisions link1="gripper_right3" link2="joint1" reason="Never"/>
<disable_collisions link1="tcp" link2="joint1" reason="Never"/>
<disable_collisions link1="tcp" link2="joint2" reason="Never"/>
<disable_collisions link1="tcp" link2="joint3" reason="Never"/>
<disable_collisions link1="tcp" link2="joint4" reason="Never"/>

<!-- Roboter-Basis – Annahme, dass der Arm sich nicht selbst berührt -->
<disable_collisions link1="robot_base" link2="joint1" reason="Adjacent"/>

<!-- joint1×joint3: WIEDER HINZUGEFUEGT (2026-06-15). Zuerst wegen Basis-Kollision gelöscht, aber
check_state_validity zeigte, dass der joint1-Kollisions-ZYLINDER (r=4.5cm, 15.6cm lang)
sich mit dem forearm(joint3) in FAST JEDER Vorwärts-Reichweiten-Pose schneidet → der Roboter
konnte keine Schraube erreichen. "Never" war korrekt (Geometrie überlappt, aber keine physische
Kollision). Echte Basis-Gehäuse-Kollision wird nun durch die zu robot_base hinzugefügte ECHT-MASS-Box
(15×11×11cm) erfasst (robot_base×joint3 löst nur bei hartem Falten aus). -->
<disable_collisions link1="joint1" link2="joint3" reason="Never"/>
<disable_collisions link1="joint2" link2="joint4" reason="Never"/>
<disable_collisions link1="joint3" link2="joint5" reason="Never"/>
<disable_collisions link1="joint4" link2="joint6" reason="Never"/>

<!-- =====
Dauerhafte Infrastruktur: Plattform + Kaiser RS 2 XA Stand (in URDF definiert).

```

- platform_top: Baseboard, Roboter-Basis immer darauf
- tutucu_kelepce: an die Plattform geschraubte Halte-Klemme des Stands
- column: vertikale Aluminium-Säule
- camera_arm: horizontaler Arm, der von column zur Kamera reicht

Zwischen direkt miteinander verbundenen Teilen ist ein Kollisions-Check sinnlos – alle sind fix und in der Realität ständig in Berührung.

===== -->

```
<disable_collisions link1="platform_top" link2="robot_base" reason="Adjacent"/>
<disable_collisions link1="platform_top" link2="joint1" reason="Adjacent"/>
<disable_collisions link1="platform_top" link2="tutucu_kelepce" reason="Adjacent"/>
<disable_collisions link1="tutucu_kelepce" link2="column" reason="Adjacent"/>
<disable_collisions link1="tutucu_kelepce" link2="robot_base" reason="Never"/>
<disable_collisions link1="column" link2="camera_arm" reason="Adjacent"/>
<disable_collisions link1="column" link2="robot_base" reason="Never"/>
```

<!-- column statisch, geometrisch weit von Roboter-Links entfernt (>30 cm) – FCL-Mesh-

Bounding liefert False-Positive (besonders gripper_base ↔ column). Alle Roboter-

Links von column trennen. Physisches Kollisionsrisiko bleibt Verantwortung des Benutzers. -->

```
<disable_collisions link1="camera_arm" link2="camera_link" reason="Adjacent"/>
<disable_collisions link1="camera_arm" link2="robot_base" reason="Never"/>
```

<!-- Stand-Teile (platform_top Baseboard, tutucu_kelepce, camera_arm) ↔ Arm/Greifer.

Gleicher Grund wie bei column: das ist statische Stand-Geometrie; da der Greifer volle .dä-Mesh-

Kollision nutzt, liefert FCL-Bounding beim Nach-unten/Nach-innen-Reichen des Arms False-Positive

(Links rot). Alle von beweglichen Links trennen. HINWEIS: die physische Anstoß-Prüfung des

Greifers ans Baseboard entfällt damit → Verantwortung des Benutzers. -->

```
<disable_collisions link1="platform_top" link2="joint2" reason="Never"/>
<disable_collisions link1="platform_top" link2="joint3" reason="Never"/>
<disable_collisions link1="platform_top" link2="joint4" reason="Never"/>
<disable_collisions link1="platform_top" link2="joint5" reason="Never"/>
<disable_collisions link1="platform_top" link2="joint6" reason="Never"/>
<disable_collisions link1="platform_top" link2="joint6_flange" reason="Never"/>
<disable_collisions link1="platform_top" link2="grripper_base" reason="Never"/>
<disable_collisions link1="platform_top" link2="grripper_left1" reason="Never"/>
<disable_collisions link1="platform_top" link2="grripper_left2" reason="Never"/>
<disable_collisions link1="platform_top" link2="grripper_left3" reason="Never"/>
<disable_collisions link1="platform_top" link2="grripper_right1" reason="Never"/>
```

```
<disable_collisions link1="platform_top" link2="gripper_right2" reason="Never"/>
```

```
<disable_collisions link1="platform_top" link2="gripper_right3" reason="Never"/>
```

```
</robot>
```

src/mycobot_moveit_config/config/ompl_planning.yaml

```
# OMPL-Planungskonfiguration (Open Motion Planning Library).

# Definiert das Planungs-Plugin, die Vor-/Nachbearbeitungs-Adapter der
# Planungsanfrage sowie die verfügbaren Sampling-basierten Planer je Gruppe.
# HINWEIS: Der Galactic-MoveItConfigsBuilder ergänzt planning_plugin und
# request_adapters NICHT automatisch, daher werden sie hier manuell gesetzt
# (siehe Projekt-Memory "OMPL request adapters").

# planning_plugin: die OMPL-Schnittstelle als aktives Planungs-Plugin für MoveIt.
planning_plugin: ompl_interface/OMPLPlanner

# request_adapters: Kette von Adaptern, die VOR/NACH der Planung laufen.
#   AddTimeOptimalParameterization: erzeugt aus dem geometrischen Pfad ein
#   zeitoptimales Geschwindigkeits-/Beschleunigungsprofil (Zeitparametrisierung).
#   FixWorkspaceBounds: setzt sinnvolle Arbeitsraumgrenzen, falls nicht angegeben.
#   FixStartStateBounds: klemmt einen leicht außerhalb der Limits liegenden
#   Startzustand zurück in die Gelenkgrenzen.
#   FixStartStateCollision: versucht, einen kollidierenden Startzustand minimal
#   zu verschieben, damit die Planung überhaupt starten kann.
#   FixStartStatePathConstraints: passt den Start an, wenn er Pfad-Constraints verletzt.
request_adapters: >-
  default_planner_request_adapters/AddTimeOptimalParameterization
  default_planner_request_adapters/FixWorkspaceBounds
  default_planner_request_adapters/FixStartStateBounds
  default_planner_request_adapters/FixStartStateCollision
  default_planner_request_adapters/FixStartStatePathConstraints

# planner_configs: Katalog konfigurierter OMPL-Planer, auf die die Gruppen
# weiter unten per Namen verweisen.
planner_configs:

  # RRTConnect: bidirektionaler RRT, sehr schnell für Freiraum-Planung –
  # der Standardplaner dieses Projekts. range=0.0 lässt OMPL die Schrittweite
  # automatisch aus dem Zustandsraum bestimmen.
  RRTConnectkConfigDefault:
    type: geometric::RRTConnect
    range: 0.0

  # RRTstar: asymptotisch optimaler RRT (kürzere/glattere Pfade), aber langsamer.
```

```

RRTstarkConfigDefault:

    type: geometric::RRTstar

    range: 0.0

    goal_bias: 0.05          # Wahrscheinlichkeit, direkt zum Ziel zu sampeln

    delay_collision_checking: 1 # Kollisionsprüfung verzögern -> schneller

# PRM: Probabilistic Roadmap, gut für mehrfache Anfragen im selben Umfeld.

PRMkConfigDefault:

    type: geometric::PRM

# BiTRRT: bidirektionaler Transition-RRT, nützlich bei Kosten-/Constraint-Landschaften.

BiTRRTkConfigDefault:

    type: geometric::BiTRRT


# arm: Planer-Zuordnung für die 6-DoF-Armgruppe.

arm:

    default_planner_config: RRTConnectkConfigDefault

    planner_configs:

        - RRTConnectkConfigDefault

        - RRTstarkConfigDefault

        - PRMkConfigDefault

        - BiTRRTkConfigDefault

# projection_evaluator: definiert die Projektion des Zustandsraums (hier die
# ersten beiden Gelenke), mit der einige Planer den Raum abdecken/diskretisieren.

projection_evaluator: joints(joint2_to_joint1,joint3_to_joint2)

# longest_valid_segment_fraction: Anteil des Zustandsraums, der beim Kollisions-
# Check als ein Segment geprüft wird. Kleiner = feinere Prüfung, weniger Gefahr
# eines "durchtunnelten" Hindernisses, aber höhere Rechenlast.

longest_valid_segment_fraction: 0.005


# gripper: Planer-Zuordnung für die 1-DoF-Greifergruppe (nur RRTConnect nötig).

gripper:

    default_planner_config: RRTConnectkConfigDefault

    planner_configs:

        - RRTConnectkConfigDefault

    longest_valid_segment_fraction: 0.005

```

src/mycobot_moveit_config/config/ros2_controllers.yaml

```
# ros2_control controllers (fake execution mit mock components).

# arm: joint_trajectory_controller (FollowJointTrajectory-Anbindung von MoveIt)

# gripper: joint_trajectory_controller (einfache Positionskontrolle, könnte auch gripper_action_controller sein)

controller_manager:

  ros__parameters:

    # update_rate: Regeltakt des controller_manager in Hz. 100 Hz = alle 10 ms

    # werden Zustände gelesen und Kommandos an die (mock-)Hardware geschrieben.

    update_rate: 100 # Hz

    # joint_state_broadcaster: liest die Gelenkzustände aus der Hardware/Mock

    # und publiziert sie auf /joint_states (kein Regler, nur Zustands-Ausgabe).

    joint_state_broadcaster:

      type: joint_state_broadcaster/JointStateBroadcaster

    # arm_controller: JointTrajectoryController für die 6-DoF-Armkette;

    # empfängt die FollowJointTrajectory-Actions von MoveIt.

    arm_controller:

      type: joint_trajectory_controller/JointTrajectoryController

    # gripper_controller: JointTrajectoryController für das Greifergelenk.

    gripper_controller:

      type: joint_trajectory_controller/JointTrajectoryController

# Parametrierung des Arm-Trajektorienreglers.

arm_controller:

  ros__parameters:

    # joints: die sechs geregelten Armgelenke (Reihenfolge = Index in der Trajektorie).

    joints:

      - joint2_to_joint1

      - joint3_to_joint2

      - joint4_to_joint3

      - joint5_to_joint4

      - joint6_to_joint5

      - joint6output_to_joint6
```

```

# command_interfaces: an die Hardware geschriebene Größen – hier reine
# Positionsvorgabe (der Mock-Hardware genügt Position).

command_interfaces:
    - position

# state_interfaces: von der Hardware gelesene Größen (Position + Geschwindigkeit),
# die zur Regelung und zur Zustandsrückmeldung genutzt werden.

state_interfaces:
    - position
    - velocity

# state_publish_rate: Rate (Hz), mit der der Controller seinen Zustand publiziert.
state_publish_rate: 50.0

# action_monitor_rate: Rate (Hz), mit der der Fortschritt einer laufenden
# FollowJointTrajectory-Action überwacht/rückgemeldet wird.
action_monitor_rate: 20.0

# allow_partial_joints_goal: false = ein Ziel muss ALLE obigen Gelenke enthalten;
# Teilziele werden abgelehnt.
allow_partial_joints_goal: false

constraints:
    # stopped_velocity_tolerance: max. Restgeschwindigkeit (rad/s), bei der ein
    # Gelenk am Zielpunkt noch als "gestoppt" gilt.
    stopped_velocity_tolerance: 0.05

    # goal_time: erlaubte Überschreitung der geplanten Endzeit (s), bevor das
    # Ziel als nicht erreicht gewertet wird.
    goal_time: 0.6

# Parametrierung des Greifer-Trajektorienreglers (analog zum Arm, nur 1 Gelenk).
gripper_controller:
    ros__parameters:
        joints:
            - gripper_controller

        command_interfaces:
            - position

        state_interfaces:
            - position
            - velocity

        state_publish_rate: 50.0

        action_monitor_rate: 20.0

```

| allow_partial_joints_goal: false

src/mycobot_moveit_config/launch/demo.launch.py

```
"""MoveIt 2 Demo Launch – Fake-Execution mit ros2_control fake components.
```

Ausführung:

```
ros2 launch mycobot_moveit_config demo.launch.py
```

Komponenten:

- robot_state_publisher
- ros2_control_node (fake_components/GenericSystem)
- joint_state_broadcaster + arm_controller + gripper_controller spawner
- move_group
- RViz (MotionPlanning Panel)

WICHTIG: Alle Nodes werden mit LC_NUMERIC=C gestartet. Bei System-Locale de_DE wird das Float-Parsing beschädigt (RViz "0.15" → "0", kinematics_solver_timeout wird zu String → loadRobotModel Fehler).

```
"""
```

```
import os
```

```
from pathlib import Path
```

```
from ament_index_python.packages import get_package_share_directory
```

```
from launch import LaunchDescription
```

```
from launch.actions import (
```

```
    DeclareLaunchArgument,
```

```
    IncludeLaunchDescription,
```

```
    SetEnvironmentVariable,
```

```
)
```

```
from launch.conditions import IfCondition
```

```
from launch.launch_description_sources import PythonLaunchDescriptionSource
```

```
from launch_ros.actions import Node
```

```
from launch_ros.substitutions import FindPackageShare
```

```
from launch.substitutions import LaunchConfiguration, PathJoinSubstitution
```

```
from moveit_configs_utils import MoveItConfigsBuilder
```

```
def generate_launch_description():
```

```

# MoveItConfigsBuilder Galactic: robot_name="mycobot" → mycobot_moveit_config Paket

world_share = get_package_share_directory("mycobot_world")

moveit_share = get_package_share_directory("mycobot_moveit_config")

xacro_path = str(Path(world_share) / "urdf" / "mycobot_world.urdf.xacro")

ros2_controllers_path = str(Path(moveit_share) / "config" / "ros2_controllers.yaml")


# Hardware-Auswahl per env – wird von start_mycobot.sh gesetzt.

# Default: echter Roboter. "sim" mode → use_fake_hardware=true.

use_fake_hw = os.environ.get("USE_FAKE_HARDWARE", "false")

robot_port = os.environ.get("MYCOBOT_PORT", "/dev/ttyTHS1")


# MoveItConfigsBuilder bündelt die gesamte MoveIt-Konfiguration aus den YAML-/SRDF-Dateien
# des Pakets zu einem Objekt, das anschließend an move_group und RViz weitergereicht wird.

moveit_config = (

    MoveItConfigsBuilder("mycobot")

    # robot_description: URDF aus der Welt-xacro; die mappings reichen die

    # Hardware-Auswahl (fake vs. echt) und den seriellen Port in die xacro-Argumente durch.

    .robot_description(

        file_path=xacro_path,

        mappings={

            "use_fake_hardware": use_fake_hw,

            "robot_port": robot_port,

        },

    )

    # SRDF: semantische Beschreibung (Planungsgruppen "arm"/"gripper", disable_collisions).

    .robot_description_semantic(file_path="config/mycobot.srdf")

    # Kinematik-Solver-Konfiguration (wird unten durch den TRAC-IK-Override ersetzt).

    .robot_description_kinematics(file_path="config/kinematics.yaml")

    # Gelenkgrenzen (Position/Geschwindigkeit/Beschleunigung) für die Planung.

    .joint_limits(file_path="config/joint_limits.yaml")

    # Zuordnung MoveIt-Planungsgruppe -> ros2_control-Controller (arm/gripper).

    .trajectory_execution(file_path="config/moveit_controllers.yaml")

    # Planungspipelines (OMPL etc.).

    .planning_pipelines()

    # planning_scene_monitor: sorgt dafür, dass die Planungsszene, Geometrie-Updates,

    # Roboterzustand und TF-Änderungen publiziert werden – nötig, damit RViz und externe

    # Nodes (z.B. der Pick-Node, der Kollisionsobjekte einfügt) dieselbe Szene sehen.

```

```

.planning_scene_monitor(
    publish_planning_scene=True,
    publish_geometry_updates=True,
    publish_state_updates=True,
    publish_transforms_updates=True,
)

.to_moveit_configs()

)

moveit_config.robot_description_kinematics = {
    "robot_description_kinematics": {
        "arm": {
            "kinematics_solver": "trac_ik_kinematics_plugin/TRAC_IKKinematicsPlugin",
            "kinematics_solver_search_resolution": float(0.005),
            "kinematics_solver_timeout": float(0.5),
            # False ERFORDERLICH (Sitzung 29 sim A/B-Test): der move_group pose-goal IK
            # Sampler nutzt diesen Parameter; bei True werden tilt-grasp hover
            # Ziele mit "goal konnte nicht gesampelt werden" SOFORT abgelehnt
            # (konsistent mit kinematics.yaml=false und pick_tilt.launch.py=False).
            "position_only_ik": False,
            "solve_type": "Distance",
            "epsilon": float(1e-5),
        }
    }
}

# robot_state_publisher: publiziert aus der URDF (robot_description) + /joint_states den
# TF-Baum. move_group und RViz stützen sich auf diese TF, um Kollisionsgeometrie und
# Roboterzustand räumlich einzuordnen.

robot_state_publisher_node = Node(
    package="robot_state_publisher",
    executable="robot_state_publisher",
    output="screen",
    parameters=[moveit_config.robot_description],
)

# ros2_control_node (controller_manager): lädt die Hardware-Schnittstelle. Je nach

```

```

# use_fake_hardware-Mapping in der URDF ist das die fake_components/GenericSystem
# (Simulation) oder das echte serielle myCobot-Interface. Er bekommt sowohl die URDF als
# auch ros2_controllers.yaml (Controller-Definitionen + Update-Rate).

ros2_control_node = Node(
    package="controller_manager",
    executable="ros2_control_node",
    parameters=[moveit_config.robot_description, ros2_controllers_path],
    output="screen",
)

# joint_state_broadcaster_spawner: aktiviert den Broadcaster, der die Gelenkzustände der
# Hardware-Schnittstelle auf /joint_states publiziert (Grundlage für den TF-Baum).
joint_state_broadcaster_spawner = Node(
    package="controller_manager",
    executable="spawner",
    arguments=["joint_state_broadcaster", "--controller-manager", "/controller_manager"],
)

# arm_controller_spawner: aktiviert den JointTrajectoryController für die 6 Armgelenke –
# er führt die von move_group geplanten Trajektorien aus.
arm_controller_spawner = Node(
    package="controller_manager",
    executable="spawner",
    arguments=["arm_controller", "--controller-manager", "/controller_manager"],
)

# gripper_controller_spawner: aktiviert den Controller für das Greifergelenk.
gripper_controller_spawner = Node(
    package="controller_manager",
    executable="spawner",
    arguments=["gripper_controller", "--controller-manager", "/controller_manager"],
)

# move_group_node: das Herzstück von MoveIt 2 – bietet Planungs-, IK- und
# Ausführungs-Services (Planungspipeline, Kollisionsprüfung, Trajektorien-Execution
# über die oben gespawnten Controller). Es erhält die gesamte MoveIt-Konfiguration
# (URDF, SRDF, Kinematik-Override, Gelenkgrenzen, Controller-Zuordnung) via to_dict().
move_group_node = Node(
    package="moveit_ros_move_group",

```

```

    executable="move_group",

    output="screen",

    parameters=[moveit_config.to_dict()],
)

# mit rviz_config:=<path> kann eine eigene .rviz übergeben werden (z.B. approach.rviz –
# mit Ziel-Marker + 2 Kamera-Images). Leer → Standard moveit.rviz.
rviz_config_file = LaunchConfiguration("rviz_config")
rviz_node = Node(
    package="rviz2",
    executable="rviz2",
    name="rviz2",
    output="log",
    arguments=["-d", rviz_config_file],
    parameters=[
        moveit_config.robot_description,
        moveit_config.robot_description_semantic,
        moveit_config.robot_description_kinematics,
        moveit_config.planning_pipelines,
    ],
    # use_rviz:=false → ohne RViz laufen, um RAM-Thrashing auf dem Jetson Nano zu
    # vermeiden (Schrauben-Verifikation mit joint_pose_gui + tf2_echo benötigt kein RViz).
    condition=IfCondition(LaunchConfiguration("use_rviz")),
)

# estop_relay: NOT-AUS-Brücke – abonniert das gelatchte /estop-Topic (estop_button
# bzw. "ros2 topic pub") und leitet den Zustand als "estop"-Kommando an den
# Kommando-Socket des seriellen Bridge-Prozesses weiter. Nur bei ECHTER Hardware
# sinnvoll (fake hardware startet keinen Bridge); der Bridge stoppt dann sofort
# jede Bewegung (mc.stop, Drehmoment bleibt) und blockiert weitere Kommandos.
estop_relay_node = Node(
    package="mycobot_hardware",
    executable="estop_relay.py",
    output="screen",
)

# ===== Optional: RealSense D435i Live-Kamera =====

```

```

# mit use_camera:=true starten Demo + Live-Kamera zusammen.

# use_camera:=false (default) – nur Simulation, kein Kamera-Node wird gestartet.

use_camera = LaunchConfiguration("use_camera")

realsense_launch = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(
        PathJoinSubstitution([
            FindPackageShare("realsense2_camera"),
            "launch",
            "rs_launch.py",
        ])
    ),
    launch_arguments={
        # pointcloud/align_depth: Punktwolke + zum Farbbild ausgerichtete Tiefe – nötig,
        # damit die Vision-Pipeline aus 2D-Detektionen 3D-Greifpunkte ableiten kann.
        "pointcloud.enable": "true",
        "align_depth.enable": "true",
        # 424x240x15: bewusst niedrige Auflösung/Framerate. Auf dem Jetson Nano führt die
        # volle Auflösung zu "Out of frame resources" bzw. RAM-/USB-Sättigung; 424x240@15
        # ist das getestete stabile Profil für den kompletten Stack.
        "depth_module.profile": "424x240x15",
        "rgb_camera.profile": "424x240x15",
        # Infrarot-, Gyro- und Accel-Streams sind für diese Anwendung ungenutzt und
        # werden abgeschaltet, um USB-Bandbreite und CPU zu sparen.
        "enable_infra1": "false",
        "enable_infra2": "false",
        "enable_gyro": "false",
        "enable_accel": "false",
        # initial_reset: setzt die Kamera-Firmware beim Start zurück – umgeht hängende
        # USB-Zustände nach einem vorherigen ungeplanten Beenden des Treibers.
        "initial_reset": "true",
    }.items(),
    # Nur starten, wenn use_camera:=true gesetzt ist.
    condition=IfCondition(use_camera),
)

launch_entities = [
    # Locale-Fix – Qt/RViz verlässt sich mehr auf LC_ALL/LANG als auf LC_NUMERIC.

```

```

# Alle zusammen auf C festsetzen.

SetEnvironmentVariable(name="LC_ALL", value="C"),

SetEnvironmentVariable(name="LC_NUMERIC", value="C"),

SetEnvironmentVariable(name="LANG", value="C"),

DeclareLaunchArgument(

    "use_camera",

    default_value="false",

    description="true → RealSense D435i Kamera-Node mit starten",

),

DeclareLaunchArgument(

    "use_rviz",

    default_value="true",

    description="false → RViz nicht starten (Jetson Nano RAM-Ersparnis)",

),

DeclareLaunchArgument(

    "rviz_config",

    default_value=PathJoinSubstitution(

        [FindPackageShare("mycobot_moveit_config"), "rviz", "moveit.rviz"]

    ),

    description="Pfad zur RViz-Config-Datei (Override fuer eigene Ziel+Kamera-Config)",

),

robot_state_publisher_node,

ros2_control_node,

joint_state_broadcaster_spawner,

arm_controller_spawner,

gripper_controller_spawner,

move_group_node,

rviz_node,

realsense_launch,

]

# NOT-AUS-Relay nur bei echter Hardware anhängen (Python-Bedingung statt

# IfCondition, weil die Hardware-Auswahl per Umgebungsvariable erfolgt).

if use_fake_hw.strip().lower() not in ("true", "1"):

    launch_entities.append(estop_relay_node)

return LaunchDescription(launch_entities)

```

src/mycobot_moveit_config/launch/moveit_rviz.launch.py

"""RViz-only launch – um sich mit einem BEREITS LAUFENDEN move_group zu verbinden.

Wird in run_pick_preview.sh Phase B verwendet: der Stack (move_group) läuft separat, dieser Launch startet NUR RViz mit den MoveIt-Parametern (robot_description + SRDF + kinematics + planning_pipelines). `ros2 run rviz2 rviz2 -d ...` erhält diese Parameter nicht und gibt daher "No Planning Scene Loaded" + Roboter unsichtbar Fehler; dieser Launch löst das.

```
ros2 launch mycobot_moveit_config moveit_rviz.launch.py rviz_config:=<path>
```

LC_NUMERIC=C erforderlich (Qt/RViz beschädigt Float-Parsing bei de_DE Locale).

"""

```
from pathlib import Path
```

```
from ament_index_python.packages import get_package_share_directory
```

```
from launch import LaunchDescription
```

```
from launch.actions import DeclareLaunchArgument, SetEnvironmentVariable
```

```
from launch.substitutions import LaunchConfiguration, PathJoinSubstitution
```

```
from launch_ros.actions import Node
```

```
from launch_ros.substitutions import FindPackageShare
```

```
from moveit_configs_utils import MoveItConfigsBuilder
```

```
import os
```

```
def generate_launch_description():
```

```
    world_share = get_package_share_directory("mycobot_world")
```

```
    xacro_path = str(Path(world_share) / "urdf" / "mycobot_world.urdf.xacro")
```

```
    use_fake_hw = os.environ.get("USE_FAKE_HARDWARE", "false")
```

```
    robot_port = os.environ.get("MYCOBOT_PORT", "/dev/ttyTHS1")
```

```
    # Dieselbe MoveIt-Konfiguration wie in demo.launch.py aufbauen, ABER ohne
```

```
    # trajectory_execution/planning_scene_monitor – dieser Launch startet kein move_group,
```

```
    # sondern nur RViz. RViz braucht lediglich robot_description, SRDF, Kinematik und
```

```
    # Planungspipelines, um das Roboter- und Planungs-Panel korrekt anzuzeigen.
```

```
    moveit_config = (
```

```
        MoveItConfigsBuilder("mycobot")
```



```

.robot_description(
    file_path=xacro_path,
    mappings={"use_fake_hardware": use_fake_hw, "robot_port": robot_port},
)

.robot_description_semantic(file_path="config/mycobot.srdf")

.robot_description_kinematics(file_path="config/kinematics.yaml")

.joint_limits(file_path="config/joint_limits.yaml")

.planning_pipelines()

.to_moveit_configs()

)

# Gleicher kinematics Override wie demo.launch.py (TRAC-IK, position_only_ik=False).
moveit_config.robot_description_kinematics = {
    "robot_description_kinematics": {
        "arm": {
            "kinematics_solver": "trac_ik_kinematics_plugin/TRAC_IKKinematicsPlugin",
            "kinematics_solver_search_resolution": float(0.005),
            "kinematics_solver_timeout": float(0.5),
            "position_only_ik": False,
            "solve_type": "Distance",
            "epsilon": float(1e-5),
        }
    }
}

# rviz_config_file: Pfad zur .rviz-Datei (per Argument überschreibbar). Der RViz-Node
# erhält zusätzlich die MoveIt-Parameter, ohne die er "No Planning Scene Loaded" meldet
# und den Roboter nicht rendern kann (siehe Modul-Docstring).

rviz_config_file = LaunchConfiguration("rviz_config")

rviz_node = Node(
    package="rviz2",
    executable="rviz2",
    name="rviz2",
    output="screen",
    arguments=["-d", rviz_config_file],
    parameters=[
        moveit_config.robot_description,

```

```

        moveit_config.robot_description_semantic,

        moveit_config.robot_description_kinematics,

        moveit_config.planning_pipelines,

    ],

)

return LaunchDescription([

    # Locale-Fix (siehe Docstring): Qt/RViz beschädigt sonst das Float-Parsing bei de_DE.

    SetEnvironmentVariable("LC_NUMERIC", "C"),

    SetEnvironmentVariable("LC_ALL", "C"),

    SetEnvironmentVariable("LANG", "C"),

    DeclareLaunchArgument(

        "rviz_config",

        default_value=PathJoinSubstitution(

            [FindPackageShare("mycobot_moveit_config"), "rviz", "moveit.rviz"]

        ),

        description="Pfad zur RViz-Config-Datei",

    ),

    rviz_node,

])

```

src/mycobot_moveit_config/package.xml

```
<?xml version="1.0"?>

<package format="3">

  <name>mycobot_moveit_config</name>

  <version>0.1.0</version>

  <description>myCobot 280 JN için MoveIt 2 konfigürasyon paketi. Robot: mycobot_world (arm + adaptive gripper + D435i).</description>

  <maintainer email="ekinciorhandijvar@gmail.com">er</maintainer>

  <license>Apache-2.0</license>


  <buildtool_depend>ament_cmake</buildtool_depend>


  <exec_depend>mycobot_world</exec_depend>

  <exec_depend>mycobot_description</exec_depend>

  <exec_depend>realsense2_description</exec_depend>


  <exec_depend>moveit_ros_move_group</exec_depend>

  <exec_depend>moveit_kinematics</exec_depend>

  <exec_depend>moveit_planners_ompl</exec_depend>

  <exec_depend>moveit_ros_visualization</exec_depend>

  <exec_depend>moveit_ros_warehouse</exec_depend>

  <exec_depend>moveit_simple_controller_manager</exec_depend>


  <exec_depend>joint_state_publisher</exec_depend>

  <exec_depend>joint_state_publisher_gui</exec_depend>

  <exec_depend>robot_state_publisher</exec_depend>

  <exec_depend>rviz2</exec_depend>

  <exec_depend>xacro</exec_depend>

  <exec_depend>tf2_ros</exec_depend>


  <export>

    <build_type>ament_cmake</build_type>

  </export>

</package>
```

src/mycobot_moveit_config/rviz/moveit.rviz

Panels:

- Class: rviz_common/Displays

Help Height: 78

Name: Displays

Property Tree Widget:

Expanded: null

Splitter Ratio: 0.5

Tree Height: 78

Visualization Manager:

Class: ''

Displays:

- Alpha: 0.5

Cell Size: 0.10000000149011612

Class: rviz_default_plugins/Grid

Color: 160; 160; 164

Enabled: true

Line Style:

Line Width: 0.029999999329447746

Value: Lines

Name: Grid

Normal Cell Count: 0

Offset:

X: 0

Y: 0

Z: 0

Plane: XY

Plane Cell Count: 10

Reference Frame: <Fixed Frame>

Value: true

- Acceleration_Scaling_Factor: 0.1

Class: moveit_rviz_plugin/MotionPlanning

Enabled: true

Move Group Namespace: ''

MoveIt_Allow_Approximate_IK: false

MoveIt_Allow_External_Program: false

MoveIt-Allow_Replanning: false

MoveIt-Allow_Sensor_Positioning: false

MoveIt_Planning_Attempts: 10

MoveIt_Planning_Time: 5

MoveIt_Use_Cartesian_Path: false

MoveIt_Use_Constraint_Aware_IK: true

MoveIt_Workspace:

Center:

X: 0

Y: 0

Z: 0

Size:

X: 2

Y: 2

Z: 2

Name: MotionPlanning

Planned Path:

Color Enabled: false

Interrupt Display: false

Links:

All Links Enabled: true

Expand Joint Details: false

Expand Link Details: false

Expand Tree: false

Link Tree Style: Links in Alphabetic Order

Loop Animation: false

Robot Alpha: 0.5

Robot Color: 150; 50; 150

Show Robot Collision: false

Show Robot Visual: true

Show Trail: false

State Display Time: 0.05 s

Trail Step Size: 1

Trajectory Topic: display_planned_path

Planning Metrics:

Payload: 1

Show Joint Torques: false

Show Manipulability: false

Show Manipulability Index: false

Show Weight Limit: false

TextHeight: 0.07999999821186066

Planning Request:

Colliding Link Color: 255; 0; 0

Goal State Alpha: 1

Goal State Color: 250; 128; 0

Interactive Marker Size: 0.029999999329447746

Joint Violation Color: 255; 0; 255

Planning Group: arm

Query Goal State: true

Query Start State: false

Show Workspace: false

Start State Alpha: 1

Start State Color: 0; 255; 0

Planning Scene Topic: monitored_planning_scene

Robot Description: robot_description

Scene Geometry:

Scene Alpha: 0.8999999761581421

Scene Color: 50; 230; 50

Scene Display Time: 9.999999747378752e-05

Show Scene Geometry: true

Voxel Coloring: Z-Axis

Voxel Rendering: Occupied Voxels

Scene Robot:

Attached Body Color: 150; 50; 150

Links:

All Links Enabled: true

Expand Joint Details: false

Expand Link Details: false

Expand Tree: false

Link Tree Style: Links in Alphabetic Order

Robot Alpha: 1

Show Robot Collision: false

Show Robot Visual: true

Value: true

```
Velocity_Scaling_Factor: 0.1

- Class: rviz_default_plugins/MarkerArray

  Enabled: true

  Name: ReachRings

  Namespaces: {}

  Topic:

    Depth: 5

    Durability Policy: Transient Local

    History Policy: Keep Last

    Reliability Policy: Reliable

    Value: /reach_rings

  Value: true

- Class: rviz_default_plugins/PointCloud2

  Color Transformer: RGB8

  Decay Time: 0

  Enabled: false

  Name: DepthCloud

  Position Transformer: XYZ

  Size (m): 0.003

  Style: Points

  Topic:

    Depth: 5

    Durability Policy: Volatile

    History Policy: Keep Last

    Reliability Policy: Best Effort

    Value: /camera/depth/color/points

  Value: false

Enabled: true

Global Options:

  Background Color: 48; 48; 48

  Fixed Frame: robot_base

  Frame Rate: 30

Name: root

Tools:

- Class: rviz_default_plugins/Interact

  Hide Inactive Objects: true

- Class: rviz_default_plugins/MoveCamera
```

- Class: rviz_default_plugins/Select
- Class: rviz_default_plugins/FocusCamera

Transformation:

Current:

Class: rviz_default_plugins/TF

Value: true

Views:

Current:

Class: rviz_default_plugins/Orbit

Distance: 0.5159747004508972

Enable Stereo Rendering:

Value: false

Focal Point:

X: 0

Y: 0

Z: 0.30000001192092896

Focal Shape Fixed Size: true

Focal Shape Size: 0.05000000074505806

Invert Z Axis: false

Name: Current View

Near Clip Distance: 0.00999999776482582

Pitch: 0.8799999952316284

Target Frame: <Fixed Frame>

Value: Orbit (rviz_default_plugins)

Yaw: 5.858185291290283

Saved: null

Window Geometry:

Displays:

collapsed: false

Height: 700

Hide Left Dock: false

Hide Right Dock: true

Width: 1024

X: 0

Y: 24

src/mycobot_world/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.5)

project(mycobot_world)


find_package(ament_cmake REQUIRED)


install(DIRECTORY urdf launch rviz
  DESTINATION share/${PROJECT_NAME}
)

ament_package()
```

src/mycobot_world/launch/world.launch.py

```
"""myCobot 280 JN + adaptive gripper + eye-to-hand RealSense D435i – RViz display.
```

Verwendung:

```
ros2 launch mycobot_world world.launch.py
```

```
ros2 launch mycobot_world world.launch.py use_gui:=false
```

```
"""
```

```
import os
```

```
from ament_index_python.packages import get_package_share_directory
```

```
from launch import LaunchDescription
```

```
from launch.actions import DeclareLaunchArgument
```

```
from launch.conditions import IfCondition
```

```
from launch.substitutions import Command, LaunchConfiguration, PathJoinSubstitution
```

```
from launch_ros.actions import Node
```

```
from launch_ros.parameter_descriptions import ParameterValue
```

```
from launch_ros.substitutions import FindPackageShare
```

```
def generate_launch_description():
```

```
    pkg = FindPackageShare("mycobot_world")
```

```
    # xacro_path: die Welt-URDF (Roboter + Greifer + fest montierte D435i). Command(["xacro ..."])
```

```
    # expandiert sie zur Laufzeit zu vollständigem URDF-XML.
```

```
    xacro_path = PathJoinSubstitution([pkg, "urdf", "mycobot_world.urdf.xacro"])
```

```
    # rviz_config: Anzeige-Layout dieser Welt-Visualisierung.
```

```
    rviz_config = PathJoinSubstitution([pkg, "rviz", "world.rviz"])
```

```
    # use_gui: schaltet die Slider-GUI zu (true) oder lässt sie weg (false), z.B. wenn die
```

```
    # Gelenkzustände von einer anderen Quelle (echte Bridge) kommen.
```

```
    use_gui = LaunchConfiguration("use_gui")
```

```
    # robot_description: expandiertes URDF als geteilter String-Parameter für rsp + Slider-GUI.
```

```
    robot_description = {
```

```
        "robot_description": ParameterValue(
```

```
            Command(["xacro ", xacro_path]),
```

```

        value_type=str,
    ),
}

return LaunchDescription([
    DeclareLaunchArgument(
        "use_gui",
        default_value="true",
        description="Joint-Slider-GUI starten.",
    ),

    # robot_state_publisher: erzeugt aus URDF + /joint_states den TF-Baum, den RViz zum
    # Rendern von Roboter und Kamera-Frame nutzt.
    Node(
        package="robot_state_publisher",
        executable="robot_state_publisher",
        name="robot_state_publisher",
        output="screen",
        parameters=[robot_description],
    ),

    # joint_state_publisher_gui: reine Visualisierungs-GUI (KEIN Remap auf /joint_commands,
    # da hier kein echter Roboter angesprochen wird) – die Slider publizieren direkt auf
    # /joint_states, damit man das Modell in RViz durchbewegen kann. Nur bei use_gui:=true.
    Node(
        package="joint_state_publisher_gui",
        executable="joint_state_publisher_gui",
        name="joint_state_publisher_gui",
        output="screen",
        parameters=[robot_description],
        condition=IfCondition(use_gui),
    ),

    # rviz2: startet die 3D-Visualisierung mit der world.rviz-Config.
    Node(
        package="rviz2",
        executable="rviz2",

```

```
    name="rviz2",  
    output="screen",  
    arguments=["-d", rviz_config],  
),  
1)
```

src/mycobot_world/package.xml

```
<?xml version="1.0"?>

<package format="3">

  <name>mycobot_world</name>

  <version>0.1.0</version>

  <description>Sahne paketi: myCobot 280 JN + adaptive gripper + eye-to-hand RealSense D435i.</description>

  <maintainer email="ekinciorhandijvar@gmail.com">er</maintainer>

  <license>Apache-2.0</license>


  <buildtool_depend>ament_cmake</buildtool_depend>


  <exec_depend>mycobot_description</exec_depend>

  <exec_depend>realsense2_description</exec_depend>

  <exec_depend>robot_state_publisher</exec_depend>

  <exec_depend>joint_state_publisher_gui</exec_depend>

  <exec_depend>rviz2</exec_depend>

  <exec_depend>xacro</exec_depend>


  <export>

    <build_type>ament_cmake</build_type>

  </export>

</package>
```

src/mycobot_world/rviz/world.rviz

Panels:

- Class: rviz_common/Displays

Help Height: 78

Name: Displays

Property Tree Widget:

Expanded:

- /Global Options1

- /Status1

- /TF1

- /RobotModel1

Splitter Ratio: 0.5

Tree Height: 677

- Class: rviz_common/Selection

Name: Selection

- Class: rviz_common/Tool Properties

Expanded:

- /2D Pose Estimate1

- /Publish Point1

Name: Tool Properties

Splitter Ratio: 0.5886790156364441

- Class: rviz_common/Views

Expanded:

- /Current View1

Name: Views

Splitter Ratio: 0.5

Visualization Manager:

Class: ""

Displays:

- Alpha: 0.5

Cell Size: 1

Class: rviz_default_plugins/Grid

Color: 160; 160; 164

Enabled: true

Line Style:

Line Width: 0.029999999329447746

Value: Lines

Name: Grid

Normal Cell Count: 0

Offset:

X: 0

Y: 0

Z: 0

Plane: XY

Plane Cell Count: 10

Reference Frame: <Fixed Frame>

Value: true

- Class: rviz_default_plugins/TF

Enabled: true

Frame Timeout: 15

Frames:

All Enabled: true

gripper_base:

Value: true

gripper_left1:

Value: true

gripper_left2:

Value: true

gripper_left3:

Value: true

gripper_right1:

Value: true

gripper_right2:

Value: true

gripper_right3:

Value: true

joint1:

Value: true

joint2:

Value: true

joint3:

Value: true

joint4:

Value: true

joint5:

Value: true

joint6:

Value: true

joint6_flange:

Value: true

Marker Scale: 0.10000000149011612

Name: TF

Show Arrows: true

Show Axes: true

Show Names: true

Tree:

joint1:

joint2:

joint3:

joint4:

joint5:

joint6:

joint6_flange:

gripper_base:

gripper_left2:

{}

gripper_left3:

gripper_left1:

{}

gripper_right2:

{}

gripper_right3:

gripper_right1:

{}

Update Interval: 0

Value: true

- Alpha: 1

Class: rviz_default_plugins/RobotModel

Collision Enabled: false

Description File: ""

Description Source: Topic

Description Topic:

Depth: 5

Durability Policy: Volatile

History Policy: Keep Last

Reliability Policy: Reliable

Value: /robot_description

Enabled: true

Links:

All Links Enabled: true

Expand Joint Details: false

Expand Link Details: false

Expand Tree: false

Link Tree Style: Links in Alphabetic Order

gripper_base:

Alpha: 1

Show Axes: false

Show Trail: false

Value: true

gripper_left1:

Alpha: 1

Show Axes: false

Show Trail: false

Value: true

gripper_left2:

Alpha: 1

Show Axes: false

Show Trail: false

Value: true

gripper_left3:

Alpha: 1

Show Axes: false

Show Trail: false

Value: true

gripper_right1:

Alpha: 1

Show Axes: false

Show Trail: false

Value: true

gripper_right2:

Alpha: 1

Show Axes: false

Show Trail: false

Value: true

gripper_right3:

Alpha: 1

Show Axes: false

Show Trail: false

Value: true

joint1:

Alpha: 1

Show Axes: false

Show Trail: false

Value: true

joint2:

Alpha: 1

Show Axes: false

Show Trail: false

Value: true

joint3:

Alpha: 1

Show Axes: false

Show Trail: false

Value: true

joint4:

Alpha: 1

Show Axes: false

Show Trail: false

Value: true

joint5:

Alpha: 1

Show Axes: false

Show Trail: false

Value: true

joint6:

Alpha: 1

Show Axes: false

Show Trail: false

Value: true

joint6_flange:

Alpha: 1

Show Axes: false

Show Trail: false

Value: true

Name: RobotModel

TF Prefix: ""

Update Interval: 0

Value: true

Visual Enabled: true

- Class: rviz_default_plugins/Image

Enabled: true

Max Value: 1

Median window: 5

Min Value: 0

Name: ChArUco Detection

Normalize Range: true

Topic:

Depth: 5

Durability Policy: Volatile

History Policy: Keep Last

Reliability Policy: Reliable

Value: /charuco_detector/image_annotated

Value: true

Enabled: true

Global Options:

Background Color: 48; 48; 48

Fixed Frame: robot_base

Frame Rate: 30

Name: root

Tools:

- Class: rviz_default_plugins/Interact

Hide Inactive Objects: true

- Class: rviz_default_plugins/MoveCamera
- Class: rviz_default_plugins/Select
- Class: rviz_default_plugins/FocusCamera
- Class: rviz_default_plugins/Measure

Line color: 128; 128; 0

- Class: rviz_default_plugins/SetInitialPose

Topic:

Depth: 5

Durability Policy: Volatile

History Policy: Keep Last

Reliability Policy: Reliable

Value: /initialpose

- Class: rviz_default_plugins/SetGoal

Topic:

Depth: 5

Durability Policy: Volatile

History Policy: Keep Last

Reliability Policy: Reliable

Value: /goal_pose

- Class: rviz_default_plugins/PublishPoint

Single click: true

Topic:

Depth: 5

Durability Policy: Volatile

History Policy: Keep Last

Reliability Policy: Reliable

Value: /clicked_point

Transformation:

Current:

Class: rviz_default_plugins/TF

Value: true

Views:

Current:

Class: rviz_default_plugins/Orbit

Distance: 1.2028908729553223

Enable Stereo Rendering:

Stereo Eye Separation: 0.05999999865889549

Stereo Focal Distance: 1

Swap Stereo Eyes: false

Value: false

Focal Point:

X: -0.07064759731292725

Y: -0.0814988762140274

Z: 0.10758385062217712

Focal Shape Fixed Size: true

Focal Shape Size: 0.05000000074505806

Invert Z Axis: false

Name: Current View

Near Clip Distance: 0.009999999776482582

Pitch: 0.44039833545684814

Target Frame: <Fixed Frame>

Value: Orbit (rviz)

Yaw: 0.43038973212242126

Saved: ~

Window Geometry:

Displays:

collapsed: false

Height: 906

Hide Left Dock: false

Hide Right Dock: false

QMainWindow State:

000000ff00000000fd00000004000000000000016a00000330fc0200000008fb00000001200530065006c0065006300740069006f006e0000000
1e10000009b0000005c00fffffffb0000001e0054006f006f006c002000500072006f007000650072007400690065007302000001ed000001df
00000185000000a3fb000000120056006900650077007300200054006f006f02000001df000002110000018500000122fb000000200054006f0
06f006c002000500072006f0070006500720074006900650073003203000002880000011d000002210000017afb000000100044006900730070
006c006100790073010000003d000000330000000c900fffffffb00000002000730065006c0065006300740069006f006e0020006200750066006
6006500720200000138000000aa0000023a00000294fb00000014005700690064006500530074006500720065006f02000000e6000000d20000
03ee0000030bfb0000000c004b0069006e0065006300740200000186000001060000030c00000261000000010000022f00000330fc020000000
3fb0000001e0054006f006f006c002000500072006f007000650072007400690065007301000000410000007800000000000000fb0000000a
00560069006500770073010000003d000000330000000a400fffffffb0000001200530065006c0065006300740069006f006e010000025a00000
0b20000000000000000000000000200000490000000a9fc0100000001fb0000000a00560069006500770073030000004e00000080000002e10000
01970000000300000073f0000003efc0100000002fb0000000800540069006d006501000000000000073f00000000000000fb0000000800540
069006d006501000000000000450000000000000000000039300000330000000400000040000000800000008fc00000001000000020000
00010000000a0054006f006f006c00730100000000fffffffb0000000000000000

Selection:

collapsed: false

Tool Properties:

collapsed: false

Views:

collapsed: false

Width: 1848

X: 72

Y: 27

src/mycobot_world/urdf/mycobot_world.urdf.xacro

```
<?xml version="1.0"?>

<robot name="mycobot_world" xmlns:xacro="http://www.ros.org/wiki/xacro">

  <!--
    mycobot_280 JN + adaptive gripper + eye-to-hand RealSense D435i.

    Frame-Hierarchie (Phase 1 stabilisiert):

      robot_base  (URDF-Wurzel, MoveIt-Planung Haupt-Referenzframe)
      |
      |— joint1 → ... → joint6_flange  (Arm-Kette, upstream URDF)
      |
      |   └─ gripper_base → fingers
      |
      |       └─ tcp  (Greifer-Spitze, Pick-&-Place-Zielframe)
      |
      └─ camera_link  (D435i, realsense2_description-Makro)

    - tcp-Pose ungefähr (~9 cm gripper_base z+) – später mit präziser Messung justierbar.
    - Kamera-Pose Platzhalter, nach Hand-Eye-Kalibrierung (Phase 5) überschrieben.
  -->

  <xacro:property name="PI" value="3.14159265358979323846"/>

  <!-- Haupt-Referenzframe (Master-Plan vorgeschrieben) +

    physisches Basis-Gehäuse des Roboters (Box mit USB/HMI-Anschluss, zwischen Arm und Boden).
    ~15×11×11 cm, auf Armachse zentriert, sitzt auf der Plattform (z=0) auf → z=0..0.11.

    HINZUGEFUEGT 2026-06-15: zuvor war robot_base ein leeres Link → Gehäuse fehlte im
    Kollisionsmodell → bei sehr naher Schraube faltet sich der Arm und stößt an dieses
    Gehäuse, MoveIt sah es nicht. Nur-Kollision (Visual ist bereits im joint1-Mesh). -->

  <link name="robot_base">

    <collision>

      <origin xyz="0 0 0.055" rpy="0 0 0"/>

      <geometry><box size="0.15 0.11 0.11"/></geometry>

    </collision>

  </link>

  <!--

    Physische Infrastruktur: Plattform auf der der Roboter steht + vertikale Kamera-Stütze.

    Diese Primitive sind Teil des dauerhaften Aufbaus (kein temporäres Debug) – müssen für
```

Kollision aktiv bleiben, damit die Planung nicht an Boden/Säule stößt. Visual halbtransparent, sodass die Roboter-Meshes dominant erscheinen.

-->

<!-- Plattform: 50×40×2.5 cm. Obere Fläche z=0 (Roboter-Basis sitzt hier auf). -->

<link name="platform_top">

<visual>

<origin xyz="0 0 0" rpy="0 0 0"/>

<geometry><box size="0.50 0.40 0.025"/></geometry>

<material name="platform_color">

<color rgba="0.35 0.35 0.40 0.55"/>

</material>

</visual>

<collision>

<origin xyz="0 0 0" rpy="0 0 0"/>

<geometry><box size="0.50 0.40 0.025"/></geometry>

</collision>

</link>

<joint name="robot_base_to_platform" type="fixed">

<parent link="robot_base"/>

<child link="platform_top"/>

<origin xyz="0.15 -0.125 -0.0125" rpy="0 0 0"/>

</joint>

<!--

Kaiser RS 2 XA Repro-Stand (Baseboard ist platform_top oben).

Drei Teile: Halte-Klemme (tutucu_kelepce), vertikale Säule (column),

von column zum Roboter reichender horizontaler Kamera-Arm (camera_arm).

Geometrie (robot_base frame):

- Halter: an der gegenüberliegenden Kante des Roboters (+X), 11 cm lang in Y-Richtung
- Stab: 2.5 × 3.5 cm rechteckiger Querschnitt, 76 cm Länge, tritt aus dem Halter aus
- Kamera-Arm: 12.5 cm (2026-06-07), ein Ende berührt die Kamera (~0.21), das andere Ende verbindet sich mit column (~0.335). Kamera-Pose aus Kalibrierung fix; column+Klemme ~5cm näher an Roboter gebracht, sodass der Stab an beide Enden anschließt.

-->


```

<link name="tutucu_kelepce">

  <visual>

    <origin xyz="0 0 0" rpy="0 0 0"/>

    <geometry><box size="0.05 0.11 0.07"/></geometry>

    <material name="mount_color">

      <color rgba="0.15 0.15 0.18 0.85"/>

    </material>

  </visual>

  <collision>

    <origin xyz="0 0 0" rpy="0 0 0"/>

    <geometry><box size="0.05 0.11 0.07"/></geometry>

  </collision>

</link>

<joint name="robot_base_to_tutucu_kelepce" type="fixed">

  <parent link="robot_base"/>

  <child link="tutucu_kelepce"/>

  <!-- 2026-06-25: zur Mitte der fernen Kante der Platte zurückgesetzt (0.3225 -> 0.3725).

    Die vorherige Position vom 2026-06-07 hatte den Stab ~5cm vor die Plattenkante verschoben. -->

  <origin xyz="0.3725 -0.125 0.035" rpy="0 0 0"/>

</joint>

<link name="column">

  <visual>

    <origin xyz="0 0 0" rpy="0 0 0"/>

    <geometry><box size="0.025 0.035 0.76"/></geometry>

    <material name="column_color">

      <color rgba="0.20 0.20 0.25 0.85"/>

    </material>

  </visual>

  <collision>

    <origin xyz="0 0 0" rpy="0 0 0"/>

    <geometry><box size="0.025 0.035 0.76"/></geometry>

  </collision>

</link>

<joint name="robot_base_to_column" type="fixed">

  <parent link="robot_base"/>

  <child link="column"/>

```

```

<!-- 2026-06-25: zur Mitte der fernen Kante der Platte zurückgesetzt (0.335 -> 0.385);

Außenfläche x=0.3975 ≈ Plattenkante 0.40. Y=-0.125 = Platten-Y-Mitte.

(Das vorherige 0.335 hatte den Stab ~5cm vor die Kante verschoben; der Kamera-Arm
war deshalb auf 12.5cm verkürzt worden – jetzt ist der Arm wieder verlängert.) -->

<origin xyz="0.385 -0.125 0.38" rpy="0 0 0"/>
</joint>

<link name="camera_arm">

  <visual>

    <origin xyz="0 0 0" rpy="0 0 0"/>

    <geometry><box size="0.229 0.025 0.025"/></geometry>

    <material name="arm_color">

      <color rgba="0.20 0.20 0.25 0.85"/>

    </material>

  </visual>

  <collision>

    <origin xyz="0 0 0" rpy="0 0 0"/>

    <geometry><box size="0.229 0.025 0.025"/></geometry>

  </collision>

</link>

<joint name="robot_base_to_camera_arm" type="fixed">

  <parent link="robot_base"/>

  <child link="camera_arm"/>

  <!-- 2026-07-14: nach Neukalibrierung (Kamera ~48cm -> ~66cm angehoben + näher an den Roboter)

wurde der Arm an die neue Kamera-Aufhängung angepasst, damit KEINE Lücke zwischen Stab
und Kamera bleibt.

z auf die neue Kamera-Aufhängehöhe: 0.486 -> 0.670 (= neues camera_bottom_screw z=0.66987).

2026-08-31: Kamera vom Benutzer auf 45 cm HERUNTERGESETZT -> z 0.670 -> 0.450

(= neues camera_bottom_screw z=0.45000). x/Länge unverändert (rein vertikale
Verschiebung entlang der column); column (0->0.76 m) weiterhin hoch genug.

x: nahes Ende = neue Kamera x=0.156 (früher 0.21, Kamera näher gerückt), fernes Ende
berührt bei 0.385 die column. Länge=0.385-0.156=0.229. Mitte=(0.156+0.385)/2=0.2705.

column (0->0.76m) ist hoch genug, unverändert. -->

  <origin xyz="0.2705 -0.125 0.450" rpy="0 0 0"/>
</joint>

<!-- mycobot-Arm + adaptive gripper: upstream-URDF-Inhalt -->

```

```

<xacro:include filename="$(find mycobot_description)/urdf/mycobot_280_jn/mycobot_280_jn_adaptive_gripper.urdf"/>

<!-- Wurzel des Roboterarms (joint1) mit robot_base verbunden -->
<joint name="robot_base_to_joint1" type="fixed">
  <parent link="robot_base"/>
  <child link="joint1"/>
  <origin xyz="0 0 0" rpy="0 0 0"/>
</joint>

<!--
  TCP (Tool Center Point) – Mittelpunkt zwischen den beiden Greifer-Fingern.
  Pose etwa 9 cm voraus in z-Achsen-Richtung von gripper_base.
  Master-Plan: Pick/Place-Bewegungen werden vom tcp-Frame aus gesteuert.
-->
<link name="tcp"/>
<joint name="gripper_base_to_tcp" type="fixed">
  <parent link="gripper_base"/>
  <child link="tcp"/>
  <!-- Greifer-Körper gemessen = 110mm (2026-06-07): 0.105 -> 0.110. -->
  <origin xyz="0 0.110 -0.01" rpy="0 0 0"/>
</joint>

<!-- IntelRealSense offizielles D435i-URDF-Makro -->
<xacro:include filename="$(find realsense2_description)/urdf/_d435i.urdf.xacro"/>

<!--
  Eye-to-hand-Kamera-Pose (Platzhalter – wird nach Phase 5 Hand-Eye aktualisiert):
    x = 0.40 m (40 cm vor dem Roboter)
    y = 0.00 m (Mitte)
    z = 0.50 m (50 cm über dem Roboter)
    pitch = pi/2 rad (Linse genau nach unten – realer Aufbau: Kamera an der Decke, senkrecht zum Boden)
    yaw = pi (Bildoberseite zeigt zur Rückseite des Roboters – bei Bedarf angepasst)
-->
<!-- KALIBRIERUNG (2026-07-14, easy_handeye2 eye-on-base, OpenCV/Park, 16 Posen, 6x3 Greifer-Board):
  robot_base->camera_color_optical_frame = trans(0.14406,-0.14646,0.65809)
  quat(0.70446,0.70947,0.01973,0.00212). Kamera höher gesetzt: ~47cm -> ~66cm.
  Der untenstehende Makro-Origin (robot_base->camera_bottom_screw_frame) wurde aus diesem

```

Ergebnis abgeleitet: $T_{origin} = T_{calib} \cdot (bottom_screw \rightarrow optical)^{-1}$ (numerisch, gegen die alte Kalibrierung verifiziert: reproduziert altes Origin auf $\sim 0.3\text{mm}/0.4\text{deg}$).
Park/Horad/Daniilidis stimmen auf $\sim 1\text{cm}$ überein $\rightarrow$ z GUT bestimmt, kein Kollaps.
Tsai-Lenz weiterhin unzuverlässig (z/Rotation abweichend) $\rightarrow$ NICHT verwendet.
Früher (2026-06-02): $trans(0.197, -0.143, 0.475)$, Origin $0.20961/-0.11021/0.48567$, $\sim 47\text{cm}$.

*** 2026-08-31 – KAMERA AUF 45 cm HERUNTERGESETZT (GEOMETRISCHE NACHFÜHRUNG, KEINE NEUE KALIBRIERUNG) ***

Der Benutzer hat die Kamera von $\sim 66\text{ cm}$ auf 45 cm abgesenkt. Angenommen wurde eine REIN VERTIKALE Verschiebung entlang der column: x, y und rpy bleiben unverändert, nur z: $0.66987 \rightarrow 0.45000$ (Delta -0.21987 m).

Daraus folgt $robot_base \rightarrow camera_color_optical_frame$ z: $0.65809 \rightarrow 0.43822$ ($x=0.14406$, $y=-0.14646$ unverändert).

⚠ Das ist eine GEOMETRISCHE SCHÄTZUNG, kein Hand-Auge-Ergebnis. Reale Neigungs- und XY-Änderungen der Halterung sind darin NICHT erfasst. Vor Präzisions-Picks: easy_handeye2-Neukalibrierung (Park/Horad, Tsai-Lenz verboten) fahren und diesen Origin ersetzen. Ebenfalls veraltet: `.place_offset.json`, `GRASP_Z_OFFSET`, `calib_check_points.json` (am 66-cm -Extrinsic gemessen). $\rightarrow$

```
<xacro:sensor_d435i parent="robot_base" use_nominal_extrinsics="true" name="camera" use_mesh="true">
  <origin xyz="0.15598 -0.11415 0.45000" rpy="-2.46695 1.53111 0.68173"/>
</xacro:sensor_d435i>
```

<!--

ros2_control hardware interface.

Phase 6: mycobot hardware/MyCobotHardware (C++ + pybind11 + pymycobot).

Mit `use_fake_hardware:=true` wird auf `fake_components/GenericSystem` zurückgegriffen.

-->

```
<xacro:arg name="use_fake_hardware" default="false"/>
<xacro:arg name="robot_port" default="/dev/ttyTHS1"/>
<xacro:arg name="robot_baud" default="1000000"/>
```

<!-- command_speed: Servo-Geschwindigkeit, die die Bridge jedem send_radians gibt (0-100, offene Schleife).

Freiraum (home/hover/Rückkehr-zu-Null) wird real durch DIESEN Wert begrenzt (nicht MoveIt-velocity-scaling). 15-30: Freiraum $\sim 2\times$ schneller; descend bleibt großteils erhalten, da er durch langsames

MoveIt-scaling zusätzlich begrenzt ist. Wenn descend ruckelt, auf 20-25 reduzieren. $\rightarrow$

```
<xacro:arg name="command_speed" default="30"/>
```

```
<ros2_control name="mycobot_hardware" type="system">
```

```

<hardware>

  <xacro:if value="$(arg use_fake_hardware)">

    <plugin>fake_components/GenericSystem</plugin>

  </xacro:if>

  <xacro:unless value="$(arg use_fake_hardware)">

    <plugin>mycobot_hardware/MyCobotHardware</plugin>

    <param name="port">$(arg robot_port)</param>

    <param name="baud">$(arg robot_baud)</param>

    <param name="command_speed">$(arg command_speed)</param>

  </xacro:unless>

</hardware>

```

```

<!-- Arm 6 DoF -->

<joint name="joint2_to_joint1">

  <command_interface name="position"/>

  <state_interface name="position"/>

  <state_interface name="velocity"/>

</joint>

<joint name="joint3_to_joint2">

  <command_interface name="position"/>

  <state_interface name="position"/>

  <state_interface name="velocity"/>

</joint>

<joint name="joint4_to_joint3">

  <command_interface name="position"/>

  <state_interface name="position"/>

  <state_interface name="velocity"/>

</joint>

<joint name="joint5_to_joint4">

  <command_interface name="position"/>

  <state_interface name="position"/>

  <state_interface name="velocity"/>

</joint>

<joint name="joint6_to_joint5">

  <command_interface name="position"/>

  <state_interface name="position"/>

  <state_interface name="velocity"/>

```

```
</joint>

<joint name="joint6output_to_joint6">
  <command_interface name="position"/>
  <state_interface name="position"/>
  <state_interface name="velocity"/>
</joint>

<!-- Greifer aktives Gelenk (Mimics in URDF definiert) -->
<joint name="gripper_controller">
  <command_interface name="position"/>
  <state_interface name="position"/>
  <state_interface name="velocity"/>
</joint>
</ros2_control>

</robot>
```

src/vida_vision/launch/vida_detector.launch.py

```
"""Startet den vida_detector Node (Phase 2 – nur Erkennung + Overlay).
```

HINWEIS: Die D435i Topics müssen aktiv sein. Starte den Haupt-Stack mit Kamera:

```
ros2 launch mycobot_moveit_config demo.launch.py use_camera:=true
```

Danach diesen Launch in einem separaten Terminal ausführen.

Alle operativen Gate-/Schwellenwert-Parameter können überschrieben werden, z.B.:

```
ros2 launch vida_vision vida_detector.launch.py conf:=0.6 mode_consensus:=5
```

```
"""
```

```
from launch import LaunchDescription
```

```
from launch.actions import DeclareLaunchArgument
```

```
from launch.substitutions import LaunchConfiguration
```

```
from launch_ros.actions import Node
```

```
def generate_launch_description():
```

```
    return LaunchDescription([
```

```
        # Alle folgenden DeclareLaunchArgument definieren überschreibbare Parameter mit
```

```
        # Standardwerten; sie werden weiter unten 1:1 an den vida_detector-Node durchgereicht.
```

```
        # image_topic: Eingangs-Farbbild vom RealSense-Treiber (Quelle der YOLO-Erkennung).
```

```
        DeclareLaunchArgument("image_topic", default_value="/camera/color/image_raw"),
```

```
        # conf: minimale YOLO-Konfidenz, damit eine Detektion überhaupt gilt. 0.75 ist ein
```

```
        # konservativer Standard gegen Fehldetektionen; für schwierige Szenen absenkbar.
```

```
        DeclareLaunchArgument("conf", default_value="0.75"),
```

```
        # rate_hz: Verarbeitungsfrequenz. 1 Hz genügt (Schrauben bewegen sich nicht) und
```

```
        # entlastet den Jetson Nano deutlich (YOLO11-seg ist rechenintensiv).
```

```
        DeclareLaunchArgument("rate_hz", default_value="1.0"),
```

```
        # enable_3d: bei true wird aus der Tiefe zusätzlich die 3D-Position/Greifachse
```

```
        # geschätzt (benötigt align_depth des Kameratreibers).
```

```
        DeclareLaunchArgument("enable_3d", default_value="true"),
```

```
        # depth_mad_k: Ausreißer-Filter für die Tiefenwerte im Segment (k*MAD um den Median).
```

```
        DeclareLaunchArgument("depth_mad_k", default_value="2.5"),
```

```
        # axis_min_linearity: Mindest-Linearität (0.1) der Segmentmaske, damit ihre
```

```
        # Haupt-Trägheitsachse als verlässliche Schraubenachse gilt.
```

```

DeclareLaunchArgument("axis_min_linearity", default_value="0.6"),

# grasp_tilt_min_deg: ab dieser Neigung (Grad) gegen die Senkrechte wird ein

# gekippter Greifansatz statt eines reinen Top-Down-Griffs gewählt.

DeclareLaunchArgument("grasp_tilt_min_deg", default_value="15.0"),

# target_z_min/target_z_max: gültiger Höhenbereich (m) des Greifziels im Basis-Frame;

# Detektionen außerhalb (Boden/zu hoch) werden verworfen – Plausibilitäts-Gate.

DeclareLaunchArgument("target_z_min", default_value="-0.02"),

DeclareLaunchArgument("target_z_max", default_value="0.12"),

# mode_consensus: so viele aufeinanderfolgende Frames müssen dasselbe Ziel bestätigen,

# bevor es als stabil ausgegeben wird (unterdrückt Flackern/Einzelaussetzer).

DeclareLaunchArgument("mode_consensus", default_value="3"),

# sticky_radius: Radius (m), innerhalb dessen das bisherige Ziel beibehalten wird, statt

# bei geringfügiger Verschiebung auf eine andere Schraube umzuspringen (Hysterese).

DeclareLaunchArgument("sticky_radius", default_value="0.05"),

# Zielauswahl: am zentralsten + am besten sichtbar, harte Eck-Eliminierung

# center_weight/conf_weight: Gewichtung von Bildzentrums-Nähe vs. Konfidenz im Score.

DeclareLaunchArgument("center_weight", default_value="1.0"),

DeclareLaunchArgument("conf_weight", default_value="1.0"),

# corner_edge_frac: Randstreifen-Breite (Bruchteil der Bildkante), der als "Ecke" gilt.

DeclareLaunchArgument("corner_edge_frac", default_value="0.15"),

# corner_min_screws: erst ab so vielen Schrauben werden randständige/eckige Ziele hart

# eliminiert (bei wenigen Schrauben ist auch eine Eck-Schraube noch akzeptabel).

DeclareLaunchArgument("corner_min_screws", default_value="4"),

# frame_margin_frac: Sicherheitsrand am Bildrand (Bruchteil), damit Ziele nicht auf der

# Bildkante liegen (dort ist die Tiefe/Geometrie unzuverlässig).

DeclareLaunchArgument("frame_margin_frac", default_value="0.04"),

# display_min_conf: Konfidenzschwelle nur für das Overlay-Rendering (Anzeige), strenger

# als conf – im Bild werden nur sehr sichere Detektionen eingezeichnet.

DeclareLaunchArgument("display_min_conf", default_value="0.9"),

# top_down: erzwingt bevorzugt einen senkrechten Greifansatz (Standard für flach

# liegende Schrauben).

DeclareLaunchArgument("top_down", default_value="true"),

# vida_detector-Node: führt die YOLO11-seg-Erkennung durch, wählt das beste Greifziel,

# schätzt (bei enable_3d) dessen 3D-Pose und zeichnet das Overlay. Alle oben

# deklarierten Argumente werden über LaunchConfiguration als Parameter übergeben.

Node(
    package="vida_vision",

```



```

executable="vida_detector",

name="vida_detector",

output="screen",

parameters=[{

    "image_topic": LaunchConfiguration("image_topic"),

    "conf": LaunchConfiguration("conf"),

    "rate_hz": LaunchConfiguration("rate_hz"),

    "enable_3d": LaunchConfiguration("enable_3d"),

    "depth_map_k": LaunchConfiguration("depth_map_k"),

    "axis_min_linearity": LaunchConfiguration("axis_min_linearity"),

    "grasp_tilt_min_deg": LaunchConfiguration("grasp_tilt_min_deg"),

    "target_z_min": LaunchConfiguration("target_z_min"),

    "target_z_max": LaunchConfiguration("target_z_max"),

    "mode_consensus": LaunchConfiguration("mode_consensus"),

    "sticky_radius": LaunchConfiguration("sticky_radius"),

    "center_weight": LaunchConfiguration("center_weight"),

    "conf_weight": LaunchConfiguration("conf_weight"),

    "corner_edge_frac": LaunchConfiguration("corner_edge_frac"),

    "corner_min_screws": LaunchConfiguration("corner_min_screws"),

    "frame_margin_frac": LaunchConfiguration("frame_margin_frac"),

    "display_min_conf": LaunchConfiguration("display_min_conf"),

    "top_down": LaunchConfiguration("top_down"),

}],

),

1)

```

src/vida_vision/package.xml

```
<?xml version="1.0"?>

<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>

<package format="3">

  <name>vida_vision</name>

  <version>0.1.0</version>

  <description>Y0L011-seg ile D435i vida tespiti ve robot pick-and-place için 3B hedef üretimi (myCobot 280 JN).</description>

  <maintainer email="ekinciorhandijvar@gmail.com">er</maintainer>

  <license>BSD-3-Clause</license>


  <buildtool_depend>ament_python</buildtool_depend>


  <exec_depend>rclpy</exec_depend>

  <exec_depend>sensor_msgs</exec_depend>

  <exec_depend>geometry_msgs</exec_depend>

  <exec_depend>std_msgs</exec_depend>

  <exec_depend>visualization_msgs</exec_depend>

  <exec_depend>tf2_ros</exec_depend>

  <exec_depend>python3-numpy</exec_depend>

  <exec_depend>python3-opencv</exec_depend>


  <export>

    <build_type>ament_python</build_type>

  </export>

</package>
```

src/vida_vision/setup.cfg

```
[develop]
```

```
script_dir=$base/lib/vida_vision
```

```
[install]
```

```
install_scripts=$base/lib/vida_vision
```

src/vida_vision/setup.py

```
from glob import glob

from setuptools import setup

package_name = 'vida_vision'

setup(
    name=package_name,
    version='0.1.0',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages', ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        ('share/' + package_name + '/launch', glob('launch/*.launch.py')),
        ('share/' + package_name + '/weights', glob('weights/*.pt')),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='er',
    maintainer_email='ekinciorhandijvar@gmail.com',
    description='YOL011-seg Schraubenerkennung + 3D-Zielerzeugung (myCobot 280 JN + D435i).',
    license='BSD-3-Clause',
    entry_points={
        'console_scripts': [
            'vida_detector = vida_vision.vida_detector_node:main',
        ],
    },
)
```

```
| src/vida_vision/vida_vision/__init__.py
```

src/vida_vision/vida_vision/detection.py

```
"""Schrauben-Erkennung – Mittelpunkt + Winkel + Farbe aus YOLO11-seg Maske.

Portiert aus inference_template.py (kakir001 / robot-vida-projesi).

Template-Mask-Refinement entfernt: für den Roboter genügen der Masken-Centroid
(PCA-Zentrum) und der Langachsen-Winkel. Dieses Modul ist ROS-unabhängig – reines numpy/cv2/ultralytics.
"""

import math

import numpy as np

# Exakt in der in best.pt eingebetteten Reihenfolge (mit model.names verifiziert):
CLASS_NAMES = ["sari", "beyaz", "siyah", "yesil", "kirmizi"]

# Physische Schrauben-Abmessung (für mm/px Skalenschätzung)
SCREW_LENGTH_MM = 44.0

def get_mask_orientation(mask_bool):
    """Mittelpunkt + Langachsen-Winkel via PCA (Hauptkomponentenanalyse) aus Binär-Maske.

    MATHEMATIK – 2D-PCA auf den Maskenpixeln:

    Die Menge der zur Maske gehörenden Pixel (x_i, y_i) wird als 2D-Punktwolke
    aufgefasst. Der Schwerpunkt (Centroid) (cx, cy) = Mittelwert aller Pixel ist der
    Schrauben-Mittelpunkt. Nach Zentrierung (Abzug des Centroids) beschreibt die
    2x2-Kovarianzmatrix cov die räumliche Streuung der Punktwolke. Ihre Eigenwert-
    zerlegung (eigh, da cov symmetrisch) liefert zwei orthogonale Eigenvektoren:

    - der Eigenvektor zum GROESSTEN Eigenwert = Richtung der größten Streuung
      = LANGACHSE der Schraube (Schafttrichtung),

    - der zum kleineren Eigenwert = KURZACHSE (Breite/Dicke).

    Der Winkel der Langachse folgt aus atan2(major_y, major_x) im Bildkoordinaten-
    system (x nach rechts, y nach UNTEN – daher ist der Winkel im Uhrzeigersinn positiv).

    Returns (cx, cy, angle_deg, major_len_px, minor_len_px) oder None.

    angle_deg: Winkel der Langachse in Bildkoordinaten (x rechts, y unten).
    """
```

```

ys, xs = np.where(mask_bool)

if len(xs) < 10:

    return None # zu wenige Pixel für eine stabile Kovarianzschätzung

# Schwerpunkt (Centroid) der Punktwolke = Schrauben-Mittelpunkt in px
cx = float(np.mean(xs))
cy = float(np.mean(ys))

# Zentrierte Koordinaten (Nx2) und 2x2-Kovarianzmatrix der Punktwolke
coords = np.column_stack([xs - cx, ys - cy]).astype(np.float64)
cov = np.cov(coords.T)

# Eigenwertzerlegung der symmetrischen Kovarianz -> Hauptachsen der Streuung
eigvals, eigvecs = np.linalg.eigh(cov)

# Größter Eigenwert -> zugehöriger Eigenvektor = Langachse (Schaftrichtung)
idx = int(np.argmax(eigvals))
major = eigvecs[:, idx]
angle = math.degrees(math.atan2(major[1], major[0]))

# Achsenlängen ~ 2*Standardabweichung (sqrt Eigenwert), zusätzlich *2 als
# Gesamtausdehnung (Näherung für die sichtbare Länge/Breite in px).
major_len = 2.0 * math.sqrt(max(eigvals[idx], 0.0)) * 2.0
minor_len = 2.0 * math.sqrt(max(eigvals[1 - idx], 0.0)) * 2.0
return cx, cy, angle, major_len, minor_len

```

```

def estimate_scale(mask_bool, angle_deg):

```

```

    """mm/px Schätzung aus Maskenlänge (px) bezogen auf physische 44 mm.

```

MATHEMATIK – Projektion auf die Langachse:

Jeder Maskenpixel wird (nach Zentrierung um den Centroid) per Skalarprodukt mit dem Einheitsvektor ($\cos a$, $\sin a$) der Langachse auf diese projiziert:

$$rx_i = (x_i - cx) \cdot \cos(a) + (y_i - cy) \cdot \sin(a).$$

rx ist die vorzeichenbehaftete Koordinate ENTLANG der Schraubenachse. Die Ausdehnung $\max(rx) - \min(rx)$ ist die sichtbare Schraubenlänge in Pixeln.

Aus der bekannten physischen Länge (44 mm) folgt der Maßstab $\text{mm/px} = 44 / \text{length_px}$.

(Fallback 0.1 mm/px bei zu kurzer/ungültiger Maske.)

```

"""

ys, xs = np.where(mask_bool)

if len(xs) < 10:

    return 0.1

cx, cy = np.mean(xs), np.mean(ys)

a = math.radians(angle_deg)

cos_a, sin_a = math.cos(a), math.sin(a)

# Projektion der zentrierten Pixel auf die Langachsenrichtung (Skalarprodukt)

rx = (xs - cx) * cos_a + (ys - cy) * sin_a

length_px = rx.max() - rx.min() # Ausdehnung entlang der Achse = Länge in px

if length_px < 5:

    return 0.1

return float(SCREW_LENGTH_MM / length_px)

def head_aware_grip(mask_bool, cx, cy, angle_deg, inset_frac=0.30, min_ratio=1.15):

    """Finde den Schrauben-KOPF aus der Maskenform und verschiebe den Greifpunkt auf den Schaft unter dem Kopf.

    Der Schraubenkopf ist breiter als der Schaft: vergleicht die Breite an beiden
    Enden entlang der Langachse (25%-Segment vom Ende); das breitere Ende = Kopf.
    Der Greifpunkt wird vom Kopf-Ende um inset_frac*L Richtung Spitze nach innen
    verschoben (direkt unter dem Kopf, gerader/paralleler Schaft = sicherster
    Parallel-Backen-Griff). Liegt das Kopf-Spitze-Breitenverhältnis unter min_ratio
    (unklar), wird der Mittelpunkt (Centroid) zurückgegeben und head_known=False.

    angle_deg = Langachsen-Winkel aus get_mask_orientation (x rechts, y unten).
    Returns dict: grip_px=(gx,gy), head_px=(hx,hy)|None, head_known(bool), ratio(float).
    """

    fallback = {"grip_px": (cx, cy), "head_px": None, "head_known": False, "ratio": 1.0}

    ys, xs = np.where(mask_bool)

    if len(xs) < 20:

        return fallback

    # Lokales, an der Schraube ausgerichtetes Koordinatensystem: die zentrierten
    # Pixel werden per Skalarprodukt in "major" (entlang der Langachse) und "minor"
    # (senkrecht dazu = Breitenrichtung) zerlegt – eine 2D-Rotation um den Winkel a.

    a = math.radians(angle_deg)

    ca, sa = math.cos(a), math.sin(a)

```



```

major = (xs - cx) * ca + (ys - cy) * sa      # vorzeichenbehaftet entlang der Langachse
minor = -(xs - cx) * sa + (ys - cy) * ca     # senkrecht zur Langachse (Breite)

t_min, t_max = float(major.min()), float(major.max())

length = t_max - t_min                       # Gesamtlänge entlang der Achse (px)

if length < 5:
    return fallback

# Die beiden End-Segmente (je 25% der Länge) an den Achsenenden auswählen
seg = 0.25 * length

hi = major >= (t_max - seg)   # Segment am oberen Ende (großes major)
lo = major <= (t_min + seg)   # Segment am unteren Ende (kleines major)

def _width(sel):
    # Breite eines End-Segments = robustes Perzentil-Intervall (90%-10%) der
    # Kurzachsen-Koordinate; ausreißerrobust gegenüber Maskenrand-Rauschen.
    if np.count_nonzero(sel) < 5:
        return 0.0
    mv = minor[sel]
    return float(np.percentile(mv, 90) - np.percentile(mv, 10))

w_hi, w_lo = _width(hi), _width(lo)

if max(w_hi, w_lo) < 1e-6:
    return fallback

# Verhältnis der breiteren zur schmalen Seite. Kopf (breit) vs. Schaft (schmal).
ratio = max(w_hi, w_lo) / max(min(w_hi, w_lo), 1e-6)

if ratio < min_ratio:
    # Zu geringer Breitenunterschied -> Kopf unklar -> Centroid als Rückfall
    out = dict(fallback)
    out["ratio"] = ratio
    return out

# Das breitere Ende ist der Kopf. Der Greifpunkt wird vom Kopf-Ende um
# inset_frac*L nach INNEN (Richtung Spitze) verschoben -> gerader Schaft.
head_at_max = w_hi >= w_lo
t_head = t_max if head_at_max else t_min
t_grip = t_head - inset_frac * length if head_at_max else t_head + inset_frac * length

return {

```

```

    "grip_px": (cx + t_grip * ca, cy + t_grip * sa),

    "head_px": (cx + t_head * ca, cy + t_head * sa),

    "head_known": True,

    "ratio": ratio,

}

```

```

def detect_screws(bgr_img, model, conf=0.70):

```

```

    """Erkenne Schrauben in einem BGR-Frame mit YOLO11-seg.

```

```

    Returns list[dict]: class_id, class_name, conf, center_px (cx,cy),
    angle_deg, bbox (x1,y1,x2,y2), mask (HxW uint8 0/255), mm_per_px.
    Absteigend nach Confidence sortiert.

```

```

    """

```

```

    results = model.predict(bgr_img, conf=conf, retina_masks=True, verbose=False)[0]

```

```

    out = []

```

```

    if results.masks is None:

```

```

        return out

```

```

    masks = results.masks.data.cpu().numpy()

```

```

    boxes = results.boxes

```

```

    h, w = bgr_img.shape[:2]

```

```

    import cv2 # lazy: in manchen Umgebungen Import-Kosten

```

```

    for i in range(len(masks)):

```

```

        cls_id = int(boxes.cls[i].item())

```

```

        conf_val = float(boxes.conf[i].item())

```

```

        x1, y1, x2, y2 = [float(v) for v in boxes.xyxy[i].cpu().numpy().tolist()]

```

```

        m = masks[i]

```

```

        if m.shape[:2] != (h, w):

```

```

            m = cv2.resize(m, (w, h))

```

```

        m_bool = m > 0.5

```

```

        orient = get_mask_orientation(m_bool)

```

```

        if orient is None:

```

```

        continue

    cx, cy, angle, major_len, minor_len = orient

    mm_per_px = estimate_scale(m_bool, angle)

    out.append({
        "class_id": cls_id,
        "class_name": CLASS_NAMES[cls_id] if cls_id < len(CLASS_NAMES) else f"cls_{cls_id}",
        "conf": conf_val,
        "center_px": (cx, cy),
        "angle_deg": float(angle),
        "bbox": (x1, y1, x2, y2),
        "mask": (m_bool * 255).astype(np.uint8),
        "major_len_px": float(major_len),
        "minor_len_px": float(minor_len),
        "mm_per_px": mm_per_px,
    })

out.sort(key=lambda d: d["conf"], reverse=True)

return out

```

src/vida_vision/vida_vision/vida_detector_node.py

```
"""vida_detector – Schraubenerkennung mit YOLO11-seg + 3D-Zielerzeugung aus D435i.
```

```
Phase 2: color topic → Erkennung → /vida/overlay (Visualisierung).
```

```
Phase 4: Mitte-Pixel der gewählten Schraube + aligned depth → Kamera-Frame 3D (deproject)
```

```
→ tf2 (camera_color_optical_frame → robot_base) → /vida/target (PoseStamped)
```

```
+ /vida/target_marker (RViz). Base-Koordinaten werden zur Prüfung geloggt.
```

```
cv_bridge WIRD NICHT BENUTZT – Image<->numpy von Hand.
```

```
"""
```

```
import math
```

```
import os
```

```
import numpy as np
```

```
import rclpy
```

```
from rclpy.node import Node
```

```
from sensor_msgs.msg import Image, CameraInfo
```

```
from geometry_msgs.msg import PoseStamped, Point
```

```
from visualization_msgs.msg import Marker
```

```
from std_msgs.msg import String, Int32
```

```
import tf2_ros
```

```
from vida_vision.detection import detect_screws, CLASS_NAMES, head_aware_grip
```

```
CLASS_BGR = [
```

```
    (0, 255, 255),    # gelb
```

```
    (255, 255, 255), # weiß
```

```
    (60, 60, 60),     # schwarz
```

```
    (0, 255, 0),      # grün
```

```
    (0, 0, 255),      # rot
```

```
]
```

```
# Deutsche Farbnamen fuer das Overlay (UI ist deutsch/ASCII; CLASS_NAMES bleiben
```

```
# die im Modell eingebetteten tuerkischen Klassennamen).
```

```
CLASS_DE = ["GELB", "WEISS", "SCHWARZ", "GRUEN", "ROT"]
```

```

def image_to_bgr(msg):
    h, w = msg.height, msg.width

    buf = np.frombuffer(msg.data, dtype=np.uint8)

    enc = msg.encoding.lower()

    if enc in ("rgb8", "bgr8"):
        img = buf.reshape(h, w, 3)

        if enc == "rgb8":
            img = img[:, :, ::-1]

        return np.ascontiguousarray(img)

    raise ValueError(f"nicht unterstuetztes color encoding: {msg.encoding}")


def depth_to_mm(msg):
    """aligned_depth_to_color (16UC1, mm) -> HxW uint16 numpy."""
    if msg.encoding.lower() not in ("16uc1", "mono16"):
        raise ValueError(f"nicht unterstuetztes depth encoding: {msg.encoding}")

    return np.frombuffer(msg.data, dtype=np.uint16).reshape(msg.height, msg.width)


def bgr_to_image(img, frame_id, stamp):
    msg = Image()

    msg.header.stamp = stamp

    msg.header.frame_id = frame_id

    msg.height, msg.width = img.shape[:2]

    msg.encoding = "bgr8"

    msg.is_bigendian = 0

    msg.step = img.shape[1] * 3

    msg.data = np.ascontiguousarray(img).tobytes()

    return msg


def qrot(q, v):
    """Vektor v(3) mit Quaternion (x,y,z,w) rotieren.

    MATHEMATIK – effiziente Quaternion-Vektor-Rotation ohne Aufbau der
    Rotationsmatrix. Statt der klassischen Formel  $v' = q * v * q^{-1}$  wird die

```

äquivalente, numerisch günstigere Variante nach der Rodrigues-Umformung genutzt:

$$t = 2 * (q_{xyz} \times v)$$

$$v' = v + w * t + (q_{xyz} \times t)$$

mit $q_{xyz} = (x, y, z)$ dem Vektorteil und w dem Skalarteil des Quaternions.

Voraussetzung: q ist normiert (Einheitsquaternion).

"""

$x, y, z, w = q$

$vx, vy, vz = v$

$\# t = 2 * \text{cross}(q_{xyz}, v)$ (Kreuzprodukt Vektorteil x Eingangsvektor)

$tx = 2.0 * (y * vz - z * vy)$

$ty = 2.0 * (z * vx - x * vz)$

$tz = 2.0 * (x * vy - y * vx)$

$\# v' = v + w * t + \text{cross}(q_{xyz}, t)$

$rx = vx + w * tx + (y * tz - z * ty)$

$ry = vy + w * ty + (z * tx - x * tz)$

$rz = vz + w * tz + (x * ty - y * tx)$

return (rx, ry, rz)

def yaw_to_quat(yaw):

return (0.0, 0.0, math.sin(yaw / 2.0), math.cos(yaw / 2.0))

def mat_to_quat(R):

"""3x3 Rotationsmatrix (Spalten = tcp-Achsen in base) -> quaternion (x,y,z,w).

MATHEMATIK – Umrechnung Rotationsmatrix -> Quaternion nach Shepperd/Shoemake.

Die Spur $t = R_{00} + R_{11} + R_{22}$ hängt mit dem Skalarteil zusammen ($t = 4w^2 - 1$).

Um numerische Instabilität (Division durch nahezu 0) zu vermeiden, wird nach

Fallunterscheidung stets über die GRÖESSTE Diagonalkomponente aufgelöst:

- $t > 0$ -> über w ,

- sonst der Fall, in dem R_{00} , R_{11} bzw. R_{22} maximal ist -> über x , y bzw. z .

Zum Schluss wird das Quaternion auf Einheitslänge normiert.

"""

$t = R[0, 0] + R[1, 1] + R[2, 2]$

if $t > 0.0$:

$s = \text{math.sqrt}(t + 1.0) * 2.0$

```

w = 0.25 * s

x = (R[2, 1] - R[1, 2]) / s

y = (R[0, 2] - R[2, 0]) / s

z = (R[1, 0] - R[0, 1]) / s

elif R[0, 0] > R[1, 1] and R[0, 0] > R[2, 2]:

    s = math.sqrt(1.0 + R[0, 0] - R[1, 1] - R[2, 2]) * 2.0

    w = (R[2, 1] - R[1, 2]) / s

    x = 0.25 * s

    y = (R[0, 1] + R[1, 0]) / s

    z = (R[0, 2] + R[2, 0]) / s

elif R[1, 1] > R[2, 2]:

    s = math.sqrt(1.0 + R[1, 1] - R[0, 0] - R[2, 2]) * 2.0

    w = (R[0, 2] - R[2, 0]) / s

    x = (R[0, 1] + R[1, 0]) / s

    y = 0.25 * s

    z = (R[1, 2] + R[2, 1]) / s

else:

    s = math.sqrt(1.0 + R[2, 2] - R[0, 0] - R[1, 1]) * 2.0

    w = (R[1, 0] - R[0, 1]) / s

    x = (R[0, 2] + R[2, 0]) / s

    y = (R[1, 2] + R[2, 1]) / s

    z = 0.25 * s

n = math.sqrt(x * x + y * y + z * z + w * w)

return (x / n, y / n, z / n, w / n)

```

```
def grasp_quat_from_axis(s_base, top_down=False):
```

"""Greifer-Grasp-Orientierung aus der 3D-Längsachse der Schraube in base aufbauen.

Greifer-Konvention (passend zu URDF/gripperOrientation): tcp +Y = Annäherungsachse.

Grasp-Frame:

- tcp +Y (eY) = Annäherung = möglichst steil tischwärts-abwärts, ABER Komponente SENKRECHT zur Schraubenachse
- tcp +Z (eZ) = entlang der Schraubenachse (-s)
- tcp +X (eX) = Finger-Schließachse = eY × eZ (SENKRECHT zur Schrauben-Längsachse)

Flach liegende Schraube (s horizontal) -> eY=(0,0,-1), also gerade-abwärts (top-down) ergibt sich natürlich.

Aufrechte Schraube (s ~vertikal) -> Annäherung undefiniert -> None (als unzuverlässig werten).

top_down=True: Annäherungsachse eY wird IMMER gerade abwärts (0,0,-1) erzwungen;

die gemessene PCA-Neigung (Seitenneigung) wird IGNORIERT, nur der Yaw (horizontale Längsachse der Schraube) wird beibehalten → Greifer fährt von oben SENKRECHT herab, Finger schließen senkrecht zum Schaft. Verhindert, dass beim schrägen Herabfahren Nachbarschrauben angestoßen und die Szene gestört wird (Nutzerwunsch).

Returns (x,y,z,w) oder None.

"""

```
s = np.asarray(s_base, dtype=np.float64)
```

```
n = np.linalg.norm(s)
```

```
if n < 1e-9:
```

```
    return None
```

```
s = s / n
```

```
if s[2] < 0.0:      # Achse so wählen, dass sie nach oben zeigt
```

```
    s = -s
```

```
if top_down:
```

```
    # SENKRECHTE ANNAEHERUNG: eY = gerade abwärts; eZ = horizontale Längsachse der Schraube (Yaw).
```

```
    eY = np.array([0.0, 0.0, -1.0])
```

```
    sh = np.array([s[0], s[1], 0.0])
```

```
    nsh = np.linalg.norm(sh)
```

```
    eZ = sh / nsh if nsh > 1e-6 else np.array([1.0, 0.0, 0.0]) # ~vertikale Schraube: beliebiger Yaw
```

```
    eX = np.cross(eY, eZ)
```

```
    nx = np.linalg.norm(eX)
```

```
    if nx < 1e-9:
```

```
        return None
```

```
    eX = eX / nx
```

```
    eZ = np.cross(eX, eY)          # voll orthonormal
```

```
    return mat_to_quat(np.column_stack([eX, eY, eZ]))
```

```
down = np.array([0.0, 0.0, -1.0])
```

```
a = down - np.dot(down, s) * s      # zur Schraubenachse senkrechte Komponente von tisch-abwärts
```

```
na = np.linalg.norm(a)
```

```
if na < 1e-3:          # Schraube ~vertikal: Annäherung undefiniert
```

```
    return None
```

```
eY = a / na
```

```
eZ = -s
```

```
eX = np.cross(eY, eZ)
```

```
nx = np.linalg.norm(eX)
```



```

if nx < 1e-9:
    return None

eX = eX / nx

eZ = np.cross(eX, eY)          # voll orthonormal garantiert

R = np.column_stack([eX, eY, eZ])

return mat_to_quat(R)


class VidaDetector(Node):

    def __init__(self):
        super().__init__("vida_detector")

        self.declare_parameter("image_topic", "/camera/color/image_raw")

        self.declare_parameter("depth_topic", "/camera/aligned_depth_to_color/image_raw")

        self.declare_parameter("caminfo_topic", "/camera/color/camera_info")

        self.declare_parameter("base_frame", "robot_base")

        self.declare_parameter("weights", "")

        self.declare_parameter("conf", 0.70)  # Standard an run_pick_preview.sh angeglichen (0.85 war zu streng:
        reale Picks lagen bei 0.72-0.75)

        # WAHRNEHMUNGS-OFFSET (base-Frame, m): Die Kamera liest systematisch

        # versetzt (Lineal-Messung 2026-07-15: ca. +7/+11 mm in X/Y). Die Korrektur

        # wird beim VERÖFFENTLICHEN des Ziels addiert (interne Geometrie/Overlay

        # bleiben roh, damit die Rückprojektion weiter auf der Welle liegt).

        # Wertequelle: ~/ros2_ws/.place_offset.json – dieselbe Datei wie

        # click_place_moveit.py; das Ablaufskript reicht sie als Parameter durch.

        self.declare_parameter("offset_x", 0.0)

        self.declare_parameter("offset_y", 0.0)

        self.declare_parameter("offset_z", 0.0)

        self.declare_parameter("rate_hz", 1.0)

        self.declare_parameter("enable_3d", True)

        # Tiefenfilter (am kleinen/glänzenden Schraubenkopf ist das D435i-IR verrauscht):

        #   depth_win: Fensterradius (px) -> (2*win+1)^2 Proben. 5 = 11x11.

        #   depth_min_valid: mindestens so viele nicht-ungültige (>0) Proben nötig.

        #   depth_mad_k: Werte mit |x-med| > k*MAD gelten als Ausreißer und werden verworfen.

        self.declare_parameter("depth_win", 5)

        self.declare_parameter("depth_min_valid", 8)

        self.declare_parameter("depth_mad_k", 2.5)

```

```

# Reichweiten-Kreis (in base-XY-Ebene, auf Plattformhöhe):

# reach_radius      = SICHERER Reichweitenradius für senkrechten Griff (grün) [m]

# reach_radius_max = KINEMATISCHES Limit für senkrechten Griff (amber) [m]

# reach_z           = Plattformhöhe (base z) [m]

# Quelle: URDF-FK-Monte-Carlo + Hardware-Test (Greifer ganz senkrecht, 10cm pre-grasp):

# sicher=200mm (r<200 wird im ersten Yaw gelöst), kinematisches Senkrecht-Limit=260mm

# (früher 220/300; 300 war UNEINGESCHRAENKTES Limit, im top-down irreführend – Sitzung 21).

# DAUERHAFTER Standard: im Kamera-Overlay werden die Kreise stets mit diesen Werten gezeichnet.

self.declare_parameter("reach_radius", 0.20)

self.declare_parameter("reach_radius_max", 0.26)

# reach_radius_min = INNERE Grenze: näher als das (r<min) faltet sich der Arm im

# top-down-Griff zu stark → Arme stoßen aneinander/an die base (self/base collision).

# Gültiger Bereich ist ein DONUT: [min, reach_radius] grün. DEVLOG: r≤130mm zu nah, 150-190mm ideal.

self.declare_parameter("reach_radius_min", 0.15)

self.declare_parameter("reach_z", 0.008)

# 3D-Neigungsmessung (Schraubenachse) – PCA aus der Tiefe der Maskenpixel:

# axis_subsample    : nur jedes N-te Maskenpixel nehmen (Geschwindigkeit)

# axis_min_points    : Mindestanzahl Punkte mit gültiger Tiefe für PCA

# axis_min_linearity: (l1-l2)/l1 Vertrauensschwelle; darunter ist die Messung UNZUVERLAESSIG

#                    -> /vida/target wird NICHT veröffentlicht (Roboter wartet, Nutzer mischt um)

# grasp_tilt_min_deg: Schwelle zur Information (darunter = praktisch top-down)

self.declare_parameter("axis_subsample", 3)

self.declare_parameter("axis_min_points", 25)

self.declare_parameter("axis_min_linearity", 0.6)

self.declare_parameter("grasp_tilt_min_deg", 15.0)

# top_down=True: Grasp-Orientierung IMMER gerade-abwärts (schräge PCA wird ignoriert,

# nur Yaw beibehalten) → Greifer fährt von oben senkrecht herab, gleitet nicht schräg

# in Nachbarschrauben. Da die Schrauben in der Kiste ~flach liegen, ist der Standard an (Nutzerwunsch).

self.declare_parameter("top_down", True)

# Ziel-Veröffentlichungs-Gatter:

# target_z_min/max: liegt BASE z außerhalb dieses Bereichs, gilt depth als Müll (m)

# mode_consensus: /vida/target nicht veröffentlichen, solange nicht die letzten N Zyklen im SELBEN Modus
übereinstimmen

# (Tiefenschwankungen verhindern ein fälschliches Ein-Frame-top-down/TILT-Latch)

self.declare_parameter("target_z_min", -0.02)

self.declare_parameter("target_z_max", 0.12)

self.declare_parameter("mode_consensus", 3)

```

```

# sticky_radius: gibt es innerhalb dieses Radius (m) um das zuletzt veröffentlichte Ziel einen
Kandidaten, DIESEN wählen

# (klebrige Verfolgung – conf-Jitter kann das Ziel nicht zwischen zwei Schrauben springen lassen)

self.declare_parameter("sticky_radius", 0.05)

# kopf-bewusster Griff: Greifpunkt vom Masken-Centroid auf den geraden Schaft

# direkt unterhalb des Schrauben-KOPFES verschieben (sicherster Parallelbacken-Griff).

# grip_under_head=False -> altes Verhalten (Centroid). grip_inset_frac =

# Anteil der Verschiebung von der Kopfkante Richtung tip nach innen (Vielfaches der Schraubenlänge).

self.declare_parameter("grip_under_head", True)

self.declare_parameter("grip_inset_frac", 0.30)

# ZIELAUSWAHL – die eine "mittigste + bestsichtbare" Schraube:

# Score = center_weight * Mittigkeit + conf_weight * confidence.

# Mitte = Pixel-Centroid ALLER erkannten Schrauben (= Schraubenhaufen/Kistenmitte;

# keine Kalibrierung nötig, passt sich automatisch an, wo die Kiste steht). Mittigkeit=1 in der Mitte,
0 ganz außen.

self.declare_parameter("center_weight", 1.0)

self.declare_parameter("conf_weight", 1.0)

# HARTE ECKEN-ELIMINIERUNG: die achsenparallele bbox der Kandidaten (= innere Kistengrenze) wird berechnet;

# Schrauben näher als corner_edge_frac*bbox am Rand (Kistenwand → breiter Greifer stößt an)

# werden KOMPLETT aus der Kandidatenliste entfernt. Sind alle am Rand, wird kein Ziel VERÖFFENTLICHT
(Roboter wartet).

# corner_min_screws: gibt es weniger Kandidaten, ist die Kistengeometrie nicht bestimmbar → Eliminierung
übersprungen

# (Sicherheit, um die letzten paar Schrauben nicht ewig als Ecke zu werten und festzusetzen).

self.declare_parameter("corner_edge_frac", 0.15)

self.declare_parameter("corner_min_screws", 4)

# KAMERA-BILDRAND (FOV) ELIMINIERUNG: Schrauben sehr nah am Rand des Kamera-Sichtfelds

# ODER über den Rahmen hinausragend (Maske berührt den Rand → halb-abgeschnitten) können KEIN Ziel sein.

# Grund: am FOV-Rand ist die Tiefe unzuverlässig und eine abgeschnittene Maske verfälscht Centroid/PCA-
Winkel/

# Greifpunkt → falsches Ziel. frame_margin_frac: sichere Zone um diesen Anteil der Bild-Breite/Höhe

# nach innen; Schrauben, deren Mitte AUSSERHALB dieser Zone liegt oder deren

# Maske den Bildrand berührt, werden eliminiert. 0 → aus.

self.declare_parameter("frame_margin_frac", 0.04)

# OVERLAY-VEREINFACHUNG: Schrauben UNTERHALB dieser conf werden auf dem Bildschirm GAR NICHT gezeichnet

# (Reduzierung der Unübersichtlichkeit). Bei den gezeichneten nur ein Punkt – kein Kreis/Text.

# (0.9 → 0.85 gesenkt, Nutzerwunsch 2026-07-16: auch etwas unsicherere

# Wellen im Fenster zeigen)

self.declare_parameter("display_min_conf", 0.85)

```

```

self.image_topic = self.get_parameter("image_topic").value

self.depth_topic = self.get_parameter("depth_topic").value

self.caminfo_topic = self.get_parameter("caminfo_topic").value

self.base_frame = self.get_parameter("base_frame").value

self.conf = float(self.get_parameter("conf").value)

rate = float(self.get_parameter("rate_hz").value)

self.enable_3d = bool(self.get_parameter("enable_3d").value)

self._depth_win = int(self.get_parameter("depth_win").value)

self._depth_min_valid = int(self.get_parameter("depth_min_valid").value)

self._depth_mad_k = float(self.get_parameter("depth_mad_k").value)

self.reach_radius = float(self.get_parameter("reach_radius").value)

self.reach_radius_max = float(self.get_parameter("reach_radius_max").value)

self.reach_radius_min = float(self.get_parameter("reach_radius_min").value)

self.reach_z = float(self.get_parameter("reach_z").value)

self._axis_subsample = max(1, int(self.get_parameter("axis_subsample").value))

self._axis_min_points = int(self.get_parameter("axis_min_points").value)

self._axis_min_linearity = float(self.get_parameter("axis_min_linearity").value)

self._grasp_tilt_min_deg = float(self.get_parameter("grasp_tilt_min_deg").value)

self._top_down = bool(self.get_parameter("top_down").value)

self._target_z_min = float(self.get_parameter("target_z_min").value)

self._target_z_max = float(self.get_parameter("target_z_max").value)

self._sticky_radius = float(self.get_parameter("sticky_radius").value)

self._grip_under_head = bool(self.get_parameter("grip_under_head").value)

self._grip_inset_frac = float(self.get_parameter("grip_inset_frac").value)

self._center_weight = float(self.get_parameter("center_weight").value)

self._conf_weight = float(self.get_parameter("conf_weight").value)

self._corner_edge_frac = float(self.get_parameter("corner_edge_frac").value)

self._corner_min_screws = int(self.get_parameter("corner_min_screws").value)

self._frame_margin_frac = float(self.get_parameter("frame_margin_frac").value)

self._display_min_conf = float(self.get_parameter("display_min_conf").value)

self._offset = (float(self.get_parameter("offset_x").value),
                float(self.get_parameter("offset_y").value),
                float(self.get_parameter("offset_z").value))

if any(abs(v) > 1e-9 for v in self._offset):
    self.get_logger().info(
        f"Ziel-Offset AKTIV: ({self._offset[0]*1000:.1f}, "

```

```

        f"{self._offset[1]*1000:.1f}, {self._offset[2]*1000:.1f}) mm "

        "wird auf /vida/target addiert")

self._last_pub_p = None # BASE-Position des zuletzt veröffentlichten Ziels (klebrige Auswahl)

self._last_axis_base = None # BASE-3D-Achse des zuletzt veröffentlichten Ziels (gesperrtes Overlay)

from collections import deque

self._mode_hist = deque(maxlen=max(1, int(self.get_parameter("mode_consensus").value)))


weights = self.get_parameter("weights").value

if not weights:

    from ament_index_python.packages import get_package_share_directory

    weights = os.path.join(

        get_package_share_directory("vida_vision"), "weights", "vida_large.pt"

    )

self.get_logger().info(f"Modell wird geladen: {weights}")

from ultralytics import YOLO

self.model = YOLO(weights)

# WARMUP: die ERSTE CUDA/cuDNN-Inferenz dauert auf dem Nano ~100s (Kernel-
# Initialisierung). Ohne Warmup fror das erste Kamerabild scheinbar minutenlang
# ein (Sitzung 2026-07-16). Dummy-Inferenz hier → "Modell bereit" heisst WIRKLICH bereit.

self.get_logger().info("CUDA-Warmup laeuft (erste Inferenz, auf dem Nano 1-2 min)...")

self.model.predict(np.zeros((480, 848, 3), dtype=np.uint8), conf=0.5, verbose=False)

self.get_logger().info(f"Modell bereit (task={self.model.task}, Klassen={CLASS_NAMES})")


self._latest = None # (bgr, stamp, frame_id)

self._depth = None # HxW uint16 (mm)

self._K = None # (fx, fy, cx, cy)


self.sub_img = self.create_subscription(Image, self.image_topic, self._on_image, 1)

self.pub_overlay = self.create_publisher(Image, "/vida/overlay", 1)


if self.enable_3d:

    self.sub_depth = self.create_subscription(Image, self.depth_topic, self._on_depth, 1)

    self.sub_info = self.create_subscription(CameraInfo, self.caminfo_topic, self._on_info, 1)

    self.pub_target = self.create_publisher(PoseStamped, "/vida/target", 1)

    self.pub_marker = self.create_publisher(Marker, "/vida/target_marker", 1)

    # Ziel-FARBE/Info (damit nach Schließen des Kamerafensters aus dem Terminal/Latch
    # ersichtlich ist, zu welcher Schraube gefahren wird). target_latch capture hört dies ab + speichert.

```

```

self.pub_target_info = self.create_publisher(String, "/vida/target_info", 1)

# FARBKLASSE der Zielschraube (class_id, 0..4 = CLASS_NAMES-Reihenfolge). pick_tilt
# liest dies GLEICHZEITIG mit /vida/target → legt die gegriffene Schraube in den richtigen Farbbehälter
# (Sortierung nach Farbe). Wird gleichzeitig mit _publish_target veröffentlicht.
self.pub_target_color = self.create_publisher(Int32, "/vida/target_color", 1)

self.pub_axis = self.create_publisher(Marker, "/vida/axis_marker", 1)

self.tf_buffer = tf2_ros.Buffer()

self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)


self.timer = self.create_timer(1.0 / max(rate, 0.1), self._on_timer)

self.get_logger().info(
    f"Lausche: {self.image_topic} | inference {rate:.1f} Hz | 3D={'an' if self.enable_3d else 'aus'}"
)


# ---- Abonnenten-Callbacks ----

def _on_image(self, msg):
    try:
        self._latest = (image_to_bgr(msg), msg.header.stamp, msg.header.frame_id)
    except Exception as e: # noqa: BLE001
        self.get_logger().warn(f"color Konvertierungsfehler: {e}", throttle_duration_sec=5.0)

def _on_depth(self, msg):
    try:
        self._depth = depth_to_mm(msg)
    except Exception as e: # noqa: BLE001
        self.get_logger().warn(f"depth Konvertierungsfehler: {e}", throttle_duration_sec=5.0)

def _on_info(self, msg):
    k = msg.k # 3x3 row-major
    self._K = (k[0], k[4], k[2], k[5]) # fx, fy, cx, cy


# ---- Hilfsfunktionen ----

def _depth_m(self, u, v, win=None):
    """Robuste Tiefe um (u,v): Median über breites Fenster + MAD-Ausreißer-
    Eliminierung. Verwirft die nah-unsinnigen IR-Messungen am kleinen/glänzenden
    Schraubenkopf; setzt sich mit den umliegenden Oberflächenproben auf die echte Tiefe. Meter oder None."""
    if self._depth is None:

```

```

        return None

    if win is None:

        win = self._depth_win

    h, w = self._depth.shape

    u, v = int(round(u)), int(round(v))

    if not (0 <= u < w and 0 <= v < h):

        return None

    patch = self._depth[max(0, v - win):v + win + 1, max(0, u - win):u + win + 1]

    vals = patch[patch > 0].astype(np.float32)

    if vals.size < self._depth_min_valid:

        return None

    med = float(np.median(vals))

    # Ausreißer-Eliminierung per MAD (nahe IR-Reflexionen + ferne Lücken-Pixel)

    mad = float(np.median(np.abs(vals - med)))

    if mad > 0.0:

        keep = vals[np.abs(vals - med) <= self._depth_mad_k * mad]

        if keep.size >= self._depth_min_valid:

            med = float(np.median(keep))

    return med / 1000.0

def _deproject(self, u, v, z):

    """Pixel + Tiefe (m) -> optischer Frame 3D (m). Z vorne, X rechts, Y unten."""

    fx, fy, cx, cy = self._K

    x = (u - cx) * z / fx

    y = (v - cy) * z / fy

    return (x, y, z)

def _to_base(self, p_opt, src_frame):

    """Punkt aus optischem Frame nach robot_base überführen. (x,y,z) oder None."""

    try:

        tf = self.tf_buffer.lookup_transform(

            self.base_frame, src_frame, rclpy.time.Time())

    except Exception as e: # noqa: BLE001

        self.get_logger().warn(f"kein TF ({src_frame}->{self.base_frame}): {e}",

                                throttle_duration_sec=5.0)

    return None

t = tf.transform.translation

```

```

    r = tf.transform.rotation

    rx, ry, rz = qrot((r.x, r.y, r.z, r.w), p_opt)

    return (rx + t.x, ry + t.y, rz + t.z)

# ---- Hilfsfunktionen Reichweiten-Kreis ----

def _opt_tf(self, src_frame):
    """robot_base -> optischer Frame Transform als (quat, trans) mit EINEM lookup holen."""
    if self._K is None or not self.enable_3d:
        return None

    try:
        tf = self.tf_buffer.lookup_transform(src_frame, self.base_frame, rclpy.time.Time())

    except Exception: # noqa: BLE001
        return None

    t = tf.transform.translation
    r = tf.transform.rotation

    return ((r.x, r.y, r.z, r.w), (t.x, t.y, t.z))

def _proj(self, p_base, otf):
    """base-Punkt -> Pixel (u,v). Hinter der Kamera -> None."""
    q, t = otf

    x, y, z = qrot(q, p_base)

    x += t[0]; y += t[1]; z += t[2]

    if z <= 1e-3:
        return None

    fx, fy, cx, cy = self._K

    return (fx * x / z + cx, fy * y / z + cy)

def _reach_polygon(self, otf, radius, n=72):
    """Kreis in base-XY auf Höhe reach_z in eine Pixelliste projizieren."""
    pts = []

    for i in range(n):
        a = 2.0 * math.pi * i / n

        p = self._proj((radius * math.cos(a), radius * math.sin(a), self.reach_z), otf)

        pts.append(None if p is None else (int(round(p[0])), int(round(p[1]))))

    return pts

def _screw_base(self, d):

```



```

        """Die base-3D (mit Tiefe) einer Erkennung zurückgeben. (x,y,z) oder None."""

        if self._K is None or not self.enable_3d or self._latest is None:

            return None

        z = self._depth_m(*d["center_px"])

        if z is None:

            return None

        return self._to_base(self._deproject(*d["center_px"], z), self._latest[2])

# ---- 3D-Achsen-(Neigungs-)Messung ----

def _mask_points_3d(self, mask):

    """Pixel der Schraubenmaske mit roher aligned-depth in optischen-Frame-3D umwandeln.

    Unter-Sampling (Geschwindigkeit), Filter depth>0, eliminiert Tiefen-Ausreißer per MAD.

    Returns Nx3 (optischer Frame, m) oder None (zu wenige Punkte)."""

    if self._depth is None or self._K is None:

        return None

    ys, xs = np.where(mask > 127)

    if xs.size < self._axis_min_points:

        return None

    st = self._axis_subsample

    if st > 1:

        ys, xs = ys[::st], xs[::st]

    h, w = self._depth.shape

    ok = (xs >= 0) & (xs < w) & (ys >= 0) & (ys < h)

    ys, xs = ys[ok], xs[ok]

    d_mm = self._depth[ys, xs].astype(np.float32)

    valid = d_mm > 0

    ys, xs, d_mm = ys[valid], xs[valid], d_mm[valid]

    if d_mm.size < self._axis_min_points:

        return None

    # MAD-Ausreißer-Eliminierung (Hintergrund / nahe IR-Reflexion am Maskenrand)

    med = float(np.median(d_mm))

    mad = float(np.median(np.abs(d_mm - med)))

    if mad > 0.0:

        keep = np.abs(d_mm - med) <= self._depth_mad_k * mad

        ys, xs, d_mm = ys[keep], xs[keep], d_mm[keep]

    if d_mm.size < self._axis_min_points:

```

```

        return None

    fx, fy, cx, cy = self._K

    z = d_mm / 1000.0

    x = (xs.astype(np.float32) - cx) * z / fx
    y = (ys.astype(np.float32) - cy) * z / fy

    return np.column_stack([x, y, z]).astype(np.float64)

def _axis_3d(self, pts):
    """PCA aus Punktwolke -> (größter Eigenvektor=Achsenrichtung(optisch), linearity, N).

    linearity = (l1-l2)/l1 ∈ [0,1]; nahe 1 = ausgeprägt linear (zuverlässig)."""

    n = pts.shape[0]

    c = pts.mean(axis=0)

    cov = np.cov((pts - c).T)

    w, V = np.linalg.eigh(cov)          # aufsteigend sortiert

    l1, l2 = float(w[2]), float(w[1])

    linearity = (l1 - l2) / l1 if l1 > 1e-12 else 0.0

    axis_opt = V[:, 2]                 # größter Eigenvektor

    return axis_opt, linearity, n

def _mask_dims_mm(self, mask, angle_deg, z):
    """Major/Minor px-Ausdehnung der Maske -> ABSOLUTE mm (Kamera-Maßstab z/fx).

    KEINE 44mm-Längen-ANNAHME; der Maßstab kommt direkt aus der Tiefe (z):

    bei Tiefe z ist lpx = z/fx Meter. Gibt (länge_mm, dicke_mm)

    oder None zurück. dicke = das die Silhouette senkrecht schneidende breiteste Maß = das Minimum,
    das der Greifer öffnen muss (Schraubenkopf = breiteste Stelle)."""

    if self._K is None:

        return None

    ys, xs = np.where(mask > 127)

    if xs.size < 10:

        return None

    fx, fy, cx, cy = self._K

    cxm, cym = float(np.mean(xs)), float(np.mean(ys))

    a = math.radians(angle_deg)

    ca, sa = math.cos(a), math.sin(a)

    major = (xs - cxm) * ca + (ys - cym) * sa      # Projektion auf Längsachse

```

```

minor = -(xs - cxm) * sa + (ys - cym) * ca    # Kurzachse (senkrecht)

len_px = float(major.max() - major.min())

# Breite: ausreißer-robustes Perzentil (97-3). max-min bläht auf.

wid_px = float(np.percentile(minor, 97) - np.percentile(minor, 3))

mmp = z / fx * 1000.0                        # z(m), fx(px) -> mm/px

return len_px * mmp, wid_px * mmp

```

```

# ---- Hauptschleife ----

```

```

def _on_timer(self):

```

```

    if self._latest is None:

```

```

        self.get_logger().warn("noch kein color (Kamera an? use_camera:=true)",
                                throttle_duration_sec=5.0)

        return

```

```

    bgr, stamp, frame_id = self._latest

```

```

    dets = detect_screws(bgr, self.model, conf=self.conf)

```

```

    if not dets:

```

```

        self.pub_overlay.publish(bgr_to_image(self._draw(bgr, []), frame_id, stamp))

        self.get_logger().info("keine Schrauben", throttle_duration_sec=3.0)

        return

```

```

    if not self.enable_3d:

```

```

        top = dets[0]

        self.pub_overlay.publish(bgr_to_image(self._draw(bgr, dets, chosen=top), frame_id, stamp))

        self.get_logger().info(
            f"{len(dets)} Schrauben | beste: {top['class_name']} {top['conf']:.2f} "
            f"Mitte=({top['center_px'][0]:.0f},{top['center_px'][1]:.0f}) Winkel={top['angle_deg']:.0f}°")

        return

```

```

    if self._K is None:

```

```

        self.pub_overlay.publish(bgr_to_image(self._draw(bgr, dets), frame_id, stamp))

        self.get_logger().warn("camera_info nicht eingetroffen", throttle_duration_sec=5.0)

        return

```

```

# Auswahl (DETERMINISTISCH): aus den Kandidaten mit gültiger Tiefe+TF –

```

```

# 1) der nächste Kandidat innerhalb sticky_radius zum zuletzt VERÖFFENTLICHTEN Ziel (klebrige

```

```

# Verfolgung: conf-Jitter kann das Ziel nicht zwischen zwei Schrauben springen lassen),

```

```

# 2) ohne klebrige Übereinstimmung die höchste confidence (dets nach conf sortiert).

cands = []

for d in dets:

    z = self._depth_m(*d["center_px"])

    if z is None:

        continue

    p_opt = self._deproject(*d["center_px"], z)

    p_base = self._to_base(p_opt, frame_id)

    if p_base is None:

        continue

    cands.append((d, z, p_opt, p_base))

if not cands:

    self.pub_overlay.publish(bgr_to_image(self._draw(bgr, dets), frame_id, stamp))

    self.get_logger().warn(f"{len(dets)} Schrauben, aber bei keiner ist Tiefe/TF gueltig",

                           throttle_duration_sec=3.0)

    return

# --- KAMERA-BILDRAND (FOV) ELIMINIERUNG ---

# VOR der Markierung als Ziel: ist die Schraube in der sicheren Zone des Kamera-Sichtfelds

# und GANZ im Bild? Am FOV-Rand ist die Tiefe unzuverlässig, eine abgeschnittene Maske verfälscht
Winkel/Greifpunkt.

# Schrauben, deren Mitte außerhalb der sicheren Zone liegt ODER deren Maske den Bildrand berührt, werden
eliminiert.

if self._frame_margin_frac > 0.0:

    H, W = bgr.shape[:2]

    mx = self._frame_margin_frac * W

    my = self._frame_margin_frac * H

    in_frame = []

    for c in cands:

        cu, cv = c[0]["center_px"]

        if not (mx <= cu <= W - mx and my <= cv <= H - my):

            continue # Mitte außerhalb der sicheren FOV-Zone

        m = c[0]["mask"] > 127

        if m[0, :].any() or m[-1, :].any() or m[:, 0].any() or m[:, -1].any():

            continue # Maske berührt den Bildrand → halb-abgeschnittene Schraube

        in_frame.append(c)

    dropped = len(cands) - len(in_frame)

    if not in_frame:

```

```

self.pub_overlay.publish(bgr_to_image(self._draw(bgr, dets), frame_id, stamp))

self.get_logger().warn(

    f"{len(cands)} Schrauben, aber alle am Rand des Kamerabilds/ragen hinaus – "

    "keine geeignete Schraube im sicheren FOV, Roboter wartet", throttle_duration_sec=3.0)

return

if dropped:

    self.get_logger().info(

        f"Kamerarand: {dropped} Schrauben eliminiert (ausserhalb FOV/abgeschnitten), "

        f"{len(in_frame)} Kandidaten verbleiben", throttle_duration_sec=3.0)

cands = in_frame

# --- REICHWEITEN- (IK) INNENGRENZEN-ELIMINIERUNG ---

# Der ROTE Innenkreis im Overlay = reach_radius_min. Schrauben mit kleinerem base-XY-Radius

# sind dem Roboter ZU NAH → im top-down-Griff biegt sich der Arm zu stark (IK-Limit).

# Der Kreis wurde NUR gezeichnet; hier wenden wir ihn auch auf die Zielauswahl an:

# Schrauben innerhalb des Innenkreises werden aus den Kandidaten AUSGESCHLOSSEN, nicht als Ziel markiert.

if self.reach_radius_min > 0.0:

    in_reach = []

    for c in cands:

        pb = c[3] # p_base (base XY, m)

        if math.hypot(pb[0], pb[1]) < self.reach_radius_min:

            continue # innerhalb des roten Innenkreises → zu nah, IK-Limit

        in_reach.append(c)

    dropped = len(cands) - len(in_reach)

    if not in_reach:

        self.pub_overlay.publish(bgr_to_image(self._draw(bgr, dets), frame_id, stamp))

        self.get_logger().warn(

            f"{len(cands)} Schrauben, aber alle im roten Innenkreis "

            f"(r<{self.reach_radius_min*1000:.0f}mm, zu nah am Roboter/IK-Limit) – "

            "Roboter wartet", throttle_duration_sec=3.0)

        return

    if dropped:

        self.get_logger().info(

            f"IK-Innengrenze: {dropped} Schrauben eliminiert (r<{self.reach_radius_min*1000:.0f}mm zu nah), "

            f"{len(in_reach)} Kandidaten verbleiben", throttle_duration_sec=3.0)

cands = in_reach

```

```

# --- ZIELAUSWAHL: die eine "mittigste + bestsichtbare" Schraube ---

# Mitte = Pixel-Centroid der Kandidatenschrauben (= Schraubenhaufen / Kistenmitte).
centers = np.array([c[0]["center_px"] for c in cands], dtype=float)
centroid = centers.mean(axis=0)
dists = np.linalg.norm(centers - centroid, axis=1)

# HARTE ECKEN-ELIMINIERUNG: bbox der Kandidaten (innere Kistengrenze); randnahe
# (Kistenwand → breiter Greifer stößt an) Schrauben aus den Kandidaten AUSSCHLIESSEN.
keep = list(range(len(cands)))

if len(cands) >= self._corner_min_screws:
    umin, vmin = centers.min(axis=0)
    umax, vmax = centers.max(axis=0)
    mx = self._corner_edge_frac * max(umax - umin, 1.0)
    my = self._corner_edge_frac * max(vmax - vmin, 1.0)
    keep = [i for i in range(len(cands))
            if umin + mx <= centers[i][0] <= umax - mx
            and vmin + my <= centers[i][1] <= vmax - my]
    if not keep:
        self.pub_overlay.publish(bgr_to_image(
            self._draw(bgr, dets, centroid=centroid), frame_id, stamp))
        self.get_logger().warn(
            f"{len(cands)} Schrauben, aber alle am Kistenrand (Ecke) – "
            "keine geeignete Schraube in der Mitte, Roboter wartet", throttle_duration_sec=3.0)
    return

# Score = center_weight*Mittigkeit + conf_weight*confidence. Höchste wählen.
maxd = float(dists.max()) if float(dists.max()) > 1e-6 else 1.0

def _score(i):
    centrality = 1.0 - dists[i] / maxd          # Mitte=1, am weitesten=0
    return (self._center_weight * centrality
            + self._conf_weight * float(cands[i][0]["conf"]))

chosen_i = max(keep, key=_score)

# KLEBRIGE VERFOLGUNG: gibt es nach der Eliminierung einen Kandidaten innerhalb
# sticky_radius zum vorherigen Ziel, diesen beibehalten – Jitter soll das Ziel nicht zwischen zwei

```

Schrauben springen lassen.

```
if self._last_pub_p is not None and len(keep) > 1:

    best = min(keep, key=lambda i: math.dist(tuple(cands[i][3]), self._last_pub_p))

    if math.dist(tuple(cands[best][3]), self._last_pub_p) <= self._sticky_radius:

        chosen_i = best

d, z, p_opt, p_base = cands[chosen_i]

u, v = d["center_px"]

# --- KOPF-BEWUSSTER GRIFF: Ziel auf den geraden Schaft unter dem Kopf verschieben ---
# Aus der Maskenform die Kopfkante finden; Greifpunkt direkt unter den Kopf verschieben.
# Bei gültiger Tiefe wird das Ziel aus diesem Punkt erzeugt (sonst bleibt der Centroid).

grip_px = None

head_px = None

if self._grip_under_head:

    g = head_aware_grip(d["mask"] > 127, d["center_px"][0], d["center_px"][1],

                        d["angle_deg"], inset_frac=self._grip_inset_frac)

    head_px = g["head_px"]

    if g["head_known"]:

        gu, gv = g["grip_px"]

        gz = self._depth_m(gu, gv)

        if gz is not None:

            gp_opt = self._deproject(gu, gv, gz)

            gp_base = self._to_base(gp_opt, frame_id)

            if gp_base is not None:

                u, v = gu, gv

                z, p_opt, p_base = gz, gp_opt, gp_base

                grip_px = (gu, gv)

# --- 3D-Achsen-(Neigungs-)Messung: PCA aus der Maskentiefe ---

pts = self._mask_points_3d(d["mask"])

axis_base = None

linearity = 0.0

npts = 0

if pts is not None:

    axis_opt, linearity, npts = self._axis_3d(pts)

    tip_base = self._to_base(

        (p_opt[0] + axis_opt[0] * 0.05,
```

```

        p_opt[1] + axis_opt[1] * 0.05,

        p_opt[2] + axis_opt[2] * 0.05), frame_id)

    if tip_base is not None:

        v_axis = np.array([tip_base[0] - p_base[0],

                           tip_base[1] - p_base[1],

                           tip_base[2] - p_base[2]])

        nb = float(np.linalg.norm(v_axis))

        if nb > 1e-9:

            axis_base = v_axis / nb

    grasp_q = (grasp_quat_from_axis(axis_base, top_down=self._top_down)

               if axis_base is not None else None)

    z_ok = self._target_z_min <= p_base[2] <= self._target_z_max

    reliable = (pts is not None and grasp_q is not None

               and npts >= self._axis_min_points

               and linearity >= self._axis_min_linearity

               and z_ok)

    tilt_deg = 0.0

    if axis_base is not None:

        s2 = abs(float(axis_base[2]))

        tilt_deg = math.degrees(math.asin(min(1.0, s2)))

    # --- Maßbestimmung (Länge + Dicke), absolute mm, KEINE 44mm-Annahme ---

    dims = self._mask_dims_mm(d["mask"], d["angle_deg"], z)

    dim_txt = f" lng={dims[0]:.0f} dck={dims[1]:.1f}mm" if dims else ""

    mode = None

    if reliable:

        if self._top_down:

            mode = "SENKRECHT"    # Annäherung immer gerade-abwärts (top_down erzwungen)

        else:

            mode = "NEIGUNGS-AUSGERICHTET" if tilt_deg >= self._grasp_tilt_min_deg else "top-down"

    self._mode_hist.append(mode)

    steady = (mode is not None

              and len(self._mode_hist) == self._mode_hist.maxlen

              and all(m == mode for m in self._mode_hist))

```



```

if steady:

    base_txt = (f"x={p_base[0]*1000:.0f} y={p_base[1]*1000:.0f} z={p_base[2]*1000:.0f}mm "

                f"tilt={tilt_deg:.0f} lin={linearity:.2f} [{mode}]{dim_txt}")

    self.pub_overlay.publish(

        bgr_to_image(self._draw(bgr, dets, chosen=d, base_txt=base_txt,

                                axis3d=(p_base, axis_base),

                                grip_px=grip_px, head_px=head_px,

                                centroid=centroid), frame_id, stamp))

    # WAHRNEHMUNGS-OFFSET nur auf das VERÖFFENTLICHTES Ziel anwenden –

    # Overlay/sticky bleiben roh (Rückprojektion soll auf der Welle liegen).

    p_pub = (p_base[0] + self._offset[0],

             p_base[1] + self._offset[1],

             p_base[2] + self._offset[2])

    self._publish_target(p_pub, grasp_q, stamp)

    self.pub_target_color.publish(Int32(data=int(d["class_id"]))) # gleichzeitig mit /vida/target

    info = (f"{d['class_name'].upper()} Schraube conf={d['conf']:.2f} | "

            f"x={p_pub[0]*1000:.0f} y={p_pub[1]*1000:.0f} z={p_pub[2]*1000:.0f}mm")

    self.pub_target_info.publish(String(data=info))

    self._last_pub_p = tuple(p_base) # klebrige Auswahl + Referenz für gesperrtes Overlay (ROH)

    self._last_axis_base = tuple(axis_base) if axis_base is not None else None

    self._publish_marker(p_pub, grasp_q, d, stamp)

    self._publish_axis(p_pub, axis_base, stamp)

    self.get_logger().info(

        f"ZIEL {d['class_name']} conf={d['conf']:.2f} | BASE(korr.) "

        f"x={p_pub[0]*1000:.0f} y={p_pub[1]*1000:.0f} z={p_pub[2]*1000:.0f}mm "

        f"| tilt={tilt_deg:.0f}° lin={linearity:.2f} N={npts}{dim_txt} | mod={mode} -> /vida/target")

elif reliable:

    # Messung gut, aber Modus noch nicht in Folge übereinstimmend – nicht veröffentlichen, warten

    base_txt = (f"x={p_base[0]*1000:.0f} y={p_base[1]*1000:.0f} z={p_base[2]*1000:.0f}mm "

                f"tilt={tilt_deg:.0f} [{mode}??] Konsens wird erwartet{dim_txt}")

    self.pub_overlay.publish(

        bgr_to_image(self._draw(bgr, dets, chosen=d, base_txt=base_txt,

                                axis3d=(p_base, axis_base),

                                grip_px=grip_px, head_px=head_px,

                                centroid=centroid), frame_id, stamp))

    self.get_logger().info(

```

```

        f"Konsens wird erwartet ({mode}, {sum(1 for m in self._mode_hist if m == mode)})"

        f"/{self._mode_hist.maxlen}) | tilt={tilt_deg:.0f}° (/vida/target NICHT)")

else:

    if pts is not None and grasp_q is not None and not z_ok:

        warn = (f"BASE z={p_base[2]*1000:.0f}mm ausserhalb Grenze "

                f"[{self._target_z_min*1000:.0f},{self._target_z_max*1000:.0f}] - depth Muell")

    elif pts is not None:

        warn = f"tilt UNZUVERLAESSIG lin={linearity:.2f} N={npts} - Schrauben umruehren"

    else:

        warn = "Tiefe unzureichend - Schrauben umruehren"

    self.pub_overlay.publish(

        bgr_to_image(self._draw(bgr, dets, chosen=d, base_txt=warn,

                                grip_px=grip_px, head_px=head_px,

                                centroid=centroid), frame_id, stamp))

    self.get_logger().warn(

        f"{warn} | conf={d['conf']:.2f} BASE x={p_base[0]*1000:.0f} "

        f"y={p_base[1]*1000:.0f} z={p_base[2]*1000:.0f}mm (/vida/target NICHT)",

        throttle_duration_sec=2.0)

def _publish_target(self, p, q, stamp):

    """q = Grasp-Orientierung (x,y,z,w), tcp +Y = Annäherungsachse. pick_tilt nutzt dies."""

    msg = PoseStamped()

    msg.header.stamp = stamp

    msg.header.frame_id = self.base_frame

    msg.pose.position.x, msg.pose.position.y, msg.pose.position.z = p

    msg.pose.orientation.x, msg.pose.orientation.y, msg.pose.orientation.z, msg.pose.orientation.w = q

    self.pub_target.publish(msg)

def _publish_marker(self, p, q, d, stamp):

    """Grasp-ANNAEHERUNGsachse (tcp +Y) mit zweiseitigem Pfeil zeigen: der Greifer fährt

    in dieser Richtung zur Schraube. Pfeilspitze = Grasp-Punkt (p), Schaftende = Gegenrichtung der

    Annäherung."""

    ay = qrot(q, (0.0, 1.0, 0.0)) # Richtung von tcp +Y in base = Annäherung

    m = Marker()

    m.header.frame_id = self.base_frame

    m.header.stamp = stamp

    m.ns = "vida_target"

```

```

m.id = 0

m.type = Marker.ARROW

m.action = Marker.ADD

tail = Point(x=p[0] - ay[0] * 0.06, y=p[1] - ay[1] * 0.06, z=p[2] - ay[2] * 0.06)

tip = Point(x=p[0], y=p[1], z=p[2])

m.points = [tail, tip]

m.scale.x, m.scale.y, m.scale.z = 0.004, 0.010, 0.0

col = CLASS_BGR[d["class_id"]] if d["class_id"] < len(CLASS_BGR) else (128, 128, 128)

m.color.b, m.color.g, m.color.r = col[0] / 255.0, col[1] / 255.0, col[2] / 255.0

m.color.a = 1.0

self.pub_marker.publish(m)

def _publish_axis(self, p, axis_base, stamp):

    """Die gemessene 3D-Längsachse der Schraube mit magenta ARROW (zweiseitig) zeigen (Diagnose)."""

    m = Marker()

    m.header.frame_id = self.base_frame

    m.header.stamp = stamp

    m.ns = "vida_axis"

    m.id = 0

    m.type = Marker.ARROW

    m.action = Marker.ADD

    half = 0.03

    p0 = Point(x=p[0] - axis_base[0] * half, y=p[1] - axis_base[1] * half,
               z=p[2] - axis_base[2] * half)

    p1 = Point(x=p[0] + axis_base[0] * half, y=p[1] + axis_base[1] * half,
               z=p[2] + axis_base[2] * half)

    m.points = [p0, p1]

    m.scale.x, m.scale.y, m.scale.z = 0.004, 0.008, 0.0 # shaft, head Durchmesser

    m.color.r, m.color.g, m.color.b, m.color.a = 1.0, 0.0, 1.0, 1.0 # magenta

    self.pub_axis.publish(m)

def _draw(self, bgr, dets, chosen=None, base_txt=None, axis3d=None,

          grip_px=None, head_px=None, centroid=None):

    import cv2

    ov = bgr.copy()

    # --- HAUFENMITTE: Mittenreferenz der Zielauswahl (magenta), Ecken-Eliminierung danach ---

```

```

if centroid is not None:

    mcx, mcy = int(centroid[0]), int(centroid[1])

    cv2.drawMarker(ov, (mcx, mcy), (255, 0, 255), cv2.MARKER_CROSS, 22, 2)

    cv2.putText(ov, "Mitte", (mcx + 10, mcy - 6),

                cv2.FONT_HERSHEY_SIMPLEX, 0.42, (255, 0, 255), 1, cv2.LINE_AA)

# --- REICHWEITEN-KREIS: um robot_base, auf Plattformhöhe ---

# Das INNERE des gezeichneten Kreises = mit senkrechtem Griff erreichbare Zone für Schrauben.
AMBER, GREENC, REDC = (0, 200, 255), (0, 200, 0), (0, 0, 255)

otf = self._opt_tf(self._latest[2]) if self._latest else None

if otf is not None:

    try:

        for radius, col, thick in ((self.reach_radius_max, AMBER, 1),

                                    (self.reach_radius, GREENC, 2),

                                    (self.reach_radius_min, REDC, 2)):

            if radius <= 0:

                continue

            poly = self._reach_polygon(otf, radius)

            valid = [p for p in poly if p is not None]

            if len(valid) >= 2:

                closed = all(p is not None for p in poly)

                cv2.polylines(ov, [np.array(valid, np.int32)], closed,

                              col, thick, cv2.LINE_AA)

            c = self._proj((0.0, 0.0, self.reach_z), otf)

            if c is not None:

                cc = (int(c[0]), int(c[1]))

                cv2.drawMarker(ov, cc, AMBER, cv2.MARKER_TILTED_CROSS, 16, 2)

                cv2.putText(ov, f"ROBOTER gruen {self.reach_radius_min*1000:.0f}-{self.reach_radius*1000:.0f}
amber<{self.reach_radius_max*1000:.0f}mm (innen rot=zu nah)",

                            (cc[0] + 8, cc[1] + 4), cv2.FONT_HERSHEY_SIMPLEX,

                            0.42, AMBER, 1, cv2.LINE_AA)

        except Exception: # noqa: BLE001

            pass

# --- GESPERRTES ZIEL: zuletzt veröffentlichtes Ziel + dessen Achse; bleibt im Overlay

# FEST (cyan), selbst wenn der Roboter die Schraube verdeckt und die Live-Erkennung wegfällt ---

if otf is not None and getattr(self, "_last_pub_p", None) is not None:

```

```

CYAN = (255, 255, 0)

lp = self._proj(self._last_pub_p, otf)

if lp is not None:
    lx, ly = int(lp[0]), int(lp[1])

    cv2.drawMarker(ov, (lx, ly), CYAN, cv2.MARKER_TILTED_CROSS, 18, 1)

    cv2.putText(ov, "GESPERRT", (lx + 8, ly + 16),
                cv2.FONT_HERSHEY_SIMPLEX, 0.42, CYAN, 1, cv2.LINE_AA)

    lab = getattr(self, "_last_axis_base", None)

    if lab is not None:
        half = 0.03

        pa = self._proj((self._last_pub_p[0] - lab[0] * half,
                        self._last_pub_p[1] - lab[1] * half,
                        self._last_pub_p[2] - lab[2] * half), otf)

        pb = self._proj((self._last_pub_p[0] + lab[0] * half,
                        self._last_pub_p[1] + lab[1] * half,
                        self._last_pub_p[2] + lab[2] * half), otf)

        if pa is not None and pb is not None:
            cv2.line(ov, (int(pa[0]), int(pa[1])), (int(pb[0]), int(pb[1])),
                    CYAN, 1, cv2.LINE_AA)

shown = 0

for d in dets:
    # OVERLAY-VEREINFACHUNG (Wunsch): Schrauben mit niedriger conf GAR NICHT zeichnen; bei den gezeichneten
    # nur ein PUNKT – KEIN Kreis (Reichweitenring) und kein Text (Farbe/conf/r=).

    if float(d.get("conf", 1.0)) < self._display_min_conf:
        continue

    shown += 1

    color = CLASS_BGR[d["class_id"]] if d["class_id"] < len(CLASS_BGR) else (128, 128, 128)

    mb = d["mask"] > 127

    ov[mb] = (ov[mb] * 0.5 + np.array(color) * 0.5).astype(np.uint8)

    cx, cy = int(d["center_px"][0]), int(d["center_px"][1])

    cv2.circle(ov, (cx, cy), 3, (0, 0, 255), -1) # nur Punkt

# GEWAELTE Schraube: Längsachsen-Linie (immer) + Kopfmarkierung + Greif-Fadenkreuz.

if chosen is not None:
    GREEN, ORANGE, REDH = (0, 255, 0), (0, 165, 255), (0, 0, 255)

    cx, cy = int(chosen["center_px"][0]), int(chosen["center_px"][1])

```

```

# LAENGSSACHSE (immer sichtbar): in welche Richtung die Schraube lang ist.

ang = math.radians(float(chosen.get("angle_deg", 0.0)))

half = 0.5 * float(chosen.get("major_len_px", 40.0))

dx, dy = math.cos(ang) * half, math.sin(ang) * half

cv2.line(ov, (int(cx - dx), int(cy - dy)), (int(cx + dx), int(cy + dy)),

        ORANGE, 2, cv2.LINE_AA)

# KOPFkante (falls erkannt): der Griff verschiebt sich direkt DARUNTER.

if head_px is not None:

    hx, hy = int(head_px[0]), int(head_px[1])

    cv2.circle(ov, (hx, hy), 6, REDH, 2, cv2.LINE_AA)

    cv2.putText(ov, "KOPF", (hx + 7, hy - 6),

                cv2.FONT_HERSHEY_SIMPLEX, 0.45, REDH, 1, cv2.LINE_AA)

# GRIFF/ZIEL-Punkt: Schaft unter dem Kopf (sonst Centroid). BLAU (Wunsch: nicht mit

#   den Schraubenfarben verwechseln → unterscheidbares blaues Kreuz + "ZIEL"-Text).

BLUE = (255, 0, 0)

gx, gy = (int(grip_px[0]), int(grip_px[1])) if grip_px is not None else (cx, cy)

cv2.drawMarker(ov, (gx, gy), BLUE, cv2.MARKER_CROSS, 16, 2)

# ZIEL + FARBE der gewaehlten Welle (Wunsch 2026-07-16): Farbname in der

# Klassenfarbe mit dunkler Kontur (auf jedem Hintergrund lesbar), duenn (1).

cid = int(chosen.get("class_id", -1))

farbe = CLASS_DE[cid] if 0 <= cid < len(CLASS_DE) else "?"

fcol = CLASS_BGR[cid] if 0 <= cid < len(CLASS_BGR) else (200, 200, 200)

ziel_txt = f"ZIEL: {farbe}"

cv2.putText(ov, ziel_txt, (gx + 10, gy - 6),

            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 3, cv2.LINE_AA)

cv2.putText(ov, ziel_txt, (gx + 10, gy - 6),

            cv2.FONT_HERSHEY_SIMPLEX, 0.5, fcol, 1, cv2.LINE_AA)

# Zusaetzlich fest oben rechts – auch wenn das Fadenkreuz am Rand liegt.

cv2.putText(ov, ziel_txt, (ov.shape[1] - 150, 20),

            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 3, cv2.LINE_AA)

cv2.putText(ov, ziel_txt, (ov.shape[1] - 150, 20),

            cv2.FONT_HERSHEY_SIMPLEX, 0.5, fcol, 1, cv2.LINE_AA)

if base_txt:

    # Untere xyz-Info: klein, duenn (thickness=1), WEISS – soll nicht ueberladen.

```

```

        cv2.putText(ov, base_txt, (10, ov.shape[0] - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.4, (255, 255, 255), 1, cv2.LINE_AA)

# --- 3D-ACHSE (PCA) der GEWAELHTEN Schraube ins Bild projiziert (Prüfung) ---
# MAGENTA-Linie = gemessene Schraubenachse; die Linie sollte entlang des Schraubenkörpers
# verlaufen. GELBER Kreis = base->rückprojizierter Greifpunkt;
# fällt er nicht auf die Schraube, sind deprojection/TF inkonsistent (Kalibrierung).
if axis3d is not None and otf is not None:
    p0, ax = axis3d

    MAGENTA, YELLOW = (255, 0, 255), (0, 255, 255)

    L = 0.04 # Achse um ±4 cm verlängern (insgesamt ~8 cm Linie)

    a = self._proj((p0[0] - ax[0] * L, p0[1] - ax[1] * L, p0[2] - ax[2] * L), otf)
    b = self._proj((p0[0] + ax[0] * L, p0[1] + ax[1] * L, p0[2] + ax[2] * L), otf)

    if a is not None and b is not None:
        cv2.line(ov, (int(a[0]), int(a[1])), (int(b[0]), int(b[1])),
                MAGENTA, 2, cv2.LINE_AA)

        cv2.circle(ov, (int(b[0]), int(b[1])), 4, MAGENTA, -1)

    g = self._proj(p0, otf)

    if g is not None:
        cv2.circle(ov, (int(g[0]), int(g[1])), 6, YELLOW, 2, cv2.LINE_AA)

cv2.putText(ov, f"{shown}/{len(dets)} Schrauben (conf>={self._display_min_conf:.2f})", (10, 20),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1, cv2.LINE_AA)

return ov

```

```

def main():
    rclpy.init()

    node = VidaDetector()

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass

    finally:
        node.destroy_node()
        rclpy.try_shutdown()

```

```
if __name__ == "__main__":  
    main()
```


start_mycobot.sh

```
#!/bin/bash

# myCobot 280 JN: MoveIt2 + RViz + ros2_control + D435i camera.

# Default: ECHTER Roboter (mycobot_hardware/MyCobotHardware via pymycobot).

# Verwendung:

#   ./start_mycobot.sh           → echter Roboter (ttyTHS1)
#   ./start_mycobot.sh sim       → fake_components (Sim ohne Roboter)

# Locale C erforderlich (Qt/RViz Punkt-Komma-Problem).

export LC_ALL=C
export LC_NUMERIC=C
export LANG=C

# CycloneDDS: Jetson default rmem_max (212KB) reicht nicht für Bild-Nachrichten
sudo sysctl -w net.core.rmem_max=8388608 net.core.wmem_max=8388608 \
    net.core.rmem_default=8388608 net.core.wmem_default=8388608 \
    2>/dev/null

# Jetson GPU: railgate aus + fan max (RViz OGRE segfault-Vorbeugung)
echo 0 | sudo tee /sys/devices/57000000.gpu/railgate_enable >/dev/null 2>&1
sudo sh -c 'echo 255 > /sys/devices/pwm-fan/target_pwm' 2>/dev/null

# D435i USB autosuspend ausschalten
for dev in /sys/bus/usb/devices/*/idVendor; do
    [ "$(cat "$dev" 2>/dev/null)" = "8086" ] && echo on | sudo tee "$(dirname "$dev")/power/control" >/dev/null 2>&1
done

source /opt/ros/galactic/setup.bash
source "$HOME/ros2_ws/install/setup.bash"

cd "$HOME/ros2_ws" || exit 1

if [ "${1:-}" = "sim" ]; then
    export USE_FAKE_HARDWARE="true"
    echo "[start_mycobot] SIM MODE — fake_components/GenericSystem"
else
```

```

export USE_FAKE_HARDWARE="false"

echo "[start_mycobot] ECHTER ROBOTER – mycobot_hardware/MyCobotHardware @ ${MYCOBOT_PORT:-/dev/ttyTHS1}"
fi

ros2 launch mycobot_moveit_config demo.launch.py use_camera:=true &
LAUNCH_PID=$!

echo "[start_mycobot] Warten bis die Controller bereit sind..."
for i in $(seq 1 30); do

    if ros2 control list_controllers 2>/dev/null | grep -q "arm_controller.*active"; then

        break
    fi

    sleep 1
done

echo "[start_mycobot] Alle Gelenke werden zum Nullpunkt gesendet..."
ros2 action send_goal /arm_controller/follow_joint_trajectory \

    control_msgs/action/FollowJointTrajectory "{

    trajectory: {

        joint_names: [joint2_to_joint1, joint3_to_joint2, joint4_to_joint3, joint5_to_joint4, joint6_to_joint5,
joint6output_to_joint6],

        points: [{positions: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0], time_from_start: {sec: 4, nanosec: 0}}]

    }

}" 2>/dev/null && echo "[start_mycobot] Arm → Null OK"

ros2 action send_goal /gripper_controller/follow_joint_trajectory \

    control_msgs/action/FollowJointTrajectory "{

    trajectory: {

        joint_names: [gripper_controller],

        points: [{positions: [0.0], time_from_start: {sec: 2, nanosec: 0}}]

    }

}" 2>/dev/null && echo "[start_mycobot] Gripper → Null OK"

wait $LAUNCH_PID

```

target_latch.py

```
#!/usr/bin/env python3

# =====
# target_latch.py - /vida/target-Brücke für zweiphasigen (Vision→RViz) Pick.
#
# Zwei Modi:
#   capture <Datei> : hört auf /vida/target (PoseStamped); zeigt das aktuellste
#                   Ziel. Wenn der Benutzer ENTER drückt, wird das letzte Ziel in
#                   <Datei> (JSON) GESPEICHERT und beendet. So bleibt die Koordinate
#                   auch dann erhalten, wenn der Detector geschlossen wird
#                   (Schraube statisch → Koordinate bleibt gültig).
#   replay <Datei> : liest die Pose aus <Datei> und veröffentlicht sie mit 5 Hz
#                   ERNEUT auf /vida/target (pick_tilt empfängt das unverändert
#                   wartende Ziel) + erzeugt einen in RViz sichtbaren Marker
#                   (grüne Kugel + Pfeil der Grasp-Achse).
#
# Verwendung:
#   python3 target_latch.py capture /tmp/vida_target.json
#   python3 target_latch.py replay /tmp/vida_target.json
# =====

import sys, json, select, time

import rclpy

from rclpy.node import Node

from geometry_msgs.msg import PoseStamped
from visualization_msgs.msg import Marker

from std_msgs.msg import String

def _fmt(p):
    return (f"BASE x={p.pose.position.x*1000:.0f} y={p.pose.position.y*1000:.0f} "
           f"z={p.pose.position.z*1000:.0f}mm  q=({p.pose.orientation.x:.3f}, "
           f"{p.pose.orientation.y:.3f},{p.pose.orientation.z:.3f},{p.pose.orientation.w:.3f})"
           f"  frame={p.header.frame_id}")

def capture(path):
```

```

rclpy.init()

n = Node("target_capture")

state = {"last": None, "n": 0, "info": None}


def cb(m):

    state["last"] = m

    state["n"] += 1


def info_cb(m):

    state["info"] = m.data    # "KIRMIZI Schraube conf=0.92 | x=.. y=.. z=..mm"


n.create_subscription(PoseStamped, "/vida/target", cb, 1)
n.create_subscription(String, "/vida/target_info", info_cb, 1)

print("[target_latch] hoere auf /vida/target. Sobald die Erkennung im Overlay sitzt,")
print("[target_latch] in diesem Terminal ENTER druecken → letztes Ziel wird gespeichert.")
print("[target_latch] (wenn noch kein Ziel da ist, sagt ENTER 'kein Ziel', warten weiter.)")

stdin_ok = True

while rclpy.ok():

    rclpy.spin_once(n, timeout_sec=0.1)

    r, _, _ = (select.select([sys.stdin], [], [], 0.0) if stdin_ok else ([], [], []))

    if r:

        line = sys.stdin.readline()

        if line == "":    # EOF (Ctrl+D / nicht-interaktiv) – keinen Busy-Loop machen

            print("[target_latch] stdin geschlossen (EOF) – kein interaktives capture; warten.")

            stdin_ok = False

            continue

        if state["last"] is None:

            print("[target_latch] /vida/target noch nicht eingetroffen – Schraube schraeg legen/mischen,
warten.")

            continue

        p = state["last"]

        data = {

            "frame_id": p.header.frame_id or "robot_base",

            "px": p.pose.position.x, "py": p.pose.position.y, "pz": p.pose.position.z,

            "ox": p.pose.orientation.x, "oy": p.pose.orientation.y,

            "oz": p.pose.orientation.z, "ow": p.pose.orientation.w,

            "info": state["info"] or "",

```

```

    }

    with open(path, "w") as f:
        json.dump(data, f, indent=2)

    print(f"[target_latch] GESPEICHERT → {path}")

    # ZU WELCHER Schraube gefahren wird (Farbe) – soll in PHASE B feststehen, wenn die Kamera aus ist.

    if state["info"]:
        print(f"[target_latch] >>> GEWAHLTES ZIEL: {state['info']}")
    else:
        print(f"[target_latch] (keine Farbinfo eingetroffen – /vida/target_info fehlt)")

    print(f"[target_latch]   {_fmt(p)}   (insgesamt {state['n']} Ziele gesehen)")

    break

if state["n"] and state["n"] % 10 == 1:
    pass

n.destroy_node()
rclpy.shutdown()

```

```

def replay(path):
    with open(path) as f:
        d = json.load(f)

    rclpy.init()

    n = Node("target_replay")

    pub = n.create_publisher(PoseStamped, "/vida/target", 1)
    mpub = n.create_publisher(Marker, "/vida/preview_marker", 1)

    frame = d.get("frame_id", "robot_base")

    print(f"[target_latch] replay: {path} → /vida/target (5Hz) + /vida/preview_marker frame={frame}")

    if d.get("info"):
        print(f"[target_latch] >>> GEWAHLTES ZIEL: {d['info']}")

    print(f"[target_latch]   x={d['px']*1000:.0f} y={d['py']*1000:.0f} z={d['pz']*1000:.0f}mm")

def make_pose():
    m = PoseStamped()

    m.header.frame_id = frame

    m.header.stamp = n.get_clock().now().to_msg()

    m.pose.position.x = d["px"]; m.pose.position.y = d["py"]; m.pose.position.z = d["pz"]

    m.pose.orientation.x = d["ox"]; m.pose.orientation.y = d["oy"]

    m.pose.orientation.z = d["oz"]; m.pose.orientation.w = d["ow"]

```

```
return m
```

```
def make_marker(mid, mtype):
```

```
    mk = Marker()
```

```
    mk.header.frame_id = frame
```

```
    mk.header.stamp = n.get_clock().now().to_msg()
```

```
    mk.ns = "vida_preview"; mk.id = mid; mk.action = Marker.ADD
```

```
    mk.pose.position.x = d["px"]; mk.pose.position.y = d["py"]; mk.pose.position.z = d["pz"]
```

```
    mk.pose.orientation.x = d["ox"]; mk.pose.orientation.y = d["oy"]
```

```
    mk.pose.orientation.z = d["oz"]; mk.pose.orientation.w = d["ow"]
```

```
    if mtype == "sphere":
```

```
        mk.type = Marker.SPHERE
```

```
        mk.scale.x = mk.scale.y = mk.scale.z = 0.012
```

```
        mk.color.r = 0.1; mk.color.g = 1.0; mk.color.b = 0.1; mk.color.a = 0.95
```

```
    else: # Pfeil der Grasp-Achse (tcp +Y = Annäherungsachse; pick_tilt-Auslegung)
```

```
        mk.type = Marker.ARROW
```

```
        mk.scale.x = 0.06; mk.scale.y = 0.006; mk.scale.z = 0.006
```

```
        mk.color.r = 1.0; mk.color.g = 0.55; mk.color.b = 0.0; mk.color.a = 0.9
```

```
    return mk
```

```
try:
```

```
    while rclpy.ok():
```

```
        pub.publish(make_pose())
```

```
        mpub.publish(make_marker(0, "sphere"))
```

```
        mpub.publish(make_marker(1, "arrow"))
```

```
        rclpy.spin_once(n, timeout_sec=0.0)
```

```
        time.sleep(0.2)
```

```
except KeyboardInterrupt:
```

```
    pass
```

```
n.destroy_node()
```

```
rclpy.shutdown()
```

```
if __name__ == "__main__":
```

```
    if len(sys.argv) < 3 or sys.argv[1] not in ("capture", "replay"):
```

```
        print("Verwendung: target_latch.py capture|replay <Datei>")
```

```
        sys.exit(2)
```

```
(capture if sys.argv[1] == "capture" else replay)(sys.argv[2])
```

teach_boxes.py

```
#!/usr/bin/env python3

# =====

# teach_boxes.py – die 5 Box-Koordinaten für das SORTIEREN nach Farbe anlernen (teach).

#

# Zweck: nachdem pick_tilt die Schraube gegriffen hat und nach Home zurückgekehrt

# ist, fährt es UEBER die zur FARBE der Schraube gehörende Box und lässt sie aus

# der Luft fallen (SCHRITT 8). Dafür wird das (x,y,z)-Zentrum der Box jeder Farbe

# in robot_base benötigt. Dieses Skript lernt sie an.

#

# METHODE: den Roboter mit RViz MoveIt MotionPlanning (interaktiver Marker → Plan &

# Execute, IK in MoveIt) über die OEFFNUNG der jeweiligen Box bringen, dann speichern →

# die aktuelle robot_base-tcp-Pose wird unter dieser Farbe abgelegt. Dieses Skript

# STEUERT den Roboter NICHT.

#

# ZWEI MODI:

#   (interaktiv) python3 teach_boxes.py

#       Fragt für jede Farbe der Reihe nach per ENTER (vom Terminal).

#   (einmalig) python3 teach_boxes.py --once kirmizi

#       Speichert NUR die aktuelle TCP-Pose von 'kirmizi', behält die anderen.

#       (Damit Claude es aufruft, wenn du "speichern" sagst – du musst nicht ins Terminal tippen.)

#

# VORBEDINGUNG: der Stack (move_group + controllers + TF) muss LAUFEN, damit

# der robot_base-tcp-TF veröffentlicht wird. (teach_setup.sh richtet das ein.)

#

# SPEICHERUNG: die kanonischen Daten liegen in ~/ros2_ws/.box_teach.json; bei jeder

# Speicherung wird sowohl diese aktualisiert als auch die von pick_tilt geladene

# box_map.yaml neu erzeugt.

#

# Verwendung:

#   python3 teach_boxes.py                # interaktiv, 5 Farben

#   python3 teach_boxes.py --once yesil   # nur yesil speichern

#   python3 teach_boxes.py --list         # vorhandene Einträge zeigen

# =====

import argparse

import json
```



```

import os

import rclpy

from rclpy.node import Node

import tf2_ros

# EXAKT dieselbe Reihenfolge wie detection.py (model.names): class_id 0..4
CLASS_NAMES = ["sari", "beyaz", "siyah", "yesil", "kirmizi"]

ROBOT_BASE = "robot_base"

TCP = "tcp"

WS = os.path.expanduser("~/ros2_ws")
JSON_PATH = os.path.join(WS, ".box_teach.json")
YAML_PATH = os.path.join(WS, "box_map.yaml")

class TeachNode(Node):
    def __init__(self):
        super().__init__("teach_boxes")

        self.tf_buffer = tf2_ros.Buffer()

        tf2_ros.TransformListener(self.tf_buffer, self)

    def tcp_pose(self, settle_ticks=12):
        """robot_base->tcp momentan (x,y,z). Kurzes Spin, um den TF-Buffer zu füllen."""
        for _ in range(settle_ticks):
            rclpy.spin_once(self, timeout_sec=0.1)

        try:
            tf = self.tf_buffer.lookup_transform(ROBOT_BASE, TCP, rclpy.time.Time())

            t = tf.transform.translation

            return [float(t.x), float(t.y), float(t.z)]

        except Exception as e:
            self.get_logger().warn(f"TF robot_base->tcp nicht lesbar: {e}")

            return None

    def load_boxes():
        if os.path.exists(JSON_PATH):

```

```

try:
    with open(JSON_PATH) as f:
        return json.load(f)
except Exception:
    pass
return {}

def save_all(boxes):
    """Kanonisches JSON + das von pick_tilt gelesene ros-params-YAML zusammen schreiben."""
    with open(JSON_PATH, "w") as f:
        json.dump(boxes, f, indent=2)

    lines = [
        "# mit teach_boxes.py angelernt – Box-Zentren fuer das Sortieren nach Farbe (robot_base, m).",
        "# pick_tilt.launch.py laedt sie per BOX_MAP env (Standard ~/ros2_ws/box_map.yaml).",
        "/*:",
        "  ros__parameters:",
    ]

    for name in CLASS_NAMES:
        xyz = boxes.get(name)
        if xyz is None:
            lines.append(f"    # box_{name}: UEBERSPRUNGEN (nicht angelernt)")
        else:
            lines.append(f"    box_{name}: [{xyz[0]:.4f}, {xyz[1]:.4f}, {xyz[2]:.4f}]")

    with open(YAML_PATH, "w") as f:
        f.write("\n".join(lines) + "\n")

def print_list(boxes):
    print("=== Vorhandene Box-Eintraege ===")

    for name in CLASS_NAMES:
        xyz = boxes.get(name)
        if xyz is None:
            print(f"  {name:8s}: – (nicht angelernt)")
        else:
            print(f"  {name:8s}: x={xyz[0]*1000:.0f} y={xyz[1]*1000:.0f} z={xyz[2]*1000:.0f} mm")

```

```

def main():

    ap = argparse.ArgumentParser()

    ap.add_argument("--once", metavar="FARBE", help=f"nur diese Farbe speichern ({'/'.join(CLASS_NAMES)}")

    ap.add_argument("--list", action="store_true", help="vorhandene Eintraege zeigen und beenden")

    args = ap.parse_args()

    boxes = load_boxes()

    if args.list:

        print_list(boxes)

        return

    rclpy.init()

    node = TeachNode()

    if args.once:

        color = args.once.strip().lower()

        if color not in CLASS_NAMES:

            print(f"△ Ungueltige Farbe '{color}'. Gueltig: {'', '.join(CLASS_NAMES)}")

        else:

            pose = node.tcp_pose()

            if pose is None:

                print("△ Kein TF – laeuft der Stack? Speichern fehlgeschlagen.")

            else:

                boxes[color] = pose

                save_all(boxes)

                print(f"✓ {color}: x={pose[0]*1000:.0f} y={pose[1]*1000:.0f} "

                      f"z={pose[2]*1000:.0f} mm → gespeichert ({YAML_PATH}")

                print_list(boxes)

            node.destroy_node(); rclpy.shutdown()

            return

    # --- interaktiver Modus ---

    print("\n=== BOX ANLERNEN (teach) – Sortieren nach Farbe ===")

    print("Mit RViz MotionPlanning den Roboter ueber die Box-Oeffnung bringen, dann ENTER.")

    print("('s'=ueberspringen, 'q'=speichern und beenden)\n")

```

```

try:
    for name in CLASS_NAMES:
        while True:
            ans = input(f"Ueber Box [{name.upper()}]? ENTER=speichern / s=ueberspringen / q=beenden:
").strip().lower()

            if ans == "q":
                raise KeyboardInterrupt

            if ans == "s":
                print(f" {name} UEBERSPRUNGEN."); break

            pose = node.tcp_pose()

            if pose is None:
                print(" ⚠ Kein TF – laeuft der Stack? Erneut versuchen."); continue

            boxes[name] = pose

            save_all(boxes)

            print(f" ✓ {name}: x={pose[0]*1000:.0f} y={pose[1]*1000:.0f} z={pose[2]*1000:.0f} mm")

            break

except KeyboardInterrupt:
    print("\n(wird beendet)")

print_list(boxes)
print(f"\n→ {YAML_PATH}")
node.destroy_node(); rclpy.shutdown()

if __name__ == "__main__":
    main()

```

teach_click.rviz

Panels:

- Class: rviz_common/Displays

Name: Displays

Property Tree Widget:

Expanded: ~

Splitter Ratio: 0.5

- Class: rviz_common/Views

Name: Views

Visualization Manager:

Class: ""

Displays:

- Class: rviz_default_plugins/Grid

Name: Grid

Color: 160; 160; 164

Cell Size: 0.1

Plane Cell Count: 20

Reference Frame: robot_base

Value: true

- Class: rviz_default_plugins/RobotModel

Name: RobotModel

Description Topic:

Value: /robot_description

Depth: 5

Durability Policy: Volatile

Reliability Policy: Reliable

History Policy: Keep Last

Visual Enabled: true

Collision Enabled: false

Value: true

- Class: rviz_default_plugins/TF

Name: TF

Show Names: false

Show Axes: true

Show Arrows: false

Marker Scale: 0.2

Value: false

- Class: rviz_default_plugins/Marker

Name: ReachRings

Topic:

Value: /reach_boundary_marker

Depth: 10

Durability Policy: Volatile

Reliability Policy: Reliable

History Policy: Keep Last

Value: true

- Class: rviz_default_plugins/Marker

Name: ClickedTarget

Topic:

Value: /clicked_target_marker

Depth: 10

Durability Policy: Volatile

Reliability Policy: Reliable

History Policy: Keep Last

Value: true

Global Options:

Background Color: 48; 48; 48

Fixed Frame: robot_base

Frame Rate: 30

Name: root

Tools:

- Class: rviz_default_plugins/MoveCamera

- Class: rviz_default_plugins/Select

- Class: rviz_default_plugins/FocusCamera

- Class: rviz_default_plugins/Measure

Line color: 128; 128; 0

- Class: rviz_default_plugins/PublishPoint

Single click: true

Topic:

Value: /clicked_point

Depth: 5

Durability Policy: Volatile

History Policy: Keep Last

Reliability Policy: Reliable

Transformation:

Current:

Class: rviz_default_plugins/TF

Value: true

Views:

Current:

Class: rviz_default_plugins/Orbit

Name: Current View

Distance: 1.2

Focal Point:

X: 0.15

Y: 0

Z: 0.1

Pitch: 0.6

Yaw: 0.8

Target Frame: robot_base

Value: Orbit (rviz)

Saved: ~

Window Geometry:

Displays:

collapsed: false

Height: 800

Width: 1200

Hide Left Dock: false

Hide Right Dock: false

teach_setup.sh

```
#!/bin/bash

# =====

# teach_setup.sh – Lernumgebung für das Anlernen von 5 Boxen zur Farb-SORTIERUNG (ISOLIERT, KLIICK-UND-GO).

#

# Teach-Umgebung mit einem Befehl:

#     1) Echt-Hardware-Stack (move_group + control + bridge)

#     2) goto_clicked_point (RViz "Publish Point" → Roboter fährt 5cm über den Punkt)

#     3) LEICHTES rviz2 (teach_click.rviz) – KEIN MotionPlanning → stürzt nicht ab.

# Kein Detector/Kamera/pick_tilt → es geht kein falsches Kommando an den Arm.

#

# VERWENDUNG: cd ~/ros2_ws && ./teach_setup.sh

# ANLERNEN:

#     - In RViz mit dem Werkzeug "Publish Point" auf den Ort der Box klicken → Roboter fährt hin.

#     - Bei Bedarf erneut klicken, anpassen. Wenn genau über der Box, speichern mit:

#         python3 teach_boxes.py --once <farbe>      (farbe: sari/beyaz/siyah/yesil/kirmizi)

# BEENDEN: diesen Prozess stoppen → Roboter wird zu 0 geparkt, alles wird beendet.

# =====

set -u

WS="$HOME/ros2_ws"

STAMP="$(date +%Y%m%d_%H%M%S)"

LOGDIR="$WS/logs/teach_$STAMP"; mkdir -p "$LOGDIR"

STACK_LOG="$LOGDIR/stack.log"; RVIZ_LOG="$LOGDIR/rviz.log"; CLICK_LOG="$LOGDIR/click2go.log"

RVIZ_CFG="$WS/teach_click.rviz"

ARM_JOINTS="[joint2_to_joint1, joint3_to_joint2, joint4_to_joint3, joint5_to_joint4, joint6_to_joint5, joint6output_to_joint6]"

log() { echo -e "\033[1;36m[teach]\033[0m $*"; }

warn() { echo -e "\033[1;33m[teach] WARNUNG:\033[0m $*"; }

err() { echo -e "\033[1;31m[teach] FEHLER:\033[0m $*"; }

export LC_ALL=C LC_NUMERIC=C LANG=C PYTHONUNBUFFERED=1

set +u

source /opt/ros/galactic/setup.bash

source "$WS/install/setup.bash"

set -u

export USE_FAKE_HARDWARE="{USE_FAKE_HARDWARE:-false}"
```



```

export MYCOBOT_PORT="${MYCOBOT_PORT:-/dev/ttyTHS1}"

export DISPLAY="${DISPLAY:-:0}"

CTRL_WAIT="${CTRL_WAIT:-60}"

STACK_PID=""; RVIZ_PID=""; CLICK_PID=""

cleanup() {

    log "AUFRAEUMEN – Roboter wird zu 0 geparkt, nodes werden beendet..."

    if ros2 action list 2>/dev/null | grep -q "/arm_controller/follow_joint_trajectory"; then

        timeout 20 ros2 action send_goal /arm_controller/follow_joint_trajectory \

            control_msgs/action/FollowJointTrajectory \

                "{trajectory: {joint_names: $ARM_JOINTS, points: [{positions: [0.0,0.0,0.0,0.0,0.0,0.0], time_from_start: {sec: 5}}]}}" \

                    >/dev/null 2>&1 && log "Roboter bei Punkt 0." || warn "Konnte nicht zu 0 geparkt werden – Arm von Hand kontrollieren!"

    fi

    [ -n "$RVIZ_PID" ] && kill -- -"$RVIZ_PID" 2>/dev/null

    [ -n "$CLICK_PID" ] && kill -- -"$CLICK_PID" 2>/dev/null

    [ -n "$STACK_PID" ] && kill -- -"$STACK_PID" 2>/dev/null

    pkill -f "rviz2 -d $RVIZ_CFG" 2>/dev/null

    pkill -f "goto_clicked_point" 2>/dev/null

    pkill -f "demo.launch.py" 2>/dev/null

    sleep 1

    pkill -9 -f "ros2_control_node" 2>/dev/null

    pkill -9 -f "mycobot_bridge.py" 2>/dev/null

    rm -f /tmp/mycobot_bridge.sock /tmp/mycobot_bridge.sock.cmd 2>/dev/null

    log "Fertig. Eintraege: cat $WS/box_map.yaml"

}

trap cleanup EXIT INT TERM

if pgrep -f "ros2_control_node" >/dev/null 2>&1; then

    err "Es laeuft bereits ein ros2_control_node. Erst diesen beenden, dann erneut versuchen."

    exit 1

fi

# --- 1) STACK ---

log "##### STACK (echte Hardware, ohne Kamera, ohne RViz) → $STACK_LOG #####"

[ "$USE_FAKE_HARDWARE" = "true" ] && warn "FAKE HARDWARE – kein echter Roboter (teach sinnlos!)"

setuid bash -c "exec ros2 launch mycobot_moveit_config demo.launch.py \

```

```

    use_camera:=false use_rviz:=false use_fake_hardware:=$USE_FAKE_HARDWARE" >"$STACK_LOG" 2>&1 &
STACK_PID=$!

log "Auf Controller warten (max ${CTRL_WAIT}s)..."

ok=0

for i in $(seq 1 "$CTRL_WAIT"); do

    kill -0 "$STACK_PID" 2>/dev/null || { err "Stack zu frueh gestorben!"; tail -n 25 "$STACK_LOG"; exit 1; }

    ros2 control list_controllers 2>/dev/null | grep -q "arm_controller.*active" && { ok=1; log "Controller AKTIV (${i}s)"; break; }

    sleep 1
done

[ "$ok" = "1" ] || { err "Controller wurde nicht aktiv. Siehe $STACK_LOG"; exit 1; }

log "Roboter wird zu Punkt 0 gesendet..."

timeout 20 ros2 action send_goal /arm_controller/follow_joint_trajectory \

    control_msgs/action/FollowJointTrajectory \

    "{trajectory: {joint_names: $ARM_JOINTS, points: [{positions: [0.0,0.0,0.0,0.0,0.0,0.0]}, time_from_start: {sec: 5}}]}}" \

    >/dev/null 2>&1 && log "Roboter bei Punkt 0." || warn "Konnte nicht zu 0 gesendet werden (weiter)."
```

--- 2) KLIK-UND-GO node ---

```

log "goto_clicked_point (RViz Publish Point → Roboter) → $CLICK_LOG"

setsid bash -c "exec ros2 launch mycobot_demo goto_clicked_point.launch.py" >"$CLICK_LOG" 2>&1 &

CLICK_PID=$!

sleep 4
```

--- 3) LEICHTES RViz (KEIN MotionPlanning)---

```

if [ -n "${DISPLAY:-}" ]; then

    log "RViz (leichte teach config, stuerzt nicht ab) → $RVIZ_LOG"

    setsid bash -c "exec ros2 run rviz2 rviz2 -d $RVIZ_CFG" >"$RVIZ_LOG" 2>&1 &

    RVIZ_PID=$!

    sleep 6
else

    warn "Kein DISPLAY – RViz konnte nicht geoeffnet werden."
fi
```

```

log "#####"
```

log " ANLERNEN BEREIT. In RViz mit 'Publish Point' auf die Box klicken → Roboter faehrt hin."

```
log " Zum Speichern (waehrend der Arm ueber der Box steht):"

log "      python3 $WS/teach_boxes.py --once <farbe>"

log " Vorhandene Eintraege: python3 $WS/teach_boxes.py --list"

log " Beenden: diesen Prozess stoppen (Roboter wird zu 0 geparkt)."
```

tools/umlaut_convert.py

```
#!/usr/bin/env python3

"""Yorum/docstring-kapsamlı ASCII-Almanca -> gerçek umlaut dönüştürücü.

String literal'lere DOKUNMAZ (cv2 metinleri + grep kontratı korunur).

Kullanım: umlaut.py [--apply]    (varsayılan: dry-run)

"""

import os, re, sys, io, tokenize, ast, glob, collections

APPLY = '--apply' in sys.argv

ROOT = os.path.expanduser('~/.ros2_ws')

OUR = ['src/mycobot_calibration', 'src/mycobot_world', 'src/mycobot_hardware',
       'src/mycobot_demo', 'src/mycobot_moveit_config', 'src/vida_vision']

VOWELS=set('aeiouäöüAEIOU')

# --- ß + özel tam-kelime dönüşümleri (elle onaylı) ---

FORCE = {

    'gross': 'groß', 'grosse': 'große', 'grosses': 'großes', 'Grosser': 'Großer', 'grossteils': 'größteils',
    'groesste': 'größte', 'groesstem': 'größtem', 'groessten': 'größten', 'groesster': 'größter',
    'Groessen': 'Größen', 'Groesster': 'Größter',
    'vergroessern': 'vergrößern', 'Boardgroesse': 'Boardgröße', 'Feldgroesse': 'Feldgröße', 'Marker-Groesse': 'Marker-Größe',
    'ausser': 'außer', 'ausserdem': 'außerdem', 'ausserhalb': 'außerhalb', 'ausssen': 'außen',
    'aeusseren': 'äußeren', 'aeusserer': 'äußerer', 'aeusseres': 'äußeres',
    'Aussenflaeche': 'Außenfläche', 'Aussenmass': 'Außenmaß',
    'schliessen': 'schließen', 'schliesst': 'schließt', 'Schliessen': 'Schließen', 'Schliess-': 'Schließ-',
    'anschliessend': 'anschließend', 'anschliesst': 'anschließt', 'umschliessen': 'umschließen',
    'ausschliesslich': 'ausschließlich', 'Finger-Schliessachse': 'Finger-Schließachse',
    'stoest': 'stößt', 'stossen': 'stoßen', 'Anstossen': 'Anstoßen', 'anstossen': 'anstoßen',
    'angestossen': 'angestoßen', 'Anstoss-Pruefung': 'Anstoß-Prüfung',
    'weiss': 'weiß', 'weissen': 'weißen', 'weisser': 'weißer',
    'Mass': 'Maß', 'Masse': 'Maße', 'Massbestimmung': 'Maßbestimmung', 'Board-Masse': 'Board-Maße',
    'Markermasse': 'Markermaße', 'Massstab': 'Maßstab', 'Kamera-Massstab': 'Kamera-Maßstab',
    'massstaeblich': 'maßstäblich', 'massgeblich': 'maßgeblich',
    'Ausreisser': 'Ausreißer', 'Ausreisser-Eliminierung': 'Ausreißer-Eliminierung', 'Ausreisser-Filter': 'Ausreißer-Filter',
    'MAD-Ausreisser-Eliminierung': 'MAD-Ausreißer-Eliminierung', 'MAD-Ausreisser-': 'MAD-Ausreißer-',
    'Tiefen-Ausreisser': 'Tiefen-Ausreißer', 'ausreisserrobust': 'ausreißerrobust', 'ausreisser-robustes': 'ausreißer-robustes',
}
```

```

'Gauss-Newton': 'Gauß-Newton',

'gleichmaessig': 'gleichmäßig', 'gleichmaessiger': 'gleichmäßiger',

'erfahrungsgemaess': 'erfahrungsgemäß', 'Standardmaessig': 'Standardmäßig',

'Muell': 'Müll',

'zurueckprojizieren': 'zurückprojizieren', # önceden var olan yazım hatası (uee) düzeltildi
}

# İngilizce/özel-ad/zuerst – hiç değişmez

KEEP = {'true', 'True', 'value', 'values', 'Daemon-Thread', 'Rodrigues', 'Rodrigues-Rotationsvektor',

        'Rodrigues-Umformung', 'Shoemake', 'Verzeichnungskoeffizienten', 'zuerst', 'Zuerst'}

# -uell Latince sıfat/zarf – ue korunur (füllen DEĞİL)

UELL_SKIP = {'aktuell', 'aktuelle', 'aktuellem', 'aktuellen', 'aktueller', 'aktuelles', 'aktuellste', 'aktuellstes',

             'Aktuelle', 'Aktuellen', 'manuell', 'manuellen', 'visuell', 'visuelle', 'visuellen'}

def conv_ue(w):

    out=[]; i=0

    while i<len(w):

        two=w[i:i+2]

        if two in ('ue', 'Ue'):

            prev=w[i-1] if i>0 else ''

            if prev in VOWELS or prev in ('q', 'Q'):

                out.append(w[i]); i+=1; continue

            out.append('ü' if two=='ue' else 'Ü'); i+=2; continue

        out.append(w[i]); i+=1

    return ''.join(out)

def rule(w):

    return conv_ue(w.replace('ae', 'ä').replace('Ae', 'Ä').replace('oe', 'ö').replace('Oe', 'Ö'))

CHANGES = collections.Counter()

def conv_word(w):

    if w in FORCE: new=FORCE[w]

    elif w in KEEP or w in UELL_SKIP or w.isupper(): new=w # UPPERCASE=UI-literal referansı, atla

    else: new=rule(w)

    if new!=w: CHANGES[(w,new)]+=1

    return new

WORD=re.compile(r'[A-Za-zÄ-ÿ][A-Za-zÄ-ÿ\-\-]*')

```

```

def conv_text(t):

    return WORD.sub(lambda m: conv_word(m.group(0)), t)

# ----- .py: tokenize span-replacement (yorum + docstring) -----

def process_py(src):

    spans=[] # (start_off, end_off, newtext)

    lines=src.splitlines(keepends=True)

    def off(row,col):

        return sum(len(lines[r]) for r in range(row-1))+col

    try:

        toks=list(tokenize.generate_tokens(io.StringIO(src).readline))

    except Exception as e:

        return src, False

    # docstring string token'larını bul (Module/Class/Func ilk ifade)

    doc_positions=set()

    try:

        tree=ast.parse(src)

        for node in ast.walk(tree):

            if isinstance(node, (ast.Module, ast.ClassDef, ast.FunctionDef, ast.AsyncFunctionDef)):

                body=getattr(node, 'body', [])

                if body and isinstance(body[0], ast.Expr) and isinstance(getattr(body[0], 'value', None), ast.Constant) and isinstance(body[0].value.value, str):

                    doc_positions.add((body[0].value.lineno, body[0].value.col_offset))

    except Exception:

        pass

    for tok in toks:

        if tok.type==tokenize.COMMENT:

            spans.append((off(*tok.start), off(*tok.end), conv_text(tok.string)))

        elif tok.type==tokenize.STRING and (tok.start[0], tok.start[1]) in doc_positions:

            spans.append((off(*tok.start), off(*tok.end), conv_text(tok.string)))

    if not spans: return src, False

    spans.sort()

    out=[]; last=0; changed=False

    for s,e,new in spans:

        out.append(src[last:s]);

        if new!=src[s:e]: changed=True

        out.append(new); last=e

```

```

    out.append(src[last:])

    return ''.join(out), changed

# ----- .sh: tam-satır # yorumları -----
def process_sh(src):
    res=[]; changed=False
    for line in src.splitlines(keepends=True):
        if line.lstrip().startswith('#'):
            nl=conv_text(line)
            if nl!=line: changed=True
            res.append(nl)
        else:
            res.append(line)
    return ''.join(res), changed

# ----- .yaml: satır-başı ya da boşluk-sonrası # -----
def process_yaml(src):
    res=[]; changed=False
    for line in src.splitlines(keepends=True):
        m=re.search(r'(^|\s)#', line)
        if m:
            i=m.start()+len(m.group(1))
            head, comment = line[:i], line[i:]
            nc=conv_text(comment)
            nl=head+nc
            if nl!=line: changed=True
            res.append(nl)
        else:
            res.append(line)
    return ''.join(res), changed

# ----- .cpp: // ve /* */ yorumları -----
def process_cpp(src):
    out=[]; i=0; n=len(src); changed=False
    while i<n:
        if src[i:i+2]=='//':
            j=src.find('\n', i); j=n if j<0 else j

```

```

        seg=src[i:j]; nseg=conv_text(seg)

        if nseg!=seg: changed=True

        out.append(nseg); i=j
    elif src[i:i+2]=='/*':

        j=src.find('*/', i); j=n if j<0 else j+2

        seg=src[i:j]; nseg=conv_text(seg)

        if nseg!=seg: changed=True

        out.append(nseg); i=j
    elif src[i]=='': # string literal - atla

        j=i+1

        while j<n and src[j]!='':

            if src[j]=='\\': j+=1

            j+=1

        out.append(src[i:j+1]); i=j+1
    elif src[i]=='':

        j=i+1

        while j<n and src[j]!='':

            if src[j]=='\\': j+=1

            j+=1

        out.append(src[i:j+1]); i=j+1
    else:

        out.append(src[i]); i+=1

    return ''.join(out), changed

```

```

# ----- .xml/.xacro/.srdf: <!-- --> -----

```

```

def process_xml(src):

    changed=[False]

    def repl(m):

        seg=m.group(0); nseg=conv_text(seg)

        if nseg!=seg: changed[0]=True

        return nseg

    out=re.sub(r'<!--.*?-->', repl, src, flags=re.S)

    return out, changed[0]

```

```

def handler(path):

    if path.endswith('.py'): return process_py

    if path.endswith('.sh'): return process_sh

```



```

    if path.endswith(('.yaml','.yml')): return process_yaml

    if path.endswith(('.cpp','.hpp','.h','.cc')): return process_cpp

    if path.endswith(('.xacro','.srdf','.urdf','.xml')): return process_xml

    return None

files=[]

for base in OUR:

    for r,dirs,fs in os.walk(os.path.join(ROOT,base)):

        dirs[:] = [d for d in dirs if d not in ('__pycache__','.git')]

        for f in fs:

            if handler(f): files.append(os.path.join(r,f))

for pat in ('*.sh','*.py','*.yaml'):

    files += glob.glob(os.path.join(ROOT,pat))

changed_files=[]

for p in sorted(set(files)):

    h=handler(p); src=open(p,encoding='utf-8').read()

    new,ch=h(src)

    if ch:

        changed_files.append(p)

        if APPLY:

            open(p,'w',encoding='utf-8').write(new)

print(f"{'UYGULANDI' if APPLY else 'DRY-RUN'}: {len(changed_files)} dosya deđiř(ecek/ti), {len(CHANGES)} benzersiz kelime dnřm, toplam {sum(CHANGES.values())} yerde")

print("=== benzersiz dnřmler (alfabetik) ===")

for (w,new),c in sorted(CHANGES.items()):

    print(f"{c:4d}  {w}  ->  {new}")

```

view_camera.sh

```
#!/bin/bash

# =====

# view_camera.sh – NUR Kamerabild + Roboter-Reach-RINGE.

#

# Zweck: ohne pick, nur das D435i-Overlay ansehen – YOLO-Erkennung + gewählte

# Schraube + Roboter-Basis-Reach-Kreise (grün/amber/rot). Um per Auge zu prüfen,

# ob die Schrauben im grünen Band liegen.

#

# RUEHRT DEN ECHTEN ROBOTER NICHT AN: use_fake_hardware:=true → TF (Kamera→robot_base) kommt aus dem URDF

# (eye-to-hand, statisch), Port/Servo werden nicht genutzt, KEIN Servo-drop-Risiko.

#

# Verwendung: cd ~/ros2_ws && ./view_camera.sh          (Ctrl+C = beenden)

#              CONF=0.5 ./view_camera.sh                (Schwelle ändern)

# =====

set -u

export LC_ALL=C LC_NUMERIC=C LANG=C PYTHONUNBUFFERED=1

WS="$HOME/ros2_ws"

STAMP="$(date +%Y%m%d_%H%M%S)"

LOGDIR="$WS/logs/view_camera_$STAMP"; mkdir -p "$LOGDIR"

STACK_LOG="$LOGDIR/stack.log"; CAM_LOG="$LOGDIR/camera.log"

DET_LOG="$LOGDIR/detector.log"; CAMV_LOG="$LOGDIR/cam_viewer.log"

CONF="{CONF:-0.5}"

CAM_WAIT="{CAM_WAIT:-40}"; DETECTOR_WAIT="{DETECTOR_WAIT:-120}"

STACK_PID=""; CAM_PID=""; DET_PID=""; CAMV_PID=""

log() { echo -e "\033[1;36m[view]\033[0m $*"; }

warn() { echo -e "\033[1;33m[view]\033[0m $*"; }

err() { echo -e "\033[1;31m[view]\033[0m $*" >&2; }

kill_group() {

    local pid="$1" name="$2"; [ -z "$pid" ] && return 0

    if kill -0 "$pid" 2>/dev/null; then

        log "$name wird beendet (pgid=$pid)..."

    fi

}
```

```

    kill -TERM -- "-$pid" 2>/dev/null

    for _ in $(seq 1 10); do kill -0 "$pid" 2>/dev/null || break; sleep 0.5; done

    kill -0 "$pid" 2>/dev/null && { warn "$name erzwungen (KILL)"; kill -KILL -- "-$pid" 2>/dev/null; }

fi
}

cleanup() {
    [ "${_CLEANED:-0}" = "1" ] && return 0
    _CLEANED=1; trap - EXIT INT TERM; echo
    log "AUFRAEUMEN – nodes werden beendet..."
    kill_group "$CAMV_PID" "cam_viewer"
    kill_group "$DET_PID" "detector"
    kill_group "$CAM_PID" "camera"
    kill_group "$STACK_PID" "stack"
    log "Fertig. (fake hardware – echter Roboter/Port wurde nicht genutzt.)"
}

trap cleanup EXIT INT TERM

# ROS setup.bash nutzt unbound-Variablen → unter set -u platzt es; das sourcen umschließen.
set +u
source /opt/ros/galactic/setup.bash
source "$WS/install/setup.bash"
set -u

# --- Stack (NUR für TF): fake hardware, kein RViz, Kamera wird separat gestartet ---
log "Stack (fake hardware, fuer TF) → $STACK_LOG"
setsid bash -c "exec ros2 launch mycobot_moveit_config demo.launch.py \
    use_camera:=false use_rviz:=false use_fake_hardware:=true" >"$STACK_LOG" 2>&1 &
STACK_PID=$!
for i in $(seq 1 30); do
    kill -0 "$STACK_PID" 2>/dev/null || { err "Stack zu frueh gestorben!"; tail -n 25 "$STACK_LOG"; exit 1; }
    ros2 topic list 2>/dev/null | grep -q "/joint_states" && { log "Stack/TF bereit (${i}s)"; break; }
    sleep 1
done

# --- Kamera (D435i, niedrige Auflösung – auf Jetson erforderlich) ---
log "Kamera (rs_launch 424x240x15) → $CAM_LOG"

```

```

setuid bash -c "exec ros2 launch realsense2_camera rs_launch.py \

    pointcloud.enable:=true align_depth.enable:=true \

    depth_module.profile:=424x240x15 rgb_camera.profile:=424x240x15" >"$CAM_LOG" 2>&1 &

CAM_PID=$!

ok=0

for i in $(seq 1 "$CAM_WAIT"); do

    kill -0 "$CAM_PID" 2>/dev/null || { err "Kamera zu frueh gestorben!"; tail -n 25 "$CAM_LOG"; exit 1; }

    [ "$(ros2 topic info /camera/color/image_raw 2>/dev/null | grep -c 'Publisher count: [1-9]')" -ge 1 ] && { ok=1;
log "Kamera sendet (${i}s)"; break; }

    sleep 1
done

[ "$ok" = "1" ] || { err "Kamera sendet nicht innerhalb ${CAM_WAIT}s (evtl. USB aus-/einstecken noetig)"; tail -n
20 "$CAM_LOG"; exit 1; }


# --- Detector (zeichnet Overlay + Reach-Ringe, sendet an /vida/overlay) ---

log "vida_detector (conf=$CONF) → $DET_LOG"

setuid bash -c "exec ros2 run vida_vision vida_detector --ros-args -p conf:=$CONF" >"$DET_LOG" 2>&1 &

DET_PID=$!

ok=0

for i in $(seq 1 "$DETECTOR_WAIT"); do

    kill -0 "$DET_PID" 2>/dev/null || { err "Detector zu frueh gestorben!"; tail -n 25 "$DET_LOG"; exit 1; }

    grep -q "Modell bereit" "$DET_LOG" 2>/dev/null && { ok=1; log "Modell geladen (${i}s)"; break; }

    sleep 1
done

[ "$ok" = "1" ] || { err "Detector nicht bereit innerhalb ${DETECTOR_WAIT}s"; tail -n 20 "$DET_LOG"; exit 1; }


# --- cam_viewer: öffnet das /vida/overlay-Fenster (hier sind die Ringe sichtbar) ---

log "cam_viewer (Overlay-Fenster) → $CAMV_LOG"

setuid bash -c "exec python3 '$WS/cam_viewer.py'" >"$CAMV_LOG" 2>&1 &

CAMV_PID=$!


log "BEREIT – Kamerafenster offen. Gruen/amber Kreis = Roboter-Reach-Grenze."

log "Schraube innerhalb des GRUENEN Kreises = ideale pick-Zone. Ctrl+C = beenden."

# Warten, solange die nodes leben (wenn einer stirbt → raus → cleanup)

while kill -0 "$CAMV_PID" 2>/dev/null && kill -0 "$DET_PID" 2>/dev/null \

    && kill -0 "$CAM_PID" 2>/dev/null && kill -0 "$STACK_PID" 2>/dev/null; do

    sleep 1
done

```

| warn "Ein node wurde unerwartet beendet – wird aufgeräumt."

view_overlay.py

```
#!/usr/bin/env python3

# view_overlay.py - /vida/overlay in einem LEICHTGEWICHTIGEN Fenster live zeigen.

# Statt RViz/rqt (Nano-RAM sparen). Die gewählte Schraube GRUEN + mit "ZIEL" markiert.

#

# Verwendung (separates Terminal):

#   source /opt/ros/galactic/setup.bash && source ~/ros2_ws/install/setup.bash
#   DISPLAY=:0 python3 ~/ros2_ws/view_overlay.py

#   # im Fenster 'q' oder ESC = Ende

#

# Wenn du headless willst (kein Fenster, nur alle N Sekunden ein PNG speichern):

#   SAVE_ONLY=1 SAVE_DIR=/tmp/vida python3 ~/ros2_ws/view_overlay.py

import os

import rclpy

from rclpy.node import Node

from sensor_msgs.msg import Image

import numpy as np

import cv2

SAVE_ONLY = os.environ.get("SAVE_ONLY", "0") == "1"

SAVE_DIR = os.environ.get("SAVE_DIR", "/tmp/vida")

WIN = "Schraube Overlay (ZIEL=gruen) - q/ESC=Ende"

class Viewer(Node):

    def __init__(self):

        super().__init__("vida_overlay_viewer")

        # /vida/overlay wird mit default (reliable) QoS veröffentlicht; zum Matchen default-Abonnent.

        self.sub = self.create_subscription(Image, "/vida/overlay", self.cb, 1)

        self.frame = None

        self.n = 0

        if SAVE_ONLY:

            os.makedirs(SAVE_DIR, exist_ok=True)

            self.get_logger().info(f"SAVE_ONLY - alle 30 Frames ein PNG → {SAVE_DIR}")

        else:

            cv2.namedWindow(WIN, cv2.WINDOW_NORMAL)
```

```

        cv2.resizeWindow(WIN, 848, 480)

        self.get_logger().info("warte auf /vida/overlay... (im Fenster mit q/ESC beenden)")

    self.timer = self.create_timer(0.03, self.tick)

def cb(self, msg: Image):

    # rgb8/bgr8 → BGR ndarray (ohne cv_bridge, wenig Abhängigkeiten)

    buf = np.frombuffer(msg.data, dtype=np.uint8).reshape(msg.height, msg.width, -1)

    if msg.encoding == "rgb8":

        buf = cv2.cvtColor(buf, cv2.COLOR_RGB2BGR)

    self.frame = buf

def tick(self):

    if self.frame is None:

        return

    if SAVE_ONLY:

        self.n += 1

        if self.n % 30 == 0:

            p = os.path.join(SAVE_DIR, "overlay_latest.png")

            cv2.imwrite(p, self.frame)

            self.get_logger().info(f"gespeichert: {p}")

        return

    cv2.imshow(WIN, self.frame)

    k = cv2.waitKey(1) & 0xFF

    if k in (ord("q"), 27): # q oder ESC

        rclpy.shutdown()

def main():

    rclpy.init()

    node = Viewer()

    try:

        rclpy.spin(node)

    except KeyboardInterrupt:

        pass

    finally:

        if not SAVE_ONLY:

            cv2.destroyAllWindows()

```

```
    if rclpy.ok():  
        node.destroy_node()  
        rclpy.shutdown()
```

```
if __name__ == "__main__":  
    main()
```


visit_boxes.py

```
#!/usr/bin/env python3

# =====

# visit_boxes.py – die 5 angelernten Boxen DER REIHE NACH abfahren (Prüfrunde).

#

# Ablauf: Roboter nach 0 → zur XY jeder Box fahren, z FEST (Standard 8cm über Plattform).

# Die Boxen liegen an der Reichweite-GRENZE des Roboters → top-down (senkrechte) IK

# löst nicht. Daher wird die Greifer-Ausrichtung wie in goto_clicked_point per

# YAW + TILT-Scan gefunden (zuerst gerade nach unten, sonst schrittweise neigen).

# Es wird mit der ersten lösbaren Ausrichtung gefahren.

# /compute_ik (move_group offen) + /arm_controller (gleicher Weg wie cartesian_jog).

#

# Verwendung: python3 visit_boxes.py          # z=0.08

#             python3 visit_boxes.py --z 0.06  # andere Höhe

# =====

import argparse, json, math, os, time

import rclpy

from rclpy.node import Node

from rclpy.action import ActionClient

from control_msgs.action import FollowJointTrajectory

from trajectory_msgs.msg import JointTrajectoryPoint

from sensor_msgs.msg import JointState

from builtin_interfaces.msg import Duration

from moveit_msgs.srv import GetPositionIK

from geometry_msgs.msg import PoseStamped

ARM_JOINTS = ['joint2_to_joint1', 'joint3_to_joint2', 'joint4_to_joint3',

              'joint5_to_joint4', 'joint6_to_joint5', 'joint6output_to_joint6']

WS = os.path.expanduser("~/ros2_ws")

JSON_PATH = os.path.join(WS, ".box_teach.json")

ORDER = ["kirmizi", "sari", "beyaz", "siyah", "yesil"]


def quat_rpy(roll, pitch, yaw):

    """exakt wie tf2 setRPY (goto_clicked_point gripper0orientation)."""

    hr, hp, hy = roll / 2, pitch / 2, yaw / 2
```

```

cr, sr = math.cos(hr), math.sin(hr)
cp, sp = math.cos(hp), math.sin(hp)
cy, sy = math.cos(hy), math.sin(hy)
return (sr * cp * cy - cr * sp * sy,
        cr * sp * cy + sr * cp * sy,
        cr * cp * sy - sr * sp * cy,
        cr * cp * cy + sr * sp * sy)

```

```

class Visitor(Node):

```

```

    def __init__(self):
        super().__init__("visit_boxes")
        self.cli = ActionClient(self, FollowJointTrajectory,
                                "/arm_controller/follow_joint_trajectory")
        self.ik = self.create_client(GetPositionIK, "/compute_ik")
        self.cur = [0.0] * 6
        self.js_ok = False
        self.create_subscription(JointState, "/joint_states", self._js, 10)

```

```

    def _js(self, msg):
        for i, n in enumerate(ARM_JOINTS):
            if n in msg.name:
                self.cur[i] = msg.position[msg.name.index(n)]
        self.js_ok = True

```

```

    def wait_js(self, ticks=40):
        for _ in range(ticks):
            rclpy.spin_once(self, timeout_sec=0.1)
            if self.js_ok:
                return True
        return False

```

```

    def solve(self, x, y, z, seed):
        """Yaw + Tilt-Scan; die erste lösbare 6-Gelenk-Lösung zurückgeben (sonst None)."""
        if not self.ik.wait_for_service(timeout_sec=2.0):
            return None
        yaw_out = math.atan2(y, x)

```

```

yaws = [yaw_out, yaw_out + math.pi/2, yaw_out - math.pi/2,
        yaw_out + math.pi, yaw_out + math.pi/4, yaw_out - math.pi/4]

tilts = [0.0, math.radians(20), math.radians(35), math.radians(45), math.radians(55)]

for tilt in tilts:
    for yaw in yaws:
        q = quat_rpy(-math.pi/2 + tilt, 0.0, yaw)

        req = GetPositionIK.Request()

        req.ik_request.group_name = 'arm'

        req.ik_request.ik_link_name = 'tcp'

        req.ik_request.avoid_collisions = False

        req.ik_request.timeout.sec = 1

        p = PoseStamped(); p.header.frame_id = 'robot_base'

        p.pose.position.x = x; p.pose.position.y = y; p.pose.position.z = z

        p.pose.orientation.x, p.pose.orientation.y, p.pose.orientation.z, p.pose.orientation.w = q

        req.ik_request.pose_stamped = p

        js = JointState(); js.name = list(ARM_JOINTS); js.position = list(seed)

        req.ik_request.robot_state.joint_state = js

        fut = self.ik.call_async(req)

        t0 = time.monotonic()

        while not fut.done() and (time.monotonic() - t0) < 2.0:

            rclpy.spin_once(self, timeout_sec=0.02)

        if fut.done() and fut.result() and fut.result().error_code.val == 1:

            nm = dict(zip(fut.result().solution.joint_state.name,
                          fut.result().solution.joint_state.position))

            return ([nm.get(j, 0.0) for j in ARM_JOINTS],
                    math.degrees(tilt), math.degrees(yaw - yaw_out))

    return None

```

```

def send(self, positions, secs):

    if not self.cli.wait_for_server(timeout_sec=3.0):

        return False

    g = FollowJointTrajectory.Goal()

    g.trajectory.joint_names = list(ARM_JOINTS)

    pt = JointTrajectoryPoint()

    pt.positions = list(map(float, positions))

    pt.time_from_start = Duration(sec=int(secs), nanosec=int((secs % 1) * 1e9))

    g.trajectory.points = [pt]

```

```

        f = self.cli.send_goal_async(g)

        rclpy.spin_until_future_complete(self, f)

        gh = f.result()

        if gh is None or not gh.accepted:

            return False

        rclpy.spin_until_future_complete(self, gh.get_result_async())

        return True

def settle(self, secs):

    for _ in range(int(secs / 0.1)):

        rclpy.spin_once(self, timeout_sec=0.1)

def main():

    ap = argparse.ArgumentParser()

    ap.add_argument("--z", type=float, default=0.08)

    args = ap.parse_args()

    boxes = json.load(open(JSON_PATH))

    rclpy.init()

    n = Visitor()

    if not n.wait_js():

        print("△ Kein /joint_states – laeuft der Stack?"); rclpy.shutdown(); return

    print(f"\n=== BOX-ABFAHRRUNDE (z={args.z*100:.0f}cm, yaw+tilt-Scan) ===")

    print("→ zum Punkt 0..."); n.send([0.0]*6, 5); n.settle(2.0); print(" ✓ bei 0.")

    for c in ORDER:

        b = boxes.get(c)

        if b is None:

            print(f"→ {c}: kein Eintrag"); continue

        res = n.solve(b[0], b[1], args.z, list(n.cur))

        if res is None:

            print(f"→ {c} (r={math.hypot(b[0],b[1])*1000:.0f}mm z={args.z*1000:.0f}mm): "

                  f"kein yaw/tilt geloest – auf dieser Hoehe NICHT ERREICHBAR."); continue

        sol, tilt, dyaw = res

        print(f"→ {c} (x={b[0]*1000:.0f} y={b[1]*1000:.0f} z={args.z*1000:.0f}mm) ")

```

```

        f"tilt={tilt:.0f}° dyaw={dyaw:+.0f}° wird angefahren...")

    if n.send(sol, 6):

        n.settle(2.5); print(f"  ✓ ueber {c}.")

    else:

        print(f"  △ {c}: Senden fehlgeschlagen.")


print("→ Rueckkehr nach 0..."); n.send([0.0]*6, 5); n.settle(2.0)

print("  FERTIG – Roboter am Punkt 0.")

n.destroy_node(); rclpy.shutdown()


if __name__ == "__main__":

    main()

```

vo_rsp.launch.py

"""Vision-only TF: nur robot_state_publisher (Kamera->robot_base statischer TF).

KEIN RViz/GUI/controller – BERUEHRT den echten Roboter NICHT. Veröffentlicht die TF-Kette, die der Detector für die Base-Frame-Berechnung braucht.

"""

```
from launch import LaunchDescription
```

```
from launch.substitutions import Command, PathJoinSubstitution
```

```
from launch_ros.actions import Node
```

```
from launch_ros.parameter_descriptions import ParameterValue
```

```
from launch_ros.substitutions import FindPackageShare
```

```
def generate_launch_description():
```

```
    xacro_path = PathJoinSubstitution(
```

```
        [FindPackageShare("mycobot_world"), "urdf", "mycobot_world.urdf.xacro"]
```

```
    )
```

```
    robot_description = {
```

```
        "robot_description": ParameterValue(
```

```
            Command(["xacro ", xacro_path]), value_type=str
```

```
        )
```

```
    }
```

```
    return LaunchDescription([
```

```
        Node(
```

```
            package="robot_state_publisher",
```

```
            executable="robot_state_publisher",
```

```
            name="robot_state_publisher",
```

```
            output="screen",
```

```
            parameters=[robot_description],
```

```
        ),
```

```
    ])
```